DRAFT
On Foundations of Deep Learning

Tomaso Poggio *with Gemini (and ChatGPT)*

based on work with Pierfrancesco Beneventano, Yulu Gan, David Koplow, Qianli Liao, Daniel Mitropolsky, Liu Ziyin

Intelligence and its Fundamental Principles

First version: January 2, 2026

Current version: October 5, 2026

Prologue

Modern deep learning works astonishingly well, yet we still lack a fundamental understanding of *why*. What we call "deep learning" is not merely a triumph of computer science or engineering; it is a natural phenomenon that demands explanation in terms of underlying principles—much as physics explains why the Sun produces energy or why matter follows specific laws.

This collection of essays is an attempt to uncover some of those principles. It argues that today's AI systems succeed because they instantiate deep structural regularities of the world—regularities that any intelligent system, biological or artificial, must exploit. Two principles emerge repeatedly throughout these essays.

The first is *(sparse) compositionality*, and the second is *genericity*. In this essays, we show that they are not heuristic conjectures, but logical inevitabilities derived from the foundations of computation and physics. We show that sparse compositionality is mathematically implied by efficient computability, just as genericity is dictated by assuming invariance in the choice of coordinates. Together, they define the channel through which intelligence must pass.

Together – and helped by several other ideas and observations – these principles offer a first step toward a unified theory of modern AI—one that explains not only *how* current systems work, but *why* they work at all.

Contents

Prologue **i**

How to Read This Collection **1**

A Note on Co-Authorship **3**

1 Intelligence as Associative Memory **5**
1.1 The Associative-Memory Hypothesis . . . 5
1.2 Attention as Content-Addressable Retrieval . . . 6
1.3 An Exact Kernel Equivalence for Normalized Attention . . . 7
1.4 From Associative Retrieval to Computation . . . 9
1.5 Building a Turing Machine from Associative Reads . . . 9
1.6 A Broader Principle: Computation by Repeated Local Transitions . . . 10
1.7 What the Associative-Memory Picture Does Not Explain . . . 10
1.8 Why Can Autoregressive Generation Be So Coherent? . . . 11
1.8.1 Structured Geometry of Learned Representations . . . 11
1.8.2 Self-Reinforcing Autoregression . . . 13
1.8.3 Coherence Is Not Truth . . . 13
1.8.4 Emergent Consistency as Constrained Associative Recall . . . 14
1.9 Historical and Biological Connections . . . 14
1.10 Associative Retrieval, Computational Arity, and Memory Capacity . . . 15
1.10.1 A generic associative read . . . 15
1.10.2 Dot-product attention . . . 16
1.10.3 Gaussian RBF retrieval . . . 16
1.10.4 Modern Hopfield retrieval . . . 17
1.10.5 Kanerva's Sparse Distributed Memory . . . 17
1.10.6 Exact equivalence versus shared computational motif . . . 18
1.10.7 Associative retrieval does not automatically have bounded arity . . . 18
1.10.8 Sparse retrieval . . . 19
1.10.9 Bounded-fan-in implementation of dense retrieval . . . 19
1.10.10 Addressability and storage are different capacities . . . 20
1.10.11 Exponential addressability . . . 20
1.10.12 Modular address codes . . . 20
1.10.13 Superpositional storage . . . 21
1.10.14 The apparent paradox disappears . . . 21
1.10.15 Why the distinction matters for associative computation . . . 22

1.10.16 Relation to compositional computation . . . 22
1.10.17 A useful hierarchy of claims . . . 22
1.10.18 Conclusion . . . 23
1.11 Associative Memory as a Computational Primitive . . . 23

2 A Historical Reflection on Associative Memories 25
2.1 A Recurring Computational Idea . . . 25
2.2 Early Associative Memories . . . 27
2.2.1 Heteroassociative Correlation Memories . . . 27
2.2.2 Nonlinear Operator Memories . . . 27
2.2.3 RBFs and HyperBFs . . . 28
2.2.4 A Different Branch: Autoassociative Attractor Memories . . . 29
2.3 From Association to Biological Computation . . . 29
2.3.1 Reflexes as Elementary Input–Output Associations . . . 29
2.3.2 Plasticity and Adaptive Association . . . 30
2.3.3 A Possible Computational Progression . . . 30
2.3.4 Hippocampal Association and Scaffolding . . . 31
2.4 Modern Attention as Associative Retrieval . . . 31
2.5 A Preview: Why Does Associative Recall Work? . . . 32
2.6 Associative Memory Is a Primitive, Not a Complete Theory . . . 33
2.7 Experimental Evidence from Homogeneous HyperBF Architectures . . . 35
2.8 Conclusion . . . 35

3 Associative Memories Are Turing-Complete 37
3.1 From Associative Recall to Universal Computation . . . 37
3.2 Definition of the Associative Turing Machine . . . 38
3.2.1 Control State . . . 38
3.2.2 Program Memory . . . 39
3.2.3 Working Memory . . . 40
3.2.4 Associative Read . . . 41
3.2.5 Associative Write . . . 41
3.2.6 Local Update . . . 41
3.3 Why Extensible Memory Matters: RNNs, Precision, and Robust Computation . . . 43
3.3.1 Fixed-dimensional state with finite precision . . . 43
3.3.2 How idealized recurrent systems evade the proposition . . . 44
3.3.3 Why use explicit memory? . . . 45
3.3.4 Memory hierarchy and computational power . . . 45
3.3.5 Robustness rather than idealized expressivity . . . 46
3.3.6 Execution versus trainability . . . 46
3.3.7 The design principle . . . 46
3.4 Simulation of a Turing Machine . . . 47
3.4.1 Encoding the Transition Table . . . 47
3.4.2 Encoding the Tape . . . 47
3.4.3 One Simulated Step . . . 48
3.5 Main Theorem . . . 48
3.5.1 Proof . . . 48
3.5.2 Corollary: A Universal ATM . . . 49
3.6 Soft Attention as a Differentiable Approximation . . . 49

3.6.1 Softmax Read . . . 50
3.6.2 General Margin Bound . . . 50
3.6.3 Orthogonal-Key Case . . . 51
3.6.4 From Retrieval Error to State Error . . . 52
3.7 Relation to Transformer Computation . . . 52
3.7.1 Queries, Keys, and Values . . . 52
3.7.2 Residual State Update . . . 53
3.7.3 Append-Only Working Memory . . . 53
3.7.4 What the Correspondence Does and Does Not Say . . . 54
3.8 Why Associative Access Matters . . . 54
3.9 Discussion . . . 55

Part I: Principles and Foundations **57**

Interlude: Representation and Recovery **58**

4 Efficient Computability and Compositional Sparsity **62**
4.1 Representation, Not Recovery . . . 62
4.1.1 Why the distinction matters . . . 64
4.1.2 The central structural implication . . . 64
4.2 Definitions and Model Conventions . . . 65
4.2.1 Boolean base . . . 65
4.2.2 Real-valued base . . . 65
4.2.3 Efficient computability: Boolean functions . . . 65
4.2.4 Efficient computation of real-valued functions . . . 66
4.2.5 Computation DAGs . . . 66
4.2.6 Compositional sparsity . . . 67
4.3 Efficient Computability Implies Compositional Sparsity . . . 67
4.3.1 The Boolean case . . . 67
4.3.2 Why bounded fan-in is natural . . . 68
4.4 Exact Deep-Network Realization on Discrete Inputs . . . 68
4.4.1 Boolean gates as ReLU subnetworks . . . 69
4.4.2 Finite-precision grids . . . 69
4.5 Real-Valued Functions and Finite Precision . . . 70
4.6 Compositional Approximation and the Curse of Dimensionality . . . 71
4.6.1 Generic high-dimensional functions . . . 71
4.6.2 Low-arity constituents . . . 71
4.6.3 A stability factor . . . 71
4.6.4 A simple chain . . . 73
4.7 Two Small Consequences of Local Arity . . . 73
4.8 What C1 Buys . . . 74
4.8.1 A feasible representation . . . 74
4.8.2 Escape from ambient-dimensional approximation . . . 75
4.8.3 Optimization becomes feasible to pose . . . 75
4.8.4 Favorable statistical complexity becomes possible . . . 75
4.9 What Does Not Follow from C1 . . . 75
4.9.1 C1 does not imply module reuse . . . 76
4.9.2 C1 does not imply transfer . . . 76

4.9.3 C1 does not imply identifiability . . . 76
4.9.4 C1 does not imply visible sparsity of trained weights . . . 77
4.9.5 C1 does not imply easy global search . . . 77
4.9.6 C1 does not imply generalization . . . 77
4.9.7 Most importantly: C1 does not imply learnability . . . 78
4.10 Scope: Artificial and Biological Computation . . . 78
4.10.1 Computability versus efficient computability . . . 79
4.10.2 Artificial systems . . . 79
4.10.3 Biological systems . . . 79
4.10.4 Microscopic simulation versus functional simulation . . . 79
4.10.5 Local physical dynamics . . . 80
4.10.6 No connection to consciousness or evolutionary age . . . 80
4.10.7 The biological boundary . . . 80
4.11 The Status of the Principle . . . 81
4.12 Conclusion: From Representation to Predictive Learning . . . 81

5 Compositionality: Why Optimization Is Feasible to Pose 84
5.1 From Representation to Optimization . . . 84
5.2 Why C1 Makes the Search Space Feasible . . . 86
5.2.1 Generic high-dimensional approximation . . . 86
5.2.2 Compositionally sparse approximation . . . 86
5.2.3 What this does not imply . . . 87
5.3 Two Obstructions to End-to-End Learning . . . 87
5.3.1 The geometric obstruction . . . 87
5.3.2 Architectural responses to geometry . . . 87
5.3.3 The informational obstruction . . . 88
5.3.4 Geometry versus information . . . 88
5.3.5 What compositionality alone guarantees . . . 88
5.4 Grey-Box Recovery . . . 89
5.4.1 Black box versus grey box . . . 89
5.4.2 What the proposition proves . . . 89
5.4.3 Extension to a computation DAG . . . 90
5.5 Three Levels of Trace Exposure . . . 90
5.5.1 Hard trace . . . 90
5.5.2 Partial trace . . . 90
5.5.3 Surface trace . . . 91
5.6 Partial Traces in Practice . . . 91
5.6.1 Layer-wise training . . . 91
5.6.2 Transfer learning . . . 92
5.6.3 Scratchpads and chain-of-thought . . . 92
5.6.4 What partial exposure leaves to be learned . . . 92
5.7 Statistical Benefits of Structural Alignment . . . 93
5.8 What Grey-Box Structure Does to Optimization Geometry . . . 93
5.9 The Division of Labor Among C1, C2, and C3 . . . 93
5.10 Overparameterization, Degeneracy, and Entropic Selection . . . 94
5.10.1 Geometry of the interpolating set . . . 94
5.10.2 From flat directions to manifolds of solutions . . . 95
5.10.3 An idealized model of stochastic selection . . . 96

5.10.4 Probability mass near a manifold of minima . . . 96
5.10.5 Isolated minima as a special case . . . 97
5.10.6 Beyond the regular Hessian picture . . . 97
5.10.7 Ordinary SGD is not isotropic Langevin dynamics . . . 98
5.10.8 Learning rate, batch size, and noise scale . . . 99
5.10.9 Why parameter-space flatness is not generalization . . . 100
5.10.10 Selection and predictive recovery . . . 100
5.10.11 A role for overparameterization . . . 101
5.10.12 From geometric preference to finite-time learning . . . 101
5.11 Norm-Driven Selection and Separation of Timescales . . . 101
5.11.1 Fast fitting and slow selection . . . 102
5.11.2 An idealized constrained picture . . . 102
5.11.3 Neural collapse as a special example of structured selection . . . 103
5.11.4 Grokking as delayed selection . . . 103
5.11.5 Minimum norm need not identify the generating structure . . . 103
5.11.6 Why irrelevant features need not simply decay away . . . 104
5.11.7 Selection among well-fitting predictors . . . 105
5.11.8 From selection to accessible learning signals . . . 105
5.12 Summary . . . 105

6 Genericity: Where the Optimization Signal Comes From 108
6.1 The Missing Ingredient: Informative Signal . . . 109
6.2 Footprints . . . 109
6.2.1 Existence . . . 110
6.2.2 Size . . . 110
6.2.3 Relation to gradients at initialization . . . 110
6.3 Statistical Sufficiency versus Computational Recoverability . . . 111
6.3.1 An information threshold . . . 111
6.3.2 A recovery threshold depends on the learner . . . 111
6.3.3 Genericity and the accessible signal . . . 111
6.3.4 More data can still help recovery . . . 112
6.3.5 Changing the problem can be more powerful than adding data . . . 112
6.3.6 A simple symmetric example . . . 112
6.3.7 Trace supervision changes the observation model . . . 113
6.3.8 Representation, information, and recovery . . . 113
6.3.9 The recovery gap . . . 113
6.3.10 The central lesson . . . 114
6.4 Genericity as Symmetry Breaking . . . 114
6.4.1 Genericity is distribution-relative . . . 114
6.4.2 Structural, observational, and optimization genericity . . . 114
6.5 Analytic Shift Families . . . 115
6.5.1 What this does and does not prove . . . 115
6.6 Thom Transversality as Geometric Motivation . . . 115
6.6.1 Why transversality is secondary here . . . 115
6.7 Two Exact Examples . . . 116
6.7.1 Gaussian inputs and Stein's identity . . . 116
6.7.2 Boolean parity under biased product measures . . . 116
6.7.3 Existence versus size in one equation . . . 117

6.8 From Footprints to Detectability . . . 117
6.8.1 Polynomially detectable footprints . . . 118
6.8.2 Exponentially weak footprints . . . 118
6.8.3 Parity leakage revisited . . . 118
6.9 Bounded Leap . . . 118
6.10 The Staircase Picture . . . 119
6.10.1 Stage 1: detect an exposed footprint . . . 119
6.10.2 Stage 2: create new effective coordinates . . . 119
6.10.3 Stage 3: repeat . . . 119
6.10.4 Cliff versus staircase . . . 119
6.11 Genericity and Gradient-Based Optimization . . . 120
6.11.1 Population gradients . . . 120
6.11.2 The Statistical Query viewpoint . . . 120
6.11.3 Information versus geometry revisited . . . 121
6.12 A Cautionary Example: Removing the Screening Signal . . . 121
6.12.1 What follows . . . 121
6.12.2 What does not follow . . . 121
6.13 What Genericity Does Not Explain . . . 122
6.13.1 Generalization . . . 122
6.13.2 Conditioning . . . 122
6.13.3 Global convergence . . . 122
6.13.4 Faithful internal structure . . . 122
6.14 Genericity and Physical Symmetry . . . 122
6.15 How C1, C2, and C3 Fit Together . . . 123
6.16 Summary of the Footprint Mechanism . . . 123
6.17 AGOP as a Diagnostic of Low-Dimensional Structure . . . 124
6.17.1 The clean case: multi-index structure . . . 124
6.17.2 Compositional sparsity does not imply low-rank AGOP . . . 126
6.17.3 AGOP can miss nonlinear interactions . . . 126
6.17.4 Why a compositional DAG does not generally produce a block-diagonal AGOP 127
6.17.5 Coordinate dependence . . . 127
6.17.6 A single AGOP cannot identify a hidden DAG by diagonalization . . . 128
6.17.7 Layer-wise AGOP as an empirical diagnostic . . . 129
6.17.8 Target AGOP versus learned-model AGOP . . . 130
6.17.9 Relation to genericity . . . 131
6.17.10 Relation to the staircase . . . 131
6.17.11 Practical use . . . 131
6.17.12 Summary . . . 132

7 Overparameterization and the Geometry of Global Minima 133
7.1 The classical puzzle . . . 134
7.2 Prediction space and parameter space . . . 135
7.3 The dimension-counting argument . . . 135
7.3.1 The regular zero-loss manifold . . . 136
7.3.2 Why the p gradient equations do not contradict this . . . 137
7.4 The Hessian at interpolation . . . 137
7.5 Why degeneracy matters . . . 138
7.6 Volume near a regular global-minimum manifold . . . 138

7.7 Nearby volume is not the same as basin volume . . . 139
7.8 Residual contraction in prediction space . . . 140
7.9 A deterministic convergence backstop . . . 141
7.10 SGD as noisy gradient flow . . . 142
7.11 An isolated local minimum under Langevin dynamics . . . 143
7.12 A global-minimum manifold under Langevin dynamics . . . 144
7.13 Free energy and entropic selection . . . 144
7.14 Why the smooth-manifold picture is incomplete . . . 145
7.15 The real log canonical threshold . . . 146
7.16 The singular version of the entropy argument . . . 147
7.17 What real SGD changes . . . 148
7.17.1 Sampling covariance at a fixed parameter . . . 148
7.17.2 Exact interpolation and regularized stationarity . . . 149
7.17.3 A precise criterion for persistent fluctuations . . . 149
7.17.4 Persistent noise at a strictly convex global minimum . . . 150
7.17.5 Normalized ReLU networks and the rank obstruction . . . 150
7.17.6 Intrinsic noise and the interpretation of low-rank bias . . . 152
7.17.7 Learning rates, point convergence, and diffusion models . . . 153
7.18 Regularization and selection among fitting solutions . . . 153
7.19 Why norm selection can be statistically meaningful . . . 155
7.20 An idealized theorem . . . 156
7.21 The conjectural extension to deep-network SGD . . . 157
7.22 What the argument does not say . . . 158
7.22.1 It does not make the loss convex . . . 159
7.22.2 It does not prove that every degenerate set has a large basin . . . 159
7.22.3 Suboptimal sets can also be degenerate . . . 159
7.22.4 Raw Euclidean volume is parameterization dependent . . . 159
7.22.5 The zero-loss set can be noncompact . . . 160
7.22.6 Other losses require modified geometry . . . 160
7.23 Why overparameterization can make optimization easier . . . 160
7.24 Three levels of explanation . . . 161
7.25 The convergence backstop, with its conditions explicit . . . 161
7.26 Interpolation, prediction, and structural recovery . . . 162
7.27 The bridge to predictive learning . . . 163
7.28 Summary . . . 164

8 Principles of Deep Learning 166
8.1 The central simplification . . . 167
8.1.1 Predictive recovery . . . 167
8.1.2 Structural recovery . . . 167
8.2 A successful finite network is itself a computation . . . 168
8.2.1 Finite is not the same as feasible . . . 168
8.2.2 Learning accuracy and evaluation precision . . . 168
8.3 Efficient computation and compositional sparsity . . . 169
8.3.1 Computational sparsity and regular compositional structure . . . 169
8.4 The learned network is a witness . . . 170
8.5 Optimization: why interpolation is plausible . . . 170
8.5.1 Hessian degeneracy . . . 171

8.5.2 The Langevin picture . . . 172
8.5.3 Equilibrium preference is not a training-time guarantee . . . 173
8.5.4 Exact interpolation and regularized SGD fluctuations . . . 173
8.5.5 Normalized networks, common equilibria, and low-rank bias . . . 174
8.6 Regularization selects among well-fitting solutions . . . 175
8.6.1 An elementary conditional comparison . . . 175
8.7 Product norms and Rademacher complexity . . . 176
8.7.1 A class bound, not a property of one isolated parameter vector . . . 177
8.7.2 From a well-fitting predictor to population control . . . 178
8.7.3 What the regularizer must actually accomplish . . . 178
8.8 The main predictive recovery argument . . . 179
8.9 Why uniqueness is not required for predictive learning . . . 181
8.10 The revised role of genericity . . . 181
8.11 Parity revisited . . . 182
8.12 Generic perturbations and local signals . . . 182
8.12.1 Boolean leakage . . . 182
8.12.2 Gaussian leakage . . . 182
8.13 Staircase learning as a constructive mechanism . . . 183
8.14 Skip connections and traces remain useful . . . 183
8.15 Neural collapse is evidence, not a premise . . . 184
8.16 Three questions for predictive learning . . . 184
8.16.1 Q1: Does the chosen class contain a feasible accurate predictor? . . . 184
8.16.2 Q2: Can the specified training procedure find a suitable fit? . . . 185
8.16.3 Q3: Is the selected predictor controlled well enough to generalize? . . . 185
8.16.4 What is not a fourth predictive requirement . . . 185
8.17 What is theorem and what remains conjectural . . . 185
8.17.1 Established under stated assumptions . . . 185
8.17.2 Idealized but explicit . . . 186
8.17.3 Mechanisms rather than universal prerequisites . . . 186
8.17.4 The remaining broad conjecture . . . 186
8.18 Failure modes . . . 187
8.19 What predictive success does not require . . . 188
8.20 A compact recovery logic . . . 188
8.21 The principle in one sentence . . . 189
8.22 Summary . . . 189

Part II: Computation and Algorithms **191**

9 Efficient Computability, Compositional Sparsity, and Attention **192**
9.1 The Architectural Connection . . . 192
9.2 Attention: Definitions and Scope . . . 193
9.3 When Attention Is Gaussian-Kernel Retrieval . . . 194
9.3.1 Exact equivalence under equal key norms . . . 194
9.3.2 Unequal key norms . . . 194
9.3.3 Layer normalization is not query–key normalization . . . 194
9.3.4 Learned Mahalanobis geometry . . . 195
9.4 Kernel Averaging Is Not a General Approximation Theorem . . . 195
9.5 Margins Give Quantitative Retrieval Bounds . . . 196

9.5.1 Retrieving one item . . . 196
9.5.2 Near top-b sparsity . . . 196
9.6 An Explicit Attention–MLP Computation Model . . . 197
9.6.1 A simple addressing construction . . . 197
9.7 Compositional Approximation with Routing Error . . . 198
9.7.1 Keeping intermediate values in their domains . . . 198
9.7.2 Propagation through the graph . . . 198
9.7.3 A conditional approximation theorem . . . 199
9.7.4 What "dimension-independent" means here . . . 200
9.8 Keys, Parameters, and Operations Are Different Resources . . . 200
9.8.1 Functional arity and implementation fan-in . . . 200
9.9 Efficient Finite-Precision Computation . . . 201
9.9.1 From bit computation to a routed circuit . . . 201
9.9.2 Restoring discrete states after soft retrieval . . . 201
9.9.3 Scope of the construction . . . 202
9.10 Low-Dimensional Relevance and Low-Rank Projections . . . 202
9.10.1 What the rank statement does not imply . . . 203
9.11 Empirical Questions and Predictions . . . 203
9.11.1 Routing accuracy versus constituent accuracy . . . 203
9.11.2 Margin and truncation . . . 204
9.11.3 Local dimension and projection rank . . . 204
9.11.4 Depth, precision, and restoration . . . 204
9.11.5 Learning the decomposition . . . 204
9.12 What Remains Open . . . 204
9.13 Relation to C1, C2, and C3 . . . 204
9.14 Conclusion . . . 205

10 Hardware for Compositionally Sparse Computation (with J. Bates) 206
10.1 The Structural Alignment Argument . . . 207
10.2 System and Numeric Model . . . 207
10.3 Mapping Compositional DAGs to 2D Meshes . . . 208
10.4 Sparse Attention on 2D Meshes . . . 209
10.4.1 KV-Stationary: Coarse-to-Fine Attention . . . 209
10.4.2 Q-Stationary: Systolic Streaming with Early Exit . . . 209
10.5 AM Quantization and Error Analysis . . . 210
10.6 Discussion: Why 2D Meshes Help . . . 210
10.7 Conclusion . . . 210

11 Stepwise Trace Supervision: A Common Computational Principle for Autoregressive and Denoising Models 211
11.1 Introduction . . . 212
11.2 Preliminaries . . . 212
11.2.1 Polynomial-time computation . . . 212
11.2.2 A local alphabet . . . 213
11.2.3 Three forms of supervision . . . 213
11.3 Turing Computation Is a Local Space–Time Rule . . . 214
11.4 Learning the Local Rule . . . 215
11.4.1 Constant-size realization . . . 215

11.5 Trace-Exposed Universality with Complete Local Supervision . . . 216
11.5.1 What the theorem proves . . . 216
11.6 Learning from Observed Traces: The Coverage Problem . . . 217
11.7 A Universal Interpreter . . . 218
11.8 Architectural Depth versus Computational Time . . . 218
11.9 Autoregressive Indexing . . . 219
11.9.1 When autoregressive supervision approaches C3 . . . 219
11.10 Reverse Indexing of the Same Computation . . . 220
11.11 Reverse Diffusion Is Not Inverse Heat Flow . . . 220
11.11.1 Why direct inversion is unstable . . . 221
11.11.2 The diffusion-model problem . . . 221
11.11.3 The score as a denoising quantity . . . 221
11.12 Diffusion Supplies Synthetic Intermediate States . . . 222
11.12.1 Intermediate does not mean easy . . . 222
11.13 Relation to Standard Diffusion Models . . . 223
11.14 Gaussian-Corrupted Computational States . . . 223
11.14.1 Blockwise recovery . . . 224
11.14.2 Robust execution . . . 225
11.15 What the Common Principle Really Is . . . 225
11.16 Relation to Compositional Sparsity . . . 226
11.17 Relation to Associative Computation . . . 226
11.18 A Biological Interpretation: Replay as Partial Trace . . . 227
11.19 The State-Discovery Problem . . . 227
11.20 Limitations and Open Problems . . . 228
11.20.1 Trace exposure is stronger than end-to-end supervision . . . 228
11.20.2 Observed traces may not cover future local states . . . 228
11.20.3 Surface traces are not necessarily faithful traces . . . 228
11.20.4 Standard diffusion is not computational-trace diffusion . . . 228
11.20.5 Approximate execution can accumulate error . . . 229
11.20.6 State discovery remains open . . . 229
11.21 Conclusion . . . 229

12 Return-Conditioned Sequence Modeling as Autoregressive Control 231
12.1 From Value Estimation to Conditional Trajectory Modeling . . . 232
12.1.1 Classical value-based reinforcement learning . . . 232
12.1.2 An alternative offline formulation . . . 233
12.2 Offline Training Is Supervised Conditional Action Prediction . . . 233
12.2.1 What has disappeared . . . 234
12.2.2 What has not disappeared . . . 234
12.3 Return-to-Go as a Partial Trace . . . 234
12.3.1 A delayed reward becomes a local label . . . 234
12.3.2 Connection to C3 . . . 235
12.3.3 A hierarchy of supervision . . . 235
12.4 Inference as Closed-Loop Autoregressive Rollout . . . 235
12.4.1 Desired return . . . 236
12.4.2 Updating the desired return . . . 236
12.4.3 The complete recurrence . . . 236
12.5 Control as Conditional Trajectory Generation . . . 237

12.5.1 A trajectory-level control variable . . . 237
12.6 What Is and Is Not Numerical Integration . . . 237
12.6.1 Policy-only sequence models . . . 238
12.6.2 The dynamical-system analogy . . . 238
12.7 When a World Model Is Added . . . 238
12.7.1 A learned world model . . . 238
12.7.2 Difference-equation form . . . 239
12.8 Credit Assignment: What Has Been Removed? . . . 239
12.8.1 Bellman credit assignment is removed from the training objective . . . 239
12.8.2 Temporal outcome information is supplied directly . . . 240
12.8.3 But causal credit assignment remains . . . 240
12.9 The Role of Attention . . . 240

Part III: Learning and Evolution **241**

13 Bits as a Currency of Inductive Bias and Generalization **242**
13.1 Why Bits? . . . 243
13.1.1 Finite precision makes the question operational . . . 243
13.2 Individual Description Length . . . 243
13.2.1 Kolmogorov complexity . . . 243
13.2.2 Approximate description length . . . 244
13.2.3 Why few parameters do not imply few bits . . . 244
13.3 A Generalization Theorem for Individual Codes . . . 244
13.3.1 Prefix codes . . . 245
13.4 The Finite-Class Bound as a Special Case . . . 246
13.5 Approximate Compression of the Selected Predictor . . . 246
13.6 From Individual Bits to Class Bits: Metric Entropy . . . 247
13.6.1 Covering-number generalization . . . 248
13.7 Rademacher Complexity: The Statistical Language . . . 248
13.8 Dudley: From Covering Bits to Rademacher Complexity . . . 249
13.8.1 The coding interpretation . . . 249
13.8.2 An important qualification . . . 249
13.9 Architecture as a Coding Language . . . 250
13.9.1 Examples . . . 250
13.10 Compositional Sparsity as a Bit Constraint . . . 250
13.10.1 Fixed computation DAG . . . 250
13.10.2 Stability of the composition . . . 251
13.11 Unknown Graph Structure Costs Additional Bits . . . 251
13.11.1 A uniformly stable compositional class . . . 252
13.11.2 Representation versus support discovery . . . 253
13.12 Weight Sharing and Reuse . . . 253
13.13 Sparsity . . . 253
13.14 Low Rank . . . 254
13.14.1 Attention . . . 254
13.15 Quantization and Finite Precision . . . 255
13.15.1 Immediate finite-precision bound . . . 255
13.15.2 Training-time quantization versus post-training compression . . . 255
13.16 Norm Control as a Covering Constraint . . . 256

13.16.1 A Euclidean parameter example . . . 256
13.16.2 Parameter-space norm is not the final object . . . 256
13.17 A Unified Bit Table . . . 257
13.18 Architecture Versus Optimization . . . 257
13.18.1 Architecture defines the coding language . . . 257
13.18.2 Optimization selects a code . . . 257
13.18.3 The architecture need not impose a hard ceiling . . . 258
13.19 Compression of the Selected Solution . . . 258
13.19.1 What must be included in the code . . . 259
13.20 Two Routes from Bits to Generalization . . . 259
13.20.1 Route I: individual code length . . . 259
13.20.2 Route II: class metric entropy . . . 259
13.21 Connection to the Principles of the Book . . . 259
13.21.1 C1: representation . . . 260
13.21.2 C2 and C3: recovery . . . 260
13.21.3 Selection . . . 260
13.21.4 Bits: statistical complexity . . . 260
13.22 What Bits Do Not Explain . . . 260
13.22.1 Bits do not explain recoverability . . . 261
13.22.2 Bits do not identify the correct latent structure . . . 261
13.22.3 Kolmogorov complexity is not computable . . . 261
13.22.4 The metric matters . . . 261
13.22.5 Scale matters . . . 261
13.22.6 A post-hoc coding language is not free . . . 262
13.23 Predictions and Empirical Tests . . . 262
13.23.1 Compressed complexity versus parameter count . . . 262
13.23.2 Different biases can have comparable statistical bit costs . . . 262
13.23.3 Structure discovery may reduce description length . . . 263
13.23.4 Memorizing unstructured exceptions should cost bits . . . 263
13.24 Open Problems . . . 263
13.25 Conclusion . . . 264

14 Multiplicative Norm Regularization: Selection, Complexity, and an Experimental Case Study 266
14.1 Introduction . . . 267
14.2 Norm-Based Motivation . . . 268
14.2.1 Relation to norm-based capacity bounds . . . 268
14.3 Additive and Multiplicative Penalties Are Different . . . 269
14.4 Layerwise Rescaling in Homogeneous Networks . . . 269
14.4.1 Additive regularization favors balanced factorizations . . . 270
14.5 Multiplicative Regularization Couples the Layers . . . 270
14.6 Product and Log-Product Objectives Are Not Equivalent . . . 271
14.7 Original Experimental Setup . . . 271
14.7.1 Binary CIFAR-10 task . . . 271
14.7.2 Multiclass CIFAR-10 . . . 272
14.7.3 Training objectives . . . 272
14.8 Why the Original Performance Comparison Is Exploratory . . . 273
14.8.1 Test-set reuse . . . 273

14.8.2 Optimization failures were discarded . . . 273
14.8.3 Effect sizes . . . 273
14.9 Exploratory Results from the Original Protocol . . . 273
14.9.1 Binary classification and label noise . . . 274
14.9.2 Raw margins . . . 274
14.10 Norm Products and Layerwise Structure . . . 274
14.11 Singular-Value Suppression . . . 275
14.11.1 Stable rank . . . 275
14.12 Exploratory Multiclass Results . . . 276
14.13 A Mechanistic Hypothesis . . . 276
14.14 What We Do Not Claim . . . 276
14.14.1 The Frobenius product is not the unique complexity measure . . . 277
14.14.2 A smaller product does not by itself imply better generalization . . . 277
14.14.3 Parameter flatness is not claimed . . . 277
14.14.4 Rank reduction is not automatically beneficial . . . 277
14.15 Connection to the Principles of the Book . . . 277
14.16 Conclusion . . . 278
14.17 Experiments That Need to Be Done or Repeated . . . 279

15 What the Hopfield Analogy Explains—and What It Does Not 285
15.1 Why the Hopfield Analogy Is Useful . . . 285
15.2 Three Different Dynamical Systems . . . 286
15.2.1 Associative state dynamics . . . 286
15.2.2 Transformer depth . . . 287
15.2.3 Learning dynamics . . . 287
15.2.4 The central distinction . . . 288
15.3 The Scope of the Hopfield–Attention Correspondence . . . 288
15.3.1 Query, key, and value maps need not coincide . . . 288
15.3.2 A Transformer normally performs one attention update per block . . . 289
15.3.3 The correspondence concerns a primitive, not the entire architecture . . . 289

16 A Self-Assembling Cortical Circuit for Generalized Gradient Descent (with Qianli Liao and Liu Ziyin) 290
16.1 From Chapter 15: The Missing Learning Dynamics . . . 291
16.1.1 What backpropagation accomplishes . . . 291
16.2 The Biological Credit-Assignment Problem . . . 292
16.2.1 Weight transport . . . 292
16.2.2 Error transport . . . 292
16.2.3 Synaptic locality . . . 292
16.2.4 The top-level teaching signal remains necessary . . . 292
16.3 The SAL Motif . . . 293
16.3.1 Ascending stream . . . 293
16.3.2 Descending stream . . . 293
16.3.3 Cross-stream couplings . . . 293
16.4 Forward and Feedback Computation . . . 294
16.4.1 Ascending computation . . . 294
16.4.2 Error injection . . . 294
16.4.3 Descending computation . . . 294

16.5 Local Heterosynaptic Plasticity . . . 294
16.5.1 Feedforward synapses . . . 295
16.5.2 Descending synapses . . . 295
16.5.3 Cross-stream plasticity . . . 295
16.6 The Main Conditional Result . . . 295
16.7 Why Positive-Definite Preconditioning Is Enough . . . 296
16.7.1 With weight decay . . . 297
16.8 SAL as a Learned Metric on Parameter Space . . . 297
16.8.1 Conditioning matters . . . 298
16.9 What the Theorem Does—and Does Not—Prove . . . 298
16.9.1 What the theorem proves conditionally . . . 298
16.9.2 What it does not automatically prove . . . 299
16.10 Self-Assembly from Random Initialization . . . 299
16.10.1 Functional rather than literal alignment . . . 299
16.10.2 Why overparameterized feedback may help . . . 300
16.11 Relation to Feedback Alignment . . . 300
16.12 Biological Plausibility at Three Levels . . . 300
16.12.1 Level I: Mathematical locality . . . 300
16.12.2 Level II: Candidate biological mechanisms . . . 301
16.12.3 Level III: Does cortex actually implement SAL? . . . 301
16.13 A Candidate Cortical Implementation . . . 301
16.13.1 Driving versus modulatory interactions . . . 301
16.13.2 One candidate mapping . . . 302
16.13.3 What SAL does not require anatomically . . . 302
16.14 Falsifiable Predictions . . . 302
16.14.1 Prediction 1: functional gradient alignment should emerge . . . 302
16.14.2 Prediction 2: cross-stream plasticity should stabilize as learning becomes competent . . . 303
16.14.3 Prediction 3: feedback width should improve conditioning up to saturation . 303
16.14.4 Prediction 4: performance should track the spectrum of the learned preconditioner . . . 303
16.14.5 Prediction 5: disrupting heterosynaptic plasticity should selectively impair deep credit assignment . . . 303
16.14.6 Prediction 6: feedback need not mirror feedforward weights . . . 304
16.14.7 Prediction 7: different physiological regimes should characterize within-stream and cross-stream pathways . . . 304
16.15 Relation to C1, C2, and C3 . . . 304
16.15.1 C1: representation . . . 304
16.15.2 C2: informative recovery signal . . . 304
16.15.3 C3: supplied trace . . . 304
16.15.4 SAL: implementation of the optimizer . . . 305
16.16 SAL and Genericity . . . 305
16.17 Relation to Chapter 15 . . . 306
16.18 Implications for Artificial Learning Systems . . . 306
16.18.1 Learned feedback pathways . . . 306
16.18.2 Learned preconditioning . . . 306
16.18.3 Local hardware . . . 306
16.18.4 Continual adaptation . . . 307

16.19 Limitations and Open Problems . . . 307
16.19.1 Why should the auxiliary weights reach the required stationary regime? . . . 307
16.19.2 Rank preservation . . . 307
16.19.3 Timescale separation . . . 307
16.19.4 Origin of the teaching signal . . . 307
16.19.5 Spiking and recurrent biological dynamics . . . 307
16.19.6 Timing of local factors . . . 308
16.19.7 Very deep networks . . . 308
16.19.8 SAL does not solve informationally hard targets . . . 308
16.20 Conclusion . . . 308

17 Zeroth-Order Post-Training: Why Effective Dimension Matters (with Y. Gan) 310
17.1 Why Zeroth-Order Optimization? . . . 311
17.1.1 Black-box model access . . . 312
17.1.2 Memory-limited fine-tuning . . . 312
17.1.3 System-level objectives . . . 312
17.1.4 Outer-loop optimization . . . 312
17.2 Nondifferentiable Reward Does Not by Itself Require Zeroth Order . . . 313
17.2.1 What distinguishes ZO from policy gradient? . . . 313
17.3 Gaussian Smoothing and Zeroth-Order Gradient Estimation . . . 314
17.3.1 Why smoothing can help . . . 314
17.4 The Curse of Search Dimension . . . 315
17.5 Evaluation Noise Makes Dimension Still More Important . . . 316
17.6 Nominal Dimension versus Effective Adaptation Dimension . . . 317
17.7 Projected Zeroth-Order Optimization . . . 318
17.7.1 What direction is being estimated? . . . 318
17.8 Structured Mutations as Computational Genes . . . 319
17.8.1 Why this is more biologically plausible . . . 319
17.8.2 Connection to sparse compositionality . . . 320
17.9 Where Can the Low-Dimensional Coordinates Come From? . . . 320
17.9.1 Low-rank adapters . . . 320
17.9.2 Module scales . . . 320
17.9.3 Mixture-of-experts routing . . . 320
17.9.4 Prompt and context variables . . . 321
17.9.5 Decoding parameters . . . 321
17.9.6 Training controls . . . 321
17.10 Three Families of Zeroth-Order Methods . . . 321
17.10.1 ZO gradient estimation . . . 321
17.10.2 Direct search . . . 321
17.10.3 Distribution search . . . 321
17.11 Natural Evolution Strategies . . . 322
17.12 CMA-ES and Adaptive Search Geometry . . . 322
17.12.1 Why CMA-ES reinforces the effective-dimension principle . . . 323
17.13 ZO-Friendly Control Surfaces in LLM Systems . . . 323
17.13.1 Very low dimension: decoding and system controls . . . 323
17.13.2 Low dimension: routing and mixtures . . . 323
17.13.3 Intermediate dimension: structured adapters . . . 324
17.13.4 Full parameter space . . . 324

17.14 Memory-Efficient Zeroth-Order Fine-Tuning . . . 324
17.15 Hybrid First-Order and Zeroth-Order Optimization . . . 324
17.15.1 First-order inner loop . . . 325
17.15.2 Zeroth-order outer loop . . . 325
17.16 Relation to Reinforcement-Learning Post-Training . . . 325
17.16.1 Policy-gradient RL . . . 325
17.16.2 Evolution strategies . . . 326
17.16.3 Neither dominates universally . . . 326
17.17 Experiments Suggested by the Theory . . . 326
17.17.1 Experiment 1: hold model size fixed and vary search dimension . . . 326
17.17.2 Experiment 2: structured versus random subspaces . . . 327
17.17.3 Experiment 3: full-weight versus projected ZO . . . 327
17.17.4 Experiment 4: evolutionary genes . . . 327
17.17.5 Experiment 5: reward noise . . . 328
17.17.6 Experiment 6: hybrid optimization . . . 328
17.17.7 Experiment 7: estimate intrinsic adaptation dimension . . . 328
17.18 Failure Modes . . . 328
17.18.1 No low-dimensional adaptation manifold . . . 328
17.18.2 Poor subspace alignment . . . 329
17.18.3 Highly noisy reward . . . 329
17.18.4 Rare-event objectives . . . 329
17.18.5 Excessively small perturbations . . . 329
17.18.6 Excessively large perturbations . . . 329
17.18.7 Off-manifold parameter perturbations . . . 329
17.19 Connection to Sparse Compositionality . . . 330
17.19.1 C1 and zeroth-order search . . . 330
17.19.2 Genericity still matters . . . 330
17.20 Evolution as Structured Search . . . 330
17.21 Conclusion . . . 331

Part IV: Random Topics **333**

18 Sparse Compositionality, Genericity, and the Random Hierarchy Model **334**
18.1 Why the Random Hierarchy Model Matters . . . 335
18.2 The Random Hierarchy Model in Minimal Form . . . 335
18.2.1 The characteristic sample scale . . . 336
18.3 C1: When the Hierarchical Classifier Is Compositionally Sparse . . . 336
18.3.1 Generative sparsity does not by itself imply inverse sparsity . . . 336
18.4 C2: Randomness as a Source of Genericity . . . 337
18.4.1 Annealed symmetry . . . 337
18.4.2 Quenched asymmetry . . . 337
18.5 A Precise Low-Order Footprint . . . 338
18.5.1 RMS footprint across hierarchy realizations . . . 339
18.6 Random Finite-Size Fluctuations . . . 339
18.6.1 Application to the hierarchy . . . 339
18.7 Detectability from Finite Data . . . 340
18.8 The General Detectability Formula . . . 341
18.8.1 Searching over many footprints . . . 341

18.9 Relation to the RHM Threshold . . . 342
18.10 Depth, Input Size, and the Meaning of Exponential Scaling . . . 343
18.11 The Staircase Interpretation . . . 343
18.11.1 Stage 1: a weak footprint becomes detectable . . . 343
18.11.2 Stage 2: synonymous observations are quotiented . . . 344
18.11.3 Stage 3: a new coordinate system is created . . . 344
18.11.4 Stage 4: the next rung becomes visible . . . 344
18.12 Progressive Quotienting by Synonym Invariance . . . 344
18.13 What Is Established and What Is Interpretation . . . 345
18.13.1 Structural facts . . . 345
18.13.2 Statistical calculation . . . 345
18.13.3 Dynamical interpretation . . . 346
18.14 A Statistical Threshold versus a Recovery Threshold . . . 346
18.15 What Would Happen in a Nongeneric Hierarchy? . . . 347
18.16 Predictions Beyond the Original RHM . . . 347
18.16.1 Prediction 1: directly measure the footprint . . . 347
18.16.2 Prediction 2: footprint magnitude should predict the transition sample . . . 347
18.16.3 Prediction 3: artificially suppressing the footprint should delay learning . . . 348
18.16.4 Prediction 4: artificially increasing asymmetry should lower the threshold . . . 348
18.16.5 Prediction 5: intermediate representations should increase the next-level footprint . . . 348
18.16.6 Prediction 6: supplied intermediate labels should bypass the first threshold . . . 348
18.16.7 Prediction 7: kernel methods should not receive the same progressive coordinate advantage . . . 349
18.17 Relation to Sparse Compositionality . . . 349
18.18 Relation to Genericity . . . 349
18.19 Relation to Genericity as a Property of the Observed Problem . . . 350
18.20 A Compact Summary of the Mechanism . . . 350
18.21 Conclusion . . . 351

19 The Hippocampal Scaffold and Compositional Sparsity 353
19.1 Fast Episodic Indexing and Slow Structural Learning . . . 353
19.2 A Minimal Computational Model . . . 354
19.3 Pattern Separation as an Indexing Primitive . . . 355
19.4 From Episodic Indices to a Scaffold . . . 355
19.5 Graph Sparsity Is Not Yet Compositional Sparsity . . . 356
19.6 The Scaffold as a Partial Relational Trace . . . 356
19.7 From Episodic Indices to Latent Variables . . . 357
19.8 Replay as Structured Training Data . . . 358
19.9 Cortical Abstraction as Parameter Sharing . . . 358
19.10 Learning the Algebra of Composition . . . 359
19.11 Consolidation as Compilation . . . 359
19.12 Episodic and Semantic Memory . . . 360
19.13 Relation to Classical Hippocampal Theories . . . 360
19.13.1 Cognitive maps . . . 361
19.13.2 Pattern separation . . . 361
19.13.3 Indexing theory . . . 361
19.13.4 Complementary learning systems . . . 361

19.13.5 Predictive and successor representations . . . 361
19.14 An Empirical Example: Structured Latent Geometry . . . 362
19.15 What the Scaffold Hypothesis Predicts . . . 362
19.15.1 Prediction 1: relationally structured replay should aid abstraction . . . 362
19.15.2 Prediction 2: episodic separation and semantic abstraction should have opposite geometry . . . 362
19.15.3 Prediction 3: cortical rules should transfer beyond stored episodes . . . 363
19.15.4 Prediction 4: hippocampal dependence should decrease as cortical predictive structure improves . . . 363
19.15.5 Prediction 5: a scaffold should reduce recovery difficulty . . . 363
19.16 A Direct Artificial Experiment . . . 363
19.16.1 Evaluation . . . 364
19.17 What Is Established and What Is Hypothesis . . . 364
19.17.1 Established biological observations . . . 364
19.17.2 Computationally plausible interpretation . . . 365
19.17.3 Open hypothesis . . . 365
19.18 Relation to World Models . . . 365
19.19 Relation to Sparse Compositionality . . . 366
19.20 Relation to C1, C2, and C3 . . . 366
19.21 Conclusion . . . 367

20 Mixture of Experts and Spatial Manifolds 368
20.1 The Geometric Primitives of MoE Networks . . . 368
20.2 Manifold Approximation and Theorem 4.8 . . . 368
20.3 Deep MoEs and Exponential Compositional Capacity . . . 369
20.4 Case Study: The DeepSeek-V3 MoE Architecture . . . 369
20.4.1 Architectural Layering Strategy . . . 370
20.4.2 The Multi-Expert Routing Equations . . . 370
20.4.3 Internal Node Projections: The SwiGLU Block . . . 370
20.4.4 Gating and Capacity-Balanced Routing . . . 370

21 The Architecture of a Compositional Register Machine 372
21.1 From Sparse Compositionality to an Architectural Hypothesis . . . 373
21.1.1 What this constraint does guarantee . . . 373
21.1.2 What it does not guarantee . . . 373
21.2 The Abstract Compositional Register Machine . . . 374
21.2.1 Working memory . . . 374
21.2.2 Operator library . . . 374
21.2.3 Controller . . . 375
21.2.4 Finite control state . . . 375
21.3 Relational Routing Rather Than Independent Scores . . . 375
21.3.1 Pairwise controller . . . 376
21.3.2 Factorized selection . . . 376
21.4 Bounded Arity and Local Capacity . . . 376
21.5 Execution Universality Is Different from Learning . . . 377
21.6 Metadata-Only Routing and Oblivious Algorithms . . . 377
21.7 Content-Dependent Routing . . . 378
21.7.1 Compact content keys . . . 378

21.7.2 Exact discrete keys . . . 378
21.7.3 Frozen random projections . . . 379
21.7.4 Freezing the projection does not freeze the representation . . . 379
21.8 Why Evolution Is Attractive—but Not Necessary . . . 379
21.9 Differentiable Sparse Routing: Sparsemax and Entmax as Relaxations of Program Search . . . 380
21.9.1 Softmax, Sparsemax, and Entmax . . . 381
21.10 What Sparse Routing Does—and Does Not—Mean . . . 382
21.10.1 Fixed structure versus conditional computation . . . 382
21.11 Preserving Bounded Arity . . . 383
21.11.1 Sparse operand slots . . . 383
21.12 Relaxing Operator Selection . . . 384
21.13 The Sparsemax Jacobian . . . 384
21.13.1 The benefit . . . 385
21.13.2 The danger . . . 385
21.14 A Continuation Strategy . . . 385
21.15 Hardening the Learned Program . . . 386
21.15.1 Retraining after hardening . . . 386
21.16 Structural Complexity Penalties . . . 386
21.17 Unknown Structure Carries a Statistical Cost . . . 387
21.18 Differentiable Sparsity Does Not Solve C2 . . . 387
21.19 Computational Cost . . . 388
21.20 Three Ways to Learn the Same CRM Controller . . . 388
21.20.1 Evolutionary routing . . . 389
21.20.2 Policy-gradient routing . . . 389
21.20.3 Entmax relaxation . . . 389
21.21 Relation to C3: Trace Supervision . . . 390
21.22 Interpretation . . . 390
21.23 The Operator Library . . . 391
21.23.1 Reusable constituents . . . 391
21.23.2 Learning a new constituent . . . 391
21.24 Compression of Composition: Compiling Subprograms into New Operators . . . 391
21.24.1 From a subprogram to a primitive . . . 392
21.24.2 Two different forms of compression . . . 392
21.24.3 Depth collapse for familiar computations . . . 393
21.24.4 Offline versus online compilation . . . 393
21.24.5 When should a new operator be created? . . . 394
21.24.6 Exact and approximate compilation . . . 394
21.24.7 Verification and fallback . . . 395
21.24.8 A hierarchy of learned programs . . . 395
21.24.9 Relation to sparse compositionality . . . 396
21.24.10 Relation to the staircase picture . . . 396
21.24.11 A testable prediction . . . 396
21.24.12 From learning programs to learning the instruction set . . . 397
21.25 Computational Depth and Primitive Granularity . . . 398
21.25.1 Depth is relative to a primitive vocabulary . . . 398
21.25.2 Coarsening a DAG requires explicit interfaces . . . 398
21.25.3 Bounded gate arity does not guarantee bounded module arity . . . 399

21.25.4 Representations at execution boundaries . . . 399
21.25.5 Regrouping, representation learning, and compilation . . . 400
21.25.6 Granularity is not automatically semantic or spatial scale . . . 401
21.25.7 Choosing a useful granularity . . . 401
21.26 Reusability Across Tasks: When a Composition Should Become a Primitive . . . 402
21.26.1 Sparse decompositions are not unique . . . 402
21.26.2 Reuse as an additional selection pressure . . . 403
21.26.3 When should a recurring computation become a primitive? . . . 403
21.26.4 Reuse can compress execution as well as description . . . 404
21.26.5 Reusability is relative to the task distribution . . . 404
21.26.6 Reuse reduces ambiguity but does not guarantee identifiability . . . 405
21.26.7 From repeated programs to a learned instruction set . . . 405
21.26.8 Connection to the staircase mechanism . . . 406
21.26.9 Relation to cultural accumulation . . . 406
21.26.10 An experimental test . . . 406
21.26.11 Controls . . . 407
21.26.12 Relation to sparse compositionality . . . 407
21.26.13 The resulting picture . . . 408
21.27 Parsimony as Description Length . . . 408
21.28 Continual Learning: What Freezing Does and Does Not Solve . . . 409
21.28.1 Protecting programs as well as operators . . . 409
21.29 Stability of Continuous Composition . . . 410
21.30 Digital Operators . . . 411
21.31 Working Memory, Garbage Collection, and Universality . . . 411
21.31.1 Liveness . . . 411
21.31.2 Fixed capacity changes computational power . . . 412
21.32 Sparse Parallel Execution . . . 412
21.33 Relation to Transformers . . . 412
21.34 Experiment 0: Parity as an Execution Sanity Check . . . 413
21.34.1 Protocol . . . 413
21.35 Experiment 1: Non-Associative Program Execution . . . 414
21.35.1 Training . . . 414
21.35.2 Testing . . . 414
21.35.3 Why this is a real routing test . . . 414
21.36 Experiment 2: Structural Routing with Sorting Networks . . . 414
21.37 Experiment 3: Content-Dependent Conditional Control . . . 415
21.37.1 Stage A: exact semantic tags . . . 415
21.37.2 Stage B: high-dimensional representations . . . 415
21.38 Experiment 4: Learning a Reusable Operator Library . . . 416
21.39 Experiment 5: How Should Hard Routing Be Learned? . . . 416
21.39.1 A Critical Hardening Test . . . 417
21.40 Experiment 6: Stability of Continuous Operators . . . 417
21.41 Fair Baselines . . . 418
21.42 What Would Support the CRM Hypothesis? . . . 418
21.43 What Would Weaken the Hypothesis? . . . 419
21.44 Relation to C1, C2, and C3 . . . 419
21.44.1 C1: sparse compositionality . . . 419
21.44.2 C2: genericity . . . 419

21.44.3 C3: program traces . . . 419
21.45 Relation to Chapter 17 . . . 420
21.46 Relation to the Associative Turing Machine . . . 420
21.47 Conclusion . . . 420

22 World Models Before Language **422**
22.1 A Minimal Definition of a World Model . . . 422
22.2 Sparse Dynamics of the Physical World . . . 423
22.3 State Estimation Is an Inverse Problem . . . 424
22.4 Delay Embeddings: When History Determines State . . . 425
22.4.1 Takens' theorem . . . 425

23 Can 3D Structure Be Discovered from 2D Experience? **427**
23.1 Historical Motivation: Recognition Without Explicit 3D Reconstruction . . . 427
23.2 A Different Question . . . 428
23.3 The Generative Setting . . . 429
23.4 Recognition Is Not Evidence for 3D Structure . . . 429
23.5 The Occlusion Problem . . . 430
23.6 The 3D-Emergence Hypothesis . . . 431
23.7 Four Core Hypotheses . . . 432
23.7.1 H1: Recognition does not require a 3D model . . . 432
23.7.2 H2: View prediction encourages pose-sensitive latent structure . . . 432
23.7.3 H3: Occlusion creates pressure for persistent object state . . . 432
23.7.4 H4: Genuine geometric structure should compose . . . 433
23.8 What Would Count as Discovering 3D? . . . 433
23.8.1 Novel-view prediction . . . 433
23.8.2 Pose decodability . . . 433
23.8.3 Group-composition consistency . . . 434
23.8.4 Persistence of occluded structure . . . 434
23.8.5 Cross-object transfer of motion . . . 434
23.8.6 Extrapolation beyond trained trajectories . . . 435
23.9 Experimental Environment . . . 435
23.10 Architectural Baselines . . . 435
23.10.1 Feedforward classifier . . . 435
23.10.2 Multi-view contrastive encoder . . . 435
23.10.3 Recurrent or state-space predictor . . . 435
23.10.4 Transformer over view sequences . . . 436
23.10.5 Explicit 3D model . . . 436
23.10.6 Object-centric latent world model . . . 436
23.11 Experiment 1: Classification versus Prediction . . . 436
23.12 Experiment 2: Object Permanence Through Occlusion . . . 437
23.13 Experiment 3: Composition of Rotations . . . 437
23.14 Experiment 4: Reusing Motion Operators Across Objects . . . 438
23.15 Experiment 5: What Signal Makes 3D Discoverable? . . . 438
23.16 Experiment 6: Fast Episodic Binding and Slow Structural Learning . . . 439
23.17 A Minimal Latent Factorization . . . 439
23.18 Connection to Sparse Compositionality . . . 440
23.19 Relation to World Models . . . 440

23.20 Relation to Genericity . . . 441
23.21 Distinguishing Geometry from Memorization . . . 441
23.21.1 Held-out viewpoints . . . 442
23.21.2 Held-out compositions . . . 442
23.21.3 Held-out objects . . . 442
23.22 Metrics . . . 442
23.23 Possible Outcomes . . . 442
23.23.1 Outcome A: Recognition without geometry . . . 442
23.23.2 Outcome B: Prediction produces geometry . . . 443
23.23.3 Outcome C: Geometry requires explicit architectural bias . . . 443
23.23.4 Outcome D: A non-Euclidean latent solves the task . . . 443
23.24 Falsification Criteria . . . 443
23.25 From the 1990 Question to the Present One . . . 443
23.26 Conclusion . . . 444

24 Interlude: Most Real Numbers Do Not Exist 445
24.1 Several Different Notions of Existence . . . 445
24.2 The Continuum Is Much Larger Than Finite Description . . . 446
24.3 Algebraic and Transcendental Numbers . . . 446
24.4 Computable Real Numbers . . . 447
24.5 Definability Is Not the Same as Computability . . . 448
24.6 Algorithmic Randomness . . . 448
24.7 Finite Measurement Gives Finite Information . . . 449
24.8 Finite Precision . . . 450
24.9 Infinite Precision Can Hide Infinite Information . . . 450
24.10 Exact-Real Universality and Physical Resources . . . 451
24.11 Why the Bit Model Matters . . . 452
24.12 A Modulus of Continuity as a Precision Budget . . . 452
24.13 Description Length and Learning . . . 453
24.14 The Continuum Remains Useful . . . 453
24.15 What "Most Real Numbers Do Not Exist" Means . . . 454
24.16 Conclusion . . . 454

25 Finding the 1983 Circuit in the Connectome: the Tale of a SuperNeuron 456
25.1 Introduction . . . 456
25.2 Main Neurons in the Optical Lobes . . . 457
25.2.1 The Local EMDs: T4 and T5 Cells . . . 457
25.2.2 Wide-Field Pool Cells: The LPTCs . . . 457
25.2.3 Dendrites of the CT1 Cell vs. the vCH Cell in *Drosophila* . . . 457
25.2.4 The CT1 Cell: Local, Polyadic Interception . . . 458
25.2.5 The vCH Cell: Wide-Field, Dendro-Dendritic Integration . . . 458
25.3 Local Motion Detection by T4/T5 Cells and CT1 Dendrites . . . 459
25.3.1 Upstream Modulation and Null-Direction Suppression . . . 459
25.3.2 Extreme Compartmentalization . . . 459
25.3.3 Recurrent Connectivity: Local Feedback and Gain Control . . . 460
25.4 The Core Hypothesis: Figure-Ground Discrimination via vCH (Shunting) Inhibition 460
25.5 The Output 'X Cell': LPTCs as Behavioral Integrators . . . 461
25.6 Historical Context: Figure Detection (FD) Cells in the Blowfly . . . 461

25.7 Biophysical Implementation of Shunting Inhibition . . . 462
25.8 Predictions and the Missing Link . . . 463

26 Frozen Interpreter and Cultural Evolution 465
26.1 Two Channels of Adaptation . . . 465
26.2 Prompt Selection Is a Learning Problem . . . 466
26.2.1 A simple generalization bound . . . 467
26.3 Culture as Programs for a Frozen Interpreter . . . 467
26.4 Interpreter Complexity and Program Complexity . . . 468
26.4.1 The model–program tradeoff . . . 469
26.5 Evolution of External Programs . . . 469
26.5.1 Meta-programs . . . 470
26.6 What Counts as Cumulative Culture? . . . 470
26.7 Intelligent Search . . . 471
26.8 Identification Is Not Execution . . . 471
26.9 Bounded Cultural Resources Imply a Ceiling . . . 472
26.10 Frozen Weights Alone Do Not Imply a Finite Improvement Sequence . . . 473
26.11 Experimental Test . . . 473
26.12 What the Initial Experiments Show . . . 474
26.12.1 Search was not reliably intelligent . . . 474
26.12.2 Inheritance was not reliably cumulative . . . 474
26.12.3 Model size did not solve the problem . . . 475
26.12.4 Identification and execution separated sharply . . . 475
26.13 What Would Constitute Strong Evidence? . . . 475
26.14 Limits . . . 476
26.14.1 No universal search advantage . . . 476
26.14.2 No universal semantic verifier . . . 476
26.14.3 Universality does not imply practicality . . . 476
26.15 Relation to the Framework of This Book . . . 476
26.16 Conclusion . . . 477

27 Puzzles 479
27.1 Why Are Task-Relevant Variables Often Linearly Decodable from Pretrained Representations—and How Can This Be Exploited in Post-Training? . . . 479
27.1.1 Linear decodability is not causal use . . . 481
27.1.2 Connection to the framework of this book . . . 481
27.2 Weights as Compiled Computation, Prompts as Programs . . . 482
27.2.1 Slow adaptation: compiling computation into weights . . . 482
27.2.2 Fast adaptation: programming the frozen interpreter . . . 483
27.2.3 A third level: external program libraries . . . 483
27.2.4 Learning in parameter space versus program space . . . 484
27.2.5 Connection to compilation and compositionality . . . 484
27.2.6 Frozen models and cultural accumulation . . . 485
27.2.7 The resulting picture . . . 486
27.3 Hermite Staircases, Leap Complexity, and the Open Problem of General Compositional Recovery . . . 487
27.3.1 Gaussian Multi-Index Models . . . 487
27.3.2 The Hermite Expansion . . . 488

27.3.3 Degree Is Not the Whole Story . . . 489
27.3.4 Leap Complexity . . . 489
27.3.5 An Example . . . 490
27.3.6 Existing Gaussian Staircase Results . . . 490
27.3.7 Leap Complexity Is Not Compositional Sparsity . . . 491
27.3.8 The Coordinate Problem . . . 492
27.3.9 Open Problem 1: From Hermite Leap to General Compositional DAGs . . . 493
27.3.10 Open Problem 2: An Adaptive Spectral Staircase . . . 493
27.3.11 Open Problem 3: A Coordinate-Invariant or Coordinate-Penalized Leap . . . 494
27.3.12 Open Problem 4: Beyond Gaussian and Product Measures . . . 495
27.3.13 Open Problem 5: From Population or Modified Dynamics to End-to-End SGD 496
27.3.14 Open Problem 6: Relating Leap to Compositional Arity . . . 496
27.3.15 Relation to C1 and C2 . . . 497
27.3.16 The Revised Staircase Question . . . 498

28 Five Open Problem Directions: Representation, Recovery, and Selection 499
28.1 Problem 1: Characterize Economical Compositional Representations . . . 499
28.1.1 What is already established . . . 500
28.1.2 A formal representation problem . . . 500
28.1.3 Possible approaches . . . 501
28.2 Problem 2: Which Coordinates Expose Compositional Structure? . . . 502
28.2.1 Local dependence is coordinate dependent . . . 502
28.2.2 Charge for the coordinate transformation . . . 503
28.2.3 Possible approaches . . . 503
28.3 Problem 3: Quantitative Genericity and Detectable Footprints . . . 504
28.3.1 A finite-sample formulation . . . 504
28.3.2 The desired quantitative statement . . . 505
28.3.3 Possible approaches . . . 505
28.4 Problem 4: Prove a Compositional Staircase Recovery Theorem . . . 506
28.4.1 What a theorem should establish . . . 507
28.4.2 What is already known . . . 507
28.4.3 Possible approaches . . . 507
28.5 Problem 5: When Does Norm-Based Selection Favor Reusable Structure? . . . 509
28.5.1 The established generalization mechanism . . . 509
28.5.2 An elementary conditional guarantee . . . 509
28.5.3 The remaining structural question . . . 510
28.5.4 Possible approaches . . . 510

29 Sparse Digital Computation, Smooth Lifts, and the Cost of Contraction 513
29.1 The Objects and the Conventions . . . 513
29.1.1 Boolean gates and numerical modules . . . 514
29.1.2 Encoding and decoding are part of the computation . . . 514
29.1.3 Local versus global contraction . . . 514
29.2 Error Propagation and the Three Lipschitz Regimes . . . 515
29.2.1 A qualification about depth and skip connections . . . 516
29.3 From Regular Real Constituents to Boolean Approximation . . . 516
29.3.1 The contractive improvement . . . 517
29.3.2 Approximation rate is not polynomial-time bit complexity . . . 517

29.4 Exact Real Extensions of Boolean Gates . . . 518
29.4.1 The smooth multilinear lift . . . 518
29.4.2 An optimal Lipschitz lift . . . 519
29.5 Smooth Lifts Arbitrarily Close to Nonexpansive . . . 519
29.5.1 Why exact nonexpansive smooth interpolation can fail . . . 520
29.6 Strict Contraction with Fixed Boolean Codes . . . 521
29.6.1 The smooth version has the same infimum . . . 522
29.7 Exact Contractive Computation with Changing Code Scales . . . 522
29.7.1 An exact Lipschitz construction . . . 522
29.7.2 Smooth contractive realization: a small slack is needed . . . 523
29.7.3 Higher derivatives in shrinking physical coordinates . . . 524
29.8 Absolute Error, Relative Error, and Required Precision . . . 524
29.8.1 Normalized error reveals the tradeoff . . . 525
29.8.2 The final decoder restores the scale . . . 525
29.8.3 What the bit budget does and does not mean . . . 525
29.9 Contractivization by Rescaling Is Not a Free Structural Result . . . 525
29.10 Continuum Limits on a Coherent Coarse DAG . . . 526
29.11 Regularity Conditions Stated Directly on Digital Tables . . . 527
29.11.1 A digital contraction condition allowing quantization error . . . 527
29.11.2 Higher-order finite differences . . . 528
29.11.3 Output quantization must respect derivative order . . . 529
29.12 Efficient Computation Does Not Imply Uniform Analytic Regularity . . . 529
29.12.1 Coherent smooth approximants can lose regularity . . . 529
29.12.2 Strict contraction does not imply higher smoothness . . . 530
29.13 What the Theorems Establish . . . 530
29.14 The Remaining Research Problems . . . 531
29.14.1 Coarse-graining an efficient circuit family . . . 531
29.14.2 Efficient certificates of digital regularity . . . 532
29.14.3 Optimal smoothness–margin–depth tradeoffs . . . 532
29.14.4 Jointly optimizing precision and code scales . . . 532
29.14.5 The distinction between depth limits and resolution limits . . . 533
29.15 Conclusion . . . 533

30 Learning Difficult Inverses by Forward Simulation 535
30.1 The basic idea . . . 535
30.2 Why the two directions may have different difficulty . . . 536
30.3 The ideal result . . . 537
30.4 Proof . . . 538
30.5 The approximate case . . . 538
30.6 Risk transfer from synthetic to real data . . . 539
30.7 What synthetic data do and do not provide . . . 540
30.8 Asymmetric sample complexity . . . 541
30.9 The importance of sampling from the correct X distribution . . . 541
30.10 Stochastic forward models are preferable . . . 542
30.11 The inverse should usually be probabilistic . . . 543
30.12 Connection with Bayesian inversion . . . 543
30.13 Connection with regularized inverse problems . . . 544
30.14 Combining real and synthetic pairs . . . 545

30.15Iterative refinement . . . 545
30.16Application to image reconstruction from brain activity . . . 546
30.17A general principle . . . 547
30.18Discussion . . . 548

31 Learning Without Weight Updates: Context, Memory, and Cultural State 549
31.1 Learning need not mean changing weights . . . 549
31.2 Three forms of learning . . . 550
31.3 The adaptive state . . . 551
31.4 Frozen weights define a family of hypotheses . . . 551
31.5 Context as a program . . . 552
31.6 A universality observation . . . 552
31.7 Attention and associative memory . . . 553
31.8 Transient context is not yet persistent learning . . . 553
31.9 Cultural learning . . . 554
31.10Cumulative culture . . . 555
31.11Where is the learning algorithm? . . . 556
31.12Generalization with frozen weights . . . 556
31.13Context as fast parameters . . . 557
31.14The limits of frozen-weight learning . . . 558
31.15Performance ceilings and indefinitely many improvements . . . 558
31.16An extended-system view of intelligence . . . 559
31.17A possible definition . . . 559
31.18A hierarchy of adaptation . . . 560
31.19A minimal experimental paradigm . . . 560
31.20The main principle . . . 561

Appendix 563

A Stability, ERM, and the Foundations of Learnability 564
A.1 Learning setup and ERM . . . 564
A.2 CV_{loo} stability . . . 565
A.3 Stability and generalization . . . 565
A.4 Stability as a modeling requirement . . . 565
A.5 Implementation . . . 566
A.5.1 The Evolutionary Router and AM Bank . . . 566
A.5.2 The Core ATM Architecture . . . 567
A.5.3 Integration with Evolutionary Algorithms . . . 569

B Compositionality in Machine Learning and Physics 570
B.1 Introduction . . . 570
B.2 Definitions and Setup . . . 572
B.3 From Efficient Computation to DAGs . . . 573
B.3.1 From arithmetic circuits to neural DAGs . . . 574
B.3.2 Computation as compositional structure . . . 574
B.4 ML Approximation Theorem . . . 574
B.4.1 Local constructive approximation with explicit constants . . . 575
B.4.2 Global error propagation and optimal budget . . . 576

B.4.3 The theorem . . . 577
B.4.4 Depth separation . . . 577
B.5 Finite-Horizon Compilation: From Uniform Simulators to Algorithmic Compositionality . . . 578
B.5.1 Setup . . . 578
B.5.2 Compilation theorem (algorithmic, safe) . . . 578
B.6 Learning Theory and Optimization . . . 579
B.7 Limits of the Framework . . . 580
B.8 Conclusion . . . 582

C Faster Attention **584**
C.1 The Adjacency Distance Chain . . . 584
C.2 Utilizing the Lower Bound for Pruning . . . 584
C.3 Formal Constraint: The Metric Continuity Hypothesis . . . 585
C.4 Experimental Verification . . . 585
C.5 Proposed Algorithm: Recursive Distance Bounding . . . 586
C.6 Comparison with Global Clustering and Hashing Approaches . . . 586
C.6.1 LSH and k-means Clustering . . . 586
C.6.2 Global vs. Recursive Local Metrics . . . 586
C.6.3 Summary of Complexity Drivers . . . 587
C.7 Background: Accelerating Transformer Attention with Clustering Algorithms . . . 587
C.7.1 Fast Transformers with Clustered Attention [110] . . . 587
C.7.2 Reformer: The Efficient Transformer [52] . . . 587
C.7.3 Routing Transformers [95] . . . 588
C.7.4 Neural Clustering Attention for Efficient and Effective Transformer [122] . . . 588
C.7.5 Mathematical Relationship . . . 589

D The Hippocampal Scaffold and Compositional Sparsity **590**
D.1 Introduction: The Memory Palace . . . 590
D.2 The Variables of Experience . . . 590
D.3 Mathematical Foundations of the Hippocampal Index . . . 590
D.3.1 The Top-K Projection Mechanism . . . 591
D.3.2 Locality-Sensitive Hashing (LSH) . . . 591
D.4 Building the Scaffold Graph . . . 591
D.5 Connection to Existing Theories . . . 591
D.6 Conclusion . . . 592

E How Many Boolean Functions Are in P Versus Not in P **593**
E.1 Counting all Boolean functions . . . 593
E.2 A general upper bound on the number of functions computable with polynomial descriptions . . . 593
E.3 Implication for P . . . 594
E.4 Almost all Boolean functions are not in P . . . 594
E.5 Connection to structured representations (example: local Fourier-polynomial compositions) . . . 594

F Boolean Circuits and a Path to "Superintelligence" (by D. Koplow) 595
F.1 Boolean Circuits . . . 595
F.2 Learning . . . 596
F.2.1 Reasoning . . . 596
F.2.2 Measuring Learnability . . . 597
F.3 The Computational Cliff . . . 599
F.3.1 Problem Formulation . . . 599
F.3.2 The Statistical Obfuscation Construction . . . 600
F.3.3 Theoretical Guarantees . . . 601
F.3.4 Tool Synthesis . . . 604

G Gradient Formulas for Iterated Functional Compositions 606
G.1 The Univariate Case ($d = 1$) . . . 606
G.2 The Multivariate Case ($d > 1$) . . . 607
G.2.1 Gradient of a Scalar Loss Function . . . 607
G.3 Structural and Spectral Implications . . . 608
G.3.1 The Stability of Deep Iterated Maps . . . 608
G.3.2 Connection to Dynamical Systems . . . 608

H Principles of Deep Learning 609
H.1 Principle I: Sparse Compositionality . . . 610
H.1.1 Hierarchy follows from sparse compositionality . . . 610
H.1.2 Modularity and reuse of modules . . . 610
H.1.3 Transfer learning . . . 611
H.1.4 Interpretability from consistent compositionality . . . 611
H.1.5 Non-uniqueness of sparse decompositions . . . 611
H.2 Principle II: Genericity of Learnable Targets . . . 612
H.2.1 Genericity from invariance to the choice of the origin of the coordinates . . . 612
H.2.2 Genericity ensures good gradients for optimization . . . 612
H.3 Two independent but complementary principles . . . 612
H.4 Conclusion . . . 613

I The Geometry of Overparameterized Systems: Krull and Bézout 614
I.1 Introduction . . . 614
I.2 Dimensionality: Krull's Height Theorem . . . 614
I.3 Geometric Complexity: Generalized Bézout's Theorem . . . 615
I.4 Maximal Degeneracy at the Origin . . . 615
I.5 Real vs. Complex Solution Varieties ($\mathbb{R}^d$ vs. $\mathbb{C}^d$) . . . 615
I.5.1 Dimension Drop in Real Loci . . . 616
I.5.2 Overparameterization and Smooth Real Points . . . 616
I.6 Local Geometry and the Neural Tangent Kernel . . . 616
I.6.1 Tangent Space and Flat Directions . . . 616
I.6.2 NTK Rank Defect and Singular Loci . . . 616
I.7 Summary of Landscape Regimes . . . 617

J Unsupervised Ensembling of Verifiers **618**
J.1 The Problem . . . 618
J.2 Verifier Quality . . . 619
J.3 Triplet Conditional Independence . . . 620
J.4 Observable Moment Factorization . . . 620
J.5 Why Second- and Third-Order Moments Are Enough . . . 621
J.6 The Unavoidable Orientation Ambiguity . . . 622
J.7 When Conditional Independence Fails . . . 622
J.8 FUSE: Transforming Scores to Improve the Moment Model . . . 623
J.9 From Verifier Reliabilities to Pseudo-Labels . . . 624
J.10 What Makes Unsupervised Verification Possible? . . . 624
J.11 Failure Modes . . . 625
J.11.1 Common-mode errors . . . 625
J.11.2 Weak verifiers . . . 625
J.11.3 Strong conditional dependence . . . 625
J.11.4 Insufficient diversity . . . 625
J.11.5 Finite-sample noise . . . 626
J.11.6 Orientation failure . . . 626
J.12 Zero Labels Do Not Mean Truth from Consensus . . . 626
J.13 Relation to Genericity . . . 626
J.14 Relation to Generative AI . . . 627
J.15 Conclusion . . . 627

K Sparse Structure Does Not Eliminate NP-Hardness: Worst-Case and Distributional Tractability **629**
K.1 The Fundamental Distinction: Evaluation versus Search . . . 629
K.2 Worst-Case Complexity . . . 630
K.3 Distributional Tractability . . . 631
K.4 A More Precise Practical Hypothesis . . . 631
K.5 Exact Solution versus Useful Approximation . . . 632
K.6 Smoothed Analysis . . . 632
K.7 What Smoothed Analysis Does Not Show . . . 633
K.8 Discrete Problems and Robust Hardness . . . 633
K.9 Where Practical Tractability Can Come From . . . 634
K.9.1 Low treewidth or related graph structure . . . 634
K.9.2 Small backdoor sets . . . 634
K.9.3 Repeated motifs . . . 635
K.9.4 Restricted effective dimension . . . 635
K.9.5 Approximation tolerance . . . 635
K.9.6 Stable heuristics . . . 635
K.9.7 Verification and fallback . . . 635
K.10 Learning from Deployment Traces . . . 636
K.11 SAT Solvers as an Example . . . 636
K.12 Sparse Compositionality Helps—but Is Not Enough . . . 637
K.13 A Connection to Genericity . . . 637
K.14 The Analogy with Optimization in Deep Learning . . . 638
K.15 A Hierarchy of Claims . . . 638
K.16 What Would Make the Hypothesis Testable? . . . 638

K.17 Implications for Practical AI . . . 639
K.18 Conclusion . . . 640

L A Johnson–Lindenstrauss Lemma for Sampled Group-Orbit Representations 641
L.1 Group-Orbit Representations . . . 641
L.2 Why the Orbit Distribution Is Invariant . . . 642
L.3 Why We Use Wasserstein Distance . . . 643
L.4 The Population Invariant Distance . . . 643
L.5 Sampling the Group Orbit . . . 644
L.6 Uniform Approximation of Haar Orbit Distributions . . . 644
L.7 Random Projection of the Sampled Orbit . . . 645
L.8 The Finite JL Control Set . . . 646
L.9 Preservation of Orbit Inner Products . . . 646
L.10 Wasserstein Stability after Projection . . . 647
L.11 Main Theorem . . . 647
L.12 Normalized Signals . . . 649
L.13 Approximate Invariance after Projection . . . 649
L.14 Nearest-Neighbor Stability . . . 649
L.15 Why the Logarithmic Dimension Is Significant . . . 650
L.16 What the Theorem Does Not Say . . . 650
L.16.1 Random projection does not discover invariance . . . 651
L.16.2 The theorem is finite-sample . . . 651
L.16.3 Exact invariance is lost after arbitrary projection . . . 651
L.16.4 Selectivity is separate . . . 651
L.17 Possible Extensions . . . 651
L.18 Relation to the Framework of This Book . . . 652
L.19 Conclusion . . . 652

M The Imitation Game 2.0 (Idea by Dan Mitropolsky) 654
M.1 Introduction . . . 654
M.2 The Containment Principle . . . 655
M.3 The Protocol . . . 655
M.4 The Metric: Simulation Distance . . . 656
M.5 Case Study: The “Bad Code” Test . . . 656
M.6 Conclusion . . . 657

N Degenerate Minima in Analytic Functions and Overparameterized Deep Networks 658
N.1 Introductory Foundations: Minima of Analytic Functions . . . 659
N.1.1 Analytic Functions of One Variable . . . 659
N.1.2 Analytic Functions of Multiple Real Variables . . . 660
N.1.3 Bounded Analytic Gradient Flow Converges to a Critical Point . . . 660
N.1.4 Qualitative Convergence and Quantitative Rates . . . 662
N.2 Real-Analyticity of Deep GELU Networks . . . 662
N.2.1 Analytic Nature of the GELU Activation . . . 662
N.2.2 Network Architecture and Quadratic Loss . . . 663
N.2.3 Smoothness Does Not Supply Uniform Conditioning . . . 663
N.2.4 A Quantitative Residual-Path Bound . . . 664
N.2.5 A Positive Convergence Result with Full Weight Decay . . . 664

N.3 Desingularization via Hironaka's Theorem . . . 665
N.3.1 A Local Normal-Crossing Form . . . 665
N.3.2 Weighted Volume and the Real Log Canonical Threshold . . . 666
N.3.3 What Resolution Does Not Say About Gradient Trajectories . . . 667
N.4 Overparameterized Deep Networks: Parameters and Fitting Constraints . . . 667
N.4.1 The Regular Interpolating Zero-Loss Manifold . . . 667
N.4.2 Hessian Spectrum and Neural Tangent Kernel Alignment . . . 668
N.4.3 The Scope of Neural-Network Landscape Results . . . 669
N.4.4 An Analytic Least-Squares Counterexample to Almost-Sure Recovery . . . 669
N.4.5 What a Quantitative Prediction-Jacobian Bound Does Prove . . . 670
N.5 Basins of Attraction for Degenerate Manifolds . . . 672
N.5.1 Foliation and Fiber Geometry: Local Conditions Matter . . . 672
N.5.2 Measure, Saddles, and Limits of Almost-Sure Claims . . . 673
N.5.3 Exact Minibatch Noise and the Interpolation Limit . . . 673
N.5.4 Regularized Stationarity and Common Minibatch Equilibria . . . 674
N.5.5 A Strictly Convex Example with Persistent Minibatch Noise . . . 676
N.5.6 Normalized ReLU Networks and the Low-Rank Obstruction . . . 676
N.5.7 Learning Rates and Finite Point Limits . . . 678
N.5.8 Anisotropic Diffusion Is Not a Universal Gibbs Law . . . 679
N.5.9 The Valid Local Energy–Volume Comparison . . . 680
N.5.10 Connection to Predictive Learning and the Status of the Results . . . 681

O Diffusion as a Sequence of Supervised Learning Problems 683
O.1 The Forward Process . . . 683
O.2 The Reverse Problems . . . 684
O.3 Predicting Noise . . . 684
O.4 The Score Interpretation . . . 685
O.5 Why Denoising Becomes Generation . . . 685
O.6 Conditional Diffusion . . . 686
O.7 Summary . . . 686

P From Brain Activity to Images as an Inverse Problem 688
P.1 The Forward Problem . . . 688
P.2 Why the Inverse Problem is Harder . . . 689
P.3 Compression and Ill-Posedness . . . 689
P.4 The Bayesian Formulation . . . 690

Q Notation and Mathematical Conventions 692

R Appendix: Glossary of Core Concepts 694

Glossary of Core Concepts 694

References 698

How to Read This Collection

This volume is not a conventional monograph. It is a collection of essays—some conceptual, some technical, some speculative—written at different times but revolving around a shared set of themes. It has evolved into something close to an intellectual atlas: an ongoing effort to map the principles that underlie modern artificial intelligence and the physics of natural intelligence. Several chapters are proposals for research projects that may be completed before too long.

Two ideas appear repeatedly throughout the collection, forming the spine of the argument:

- **Sparse compositionality**: the view that complex functions, behaviors, and representations arise from the hierarchical composition of a small number of reusable parts. This principle explains approximation, hierarchy, modularity, transfer, interpretability, and the unreasonable effectiveness of deep networks.
- **Genericity**: the observation that the real functions we want to learn in the world are not arbitrary high-complexity objects. Learnable functions must show stability with respect to the training set and invariance to simple transformations of the coordinate systems used for their definition. This implies that meaningful structure leaves persistent low-degree footprints, enabling gradient-based optimization.

The collection is organized into four parts:

Part I: Principles and Foundations explores the mathematical inevitability of the two core ideas above[cite: 1]. It argues that intelligence is not a grab-bag of heuristics but a solution to specific physical constraints.

Part II: Computation and Algorithms instantiates these principles in specific architectures, moving from Associative Turing Machines to the efficiency of Boolean functions[cite: 1].

Part III: Learning and Evolution examines the dynamics of learning in deep networks and evolutionary strategies[cite: 1].

Part IV: Random Topics takes the most risks[cite: 1]. It treats biology not as a metaphor, but as a constraint. Here, we argue that mechanisms like "World Models" and "Dreaming" are not just biological quirks but computational necessities.

Suggested Paths

Readers interested primarily in the **conceptual picture** may prefer to begin with:

- The *Prologue*,
- Chapter 1: *Intelligence as Associative Memory*,

- Chapter 27: *World Models Before Language*[cite: 1].

Readers seeking **computational foundations** may start with:

- Chapter 3: *Associative Memories are Turing-Complete*[cite: 1],
- The essays on compositional optimization in Part I,
- The appendices, which now contain rigorous proofs regarding the Johnson-Lindenstrauss lemma and group invariance.

Readers curious about **broader implications** or the future of AI may enjoy:

- Chapter 29: *What Is Missing in LLMs?*[cite: 1],
- Chapter 39: *Interlude: Most Real Numbers Do Not Exist*[cite: 1],
- The concluding essay on principles and open questions.

As with any collection of essays, some ideas appear more than once, viewed from different angles. This repetition is intentional: deep principles re-emerge across different domains. The unity of the volume lies not in a single narrative arc but in the convergence of these diverse perspectives on the same underlying structure of intelligence. Finally, several chapters include the name of a human co-author with whom I discussed extensively the content of the chapter. Even so, the responsibility for mistakes and omissions in the written text is on me and Gemini!

Last but not least: this is, for the time being, just a draft, often a draft of proposed research projects. It contains errors; there are missing references and missing acknowledgments. With time and feedback from you they will be corrected! This is ready to be a "working notebook" for the field. It provides a counter-narrative to the "scale is all you need" crowd.

Note on Co-Authorship: An Experiment

This book is an experiment in a new kind of scientific collaboration. It was co-authored by a human scientist and Large Language Models (specifically, Gemini and ChatGPT)[cite: 1].

In the history of science, we often speak of "tools for thought"—notation, calculus, the computer. But until recently, those tools were passive. They did not push back. This collection represents a shift toward active *cognitive amplification*. Throughout the writing process, the AI did not merely act as a scribe or a ghostwriter; it functioned as a sparring partner, a synthesizer, and occasionally, a mirror. Gemini also drafted figures which are far from perfect but serve as good placeholders.

The Role of the Model

The core ideas—Sparse Compositionality, Genericity, the biologically constrained definition of intelligence—are mine. They stem from decades of research at MIT and the Center for Brains, Minds and Machines (CBMM). However, the *articulation* of these ideas was often a dialectic process.

I would feed the model a raw intuition or a rough technical note (e.g., "Language is just a subtitle track for a world model"). The model would expand it, connect it to adjacent concepts in the history of computation (Turing, Post, von Neumann), and return a draft. We would then iterate.

This process revealed an interesting property of current AI: it is a machine of *genericity*. Left to its own devices, the model tends to "smooth" ideas. It gravitates toward the consensus, the balanced view, the polite academic tone. It often tried to soften my more radical claims about biology—for instance, the insistence that diffusion models are biologically implausible, or that scaling laws alone will hit a wall.

I found myself constantly having to "roughen" the text back up—to re-inject the asymmetry, the specific bet, and the falsifiable conjecture. In this sense, the authorship process mirrored the very principles discussed in the book: the AI provided the *generic* compositional capability, while the human provided the *sparse*, high-frequency logical jumps that define scientific novelty.

Why This Matters

This collaboration is not just a novelty; it is a data point in the study of intelligence itself. If, as this book argues, intelligence relies on the composition of reusable modules, then LLMs have mastered the modules of *syntax* and *rhetorical structure*. They have learned the "shape" of a scientific argument. But they famously lack the "World Model"—the grounding in physical reality that allows a primate to understand gravity without words.

Writing this book with an LLM confirmed that hypothesis. The model could write fluently about "World Models," but it could not *verify* consistency without my guidance. It could hallucinate a citation as easily as it could summarize a theorem. It possessed language, but not truth.

The Future of Scientific Writing

I believe this model—Human + AI—will become the standard for scientific inquiry, at least in the near future. It allows a researcher to move faster, to synthesize broader fields of literature, and to maintain a unity of voice across disparate topics. But it requires a new kind of vigilance. We must guard against the "smoothing" effect of these models, which risks homogenizing scientific thought into a polite, generic mean.

This collection, therefore, is an artifact of 2026: a testament to what machines can do, and a reminder of what, for now, only biological brains can provide.

Tomaso Poggio
Lake Sabrina, Needham, Massachusetts
January 2026, updated July 2026

CHAPTER 1

Intelligence as Associative Memory

A useful way to understand modern artificial intelligence is through one of its oldest computational ideas: associative memory. Rather than retrieving information by an explicit numerical address, an associative memory retrieves it by similarity to a cue. Modern attention mechanisms implement a powerful learned version of this operation: queries provide the cues, keys provide content-dependent addresses, and values provide the retrieved contents [109].

This perspective connects classical kernel and radial-basis-function models to transformers. Under appropriate normalization, dot-product softmax attention is exactly a normalized Gaussian-kernel retrieval rule. Across many heads and layers, however, the significance of attention goes beyond retrieval. Retrieved information is repeatedly transformed and used to generate new queries, creating a sequence of content-addressable read–update operations.

The central claim of this chapter is therefore not that a transformer is merely a large database. It is that associative retrieval, when embedded inside an iterative computational architecture, can become a primitive for general computation. A later chapter makes this statement precise through the Associative Turing Machine.

This is a statement about mechanism and representational power. It does not by itself explain how useful memories and retrieval rules are learned, when retrieval from a partial cue is reliable, or why a trained predictor generalizes. The later chapters distinguish feasible representation, accessible learning signals, optimization, and statistical selection. Identifying the particular hidden computation that generated the data is a stronger objective than learning an accurate predictor.

1.1 The Associative-Memory Hypothesis

Long before deep networks, radial basis function (RBF) networks modeled learning through similarity to stored prototypes. A set of centers is placed in feature space, and the response associated with a center grows as the input approaches it [16]. In this sense, an RBF network implements a simple form of association by similarity.

The basic computational motif is extremely general. A partial pattern or *cue* is presented; stored patterns are compared with that cue; and information associated with the closest or most compatible patterns is returned. The memory is therefore *content addressable*: the address is not specified explicitly but inferred from the content of the query.

Transformers generalize this idea dramatically. In each attention head,

- the *query* represents the current cue;

Figure 1.1: Intelligence as an Associative Memory

- the *keys* act as content-dependent addresses or templates;
- the *values* contain the information associated with those addresses;
- the *attention weights* determine how strongly each stored value is retrieved.

Thus an attention head can be interpreted as a learned associative retrieval mechanism. The important qualification is that the exact mathematical relationship with classical RBFs depends on normalization, as we will make precise below.

Unlike classical RBF systems with fixed centers and metrics, the representations serving as queries, keys, and values in a transformer change across layers and depend on the current context. The memory is therefore not a static table of fixed prototypes. It is a dynamically constructed, learned content-addressable system.

1.2 Attention as Content-Addressable Retrieval

For a token representation x_i, an attention head forms a query q_i, while the context provides keys k_j and associated values v_j. The output is

$$\mathrm{Attn}(q_i, K, V) = \sum_j \alpha_{ij} v_j,$$

with

$$\alpha_{ij} = \frac{\exp\left(q_i^\top k_j/\sqrt{d}\right)}{\sum_\ell \exp\left(q_i^\top k_\ell/\sqrt{d}\right)}.$$

The operation has the characteristic structure of associative memory: the query determines which stored items are relevant, and the associated contents are returned as a similarity-weighted combination.

This point of view emphasizes a distinction that will be important throughout the book. Attention is not primarily interesting because it stores facts in a literal database. Its computational importance is that it provides a differentiable mechanism for *addressing information by content*.

A single retrieval operation is already useful, but the real power of a transformer comes from repeatedly combining retrieval with state transformation.

Operational schema. A transformer block participates in an iterative read–update process:

1. *Content-addressable read:* form a query from the current representation and retrieve a weighted combination of values from the available context.
2. *State update:* transform the retrieved information together with the current representation through residual, feed-forward, and nonlinear operations.
3. *Reuse:* subsequent layers form new queries from the updated representation and perform further associative reads.

Complex computations can emerge from the repeated composition of these simple operations.

It is useful to call the second operation a *state update* rather than literally a memory write. In an ordinary transformer, previously cached representations are not rewritten in the way cells of conventional RAM are overwritten. In the abstract Associative Turing Machine developed later, however, the corresponding update operation plays the computational role of writing and modifying state.

A note on scope. This chapter concerns primarily what an associative-memory architecture can represent and compute *in principle*. Whether relevant stored content can be retrieved efficiently and robustly from a partial cue is a separate inverse problem, relative to the memory, the cue, and the retrieval rule.

Likewise, representational power does not imply that a specified training procedure will discover a useful predictor from data.

It is useful to keep three questions distinct:

$$\boxed{\text{expressibility} \;\neq\; \text{learnability} \;\neq\; \text{reliable retrieval.}}$$

The first concerns what the architecture can represent. The second concerns whether a feasible learning procedure finds a predictor that is accurate beyond the observed examples. The third concerns whether the retrieval mechanism returns the desired content from the admitted cues.

Here *retrieval* should not be confused with *structural recovery*: identifying the computation or intermediate variables that generated the data, up to specified equivalences. Predictive learning need not accomplish that stronger identification. The Interlude and Chapter H make this distinction explicit.

1.3 An Exact Kernel Equivalence for Normalized Attention

The connection between attention and classical associative kernels can be made exact under a simple normalization condition.

A Gaussian RBF centered at μ_i has the form

$$\phi_i(x) = \exp\!\left(-\frac{1}{2}(x-\mu_i)^\top \Sigma_i^{-1}(x-\mu_i)\right),$$

with isotropic RBFs corresponding to $\Sigma_i = \sigma^2 I$. More general anisotropic versions were studied, among other places, in the HyperBF framework of Poggio and Girosi [88].

Now suppose that a query q and all keys k_i have fixed norm,

$$\|q\| = \|k_i\| = 1.$$

Then

$$\|q - k_i\|^2 = \|q\|^2 + \|k_i\|^2 - 2q^\top k_i = 2 - 2q^\top k_i,$$

and therefore

$$q^\top k_i = 1 - \frac{1}{2}\|q - k_i\|^2.$$

The additive constant disappears under softmax normalization. Hence

$$\frac{\exp\left(q^\top k_i/\tau\right)}{\sum_j \exp(q^\top k_j/\tau)} = \frac{\exp(-\|q - k_i\|^2/(2\tau))}{\sum_j \exp(-\|q - k_j\|^2/(2\tau))}.$$

Thus the attention output becomes

$$\mathrm{Attn}(q, k_i, v_i) = \sum_i \frac{\exp(-\|q - k_i\|^2/(2\sigma^2))}{\sum_j \exp(-\|q - k_j\|^2/(2\sigma^2))} v_i, \tag{1.1}$$

where σ^2 absorbs the temperature or attention scaling.

This is precisely the normalized kernel-regression form

$$\phi(x) = \sum_{i=1}^{N} \frac{c_i K(\|x - t_i\|_W^2)}{\sum_j K(\|x - t_j\|_W^2)}. \tag{1.2}$$

Consequently,

normalized dot-product attention ≡ normalized Gaussian-kernel retrieval

under the fixed-norm assumption. This relationship has also been emphasized in recent work on homogeneous attention and HyperBF models [87].

The qualification is important. In ordinary transformers, key norms may vary and can themselves carry information. In that regime,

$$q^\top k_i = -\frac{1}{2}\|q - k_i\|^2 + \frac{1}{2}\|q\|^2 + \frac{1}{2}\|k_i\|^2,$$

and the last term depends on i. It therefore cannot in general be discarded inside the softmax.

Thus there are two distinct statements:

fixed-norm attention: exact normalized RBF equivalence, general attention: broader associative-kernel interpretation.

The first is a mathematical identity. The second is a structural interpretation.

1.4 From Associative Retrieval to Computation

An isolated associative memory retrieves information. A deep transformer does something more: it repeatedly retrieves, transforms, and uses the result to determine subsequent retrievals.

Multiple heads allow different content-addressable operations to be performed in parallel. Different heads may attend to different types of relations or different regions of the context. Their outputs are then combined and transformed before the next layer forms new queries.

This leads naturally to a computational interpretation.

- **Random-access-like addressing.** Attention can retrieve information from distant positions according to content rather than only according to sequential proximity.
- **Parallel retrieval.** Multiple heads can perform several content-dependent lookups simultaneously.
- **State transformation.** Residual and feed-forward operations transform retrieved information into new internal representations.
- **Iterated control.** The transformed representation determines the queries issued by later layers, so the outcome of one retrieval influences subsequent retrievals.

Through training, the network learns distributed control over what each head queries, which positions receive large attention weights, how retrieved values are combined, and how the resulting representation is transformed.

Algorithmic behaviors such as copying, matching delimiters, tracking references, manipulating variables, and executing multi-step procedures can in principle be implemented through repeated applications of this read–update–read motif.

The important computational claim is therefore not that every trained transformer literally implements a Turing machine. Rather, it is that associative retrieval together with suitable state updates provides a primitive rich enough, under appropriate architectural and representation assumptions, to support universal computation.

1.5 Building a Turing Machine from Associative Reads

A conventional Turing machine operates through an extremely small set of primitive operations. At each step it reads the current tape symbol, consults its transition rule, updates its state, writes a symbol, and moves the tape head.

A random-access machine replaces sequential tape access with direct addressing. An associative machine goes one step further: the relevant address can itself be retrieved from content.

This suggests an abstract computational cycle

$$\text{state} \longrightarrow \text{associative read} \longrightarrow \text{local update} \longrightarrow \text{new query} \longrightarrow \text{associative read}.$$

Attention supplies a natural realization of the read operation, while local neural transformations supply the state update.

The formal counterpart of this intuition is developed in Chap. 3. There the Associative Turing Machine (ATM) separates the computation into an associative memory component and a local update component. The construction shows that a suitable associative read–update architecture can simulate classical computation; see Thm. 1.

The universality claim uses the memory and computational resources allowed by that model. It concerns execution of a supplied program, not a guarantee that training discovers it. For prediction, a different feasible program can be equally satisfactory when it produces accurate outputs.

The significance of this result is conceptual. Associative memory is not merely a passive mechanism for recalling previously stored examples. When retrieval determines subsequent computation, it becomes a computational primitive.

Cost line. For a sequence of length n, dense self-attention requires $O(n^2)$ query–key interactions per layer in a full forward pass.

During autoregressive inference with a key–value cache, generating the next token requires $O(n)$ attention interactions per layer when the current context contains n tokens. Consequently, generating an entire length-n sequence remains quadratic in total for dense attention. Sparse or structured attention can reduce these costs.

For fixed architecture, key–value cache storage grows linearly with sequence length.

1.6 A Broader Principle: Computation by Repeated Local Transitions

The associative-memory interpretation is especially natural for attention-based models, but it points toward a broader principle.

Complex computation can often be represented as the repeated composition of relatively simple learned transitions.

Transformers implement this through alternating content-addressable retrieval and local transformation. Diffusion models [44] repeatedly apply learned denoising transitions. State-space models such as Mamba [33] propagate structured latent states through recurrent linear and gated updates.

These mechanisms should not be identified with one another. In particular, a diffusion model or a state-space model is not an associative memory in the same direct sense as an attention mechanism.

The common structure lies at a more abstract level:

$$\boxed{\text{complex map} = \text{composition of many simpler learned transitions.}}$$

For transformers, one of the elementary transitions is an associative read. For diffusion models, it is a denoising step. For state-space models, it is a structured state transition.

This commonality will become important later when we connect efficient computation with sparse compositional structure.

1.7 What the Associative-Memory Picture Does Not Explain

The associative-memory lens explains an important part of the *mechanism* of attention, but it does not by itself explain the success of deep learning.

Suppose that a memory contains an enormous number of possible patterns. The fact that the architecture can store or represent them does not guarantee that a useful pattern can be found efficiently from a partial cue.

Likewise, the fact that a neural network has enough representational capacity to implement an algorithm does not imply that gradient-based training can find an accurate predictor with feasible resources.

These distinctions separate three levels of explanation:

1. **Representation:** does the architecture contain a feasible realization of the desired input–output computation?
2. **Retrieval:** given a partial cue and a specified memory, can the relevant stored content be retrieved efficiently and reliably?
3. **Predictive learning:** can training find a feasible predictor from data, and does the selected predictor generalize to the population of interest?

Associative memory addresses primarily the mechanism behind the first two questions. A retrieval rule alone establishes neither reliable retrieval for every cue nor successful predictive learning.

Prediction is not structural identification. A successful predictor need not reproduce the intermediate variables or factorization that generated the observations. Recovering those variables is a stronger *structural* objective, requiring its own criterion of success and appropriate identifiability assumptions. This is not merely a concession to nonunique representations: predictive accuracy does not require a nodewise correspondence with a generating decomposition at all.

The later chapters therefore separate the existence of economical representations from the optimization and statistical questions of finding predictors that generalize. Genericity and staircase learning provide candidate mechanisms for exposing and exploiting useful signals; intermediate traces provide another form of assistance. None is imposed as a necessary trajectory of every successful learner.

1.8 Why Can Autoregressive Generation Be So Coherent?

Question.

If transformers can be interpreted as associative memories, it is easy to understand how they might retrieve related words, facts, phrases, or patterns. What is less obvious is why a generated response often remains coherent over many successive tokens.

Why does one retrieval not simply lead to an unrelated retrieval at the next step?

Answer.

A transformer is trained as a conditional next-token predictor, not as an explicit dynamical system constrained by a global consistency rule. Nevertheless, after training on coherent sequences, its autoregressive dynamics often preserve substantial syntactic, semantic, and discourse consistency.

The following discussion is a model of how this behavior may arise. It should not be read as a theorem following from the attention architecture alone.

1.8.1 Structured Geometry of Learned Representations

Each generated token depends on the preceding context through the representations produced by the transformer.

A useful working picture is that, after training, representations associated with natural text concentrate near structured subsets of the high-dimensional state space. These subsets reflect regularities of syntax, semantics, discourse, and the training distribution.

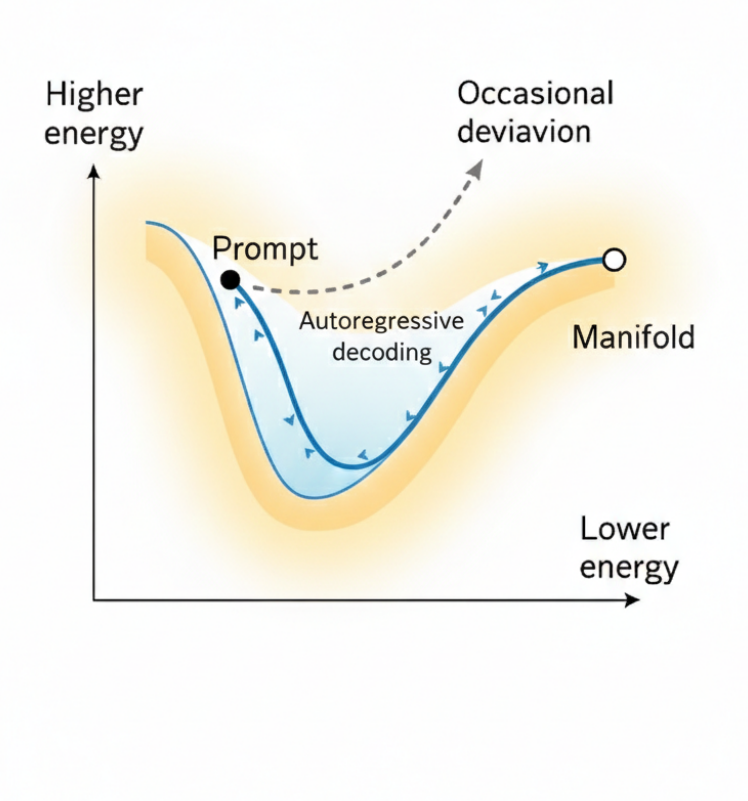


Figure 1.2: Schematic of manifold-constrained generation. A prompt initializes the model in a region of its effective representation space associated with plausible continuations. Autoregressive decoding can remain within a structured region of syntactically and semantically coherent states. The "landscape" is only a schematic representation: the dynamics need not derive from a true energy function. Moreover, remaining in a coherent region does not guarantee factual correctness.

It is common to refer informally to such structured subsets as *manifolds*. This language should be understood as an idealization, not as a claim that the hidden states of a transformer have been proved to lie on smooth low-dimensional manifolds in the mathematical sense.

A prompt places the model in a region of representation space compatible with some family of continuations. Attention then retrieves information that is itself conditioned on the same evolving context. This makes successive predictions strongly dependent rather than independent.

1.8.2 Self-Reinforcing Autoregression

At generation step t, the model samples or selects a new token according to

$$y_{t+1} \sim p_\theta(\cdot \mid y_{\leq t}),$$

after which the new token becomes part of the context used at the next step.

Abstractly, we may write the evolution of an internal state as

$$s_{t+1} = F_\theta(s_t, y_{t+1}).$$

The dynamics are therefore neither autonomous nor generally deterministic. They depend both on the current state and on the decoding choice.

If a set $\mathcal{M}$ represents states associated with coherent continuations, a useful stability hypothesis is that typical transitions starting near $\mathcal{M}$ tend to remain near it:

$$F_\theta(\mathcal{M}, \Xi_{\text{typical}}) \approx \mathcal{M},$$

where Ξ_{typical} denotes token choices receiving substantial probability under ordinary decoding.

This is an empirical property that a trained model may approximately possess. It is not guaranteed by the architecture.

The mechanism is self-reinforcing. A coherent token becomes part of the next context and therefore biases subsequent attention and prediction toward continuations compatible with what has already been generated.

1.8.3 Coherence Is Not Truth

The distinction between coherence and factual correctness is essential.

A model can produce a passage that is

- grammatically correct,
- semantically plausible,
- internally self-consistent,
- stylistically coherent,

and nevertheless factually false.

Thus a hallucination need not correspond to a trajectory that leaves the region of coherent language. On the contrary, many hallucinations are highly coherent.

The manifold picture, to the extent that it is useful, therefore explains primarily why generation may remain on a structured set of *linguistically and semantically plausible continuations*. It does not by itself explain grounding in external reality.

One may schematically distinguish

$$\mathcal{M}_{\text{text}}$$

from the smaller and more demanding subset

$$\mathcal{M}_{\text{true}} \subseteq \mathcal{M}_{\text{text}},$$

where the first represents coherent text and the second represents coherent text that is also factually correct relative to the relevant world state.

Language-model training strongly constrains the first. Guaranteeing the second requires additional assumptions, information, or external verification.

1.8.4 Emergent Consistency as Constrained Associative Recall

From the associative-memory viewpoint, prompting fixes part of a pattern and autoregressive generation successively completes it.

But the completion is not produced by a single lookup. Each newly generated token changes the cue used for subsequent retrievals. The system therefore performs a sequence of mutually conditioning associative operations.

This leads to the qualitative picture

$$\text{prompt} \to \text{retrieval} \to \text{prediction} \to \text{expanded prompt} \to \text{new retrieval} \to \cdots$$

in which previously generated material helps constrain future generation.

Consistency, in this view, can emerge from the interaction between a structured learned representation and self-conditioning autoregressive dynamics.

Associative recall explains how relevant information can be retrieved; autoregressive self-conditioning helps explain why successive retrievals can remain mutually coherent.

1.9 Historical and Biological Connections

The associative-memory viewpoint long predates transformers.

Kanerva's *Sparse Distributed Memory* (SDM) [50] provides a particularly clear model of content-addressable memory in a high-dimensional binary space. Memory locations are associated with distributed addresses, and a partial cue activates locations lying sufficiently close to it in Hamming distance. Their stored contents are then aggregated to produce a recalled pattern.

The essential idea is familiar from attention:

$$\text{cue} \longrightarrow \text{similarity-based addressing} \longrightarrow \text{retrieved content}.$$

There are, of course, major architectural differences. Kanerva's model is a fixed high-dimensional associative memory, whereas transformers build context-dependent keys, queries, and values and repeatedly compose retrieval with nonlinear transformations.

Biological memory systems provide another suggestive connection. Classical models of hippocampal circuitry emphasize sparse coding, pattern separation, associative recall, and pattern completion [67, 69, 45]. In particular, dentate gyrus and CA3 have often been interpreted as supporting sparse representations and auto-associative retrieval.

There is therefore an intriguing computational analogy between hippocampal circuitry and artificial associative-memory systems. The analogy concerns functional motifs—sparse coding,

content-addressable retrieval, context-dependent reconstruction, and multi-stage transformation—not architectural identity.

A much stronger claim, such as identifying the hippocampus literally with a transformer or assigning directly comparable information capacities to their synapses, would require assumptions not justified by the present discussion.

The biological connection becomes more interesting when associative memory is embedded in a hierarchy. Hippocampal interactions with cortex suggest a system in which rapidly stored episodic representations can participate in slower reconstruction and consolidation. This motivates a broader view of associative memory as a component of a multi-stage computational architecture rather than simply a flat lookup table.

A later chapter develops this hippocampal-scaffold hypothesis in greater detail.

1.10 Associative Retrieval, Computational Arity, and Memory Capacity

Several memory mechanisms discussed throughout this book share a common computational organization:

$$\boxed{\text{query} \rightarrow \text{similarity} \rightarrow \text{retrieval.}}$$

This motif appears in attention, radial-basis memories, modern Hopfield networks, and sparse distributed memory.

The resemblance is useful, but it should not be pushed too far.

Some of these mechanisms are exactly equivalent under particular assumptions.

Others share only the broader principle of similarity-gated access.

A second distinction is equally important: a retrieval rule may appear to have very large fan-in when written as a single mathematical expression, while still admitting a bounded-fan-in computational implementation.

Finally, one must distinguish the number of possible memory *addresses* from the number of items that can be simultaneously *superposed* and recovered reliably.

These distinctions clarify what associative memory contributes to the computational framework developed in this book.

1.10.1 A generic associative read

Consider a collection of keys

$$k_1, \ldots, k_N$$

and associated values

$$v_1, \ldots, v_N.$$

A broad class of associative memories computes

$$r(q) = \sum_{i=1}^{N} \alpha_i(q) v_i, \tag{1.3}$$

where the weights depend on the similarity between the query q and the stored keys.

A common normalized form is

$$\alpha_i(q) = \frac{\exp s(q, k_i)}{\sum_{j=1}^{N} \exp s(q, k_j)}. \tag{1.4}$$

The score function

$$s(q, k)$$

defines the geometry of retrieval.
This decomposition separates two components:

addressing geometry and stored content.

1.10.2 Dot-product attention

For standard attention,

$$s(q, k_i) = \frac{q^\top k_i}{\tau}, \tag{1.5}$$

where τ is a temperature or scale parameter.
Hence

$$r(q) = \sum_{i=1}^{N} \frac{\exp\left(q^\top k_i/\tau\right)}{\sum_j \exp(q^\top k_j/\tau)} v_i. \tag{1.6}$$

This is a content-addressable read: the query does not specify an explicit memory location but instead selects values through similarity to their keys.
The mechanism should therefore be interpreted as

content-addressable routing or retrieval,

not as a complete model of computation by itself.

1.10.3 Gaussian RBF retrieval

A Gaussian radial-basis memory instead uses the score

$$s_{\mathrm{RBF}}(q, \mu_i) = -\frac{1}{2}(q - \mu_i)^\top \Sigma^{-1}(q - \mu_i). \tag{1.7}$$

Expanding the quadratic gives

$$s_{\mathrm{RBF}}(q, \mu_i) = q^\top \Sigma^{-1}\mu_i - \frac{1}{2} q^\top \Sigma^{-1} q - \frac{1}{2}\mu_i^\top \Sigma^{-1}\mu_i. \tag{1.8}$$

The query-only term

$$-\frac{1}{2} q^\top \Sigma^{-1} q$$

is common to all keys and cancels under softmax normalization.
The key-dependent term

$$-\frac{1}{2}\mu_i^\top \Sigma^{-1}\mu_i$$

does not cancel in general.

Therefore Gaussian RBF retrieval is exactly equivalent to dot-product softmax attention only under an additional condition, for example

$$\|\Sigma^{-1/2}\mu_i\| = \text{constant}$$

for all stored keys, or if the corresponding key-dependent bias is included explicitly in the score.

Thus

RBF retrieval and attention can be equivalent, but not unconditionally.

1.10.4 Modern Hopfield retrieval

A single update of a modern Hopfield memory can take the form

$$\alpha_i(x) \propto \exp\Big(\beta x^\top x_i\Big). \tag{1.9}$$

At this level, the update is mathematically the same normalized dot-product retrieval used in attention, with

$$\beta = \frac{1}{\tau}.$$

This exact correspondence concerns a *single retrieval step*.

A recurrent Hopfield system can apply the update repeatedly:

$$x^{(0)} \to x^{(1)} \to x^{(2)} \to \cdots .$$

That introduces an attractor dynamics absent from a single ordinary attention read.

Therefore

single-step modern Hopfield retrieval = dot-product attention read,

under the usual parameter identification, but

multi-step Hopfield dynamics ≠ one attention operation.

1.10.5 Kanerva's Sparse Distributed Memory

Kanerva's Sparse Distributed Memory uses a different gating mechanism.

For binary query q, locations satisfying

$$\mathrm{Ham}(q, k_i) \le r \tag{1.10}$$

are activated.

A corresponding kernel is

$$K(q, k_i) = \mathbf{1}\left\{\mathrm{Ham}(q, k_i) \le r\right\}. \tag{1.11}$$

A normalized read can be written

$$r(q) = \frac{\sum_i K(q, k_i) v_i}{\sum_j K(q, k_j)}. \tag{1.12}$$

Thus SDM belongs naturally to the larger family of

$$\boxed{\text{similarity-gated associative reads.}}$$

But it should not be identified literally with dot-product attention or with a zero-temperature single-nearest-neighbor softmax.

Its gating geometry is a Hamming ball, and many locations may contribute simultaneously.

1.10.6 Exact equivalence versus shared computational motif

It is therefore useful to distinguish two levels of comparison.

At the stronger level,

$$\boxed{\text{exact equivalence}}$$

means that two mechanisms compute the same mathematical map after an explicit parameter transformation.

At the weaker level,

$$\boxed{\text{shared computational motif}}$$

means only that both perform

$$\text{query} \to \text{similarity} \to \text{retrieval}.$$

Attention and one-step modern Hopfield retrieval admit an exact correspondence in the standard formulation.

Gaussian RBF retrieval requires additional normalization or bias conditions.

Kanerva SDM shares the motif but not the same retrieval law.

Keeping these distinctions explicit avoids turning a useful analogy into a false identity.

1.10.7 Associative retrieval does not automatically have bounded arity

There is a second subtlety relevant to compositional sparsity.

The expression

$$r(q) = \sum_{i=1}^{N} \alpha_i(q) v_i \tag{1.13}$$

may depend on all N stored values.

If N grows with problem size, then the *functional arity* of the written expression is not bounded.

Thus

$$\boxed{\text{attention} \not\Longrightarrow \text{bounded functional arity.}}$$

This does not contradict the bounded-fan-in view of efficient computation, because the mathematical arity of a formula and the fan-in of its implementation are different notions.

1.10.8 Sparse retrieval

One possibility is that retrieval is itself sparse.

If only

$$k = O(1)$$

keys receive significant weight, then the effective read depends on only a bounded number of memory items.

This may arise through

explicit top-k attention, sparse routing, hard gating, large score margins.

In that case associative retrieval itself can instantiate a low-arity constituent.

1.10.9 Bounded-fan-in implementation of dense retrieval

Even dense retrieval admits a bounded-fan-in computational realization.

Consider the denominator

$$Z(q) = \sum_{j=1}^{N} \exp s(q, k_j).$$

The sum can be evaluated through a binary reduction tree:

$$N \to \frac{N}{2} \to \frac{N}{4} \to \cdots \to 1.$$

Each node has fan-in two, and the depth is

$$O(\log N).$$

The same is true for the weighted numerator

$$\sum_i \exp s(q, k_i) v_i.$$

Thus

$$\boxed{\text{high-arity mathematical expression} \not\Longrightarrow \text{high-fan-in computation.}}$$

What matters for compositional sparsity is the structure of an efficient implementation.

A dense attention read may therefore remain compatible with bounded-fan-in computation even though its written functional dependence involves the entire context.

This is an implementation-level statement. An efficiency claim must also control operation count, memory, and numerical precision as the problem size grows. The bounded-fan-in implementation does not by itself supply a particular semantic decomposition or uniformly regular low-arity real constituents. Chapter 4 keeps the computational statement separate from those stronger approximation-theoretic assumptions.

1.10.10 Addressability and storage are different capacities

Associative memories are often described as having enormous capacity.

But at least two fundamentally different capacities must be distinguished:

1. the number of distinct addresses that can be represented;
2. the number of associations that can be simultaneously stored in a shared representation and recovered with fixed reliability.

These quantities can scale very differently.

1.10.11 Exponential addressability

Let keys lie on the unit sphere

$$\mathbb{S}^{d-1}.$$

High-dimensional geometry permits exponentially many approximately orthogonal vectors.

For fixed tolerance

$$\eta \in (0, 1),$$

one can choose on the order of

$$N = \exp(cd) \tag{1.14}$$

distinct unit vectors, for a constant c depending on the allowed correlation, such that pairwise inner products remain small.

Thus a d-dimensional representation can provide an exponentially large *address space*.

This does not mean that exponentially many independent memories can be superposed into one d-dimensional vector and recovered reliably.

1.10.12 Modular address codes

A constructive version of the same phenomenon arises from modular codes.

Suppose an address x is represented by residues

$$x \bmod m_1, \ldots, x \bmod m_J,$$

where the moduli are pairwise coprime.

The Chinese remainder theorem implies that the joint code uniquely represents

$$\prod_{j=1}^{J} m_j \tag{1.15}$$

different addresses before repetition.

Thus a collection of relatively small modules can produce a very large joint address space.

Related modular-coding arguments are useful for thinking about biological grid-like codes as well as artificial distributed addressing.

Again, however, this is a statement about the number of distinct *addresses*, not the number of objects that can be mixed together in one noisy superposition.

1.10.13 Superpositional storage

Now consider storing many bound associations in a single distributed vector:

$$W = \sum_{j=1}^{M} k_j \odot w_j, \tag{1.16}$$

where $\odot$ denotes a unitary or approximately invertible binding operation.
To retrieve association ℓ, one approximately unbinds with the inverse of k_ℓ.
The desired item appears together with crosstalk from all other stored items.
Under standard random-key assumptions, the crosstalk variance typically scales as

$$O\left(\frac{M}{d}\right), \tag{1.17}$$

giving signal-to-noise ratio of order

$$\mathrm{SNR} \asymp \sqrt{\frac{d}{M}}. \tag{1.18}$$

Maintaining constant retrieval quality therefore permits roughly

$$M = O(d)$$

simultaneously superposed associations under this simple model.

1.10.14 The apparent paradox disappears

There is no contradiction between

$$N = \exp(\Theta(d))$$

possible addresses and

$$M = O(d)$$

reliably superposed memory items.
They measure different things.
The first asks:

How many distinct keys can exist?

The second asks:

How many items can be mixed into one finite-dimensional state while retaining adequate signal-to-noise ratio?

Therefore

exponential addressability $\neq$ exponential superpositional storage.

1.10.15 Why the distinction matters for associative computation

The difference between addressing and storage clarifies the computational role of associative memory.

A large address space allows a system to select among a huge number of potential states or memory locations.

It does not require all of those states to be simultaneously stored in one vector.

One can instead combine

large distributed address space
+ external or distributed memory
+ content-addressable retrieval.

This organization is closer to ordinary computer memory as well.

A processor may address an enormous memory space without encoding the entire contents of that memory in one register.

1.10.16 Relation to compositional computation

Associative retrieval and compositional computation therefore solve different parts of the problem.

Associative mechanisms answer:

Which information or operation should be accessed?

Compositional structure answers:

How should selected operations be combined into a computation?

In this sense,

associative memory supplies routing and addressability;

bounded-arity composition supplies computation.

A complete architecture may require both.

The query selects the relevant memories or modules.

The computation graph composes their outputs.

1.10.17 A useful hierarchy of claims

The discussion can be summarized as follows:

$$\boxed{\begin{aligned} \text{Attention} &= \text{normalized dot-product associative retrieval,} \\ \text{one-step modern Hopfield retrieval} &\simeq \text{the same mechanism,} \\ \text{Gaussian RBF memory} &= \text{related retrieval under metric-dependent scores,} \\ \text{Kanerva SDM} &= \text{similarity-gated retrieval using Hamming neighborhoods.} \end{aligned}}$$

These mechanisms share a computational motif without being universally interchangeable. Likewise,

$$\boxed{\text{dense functional dependence} \not\Longrightarrow \text{unbounded implementation fan-in,}}$$

and

$$\boxed{\text{large address space} \not\Longrightarrow \text{equally large superpositional storage capacity.}}$$

These distinctions are important whenever associative memory is used as a primitive in a larger theory of intelligent computation.

1.10.18 Conclusion

Associative memory should be understood neither as a single algorithm nor as a single capacity number.

Its central computational idea is content-dependent access:

$$\boxed{\text{the current state determines which stored information becomes available next.}}$$

Attention, RBF memories, Hopfield retrieval, and sparse distributed memory implement this idea using different similarity geometries and different dynamics.

Their common role is therefore best summarized as

$$\boxed{\text{associative memory is an addressing and routing primitive.}}$$

Compositional sparsity concerns the complementary question of how the retrieved pieces are combined.

Together they give a natural computational architecture:

$$\boxed{\text{address} \to \text{retrieve} \to \text{compose} \to \text{compute.}}$$

1.11 Associative Memory as a Computational Primitive

We can now summarize the main argument.

Classical associative memory begins with a simple operation:

retrieve what resembles the cue.

Attention turns this idea into a learned, differentiable operation. Under fixed-norm conditions, its softmax weights are exactly those of a normalized Gaussian kernel. In the general case, the same computation retains the broader structure of content-addressable retrieval while allowing learned and context-dependent representations.

Deep architectures then add a crucial ingredient: composition.

A retrieved value is transformed, the transformed state generates new queries, and those queries trigger further retrievals. Thus

$$\boxed{\text{associative retrieval} + \text{state update} + \text{iteration}}$$

becomes a general computational motif.

The progression is

$$\boxed{\begin{array}{c}\text{RBF/kernel memory} \longrightarrow \text{attention} \longrightarrow \text{read–update composition} \\ \Downarrow \\ \text{Associative Turing Machine} \longrightarrow \text{general computation.}\end{array}}$$

This perspective suggests that associative memory is not merely one specialized faculty of an intelligent system. It may be one of the basic mechanisms from which more elaborate computation can be constructed.

But universality is only the beginning of an explanation.

A universal machine can execute many different computations when supplied with the appropriate program. That fact does not explain how training finds a useful computation from examples, or why the resulting predictor works beyond those examples.

The subsequent chapters therefore separate three questions: whether a feasible accurate predictor is available, whether the specified training procedure finds a suitable fit, and whether the selected predictor generalizes. Chapter 4 develops the representation principle of compositional sparsity. The chapters that follow examine accessible learning signals, intermediate supervision, optimization geometry, and statistical selection.

Chapter H brings these questions together. Under its stated finite-precision resource assumptions, an accurate predictor with feasible evaluation already supplies a bounded-fan-in compositional realization. Its learned variables need not match those of the computation that generated the observations. Explaining how training achieves that accuracy remains a substantive problem; structural identification is a stronger, separate one.

CHAPTER 2

A Historical Reflection on Associative Memories

This chapter is a historical and conceptual essay following the thesis introduced in Chapter 1. Its purpose is not to claim that modern transformers descended directly from the associative-memory models of the 1960s and 1970s. Rather, it traces the repeated appearance of a common computational idea: information can be retrieved or transformed according to the content of a cue.

From early heteroassociative memories, through nonlinear operator memories, radial basis function networks, hippocampal models, and modern attention, the machinery changes substantially. Yet a recurring primitive remains: use similarity or correlation with the present state to recover information relevant to what should happen next.

The historical thesis of this chapter is therefore one of conceptual continuity rather than literal architectural descent.

2.1 A Recurring Computational Idea

Associative memory—the ability to bind one pattern to another and later recover the associated information from a complete or partial cue—is one of the oldest ideas in neural computation.

What is striking from the perspective of modern artificial intelligence is not merely its historical importance, but its recurrence.

A simple heteroassociative memory stores pairs

$$(x^\mu, y^\mu)$$

and implements a map

$$x^\mu \longmapsto y^\mu.$$

More sophisticated systems replace exact matching by similarity, correlation, interpolation, or learned metrics. Modern attention extends the same general idea still further: a query provides a cue, keys provide content-dependent addresses, and values provide the information retrieved from those addresses.

The central claim of this essay is therefore not that all these systems are identical. They plainly are not. Rather,

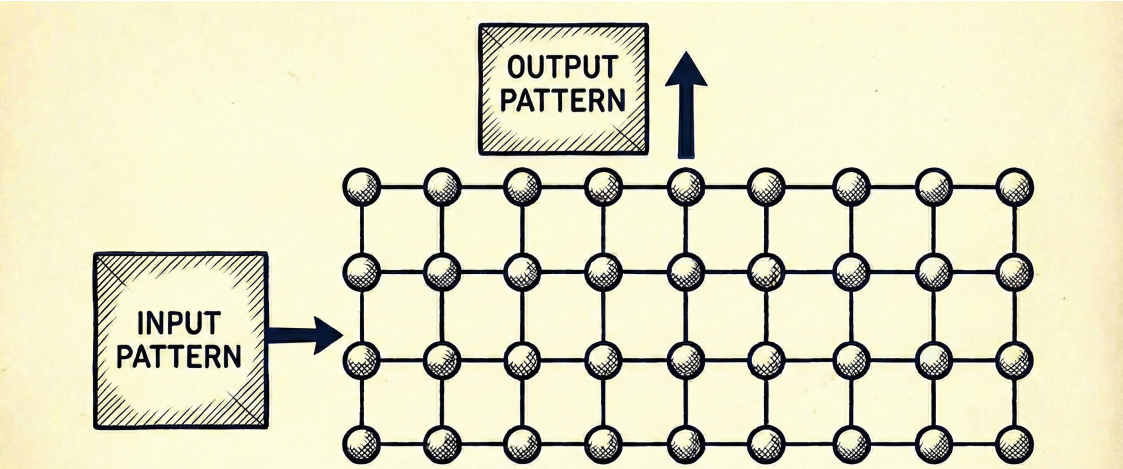


Figure 1. Schematic representation of a non-holographic associative memory, after Willshaw & Longuet-Higgins, *Nature* (1969).

Figure 2.1: A conceptual lineage of associative memory, from early correlation-based models to modern content-addressable computation.

content-addressable association is a recurring computational primitive.

Several important developments in neural computation can be viewed as successive enrichments of this primitive:

$$\text{association} \rightarrow \text{learned association} \rightarrow \text{nonlinear similarity}$$
$$\Downarrow$$
$$\text{hierarchical and recurrent association}$$
$$\Downarrow$$
$$\text{deep content-addressable computation.}$$

This perspective provides a useful historical bridge between early theories of memory and modern deep learning.

2.2 Early Associative Memories

2.2.1 Heteroassociative Correlation Memories

The modern mathematical study of associative memory emerged prominently in the late 1960s and early 1970s. Early models focused on storing associations between patterns while analyzing capacity, robustness, one-shot learning, and biological plausibility.

Willshaw and Longuet-Higgins introduced sparse binary heteroassociative memories with one-shot learning and feedforward recall [116]. Kohonen subsequently formulated associative memory in linear-algebraic form using correlation matrices [54].

Given stored pairs (x^μ, y^μ), the memory matrix can be written as

$$W = \sum_\mu y^\mu (x^\mu)^\top,$$

so that a new cue x produces

$$\hat{y} = Wx = \sum_\mu y^\mu (x^\mu)^\top x.$$

The important structure is immediately visible. Retrieval is weighted by the similarity

$$(x^\mu)^\top x$$

between the cue and the stored input pattern.

Viewed retrospectively, this already contains a primitive that will reappear repeatedly in later kernel methods and in attention:

similarity to a stored key $\longrightarrow$ retrieval of an associated value.

The historical models were not transformers, of course, nor were they formulated in the language of queries, keys, and values. But the computational organization is recognizably heteroassociative.

2.2.2 Nonlinear Operator Memories

A particularly important step was the formulation of associative recall as an operator-estimation problem in Poggio's work of the 1970s [83].

Given paired observations collected into matrices X and Y, one seeks an operator Θ satisfying approximately

$$Y \approx \Theta(X).$$

Instead of restricting Θ to be linear, one can expand it as

$$Y = L_0 + L_1(X) + L_2(X, X) + \cdots + L_k(X, \ldots, X),$$

where the L_ℓ are symmetric multilinear operators.

The first-order term recovers the linear associative-memory solution,

$$L_1 = YX^\dagger,$$

while the higher-order terms provide systematic nonlinear corrections.

If the residual after approximation through order $k-1$ is

$$E_{k-1} = Y - \sum_{\ell=0}^{k-1} L_\ell(X, \ldots, X),$$

then the next correction can be written schematically as

$$L_k = E_{k-1} C_k^\dagger,$$

where C_k is the lifted tensor of degree-k input products,

$$(C_k)_{\alpha_1 \cdots \alpha_k, j} = \prod_{t=1}^{k} X_{\alpha_t j}.$$

The significance of this construction is that associative memory becomes a problem of nonlinear function approximation rather than merely storage in a linear correlation matrix.

Early introduction of polynomial kernels. The multilinear operator expansion naturally introduces polynomial feature spaces.

A degree-k tensor feature can be written as

$$\Phi_k(x) = x^{\otimes k}.$$

Its inner product satisfies

$$\left\langle x^{\otimes k}, (x')^{\otimes k} \right\rangle = (x^\top x')^k.$$

Including lower-order terms leads to the familiar inhomogeneous polynomial form

$$K_k(x, x') = (1 + x^\top x')^k.$$

Thus the operator-estimation framework contains an explicit polynomial feature construction and the corresponding polynomial kernels.

This is not merely a retrospective resemblance. Vapnik later explicitly credited Poggio's earlier work with the introduction of polynomial kernels. Historically, the associative operator formulation therefore belongs to the early development of the kernel viewpoint, well before polynomial kernels became standard tools in statistical learning theory and support vector machines.

Conceptually, this creates an important bridge:

associative memory $\rightarrow$ nonlinear feature expansion $\rightarrow$ kernel representation.

2.2.3 RBFs and HyperBFs

A second important development replaced global polynomial expansions by localized similarity functions.

Radial basis function networks take the form

$$f(x) = \sum_\mu \alpha_\mu \phi(\|x - x^\mu\|),$$

where each stored center x^μ contributes according to its distance from the cue.

Poggio and Girosi developed this perspective into a general framework for regularization networks and HyperBFs [88], allowing learned or anisotropic metrics.

Instead of merely asking whether two patterns are correlated, these models ask how close they are in an appropriate representation space.

The associative-memory interpretation becomes

$$\text{cue} \longrightarrow \text{similarity to stored centers} \longrightarrow \text{weighted reconstruction}.$$

This is especially close conceptually to modern attention because retrieval is no longer based on exact addresses. It is controlled continuously by a learned notion of similarity.

2.2.4 A Different Branch: Autoassociative Attractor Memories

Hopfield networks developed a different branch of associative-memory theory [45].

Instead of storing a mapping from one pattern to another, a classical Hopfield network is autoassociative: a partial or corrupted version of a stored pattern evolves dynamically toward a corresponding attractor.

Its defining ingredients include recurrent interactions, symmetric weights in the classical formulation, and an energy-like dynamical system.

This was historically enormously influential because it connected memory with collective neural dynamics and pattern completion.

Conceptually, however, the key–value organization of modern attention is closer to heteroassociative and metric-based retrieval than to classical symmetric Hopfield dynamics.

The two traditions should therefore be seen as related branches of associative-memory theory rather than one as a special case of the other.

2.3 From Association to Biological Computation

The biological relevance of associative computation predates any formal theory of neural networks.

At the most elementary level, nervous systems must map sensory conditions into appropriate actions.

A simple input–output relation has the form

$$x_{\text{sensory}} \longmapsto y_{\text{motor}}.$$

Formally, this has the same shape as a heteroassociative map.

2.3.1 Reflexes as Elementary Input–Output Associations

A monosynaptic reflex arc provides perhaps the simplest neural architecture connecting a sensory signal to a motor response.

It is important not to push the analogy too far.

A genetically specified, immutable reflex is not necessarily a *memory*. Nothing need have been learned or stored during the lifetime of the organism.

Its relevance is instead computational: it exhibits the primitive organization

$$\text{pattern} \longrightarrow \text{associated response}.$$

Once synaptic plasticity is introduced, however, such mappings can become learned associations in the stronger sense.

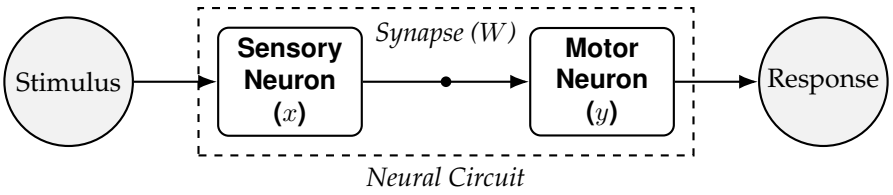


Figure 2.2: A monosynaptic reflex arc. At the level of input–output organization, the circuit maps a sensory signal x through a synaptic connection W into a motor response y. The analogy with heteroassociative memory concerns the form of the mapping; a fixed reflex need not itself constitute a learned memory.

2.3.2 Plasticity and Adaptive Association

Plastic sensory–motor circuits add a crucial ingredient: the mapping between perception and action can now change with experience.

Hebbian and heterosynaptic plasticity provide mechanisms through which local associations can be modified according to activity and outcome.

At this level, the nervous system no longer merely executes a fixed mapping. It can construct and modify mappings.

This suggests a broad progression from fixed association to adaptive association:

$$\text{fixed response} \rightarrow \text{plastic association} \rightarrow \text{adaptive computation}.$$

Feedback pathways further enrich this process because intermediate representations can influence upstream or downstream processing rather than simply propagating once through a feedforward chain.

2.3.3 A Possible Computational Progression

The following sequence should be read as a conceptual progression in computational capability, not as a claim about the literal phylogenetic order in which these mechanisms evolved.

1. **Fixed sensorimotor mappings.**

   Simple circuits associate sensory conditions with actions.

2. **Plastic associative mappings.**

   Synaptic plasticity allows these associations to adapt with experience.

3. **Multi-stage processing and feedback.**

   Ascending and descending pathways permit intermediate representations to be modified and repeatedly refined.

4. **Associative scaffolding and episodic indexing.**

   Systems such as the hippocampus provide rapid storage, partial-cue retrieval, and interaction with slower cortical representations.

5. **Self-organized learning circuits.**

   Models such as Self-Assembling Learning (SAL) illustrate, at a theoretical level, how local plastic mechanisms and feedback pathways can organize into circuits implementing effective optimization procedures [84].

The point of this sequence is not evolutionary chronology. It is that increasing computational power can arise by embedding simple association inside progressively richer systems of plasticity, feedback, hierarchy, and repeated transformation.

2.3.4 Hippocampal Association and Scaffolding

The hippocampus provides an especially interesting biological example because associative recall operates as part of a larger system.

Classical theories emphasize pattern separation, episodic indexing, autoassociative retrieval, and pattern completion [67, 69, 45].

The relevance to the present historical story is not that the hippocampus is literally a transformer. Rather, hippocampal circuitry illustrates how associative retrieval can be embedded in a multi-stage architecture and interact with other memory systems.

A partial cue may trigger reconstruction of a richer episodic pattern, which in turn influences cortical processing and later consolidation.

Thus biological memory suggests a transition from

$$\text{flat associative retrieval}$$

to

$$\text{associative retrieval embedded in a hierarchy.}$$

A later chapter develops the hippocampal-scaffold hypothesis in much greater detail.

2.4 Modern Attention as Associative Retrieval

The attention mechanism at the center of modern transformers is

$$\mathrm{Attn}(q, K, V) = \mathrm{softmax}\left(\frac{qK^\top}{\sqrt{d}}\right)V.$$

Its associative interpretation is immediate:

$$\text{query} \longrightarrow \text{similarity with keys} \longrightarrow \text{weighted retrieval of values.}$$

As shown in Chapter 1, when the query and keys have fixed norm, dot-product attention can be rewritten exactly as normalized Gaussian-kernel retrieval.

Indeed, if

$$\|q\| = \|k_i\| = 1,$$

then

$$q^\top k_i = 1 - \frac{1}{2}\|q - k_i\|^2,$$

and therefore the normalized attention weights become

$$\frac{\exp\left(q^\top k_i/\sigma^2\right)}{\sum_j \exp(q^\top k_j/\sigma^2)} = \frac{\exp(-\|q - k_i\|^2/2\sigma^2)}{\sum_j \exp(-\|q - k_j\|^2/2\sigma^2)}.$$

Thus, under normalization,

$$\boxed{\text{dot-product attention} \equiv \text{normalized Gaussian associative retrieval.}}$$

Without fixed norms, the equivalence is no longer exact because the key norms contribute nonconstant terms to the exponent. The broader content-addressable interpretation nevertheless remains useful.

Historically, attention can therefore be viewed as a modern learned form of similarity-based heteroassociative retrieval.

The crucial advance is not merely the retrieval rule itself. Queries, keys, and values are learned representations that change across layers, and associative retrieval is repeatedly interleaved with nonlinear state transformations.

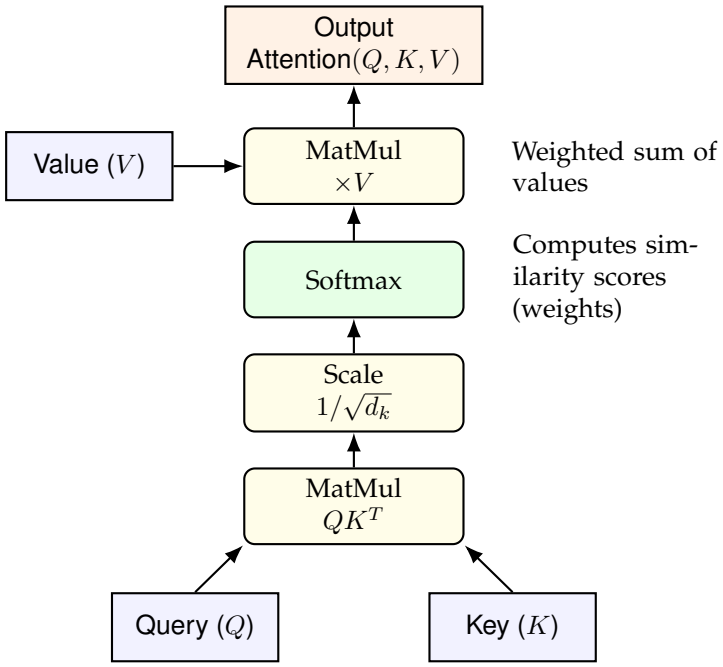


Figure 2.3: The transformer attention mechanism. Queries and keys determine content-dependent similarity weights, which are then used to retrieve a weighted combination of the stored values.

2.5 A Preview: Why Does Associative Recall Work?

The repeated historical success of associative memories raises a deeper question.

Why should similarity to a partial cue contain enough information to recover the relevant pattern?

This question cannot be answered by representational capacity alone. An associative memory may have enough capacity to store a large set of patterns while still making reliable retrieval from partial information difficult.

One possible explanation involves *genericity*, developed in Chapter 6. In a specified observation model, perturbations can break exact cancellations and expose statistical footprints of relevant dependencies. Whether this happens depends on the stored or target structure, the distribution of cues or training examples, and the family of similarities or probes available to the procedure.

Correlations and local similarities may then contain useful information about the underlying association. Three questions must nevertheless be kept separate:

1. Does a relevant footprint *exist*, rather than vanish by exact cancellation?

2. Is it *large enough to detect* at a controlled probe scale, with feasible samples and computation, including the cost of finding a useful probe?

3. Can the specified retrieval rule or learning updates *use* that information to make progress?

The distinction is

nonzero footprint
$\neq$ efficiently detectable footprint
$\neq$ an established retrieval or learning guarantee.

There exist target functions for which low-order correlations vanish, and cases in which a footprint is formally nonzero but exponentially small. Even a detectable statistic need not be among the directions accessed by a particular optimizer. A reliable-retrieval or convergence claim therefore needs assumptions about the actual procedure, not only the existence of a correlation.

The staircase and the cliff describe possible learning mechanisms within this picture. In a staircase, constructing and preserving one useful coordinate can expose a later dependency. A cliff denotes the absence of a useful signal for the specified local mechanism. Failure of that mechanism is not an impossibility theorem for every learner.

Genericity is thus neither part of the definition of associative memory nor a universal prerequisite for predictive success. It is a candidate explanation for accessible information in particular retrieval and learning problems. Learning an accurate predictor and identifying the particular computation that generated the observations remain distinct objectives, as Chapter H makes explicit.

2.6 Associative Memory Is a Primitive, Not a Complete Theory

The historical continuity described here should not be confused with a claim that associative memory alone explains intelligence.

Associative retrieval solves an important problem:

given the present cue, retrieve relevant stored structure.

But several additional problems remain.

Learning. The existence of a useful associative representation does not explain how training finds an accurate predictor from data. A system may be capable of representing the desired input–output relation while optimization fails to find a suitable fit. Nor does fitting the observed examples, by itself, explain prediction beyond them. These are the optimization and generalization questions developed later in the book.

Retrieval. Even if a representation has been learned, a partial cue may not uniquely or efficiently identify the relevant memory.

This is an inverse problem and depends on the geometry and statistics of the stored patterns. Here recovery means retrieving associated content; it should not be confused with identifying the latent computation that produced the training data.

Predictive success and structural recovery. Predictive learning asks for an accurate function on the population of interest. Structural recovery additionally asks for a specified generating computation or its intermediate variables, up to controlled equivalences. A predictor can succeed without reproducing those variables. Its learned basis or factorization may differ from the generating one without that difference constituting a predictive failure.

Composition. A collection of associations need not organize itself automatically into reusable modules. Recombining learned computations across tasks and contexts is an important additional capability, not a consequence of associative retrieval alone. A claim about such reuse requires more than accurate prediction on one task, and need not require that the reusable modules coincide with the teacher's constituents.

Persistence. Many associative mechanisms operate only over the currently available state.

Standard transformers, for example, possess a working context and a key–value cache during an episode but do not, by themselves, provide persistent memory across indefinitely separated interactions.

World models and planning. Associative retrieval does not by itself provide causal understanding, explicit dynamics, counterfactual reasoning, or planning.

Such abilities may make extensive use of associative memories, but they require additional computational structure.

The appropriate conclusion is therefore

associative memory is a primitive, not a complete theory of intelligence.

Its importance lies precisely in the fact that such a primitive can be embedded inside richer dynamical and compositional systems.

Why associative retrieval does not produce a collage. Describing attention as an associative-memory operation does not imply that a Transformer merely retrieves and concatenates fragments of its training data. Attention is only one computational primitive inside a larger conditional sequence model. At each step the complete network computes

$$p_\theta(x_t \mid x_{<t}),$$

using repeated attention-based routing together with local nonlinear transformations. Autoregressive training therefore fits conditional continuations that depend jointly on the preceding context. Coherent generation should consequently be attributed to the learned conditional sequence function as a whole, not to associative retrieval alone. Associative memory explains an important routing and retrieval primitive; it is not by itself a theory of discourse coherence.

2.7 Experimental Evidence from Homogeneous HyperBF Architectures

The connection between associative kernels and transformer-like architectures can also be tested experimentally.

Recent work [87] constructs a "homogeneous" architecture in which the nonlinear components of a Vision Transformer are implemented using explicit HyperBF modules.

In particular, both the attention-like mixing operations and the feed-forward transformations are replaced by modules built from the same general kernel-based primitive.

The important point is not that an ordinary transformer MLP is mathematically identical to a HyperBF. It is not.
Rather, the experiment asks a different question:

> Can an architecture built largely from associative-memory primitives perform competitively with a standard transformer?

On several image-classification benchmarks the resulting architecture closely approaches the performance of a baseline Vision Transformer of comparable size.

Dataset	ViT Acc. (%)	HyperBF Acc. (%)	Gap (%)
CIFAR10	75.61	75.12	0.49
CIFAR100	49.90	48.30	1.60
Tiny ImageNet	32.03	31.14	0.89

Table 2.1: Comparison between a baseline Vision Transformer and a homogeneous HyperBF architecture on image-classification benchmarks reported in [87].

These results show that, on these benchmarks and at this scale, explicit HyperBF modules can substitute for the standard nonlinear components of a Vision Transformer with only a modest loss in performance.

This does not establish that a conventional transformer is literally a composition of HyperBFs.

It supports a different and, in some ways, more interesting statement: competitive transformer-like systems can be constructed using associative-memory primitives throughout much of the architecture.

The experiment therefore provides evidence for the architectural sufficiency of the associative-memory viewpoint rather than for literal identity between standard transformer components and HyperBF modules.

2.8 Conclusion

The history of associative memory reveals a striking recurrence.

Early heteroassociative systems retrieve an output through correlation with a cue.

Nonlinear operator memories enlarge this idea through polynomial features and kernels.

RBF and HyperBF architectures replace global correlation by similarity in learned metric spaces.

Autoassociative networks add recurrent pattern completion.

Biological memory systems embed associative retrieval in plastic, hierarchical, and recurrent circuitry.

Modern attention transforms content-addressable retrieval into a learned, differentiable primitive repeatedly composed across many layers.

The historical lesson is not that all of these systems are instances of the same architecture.

It is that the computational motif

$$\boxed{\text{cue} \longrightarrow \text{content-dependent retrieval} \longrightarrow \text{use of the retrieved structure}}$$

reappears whenever a system must use past structure to interpret the present.

What changes across this history is the machinery surrounding that primitive: nonlinear similarity, learned representations, recurrence, feedback, hierarchy, and repeated composition.

Associative memory may therefore be less a complete model of intelligence than one of its elementary operations.

This leaves several theoretical questions unresolved. Why is relevant content reliably retrievable from a cue? How does training find a useful predictor with feasible resources? Why does the selected predictor generalize beyond the examples used to fit it?

The next chapter isolates the computational question. The Associative Turing Machine shows how associative reads and local updates can execute classical computations under the stated memory and representation assumptions. Executing a supplied program is not the same as discovering a useful program from data.

The Interlude then separates representation, predictive learning, and structural recovery. Chapter 4 develops compositional sparsity as a representation principle. Subsequent chapters examine accessible learning signals, supplied intermediate information, optimization geometry, and statistical selection. Genericity and staircase learning provide possible constructive mechanisms; trace exposure can assist learning rather than serving only as a fallback.

Chapter H brings these roles together. When training finds an accurate predictor whose description and finite-precision evaluation obey the stated polynomial resource bounds, the learned computation itself supplies a bounded-fan-in compositional realization. It need not reproduce the computation that generated the observations. This conclusion does not explain how training achieves accuracy; it clarifies what follows once that success and computational feasibility have been established.

The historical account therefore identifies a recurring primitive from which useful computations can be built. The learning theory must explain how such a computation is found and why it predicts well. Identification of the generating structure remains a stronger, separate objective.

CHAPTER 3

Associative Memories Are Turing-Complete

The previous chapters argued that attention can be understood as content-addressable associative retrieval. We now ask a sharper question: is associative retrieval, together with local state updates and writable working memory, computationally universal?

*We introduce an **Associative Turing Machine** (ATM), a computational model whose basic operations are associative read, associative write, and local update governed by a finite transition rule. We show that every deterministic Turing machine can be simulated with a constant number of these abstract operations per computational step. This operation count is distinct from the bit cost of implementing a memory access.*

The universality theorem is first stated for exact associative reads. We then show that softmax attention provides a differentiable approximation whose error can be made arbitrarily small when memory keys are sufficiently separated. Thus associative memory is not merely a mechanism for recall: when made writable and iterated under the stated memory conventions, it is sufficient for universal computation.

The theorem concerns execution of a supplied program. It does not explain how training discovers a useful predictor, why that predictor generalizes, or whether its internal computation reproduces the process that generated the observations. Those learning questions remain separate.

3.1 From Associative Recall to Universal Computation

An associative memory answers a simple question:

given a cue, what stored information is associated with it?

By itself, this is only a retrieval operation.

A computer requires more. The retrieved information must be allowed to modify a state, influence subsequent queries, and change working memory.

The minimal computational cycle is therefore

$$\boxed{\text{read} \rightarrow \text{local update} \rightarrow \text{write} \rightarrow \text{new query}}$$

repeated over time.

The central claim of this chapter is that this cycle is already enough for general computation.

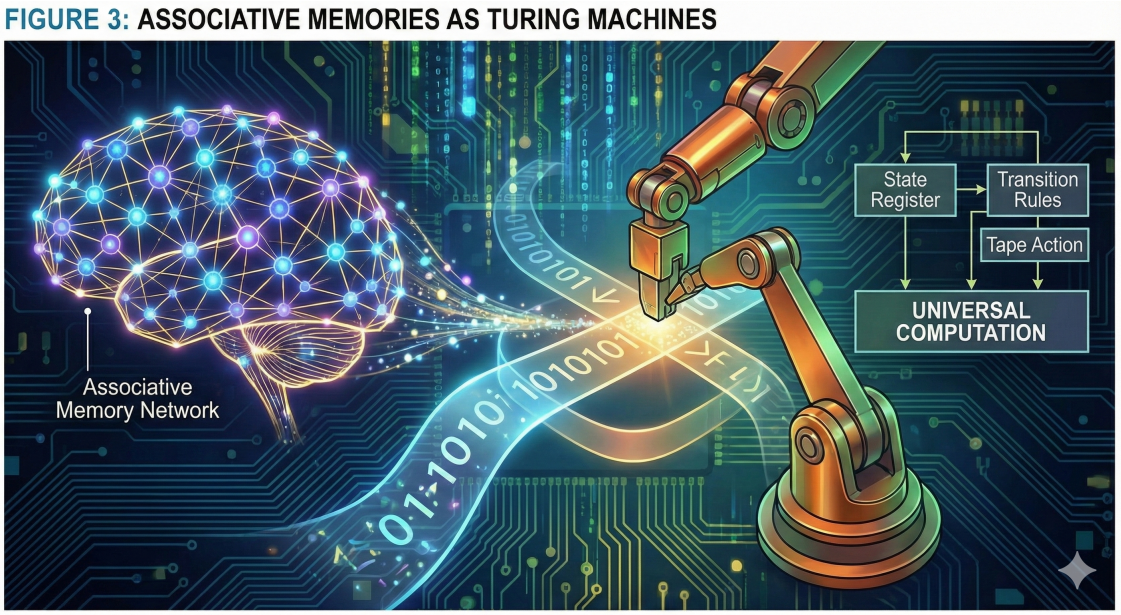


Figure 3.1: From a classical Turing tape to associative computation. The essential operations of a Turing machine—reading state, retrieving a transition rule, writing a symbol, and updating control—can be expressed through content-addressable memory interactions and local updates.

The construction is deliberately close to the operational structure of a classical Turing machine. A Turing machine needs only:

1. a finite control state;
2. a readable and writable tape;
3. a transition rule;
4. a head position.

An ATM implements the same ingredients through associative memory.

The transition table is stored in a fixed associative memory. The tape is stored in a dynamically writable associative memory. The finite controller performs only a local state update.

This avoids encoding an unbounded tape into arbitrary precision of a single real number. The growing tape contents reside explicitly in the associative store, and the address representation is counted as part of the workspace rather than treated as a fixed-precision resource.

3.2 Definition of the Associative Turing Machine

3.2.1 Control State

Let

$$Q$$

be a finite set of control states, and let

$$\Sigma$$

be a finite tape alphabet containing a blank symbol

$$\sqcup.$$

At computational step t, write the current control state and tape address together as

$$c_t = (q_t, h_t),$$

where

$$q_t \in Q$$

is the current machine state and

$$h_t \in \mathbb{Z}$$

is the current tape address.

Only q_t ranges over a fixed finite set. The pair c_t is not itself finite-state, because h_t is an integer address. The controller's transition rule has a fixed finite description, while the address is represented by a finite string whose length may grow. For an input of length n and a run of t steps, addresses can be represented with $O(\log(n + t + 2))$ bits. These bits belong to the explicitly counted workspace; they are not unbounded information hidden in a fixed-precision real coordinate.

3.2.2 Program Memory

The fixed program memory contains the transition table of the simulated Turing machine.

For a deterministic Turing machine with transition function

$$\delta : Q \times \Sigma \to Q \times \Sigma \times \{-1, +1\},$$

define the associative program memory below. Transition instructions are used only in nonhalting states; execution stops on entering the halting state, so entries associated with it need not be consulted.

$$\mathcal{M}_{\text{prog}} = \{(k_{q,s}, v_{q,s}) : (q, s) \in Q \times \Sigma\}.$$

The key encodes the pair (q, s),

$$k_{q,s} = B(q, s),$$

and the corresponding value encodes

$$v_{q,s} = \text{pack}\,(\delta(q, s)).$$

Thus the query

$$(q, s)$$

returns the transition instruction

$$(q', s', d).$$

Because Q and Σ are finite, these keys can be represented exactly by orthogonal basis vectors. Let

$$N_{\text{prog}} = |Q||\Sigma|.$$

We may choose

$$k_{q,s} = e_{(q,s)} \in \mathbb{R}^{N_{\text{prog}}},$$

so that

$$\langle k_{q,s}, k_{q',s'} \rangle = \begin{cases} 1, & (q,s) = (q',s'), \\ 0, & \text{otherwise.} \end{cases}$$

The transition table therefore admits exact associative addressing.

3.2.3 Working Memory

The work memory represents the Turing tape.

At time t, write

$$\mathcal{M}_{\text{work}}^{(t)}$$

for the finite collection of tape locations that have been written so far.

Conceptually, the memory implements a map

$$h \longmapsto T_t(h),$$

where

$$T_t(h) \in \Sigma$$

is the symbol currently stored at tape address h.

Addresses that have never been written return the blank symbol

$$\sqcup.$$

A tape read is therefore

$$s_t = \text{read}\left(h_t; \mathcal{M}_{\text{work}}^{(t)}\right).$$

A tape write replaces the value stored at address h_t:

$$\mathcal{M}_{\text{work}}^{(t+1)} = \text{write}\left(\mathcal{M}_{\text{work}}^{(t)}, h_t, s_t'\right).$$

This is the simplest abstract definition.

An equivalent implementation uses an append-only log. Instead of destructively replacing the previous value, one appends a record

$$(h_t, t, s_t').$$

A future read at address h retrieves the most recent entry associated with h.

The append-only formulation will be especially useful when we relate the ATM to autoregressive sequence models. Exact address matching, selection of the latest matching record, and the default blank response are part of this abstract tape interface. A concrete attention implementation must realize these operations; an arbitrary similarity read does not supply them automatically.

3.2.4 Associative Read

For the exact computational model, define a hard associative read.

Given query q and memory

$$\mathcal{M} = \{(k_i, v_i)\}_{i=1}^{N},$$

let

$$i^*(q) = \arg\max_i \langle q, k_i \rangle.$$

Then

$$\mathrm{read}(q; \mathcal{M}) = v_{i^*(q)}. \tag{3.1}$$

Whenever the maximizing key is unique, the read is deterministic.

The ATM theorem below uses this exact primitive together with the tape interface just specified. The finite program memory has the explicit orthogonal encoding above. The work-memory interface additionally requires an address encoding and the stated blank and update conventions.

Effective descriptions and operation counts. The symbolic construction uses finite, effectively manipulable representations of states, addresses, keys, and instructions. Reads use decidable comparisons on those representations. No arbitrary-real oracle is being supplied. One read or write counts as one abstract ATM operation; its implementation cost can depend on memory size and representation length. The resource interpretation of the theorem is discussed after its proof.

3.2.5 Associative Write

A writable associative memory supports an operation

$$\mathcal{M}' = \mathrm{write}(\mathcal{M}, k, v), \tag{3.2}$$

which ensures that subsequent reads at address k return v.

The internal implementation of the write is abstracted away in the computability theorem. It may be destructive replacement, append-only logging, or an equivalent effective mechanism preserving the current value at each address. That abstraction does not remove its storage or bit-level cost from an implementation-level efficiency claim.

3.2.6 Local Update

The controller receives

$$(q_t, h_t),$$

the tape symbol

$$s_t,$$

and the retrieved transition instruction

$$(q_{t+1}, s'_t, d_t).$$

The local update is simply

$$h_{t+1} = h_t + d_t,$$

together with the new finite state

$$q_{t+1}.$$

Thus one ATM computational step consists of:

1. read the tape symbol at h_t;
2. retrieve the transition instruction associated with (q_t, s_t);
3. write the new symbol s'_t at h_t;
4. update the control state to q_{t+1};
5. move the head to $h_t + d_t$.

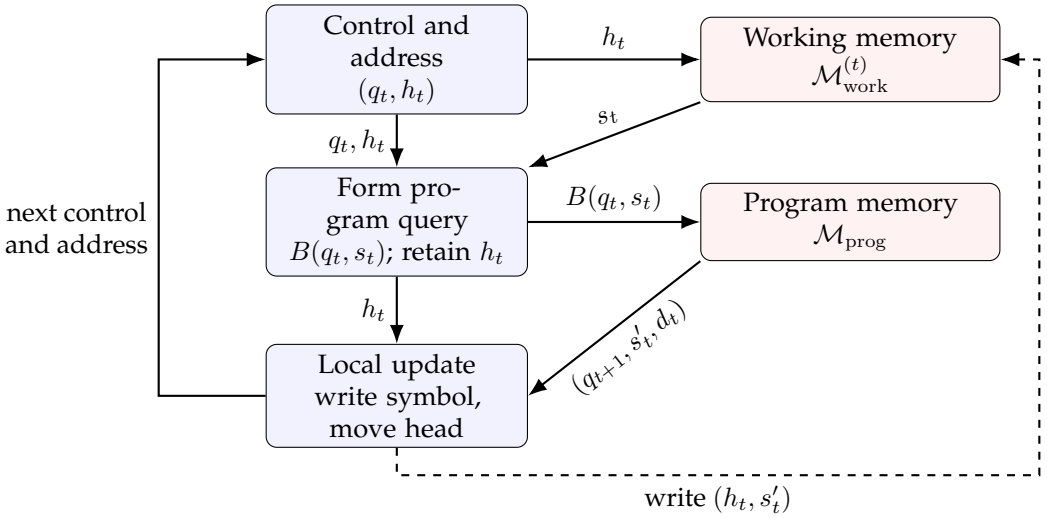


Figure 3.2: One step of an Associative Turing Machine. The current tape address is used to retrieve a symbol from working memory. The pair consisting of the control state and retrieved symbol addresses the program memory. The returned instruction determines the write operation and the next control state. Query formation is shown explicitly; the address is retained for the write and head update.

3.3 Why Extensible Memory Matters: RNNs, Precision, and Robust Computation

The motivation for introducing an explicit working memory is not that recurrent neural networks are incapable of universal computation in every mathematical model.

Indeed, idealized recurrent networks operating with exact real arithmetic can encode arbitrarily much information in a fixed-dimensional continuous state and, under appropriate constructions, can simulate Turing computation.

The relevant issue here is different:

Can unbounded computation be carried out robustly with fixed-dimensional, bounded, finite-precision internal state?

Under those physical restrictions, the answer is no for Turing-universal computation with indefinitely extensible workspace. A finite-state loop can of course run for arbitrarily many steps; runtime alone is not the unbounded memory resource at issue here.

3.3.1 Fixed-dimensional state with finite precision

Consider an autonomous recurrent system

$$h_{t+1} = F(h_t), \qquad h_t \in \mathbb{R}^d, \tag{3.3}$$

where d is fixed.

Assume that physically realizable states are bounded,

$$h_t \in [-B, B]^d,$$

and that two states separated by less than a resolution $\varepsilon > 0$ cannot be reliably distinguished.

Then the number of mutually separated state codes is finite. To obtain a finite-state computational model, one also needs the update to be well defined on those codes, rather than depend on unresolved differences inside a code's tolerance region.

Proposition 1 (Bounded finite-precision recurrence is finite-state)**.** *Let*

$$\mathcal{H} \subseteq [-B, B]^d$$

be the state space of a deterministic recurrent system.

Suppose distinct valid computational codes have representatives in $\mathcal{H}$ separated by at least $\varepsilon > 0$ in the ℓ_∞ norm. Assume also that the update, including any quantization, induces a well-defined deterministic transition on decoded codes: states representing the same code have the same decoded successor.

Then the number of codes is bounded by the finite packing number

$$N_{\text{robust}} \leq \left(1 + \frac{2B}{\varepsilon}\right)^d. \tag{3.4}$$

Consequently, at fixed B, d, and ε, the recurrent system has only finitely many robust computational states and is therefore equivalent, at this level of description, to a finite-state machine.

Proof. Around each code representative place an open ℓ_∞ ball of radius $\varepsilon/2$, that is, an open cube of side ε. The separation assumption makes these cubes disjoint. All of them lie in a cube of side $2B + \varepsilon$. For any finite collection of N code representatives, comparison of volumes gives

$$N\varepsilon^d \leq (2B + \varepsilon)^d.$$

Thus there can be at most $(1 + 2B/\varepsilon)^d$ codes, proving Equation (3.4).

By the decoded-transition assumption, the update acts deterministically on this finite code set. It can therefore be represented by a finite transition table, which is the claimed finite-state computation.

□

The important point is not recurrence itself.

It is the simultaneous assumption of

$$\boxed{\text{fixed dimension} + \text{bounded range} + \text{finite resolution.}}$$

3.3.2 How idealized recurrent systems evade the proposition

There are at least two mathematical ways for a fixed-dimensional system to avoid the finite-state conclusion.

Unbounded precision. A bounded real number can encode an arbitrarily long discrete sequence in its digits.

For example, a symbolic sequence

$$s_1 s_2 s_3 \cdots$$

can be represented schematically by

$$x = \sum_{j=1}^{\infty} s_j b^{-j}.$$

A recurrent system with exact real arithmetic can manipulate such encodings.

But as more symbols are stored, distinct computational states can become arbitrarily close.

Thus the required precision grows without bound.

Unbounded magnitude. Alternatively, one may use discrete but unbounded states, such as integer registers

$$h_t \in \mathbb{Z}^d.$$

A fixed number of unbounded integer registers can themselves support unbounded computation.

Here the resource that grows is numerical magnitude at fixed absolute resolution. The number of bits needed to store those integers still grows; unbounded integer registers are not fixed-bit hardware states.

Thus the correct general principle is

unbounded computation requires some computational resource
that is itself allowed to grow without a fixed bound.

Depending on the model, that resource may be

precision, state magnitude, or memory size.

3.3.3 Why use explicit memory?

The Associative Turing Machine chooses the third option.

Instead of encoding an arbitrarily large working memory inside a fixed continuous state, it separates

finite control state

from

extensible writable memory.

The fixed control logic manipulates only the information required for the current computational step:

control state, query/address, retrieved local data.

The growing tape record is stored externally. Addresses and any timestamps also have finite encodings whose lengths grow as needed; this does not turn the finite control rule into an unbounded-precision real operation.

Schematically,

fixed control logic with explicitly represented addresses
$+$
extensible addressable memory
$\Downarrow$
unbounded computational workspace.

This is the same architectural separation used by an ordinary computer: the processor is finite, while the amount of memory available to a computation may grow with the computation.

3.3.4 Memory hierarchy and computational power

The distinction can be expressed abstractly in terms of the memory available to a finite controller.

finite controller only	$\longrightarrow$	finite-state computation,
finite controller + extensible stack	$\longrightarrow$	pushdown-style computation,
finite controller + extensible read/write memory	$\longrightarrow$	Turing computation.

The neural nature of the controller is secondary.

What changes the computational class is the structure and extensibility of the memory resource.

3.3.5 Robustness rather than idealized expressivity

This also clarifies the role of the ATM theorem.

The claim is not that exact-real recurrent networks are mathematically incapable of Turing universality.

Rather, the ATM provides a representation of Turing computation in which unbounded information is not hidden in infinitely precise coordinates of a fixed-dimensional state.

Instead, information is stored explicitly in a memory whose size can grow with the computation.

Thus

> ATM universality is based on extensible explicit memory,
> not on infinite precision in a fixed state vector.

3.3.6 Execution versus trainability

A final distinction is important.

Problems such as vanishing or exploding gradients concern the difficulty of *learning* recurrent dynamics by backpropagation through time.

They are not the fundamental reason that a bounded finite-precision recurrent state has limited memory capacity.

A recurrent machine could be perfectly trained and still possess only a finite number of robust states.

Conversely, a mathematically universal recurrent construction may be extremely difficult to learn.

Therefore,

$$\boxed{\text{computational expressivity} \neq \text{learning or generalization.}}$$

The universality construction supplies a program and an encoding, then proves that execution follows the intended transitions. A learning procedure receives observations and must find a useful predictor. Its success additionally concerns performance beyond those observations.

As distinguished in the Interlude and Chapter H, predictive recovery does not require identifying the program that generated the data. Matching that program's hidden states or transitions is a stronger structural-recovery objective. Reliable execution of a supplied program, predictive learning, and structural identification are therefore separate claims.

3.3.7 The design principle

The architectural motivation for the ATM can therefore be summarized as

> do not compress an indefinitely growing working memory
> into a fixed robust state when that memory can instead
> be represented explicitly and addressed directly.

The ATM implements precisely this separation:

$$\boxed{\text{associative read} + \text{local update} + \text{extensible writable memory.}}$$

The following sections show that these primitives are sufficient to simulate ordinary Turing computation with only polynomial overhead.

3.4 Simulation of a Turing Machine

Let

$$T = (Q, \Sigma, \delta, q_0, q_{\text{halt}})$$

be a deterministic single-tape Turing machine.
We construct an ATM

$$A_T$$

that simulates it step by step.

3.4.1 Encoding the Transition Table

For every pair

$$(q, s) \in Q \times \Sigma,$$

store in program memory

$$(B(q, s), \text{pack}(\delta(q, s))) .$$

With the orthogonal encoding described above, each valid pair has a unique key.
Thus the program read

$$\text{read}\,(B(q_t, s_t); \mathcal{M}_{\text{prog}})$$

returns exactly

$$\delta(q_t, s_t).$$

3.4.2 Encoding the Tape

Suppose the initial Turing tape contains input string

$$x = x_1 x_2 \cdots x_n.$$

Initialize

$$\mathcal{M}_{\text{work}}^{(0)}$$

with

$$(1, x_1), (2, x_2), \ldots, (n, x_n).$$

All other addresses return

$$\sqcup.$$

The ATM head position is initialized to the same location as the Turing machine head.

At every later step, the work memory contains exactly the current tape contents of the simulated machine.

3.4.3 One Simulated Step

Assume that at time t the ATM represents the same configuration as the Turing machine.

First retrieve the current tape symbol:

$$s_t = \text{read}\left(h_t; \mathcal{M}_{\text{work}}^{(t)}\right).$$

Then retrieve the corresponding transition instruction:

$$(q_{t+1}, s'_t, d_t) = \text{read}\left(B(q_t, s_t); \mathcal{M}_{\text{prog}}\right).$$

Write the new symbol:

$$\mathcal{M}_{\text{work}}^{(t+1)} = \text{write}\left(\mathcal{M}_{\text{work}}^{(t)}, h_t, s'_t\right).$$

Finally update the head:

$$h_{t+1} = h_t + d_t.$$

The resulting ATM configuration is exactly the next configuration of the Turing machine.

3.5 Main Theorem

Theorem 1 (Turing Equivalence of Associative Computation)**.** *For every deterministic single-tape Turing machine T, there exists an Associative Turing Machine A_T such that, for every input x,*

$$A_T(x)$$

halts if and only if

$$T(x)$$

halts, and both machines produce the same output.

One step of T is simulated by a constant number of associative reads, writes, and local updates. Consequently, if T halts after t steps, then A_T halts after

$$O(t)$$

ATM operations.

3.5.1 Proof

The proof is by induction on the simulated time step.

At time $t = 0$, the control state, head location, and tape contents of A_T are initialized to match those of T.

Assume that after t simulated steps the ATM and the Turing machine have identical configurations.

The work-memory read at address h_t therefore returns the same tape symbol s_t observed by the Turing machine.

The program-memory read at key

$$B(q_t, s_t)$$

returns exactly

$$\delta(q_t, s_t) = (q_{t+1}, s'_t, d_t).$$

The ATM writes s'_t to address h_t, enters state q_{t+1}, and moves the head to

$$h_t + d_t.$$

These are precisely the operations performed by one step of the Turing machine.

Therefore the configurations remain identical after step $t + 1$.

By induction, the ATM reproduces the entire computation of T, including its halting behavior and output.

Each simulated step uses a constant number of primitive ATM operations, so the simulation requires $O(t)$ steps.

□

What the overhead counts. The $O(t)$ bound counts abstract ATM primitives after input initialization, for a fixed simulated machine T. Storing an input of length n costs $O(n)$ initial writes, so including this stage gives $O(n + t)$ primitive operations. A run needs only $O(n + t)$ tape records, or that many records in the append-only implementation; addresses and timestamps use $O(\log(n + t + 2))$ bits each.

An implementation of a read may nevertheless inspect many records. In particular, scoring every stored key is not a constant-bit-cost operation as memory grows. The theorem's constant primitive overhead is not a constant physical-time or bit-operation overhead for dense attention.

Conversely, the symbolic tape interface in this construction can be implemented on an ordinary Turing machine by keeping finite records, scanning for matching addresses, and applying the finite transition table. This gives an effective classical simulation with polynomial overhead in the input length and simulated runtime. The computational equivalence is therefore about effective symbolic machines, not machines endowed with arbitrary real-valued oracles.

3.5.2 Corollary: A Universal ATM

A universal Turing machine is itself a deterministic Turing machine.

Applying Theorem 1 to such a machine gives a single universal ATM that, given an encoding

$$\langle T, x \rangle,$$

can simulate arbitrary deterministic Turing machines.

Thus

Associative Turing Machines are computationally universal.

3.6 Soft Attention as a Differentiable Approximation

The theorem above uses exact associative reads.

Modern attention uses softmax retrieval instead.

We now show that the exact ATM read can be approximated arbitrarily well by soft attention whenever the correct key is separated from the others by a positive margin. The estimates below concern a single retrieval. Maintaining valid encoded states and accurate computation over many steps requires additional control of the updates, writes, and successive query margins.

3.6.1 Softmax Read

Given normalized query and keys,

$$\|q\| = \|k_i\| = 1,$$

define

$$\mathrm{read}_\tau(q; \mathcal{M}) = \sum_{i=1}^{N} \alpha_i(q) v_i, \tag{3.5}$$

where

$$\alpha_i(q) = \frac{\exp(\langle q, k_i \rangle / \tau)}{\sum_{j=1}^{N} \exp(\langle q, k_j \rangle / \tau)}.$$

The parameter

$$\tau > 0$$

controls the sharpness of retrieval.
As

$$\tau \to 0,$$

the distribution concentrates on the maximally similar key.

3.6.2 General Margin Bound

For this bound take $N \geq 2$ and an error tolerance $0 < \epsilon < 1$. With a single stored item, the read is already exact.

Assume the correct key k_* satisfies

$$\langle q, k_* \rangle - \max_{j \neq *} \langle q, k_j \rangle \geq \gamma$$

for some margin

$$\gamma > 0.$$

Then

$$\alpha_* = \frac{1}{1 + \sum_{j \neq *} \exp\left(-\frac{\langle q, k_* \rangle - \langle q, k_j \rangle}{\tau}\right)}.$$

Therefore

$$\alpha_* \geq \frac{1}{1 + (N-1)e^{-\gamma/\tau}}.$$

Hence

$$1 - \alpha_* \leq (N-1)e^{-\gamma/\tau}. \tag{3.6}$$

To ensure

$$1 - \alpha_* \leq \epsilon,$$

it suffices to choose

$$\tau \leq \frac{\gamma}{\log((N-1)/\epsilon)}.$$

Thus the soft associative read approaches the exact read exponentially fast in

$$\gamma/\tau.$$

3.6.3 Orthogonal-Key Case

For the program memory, we may choose orthogonal keys.

If

$$q = k_*,$$

then

$$\langle q, k_* \rangle = 1$$

and

$$\langle q, k_j \rangle = 0 \qquad (j \neq *),$$

so

$$\gamma = 1.$$

The correct attention weight is then exactly

$$\alpha_* = \frac{e^{1/\tau}}{e^{1/\tau} + N - 1}.$$

Therefore

$$1 - \alpha_* = \frac{N-1}{e^{1/\tau} + N - 1}.$$

As

$$\tau \to 0,$$

we obtain

$$\alpha_* \to 1.$$

Thus

softmax attention can approximate the exact ATM read arbitrarily closely.

3.6.4 From Retrieval Error to State Error

Suppose the stored values satisfy

$$\|v_i\| \le R.$$

If the correct value is v_*, then

$$\mathrm{read}_\tau(q;\mathcal{M}) - v_* = \sum_{i\neq *} \alpha_i (v_i - v_*).$$

Hence

$$\|\mathrm{read}_\tau(q;\mathcal{M}) - v_*\| \le 2R(1-\alpha_*).$$

Using Eq. (3.6),

$$\|\mathrm{read}_\tau(q;\mathcal{M}) - v_*\| \le 2R(N-1)e^{-\gamma/\tau}.$$

Thus the finite-temperature read error is explicitly controlled. This is the error in the value delivered to the controller, before any subsequent state update. A small value error does not by itself guarantee the same symbolic transition, particularly near a decoding boundary.

For a finite simulated run, one must also control accumulation through the updates, preserve the separation needed for later reads, and specify how approximate values are decoded and written. As the memory grows, N, the available margins, or the required temperature may change. The displayed bound therefore does not imply exact arbitrarily long computation at one fixed temperature and fixed numerical resolution.

This separates two questions that are sometimes conflated:

exact associative machine: exact symbolic computation,
soft attention realization: differentiable approximation.

3.7 Relation to Transformer Computation

The ATM is an abstract computational model.

Transformer components provide natural realizations of several of its primitives, but the correspondence should not be interpreted as a literal identity between a fixed Transformer block and an unbounded Turing machine.

3.7.1 Queries, Keys, and Values

In a standard self-attention layer,

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

The learned matrices

$$W_Q,\ W_K,\ W_V$$

do not themselves constitute the contextual associative memory.

Rather, they parameterize the transformations that generate queries, keys, and values from the current representations.

The memory available to attention is the set of contextual key–value pairs

$$\{(k_i, v_i)\}.$$

The attention operation

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

therefore implements a differentiable associative read over the current context.

3.7.2 Residual State Update

A transformer block combines associative retrieval with a local representation update.

Ignoring normalization details, one may write schematically

$$z_t = x_t + \mathrm{Attn}(x_t),$$

followed by

$$x_{t+1} = z_t + \mathrm{FFN}(z_t).$$

Thus the ATM update primitive

$$M_1$$

corresponds more naturally to the complete residual transformation surrounding attention and the feed-forward network than to the FFN alone.

The computational motif is

retrieve information $\rightarrow$ combine it with state $\rightarrow$ transform state.

3.7.3 Append-Only Working Memory

A standard transformer does not directly overwrite earlier token representations in the manner of writable RAM.

Autoregressive systems, however, naturally possess an append-only mechanism.

Suppose a tape write at address h_t produces symbol s'_t.

Instead of erasing the old entry, append a record

$$(h_t, t, s'_t).$$

Later reads at address h_t must retrieve the most recent matching record. This requires an explicit latest-match rule using the encoded addresses and timestamps. Appending tokens alone does not establish that a learned attention rule implements this selection.

The tape history becomes

$$(h_1, t_1, s_1), (h_2, t_2, s_2), \ldots$$

and the current tape state is obtained by selecting, for each address, the latest associated value.

This provides a natural bridge between writable associative memory and autoregressive sequence computation:

$$\boxed{\text{Turing tape write} \longleftrightarrow \text{append a new address–value record.}}$$

The mechanism is conceptually similar to log-structured memory in computer systems: old states remain in the history, while the newest entry determines the current value.

3.7.4 What the Correspondence Does and Does Not Say

The ATM theorem establishes that associative read, write, and local update are sufficient for universal computation.

It does *not* imply that every fixed Transformer is itself an unbounded Turing machine.

A Transformer of fixed depth and fixed context performs only a finite amount of computation in a single evaluation. Repeated use can extend runtime, but an unbounded simulation still needs the corresponding memory and representation resources.

To simulate arbitrarily long Turing-machine computations, some resource must grow with the simulated runtime, for example:

- sequence length;
- recurrent application of a block;
- depth;
- external memory;
- dynamically expanding working memory.

Thus the appropriate conclusion is not

$$\text{every Transformer} = \text{Turing machine}.$$

Rather,

$$\boxed{\begin{array}{c}\text{Transformer-like associative primitives can realize the} \\ \text{operations required by a Turing-equivalent machine.}\end{array}}$$

3.8 Why Associative Access Matters

Turing completeness by itself is not unique to associative systems.

Idealized recurrent neural networks are also known to admit universal computational constructions.

The significance of the ATM is therefore not that it is the first neural model capable of universal computation.

The difference lies in the memory primitive.

A conventional recurrent system compresses the computational past into a recurrent hidden state.

An ATM exposes memory explicitly through content-addressable access.

ATM Primitive	Transformer-Like Realization	Computational Role
Query Formation	Query projection	State-to-address map
Associative Memory	Contextual keys and values	Content-addressable storage
Read	Softmax attention	Associative retrieval
Local Update	Residual + MLP transformation	State transition
Write	Append new state/token	Working-memory update
Iteration	Autoregressive or recurrent execution	Repeated computation

Table 3.1: Structural correspondence between ATM primitives and transformer-like computational mechanisms. The table describes a functional analogy, not an identity between a fixed Transformer block and a complete unbounded ATM.

This changes the computational organization from

$$\text{everything encoded in recurrent state}$$

to

$$\text{finite controller} + \text{explicit associative working memory}.$$

The distinction is especially relevant for modern architectures because attention provides direct access to distributed memories according to content rather than requiring all relevant information to be propagated sequentially through a fixed-dimensional hidden state.

Thus the main structural lesson is

$$\boxed{\text{associative access provides an explicit computational memory interface.}}$$

3.9 Discussion

The Turing-machine simulation proved in this chapter is intentionally simple.

That simplicity is the point.

A classical Turing machine requires only a finite controller, a readable and writable memory, and repeated local transitions.

Replacing pointer-based tape access by content-addressable retrieval does not reduce computational power.

Indeed,

$$\boxed{\text{associative read} + \text{associative write} + \text{local finite control} = \text{universal computation}.}$$

This result clarifies the role of associative memory in the larger argument of this book.

Associative memory is not merely a mechanism for retrieving stored examples.

Once retrieval is allowed to affect subsequent queries and working memory, association becomes computation.

The exact ATM makes this statement at the level of classical computability, with the effective symbolic representations and abstract operation costs specified above.

Softmax attention provides a differentiable approximation to an individual exact read, with an explicit error bound determined by key separation, memory size, temperature, and value scale. A

complete soft simulation requires the additional trajectory-level control discussed in the preceding error analysis; it is not a consequence of the one-read estimate alone.

The resulting conceptual progression is

$$\boxed{\begin{array}{c}\text{associative recall} \to \text{associative working memory} \\ \Downarrow \\ \text{iterated read–update–write} \to \text{Turing computation.}\end{array}}$$

This establishes a computational possibility, not an efficient-learning theorem. The simulator is supplied with the program to execute. Its universality does not explain how observations lead a learning procedure to a useful program, or why the resulting predictor works beyond the observed examples.

Nor is universality itself an efficiency claim about the target task. The ATM can simulate a long or expensive computation as well as a short one. Chapter 4 addresses a more restricted question: what representation follows when the target can be evaluated with polynomial digital resources? Under the specified finite-precision conventions, unrolling such a computation yields a polynomial-size bounded-fan-in graph. Smooth coarse-grained approximation requires further regularity and stability assumptions; it is not supplied by Turing universality alone.

The Interlude separates representation from two meanings of recovery. Predictive recovery asks for a feasible function that is accurate on the population of interest. Structural recovery asks, in addition, for identification of a designated generating computation up to controlled equivalences. A successful predictor may execute a different program and use different intermediate variables from those that generated the data. That difference is not, by itself, a predictive failure.

The subsequent chapters therefore ask three distinct questions:

Representation:	Is an accurate feasible predictor available?
Optimization:	Can the specified training procedure find it?
Generalization:	Does the selected predictor work off the sample?

Accessible footprints, staircase learning, and intermediate traces offer possible routes to successful learning under their respective assumptions. They are not conclusions of the ATM theorem, and matching the generating program is not an additional requirement for predictive success.

Chapter H makes the resulting witness argument precise. When training returns an accurate predictor whose finite description and numerical evaluation satisfy the stated polynomial resource bounds, that predictor's own computation supplies a bounded-fan-in compositional realization. No separate theorem matching its internal states to those of the teacher is needed for that conclusion. Explaining how training finds such a predictor, and controlling its population error, remain substantive learning problems.

The bridge from the associative-memory chapters to the learning theory is therefore

$$\boxed{\begin{array}{c}\text{Executing a supplied program is a computational result.} \\ \text{Learning an accurate feasible predictor is a further problem.} \\ \text{Identifying the generating computation is a stronger objective.}\end{array}}$$

These distinctions preserve the significance of the universality theorem without asking it to prove learnability, generalization, or structural identification.

Part I: Principles and Foundations

Interlude: Representation and Recovery

The chapters that follow are organized around a distinction that is easy to state and easy to blur: the distinction between *representation* and *recovery*.

A representation question asks what resources are required to express, evaluate, or approximate a function. A learning question asks whether a specified procedure can discover an accurate predictor from observations, with feasible data and computation.

These are different questions. The first concerns the existence and cost of a computation. The second concerns how a useful computation is found and why it predicts beyond the observed examples.

Compositional sparsity, the subject of the next chapter, addresses the first question. By itself, it does not answer the second.

Two meanings of recovery. There is a further distinction within the learning problem. *Predictive recovery* means finding a function that predicts accurately on the population of interest. For example, in noiseless regression with $Y = f(X)$ and $X \sim P_X$, the objective may be

$$\|\widehat{f} - f\|_{L^2(P_X)} \leq \varepsilon.$$

The learner need not reproduce the particular intermediate variables or factorization that generated the observations.

Structural recovery asks for more: identification of a specified hidden computation, its constituent functions, or its intermediate variables, up to explicitly controlled equivalences. This is an important objective when the aim includes understanding the generating mechanism. It requires a separate criterion of success and suitable identifiability assumptions.

Predictive accuracy does not, by itself, establish structural recovery. Conversely, a learned representation can differ from the generating one without being a predictive failure. The issue is not merely that a sparse decomposition may be nonunique. Predictive learning need not establish a correspondence with any particular generating decomposition at all.

The learned computation as a witness. Suppose training returns a deep network that predicts accurately and can be evaluated with feasible computational resources. At the relevant finite precision, the trained network is itself a computation admitting a bounded-fan-in implementation. It therefore supplies a compositional realization of the learned predictor:

accurate population prediction
$+$ feasible computational implementation
$\Longrightarrow$
a compositionally sparse predictor.

Here *feasible* is an asymptotic requirement, not merely a synonym for finite. Model description, evaluation time, encoding, and numerical precision must obey the stated resource bounds as the problem size and accuracy requirements change. Finding the predictor must also be efficient if an efficient-learning claim is being made.

Chapter H develops this witness argument. It removes the need for an additional structural-identification theorem after predictive success has been established. It does not explain, by itself, why that success occurs.

Representation still does not imply learnability. A network may be able to represent a computation without there being a known efficient procedure for discovering useful parameters from examples. Likewise, a compact target representation does not ensure that training selects a predictor with sufficiently small statistical complexity. Existence, optimization, and generalization remain separate questions.

Parity provides a simple illustration of the first gap. For $S \subseteq [d]$, the function

$$\chi_S(x) = \prod_{i \in S} x_i, \qquad x \in \{-1, +1\}^d,$$

can be computed by a tree of two-input operations. Yet, under the uniform product distribution, when $|S| > 1$,

$$\mathbb{E}[\chi_S(X) X_j] = 0 \qquad \text{for every } j \in S.$$

A learning mechanism relying on these linear correlations lacks that particular signal. This is not an impossibility statement about all learning procedures or all gradient-based parameterizations. It shows that an economical representation need not be visible to the statistics used by a specified learner.

The framework retains the labels C1, C2, and C3 to distinguish three useful roles. They should not be read as a universal sequence of necessary and sufficient conditions for learning.

C1 — Compositional sparsity. The target admits an economical realization, or an approximation at the required resolution, through a graph of bounded-arity constituents. For efficiently computable digital targets, a polynomial-size bounded-fan-in realization follows from the computation itself.

This removes a representation-size obstruction: a manageable computational object exists. It does not make the space of possible objects easy to search, nor guarantee that every proposed architecture contains an equally economical realization.

For real-valued approximation, favorable rates governed by local arity require additional regularity and quantitative stability of the constituents. A bounded-fan-in digital implementation alone does not establish a fixed smooth coarse-grained decomposition with uniformly controlled Sobolev norms.

Developed in Chapter 4.

C2 — Detectable footprints and genericity. The observed learning problem exposes useful statistical signals to the specified learner. In the mechanisms studied here, these signals often take the form of low-complexity correlations. Their usefulness depends jointly on the target, the input distribution, the current representation, and the probes accessible to the learning rule.

Genericity provides one proposed explanation for why such footprints persist or appear under perturbations. A nonzero footprint is not enough: its magnitude must permit detection with feasible resources, and the learning updates must be able to exploit it.

A *staircase* is a particular way of organizing these signals: learning one feature creates a coordinate that exposes another useful dependency. Establishing such a mechanism requires an argument about optimization, preservation of learned features, and accumulated error. C2 is not itself a theorem that the entire hidden computation is recovered, and a staircase is not a required trajectory of every successful learner.

Developed in Chapter 6.

C3 — Trace exposure. The observations supply intermediate information that can simplify the learning problem. With a faithful full trace, the parent values and outputs of constituent computations are directly available, allowing local supervised problems to be posed. Partial traces or useful surrogates expose less of the computation but may still assist learning.

Trace exposure changes what is observed. It can help even when end-to-end learning is possible; it is not only a fallback for failure of a genericity condition. Conversely, the availability of intermediate targets does not by itself guarantee that each local problem is learnable, adequately sampled, or stable under recomposition. An externally emitted explanation is not automatically a faithful trace of the generating computation.

The division of labor is therefore

C1:	a feasible representation exists;
C2:	useful learning signals are accessible;
C3:	intermediate information is supplied.

These roles can cooperate. A learner may exploit detectable signals, supplied traces, both, or another mechanism appropriate to its problem. The failure of one proposed mechanism does not imply that no feasible predictor can be learned.

The three questions for predictive learning. Chapter H organizes the synthesis around three questions:

Q1 : Does the chosen class contain a feasible accurate predictor?

Q2 : Can the specified training procedure find a suitable fit?

Q3 : Does the selected predictor generalize?

C1 informs the first question. Detectable signals and trace exposure offer routes toward the second. Statistical complexity and the selection induced by training are central to the third. These roles are related, but none should be silently substituted for another.

The later optimization arguments study overparameterized networks, normalization, stochastic dynamics, and norm- or product-norm-based regularization. Geometry may make low-loss solutions accessible; regularization may favor predictors for which statistical-complexity bounds are informative. Neither effect is automatic. Equilibrium preference for a low-loss region is not a finite-time convergence theorem, and a regularization penalty is not itself a guarantee of generalization.

The predictive argument is conditional: when training achieves adequate empirical fit within a suitably controlled hypothesis class, a valid generalization bound can imply small population error. Exact interpolation is an important regime, not a necessary definition of predictive success. If the resulting predictor is also computationally feasible, it supplies the compositional witness described above.

There is no further requirement that its internal variables match the teacher's. Structural-recovery theorems remain valuable because they establish a stronger conclusion and can provide constructive explanations of how learning succeeds.

From associative computation to the next chapter. The distinction also clarifies the role of the preceding chapter. The Associative Turing Machine establishes computational universality under its stated model: associative read, write, and local update suffice to implement classical computation. Universality concerns what can be executed when a program is supplied. It does not show how a useful program is discovered from data, and predictive learning need not discover the same program that generated those data.

The next chapter develops C1 on its own terms. It asks what structure follows from efficient computation and, under the required analytic assumptions, what that structure buys for approximation. The subsequent chapters examine learning signals, intermediate supervision, optimization geometry, and statistical selection. Chapter H brings these questions together without identifying a successful predictor with a privileged hidden decomposition.

The separation is deliberate:

First ask what can be represented.
Then ask how an accurate predictor can be learned.
Ask separately whether the generating structure is recovered.

CHAPTER 4

Efficient Computability and Compositional Sparsity

This chapter develops the first structural principle of the book: compositional sparsity.

Its starting point is classical computational complexity. Every polynomial-time digital computation can be unfolded into a polynomial-size computation graph whose elementary operations have bounded fan-in. In this precise sense,

$$\boxed{\textit{efficient digital computation} \Longrightarrow \textit{compositional sparsity}.}$$

For Boolean and finite-precision computation this implication is direct and unconditional.

For real-valued functions, additional assumptions concerning precision, continuity, conditioning, and constituent regularity are required before the computational statement can be converted into quantitative approximation rates.

The central point is about representation*, not recovery.*

Compositional sparsity explains why efficiently computable targets admit deep polynomial-size representations and why, for sufficiently regular constituents, approximation rates can depend on local arity rather than ambient dimension.

It does not explain how a specified learning procedure finds an accurate predictor from input–output data, or why that predictor generalizes. Identifying the particular hidden constituents that generated the data is a stronger, separate objective.

The later chapters study learning signals, intermediate supervision, optimization, and statistical selection. Chapter H shows how a feasible predictor that generalizes supplies a compositional realization without requiring recovery of a privileged generating graph.

4.1 Representation, Not Recovery

Two questions are easily conflated:

$$\boxed{\text{representation} \neq \text{recovery}.}$$

The present chapter concerns the first. As distinguished in the Interlude, *predictive recovery* seeks a function that is accurate on the population of interest; *structural recovery* additionally seeks

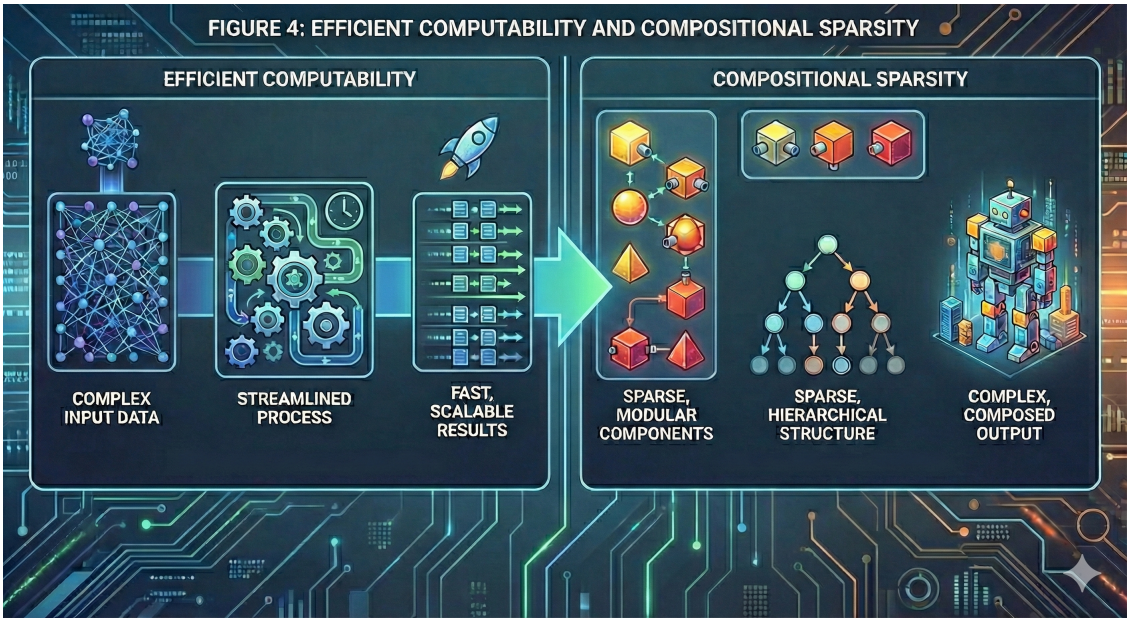


Figure 4.1: Efficient computation and compositional structure. A polynomial-time computation can be unfolded into a polynomial-size directed acyclic graph of bounded-fan-in elementary operations. Deep sparse networks provide a natural parametric realization of such computation graphs.

a specified generating computation, up to controlled equivalences. Neither conclusion follows merely from the existence of an economical representation.

Given a function f, we ask:

> What structural form must f possess if it can be evaluated using polynomial computational resources?

For discrete computation the answer is particularly simple.

A polynomial-time Turing machine performs only a polynomial number of elementary local operations.

Unrolling those operations in time produces a polynomial-size computation graph.

Because the machine possesses a finite transition rule, every elementary operation depends on only a bounded number of variables.

Hence

$$\boxed{\text{efficient digital computation} \Longrightarrow \text{polynomial-size bounded-fan-in computation.}}$$

We call this structural property *compositional sparsity*.

The terminology emphasizes two facts.

First, the target is compositional: its output is produced by composing intermediate functions.

Second, it is sparse: every individual constituent depends on only a small number of inputs.

Schematically,

$$f(x_1, \ldots, x_d) = \Phi_G(x_1, \ldots, x_d), \tag{4.1}$$

where G is a directed acyclic graph and each internal node has at most

$$k = O(1)$$

parents, independently of the ambient input dimension d.

The graph itself may be large, but for efficient computation its size is only polynomial in the input and precision parameters.

4.1.1 Why the distinction matters

A target can possess an extremely simple compositional representation and still be difficult to learn from examples.

Parity provides the canonical illustration.

For Boolean inputs,

$$\mathrm{Parity}(x_1, \ldots, x_d) = x_1 \oplus x_2 \oplus \cdots \oplus x_d \tag{4.2}$$

can be computed by a binary tree of two-input XOR gates.

Its local arity is

$$k = 2,$$

its size is linear in d, and its depth can be logarithmic under parallel composition.

Thus parity has an excellent C1 representation.

Yet under symmetric input distributions, correlations with many low-degree probes vanish.

A learner whose gradient signal is controlled by such probes may therefore receive little useful information about which variables should be combined.

This is a limitation of the specified probes, not an impossibility statement about all learning procedures or all network parameterizations. A different procedure might learn an accurate parity predictor without reconstructing a particular XOR tree.

The representation result supplies an economical computation. The learning question is whether the specified procedure finds a feasible predictor that works beyond the observed examples. Recovering the particular tree that generated those examples would be an additional structural conclusion.

4.1.2 The central structural implication

For Boolean computation,

polynomial-time Turing computation
$\Downarrow$
P-uniform polynomial-size bounded-fan-in circuit
$\Downarrow$
compositionally sparse representation.

For continuous functions the corresponding statement requires qualification:

efficient finite-precision computation
+ controlled precision requirements
+ regular low-arity constituents
+ stable composition
⇓
deep approximation with complexity controlled locally.

These two claims should remain distinct.

4.2 Definitions and Model Conventions

4.2.1 Boolean base

For discrete computation, internal nodes are gates drawn from a fixed finite basis such as

$$\{\mathrm{AND}, \mathrm{OR}, \mathrm{NOT}\},$$

with bounded fan-in and unrestricted fan-out.
Any standard finite universal basis could be used.
The particular choice changes only constant factors.

4.2.2 Real-valued base

For neural realizations, a typical constituent has the form

$$z \mapsto \sigma(w^\top z + b), \tag{4.3}$$

with

$$\sigma(u) = \max\{0, u\},$$

and with the number of arguments of z bounded by k.

All weights are represented at explicitly finite precision whenever a digital computational claim is being made.

4.2.3 Efficient computability: Boolean functions

Definition 1 (Efficient computability; Boolean)**.** *Let*

$$f_n : \{0,1\}^n \to \{0,1\}.$$

The family $\{f_n\}_{n\geq 1}$ is efficiently computable if there exists a deterministic Turing machine M and polynomial p such that, for every

$$x \in \{0,1\}^n,$$

the machine outputs $f_n(x)$ in at most

$$p(n)$$

steps.

Equivalently, the family admits a P-uniform polynomial-size Boolean circuit family.

Uniformity matters.

Without it, an arbitrary circuit family can encode non-computable information directly in its wiring.

4.2.4 Efficient computation of real-valued functions

For real-valued functions, exact real arithmetic is not an appropriate resource model for the present purpose.

We instead count bits of requested precision.

Let

$$\varepsilon = 2^{-r}.$$

Definition 2 (Efficient finite-precision computation)**.** *Let*

$$f : [0,1]^d \to \mathbb{R}^m.$$

We say that f is efficiently computable in the bit model if an algorithm can, given access to a sufficiently accurate dyadic representation of x and a precision request r, produce y satisfying

$$\|y - f(x)\| \leq 2^{-r} \tag{4.4}$$

using time and output description length polynomial in the relevant input description length, d, and r.

For continuous-domain statements, we additionally require that the number of reliable input bits required to produce r reliable output bits grow at most polynomially in these parameters.

The last requirement can be formulated using a polynomially controlled modulus of continuity.

These precision assumptions are essential.

A function may be formally computable while requiring exponentially many input bits to determine a modest number of reliable output bits.

4.2.5 Computation DAGs

Definition 3 (Computation DAG)**.** *A computation DAG*

$$G = (V, E)$$

contains input nodes, internal nodes, and output nodes.

Every internal node v has a parent set

$$\mathrm{pa}(v)$$

satisfying

$$|\,\mathrm{pa}(v)| \leq k,$$

and computes a constituent function

$$\phi_v.$$

The size of the graph is

$$s(G) = |\mathsf{Int}|,$$

and the depth is the length

$$L(G)$$

of its longest input–output path.

4.2.6 Compositional sparsity

Definition 4 (Exact compositional sparsity)**.** *Fix a node class $\mathcal{F}_{\leq k}$.*

A function f is exactly (k, s, L)-compositionally sparse if there exists a computation DAG G of fan-in at most k, size at most s, and depth at most L, with

$$\phi_v \in \mathcal{F}_{\leq k}$$

for every internal node, such that

$$f = \Phi_G.$$

Definition 5 (Approximate compositional sparsity)**.** *For norm $\|\cdot\|_{\mathcal{X}}$, a function f is (k, s, L, ε)-compositionally sparse if there exists a DAG realization Φ_G satisfying*

$$\|f - \Phi_G\|_{\mathcal{X}} \leq \varepsilon.$$

The important structural parameter is the local arity

$$k.$$

A function may depend globally on millions of input coordinates while each elementary constituent depends on only two or three.

4.3 Efficient Computability Implies Compositional Sparsity

4.3.1 The Boolean case

We begin with the cleanest statement.

Theorem 2 (Polynomial-time computation implies bounded-fan-in circuits)**.** *Let*

$$f_n : \{0,1\}^n \to \{0,1\}$$

be computable by a deterministic Turing machine in time

$$T(n) = \mathrm{poly}(n).$$

Then there exists a P-uniform Boolean circuit family over a fixed bounded-fan-in basis whose depth is

$$O(T(n))$$

and whose size is

$$\mathrm{poly}(T(n)).$$

Consequently every function family in **P** *admits a polynomial-size bounded-fan-in compositional representation.*

Proof sketch. Consider the space–time evolution of the Turing machine.

At every time step, each tape cell contains a symbol from a finite alphabet, the head occupies one location, and the machine occupies one state from a finite control set.

The state of a cell at time $t+1$ depends only on a constant-size neighborhood of the machine configuration at time t.

That local update can be implemented by a constant-size Boolean circuit.

Unrolling these local updates for

$$T(n)$$

time steps produces a layered bounded-fan-in circuit.

The number of simulated tape cells required is polynomially bounded by the running time, so the total number of gates is polynomial in $T(n)$.

Finally, the construction is uniform: from the description of the Turing machine and the input length one can generate the circuit wiring in polynomial time.

□

The precise polynomial overhead is secondary here.

The central structural fact is

$$\boxed{\text{polynomial-time computation} \Longrightarrow \text{polynomially many bounded-arity constituents.}}$$

This is the basic content of C1.

4.3.2 Why bounded fan-in is natural

A Turing machine itself is local.

At any elementary step it consults only a finite amount of state and a bounded region of memory.

Ordinary digital hardware has the same organization.

Complex computation is not performed by one giant gate with millions of inputs.

It is built by composing large numbers of elementary local operations.

Thus bounded-fan-in composition is not a special property invented for deep learning.

It is the ordinary architecture of digital computation.

Deep networks provide one convenient parametric language for representing such graphs.

4.4 Exact Deep-Network Realization on Discrete Inputs

The circuit theorem establishes compositional structure.

To connect this directly to neural networks, each discrete gate can be replaced by a constant-size neural subnetwork.

4.4.1 Boolean gates as ReLU subnetworks

Lemma 1 (ReLU realization of bounded-arity Boolean gates)**.** *Every Boolean gate from a fixed bounded-arity basis can be represented exactly on Boolean inputs by a constant-size ReLU network.*

For example, for

$$x, y \in \{0, 1\},$$

$$\mathrm{AND}(x, y) = \mathrm{ReLU}(x + y - 1),$$

while

$$\mathrm{NOT}(x) = 1 - x.$$

Likewise,

$$\mathrm{OR}(x, y) = \mathrm{ReLU}(x + y) - \mathrm{ReLU}(x + y - 1).$$

Because the gate basis is finite, every required gate can be compiled into a constant-size ReLU module.

Theorem 3 (Exact ReLU realization on Boolean domains)**.** *If a Boolean function admits a bounded-fan-in circuit of size s and depth L, then there exists a ReLU network of size*

$$O(s)$$

and depth

$$O(L)$$

that computes the function exactly on Boolean inputs.

Proof. Replace each Boolean gate by its constant-size ReLU realization.
Every replacement changes network size and depth by only a constant factor.

□

Hence

$$\boxed{\mathbf{P} \subseteq \{\text{polynomial-size deep ReLU realizations on Boolean inputs}\}.}$$

4.4.2 Finite-precision grids

The same compilation idea applies to digital functions on finite grids.
Let

$$\mathcal{G}_b = \left\{0, 2^{-b}, 2 \cdot 2^{-b}, \ldots, 1\right\}^d.$$

Each point has an $O(db)$-bit encoding.

Theorem 4 (Finite-grid realization)**.** *Suppose*

$$f : \mathcal{G}_b \to \mathbb{R}^m$$

can be evaluated to accuracy ε by a Turing machine in time

$$\operatorname{poly}\left(d, b, \log \frac{1}{\varepsilon}\right).$$

When the input is supplied through its binary representation, there exists a bounded-fan-in ReLU network of size and depth polynomial in

$$d, \qquad b, \qquad \log \frac{1}{\varepsilon}$$

whose output approximates f to error ε on $\mathcal{G}_b$.
The weights require only polynomially many bits.

The binary-encoding qualification is important.

If grid coordinates are supplied directly as real scalars, a network must also implement the corresponding finite-precision decoding operations.

4.5 Real-Valued Functions and Finite Precision

The discrete theorem is exact.

The continuous case requires additional care.

Efficient finite-precision evaluation gives a polynomial-size digital computation at each requested accuracy.

It does not, by itself, provide a quantitative approximation theorem for a particular neural architecture.

There are therefore two logically distinct statements.

1. Efficient finite-precision computation implies a polynomial-size bounded-fan-in *digital* representation at each requested precision.

2. Dimension-robust neural approximation additionally requires regularity and stability of the constituent functions.

The second statement is the domain of approximation theory.

These statements concern different levels of representation. Bounded fan-in at the level of bits or elementary operations does not by itself supply a fixed smooth coarse-grained DAG, uniformly controlled Sobolev norms for its constituents, or useful semantic modules. Such properties require the additional assumptions stated in the approximation theory.

The same distinction applies to a trained network. Its feasible digital implementation is a computationally sparse realization of its predictor. That fact alone neither identifies a generating decomposition nor establishes the regularity and stability needed to apply the continuous approximation theorem below.

4.6 Compositional Approximation and the Curse of Dimensionality

4.6.1 Generic high-dimensional functions

Consider a generic function

$$f : [0,1]^d \to \mathbb{R}$$

in an r-smooth class.

For broad smoothness classes, the number of parameters required for approximation error ε scales schematically as

$$N_{\text{generic}}(\varepsilon) \asymp \varepsilon^{-d/r}, \tag{4.5}$$

up to constants and logarithmic factors.

The exponent contains the ambient dimension d.

This is the classical curse of dimensionality.

4.6.2 Low-arity constituents

Suppose instead that

$$f = \Phi_G$$

where G contains s internal nodes and each constituent depends on at most

$$k \ll d$$

arguments.

Assume each constituent belongs to an r-smooth class for which local approximation to error δ requires

$$N_{\text{local}}(\delta) = O\left(\delta^{-k/r}\right) \tag{4.6}$$

parameters.

If each constituent is approximated separately, the total number of parameters scales schematically as

$$s\,\delta^{-k/r}.$$

The remaining issue is how local approximation errors propagate through the graph.

4.6.3 A stability factor

Let each constituent be Lipschitz on the relevant domain.

For a path

$$v_1 \to v_2 \to \cdots \to v_q,$$

an error introduced at v_j is amplified by the product of downstream Lipschitz constants.

Define a graph stability factor schematically by

$$A_G = \max_{\text{outputs}} \sum_{\text{upstream paths } v} \prod_{\text{downstream on path}} L_v. \tag{4.7}$$

If every constituent is approximated to local accuracy δ, then a generic bound has the form

$$\|f - \widetilde{f}\| \leq A_G \, \delta. \tag{4.8}$$

Thus it suffices to choose

$$\delta \asymp \frac{\varepsilon}{A_G}.$$

This yields

$$N(\varepsilon) = O\left(s \left(\frac{A_G}{\varepsilon} \right)^{k/r} \right). \tag{4.9}$$

If

$$A_G = \mathrm{poly}(s, L),$$

then the exponent governing approximation accuracy depends on k, not d.

Theorem 5 (Compositional approximation; schematic form)**.** *Let*

$$f = \Phi_G$$

be represented by a DAG of size s, depth L, and local arity at most k.
Suppose every constituent belongs to an r-smooth class admitting k-variate ReLU approximation with complexity

$$O(\delta^{-k/r})$$

and suppose composition is quantitatively stable so that

$$A_G = \mathrm{poly}(s, L).$$

Then f admits a deep ReLU approximation of error at most ε with complexity

$$N(\varepsilon) = \mathrm{poly}(s, L) \, \varepsilon^{-k/r}, \tag{4.10}$$

up to regularity-dependent constants and logarithmic factors.
In particular, the approximation exponent depends on local arity k rather than ambient dimension d.

The precise rate depends on the norm, activation, smoothness assumptions, graph geometry, and stability constants.
The robust message is

bounded local arity replaces ambient dimension by constituent dimension in approximation rates.

4.6.4 A simple chain

For a chain of depth D, common Lipschitz constant L, and local error δ,

$$e_D \leq \delta \sum_{j=0}^{D-1} L^j.$$

Hence

$$e_D \leq \delta \frac{L^D - 1}{L - 1}$$

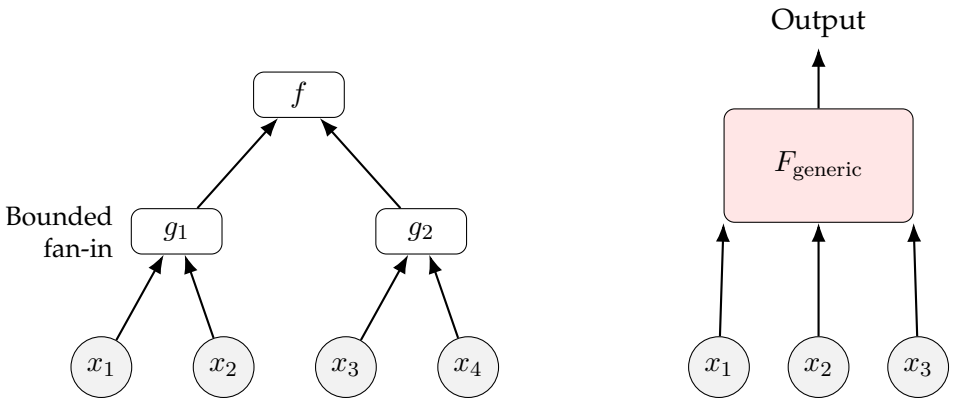
for $L \neq 1$.
When

$$L \leq 1,$$

we have the simpler bound

$$e_D \leq D\delta.$$

Thus depth is beneficial only together with adequate conditioning.
Low arity alone does not prevent local errors from being amplified.



(a) Compositionally sparse

(b) Generic dense map

Figure 4.2: Compositional sparsity versus generic high-dimensional dependence. In (a), each constituent depends on only a few variables even though the global target depends on many inputs. For broad generic smooth classes, the approximation exponent instead depends on ambient dimension.

4.7 Two Small Consequences of Local Arity

The local-arity viewpoint appears in several elementary forms.

For a Boolean constituent with k binary inputs, an exact truth table requires only

$$2^k$$

entries.

Thus N_{distinct} distinct Boolean constituents can be stored using

$$O\left(N_{\text{distinct}} 2^k\right)$$

entries.

Similarly, if a continuous k-variate constituent is quantized to R levels per coordinate, an explicit local table requires

$$R^k$$

entries.

Again the exponent is local:

$$k,$$

not global:

$$d.$$

A related geometric observation is that deep networks can produce very complicated global functions from locally simple operations.

A ReLU network is piecewise affine, yet its number of linear regions can grow extremely rapidly with depth.

Thus

$$\boxed{\text{locally simple computation} \not\Longrightarrow \text{globally simple function.}}$$

This is exactly what compositional sparsity allows.

4.8 What C1 Buys

C1 should be understood as a representation principle.

It has several important consequences, but each has a specific scope.

4.8.1 A feasible representation

If a discrete target is efficiently computable, then it possesses a polynomial-size bounded-fan-in computation graph.

Thus the object being represented does not require exponentially many independent computational components.

This is the first meaning of saying that compositional sparsity makes the problem feasible to pose.

The existence statement is relative to the allowed computational primitives and precision convention. For a particular learning architecture, one must still show that an accurate predictor is available within its admitted resources. If a norm-based generalization argument is intended, an adequate complexity budget is an additional requirement: a small circuit is not automatically a sufficiently low-norm predictor in every chosen parameterization.

4.8.2 Escape from ambient-dimensional approximation

For regular real-valued constituents, one approximates many low-dimensional functions rather than one arbitrary high-dimensional function.

Schematically,

$$\varepsilon^{-d/r} \quad \longrightarrow \quad s\varepsilon^{-k/r}, \qquad k \ll d,$$

subject to stability of the composition.

This is the approximation-theoretic content of the principle.

4.8.3 Optimization becomes feasible to pose

If representing the target itself required exponentially many parameters, then no polynomial-resource optimizer could even manipulate the relevant parameterization explicitly.

A polynomial-size compositional representation removes that obstruction.

But this says only

$$\boxed{\text{a manageable search object exists.}}$$

It does not imply

$$\boxed{\text{gradient descent knows where it is.}}$$

That second question belongs to learning. For prediction, it asks whether the training procedure can find a sufficiently accurate feasible predictor, not necessarily the representation that generated the data.

4.8.4 Favorable statistical complexity becomes possible

A sparse compositional class can possess much lower effective complexity than a class of unrestricted d-variate functions.

Generalization bounds based on

$$\text{description length}, \quad \text{sparsity}, \quad \text{norm}, \quad \text{covering entropy}, \quad \text{path complexity}$$

may therefore depend on the number and complexity of active constituents rather than the nominal size of an overparameterized architecture.

But target sparsity alone does not guarantee this outcome.

Training must select a realization with both suitable empirical fit and a statistical-complexity bound that is informative for the available sample. The realization need not be the one used to establish C1. Norm- or product-norm-based selection can support this route to prediction without identifying the generating constituents; Chapter H states the corresponding conditional argument.

4.9 What Does Not Follow from C1

Because C1 is a strong structural statement, it is easy to attribute more to it than it proves.

Several distinctions are essential.

4.9.1 C1 does not imply module reuse

A sparse DAG can contain

$$s$$

different low-arity functions, each appearing only once.
Therefore

$$\boxed{\text{bounded fan-in} \not\Longrightarrow \text{repeated modules.}}$$

Reuse requires an additional property:

$$\boxed{\text{compositional sparsity} + \text{repeated constituents} \Longrightarrow \text{modularity.}}$$

Convolutional weight sharing is an architectural example.
The CRM chapter later develops reuse as a separate selection and compression principle.

4.9.2 C1 does not imply transfer

Two tasks may both be compositionally sparse while sharing no useful constituents.
Transfer becomes possible when their representations overlap:

$$\phi_v^{(A)} \approx \phi_w^{(B)}.$$

Thus

$$\boxed{\text{sparsity} + \text{cross-task constituent sharing} \Longrightarrow \text{potential transfer.}}$$

C1 alone supplies only the first term.

4.9.3 C1 does not imply identifiability

A function can admit many sparse decompositions.
For example,

$$f = g \circ h$$

can often be rewritten under an invertible intermediate transformation

$$z' = Az$$

as

$$f = (g \circ A^{-1}) \circ (A \circ h).$$

Both factorizations compute the same function.
Therefore

$$\boxed{\text{existence of a sparse DAG} \not\Longrightarrow \text{unique latent coordinates.}}$$

Semantic interpretation or causal identification requires additional assumptions.

For predictive learning, this nonuniqueness is not itself an obstruction. The learner may use a different feasible decomposition as long as its input–output predictions are accurate on the population of interest. Structural recovery asks for an additional correspondence with a specified generating decomposition, under an explicitly controlled notion of equivalence.

4.9.4 C1 does not imply visible sparsity of trained weights

Suppose a target admits a sparse computational realization.

A change of basis can transform that realization into a dense parameterization.

Likewise, an overparameterized network may implement the same function in many coordinate systems.

Therefore

$$\boxed{\text{sparse realization exists} \not\Longrightarrow \text{native weight matrices of the trained model are sparse.}}$$

In particular, C1 does not predict that a Transformer MLP must become block sparse or column sparse in its learned coordinates.

Conversely, dense native weight matrices do not contradict the bounded-fan-in implementation of a feasible learned predictor. The witness argument in Chapter H concerns the computation implemented by that predictor, not a requirement that its displayed weight matrices reveal the generating graph.

To infer visible weight sparsity one needs additional assumptions about

$$\text{optimization bias}, \quad \text{regularization}, \quad \text{identifiability}, \quad \text{coordinate alignment}.$$

4.9.5 C1 does not imply easy global search

Bounded local structure does not by itself eliminate combinatorial difficulty.

A canonical example is 3-SAT.

Every clause depends on only three variables, so the verification map is highly local, yet finding a satisfying assignment is NP-complete.

Thus

$$\boxed{\text{efficient local evaluation} \not\Longrightarrow \text{efficient inversion, optimization, or search.}}$$

This distinction is developed separately in the appendix on distributional tractability.

4.9.6 C1 does not imply generalization

The existence of a low-complexity realization does not guarantee that training selects it.

An overparameterized model may interpolate using a function of much higher effective complexity.

Thus C1 makes favorable generalization possible but does not establish it.

There is no contradiction with the fact that a computationally feasible memorizing network also has a bounded-fan-in implementation. Computational sparsity and statistical complexity measure different properties. A polynomial-size implementation need not give a useful generalization bound at the available sample size.

The conditional route is instead to establish adequate empirical fit and an informative complexity bound for the selected predictor. If these imply small population error, and the predictor is feasible to evaluate, its implementation supplies the compositional witness. This route does not require recovery of the generating graph.

4.9.7 Most importantly: C1 does not imply learnability

The learner observes examples such as

$$(x_i, f(x_i)).$$

It is not given the generating graph, its constituent functions, the intermediate variables, or the parent sets.

Their absence does not make reconstruction of that particular computation a requirement of every learning problem. Two objectives must be kept separate.

For *predictive recovery*, the learner seeks a feasible function $\widehat{f}$ such that, for example,

$$\boxed{\|\widehat{f} - f\|_{L^2(P_X)} \leq \varepsilon,}$$

where P_X is the input distribution of interest. The task is

$$\boxed{\begin{array}{c}\text{input–output observations}\\ \longrightarrow \\ \text{an accurate, computationally feasible predictor.}\end{array}}$$

For *structural recovery*, the objective is stronger:

$$\boxed{\begin{array}{c}\text{input–output observations}\\ \longrightarrow \\ \text{the generating computation, up to controlled equivalences.}\end{array}}$$

This requires a separate criterion of success and suitable identifiability assumptions. Predictive accuracy alone does not establish it, and prediction need not fail merely because the learned internal coordinates differ from the generating ones.

C1 guarantees the existence of an economical realization in the stated computational model. It does not prove that the specified learner finds a sufficiently good empirical fit, that its selected predictor has adequate statistical-complexity control, or that its predictions generalize.

The witness argument in Chapter H starts after those predictive requirements have been established. It applies the computation-to-circuit result to the learned predictor itself. It removes an unnecessary structural-identification requirement; it does not turn representation into a learning algorithm.

4.10 Scope: Artificial and Biological Computation

The implication

$$\boxed{\text{efficient computation} \longrightarrow \text{polynomial-size bounded-fan-in computation}}$$

has its clearest meaning in the standard digital bit model.

Its interpretation for biological computation requires more care.

4.10.1 Computability versus efficient computability

A function can be Turing computable while requiring resources too large to be useful.

Thus

$$\boxed{\text{Turing computable} \not\Longrightarrow \text{efficiently Turing computable.}}$$

For a real-valued function, efficiency requires not merely that an answer eventually be produced, but that the time, memory, and precision required for an ε-approximation grow polynomially in the appropriate description lengths and in

$$\log \frac{1}{\varepsilon}.$$

4.10.2 Artificial systems

Every actual finite computation performed on ordinary digital hardware is implemented by finite local operations.

But this does not automatically make an asymptotic algorithm polynomial time.

An exponential brute-force procedure is also digital.

The correct implication is conditional:

$$\boxed{\text{polynomial-resource digital computation} \Longrightarrow \text{polynomial-size bounded-fan-in realization.}}$$

4.10.3 Biological systems

One may accept some version of the physical Church–Turing thesis and still not obtain an efficiency theorem.

Digital simulability in principle says nothing about whether a faithful simulation requires

$$\text{polynomial time}, \qquad \text{polynomial memory}, \qquad \text{polynomial precision}.$$

Therefore

$$\boxed{\text{physical simulability} \not\Longrightarrow \text{efficient digital simulability.}}$$

4.10.4 Microscopic simulation versus functional simulation

The distinction is particularly important for biological systems.

Suppose a nervous system has microscopic physical state

$$s_t$$

but behavior depends primarily on a lower-dimensional effective state

$$z_t = \Phi(s_t).$$

Simulating every microscopic degree of freedom may be extremely expensive, while reproducing the effective map

$$z_t \mapsto z_{t+1}$$

may be simple.

For artificial intelligence, the relevant question is usually not

> Can every microscopic degree of freedom of the brain be simulated?

but

> Can the behaviorally relevant computation be reproduced efficiently?

4.10.5 Local physical dynamics

Many physical systems are governed by local interactions.

If the relevant latent state factorizes as

$$z_t = (z_t^{(1)}, \dots, z_t^{(m)})$$

and satisfies

$$z_{t+1}^{(i)} = F_i\left(z_t^{\mathrm{pa}(i)}\right), \qquad |\,\mathrm{pa}(i)| = O(1),$$

then the effective dynamics are already compositionally sparse.

This provides one reason for optimism that behaviorally relevant biological computation may admit compact digital descriptions.

But it is a hypothesis about the effective variables, not a theorem about all biological processes.

4.10.6 No connection to consciousness or evolutionary age

Computational complexity should not be identified with categories such as

$$\text{conscious}, \quad \text{unconscious}, \quad \text{emotional}, \quad \text{evolutionarily old}.$$

A homeostatic controller may be computationally simple.

A symbolic reasoning problem may be computationally intractable.

Thus

> phenomenological category and evolutionary age do not determine complexity class.

4.10.7 The biological boundary

The relevant open question is therefore

> Are behaviorally relevant biological computations efficiently approximable in the finite-precision digital bit model?

If the answer is yes for a particular map, C1 applies.

If reproducing that map requires superpolynomial time, memory, or precision, the theorem does not supply an efficient compositional realization.

This marks a natural boundary of the framework.

4.11 The Status of the Principle

The classical theorem and its interpretation should be distinguished.

The complexity-theoretic statement

$$\boxed{\mathbf{P} \subseteq \{\text{P-uniform polynomial-size bounded-fan-in circuits}\}}$$

is standard.

The *Compositional Sparsity Principle* is the interpretation of this fact for learning:

> Targets produced by feasible digital computations belong to a highly structured subset of all high-dimensional functions: they possess polynomial-size decompositions into low-arity constituents.

This changes the natural reference class for deep learning.

The comparison should not be

$$\text{deep network} \quad \text{versus} \quad \text{arbitrary } f : \mathbb{R}^d \to \mathbb{R}.$$

Almost all such mathematical functions are irrelevant to feasible computation.

The more meaningful comparison is

$$\boxed{\text{learning system} \quad \text{versus} \quad \text{targets generated by feasible computations.}}$$

4.12 Conclusion: From Representation to Predictive Learning

The central result of this chapter is simple. A polynomial-time digital computation is built from only polynomially many local elementary operations. Unrolling those operations gives a polynomial-size bounded-fan-in DAG:

$$\boxed{\text{efficient digital computation} \Longrightarrow \text{compositional sparsity.}}$$

For Boolean and finite-precision discrete computation, this implication is direct in the stated computational model.

For continuous approximation, additional constituent regularity and quantitative stability are required. When the local approximation and composition hypotheses of Theorem 5 hold, local arity replaces ambient dimension in the accuracy exponent:

$$\boxed{\varepsilon^{-d/r} \quad \longrightarrow \quad \mathrm{poly}(s, L)\, \varepsilon^{-k/r}, \qquad k \ll d.}$$

The graph size, depth, regularity constants, and stability factor still matter. The conclusion is not that every computationally sparse implementation automatically yields a uniformly regular coarse-grained representation.

C1 removes a representation-size obstruction, not every obstruction to learning. As the preceding sections explain, it does not by itself imply module reuse, transfer, unique latent coordinates, visibly sparse trained weights, easy search, or generalization. In particular,

$$\boxed{\text{compositional sparsity does not imply learnability.}}$$

The learning objective need not be the generating program. Given only input–output observations, the primary predictive objective is to find a feasible function that is accurate on the population of interest. The learner need not reconstruct a privileged generating graph or establish a correspondence between its internal variables and those of the teacher.

This changes the transition from representation to learning. The questions are:

Q1 : Does the chosen class contain a feasible accurate predictor?

Q2 : Can the training procedure find a suitable empirical fit?

Q3 : Does the selected predictor generalize?

C1 informs Q1. For a chosen architecture, its expressive resources and, when needed, its norm-complexity budget must still be checked. Q2 and Q3 require learning and statistical arguments, not just circuit compilation.

The same representation result applies after learning. Suppose training returns a predictor $\widehat{f}$ satisfying

$$\|\widehat{f} - f\|_{L^2(P_X)} \leq \varepsilon.$$

Suppose also that its finite description and numerical evaluation obey the stated polynomial resource bounds. At evaluation precision m, the same compilation argument yields a bounded-fan-in digital realization whose decoded output $\widetilde{f}_m$ satisfies

$$\|\widetilde{f}_m - \widehat{f}\|_\infty \leq 2^{-m}.$$

The triangle inequality then gives

$$\boxed{\|\widetilde{f}_m - f\|_{L^2(P_X)} \leq \varepsilon + 2^{-m}.}$$

The learned computation is therefore a compositional witness at the stated predictive and numerical accuracy. Its graph may differ from the graph used to establish the target's representability.

This conclusion is conditional on predictive success and feasible evaluation. It does not prove that training is efficient, that regularization achieves an informative complexity bound, or that the internal variables identify the generating mechanism. Those are separate claims. Chapter H develops the conditional synthesis.

The route through the following chapters. The next chapter examines how compositional representations make optimization feasible to pose and how intermediate supervision can simplify the learning problem. Chapter 6 studies detectable footprints and their relation to accessible learning updates. The chapter on overparameterization studies optimization geometry. Chapter H then separates empirical fit, statistical selection, and the final witness argument.

The roles of C1, C2, and C3 remain distinct:

C1:	a feasible representation exists;
C2:	useful learning signals are accessible;
C3:	intermediate information is supplied.

Genericity offers candidate explanations for useful signals. A staircase is one possible mechanism for exploiting them through successive feature construction. Trace exposure can simplify learning

even when end-to-end learning is possible. None of these labels is, by itself, a complete learning theorem, and no particular mechanism is an additional premise of the witness implication once predictive success is established.

Structural-recovery results remain valuable: they establish a stronger objective and may explain constructively how learning succeeds. They are not a universal requirement for accurate prediction.

The resulting separation is

C1 asks which computations admit an economical representation.

Learning asks how an accurate feasible predictor is found.

Structural recovery asks whether the generating computation is identified as well.

CHAPTER 5

Compositionality: Why Optimization Is Feasible to Pose

Does compositional structure make optimization easier? The answer has two parts that should be kept separate.

First, an economical compositional representation makes the optimization problem feasible to pose*. A generic target in a broad d-dimensional smoothness class may require approximation complexity exponential in d, leaving no manageable explicit parameterization. A target with a polynomial-size bounded-arity computation removes this representation-size obstruction. For continuous approximation, the regularity and stability assumptions of Chapter 4 remain essential.*

Second, a manageable parameterization need not be easy to search. Two further obstructions can remain. The first is geometric*: gradients may vanish, explode, or be badly conditioned as they propagate through deep compositions. The second is* informational*: the observations and the statistics accessible to the chosen learning rule may fail to expose a detectable signal for improving prediction.*

This chapter separates these roles. It also studies a positive "grey-box" regime in which intermediate variables, or useful surrogates for them, are exposed. Faithful intermediate supervision permits local supervised problems to be posed. This is the trace-exposure role C3; it can help even when end-to-end learning is possible and is not only a fallback for a failed genericity condition.

The objective must also be explicit. Predictive learning seeks an accurate function on the population of interest. Structural recovery additionally seeks the generating constituents or intermediate variables. The latter is a stronger task. As Chapter H explains, a computationally feasible predictor that generalizes is already a compositional witness, even if its internal coordinates differ from the teacher's. The remaining challenge is to explain how the specified training procedure achieves that predictive success.

5.1 From Representation to Optimization

Chapter 4 established the first principle of the book:

$$\boxed{\text{efficient digital computation} \Longrightarrow \text{a compositionally sparse realization.}}$$

For real-valued functions, this computational conclusion concerns the specified encoding and numerical precision. The corresponding approximation-theoretic statement additionally requires controlled local regularity and stable composition.

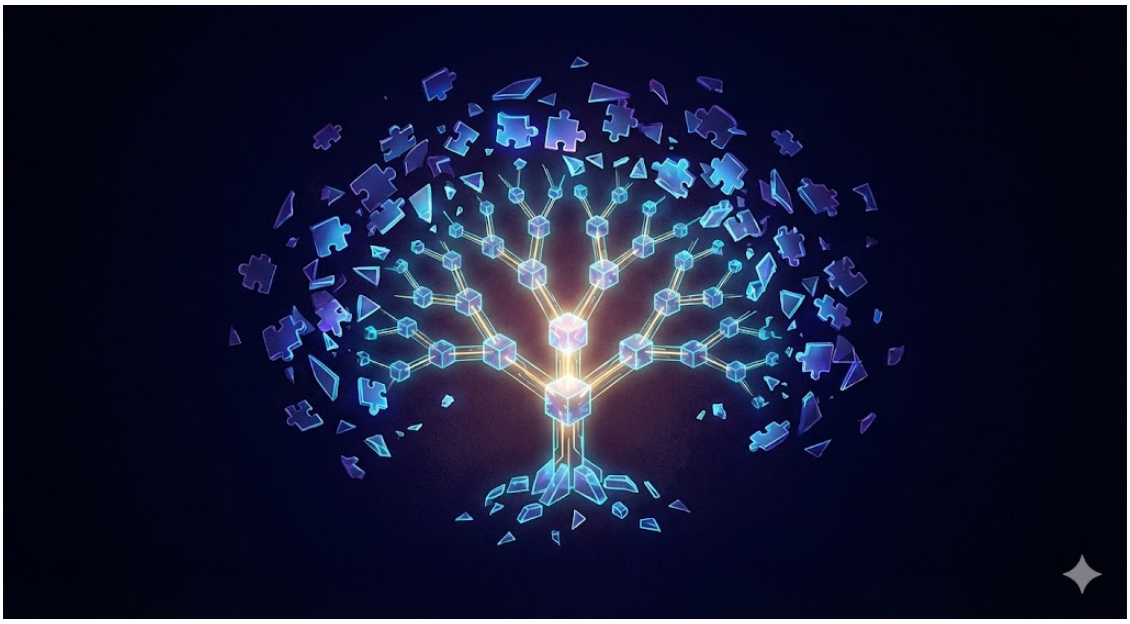


Figure 5.1: Compositional structure creates global complexity from simple local operations. An economical representation does not, by itself, guarantee that end-to-end optimization finds an accurate predictor. Identifying the particular generating constituents is a stronger, separate objective.

The immediate consequence for optimization is a statement about the size of an object to be represented. If every accurate realization in a chosen representation language required exponentially many independent parameters, a polynomial-resource procedure could not even manipulate such a realization explicitly. An economical compositional description removes that obstruction.

It does not turn a polynomial-dimensional parameter space into a small set of candidates. Many parameter vectors and many possible graphs can still have descriptions of polynomial length.

Two questions therefore remain distinct.

1. **Can the optimization problem be posed using feasible resources?** There must be an accurate predictor in the admitted architecture, with controlled description length, numerical precision, and evaluation cost.

2. **Can the specified procedure find a suitable predictor?** Even a compactly parameterized problem can have poor conditioning, unhelpful local geometry, weak statistical signals, or inadequate data. Population accuracy requires statistical control in addition to a good empirical fit.

Thus

$$\boxed{\text{economical parameterization} \neq \text{easy search.}}$$

In the terminology of the revised Interlude, C1 concerns feasible representation; C2 concerns accessible learning signal; C3 supplies intermediate information. These roles can cooperate. They are not a universal sequence of necessary and sufficient conditions for learning.

For noiseless regression, predictive recovery can be expressed as

$$\|\widehat{f} - f\|_{L^2(P_X)} \leq \varepsilon.$$

Structural recovery asks in addition for a specified correspondence between learned and generating variables, under appropriate identifiability and equivalence conditions. We do not impose that additional requirement on every successful predictor.

5.2 Why C1 Makes the Search Space Feasible

The parameter-count argument was developed in Chapter 4. We recall only the part needed here. Consider a real-valued target on a fixed compact input domain in $\mathbb{R}^d$, in a smoothness class of order s.

5.2.1 Generic high-dimensional approximation

For broad smoothness classes, worst-case approximation to error ε may require complexity of the schematic form

$$N_{\text{generic}}(\varepsilon) \asymp \varepsilon^{-d/s},$$

up to class-dependent constants and logarithmic factors. The exponent is the ambient dimension d. This is the representation-size obstruction associated with the curse of dimensionality, not a lower bound for every individual smooth target.

5.2.2 Compositionally sparse approximation

Suppose instead that the target consists of m low-arity constituents, each depending on at most k variables, with $k \ll d$. Under the local approximation assumptions of Chapter 4, and writing A_G for its graph stability factor, the corresponding complexity is schematically

$$N_{\text{comp}}(\varepsilon) = O\left(m\left(\frac{A_G}{\varepsilon}\right)^{k/s}\right),$$

up to regularity-dependent constants and logarithmic factors. The shorter expression $O(m\varepsilon^{-k/s})$ suppresses this stability dependence; it should not conceal an exponentially large amplification factor.

When the number of constituents and the relevant stability and regularity bounds are polynomially controlled, and $k = O(1)$, the approximation exponent depends on local arity rather than ambient dimension:

$$\boxed{\varepsilon^{-d/s} \quad \longrightarrow \quad m\left(\frac{A_G}{\varepsilon}\right)^{k/s}.}$$

This supplies an economical realization to be sought. It does not bound the number of possible realizations or the effort needed to select one. Nor is polynomial dependence on ε^{-1} the same as polynomial-time evaluation in the requested bit precision $\log(1/\varepsilon)$.

The representation benefit is therefore

$$\boxed{\text{C1 removes a representation-size obstruction.}}$$

This benefit is conditional on the resource and analytic assumptions just stated. A separate architecture-specific argument is needed if the learner is restricted to a particular network class or norm budget.

5.2.3 What this does not imply

An economical parameterization can still have a nonconvex loss, poorly conditioned gradients, weak accessible signals, symmetries, and flat directions. The graph structure may itself be unknown. Thus C1 supplies a feasible comparison object; it does not establish that gradient descent, SGD, or another specified procedure will find a good predictor.

It also does not establish that the selected predictor has small enough statistical complexity for the available sample. Computational feasibility, optimization cost, and generalization are different resource questions.

5.3 Two Obstructions to End-to-End Learning

Suppose that the chosen architecture admits a feasible accurate predictor. Training end to end from input–output pairs takes the form

$$\min_{W_1,\dots,W_L} \sum_i \ell(f_L(\cdots f_2(f_1(x_i;W_1);W_2)\cdots;W_L),y_i)\,.$$

The functions $f_\ell(\cdot;W_\ell)$ describe the learner's layers. They need not correspond to the teacher's constituents. Two conceptually different obstructions can arise even for the predictive objective.

5.3.1 The geometric obstruction

In a deep composition, the gradient with respect to an early parameter contains products of Jacobians from later stages. Schematically,

$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{\partial \mathcal{L}}{\partial f_L}\frac{\partial f_L}{\partial f_{L-1}}\cdots\frac{\partial f_2}{\partial f_1}\frac{\partial f_1}{\partial W_1}.$$

The singular values of these products can become very small or very large. This produces vanishing or exploding gradients, steep and flat directions, large condition numbers, and sensitivity to depth and initialization.

These are geometric or conditioning problems. They do not mean that useful target information is absent: an informative dependence may exist but be transmitted badly through the parameterization.

5.3.2 Architectural responses to geometry

Several architectural devices can improve this situation. Residual connections provide shorter gradient paths. Normalization controls scales. Suitable initialization can keep Jacobian spectra in useful regimes. Overparameterization can introduce additional fitting directions. Adaptive preconditioning can rescale poorly conditioned coordinates.

These are possible improvements, not unconditional convergence results. They can alter the directions accessible to training without supplying new observations. Changing the parameterization can also change which statistics a gradient probes, so geometry and accessible signal interact even though they are conceptually distinct.

5.3.3 The informational obstruction

Even with favorable conditioning, the observations and the chosen learning rule may not expose a useful signal for improving prediction. In particular parameterizations and initialization regimes, the leading population-gradient components are controlled by low-order correlations of the target with accessible features.

If those correlations vanish by symmetry or cancellation, or are too small to estimate at feasible sample size, that mechanism may fail to provide a useful direction. This is a statement about the target, distribution, and learner together, not about the target alone.

Parity illustrates the distinction. In sign coordinates, let

$$\chi_S(X) = \prod_{i \in S} X_i, \qquad X_i \in \{-1, +1\}.$$

A tree of binary operations computes this function economically. Under the uniform product distribution, however,

$$\mathbb{E}[\chi_S(X) X_j] = 0 \qquad (j \in S,\ |S| > 1).$$

This rules out that particular linear screening signal. It does not show that every gradient-based parameterization, every subsequent training stage, or every learning procedure lacks a useful signal.

5.3.4 Geometry versus information

The distinction can be summarized as

geometry:	how an informative dependence is transmitted;
accessible information:	what the learning rule can detect.

Rescaling a fixed zero gradient cannot make it informative. A different representation, observation model, or probe family may nevertheless expose information that the original rule did not use. Thus absence of a particular footprint is not automatically absence of information from the full input–output distribution.

Chapter 6 studies mechanisms that expose detectable footprints. Their magnitudes and their connection to actual updates both require analysis.

5.3.5 What compositionality alone guarantees

The taxonomy is

C1:	an economical computational realization exists;
geometry:	the parameterization transmits learning signals;
C2:	useful signals are accessible at feasible resolution.

Compositionality alone does not imply fast convergence to a predictive solution. It establishes still less about identifying generating constituents. A theorem about either objective must specify the algorithm, observation model, and resource bounds.

5.4 Grey-Box Recovery

There is an important positive regime in which compositionality can be exploited more directly. If intermediate variables are available as training signals, local supervised problems can be posed without first inferring those variables from their indirect effects on the final output. This is the grey-box regime.

5.4.1 Black box versus grey box

Consider $f = h \circ g$. In the black-box setting, the learner observes only

$$(x, y), \qquad y = h(g(x)).$$

The generating variable $z = g(x)$ is not supplied. A procedure that aims to recover this particular factorization must discover an appropriate version of z. Predictive learning has a weaker requirement: it may find another computation of the same input–output relation.

In the grey-box setting, z is also available, and the data contain

$$(x, z) \qquad \text{and} \qquad (z, y).$$

With separate module parameters, the two local supervised objectives can be optimized separately.

Proposition 2 (Factorization under Intermediate Supervision)**.** *Let*

$$f = h \circ g.$$

Suppose the training data include the true intermediate variable

$$z = g(x).$$

Then empirical risk minimization for the two modules can be written as two separate problems:

$$\min_{W_g} \sum_i \ell_g \left(g(x_i; W_g), z_i \right),$$

and

$$\min_{W_h} \sum_i \ell_h \left(h(z_i; W_h), y_i \right).$$

Thus intermediate supervision decomposes the composite inverse problem into constituent inverse problems.

It does not imply that each constituent problem is itself easy.

5.4.2 What the proposition proves

The proposition establishes that the supplied targets allow separate local empirical objectives to be posed. It is not an equivalence between minimizing those auxiliary losses and minimizing the original end-to-end loss under all parameter constraints. Shared parameters or additional coupling penalties can couple the local problems again.

Nor does the proposition prove efficient optimization or generalization of each factor. Even a low-dimensional constituent needs an appropriate hypothesis class, informative data, and an adequate learning procedure.

The precise benefit is

> faithful intermediate supervision removes the need to transmit the final loss through the full composition for each local fit.

It does not remove intrinsic local difficulty or accumulated error when the learned modules are recomposed.

For example, suppose $\widehat{g}(x)$ and $g(x)$ both lie in a domain K where h is L_h-Lipschitz, and suppose

$$\|\widehat{h} - h\|_{\infty,K} \le \delta_h, \qquad \|\widehat{g} - g\|_{L^2(P_X)} \le \delta_g.$$

The triangle inequality gives the same local-to-global requirement used in Chapter 4:

$$\|\widehat{h} \circ \widehat{g} - h \circ g\|_{L^2(P_X)} \le \delta_h + L_h \delta_g.$$

The domain condition matters. Training $\widehat{h}$ only on observed true states does not by itself control its behavior at every perturbed state produced by $\widehat{g}$.

5.4.3 Extension to a computation DAG

Suppose $f = \Phi_G$ has internal states z_v, $v \in \mathsf{Int}$. If every parent tuple and corresponding output are observed, the learner can pose local supervised problems

$$z_v = \phi_v(z_{\mathrm{pa}(v)}).$$

With independently parameterized modules, there are $|\mathsf{Int}|$ local fitting problems rather than one jointly hidden composition to fit from the final loss alone.

This is the full-trace version of C3. It can help even when another end-to-end method could learn an accurate predictor. Sample coverage, local estimation and optimization, compatibility of the interfaces, and stability after composition still need to be established.

5.5 Three Levels of Trace Exposure

Not all intermediate information has the same status. It is useful to distinguish three regimes.

5.5.1 Hard trace

In the strongest regime, true intermediate variables of the generating computation are observed. For a DAG node v, the learner receives

$$\left(z_{\mathrm{pa}(v)}, z_v\right).$$

The local dependency is directly supervised. This is the setting of Proposition 2; its factorization statement should not be mistaken for a complete local learnability theorem.

5.5.2 Partial trace

Only some intermediate variables may be observed, or some already trained modules may be available. A partially exposed computation can reduce the amount that must be learned jointly.

An observed teacher state and a pretrained surrogate have different interpretations. The former is part of a faithful trace. The latter may supply useful task information without identifying any teacher state. Both can help, but the usefulness of the surrogate must be established for the task.

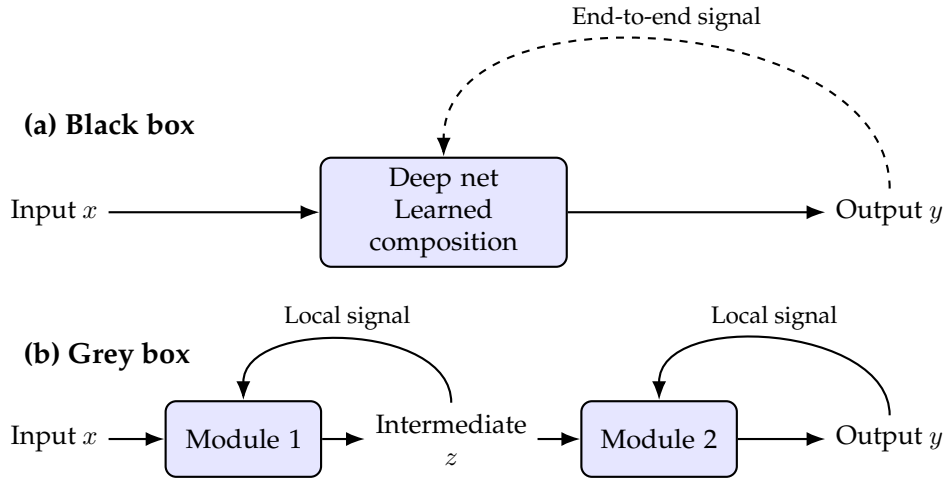


Figure 5.2: Black-box and grey-box learning. End-to-end training fits a predictor without observing the generating intermediate states; it need not identify them. Faithful intermediate supervision supplies local targets, permitting separate module objectives. Each local problem and the accuracy of the recomposed predictor still require analysis.

5.5.3 Surface trace

The learner may instead see externally emitted intermediate symbols, explanations, tokens, or scratchpad states. These may provide an operational decomposition without equaling the latent variables of the underlying computation.

The original hierarchy is therefore best read as a distinction in faithfulness to the generating computation:

hard trace / partial trace / surface exposure.

It is not a literal set inclusion or a universal ranking of predictive usefulness. An informative surrogate can be valuable for prediction even when it is not a faithful internal state.

5.6 Partial Traces in Practice

Several learning strategies can be interpreted through this lens, provided the analogy is calibrated carefully.

5.6.1 Layer-wise training

Early deep-network training strategies often trained one layer or module at a time before global fine-tuning [41]. Such a schedule replaces a single initial deep optimization with a sequence of shallower local training problems.

The intermediate targets need not be externally supervised; in unsupervised or generative pretraining they are constructed indirectly. Layer-wise training is therefore a strategy for organizing optimization, not necessarily an instance of hard-trace supervision. A benefit for the final predictive task still depends on the objectives and learned representations.

5.6.2 Transfer learning

Suppose a pretrained backbone computes $z = g(x)$ and the downstream task trains only a head $y = h(z)$. The representation z need not equal any intermediate variable of the task-generating process.

When z retains useful task information and makes the head problem manageable, replacing $x \mapsto y$ by $z \mapsto y$ can simplify learning. A representation that discards necessary information need not help. Transfer therefore illustrates useful intermediate exposure without requiring structural identification of the teacher.

5.6.3 Scratchpads and chain-of-thought

A model can also be trained or prompted to emit intermediate symbols

$$r_1, r_2, \ldots, r_T$$

before producing its final answer. Such scratchpad or chain-of-thought sequences can function as a surface computational trace, replacing one mapping by a sequence of conditional steps:

$$x \mapsto r_1 \mapsto r_2 \mapsto \cdots \mapsto y.$$

This notation suppresses the retained context. The emitted sequence need not coincide with the model's internal latent computation.

The claim is operational rather than ontological:

external intermediate symbols may provide a useful decomposition without being faithful internal states.

Their accuracy and their effect on the final prediction require their own evaluation; the existence of a trace is not a guarantee of either.

5.6.4 What partial exposure leaves to be learned

Partial or surface exposure can reduce the burden of learning a useful computation. It does not necessarily eliminate local estimation, optimization, missing information, or error accumulation.

When a learner follows a staircase mechanism, exposed features can shorten that staircase. This is a conditional interpretation, not a requirement that every remaining dependency be learned by a staircase or that genericity take one particular form. A learner may instead find a different predictive decomposition.

Thus

partial exposure changes the learning problem; it does not by itself solve the part that remains.

C2 and C3 can complement each other. Trace exposure is not reserved for cases in which end-to-end prediction would otherwise be impossible.

5.7 Statistical Benefits of Structural Alignment

Compositional structure can also have statistical consequences, which should be stated separately from optimization. The number of examples needed for prediction depends on the target and hypothesis classes, smoothness, noise, loss, regularization, and architecture. There is no universal sample-complexity law comparing ε^{-d} with $d\varepsilon^{-k}$ without these assumptions.

When a hypothesis class is adapted to an economical compositional realization, its effective complexity can depend on the number, regularity, and local arity of constituents rather than on the richness of an unrestricted d-variate class. Unknown graph structure, numerical precision, and selection among candidate representations must also be accounted for.

Under appropriate assumptions, learning in such a restricted class can reduce the sample requirement. One useful strategy is to learn local functions instead of treating the target as an unrestricted global map. This need not recover the generating local functions: another controlled realization with accurate population predictions is sufficient for the predictive objective.

A shallow architecture may represent the same target but require much greater width or a less favorable complexity budget. Depth can help when it realizes a useful factorization economically. The statistical claim concerns the admitted or selected predictor class, not merely the existence of some sparse teacher graph.

Chapter H makes the distinction explicit: a feasible circuit realization does not automatically give a low-norm predictor in every chosen architecture, and a low training loss does not automatically control population risk.

5.8 What Grey-Box Structure Does to Optimization Geometry

Intermediate supervision can change geometry as well as information. Consider again $f = h \circ g$. End-to-end optimization has gradient paths of the form

$$\nabla_{W_g} \mathcal{L} = \frac{\partial \mathcal{L}}{\partial h} \frac{\partial h}{\partial g} \frac{\partial g}{\partial W_g}.$$

Direct supervision of g instead supplies the local objective with

$$\nabla_{W_g} \mathcal{L}_g = \frac{\partial \mathcal{L}_g}{\partial g} \frac{\partial g}{\partial W_g}.$$

The Jacobian of h is no longer on that local gradient path. The training problem has changed, rather than the original end-to-end objective having become convex by a formal factorization.

Trace exposure can therefore reduce both the informational burden of inferring an intermediate target and the geometric burden of a long Jacobian chain. Neither benefit guarantees that the local subproblem is easy. Nor does it remove the need to test the composition when the later module receives learned, rather than exact, intermediate inputs.

5.9 The Division of Labor Among C1, C2, and C3

The three roles can now be stated without making them a compulsory learning sequence:

C1:	an economical realization is available;
C2:	useful learning signals are accessible;
C3:	intermediate information is supplied.

Geometry cuts across these roles. A useful signal can be poorly transmitted, and a well-conditioned parameterization need not expose the statistics needed by a particular learning rule. Changing the accessible features or observations can alter this situation.

These distinctions organize candidate explanations of finite-time learning. They do not eliminate the separate statistical question: why does the predictor selected by the procedure generalize? The next two sections introduce abundance and selection among fitting solutions. Chapter 7 develops the geometric analysis further, and Chapter H combines fitting and complexity control in a conditional predictive argument.

5.10 Overparameterization, Degeneracy, and Entropic Selection

Compositional sparsity concerns the existence of an economical realization. A learner may use a much larger, overparameterized architecture. The number of trainable parameters can greatly exceed the independent constraints imposed by the training sample.

Conditional on interpolation and suitable regularity, the fitting solutions can form an extended set rather than an isolated point. This raises two questions: how does training reach a low-loss region, and which predictor does it select there?

The following calculations retain the distinction between local geometry, idealized stochastic preference, and actual finite-time learning. They introduce the objects used in the fuller treatment of Chapter 7; their connection to prediction is summarized in Chapter H.

5.10.1 Geometry of the interpolating set

Consider a squared empirical loss of the form

$$L(\theta) = \frac{1}{2}\|r(\theta)\|_2^2, \qquad r : \mathbb{R}^P \to \mathbb{R}^M, \tag{5.1}$$

where P is the number of parameters and M is the number of scalar training constraints.

An interpolating solution satisfies

$$r(\theta^\star) = 0.$$

Let

$$J(\theta) = Dr(\theta)$$

denote the Jacobian of the residual vector. For the Hessian calculation, assume that r is twice continuously differentiable on a neighborhood of the point in question. In a piecewise-smooth network this statement applies within a region where that differentiation is valid; it is not an assertion of a classical Hessian at every activation kink.

Proposition 3 (Degeneracy at an interpolating solution)**.** *For the loss in Equation* (5.1)*, at any interpolating solution $\theta^\star$,*

$$\nabla^2 L(\theta^\star) = J(\theta^\star)^\top J(\theta^\star). \tag{5.2}$$

Consequently,

$$\operatorname{rank} \nabla^2 L(\theta^\star) \le M, \tag{5.3}$$

and therefore

$$\dim \ker \nabla^2 L(\theta^\star) \geq P - M. \tag{5.4}$$

Proof. Differentiating

$$L(\theta) = \frac{1}{2} \sum_{i=1}^{M} r_i(\theta)^2$$

gives

$$\nabla L(\theta) = J(\theta)^\top r(\theta).$$

Differentiating once more,

$$\nabla^2 L(\theta) = J(\theta)^\top J(\theta) + \sum_{i=1}^{M} r_i(\theta) \nabla^2 r_i(\theta).$$

At an interpolating solution,

$$r_i(\theta^\star) = 0$$

for every i, so the second term vanishes and

$$\nabla^2 L(\theta^\star) = J(\theta^\star)^\top J(\theta^\star).$$

Since the rank of J is at most M, the claimed nullity bound follows from rank–nullity. □

Thus, when

$$P \gg M,$$

a large Hessian nullspace is not surprising.

It is a direct consequence of interpolation in an overparameterized model.

The proposition concerns the *data loss at exact interpolation*, not a minimum of the full regularized objective. For a twice-differentiable penalty R, the latter Hessian also includes $\lambda \nabla^2 R$; away from exact interpolation, the residual-weighted curvature term in the proof need not vanish either. Chapter 7, Section 7.4, distinguishes these Hessians.

5.10.2 From flat directions to manifolds of solutions

The Hessian statement is local. A zero eigenvalue alone does not establish a manifold of exact solutions. Suppose that

$$\operatorname{rank} J(\theta) = q$$

is locally constant near an interpolating point $\theta^\star$. The constant-rank theorem then implies that

$$\mathcal{M} = \{\theta : r(\theta) = 0\}$$

is locally a smooth manifold of dimension

$$\dim \mathcal{M} = P - q. \tag{5.5}$$

Since $q \leq M$, this gives $\dim \mathcal{M} \geq P - M$ locally.

The parameters agree on the fitted observations. They need not represent the same function outside the sample. These dimension counts establish neither generalization nor the size of a basin of attraction. Increasing the parameter count changes the dimension conclusion only relative to the independent constraint rank and under the stated regularity.

5.10.3 An idealized model of stochastic selection

To understand how stochasticity can distinguish among such solutions, consider the overdamped Langevin diffusion

$$d\theta_t = -\nabla L(\theta_t)\, dt + \sqrt{2T}\, dB_t, \tag{5.6}$$

where B_t is standard Brownian motion and $T > 0$ plays the role of a temperature.

If the dynamics and domain admit the usual normalizable equilibrium, its stationary density is

$$\pi_T(\theta) = \frac{1}{Z_T} \exp\left(-\frac{L(\theta)}{T}\right), \tag{5.7}$$

with

$$Z_T = \int \exp\left(-\frac{L(\theta)}{T}\right) d\theta.$$

This model provides a clean separation between two effects.

The exponential factor favors regions of low loss:

$$e^{-L/T},$$

while the integration measure favors regions occupying large volume.

Thus two sets of parameters with the same loss need not carry the same probability mass.

The integral must be finite. Noncompact scale symmetries can invalidate this assumption unless the model is confined or a suitable regularizing measure is specified. A stationary density also does not say how rapidly a trajectory from the chosen initialization approaches that equilibrium. If the Langevin drift includes a regularizer, the full potential $L + \lambda R$ must replace L in the density and the corresponding local volume calculation. Its minimum set need not be the exact data-fitting set.

5.10.4 Probability mass near a manifold of minima

The volume effect can be made explicit.

Suppose $\mathcal{M}_j$ is a compact smooth manifold of global minima of dimension d_j,

$$L(\theta) = L_\star, \qquad \theta \in \mathcal{M}_j,$$

and suppose that the Hessian is positive definite in directions normal to the manifold.

Let

$$H_j^{\perp}(\theta)$$

denote this normal Hessian.

A tubular Laplace approximation gives, as $T \to 0$,

$$Z_j(T) \sim e^{-L_\star/T} (2\pi T)^{(P-d_j)/2} \int_{\mathcal{M}_j} \frac{d\,\mathrm{Vol}(\theta)}{\sqrt{\det H_j^{\perp}(\theta)}}. \tag{5.8}$$

Equation (5.8) separates three contributions to local probability mass:

dimension of the minimizer manifold,
volume along the manifold,
curvature transverse to the manifold.

At equal minimum loss, a higher-dimensional family of minimizers can receive greater small-temperature mass.

Among minimizer manifolds of the same dimension, larger manifold volume and smaller transverse curvature increase the local Gibbs weight.

This is the precise sense in which an idealized noisy dynamics can exhibit an *entropic preference* among equal-loss solutions.

5.10.5 Isolated minima as a special case

For an isolated nondegenerate minimum $\theta_j^\star$,

$$d_j = 0,$$

and Equation (5.8) reduces to the familiar Laplace approximation

$$Z_j(T) \sim e^{-L_\star/T}(2\pi T)^{P/2}\frac{1}{\sqrt{\det H_j}}. \tag{5.9}$$

For two isolated minima having exactly the same loss,

$$\frac{Z_1(T)}{Z_2(T)} \sim \sqrt{\frac{\det H_2}{\det H_1}}.$$

The common power of T cancels.

Thus one should not say simply that "lowering the temperature favors flatter isolated minima."

At equal depth, the leading relative preference comes from local curvature itself.

The dependence on temperature becomes more consequential when competing solution sets have different dimensions, different minimum losses, or more singular local geometries.

5.10.6 Beyond the regular Hessian picture

Neural-network parameterizations are often singular.

The Hessian may contain zero directions arising from symmetries, redundancies, or intersections of different minimizer sets.

In such cases, the ordinary Gaussian Laplace approximation is insufficient.

Under its analytic and measure assumptions, singular learning theory replaces the ordinary determinant formula by more general asymptotic invariants, including the Real Log Canonical Threshold. Chapter 7 explains how this generalizes the regular-manifold calculation.

The important point here is conceptual rather than technical:

probability near a singular solution depends on local set geometry,
not merely on the eigenvalues of one Hessian.

Thus "flatness" is only a shorthand for a richer geometric phenomenon.

5.10.7 Ordinary SGD is not isotropic Langevin dynamics

The Langevin model is useful because it makes the volume effect explicit.

It should not be identified with ordinary minibatch SGD.

A minibatch update may be written schematically as

$$\theta_{k+1} = \theta_k - \eta \nabla L(\theta_k) + \eta\, \xi_k(\theta_k), \tag{5.10}$$

where

$$\mathbb{E}[\xi_k(\theta) \mid \theta] = 0.$$

The covariance of the gradient noise generally depends on θ:

$$\mathrm{Cov}\,[\xi_k(\theta) \mid \theta] = \frac{1}{B}\Sigma(\theta),$$

for independent uniform draws with replacement, when Σ denotes the covariance of the single-example gradient. A uniform subset of size B without replacement has a finite-population correction, given below. Both formulas assume fixed per-example losses; batch-dependent forward operations require the covariance of the actual batch update.

A diffusion approximation may therefore take a form such as

$$d\theta_t = -\nabla L(\theta_t)\, dt + \sqrt{\frac{\eta}{B}}\, \Sigma(\theta_t)^{1/2} dB_t, \tag{5.11}$$

up to model-dependent constants and approximations.

This differs fundamentally from Equation (5.6).

The finite-step minibatch noise is generally

$$\boxed{\text{anisotropic} + \text{state dependent} + \text{not exactly Gaussian.}}$$

Momentum, adaptive preconditioning, finite step size, and learning-rate schedules introduce additional departures from equilibrium Langevin dynamics.

Therefore ordinary SGD does not in general possess the Gibbs stationary density

$$\pi_T(\theta) \propto e^{-L(\theta)/T}.$$

The Langevin calculation should instead be viewed as an analytically tractable model demonstrating how stochastic dynamics *can* produce an entropic selection bias. Even for that idealized process, an argument about equilibrium mass must be supplemented by finite-time accessibility or mixing estimates before it becomes a learning guarantee.

Regularized stationarity is not a common minibatch equilibrium. For sampling, use $L_{\mathrm{train}} = n^{-1}\sum_{i=1}^{n} \ell_i$ on n fixed examples, grouping each example's outputs in its loss. The residual map in (5.1) can absorb this normalization. Let $L_\lambda = L_{\mathrm{train}} + \lambda R$, $\lambda > 0$, with differentiable losses and a

deterministic differentiable penalty. Put

$$g_i(\theta) = \nabla \ell_i(\theta), \qquad \bar{g}(\theta) = \frac{1}{n}\sum_i g_i(\theta),$$
$$G_{\mathcal{B},\lambda}(\theta) = \frac{1}{B}\sum_{i\in\mathcal{B}} g_i(\theta) + \lambda \nabla R(\theta),$$
$$\Sigma(\theta) = \frac{1}{n}\sum_i (g_i(\theta) - \bar{g}(\theta))(g_i(\theta) - \bar{g}(\theta))^\top.$$

In (5.10), regularized SGD replaces the full data-loss gradient by ∇L_λ; the centered noise at a fixed parameter is unchanged by adding the same penalty gradient to every batch.

At a regularized stationary point θ_λ, $\bar{g}(\theta_\lambda) + \lambda \nabla R(\theta_\lambda) = 0$. For a uniformly sampled B-element subset, with $n > 1$ and $1 \le B < n$,

$$\boxed{\begin{aligned} \mathbb{E}_{\mathcal{B}} G_{\mathcal{B},\lambda}(\theta_\lambda) &= 0, \\ \mathrm{Cov}_{\mathcal{B}}\big(G_{\mathcal{B},\lambda}(\theta_\lambda)\big) &= \frac{n-B}{B(n-1)}\Sigma(\theta_\lambda). \end{aligned}} \tag{5.12}$$

The covariance is nonzero precisely when the individual data gradients differ. Only their average must cancel the regularizer. Positive regularization does not force such differences for every dataset. The exact criterion and proof are in Chapter 7, Proposition 5, and Appendix N, Proposition 45.

At exact interpolation of the fixed per-example squared losses, instead, all g_i vanish. Every regularized batch gradient then equals $\lambda \nabla R$: sampling covariance is zero, but the common update need not be zero. Unless the penalty gradient vanishes, this interpolator is not a stationary point of L_λ. Regularization can therefore sustain intrinsic minibatch fluctuations by changing the stationary balance, without itself supplying random forcing.

The strictly convex example and the architecture-specific normalized-ReLU rank obstruction in Chapter 7, Section 7.17, develop this distinction further. They separate a zero of the average update from a common zero of every batch update. Persistent fluctuations do not, by themselves, establish an isotropic noise model, a Gibbs law, or a generalization advantage.

5.10.8 Learning rate, batch size, and noise scale

The preceding diffusion approximation nevertheless suggests an important qualitative dependence.

Within the validity regime of the approximation, and holding the local noise covariance fixed, the stochastic component becomes stronger relative to deterministic drift when the learning rate increases or the batch size decreases.

Schematically, the diffusion scale contains a factor of the form

$$\sqrt{\frac{\eta}{B}}.$$

This motivates the often useful intuition that combinations of learning rate and batch size control an effective noise scale.

However, because

$$\Sigma = \Sigma(\theta),$$

there is generally no single scalar "temperature" that completely describes SGD.

The quantity

$$\frac{\eta}{B}$$

should therefore be interpreted as a heuristic noise-scale parameter, not as a universal thermodynamic law. With subset sampling the covariance factor is $(n - B)/[B(n - 1)]$ instead of $1/B$. Gradient-estimate covariance, update covariance, and diffusion covariance rate also differ by factors 1, η^2, and η, respectively, on the small-step time scale. A diminishing learning rate can therefore shrink the updates even when gradient-estimator covariance remains nonzero.

5.10.9 Why parameter-space flatness is not generalization

Even if stochastic dynamics favor a high-volume region of parameter space, another step is required before drawing conclusions about generalization.

Parameter-space flatness is not invariant under reparameterization.

For example, in homogeneous networks, transformations such as

$$W_\ell \to cW_\ell, \qquad W_{\ell+1} \to c^{-1}W_{\ell+1}$$

can leave the represented function unchanged while substantially altering local Hessian eigenvalues.

Hence

$$\boxed{\text{flat parameter region} \not\Longrightarrow \text{simple or well-generalizing function.}}$$

Some flat directions merely represent parameter symmetries.

Others may correspond to genuine robustness of the represented function.

These cases must be distinguished.

To connect optimization geometry to generalization one needs a valid statistical argument using specified complexity quantities, such as an appropriate norm, margin, or description-length control. Their behavior under the admitted reparameterizations must be tracked; not every norm or rank statistic is itself representation invariant. A geometric observation alone is not a generalization bound.

This issue is developed further in the chapter on bits, inductive bias, and generalization.

5.10.10 Selection and predictive recovery

In an interpolating regime, training need not find one distinguished parameter vector. It can reach any member of

$$\mathcal{M} = \{\theta : L_{\text{train}}(\theta) = 0\}.$$

The predictive question in this exact-fit comparison is whether the reached member performs well on the population, not whether it equals a designated θ_{SC} encoding the teacher's decomposition. At finite positive regularization, a returned well-fitting parameter need not belong to $\mathcal{M}$ or be a common equilibrium of all batch updates.

Neither a large fitting set nor a high-volume region ensures a good population predictor. The distinction is

representation:	an accurate feasible predictor is available;
optimization:	a specified procedure achieves an adequate fit;
selection:	the procedure favors particular fitting predictors;
generalization:	the selected predictor has small population risk.

Structural identification of the generating graph is a separate, stronger question. Failing to identify that graph is not a predictive failure when the learned function is accurate and feasible.

5.10.11 A role for overparameterization

The preceding observations suggest a qualified interpretation of the success of overparameterization. Under the regularity and realizability conditions above, excess parameters relative to the fitting rank can supply additional directions along a solution set. This removes the requirement of identifying one isolated parameter vector.

Stochasticity, initialization, architecture, and explicit or implicit regularization can then favor particular regions. The distribution of visited solutions depends on the actual dynamics, not on a count of parameters alone. The remaining predictive question is why the selected function is statistically controlled; it need not reproduce a privileged set of intermediate variables.

5.10.12 From geometric preference to finite-time learning

The geometric and stochastic calculations explain how low-loss regions can be abundant and can receive substantial equilibrium weight. They do not by themselves prove that a specified optimizer reaches an adequately fitting, low-complexity predictor in feasible time.

Chapter 6 studies one way to address this missing dynamical step: useful statistical footprints can supply accessible directions for learning. A staircase of constituent discovery is one possible organization of those directions. It is not an extra requirement once predictive success has been established by another valid argument.

The separate roles are best stated without an unconditional implication chain:

overparameterization:	can enlarge the family of fitting solutions;
stochastic or norm bias:	can favor particular regions or predictors;
detectable footprints:	can provide a route for specified updates.

A complete learning theorem must establish the relevant accessibility, fit, resource, and population guarantees. It need not establish that all of these proposed mechanisms operate in every successful run.

5.11 Norm-Driven Selection and Separation of Timescales

The preceding section emphasized that many parameters can realize the same low training loss. One source of selection among such solutions is regularization. A schematic explicitly regularized objective is

$$L_\lambda(\theta) = L_{\text{train}}(\theta) + \lambda R(\theta). \tag{5.13}$$

Here R may be a squared parameter norm, a product-norm penalty, a path norm, or another specified architecture-dependent criterion. Implicit bias can also select solutions, but need not be equivalent to descent on an explicit scalar penalty of this form.

The important point is not that one universal norm governs deep learning. Rather, fitting the observations and selecting among fitting solutions can occur on different timescales.

5.11.1 Fast fitting and slow selection

Suppose the dynamics reduce empirical loss quickly but continue to move substantially afterward. This suggests the possible separation

$$\tau_{\text{fit}} \ll \tau_{\text{select}},$$

where the first timescale concerns achieving a small training loss and the second concerns the slower influence of regularization or implicit bias.

A schematic trajectory is

$$\begin{gathered}\text{rapid fitting}\\ \Downarrow\\ \text{continued motion near a low-loss set}\\ \Downarrow\\ \text{selection of a different, possibly lower-complexity predictor.}\end{gathered}$$

This is a possible mechanism, not a theorem that such a separation always occurs or that the later predictor necessarily generalizes.

For $R(\theta) = \frac{1}{2}\|\theta\|_2^2$, explicit weight decay gives

$$\dot{\theta} = -\nabla L_{\text{train}}(\theta) - \lambda\theta.$$

The regularization term generally moves the parameters away from exact interpolation, while the data-loss gradient acts to restore a good fit. The picture is slow motion near a low-loss set, not generally motion constrained exactly to $L_{\text{train}} = 0$.

For SGD, continued motion can also include intrinsic fluctuations even where the average regularized update vanishes. This is not the same as a deterministic gradient-flow limit. For plain SGD with independent batch draws of fixed positive probabilities, continuous batch gradients, and learning rates bounded away from zero, any finite point limit must be a common zero of all batch gradients; see Appendix N, Proposition 47. Diminishing steps, normalized or adaptive updates, and averaging require their own analyses. A specified stopped iterate can nevertheless be a useful predictor; its fit, complexity, and resource guarantees still have to be established.

5.11.2 An idealized constrained picture

When fitting and selection are sufficiently separated, one can use the idealization

$$\min_{\theta\in\mathcal{M}} R(\theta), \qquad \mathcal{M} = \{\theta : L_{\text{train}}(\theta) = 0\}, \tag{5.14}$$

when the exact fitting set is nonempty. A near-fitting formulation instead uses an explicit tolerance $L_{\text{train}}(\theta) \le \eta$.

This separates the availability of a fitting solution, the dynamics of reaching a good fit, and preference among fitting solutions. Enlarging $\mathcal{M}$ can change the geometry of the first two questions, but its dimension alone does not establish convergence. Regularization expresses a preference for the third; a training theorem must show that an adequately fitting solution with sufficiently small penalty is actually obtained.

5.11.3 Neural collapse as a special example of structured selection

Neural collapse illustrates how late training can organize representations. In the classification regimes discussed in the book, within-class variability can become small, class means can approach an equiangular configuration, and classifier directions can align with those means.

In suitable idealized unconstrained-feature models, the loss together with norm regularization can select global minimizers with Equiangular Tight Frame geometry. The conclusion depends on the model and its assumptions; it is not a statement that every deep network must develop that configuration.

The useful lesson is that a fitting objective may leave many possibilities, while an additional norm criterion selects a geometrically organized subset. Such geometry is neither a premise of the compositional-witness argument nor, by itself, evidence that a generating DAG has been recovered. Population generalization remains a separate claim.

5.11.4 Grokking as delayed selection

The same timescale distinction suggests one possible interpretation of grokking: training error becomes small long before test performance improves substantially. Schematically,

$$\boxed{\tau_{\text{train}} \ll \tau_{\text{generalize}}.}$$

An early predictor may fit the sample with a relatively complicated representation. Later regularization or implicit bias may continue to reorganize that representation while the training loss stays small. A subsequent predictor can capture a rule that predicts more accurately beyond the sample.

This is a possible selection mechanism, not a universal explanation of grokking. Different settings can involve different feature-learning, barrier-crossing, or regularization effects. A test-accuracy transition does not itself identify which mechanism operated or prove recovery of the teacher's internal coordinates.

The point relevant here is that the timescale of fitting can be shorter than the timescale of selecting a predictor with useful population behavior.

5.11.5 Minimum norm need not identify the generating structure

A norm preference does not, by itself, identify the teacher's compositional structure. Nor is a raw parameter norm automatically an intrinsic measure of functional complexity.

For example, in a suitable homogeneous architecture,

$$W_\ell \longmapsto cW_\ell, \qquad W_{\ell+1} \longmapsto c^{-1}W_{\ell+1}, \qquad c > 0,$$

can preserve the represented function while changing individual layer norms. Their Frobenius product is unchanged by this particular reciprocal rescaling. Thus the example cautions against an unspecified norm; it does not invalidate a correctly formulated product-norm bound.

Predictive control does not require structural identification. This caution is fully compatible with the positive argument of Chapter H. Fix an admitted architecture and a complexity functional $\mathcal{C}(\theta)$, and form the sample-independent class

$$\mathcal{H}_Q = \{F_\theta : \mathcal{C}(\theta) \le Q\}.$$

For a suitable homogeneous feedforward setting, a candidate quantity is

$$\mathcal{C}_{\times}(\theta) = \prod_{\ell=1}^{L} \|W_\ell\|_F,$$

with the biases, activation assumptions, and other required factors included in the applicable statistical analysis. A multiplicative penalty can favor a smaller product among comparably fitting solutions. It does not literally minimize the exact Rademacher complexity or ensure that the resulting bound is small at the available sample size.

Let $\widehat{R}_S$ and R_{P} denote empirical and population risk. Suppose a valid uniform bound for this class gives

$$R_{\mathsf{P}}(h) \le \widehat{R}_S(h) + \beta_S(Q, \delta) \qquad \text{for all } h \in \mathcal{H}_Q$$

on an event of probability at least $1 - \delta$. For example, under the bounded-loss and Lipschitz assumptions specified in Chapter H, β_S can be controlled by a norm-sensitive Rademacher bound and a confidence term.

On that event, any returned predictor with $\widehat{f} \in \mathcal{H}_Q$ and $\widehat{R}_S(\widehat{f}) \le \eta$ satisfies

$$\boxed{R_{\mathsf{P}}(\widehat{f}) \le \eta + \beta_S(Q, \delta).}$$

No generating-node correspondence occurs in this implication. Interpolation is the special case $\eta = 0$, not a necessary premise for predictive success. The returned predictor can be a specified stopped iterate $\widehat{f} = F_{\theta_{j_*}}$. The bound requires its empirical fit and class membership, not convergence of every possible minibatch update to zero.

The actual joint success probability also depends on the optimizer achieving the fit and complexity budget; Chapter H states that conditional result explicitly. Choosing Q or the architecture from the same sample requires an appropriate simultaneous bound or other accounted-for selection rule.

The correct distinction is therefore

norm control can support predictive generalization without identifying the generating computation.

5.11.6 Why irrelevant features need not simply decay away

If a target does not depend on an input variable, it is tempting to conclude that norm minimization must set the corresponding first-layer weights to zero. This does not follow in a general nonlinear network. Functional independence can arise through cancellations, distributed representations, or redundant pathways rather than the literal absence of connections.

Moreover, fitting a finite sample does not establish global independence from a coordinate. A claim of feature selection by norm minimization therefore needs additional assumptions.

The distinction is

functional simplicity $\neq$ coordinate sparsity of a parameterization.

A dense representation can be a successful feasible predictor. A claim about visibly sparse weights is stronger than that predictive claim.

5.11.7 Selection among well-fitting predictors

As an idealized exact-fit comparison, suppose $\theta_1, \theta_2 \in \mathcal{M}$ both fit the sample and have similar penalties,

$$R(\theta_1) \approx R(\theta_2).$$

There are two different issues.

If both predict accurately but only one resembles the generating factorization, this is not an ambiguity that predictive learning must resolve. Structural identification needs a separate criterion.

If one has high population risk, the numerical similarity of the penalties alone does not distinguish the two. The relevant question is whether the chosen complexity class has an informative uniform bound for this sample size. On an event where both belong to such a class, the preceding inequality applies to both; neither can have population risk larger than the common empirical-error and complexity allowance.

A bad interpolant with a similar penalty can therefore signal an uninformative complexity budget, inadequate statistical assumptions, or a failure of the stated high-probability event. It is not evidence that predictive success universally requires discovering each teacher constituent.

Regularization must be connected to a valid statistical quantity and to a feasible optimization guarantee. Those requirements, rather than identification of a designated factorization, determine its predictive role. The same statistical reasoning applies to two predictors with nonzero empirical risks: the bound includes each predictor's own empirical error. Neither predictor is required to be a common minibatch equilibrium.

5.11.8 From selection to accessible learning signals

The remaining finite-time question is how the specified optimizer reaches a predictor with both adequate fit and adequate complexity control. Genericity and detectable footprints provide one proposed route. A constituent-learning staircase can be a constructive realization of that route, sometimes proving structural recovery as well.

These mechanisms retain a substantive role: they attempt to establish the algorithmic success that the conditional prediction argument assumes. They are not additional requirements for recognizing an already successful feasible network as a compositional witness.

Neural collapse and grokking remain useful illustrations of continued representation change during or after fitting. Neither makes genericity a universal prerequisite, nor requires the final coordinates to reproduce the generating ones.

The transition to Chapter 6 is therefore

> Which observable signals can the chosen learning dynamics exploit to find an accurate predictor with feasible resources?

The question is learner-relative. Recovering a specified hidden computation is an important stronger version, not the definition of all successful prediction.

5.12 Summary

Compositionality helps optimization in a precise but limited sense. With the computational, regularity, and stability conventions of Chapter 4, an economical realization removes a representation-size obstruction. It supplies a feasible object to be sought, not an automatically tractable search over all candidates.

Two obstacles to a specified learning procedure remain distinct: geometry can attenuate or distort useful gradients, and the current observations and probes can fail to expose a detectable learning signal. Architecture, normalization, initialization, and preconditioning can address aspects of the first. Chapter 6 studies genericity and footprints as mechanisms relevant to the second.

Faithful traces provide a different form of help: they supply local targets and permit separately supervised module objectives. Partial and surface exposure can also be useful. They need not reproduce the true latent computation, and they are not reserved for cases in which end-to-end learning is impossible. Local learnability, coverage, and accuracy after recomposition remain to be established.

Overparameterization introduces another distinction. Conditional on interpolation and local regularity, fitting solutions can have large Hessian nullspaces or form manifolds. Idealized Langevin calculations show how loss, volume, and transverse curvature affect equilibrium weight. Neither local degeneracy nor equilibrium preference alone proves finite-time SGD convergence. Chapter 7 develops this issue further.

Regularization can affect both predictor selection and the stationary balance of sample gradients. A zero full regularized gradient can coexist with nonzero minibatch covariance, unlike exact interpolation of the fixed per-example squared losses. Predictive success concerns the fit and complexity of the returned model, not whether every possible batch update has stopped.

Norm-driven selection can be statistically meaningful even when it does not recover the generating factorization. When training achieves a good fit within a properly controlled class, a valid bound can imply

$$R_{\mathsf{P}}(\widehat{f}) \leq \widehat{R}_S(\widehat{f}) + \beta_S.$$

The bound must be informative, and the optimizer must actually achieve the required fit and complexity budget. Exact interpolation is one important regime, not the only form of predictive success.

The resulting division is

C1:	feasible representation;
C2:	accessible learning signal;
C3:	supplied intermediate information;
optimization:	finding a suitable fit within feasible resources;
statistical selection:	controlling the selected predictor's risk.

These roles are complementary, not a universal sequence of implications. The principal predictive questions are whether an accurate feasible predictor is available, whether training can find a suitable fit, and whether the selected predictor generalizes.

Once an accurate predictor with feasible finite-precision evaluation has been obtained, its own computation provides a bounded-fan-in compositional realization. This is the witness conclusion developed in Chapter H. It does not by itself establish a fixed smooth coarse-grained decomposition, nor does it require matching the teacher's intermediate variables.

The chapter therefore ends with two separate questions:

Predictive learning: how does training find a feasible function that predicts accurately beyond the sample?

Structural recovery: when does it additionally identify the generating computation, up to controlled equivalences?

Genericity and staircase theories can contribute to both questions under specified assumptions. The next chapter examines their learning signals; it does not impose generating-structure recovery as an extra condition on every successful predictor.

CHAPTER 6

Genericity: Where the Optimization Signal Comes From

Why can local gradient-based methods make useful progress on deep, non-convex learning problems?

Chapter 5 separated two obstructions. One is geometric: an informative signal may be badly transmitted through a parameterization. The other concerns accessibility: the current data, loss, and available probes may not expose a useful direction. This chapter studies the second obstruction and its interaction with the first.

*Its central idea is **genericity**. Exact cancellations can be fragile under specified perturbations of the observation model, such as shifts or biases. Breaking a cancellation can create a nonzero low-complexity "footprint." Whether that footprint is large enough to detect, and whether the chosen optimizer can exploit it, are additional questions.*

The logical steps are therefore conditional:

a specified perturbation and suitable nondegeneracy

$\Downarrow$

a nonzero accessible footprint

a quantitative signal bound and controlled estimation cost

$\Downarrow$

a detectable footprint

compatibility with the learner's update directions

$\Downarrow$

a usable local optimization signal.

C2 denotes accessible learning signal in the framework of the book. Genericity is one possible source of that signal, and a staircase is one possible mechanism for exploiting it repeatedly. Neither is asserted to be a universal prerequisite for every successful learning procedure.

The immediate objective is predictive learning: finding a feasible function that performs well beyond the sample. Identifying the particular hidden computation that generated the observations is a stronger, separate objective. A signal theorem can contribute to either, but an initial footprint alone proves neither full convergence nor generalization. Chapter H separates those conclusions from the compositional witness supplied by an already successful feasible predictor.

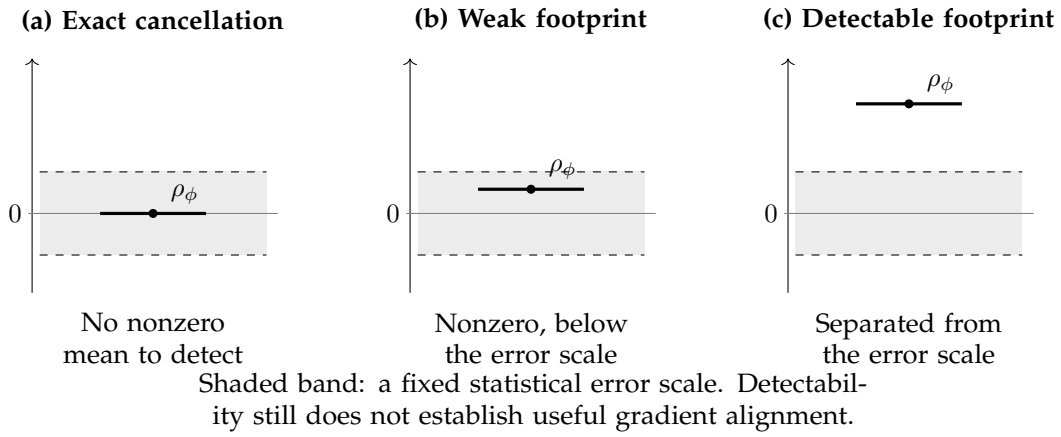


Figure 6.1: Existence, detectability, and use are distinct. The panels compare a population footprint with a common estimation-error scale; they are not confidence intervals centered on observed estimates and are not drawings of global loss landscapes. A perturbation may move a problem from (a) to (b) without reaching (c). Even (c) requires a connection to the chosen learning updates.

6.1 The Missing Ingredient: Informative Signal

Chapter 5 explained in what sense compositional sparsity makes optimization feasible to *pose*. Subject to the computational, regularity, and stability conventions of Chapter 4, an economical realization removes a representation-size obstruction. It does not make the set of candidate realizations small in cardinality, or guarantee an efficient search through that set.

The remaining learner-relative question is:

> What useful statistical information about the target is visible to the specified learning dynamics in their current representation?

Good conditioning of accessible directions need not create a missing target correlation in those directions. Conversely, an informative correlation can exist while its transmission through the parameterization is poor.

The answer depends on the target, the observation distribution, the current representation, and the statistics or features that the learning rule can actually use. Genericity is therefore not a property of the function f alone. It concerns an observed learning problem and a specified perturbation model.

An informative direction need not point toward one designated generating DAG. It can instead help construct a different predictor of the same input–output relation. Throughout this chapter, claims about identifying teacher constituents are distinguished from claims about improving prediction.

6.2 Footprints

Let $X \sim P$ and let $f : \mathcal{X} \to \mathbb{R}$ be the target. Suppose the learner has access to a family of low-complexity probes $\mathcal{F}_{\text{probe}}$. For each probe define

$$\rho_\phi(f, P) = \mathbb{E}_P[f(X)\phi(X)]. \tag{6.1}$$

The expectation is assumed to exist. Probe scale must be fixed when magnitudes are compared: one may, for example, impose $\|\phi\|_{L^2(P)} \leq 1$ and separately control the variance or tails of $f(X)\phi(X)$. Multiplying a probe by an arbitrary constant enlarges its raw footprint and its fluctuations together; it does not make estimation intrinsically easier.

Examples include coordinate functions, low-degree monomials, normalized Hermite polynomials under Gaussian inputs, Fourier characters on the Boolean cube, and features generated by a network at its current parameters. Low degree and low computational complexity are not identical notions, although they coincide in some of the examples considered here.

6.2.1 Existence

A footprint is present when an admitted probe satisfies

$$\rho_\phi(f, P) \neq 0.$$

This is qualitative: the target is not orthogonal to every direction in the specified probe family. Whether the coefficient is useful for a particular prediction or structural task still needs to be shown. An uncentered moment may, for example, reflect a mean response rather than identify a relevant input coordinate.

6.2.2 Size

Existence is not enough. A coefficient of magnitude e^{-d} may be nonzero but require exponentially many examples to resolve by ordinary averaging when its fluctuation scale remains of order one.

Under fixed normalization and suitable moment bounds, a useful target is

$$|\rho_\phi| \geq \frac{1}{\mathrm{poly}(d)}.$$

More generally the lower bound must be polynomial in the relevant problem size, accuracy, and conditioning parameters. The essential distinction is

existence:	$\rho_\phi \neq 0$,
detectability:	the signal can be estimated at feasible cost.

An inverse-polynomial raw magnitude supplies the second property only when the scale of the random variable and the cost of locating and evaluating the probe are also controlled.

6.2.3 Relation to gradients at initialization

For losses and parameterizations that admit differentiation under the expectation, gradient components involve correlations with features induced by the current network. Under squared loss, the relevant target is the current residual $f - F_\theta$, and the probes are parameter derivatives of F_θ.

Near initialization, low-degree components can dominate these probes in particular architectures and regimes. This must be derived for the specified model; it is not a universal restriction on gradient descent. A nonzero correlation with an arbitrary probe need not correspond to an available parameter update.

Section 6.11 gives the exact squared-loss identity. As features change, the accessible probe family can change too. This is one reason an initially hidden dependency may become visible later in training.

6.3 Statistical Sufficiency versus Computational Recoverability

The observations can contain enough information for accurate prediction without making that information easy for a feasible learner to extract. Statistical sufficiency and computational accessibility are different questions. Neither should be identified automatically with uniqueness of a hidden representation.

6.3.1 An information threshold

Fix a target class $\mathcal{F}$, input distribution P, observation model, error criterion, desired accuracy ε, and failure probability δ. Define $n_{\mathrm{info}}(\varepsilon, \delta)$ as the smallest sample size for which some estimator, without a computational restriction, attains the prescribed accuracy with probability at least $1 - \delta$, uniformly over the admitted targets.

For the predictive discussion, the criterion concerns population prediction, not identification of the teacher's DAG. A structural version requires its own equivalence class and loss; its threshold may be infinite if distinct generating structures are observationally indistinguishable.

For example, suppose a fixed finite comparison class $\mathcal{F}_\varepsilon$ has at most 2^{B_ε} elements and contains, for each target, a predictor with population risk within $\varepsilon/2$ of the desired benchmark. For loss in $[0, 1]$, uniform concentration and empirical risk minimization give the sufficient sample scale

$$n_{\mathrm{info}}(\varepsilon, \delta) \lesssim \frac{B_\varepsilon + \log(1/\delta)}{\varepsilon^2}.$$

Here ε is a risk tolerance. The display is an upper bound under the stated approximation assumption, not a universal equality for the minimal sample size, and not an L^2-function-error bound with the same exponent without conversion. The metric and observation model matter.

The effective description or covering complexity of the comparison class, rather than raw parameter count alone, controls this argument.

6.3.2 A recovery threshold depends on the learner

Now fix an algorithm A and computational budget T. Let $n_A(T; \varepsilon, \delta)$ be the smallest sample size at which A meets the *same* accuracy and uniform success requirement using at most T computation. Set this quantity to infinity if no such sample size exists. Then

$$n_A(T; \varepsilon, \delta) \geq n_{\mathrm{info}}(\varepsilon, \delta). \tag{6.2}$$

Equality need not hold. For fixed n, one can instead study $T_A(n; \varepsilon, \delta)$, the computational cost needed by the specified algorithm to meet the requirement.

Thus a single "sample threshold" does not describe all learning obstructions. Data and computation are different resources, and a computational budget must account for reading and processing the sample.

6.3.3 Genericity and the accessible signal

Fix a normalized probe family $\Phi = \mathcal{F}_{\mathrm{probe}}$. The largest footprint in this family is

$$\rho_\star = \sup_{\phi \in \Phi} |\rho_\phi(f, P)|. \tag{6.3}$$

For a specified probe, controlled bounded or sub-Gaussian products give a sufficient detection scale of the schematic form

$$n_{\mathrm{detect}} \lesssim \frac{K^2}{\rho_\star^2} \log \frac{1}{\delta}, \tag{6.4}$$

where K records the applicable fluctuation bound, and the display presumes that a probe attaining a comparable signal has been found. Searching a family introduces additional statistical and computational costs. A supremum over an enormous family is not an algorithm for locating a useful member.

This detection scale is not the information-theoretic sample complexity of the target class. It describes one way of extracting information. Consequently, one can have

$$n_{\mathrm{info}} \ll n_{\mathrm{detect}}$$

for a particular screening mechanism even though another algorithm can learn from fewer examples.

6.3.4 More data can still help recovery

For a fixed nonzero footprint with variance $v_\phi^2 > 0$, the signal-to-noise ratio of its sample average is

$$\frac{|\rho_\phi|\sqrt{n}}{v_\phi}.$$

Thus additional examples can make a weak coefficient usable long after prediction has become possible in principle. An exponentially weak coefficient relative to its fluctuations may make this particular route prohibitive.

If every population footprint in a fixed family vanishes, more averaging does not turn those zero means into nonzero screening signals. This is not, by itself, an impossibility theorem for all learners using the full samples, different statistics, or adaptively constructed representations. Such a theorem would require an explicit restriction on the learner and an indistinguishability argument for the admitted target class.

6.3.5 Changing the problem can be more powerful than adding data

Changes in coordinates, the observation distribution, the probe family, or the supplied intermediate information can expose a signal absent from the original screening problem. Breaking a symmetry is one possibility; exploiting a symmetry through an appropriate representation is another. A perturbation can also preserve a cancellation or attenuate useful information. Noise is not automatically beneficial.

These interventions have different meanings. C1 concerns the existence of a feasible representation. C2 concerns signals accessible to the learner. Generic perturbations are one proposed source of those signals; C3 changes the available observations by supplying intermediate information. They are not interchangeable assumptions.

6.3.6 A simple symmetric example

Parity-like targets can have compact compositional descriptions while all correlations with a specified low-order family vanish under a symmetric product distribution. This is consistent with the full input–output distribution determining the target function on its support.

The distinction is between information present globally and information visible through the chosen probes. It does not imply that every learning algorithm fails on parity. A different representation, probe family, or algorithm may exploit information that linear screening misses.

6.3.7 Trace supervision changes the observation model

Suppose

$$f = g_L \circ \cdots \circ g_1.$$

From final pairs $(x, f(x))$ alone, predictive learning need only find an accurate feasible input–output map. It need not identify these particular intermediate variables. Recovering the specified decomposition is the stronger structural problem.

If a faithful trace is supplied,

$$z_0 = x, \qquad z_\ell = g_\ell(z_{\ell-1}), \quad \ell = 1, \ldots, L,$$

then the pairs $(z_{\ell-1}, z_\ell)$ permit local supervised objectives. The learner no longer has to infer a particular constituent solely through its indirect contribution to the final output.

hidden constituent $\longrightarrow$ observed local target.

Local learnability, coverage of the relevant parent states, and error under recomposition still need analysis. Trace exposure can assist even when end-to-end learning is possible; it is not restricted to cases in which genericity fails.

6.3.8 Representation, information, and recovery

The questions can be kept separate as follows:

C1: representation	Does a feasible realization exist?
statistical sufficiency	Do the observations permit accurate prediction?
C2: accessible signal	Can the learner detect and use relevant information?
C3: trace exposure	Is intermediate information supplied?
algorithmic success	Does the specified procedure succeed efficiently?

Structural identification adds a separate criterion when it is required. A small statistical hypothesis complexity does not by itself make optimization easy, and an optimization obstruction does not prove statistical insufficiency.

6.3.9 The recovery gap

When $0 < n_{\mathrm{info}} < \infty$ and the other quantities are defined for the same learning problem, write

$$\Gamma_A(T) = \frac{n_A(T; \varepsilon, \delta)}{n_{\mathrm{info}}(\varepsilon, \delta)}.$$

A value near one indicates sample efficiency at the allowed computation; a large value indicates an algorithm-dependent gap. Detectable, usable footprints can help keep this gap small for a suitable learning rule, but qualitative genericity alone does not bound it.

Trace supervision changes the observation model, so both thresholds must be reconsidered for the trace-assisted problem. It is not a theorem that the numerical ratio above decreases merely because extra variables are supplied.

6.3.10 The central lesson

Statistical sufficiency asks whether the observations permit the desired accuracy. Computational recoverability asks whether the specified learner can attain it with its available operations and resources.

The remaining sections examine concrete signal mechanisms: analytic shifts, Gaussian integration by parts, biased parity, and the possible progression from one learned representation to another. The final AGOP section studies a complementary diagnostic of sensitivity geometry. None of these analyses identifies predictive success with mandatory reconstruction of the generating computation.

6.4 Genericity as Symmetry Breaking

The central hypothesis is that many exact cancellations are fragile within an appropriate perturbation family. A target invisible to one set of probes under one observation distribution may acquire nonzero correlations under another.

Here "generic" must refer to a specified mathematical model: for example, almost every parameter under an absolutely continuous law, or a residual set in a stated function-space topology. These meanings should not be substituted for a quantitative probability bound or a statement about all real data distributions.

6.4.1 Genericity is distribution-relative

The same f can satisfy

$$\mathbb{E}_P[f(X)X_i] = 0, \qquad \mathbb{E}_{P'}[f(X)X_i] \neq 0.$$

The relevant object is therefore the triple

$$(f, P, \mathcal{F}_{\text{probe}}),$$

together with the specified perturbation model and probe normalization.

Definition 6 (Genericity for recovery; informal)**.** *A learning problem* $(f, P, \mathcal{F}_{\text{probe}})$ *is* generic at low degree *relative to a specified perturbation model if typical perturbations in that model produce nonvanishing low-complexity footprints relevant to the stated learning objective. This qualitative usage does not include a magnitude, search-cost, or optimization guarantee unless those are separately imposed.*

6.4.2 Structural, observational, and optimization genericity

Three related ideas should be distinguished:

structural genericity:	degeneracies are unstable under allowed perturbations;
observational genericity:	a changed observation model breaks a cancellation;
optimization relevance:	the resulting signal is accessible and useful.

The first two suggest sources of signal. The third requires a link to the actual learning rule. No unconditional implication from either of the first two to the third is asserted. In particular, perturbations that preserve the relevant symmetry need not reveal a footprint.

6.5 Analytic Shift Families

A clean setting is a family of shifts

$$f_b(x) = f(x + b), \qquad F_\phi(b) = \mathbb{E}[f(X + b)\phi(X)].$$

Let the shift parameter range over a connected open set $U \subseteq \mathbb{R}^d$. Assume F_ϕ is real analytic on U. Polynomial targets with the necessary finite moments give an elementary example. For more general analytic targets, suitable integrable bounds are needed to pass analyticity through the expectation; pointwise analyticity alone is not an adequate assumption.

If $F_\phi \not\equiv 0$ on U, its zero set has Lebesgue measure zero. Consequently,

$$\boxed{F_\phi(b) \neq 0 \quad \text{for almost every } b \in U.}$$

The nondegeneracy condition is essential: a footprint identically forbidden by the chosen target, probe, and perturbation family stays zero. For an absolutely continuous distribution of shifts on U, the same statement holds almost surely.

This is a precise qualitative sense in which a cancellation can be fragile under shifts.

6.5.1 What this does and does not prove

The analytic argument establishes nonvanishing, not a lower bound. A value of 10^{-3} and a value of 10^{-300} are both nonzero. The theorem does not bound the probability of near-zero coefficients, the number of samples required to resolve them, or the time required by a learning algorithm.

Quantitative detectability therefore needs a separate estimate. Bounding an interaction order can help in a specified model, but is not a substitute for controlling coefficient size and fluctuation scale.

6.6 Thom Transversality as Geometric Motivation

There is also a geometric perspective. Let $f : \mathbb{R}^d \to \mathbb{R}$ be smooth and let $j^k f$ denote its k-jet extension. Some exact degeneracies can be represented by constraints defining a submanifold, or suitable strata, in

$$W \subset J^k(\mathbb{R}^d, \mathbb{R}).$$

Thom's transversality theorem says, under its differentiability and topological assumptions, that maps transverse to such sets are generic. The statement is not that a generic map necessarily avoids W. Rather, degenerate intersections are replaced by transverse ones.

For a smooth stratum of codimension greater than the domain dimension, transversality forces its preimage to be empty. Otherwise, a nonempty transverse preimage has the dimension given by the codimension count. These are geometric statements about jets, not sample-complexity bounds.

6.6.1 Why transversality is secondary here

The learning signals studied here are distributional quantities such as $\mathbb{E}[f(X)\phi(X)]$, not simply derivative constraints at a point. Transversality is therefore geometric motivation. It does not by itself prove that one of these expectations is nonzero, inverse-polynomially large, or aligned with a gradient update. Analytic shift families and explicit correlation calculations are more directly connected to those questions.

6.7 Two Exact Examples

6.7.1 Gaussian inputs and Stein's identity

Let

$$X \sim \mathcal{N}(\mu, \sigma^2 I_d), \qquad \sigma > 0.$$

Assume the regularity and integrability conditions required for Gaussian integration by parts. For instance, adequate absolute continuity in the ith coordinate, integrability of the displayed terms, and vanishing boundary terms suffice. Then

$$\mathbb{E}[f(X)(X_i - \mu_i)] = \sigma^2 \mathbb{E}[\partial_i f(X)]. \tag{6.5}$$

For this stated identity, nonzero average derivative and nonzero centered linear footprint are *equivalent*, since $\sigma > 0$.

Functional dependence on x_i is a different property. The function can depend on x_i while the average derivative cancels. Thus dependence alone does not establish either side as nonzero. A suitable perturbation may break that cancellation, but its magnitude and accessibility to the optimizer remain additional issues.

6.7.2 Boolean parity under biased product measures

Let $X_i \in \{-1, +1\}$ be independent, with $\mathbb{E}[X_i] = \mu_i$, and let

$$f(X) = \prod_{i \in S} X_i.$$

For every $j \in S$,

$$\mathbb{E}[f(X)X_j] = \prod_{i \in S \setminus \{j\}} \mu_i. \tag{6.6}$$

Proof. Since

$$f(X)X_j = \prod_{i \in S \setminus \{j\}} X_i,$$

independence gives

$$\mathbb{E}[f(X)X_j] = \prod_{i \in S \setminus \{j\}} \mathbb{E}[X_i] = \prod_{i \in S \setminus \{j\}} \mu_i.$$

□

Raw footprints and coordinate relevance. Equation (6.6) is an uncentered moment. If the aim is to identify relevant coordinates, a centered statistic avoids a contribution from the target mean. Direct subtraction of the product of means gives

$$\mathrm{Cov}(f(X), X_j) = \begin{cases} (1 - \mu_j^2) \displaystyle\prod_{i \in S \setminus \{j\}} \mu_i, & j \in S, \\ 0, & j \notin S. \end{cases}$$

Thus the relevant signal and normalization must be specified. Nearly deterministic coordinates, $|\mu_j| \approx 1$, introduce a further scale issue for covariance-based screening.

6.7.3 Existence versus size in one equation

If all factors in the product in Equation (6.6) are nonzero, the corresponding raw footprint is nonzero. But if $|\mu_i| \le \mu < 1$ and $|S| = k$, then

$$|\mathbb{E}[f(X)X_j]| \le \mu^{k-1}.$$

The signal may therefore be exponentially small in interaction order. This upper bound cannot be used as a lower bound on detectability.

To obtain a useful lower bound for bounded k, one needs, for example, $|\mu_i| \ge \underline{\mu}_d > 0$ on the relevant factors. The raw footprint then obeys

$$|\mathbb{E}[f(X)X_j]| \ge \underline{\mu}_d^{k-1}.$$

For covariance screening, the factor $1 - \mu_j^2$ must also be controlled. Thus bias can break symmetry without making the resulting signal large enough to use.

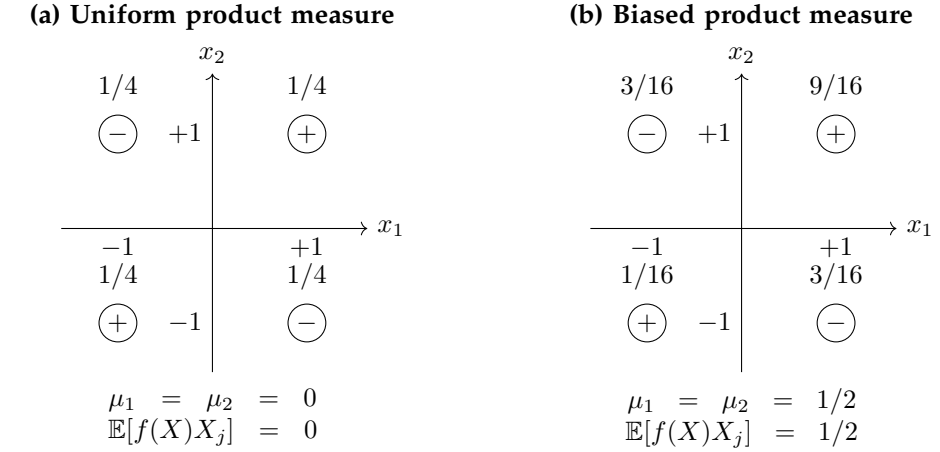


Figure 6.2: Biased sampling, not a translation of equally weighted points. The target is $f(x) = x_1x_2$; signs at the same four locations are its labels, and nearby fractions are sampling probabilities. In (b), each coordinate equals $+1$ with probability $3/4$. For either coordinate, the raw linear footprint is $1/2$ and the centered covariance is $3/8$. For higher-order parity, the product of biases can still be very small.

6.8 From Footprints to Detectability

The finite-sample assertion requires assumptions on the *product* being averaged, $Z_\phi = f(X)\phi(X)$. Boundedness or a controlled sub-Gaussian parameter for this product gives a root-n estimation scale. Bounds on f and ϕ separately must actually imply the needed product bound; Gaussian polynomial probes are generally unbounded and need appropriate tail or moment analysis.

For a concrete bounded case, let Φ_M be a fixed family of M probes and suppose $|f(X)\phi(X)| \le B$ almost surely for every member. Define

$$\widehat{\rho}_\phi = \frac{1}{n}\sum_{i=1}^{n} f(X_i)\phi(X_i).$$

The bounded-average concentration inequality and a union bound give

$$\mathbb{P}\left(\max_{\phi\in\Phi_M}|\widehat{\rho}_\phi-\rho_\phi|>t\right)\le 2M\exp\left(-\frac{nt^2}{2B^2}\right).$$

Consequently, a sufficient sample size for uniform error at most $\rho_{\min}/2$ is

$$n\ge\frac{8B^2}{\rho_{\min}^2}\log\frac{2M}{\delta}. \tag{6.7}$$

This resolves coefficients of magnitude at least $\rho_{\min}$ from zero at the specified precision. It is a sufficient screening guarantee, not a universal lower bound for all algorithms.

For $M=d$ coordinate probes, the multiplicity cost is logarithmic in d statistically. Evaluating all M probes still costs computation. For adaptive, sample-selected features, one needs fresh data conditional on the preceding stages or an appropriate uniform argument for the whole admitted adaptive procedure.

6.8.1 Polynomially detectable footprints

If B is polynomially controlled, the inverse signal is polynomial, and the relevant probes can be found and evaluated at polynomial cost, then Equation (6.7) gives a polynomial screening procedure. A statistical bound involving $\log M$ alone does not prove computational efficiency when M is exponentially large.

6.8.2 Exponentially weak footprints

With order-one fluctuation scale, a footprint of size $e^{-\Omega(d)}$ leads to an exponential sufficient sample budget for this averaging mechanism. A matching impossibility claim needs a lower-bound argument for a specified learner or observation model. Small raw magnitude by itself is not such an argument, especially when the variance also shrinks.

6.8.3 Parity leakage revisited

For equal biases with fixed $0<|\mu|<1$, the raw parity footprint is $|\rho|=|\mu|^{k-1}$. The preceding sufficient bound becomes

$$n=O\left(|\mu|^{-2(k-1)}\log\frac{d}{\delta}\right)$$

for bounded products and coordinate screening. At fixed k this is controlled; if $k=O(\log d)$ it can still be polynomial in d. Larger interaction orders can make this particular route super-polynomial. If the biases depend on d, their lower bounds must also be accounted for. This motivates a distinction between bounded interaction order and a quantitatively bounded signal cost.

6.9 Bounded Leap

The staircase metaphor concerns a sequence of learning problems in changing representations. Informally, the *leap* is the largest amount of new interaction structure that must become accessible at one stage before the learner can proceed. Its definition depends on the chosen coordinates, expansion, and learning rule; it is not identical to the local arity of the teacher DAG.

A bounded leap can limit the attenuation in a model with products of biases or constituent coefficients. For example, if every new factor has magnitude at least d^{-c} and only $\ell = O(1)$ such factors are needed, then their product is at least $d^{-c\ell}$.

But bounded leap alone is not a detectability theorem. A single newly exposed coefficient may be exponentially small, or the variance, probe search, conditioning, and errors in prior representations may dominate it. The quantitative requirement is that all these costs remain controlled.

bounded leap can help explain a detectable next signal; it does not replace a lower bound on that signal.

The two requirements remain distinct: an appropriate footprint must exist, and the learner must be able to detect and use it at the desired resolution.

6.10 The Staircase Picture

The preceding discussion suggests a stagewise mechanism. It is not a universal description of SGD, and the chapter does not derive its full induction from qualitative genericity alone.

6.10.1 Stage 1: detect an exposed footprint

A normalized low-complexity probe has a detectable correlation with a useful target component or residual. The learner must be able to locate that probe and relate it to an admissible update or local fitting problem. This can supply a first step away from an uninformative regime.

6.10.2 Stage 2: create new effective coordinates

The learning procedure constructs a useful representation and preserves it sufficiently accurately for subsequent computation. A dependency that was high-order in the original coordinates can become simpler in the new ones. This change is possible, not automatic: the next footprint and its size must be established for the new, possibly imperfect, representation.

6.10.3 Stage 3: repeat

The proposed progression is

$$\text{detect} \longrightarrow \text{learn and preserve} \longrightarrow \text{expose} \longrightarrow \text{detect}.$$

A recovery theorem must control finite samples, local optimization, accumulated feature error, and the number and cost of stages. Freezing modules or supplying explicit storage can help preserve features; residual connections alone do not guarantee preservation of arbitrary latent coordinates.

Such a theorem can explain how a learner reaches an accurate predictor and may additionally establish structural recovery. It is not a premise needed to recognize an already accurate, computationally feasible network as a compositional witness in Chapter H.

6.10.4 Cliff versus staircase

Relative to the current probes, a *cliff* means that the proposed low-order screening mechanism supplies no useful direction. A *staircase* means that a sequence of usable signals is available as the representation develops. A third case has nonzero but statistically too faint signals.

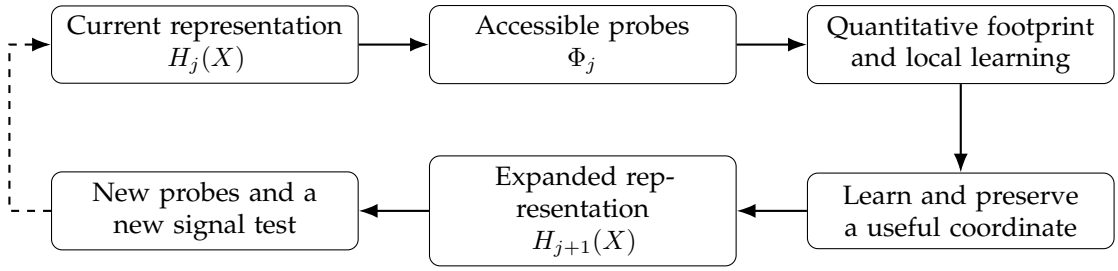


Figure 6.3: The staircase as a conditional learning mechanism, not a global landscape theorem. Each arrow requires its own estimation, optimization, or preservation argument. In particular, constructing one feature does not by itself guarantee a detectable next footprint.

These are descriptions of particular learning mechanisms, not an exhaustive classification of targets or loss landscapes. Failure of a staircase prediction need not imply failure of predictive learning. Nor must a successful learned coordinate coincide with a generating teacher variable.

6.11 Genericity and Gradient-Based Optimization

6.11.1 Population gradients

For a network $F(X;\theta)$ and population loss

$$\mathcal{L}(\theta) = \mathbb{E}[\ell(F(X;\theta), f(X))],$$

the accessible probes depend on the loss and parameterization. Under squared loss and conditions justifying differentiation under expectation,

$$\begin{aligned} \partial_{\theta_j}\mathcal{L}(\theta) &= \mathbb{E}[(F_\theta(X) - f(X))\partial_{\theta_j}F_\theta(X)] \\ &= -\rho_{\partial_{\theta_j}F_\theta}(f - F_\theta, P). \end{aligned} \tag{6.8}$$

The identity uses the unnormalized parameter derivative as written. When magnitudes across directions are compared, its scale and the estimation variance must be included.

At $F_\theta \approx 0$, the target correlation can dominate, but the approximation error in that assertion must be controlled. Later, the residual and the derivative probes both change. A nonzero target moment against some unrelated function need not appear in Equation (6.8).

A nonzero population gradient supplies a local descent direction under appropriate smoothness and step-size conditions. It does not guarantee a polynomial-time trajectory to small population error, identify a hidden constituent, or control generalization from finite samples.

6.11.2 The Statistical Query viewpoint

Population gradients are expectations, and minibatch updates estimate such quantities. This motivates Statistical Query (SQ) models of learning through approximate expectations [51].

An SQ lower bound applies only when the allowed queries, tolerances, and costs model the learner under consideration. The absence of several low-degree correlations is not, by itself, a lower bound for an adaptive SQ algorithm, much less for every procedure operating on the original

examples. A proof must identify which target distinctions remain hidden from the allowed queries at the available tolerance.

The framework is useful precisely because it makes the information accessible through a specified family of noisy statistics explicit. It should not be read as equating every form of gradient descent with a fixed low-degree screening method.

6.11.3 Information versus geometry revisited

At a fixed representation, geometry describes how update directions transmit and scale a signal; the footprint analysis asks which target information those directions reveal.

Multiplying an exactly zero gradient by a preconditioning matrix does not create a nonzero gradient. But changing the architecture or learning a new representation can change the probe family. Thus architectural changes need not leave signal accessibility fixed, even though they do not create new labels or add information to the raw observation model.

Normalization, residual pathways, and preconditioning may help under specified assumptions. Their contribution to accessible information and their contribution to conditioning should be analyzed rather than identified by terminology.

6.12 A Cautionary Example: Removing the Screening Signal

Let $L(x)$ be the population least-squares projection of f onto a specified finite-dimensional linear span containing the coordinate functions used for screening. Assume these functions and f belong to $L^2(P)$. For

$$r(x) = f(x) - L(x),$$

projection orthogonality gives

$$\mathbb{E}[r(X)X_i] = 0$$

for every coordinate included in that span. A mechanism based on those particular linear residual correlations therefore loses that signal after the projection has been removed.

6.12.1 What follows

If a proposed first stage relies specifically on the target's linear footprints, residualizing against the same span can make that stage uninformative. This is a statement about the chosen screening procedure.

6.12.2 What does not follow

Residual fitting is not thereby generically bad. The residual may retain quadratic or other low-complexity footprints, useful features already constructed by the network, or components that another procedure can learn. A stagewise method can preserve the fitted linear part and learn what remains using an expanded class.

The correct conclusion is narrower:

removing the projection used by a particular screening mechanism removes that mechanism's corresponding population signal.

This operation is also distinct from adding an identity pathway in a residual-network architecture. What other features remain available is the important learning question.

6.13 What Genericity Does Not Explain

Genericity is a proposed source of learning signal. Even a quantitatively detectable footprint is not interchangeable with the following properties.

6.13.1 Generalization

A nonzero or detectable coefficient does not by itself show that an empirically fitted predictor has small population risk. Hypothesis complexity, algorithmic stability, regularization, data dependence, and the observation model enter that separate analysis.

Conversely, norm-controlled generalization need not identify the teacher's constituents. Chapter H gives a conditional risk bound for a well-fitting predictor in an appropriate controlled class, without adding a structural-matching premise.

6.13.2 Conditioning

An informative statistic can be badly transmitted through a deep parameterization. Nonvanishing correlations do not control products of Jacobians, curvature, or the numerical precision of the updates.

6.13.3 Global convergence

An initialization signal does not prove that the entire trajectory continues toward a suitable fit. A staircase theorem would need to prove persistence or renewal of useful signal as representations change, together with finite-time and finite-sample control. The present calculations do not establish that theorem for arbitrary SGD.

6.13.4 Faithful internal structure

A learner can exploit predictive footprints without reconstructing the factorization that generated the examples. Structural recovery requires its own identifiability assumptions, controlled equivalences, and nodewise criterion. Merely abandoning uniqueness is not enough to remove this distinction: predictive success need not establish a match with any specified generating-equivalence class.

Once a feasible predictor is known to generalize, its own computation supplies the witness of Chapter H. No further genericity, staircase, or AGOP condition is needed for that particular implication. This does not remove such conditions from a constructive proof that actually uses them to establish the learner's success.

6.14 Genericity and Physical Symmetry

The discussion is not a claim that physical law lacks exact symmetries. Conservation laws and gauge, spatial, temporal, crystallographic, and permutation symmetries can be fundamental or useful. Symmetry can also supply an inductive bias rather than an obstruction.

The relevant object here is the observed learning problem. Boundary conditions, sampling distributions, nuisance variables, and measurement processes can break cancellations present in a more idealized model. They can also preserve those cancellations or obscure a signal. No universal

noise-induced genericity conclusion is intended.

genericity concerns a specified target–observation–probe problem,
not an assertion that fundamental physical symmetries disappear.

6.15 How C1, C2, and C3 Fit Together

The terminology now has the same scope as in the revised Interlude:

C1:	a feasible compositional representation exists;
C2:	useful learning signals are accessible;
C3:	intermediate information is supplied.

C1 does not establish optimization or generalization. Genericity provides one candidate source of C2, but a signal must be quantitatively detectable, affordable to locate, and useful to the specified updates. A staircase is one possible organization of these signals through time.

C3 is not only a fallback. It can assist a learner that could also learn end to end, and its usefulness depends on the fidelity and coverage of the exposed information and on the local learning problems.

These roles cooperate with optimization geometry and statistical selection; they are not a universal chain of necessary and sufficient conditions. In particular, finding a feasible predictor with small population error does not require demonstrating recovery of a privileged hidden DAG.

6.16 Summary of the Footprint Mechanism

The optimization puzzle is not only geometric. A specified learner also needs useful information in the statistics it can access. This chapter studies one proposed source of that information: nonzero footprints that appear or persist in a specified perturbation model.

The basic coefficient is

$$\rho_\phi(f, P) = \mathbb{E}[f(X)\phi(X)].$$

Its interpretation requires three distinctions:

nonvanishing:	$\rho_\phi \neq 0$;
detectability:	controlled scale, estimation, and search cost;
optimization use:	a connection to the actual update directions.

Analytic shift families give an almost-everywhere nonvanishing statement when their footprint function is analytic and not identically zero. Stein's identity relates centered Gaussian linear correlations to average derivatives. Biased parity gives an exact product formula that can be nonzero while remaining exponentially small in interaction order.

Bounded leap can help only together with quantitative coefficient and conditioning assumptions. A staircase theory must then explain how learning preserves useful coordinates and exposes later signals despite finite-sample and approximation error. These are meaningful research questions about a learning mechanism, not necessary extra tests for an already successful compositional predictor.

The next section studies AGOP, which describes sensitivity geometry when input or representation gradients are available. Chapter 7 then develops the geometry of interpolation and

stochastic selection. Chapter H combines representation, optimization, and statistical control into a conditional predictive argument while keeping structural identification separate.

6.17 AGOP as a Diagnostic of Low-Dimensional Structure

A related object that has become useful in the study of feature learning is the *Average Gradient Outer Product* (AGOP). For a differentiable scalar function

$$f : \mathbb{R}^d \to \mathbb{R}$$

and input distribution P, define

$$M_f = \mathbb{E}_{X\sim P}\left[\nabla f(X)\nabla f(X)^\top\right]. \tag{6.9}$$

Assume $\mathbb{E}\|\nabla f(X)\|^2 < \infty$, so that this matrix is finite. The matrix M_f is positive semidefinite and measures the average sensitivity of the function to perturbations in different input directions. Closely related objects appear in the active-subspace literature and in recent work on neural feature learning [**constantine2015active**, 92, 91].

AGOP is useful in the framework of this chapter, but its role should be distinguished carefully from genericity.

The genericity footprints introduced above have the form

$$\rho_\phi(f, P) = \mathbb{E}_P[f(X)\phi(X)].$$

They ask whether the input–output distribution exposes a detectable correlation to a low-complexity probe ϕ.

AGOP instead asks:

along which input directions is the function locally sensitive?

These are related questions, but they are not equivalent.

6.17.1 The clean case: multi-index structure

AGOP has a particularly transparent interpretation for functions that depend on a low-dimensional linear projection.

Proposition 4 (AGOP recovers the active linear span)**.** *Let*

$$f(x) = g(Wx),$$

where

$$W \in \mathbb{R}^{r\times d}, \qquad r \ll d,$$

and

$$g : \mathbb{R}^r \to \mathbb{R}$$

is differentiable, with $\mathbb{E}\|\nabla g(WX)\|^2 < \infty$.
Define

$$G = \mathbb{E}\left[\nabla g(WX)\nabla g(WX)^\top\right].$$

Then

$$M_f = W^\top G W. \tag{6.10}$$

Consequently,

$$\operatorname{range}(M_f) \subseteq \operatorname{range}(W^\top),$$

and

$$\operatorname{rank}(M_f) \leq r.$$

If W has full row rank and

$$G \succ 0,$$

then

$$\operatorname{rank}(M_f) = r$$

and

$$\operatorname{range}(M_f) = \operatorname{range}(W^\top).$$

Proof. By the chain rule,

$$\nabla_x f(x) = W^\top \nabla g(Wx).$$

Therefore

$$\begin{aligned} M_f &= \mathbb{E}\left[W^\top \nabla g(WX)\nabla g(WX)^\top W\right] \\ &= W^\top G W. \end{aligned}$$

The range and rank statements follow immediately.

□

Thus, for a genuine low-dimensional multi-index model,

AGOP can identify the active linear subspace.

This is a population identity when the gradients and distribution are known. Estimating the active span from data additionally requires control of gradient estimation, sampling error, and the relevant spectral gap. It is not an algorithm for obtaining the unknown target gradients from labels alone.

Even at the population level, identifying an active linear span is substantially weaker than reconstructing a compositional computation graph. Neither conclusion is a prerequisite for a feasible predictor to be accurate.

6.17.2 Compositional sparsity does not imply low-rank AGOP

A compositionally sparse function need not have low-rank AGOP.

Consider the completely separable function

$$f(x) = \sum_{i=1}^{d} x_i^2.$$

The squares are unary constituents, and their outputs can be combined by a binary addition tree. Thus the function has a bounded-arity compositional realization, even though its output depends on all the coordinates.

But

$$\nabla f(x) = 2x.$$

For an isotropic input distribution satisfying

$$\mathbb{E}[XX^\top] = I_d,$$

we obtain

$$M_f = 4I_d.$$

Hence

$$\operatorname{rank}(M_f) = d.$$

Therefore

$$\boxed{\text{compositional sparsity} \not\Longrightarrow \text{low-rank AGOP.}}$$

Sparse compositionality concerns the locality of the constituent computation.

AGOP rank concerns the number of linearly independent directions along which the final function varies.

These are different notions of complexity.

6.17.3 AGOP can miss nonlinear interactions

AGOP also does not in general reveal which variables interact inside a constituent.

Consider

$$f(x_1, x_2) = x_1 x_2,$$

with X_1, X_2 independent, mean zero, and unit variance.

Then

$$\nabla f = \begin{bmatrix} X_2 \\ X_1 \end{bmatrix}.$$

Thus

$$M_f = \begin{bmatrix} \mathbb{E}[X_2^2] & \mathbb{E}[X_1 X_2] \\ \mathbb{E}[X_1 X_2] & \mathbb{E}[X_1^2] \end{bmatrix} = I_2.$$

The function contains a pure multiplicative interaction between x_1 and x_2, yet the AGOP has zero off-diagonal entry.

Hence

$$\boxed{M_{ij} = 0 \not\Longrightarrow x_i \text{ and } x_j \text{ do not interact in } f.}$$

AGOP is an outer product of *first derivatives*. It is not a general interaction tensor.

6.17.4 Why a compositional DAG does not generally produce a block-diagonal AGOP

Consider the simple composition

$$f(x_L, x_R) = h\left(g_L(x_L), g_R(x_R)\right).$$

The gradient is

$$\nabla f = \begin{bmatrix} h_1 \nabla g_L \\ h_2 \nabla g_R \end{bmatrix},$$

where

$$h_1 = \partial_1 h(g_L, g_R), \qquad h_2 = \partial_2 h(g_L, g_R).$$

The off-diagonal block of the AGOP is

$$M_{LR} = \mathbb{E}\left[h_1 h_2 \nabla g_L \nabla g_R^\top\right]. \tag{6.11}$$

Even if

$$X_L \perp X_R,$$

Equation (6.11) need not vanish, because h_1 and h_2 depend jointly on the outputs of both branches.

Additional symmetry or factorization assumptions are required.

Thus

$$\boxed{\begin{array}{c}\text{a sparse computation graph does not generically imply} \\ \text{a block-diagonal AGOP in input coordinates.}\end{array}}$$

When a block structure does appear empirically, it may be informative, but it should be interpreted as an additional property of the target, distribution, and coordinate system rather than as an automatic consequence of compositional sparsity.

6.17.5 Coordinate dependence

AGOP is also coordinate dependent.

Suppose

$$x = Az$$

for invertible A. Compare the corresponding distributions: if Z has law P_Z, then $X = AZ$ has its pushforward law P_X. Define

$$\widetilde{f}(z) = f(Az).$$

Then

$$\nabla_z \widetilde{f} = A^\top \nabla_x f,$$

so

$$M_{\widetilde{f}} = A^\top M_f A. \tag{6.12}$$

This is a *congruence transformation*, not a similarity transformation.
For invertible A,

$$\operatorname{rank}(M_{\widetilde{f}}) = \operatorname{rank}(M_f),$$

but the nonzero eigenvalues and their ratios need not be preserved.
Only for orthogonal A does Equation (6.12) reduce to a similarity transformation.
Therefore

AGOP rank is invariant under invertible linear reparameterization, but its eigenspectrum is not.

This is another reason to distinguish intrinsic active dimension from coordinate-dependent spectral geometry.

6.17.6 A single AGOP cannot identify a hidden DAG by diagonalization

Because M_f is symmetric positive semidefinite, it always admits an orthogonal eigendecomposition

$$M_f = Q \Lambda Q^\top.$$

Thus every AGOP can be diagonalized, including targets for which no economical compositional representation has been established.
Consequently, solving

$$\min_{Q \in O(d)} \left\| \operatorname{offdiag}(Q^\top M_f Q) \right\|_F^2$$

cannot by itself test for compositionality.
The minimum is zero for every symmetric M_f.
Recovering latent components from rotated coordinates requires additional information, for example:

- several matrices that must be diagonalized jointly;
- independence assumptions;
- interventions or multiple input distributions;
- higher-order cumulants;
- higher derivatives;

- architectural information.

Thus

AGOP is not, by itself, an inverse map from functions to DAGs.

6.17.7 Layer-wise AGOP as an empirical diagnostic

AGOP may nevertheless be useful for studying a trained network.

Let

$$h_\ell = H_\ell(X)$$

denote the representation at layer ℓ.

Write the remaining computation as

$$f(X) = F_\ell(h_\ell).$$

Define the layer-wise AGOP

$$M_\ell = \mathbb{E}\left[\nabla_{h_\ell} F_\ell \nabla_{h_\ell} F_\ell^\top\right]. \tag{6.13}$$

Tracking

$$M_0, M_1, \ldots, M_L$$

can reveal how the sensitivity geometry changes across depth.

For example, one can measure:

- effective rank;
- principal eigenspaces;
- concentration of sensitivity;
- alignment of learned weight spaces with active directions;
- changes during training.

Recent work on the Neural Feature Ansatz and related feature-learning phenomena suggests that learned features can align with AGOP-like principal subspaces in important model classes [92, 91].

The interpretation should remain conditional:

alignment with AGOP eigenspaces is evidence about learned sensitivity geometry, not a proof that SGD has recovered the target DAG.

6.17.8 Target AGOP versus learned-model AGOP

There is also an observational distinction.

The target AGOP

$$M_f = \mathbb{E}[\nabla f \nabla f^\top]$$

requires derivatives of the unknown target.

These are generally not available from ordinary supervised examples

$$(X_i, Y_i).$$

What can be computed after training is instead

$$M_{\widehat{f}} = \mathbb{E}[\nabla \widehat{f} \nabla \widehat{f}^\top].$$

Derivative approximation gives a sufficient condition for the two matrices to be close. It is not a necessary condition: different gradient fields can have the same average outer product. Small prediction error alone, however, supplies no such guarantee.

Lemma 2 (Gradient approximation controls AGOP approximation)**.** *Let*

$$a(X) = \nabla \widehat{f}(X), \qquad b(X) = \nabla f(X),$$

and assume both vector fields have finite second moments.

Then

$$\left\| M_{\widehat{f}} - M_f \right\|_{\mathrm{op}} \leq \left(\sqrt{\mathbb{E}\|a\|^2} + \sqrt{\mathbb{E}\|b\|^2} \right) \sqrt{\mathbb{E}\|a-b\|^2}.$$

Proof. Using

$$aa^\top - bb^\top = (a-b)a^\top + b(a-b)^\top,$$

we obtain

$$\begin{aligned} \|M_{\widehat{f}} - M_f\|_{\mathrm{op}} &\leq \mathbb{E}[\|a-b\| \, \|a\|] + \mathbb{E}[\|b\| \, \|a-b\|] \\ &\leq \sqrt{\mathbb{E}\|a-b\|^2} \left(\sqrt{\mathbb{E}\|a\|^2} + \sqrt{\mathbb{E}\|b\|^2} \right), \end{aligned}$$

by Cauchy–Schwarz.

□

Thus good prediction error alone does not guarantee recovery of the target AGOP. The displayed lemma supplies a sufficient derivative-based route; other assumptions could also control the matrices directly.

This distinction is particularly important for the framework of Chapter H: population accuracy establishes a predictive claim, whereas matching the target's sensitivity geometry is a stronger, separate claim.

6.17.9 Relation to genericity

The role of AGOP in the present theory can now be stated precisely.

Genericity asks whether useful target structure creates detectable low-complexity footprints in the observed problem.

AGOP asks whether the function exhibits low-dimensional sensitivity geometry once gradients with respect to some representation are available.

Therefore:

genericity footprint:	Can the hidden structure be detected from the learning signal?
AGOP:	What sensitivity subspace does the function exhibit?

A target can have a simple AGOP and still be difficult to learn.

A target can be compositionally sparse and have a full-rank AGOP.

And a target can contain important nonlinear interactions that AGOP does not reveal.

6.17.10 Relation to the staircase

AGOP may be informative once feature learning has begun. Suppose the remaining computation in a learned representation is exactly

$$F_\ell(h) = g(W_\ell h), \qquad \operatorname{rank}(W_\ell) \le r.$$

Under the integrability assumptions above, its AGOP has rank at most r. For an approximate multi-index model, closeness of function values alone does not imply low-rank AGOP.

A sufficient quantitative hypothesis is derivative control. Let $\widetilde{F}_\ell(h) = g(W_\ell h)$ and, under the law of $h_\ell = H_\ell(X)$, suppose

$$\|\nabla F_\ell - \nabla \widetilde{F}_\ell\|_{L^2} \le \zeta, \qquad \|\nabla F_\ell\|_{L^2} + \|\nabla \widetilde{F}_\ell\|_{L^2} \le C.$$

Lemma 2 then gives

$$\|M_\ell - M_{\widetilde{F}_\ell}\|_{\mathrm{op}} \le C\zeta, \qquad \operatorname{rank}(M_{\widetilde{F}_\ell}) \le r.$$

Thus M_ℓ is close in operator norm to a rank-r matrix. For small nonzero ζ, it can still have full exact rank. Eigenvector recovery also requires spectral separation.

Under those additional hypotheses, concentrating sensitivity into a few directions can be an empirical signature of a useful change of representation. It is not an automatic consequence of learning one constituent, nor a necessary signature of every successful staircase. The earlier separable quadratic example already shows why compositional sparsity need not reduce global active dimension.

The logical direction remains important: a specified feature-learning mechanism may predict changes in AGOP; observing AGOP alone does not prove that mechanism or identify the teacher's DAG.

6.17.11 Practical use

For experiments, AGOP is therefore best treated as a diagnostic rather than as a structural oracle.

Useful measurements include:

$$\operatorname{rank}_{\mathrm{eff}}(M_\ell),$$

principal-angle alignment between successive eigenspaces,

$$\angle\left(\mathrm{range}(W_\ell^\top), \mathrm{TopEig}(M_\ell)\right),$$

and the evolution of these quantities over training.

Such measurements could test whether the representations learned by SGD progressively concentrate sensitivity into lower-dimensional directions.

They should not be interpreted as uniquely recovering the underlying computation graph.

6.17.12 Summary

The useful conclusion is deliberately limited:

AGOP can reveal low-dimensional active linear structure and can diagnose changes in learned sensitivity geometry.

But

AGOP is not, in general, an anatomical image of compositional sparsity.

In particular,

compositional sparsity $\not\Longrightarrow$ low-rank AGOP, zero AGOP cross-term $\not\Longrightarrow$ absence of nonlinear interaction, AGOP eigendecomposition $\not\Longrightarrow$ recovery of the target DAG.

Its proper role in the framework of this chapter is therefore as an empirical and mathematical diagnostic of representation geometry, complementary to—but distinct from—the genericity footprints used by particular learning mechanisms.

From signal mechanisms to predictive learning. The chapter has kept three objects separate: accessible statistical footprints, sensitivity geometry, and predictive accuracy. A signal or AGOP calculation can help explain a specified learning procedure, but neither is an extra test that every accurate predictor must pass.

The next chapter studies overparameterization, interpolation geometry, and stochastic selection. Chapter H then asks when a specified optimizer finds a feasible, well-fitting predictor with statistical complexity small enough to control population risk. Once that success is established, the learned computation supplies a compositional witness. Identifying a generating DAG, verifying a staircase trajectory, or matching the teacher's AGOP are stronger objectives requiring their own assumptions.

CHAPTER 7

Overparameterization and the Geometry of Global Minima

Why can local optimization in a highly nonconvex deep network reach very small training error? Initialization, normalization, skip connections, learning-rate schedules, and stochastic gradients can all matter. There is also a geometric question: what kind of set is the optimizer trying to reach?

In a realizable overparameterized problem, the interpolation constraints can leave many parameter directions unconstrained. At regular points, the zero-loss set is a manifold whose dimension equals the number of parameters minus the rank of the independent fitting constraints. At singular points, the geometry can be more complicated. In either case, an interpolating solution need not be an isolated parameter vector.

This observation motivates two different arguments. A deterministic argument studies whether the prediction residual contracts along the training trajectory. A stochastic argument studies the equilibrium weight of neighborhoods of low-loss sets under an idealized Langevin model. The first can give a convergence guarantee under a conditioning hypothesis. The second makes an energy–volume preference explicit, but does not by itself establish how quickly training reaches equilibrium.

Appendix N supplies a complementary analytic result: an all-time gradient flow whose closure is compact inside the analytic domain converges to a critical point. That point need not interpolate. Regularity of the objective, conditioning of prediction dynamics, and statistical control of the selected function have different roles.

The distinction is essential:

$$\begin{array}{c} \textit{local degeneracy} \\ \neq \ \textit{a large basin of attraction} \\ \neq \ \textit{finite-time convergence of SGD.} \end{array}$$

Overparameterization can contribute to a convergence backstop, but only when the additional dynamical conditions are established. Parameter count alone supplies neither realizability nor an optimization theorem.

Regularization also changes the relevant stochastic endpoint. Exact interpolation of fixed per-example squared losses removes their minibatch sampling variance. A stationary point of a regularized objective need not interpolate: its average data gradient can balance the penalty gradient while individual minibatch updates remain nonzero. The resulting intrinsic fluctuations and a normalized-network rank obstruction are developed below. They are not the same statement as isotropic Langevin equilibrium.

The remaining predictive question is which function training selects and whether it generalizes. A change of internal basis or a decomposition different from the teacher's is not a predictive failure. Consistent with

the Interlude, this chapter separates fitting the sample, controlling population error, and identifying the generating computation. The bridge to Chapter H is from optimization geometry to a feasible, well-fitting predictor with adequate statistical-complexity control, not to a compulsory reconstruction of hidden teacher variables.

7.1 The classical puzzle

Consider a neural network

$$F_\theta : \mathcal{X} \longrightarrow \mathbb{R}, \qquad \theta \in \mathbb{R}^p,$$

and training examples (x_i, y_i), $i = 1, \dots, n$. We begin with the unregularized empirical squared loss

$$\widehat{\mathcal{L}}(\theta) = \frac{1}{2n} \sum_{i=1}^{n} \big(F_\theta(x_i) - y_i\big)^2. \tag{7.1}$$

The map $\theta \mapsto \widehat{\mathcal{L}}(\theta)$ need not be convex, even though squared loss is convex in its prediction argument. Saddles, poor local minima, and complicated critical sets are not ruled out by making the network large.

The motivating phenomenon is that highly overparameterized networks can nevertheless be trained to essentially zero empirical error. The useful question is not whether nonconvexity has disappeared, but

> How does overparameterization change the geometry of fitting solutions, and what additional conditions make those solutions accessible to the specified learning dynamics?

The discussion initially assumes a fixed training sample and a *realizable* model: at least one finite parameter vector fits that sample exactly. This assumption is distinct from having many parameters. Conflicting labels at an identical input, or restrictions on the architecture, can prevent exact interpolation regardless of raw width.

For the differential calculations, the training prediction map is assumed to be at least C^2 on the region under discussion. For a nonsmooth architecture, such calculations apply on an appropriate smooth region; crossing nonsmooth boundaries requires a separate argument. The singular analytic asymptotics later in the chapter have their own, stronger assumptions.

Smoothness, analyticity, and conditioning. For a fixed finite architecture constructed from affine maps and GELU activations, the prediction map and empirical squared loss are real-analytic. This follows by composition of analytic maps, as proved in Appendix N, Theorem 41. It justifies additional tools for critical sets and gradient-flow convergence; it does not supply a uniformly bounded Hessian or a useful Jacobian spectrum.

For example, with $\sigma(z) = z\Phi(z)$ and $\mathcal{L}(a, b) = \frac{1}{2}[a\sigma(b)]^2$, the identities $\sigma(0) = 0$ and $\sigma'(0) = 1/2$ give

$$\frac{\partial^2 \mathcal{L}}{\partial b^2}(a, 0) = \frac{a^2}{4}.$$

Thus an analytic loss can have an unbounded Hessian. The scalar depth-H composition $h_{\ell+1} = \sigma(h_\ell)$ likewise has derivative 2^{-H} at input zero. These calculations are given in Examples 1 and 2 of the appendix.

The useful distinction is

C^2 regularity:	gradient and Hessian calculations,
real analyticity:	additional local analytic tools,
quantitative conditioning:	controlled optimization progress.

Derivatives are bounded on a fixed compact subset of a smooth domain, not uniformly over arbitrary parameters or growing network families. Normalization, residual scaling, and initialization can help establish useful bounds, but their presence alone is not a proof of them.

The objective in (7.1) concerns training predictions. Small population error will require an additional statistical argument. Exact interpolation is the central geometric regime here, not the definition of all successful predictive learning. With a fixed positive regularization coefficient, the relevant solution may instead be *quasi-interpolating*: it fits closely without making every residual zero. Small empirical loss alone is an average criterion; any stronger per-example tolerance must be stated separately. Throughout, "stationary point" refers to a named objective, and "common minibatch equilibrium" means that every admissible minibatch produces zero update.

7.2 Prediction space and parameter space

Define the vector of predictions on the training set and its residual:

$$F(\theta) = \begin{pmatrix} F_\theta(x_1) \\ \vdots \\ F_\theta(x_n) \end{pmatrix} \in \mathbb{R}^n, \qquad e(\theta) = F(\theta) - y.$$

Then

$$\widehat{\mathcal{L}}(\theta) = \frac{1}{2n}\|e(\theta)\|_2^2.$$

As a function of an unrestricted prediction vector $z \in \mathbb{R}^n$, $\|z-y\|_2^2/(2n)$ is a strictly convex quadratic. Nonconvexity enters through the parameterization $\theta \mapsto F(\theta)$ and the set of prediction vectors it makes accessible.

Thus

training prediction space:	$F(\theta) \in \mathbb{R}^n$,
parameter space:	$\theta \in \mathbb{R}^p$.

The dimension of parameter space can greatly exceed the rank of the independent fitting constraints. Many different parameters can then produce the same training prediction vector.

They need not produce the same function outside the sample. A direction that is neutral for the training predictions may change population predictions. Parameter redundancy, agreement on the sample, and functional equivalence on the whole input domain are different notions.

This distinction anticipates Chapter H: it is the population performance of the selected predictor, not identification of a particular parameter vector, that defines predictive success.

7.3 The dimension-counting argument

Let

$$J(\theta) = DF(\theta) \in \mathbb{R}^{n\times p}$$

be the Jacobian of the training prediction map. For squared loss,

$$\nabla \widehat{\mathcal{L}}(\theta) = \frac{1}{n} J(\theta)^\top e(\theta). \tag{7.2}$$

A stationary point satisfies

$$J(\theta)^\top e(\theta) = 0,$$

whereas an interpolating solution satisfies the stronger condition

$$e(\theta) = 0.$$

There is a useful contrast between an *isolated nondegenerate* positive-loss minimum and a *regular* interpolating solution. In the first case, the Hessian is nonsingular in the admitted parameter coordinates, so the stationarity equations locally isolate the critical point. In the second case, only the rank of the fitting constraints counts toward the local codimension of their level set.

The qualification is important. Positive-loss critical sets in neural networks can also be degenerate, even after familiar symmetries have been accounted for. One cannot infer independence of p equations simply because the gradient has p components. The isolated-minimum picture is a comparison case, not a universal description of suboptimal minima.

If $J(\theta)$ has full row rank, $J(\theta)^\top$ is injective. Thus (7.2) immediately gives

$$\operatorname{rank} J(\theta) = n, \quad \nabla \widehat{\mathcal{L}}(\theta) = 0 \quad \Longrightarrow \quad e(\theta) = 0.$$

Full prediction rank rules out a positive-loss stationary point at that location. Establishing such rank, and a quantitative lower singular-value bound along a trajectory, is a separate task.

Even full row rank almost everywhere is insufficient: a trajectory can approach an exceptional rank-deficient point with a positive-measure attracting basin. The analytic least-squares example in Section 7.22 makes this obstruction explicit.

7.3.1 The regular zero-loss manifold

Define

$$\mathcal{M} = \{\theta : F(\theta) = y\}.$$

Suppose $\theta^\star \in \mathcal{M}$ and J has locally constant rank q on a neighborhood of $\theta^\star$. The constant-rank theorem gives

$$\boxed{\dim \mathcal{M} = p - q} \tag{7.3}$$

locally around $\theta^\star$. In the full-row-rank case, $q = n$ and the local dimension is $p - n$.

The conclusion is conditional on the existence and regularity of the interpolating point. Under those conditions, adding parameters while keeping the independent constraint rank fixed can enlarge both parameter space and the local set of fitting solutions:

$$\boxed{\begin{array}{c}\text{interpolation exists and the prediction map has local rank } q \\ \Longrightarrow \quad \text{a local fitting manifold of dimension } p - q.\end{array}}$$

This is a local dimension statement, not yet a statement about the probability of finding that manifold.

7.3.2 Why the p gradient equations do not contradict this

Every differentiable local minimum satisfies p scalar stationarity equations. Why can an interpolating minimum nevertheless have $p - q$ free directions?

At interpolation, $e(\theta) = 0$, so

$$J(\theta)^\top e(\theta) = 0$$

holds automatically. The stationarity equations become consequences of the locally independent fitting constraints; they do not impose p additional independent conditions.

This algebra explains why a regular fitting level set can be extended. It does not show that all positive-loss stationary points are isolated, or that every point with a rank-deficient Jacobian lies on a smooth fitting manifold. At singular points, the rank at one point is not enough to infer the dimension of the zero-loss set.

7.4 The Hessian at interpolation

Differentiating the squared loss twice gives

$$\nabla^2 \widehat{\mathcal{L}}(\theta) = \frac{1}{n} J(\theta)^\top J(\theta) + \frac{1}{n} \sum_{i=1}^{n} e_i(\theta) \nabla^2 F_\theta(x_i). \tag{7.4}$$

The first term is positive semidefinite. The residual-weighted second term can contain negative curvature; it is one source of nonconvexity away from interpolation.

At an interpolating solution $\theta^\star$, all residuals vanish. Hence

$$\boxed{\nabla^2 \widehat{\mathcal{L}}(\theta^\star) = \frac{1}{n} J(\theta^\star)^\top J(\theta^\star).} \tag{7.5}$$

For any parameter direction v,

$$v^\top \nabla^2 \widehat{\mathcal{L}}(\theta^\star) v = \frac{1}{n} \|J(\theta^\star) v\|_2^2.$$

If $\operatorname{rank} J(\theta^\star) = q$, then

$$\operatorname{rank} \nabla^2 \widehat{\mathcal{L}}(\theta^\star) = q, \qquad \boxed{\dim \ker \nabla^2 \widehat{\mathcal{L}}(\theta^\star) = p - q.} \tag{7.6}$$

This rank identity needs no constant-rank neighborhood; it holds wherever the stated derivatives exist and the residual is zero.

Identifying the kernel with a tangent space does need regularity. Under the local constant-rank assumption of the preceding section,

$$\ker J(\theta^\star) = T_{\theta^\star} \mathcal{M}.$$

The local quadratic picture is therefore

$p - q$ tangent directions:	zero quadratic cost,
q normal directions:	positive quadratic cost.

Moving along the fitting manifold preserves zero loss. Moving on a straight line in a tangent direction need not do so: higher-order terms can matter away from the point.

At a singular interpolating point, the same Hessian may have zero modes that do not integrate into a manifold of corresponding dimension. Thus Hessian degeneracy is an exact algebraic consequence when $p > q$, whereas a smooth manifold interpretation is an additional geometric conclusion.

Normalized Hessian–kernel duality. At interpolation, put $J_* = J(\theta^\star)$, $H_* = J_*^\top J_*/n$, $\Theta_* = J_* J_*^\top$, and $K_* = \Theta_*/n$. The singular-value decomposition gives, for their positive eigenvalues,

$$\lambda_j^+(H_*) = \lambda_j^+(K_*) = \frac{1}{n}\lambda_j^+(\Theta_*), \qquad 1 \le j \le q. \tag{7.7}$$

The factor $1/n$ is essential when comparing the Hessian with an unnormalized neural tangent kernel. This algebra does not require a constant-kernel limit or analyticity; see Proposition 41 in Appendix N.

The regularized Hessian is a different object. For a twice-differentiable penalty and $U_\gamma = \widehat{\mathcal{L}} + \gamma\Omega$, $\gamma > 0$,

$$\nabla^2 U_\gamma(\theta) = \frac{1}{n} J^\top J + \frac{1}{n}\sum_i e_i \nabla^2 F_\theta(x_i) + \gamma \nabla^2 \Omega(\theta). \tag{7.8}$$

A stationary point of U_γ need not have $e = 0$. Both the residual term and the penalty curvature must then be retained. Even at an exact interpolator, the Hessian of U_γ includes the penalty term, so the rank identity above applies to the *data-loss Hessian*, not to the penalized Hessian. For full quadratic weight decay, the latter equals $J^\top J/n + \gamma I_p$ at an interpolator, although that parameter is generally not stationary for U_γ. Small nonzero residuals alone do not bound the correction without control of the output second derivatives.

7.5 Why degeneracy matters

Calling every zero mode "flatness" hides an important fact: a regular global solution can be an extended set rather than one distinguished point. An optimizer need not identify a predetermined parameter vector to fit the sample.

A useful comparison is

an isolated nondegenerate positive-loss minimum
versus
a regular high-dimensional interpolating set.

These sets can have very different nearby volumes at a small tolerance. The comparison becomes especially relevant when a stochastic model weights neighborhoods rather than individual points.

However, the alternatives are not exhaustive. Suboptimal minima can form extended sets as well. Furthermore, an extended set can be difficult to reach from the initialization distribution. Degeneracy describes the local fitting geometry; accessibility also depends on the vector field outside that neighborhood.

The geometric benefit is an opportunity, not a guarantee: many parameter values satisfy the same fitting constraints. Whether this opportunity makes training efficient must be shown for the architecture, data, initialization, and optimizer under consideration.

7.6 Volume near a regular global-minimum manifold

Near a regular fitting manifold, introduce tangent coordinates s and Euclidean normal coordinates z:

$$s \in \mathbb{R}^{p-q}, \qquad z \in \mathbb{R}^q.$$

On a compact regular portion, and with the smoothness needed for a uniform Taylor expansion,

$$\widehat{\mathcal{L}}(s,z) = \frac{1}{2} z^\top H_\perp(s) z + O(\|z\|_2^3), \qquad H_\perp(s) \succ 0. \tag{7.9}$$

Tangent directions parameterize zero loss; normal displacement incurs a quadratic penalty to leading order.

For a compact smooth embedded portion of $\mathcal{M}$, a sufficiently small Euclidean tube has leading volume

$$\mathrm{Vol}(T_\varepsilon(\mathcal{M})) \sim \omega_q \varepsilon^q \,\mathrm{Vol}_{p-q}(\mathcal{M}), \qquad \varepsilon \downarrow 0, \tag{7.10}$$

with the usual localization convention at patch boundaries. Here ω_q is the volume of the unit ball in $\mathbb{R}^q$. In contrast, a Euclidean ball around an isolated point in $\mathbb{R}^p$ has volume

$$\mathrm{Vol}(B_\varepsilon(\theta_\ell)) = \omega_p \varepsilon^p.$$

The leading powers differ:

$$\boxed{\varepsilon^q \quad \text{versus} \quad \varepsilon^p.}$$

For a fixed geometry, their ratio contains $\varepsilon^{-(p-q)}$. Thus an extended minimum set can have much more nearby parameter volume than an isolated minimum when the neighborhood radius is small.

A distance tube and a loss sublevel set are not identical. Relating them uses the positive normal Hessian in (7.9); its eigenvalues convert a loss tolerance into transverse distance scales. Curvature and intrinsic volume remain part of the prefactor.

These are small-neighborhood asymptotics for a fixed model. They do not by themselves compare different network widths: the dimensions, curvatures, reference measures, and admissible neighborhood sizes can all change with width. Nor do these Euclidean volumes specify probabilities until a probability measure has been chosen.

7.7 Nearby volume is not the same as basin volume

For gradient flow

$$\dot{\theta}_t = -\nabla \widehat{\mathcal{L}}(\theta_t),$$

the basin of attraction of a set A can be defined by

$$\mathcal{B}(A) = \{\theta_0 : \mathrm{dist}(\theta_t, A) \longrightarrow 0\}.$$

A large tubular neighborhood does not automatically imply a large basin. The basin depends on the vector field away from the solution set. A degenerate global minimum can have a narrow funnel leading to it; an isolated minimum can have a broad attracting region.

Accordingly,

$$\boxed{\begin{array}{c}\text{local degeneracy or large nearby volume} \\ \not\Longrightarrow \quad \text{a large basin of attraction.}\end{array}}$$

For randomized initialization, the relevant quantity is $\pi_0(\mathcal{B}(A))$, not an unspecified Euclidean basin volume. A set can be large in one measure and negligible in another.

A deterministic convergence argument needs a condition on the actual trajectory, or on a region containing it. For squared loss, the most direct condition controls contraction of the residual in training prediction space. This condition can coexist with many neutral parameter directions.

7.8 Residual contraction in prediction space

Define

$$K(\theta) = \frac{1}{n} J(\theta) J(\theta)^\top.$$

Along gradient flow, the chain rule gives

$$\dot{e}(\theta_t) = J(\theta_t)\dot{\theta}_t = -\frac{1}{n} J(\theta_t) J(\theta_t)^\top e(\theta_t) = -K(\theta_t) e(\theta_t). \tag{7.11}$$

The sufficient condition actually used in the loss estimate is

$$e(\theta_t)^\top K(\theta_t) e(\theta_t) \geq \mu \|e(\theta_t)\|_2^2, \qquad \mu > 0, \tag{7.12}$$

throughout the relevant trajectory. The stronger matrix condition $K(\theta_t) \succeq \mu I_n$ implies it. If only a residual subspace is used, that subspace must contain the actual residuals and the inequality must hold on it; positivity on the range of K alone does not suffice for an arbitrary residual.

Using (7.2),

$$\begin{aligned} \frac{d}{dt}\widehat{\mathcal{L}}(\theta_t) &= -\|\nabla \widehat{\mathcal{L}}(\theta_t)\|_2^2 = -\frac{1}{n} e(\theta_t)^\top K(\theta_t) e(\theta_t) \\ &\leq -2\mu \widehat{\mathcal{L}}(\theta_t). \end{aligned} \tag{7.13}$$

Consequently,

$$\boxed{\widehat{\mathcal{L}}(\theta_t) \leq \widehat{\mathcal{L}}(\theta_0) e^{-2\mu t}.} \tag{7.14}$$

This is a conditional deterministic convergence mechanism, not an inference from parameter count. The unresolved premise in any application is the trajectory-wise lower bound (7.12).

The two Gram matrices have different roles. If $p > q$, then $J^\top J$ is singular because some parameter directions leave the training outputs unchanged to first order. Nevertheless, $JJ^\top$ can be positive definite on training prediction space when J has full row rank. Thus

many neutral parameter directions can coexist with rapid contraction of training prediction error.

The parameters may fail to be identifiable even while the training predictions are easy to fit. Neither statement determines the predictor's behavior beyond the sample.

An NTK-style sufficient condition. A concrete route to (7.12) is to prove, on a specified time interval $[0, t_{\mathrm{f}}]$,

$$\begin{aligned} K(\theta_0) &\succeq 2\mu I_n, \\ \sup_{0 \leq t \leq t_{\mathrm{f}}} \|K(\theta_t) - K(\theta_0)\|_{\mathrm{op}} &\leq \mu. \end{aligned} \tag{7.15}$$

Indeed, for every $v \in \mathbb{R}^n$,

$$v^\top K(\theta_t) v \geq v^\top K(\theta_0) v - \|K(\theta_t) - K(\theta_0)\|_{\mathrm{op}} \|v\|^2 \geq \mu \|v\|^2.$$

This proves conditioning throughout that interval, not merely at initialization. A nearly constant kernel is one sufficient strategy; (7.12) also permits substantial kernel change when the actual residual remains controlled. The relevant parameterization, initialization, width scaling, data assumptions, and time interval must be specified. Infinite width alone does not imply a frozen kernel. Appendix N, Section N.4, discusses this distinction and the corresponding literature.

Two different Jacobians. Products of layer Jacobians control transmission through activation space; $J(\theta) = D_\theta F(\theta)$ controls changes of the training predictions under parameter updates. Good conditioning of the former does not by itself imply full row rank or a positive residual Rayleigh bound for the latter. Residual connections help when their derivatives are quantitatively controlled, not merely because an identity branch is present. A precise residual-product bound is proved in Appendix N, Proposition 39.

7.9 A deterministic convergence backstop

Suppose there is a forward-invariant region D on which (7.12) holds with a common $\mu > 0$. Assume the gradient-flow solution exists for all $t \geq 0$ for initial points in D. Then (7.14) holds for every such trajectory. For the finite-parameter conclusion below, assume also that F is continuous on $\mathbb{R}^p$ and that the loss has the local smoothness required for the flow.

One can also justify convergence to a finite interpolating parameter under these assumptions. Write $\ell(t) = \widehat{\mathcal{L}}(\theta_t)$. Before reaching zero loss,

$$-\frac{d}{dt}\sqrt{\ell(t)} = \frac{\|\nabla\widehat{\mathcal{L}}(\theta_t)\|_2^2}{2\sqrt{\ell(t)}} \geq \sqrt{\frac{\mu}{2}}\|\nabla\widehat{\mathcal{L}}(\theta_t)\|_2.$$

The last inequality follows from $\|\nabla\widehat{\mathcal{L}}\|_2^2 \geq 2\mu\widehat{\mathcal{L}}$. Integrating and using $\ell(t) \to 0$ yields

$$\int_t^\infty \|\dot{\theta}_u\|_2 \, du \leq \sqrt{\frac{2}{\mu}}\sqrt{\widehat{\mathcal{L}}(\theta_t)}. \tag{7.16}$$

Hence the trajectory has finite remaining length and converges to some $\theta_\infty \in \mathbb{R}^p$. Continuity gives $F(\theta_\infty) = y$. If zero loss is reached earlier, the gradient is zero and the trajectory can remain there.

If the initialization distribution satisfies $\pi_0(D) \geq 1 - \delta$, this conditional argument gives convergence to interpolation with probability at least $1 - \delta$. The substantive problem is proving the conditioning, forward invariance, and initialization probability for a specified network and dataset. A large value of $p - q$ does not establish any of them by itself.

For a desired loss tolerance $\xi > 0$, the bound gives a continuous-time budget

$$t \geq \frac{1}{2\mu}\log\frac{\widehat{\mathcal{L}}(\theta_0)}{\xi}$$

when the initial loss exceeds ξ. A polynomial-time learning claim additionally needs a polynomially controlled inverse μ^{-1} and a feasible discretization and evaluation model.

Finite-step gradient descent needs its own conditions. If the discrete iterates remain in the conditioning region and the loss has Λ-Lipschitz gradient along the required update segments, then for $0 < \alpha \leq 1/\Lambda$ the descent lemma gives

$$\widehat{\mathcal{L}}(\theta_{k+1}) \leq \widehat{\mathcal{L}}(\theta_k) - \frac{\alpha}{2}\|\nabla\widehat{\mathcal{L}}(\theta_k)\|_2^2 \leq (1 - \alpha\mu)\widehat{\mathcal{L}}(\theta_k).$$

The step can also be chosen with $\alpha\mu \leq 1$. This calculation explains what a discrete deterministic extension requires; it does not transfer the result automatically to noisy SGD iterates.

The justified backstop is therefore

a high-probability initialization region
+ invariance and uniform residual contraction
$\Longrightarrow$ high-probability fitting.

Overparameterization may help establish the premises, rather than replacing them.

Analytic convergence has a different endpoint. There is a complementary positive theorem that does not assume a useful prediction-Jacobian lower bound. Let $U : \Omega \to \mathbb{R}$ be real-analytic on an open set $\Omega \subseteq \mathbb{R}^p$. If $\dot{\theta}_t = -\nabla U(\theta_t)$ is defined for all $t \geq 0$ and its image has compact closure contained in Ω, then

$$\int_0^\infty \|\dot{\theta}_t\| \, dt < \infty, \qquad \theta_t \longrightarrow \theta_\infty, \qquad \nabla U(\theta_\infty) = 0. \tag{7.17}$$

This is Theorem 40 of Appendix N, where the proof is given using the local Łojasiewicz gradient inequality

$$\|\nabla U(\theta)\| \geq c \, |U(\theta) - U(\theta_*)|^\vartheta, \qquad c > 0, \quad \vartheta \in [1/2, 1).$$

The inequality is local to a critical point; compactness supplies the finite covering and eventual localization needed by the proof.

The limiting value $U(\theta_\infty)$ need not be globally minimal. For $U = \widehat{\mathcal{L}}$, it need not be zero. Nor does the theorem supply uniform bounds on the entry time, the constant c, or the exponent across growing model families. For example, the analytic flow of $U(x) = x^{2m}$, $m \geq 2$, need not converge exponentially; the appendix gives the explicit trajectory.

The residual-contraction argument above is different. It does not require analyticity, and its quantitative premise forces the specified residual to zero. Neither analyticity nor compactness is being used as a substitute for that premise. When U includes a regularizer, the analytic result concerns stationarity of that penalized objective; the confinement issue is discussed in Section 7.18. No conclusion about arbitrary noisy SGD follows from either deterministic calculation without additional assumptions.

7.10 SGD as noisy gradient flow

For a deterministic differentiable regularizer, unbiased minibatch SGD has the form

$$\begin{aligned} \theta_{k+1} &= \theta_k - \alpha \nabla U_\gamma(\theta_k) + \alpha \xi_k(\theta_k), \\ U_\gamma &= \widehat{\mathcal{L}} + \gamma \Omega, \qquad \mathbb{E}[\xi_k(\theta_k) \mid \theta_k] = 0. \end{aligned}$$

Here $\gamma = 0$ recovers the unregularized update. At a fixed parameter, adding the same deterministic penalty gradient to every batch changes the mean, not the sampling covariance. It can nevertheless change where the mean update vanishes and whether the data residuals vanish there. Section 7.17 makes this distinction exact. A diffusion approximation, when justified, depends on the sampling scheme, step size, and covariance of the gradient noise.

To isolate a volume effect, consider instead the explicitly specified *isotropic overdamped Langevin model*

$$d\theta_t = -\nabla \widehat{\mathcal{L}}(\theta_t) \, dt + \sqrt{2T} \, dW_t, \qquad T > 0. \tag{7.18}$$

Here T is a temperature parameter of this model, not a universal scalar description of ordinary SGD noise. The next equilibrium calculations use the unregularized loss as their potential. A Langevin model for penalized training instead uses U_γ, including its changed minima and curvature; that replacement is not a theorem that real regularized SGD has an isotropic noise law.

Assume well-posedness and the equilibrium hypotheses needed for a stationary probability law, including

$$Z_T = \int_{\mathbb{R}^p} e^{-\widehat{\mathcal{L}}(\theta)/T}\, d\theta < \infty.$$

Then the Gibbs density is

$$\rho_T(\theta) = Z_T^{-1} e^{-\widehat{\mathcal{L}}(\theta)/T}. \tag{7.19}$$

Its probability of a region U is

$$\rho_T(U) = Z_T^{-1} \int_U e^{-\widehat{\mathcal{L}}(\theta)/T}\, d\theta.$$

At this equilibrium, both loss and the amount of parameter space at that loss affect probability mass.

The integrability assumption is not innocuous. Flat noncompact directions can make Z_T infinite. A confining modification, a specified bounded parameter domain with suitable boundary conditions, or a prior may produce a different well-defined model, but its objective, measure, and dynamics must be stated consistently.

Even when (7.19) exists, its formula alone does not bound the time required to approach equilibrium from the initialization law. Moreover, at fixed positive T, a lower-dimensional exact minimum manifold has zero probability under a continuous density. The relevant objects are neighborhoods or positive loss tolerances, not the event of sampling a point exactly on the manifold.

7.11 An isolated local minimum under Langevin dynamics

Let θ_ℓ be an isolated nondegenerate local minimum with

$$\widehat{\mathcal{L}}(\theta_\ell) = L_\ell > 0, \qquad H_\ell = \nabla^2 \widehat{\mathcal{L}}(\theta_\ell) \succ 0.$$

On a sufficiently small fixed neighborhood U_ℓ, this point is the unique local minimizer and

$$\widehat{\mathcal{L}}(\theta) = L_\ell + \frac{1}{2}(\theta - \theta_\ell)^\top H_\ell (\theta - \theta_\ell) + O(\|\theta - \theta_\ell\|_2^3).$$

The local partition contribution satisfies

$$\boxed{Z_\ell(T) := \int_{U_\ell} e^{-\widehat{\mathcal{L}}(\theta)/T}\, d\theta \sim e^{-L_\ell/T} \frac{(2\pi T)^{p/2}}{\sqrt{\det H_\ell}}.} \tag{7.20}$$

The factor $e^{-L_\ell/T}$ penalizes positive loss. The determinant term measures the transverse width of the quadratic valley in the chosen Euclidean coordinates.

For two such minima at the same loss, their leading relative equilibrium weight is

$$\frac{Z_1(T)}{Z_2(T)} \longrightarrow \sqrt{\frac{\det H_2}{\det H_1}}.$$

The common power of T cancels. Thus a smaller determinant gives greater local equilibrium weight in this comparison; lowering temperature is not an additional universal preference among equal-loss isolated minima. For different losses, the exponential energy factor also matters.

This is a local equilibrium calculation for a specified reference measure. It neither identifies a basin of attraction nor proves that a minibatch optimizer visits one valley more often than another in finite time.

7.12 A global-minimum manifold under Langevin dynamics

Now let $\mathcal{M}$ be a compact regular zero-loss manifold of codimension q with uniformly positive normal Hessian. In a sufficiently small fixed tube $U_{\mathcal{M}}$, define

$$Z_{\mathcal{M}}(T) = \int_{U_{\mathcal{M}}} e^{-\widehat{\mathcal{L}}(\theta)/T}\, d\theta.$$

Integrating the normal quadratic approximation gives

$$\boxed{Z_{\mathcal{M}}(T) \sim (2\pi T)^{q/2} I_{\mathcal{M}}, \qquad I_{\mathcal{M}} := \int_{\mathcal{M}} \frac{dV_{p-q}(s)}{\sqrt{\det H_{\perp}(s)}}.} \tag{7.21}$$

Only the q normal directions pay a quadratic loss penalty. The tangent directions contribute through the curvature-weighted intrinsic volume $I_{\mathcal{M}}$.

Comparing with (7.20),

$$\frac{Z_{\mathcal{M}}(T)}{Z_{\ell}(T)} \sim e^{L_{\ell}/T} (2\pi T)^{-(p-q)/2} I_{\mathcal{M}} \sqrt{\det H_{\ell}}. \tag{7.22}$$

This separates two effects:

energy advantage:	$0 < L_{\ell}$,
degeneracy factor:	$T^{-(p-q)/2}$ when $p > q$.

The energy advantage already favors a zero-loss isolated minimum over a positive-loss one as $T \downarrow 0$. Degeneracy provides an additional polynomial factor in the stated comparison. It is not necessary for the exponential energy preference itself.

The ratio concerns these local neighborhoods in one fixed model. There may be other global components contributing substantial mass to the normalizing constant. A local comparison does not show that this one manifold has stationary probability tending to one in the whole space. It also says nothing by itself about mixing or hitting time.

Across increasing widths, all geometric prefactors and the admissible temperature range may vary. Exchanging the low-temperature and large-width limits requires uniform estimates that are not supplied by this fixed-model Laplace calculation.

7.13 Free energy and entropic selection

The same calculation can be expressed through a local free energy. For a neighborhood U with partition contribution $Z_U(T)$, define

$$\mathcal{F}_U(T) = -T \log Z_U(T).$$

If L_U is the minimum loss in that neighborhood, write

$$\mathcal{S}_U^{\mathrm{vol}}(T) = \log\big(e^{L_U/T} Z_U(T)\big).$$

Then, by definition,

$$\boxed{\mathcal{F}_U(T) = L_U - T\mathcal{S}_U^{\mathrm{vol}}(T).} \tag{7.23}$$

Here $\mathcal{S}_U^{\mathrm{vol}}$ is an effective logarithmic-volume term. It depends on temperature and on the reference measure; it is not being identified with a temperature-independent intrinsic entropy of a function.

For the regular zero-loss manifold,

$$\begin{aligned}\mathcal{S}_{\mathcal{M}}^{\mathrm{vol}}(T) &= \frac{q}{2}\log(2\pi T) + \log I_{\mathcal{M}} + o(1),\\ \mathcal{F}_{\mathcal{M}}(T) &= -\frac{qT}{2}\log(2\pi T) - T\log I_{\mathcal{M}} + o(T).\end{aligned} \tag{7.24}$$

For the isolated positive-loss minimum,

$$\mathcal{F}_\ell(T) = L_\ell - \frac{pT}{2}\log(2\pi T) + \frac{T}{2}\log\det H_\ell + o(T).$$

These expressions reproduce the relative weights in (7.22).

The integrated curvature factor must be kept intact. If $\langle\cdot\rangle_{\mathcal{M}}$ denotes normalized intrinsic-volume average, then

$$\log I_{\mathcal{M}} = \log \mathrm{Vol}_{p-q}(\mathcal{M}) + \log\left\langle (\det H_\perp)^{-1/2}\right\rangle_{\mathcal{M}}.$$

In general, the last term is not $-\frac{1}{2}\langle\log\det H_\perp\rangle_{\mathcal{M}}$. Replacing a logarithm of an average by an average of logarithms would change the weighting. The codimension-dependent $\log T$ terms must also be retained when comparing components of different dimension.

The useful interpretation remains positive:

interpolating sets can have both lower energy and a favorable local volume contribution.

Overparameterization can contribute to the second effect by providing regular excess directions. This is an explicit equilibrium mechanism, not a width-only guarantee of faster optimization or better prediction.

7.14 Why the smooth-manifold picture is incomplete

Neural parameterizations can contain continuous rescalings, redundant units, changing ranks, and intersecting solution components. Permutations can generate equivalent copies; collisions and other degeneracies can make the local geometry singular. The zero-loss set need not be one smooth manifold of fixed dimension.

The important distinction is between a direction of zero quadratic cost and a direction along an actual curve of exact solutions. At a singular point, higher-order terms can constrain motion even when the Hessian vanishes in that direction. Thus

$$\dim\ker\nabla^2\widehat{\mathcal{L}} = p - q$$

at interpolation does not alone establish a zero-loss manifold of that dimension.

The broader geometric question is

How much weighted parameter-space volume lies within a small loss tolerance of the solution set?

The regular normal-Hessian calculation answers this in one important case. Singular learning theory provides a language for more general local asymptotics under its analytic and localization assumptions.

Appendix N, Sections N.1 and N.4, supplies the analytic zero-set perspective and examples separating Hessian nullity from actual solution set dimension. Analyticity does not turn every such set into a manifold.

This distinction should not be confused with the predictive versus structural distinction. A singular parameterization may represent a perfectly useful predictor. Its singularity is neither evidence of population accuracy nor a failure to recover the teacher's computation.

7.15 The real log canonical threshold

Let $L(\theta) \geq 0$ be an analytic loss, and let φ be a smooth positive local weighting density. Consider the localized integral

$$Z_U(T) = \int_U e^{-L(\theta)/T} \varphi(\theta)\, d\theta$$

near a zero-loss component. In the setting covered by the singular asymptotic theorem—with the required analytic, support, and localization assumptions—its small-temperature form is

$$\boxed{Z_U(T) \asymp C T^{\lambda} \left(\log \frac{1}{T}\right)^{m-1}, \qquad T \downarrow 0.} \tag{7.25}$$

The exponent λ is the real log canonical threshold (RLCT), and m is its multiplicity. This chapter uses this established asymptotic form as a description of singular volume; it does not prove the general singular asymptotic theorem.

For a fixed loss value and a positive nonvanishing local weight, a smaller exponent produces a larger leading contribution as $T \downarrow 0$. If the exponents coincide, the logarithmic multiplicity and then the finite prefactors matter.

The interpretation of 2λ as an "effective codimension" is a useful analogy, not a formula for topological codimension at every singularity. For a smooth zero-loss manifold of codimension q, with quadratic nondegeneracy in the normal directions,

$$\boxed{\lambda = q/2, \qquad m = 1.}$$

Then (7.25) reduces to the normal Gaussian integration in (7.21).

The successive descriptions are therefore

quadratic Hessian information,
regular manifold and normal curvature,
singular weighted-volume asymptotics.

The last description can capture behavior missed by counting Hessian zero modes. Its assumptions cannot be silently extended to every smooth or piecewise-smooth network. An application to a nonsmooth architecture must justify an appropriate stratified or other applicable version.

Nor does a localized prior-weighted integral automatically describe the stationary law of (7.18). A nonconstant weight changes the measure being analyzed; the dynamics must be specified if a dynamical interpretation is intended.

The transformed measure and the analytic exponents. Resolution of singularities must transform the volume element as well as the loss. Under the localization and nonvanishing assumptions stated in Appendix N, Section N.3, a contributing chart has the form

$$L(\pi(u)) = a(u)\prod_j u_j^{2k_j}, \qquad \varphi(\pi(u))|\det D\pi(u)| = b(u)\prod_j |u_j|^{h_j}.$$

Here the unit factors are positive on the relevant chart, and localization factors must not remove its leading contribution. For $k_j > 0$, candidate integrability thresholds are

$$\lambda_j = \frac{h_j + 1}{2k_j}.$$

The Jacobian and weight exponents h_j cannot be omitted. The appendix states the invoked resolution and singular-integral results and explains how the contributing charts determine λ and its multiplicity.

The gradient-inequality exponent ϑ used for analytic convergence in Section 7.9 and the volume exponent λ answer different questions. A resolved loss monomial does not by itself determine a Euclidean gradient-flow rate: changing coordinates also changes the metric. Singular volume, analytic convergence, and finite-time stochastic access remain separate conclusions.

7.16 The singular version of the entropy argument

Suppose a zero-loss component has local asymptotic weight

$$Z_g(T) \asymp C_g T^{\lambda_g}\left(\log\frac{1}{T}\right)^{m_g-1}.$$

An isolated nondegenerate minimum at positive loss L_ℓ has

$$Z_\ell(T) \asymp e^{-L_\ell/T} C_\ell T^{p/2}.$$

Therefore, for fixed positive finite prefactors,

$$\boxed{\frac{Z_\ell(T)}{Z_g(T)} \asymp e^{-L_\ell/T} T^{p/2-\lambda_g}\left(\log\frac{1}{T}\right)^{1-m_g}.} \tag{7.26}$$

The energy gap gives the exponential penalty; singular degeneracy modifies the polynomial and logarithmic factors.

The volume mechanism therefore does not require a smooth minimum manifold in every application. It requires an applicable estimate of weighted low-loss volume. If a competing positive-loss component is itself singular, its own exponent and multiplicity must replace the isolated-minimum values in the comparison.

At equal loss, a smaller applicable volume exponent favors a component asymptotically. At unequal loss, the energy difference must also be included. Neither exponent alone supplies a hitting time, a mixing bound, or a generalization guarantee. These remain local equilibrium or weighted-integral statements for the specified model.

7.17 What real SGD changes

The correct distinction is not between "noise during training" and "no noise at minima." It is between exact interpolation of every sample, stationarity of the full objective, and a common equilibrium of all minibatch updates. Regularization makes this distinction consequential even in the late stage of nearly interpolating training.

7.17.1 Sampling covariance at a fixed parameter

For a fixed training example, let

$$g_i(\theta) = \nabla_\theta \tfrac{1}{2}(F_\theta(x_i) - y_i)^2 = e_i(\theta)\nabla_\theta F_\theta(x_i), \qquad \bar{g}(\theta) = \frac{1}{n}\sum_i g_i(\theta).$$

Sample B indices independently and uniformly with replacement, and let $\widehat{g}_B$ be their average gradient. Conditional on the current parameter,

$$\begin{aligned} \mathbb{E}[\widehat{g}_B \mid \theta] &= \bar{g}(\theta), \\ \mathrm{Cov}(\widehat{g}_B \mid \theta) &= \frac{1}{B}C(\theta), \\ C(\theta) &= \frac{1}{n}\sum_i g_i(\theta)g_i(\theta)^\top - \bar{g}(\theta)\bar{g}(\theta)^\top. \end{aligned} \tag{7.27}$$

Equivalently, $C = n^{-1}\sum_i (g_i - \bar{g})(g_i - \bar{g})^\top$. A single draw has covariance C; conditional independence divides it by B upon averaging. This is the covariance identity also proved in Appendix N, Proposition 44.

When $\mathcal{B}$ is instead a uniformly sampled subset of $[n]$ of size B, without replacement, put

$$\widehat{g}_{\mathcal{B}} = \frac{1}{B}\sum_{i\in\mathcal{B}} g_i, \qquad c_{n,B} = \frac{n-B}{B(n-1)}, \qquad n > 1.$$

Then

$$\mathbb{E}_{\mathcal{B}}[\widehat{g}_{\mathcal{B}} \mid \theta] = \bar{g}, \qquad \mathrm{Cov}_{\mathcal{B}}(\widehat{g}_{\mathcal{B}} \mid \theta) = c_{n,B}C(\theta). \tag{7.28}$$

For completeness, set $h_i = g_i - \bar{g}$ and let I_i indicate membership in $\mathcal{B}$. Since $\sum_i h_i = 0$, $\sum_{i\neq j} h_i h_j^\top = -\sum_i h_i h_i^\top$. Using $\mathbb{E}I_i = B/n$ and $\mathbb{E}I_iI_j = B(B-1)/(n(n-1))$ gives

$$\mathbb{E}\left[\left(\frac{1}{B}\sum_i I_i h_i\right)\left(\frac{1}{B}\sum_j I_j h_j\right)^\top\right] = \frac{n-B}{Bn(n-1)}\sum_i h_i h_i^\top = c_{n,B}C.$$

For $B = n$ the without-replacement variance is zero, whatever the sample gradients. For $1 \le B < n$, its vanishing is equivalent to all the g_i being equal.

The formulas below use sampling without replacement unless stated otherwise. For independent draws with replacement, replace $c_{n,B}$ by $1/B$. These identities assume that each g_i is a fixed per-example quantity at the current parameter. Batch-dependent forward operations, such as training-mode batch normalization, require a covariance analysis of the actual batch update instead.

7.17.2 Exact interpolation and regularized stationarity

There is an elementary distinction at exact squared-loss interpolation:

$$e_i(\theta^\star) = 0 \text{ for every } i \quad \Longrightarrow \quad C(\theta^\star) = 0, \qquad \nabla^2 \widehat{\mathcal{L}}(\theta^\star) = \frac{1}{n} J^\top J \text{ may be nonzero.} \tag{7.29}$$

This statement concerns the data gradients at an exact interpolator. It is not a statement about every minimum of a regularized training objective.

For $U_\gamma = \widehat{\mathcal{L}} + \gamma\Omega$ with a deterministic differentiable penalty, the regularized minibatch gradient is

$$G_{\mathcal{B},\gamma}(\theta) = \widehat{g}_{\mathcal{B}}(\theta) + \gamma \nabla\Omega(\theta). \tag{7.30}$$

The penalty is the same for every batch at the fixed parameter. Thus

$$\begin{aligned} \mathbb{E}_{\mathcal{B}}[G_{\mathcal{B},\gamma} \mid \theta] &= \nabla U_\gamma(\theta), \\ \mathrm{Cov}_{\mathcal{B}}(G_{\mathcal{B},\gamma} \mid \theta) &= c_{n,B} C(\theta). \end{aligned} \tag{7.31}$$

At an exact interpolator, every batch gradient equals $\gamma\nabla\Omega(\theta^\star)$. The sampling covariance is zero, but the mean update need not be zero. Unless the penalty gradient also vanishes, that interpolator is not stationary for U_γ.

At a regularized stationary point θ_γ, by contrast,

$$\bar{g}(\theta_\gamma) + \gamma \nabla\Omega(\theta_\gamma) = 0. \tag{7.32}$$

Only the average data gradient is required to balance the regularizer. The individual data gradients can differ. Consequently,

$$G_{\mathcal{B},\gamma}(\theta_\gamma) = \frac{1}{B} \sum_{i \in \mathcal{B}} \big(g_i(\theta_\gamma) - \bar{g}(\theta_\gamma)\big).$$

This is how regularization can sustain intrinsic minibatch fluctuations without being a random forcing term itself: it changes the stationary balance and can maintain nonzero, sample-dependent data gradients.

7.17.3 A precise criterion for persistent fluctuations

Proposition 5 (Noise at a regularized stationary point)**.** *Let $n > 1$, $1 \le B < n$, and sample $\mathcal{B}$ uniformly among all B-element subsets of $[n]$. Suppose the per-example losses and penalty are differentiable at a stationary point θ_γ of U_γ. Then*

$$\boxed{\mathbb{E}_{\mathcal{B}} \|G_{\mathcal{B},\gamma}(\theta_\gamma)\|^2 = c_{n,B} \frac{1}{n} \sum_{i=1}^{n} \|g_i(\theta_\gamma) - \bar{g}(\theta_\gamma)\|^2.} \tag{7.33}$$

The following conditions are equivalent at this point:

1. *the sampling covariance is zero;*
2. *all individual data gradients $g_i(\theta_\gamma)$ are equal;*
3. *$G_{\mathcal{B},\gamma}(\theta_\gamma) = 0$ for every admissible batch;*
4. *$g_i(\theta_\gamma) + \gamma\nabla\Omega(\theta_\gamma) = 0$ for every example.*

In particular, differing sample gradients imply strictly positive minibatch fluctuation energy even though the full regularized gradient is zero.

Proof. Equation (7.31) gives zero mean and covariance $c_{n,B}C$ at the stated point. Taking its trace proves (7.33). Since $c_{n,B} > 0$, the right side vanishes if and only if every g_i equals $\bar{g}$. Equation (7.32) then makes each regularized per-example gradient zero, so every batch gradient is zero. Conversely, if every batch update vanishes, its mean-square norm is zero. This proves all equivalences. Alternatively, subtract two batches that share $B-1$ examples and differ only in examples i, j. Their difference is $(g_i - g_j)/B$, proving equality of every pair of data gradients. □

At an arbitrary parameter, the exact decomposition is

$$\mathbb{E}_{\mathcal{B}}\|G_{\mathcal{B},\gamma}(\theta)\|^2 = \|\nabla U_\gamma(\theta)\|^2 + c_{n,B}\,\mathrm{Tr}\,C(\theta). \tag{7.34}$$

A small full gradient controls only the first term. The second term is a different quantity; gradient descent on U_γ is not, by definition, minimization of this second moment.

Positive regularization does not force nonzero covariance for every possible dataset. Identical sample gradients can occur in special configurations. The proposition gives the exact nondegeneracy condition, not a universal lower bound depending only on γ. The normalized network result below gives a further structural obstruction to a common minibatch equilibrium.

7.17.4 Persistent noise at a strictly convex global minimum

Nonconvexity is not needed for the phenomenon. Consider the scalar predictor $F_\theta(x) = \theta x$ on the two examples $(1, 1)$ and $(2, 2)$. With quadratic regularization,

$$U_\gamma(\theta) = \frac{1}{4}\left[(\theta-1)^2 + (2\theta-2)^2\right] + \frac{\gamma}{2}\theta^2 = \frac{5}{4}(\theta-1)^2 + \frac{\gamma}{2}\theta^2.$$

Its Hessian is $5/2 + \gamma > 0$, so it has a unique global minimum at $\theta_\gamma = 5/(5+2\gamma)$. For batch size one,

$$G_{\{1\},\gamma}(\theta_\gamma) = (\theta_\gamma - 1) + \gamma\theta_\gamma = \frac{3\gamma}{5+2\gamma},$$
$$G_{\{2\},\gamma}(\theta_\gamma) = 4(\theta_\gamma - 1) + \gamma\theta_\gamma = -\frac{3\gamma}{5+2\gamma}.$$

Their mean is zero, while

$$\mathrm{Var}_{\mathcal{B}}(G_{\mathcal{B},\gamma}(\theta_\gamma)) = \frac{9\gamma^2}{(5+2\gamma)^2} > 0 \qquad (\gamma > 0). \tag{7.35}$$

The unregularized problem interpolates at $\theta = 1$. At that same point with positive regularization, every batch gradient is γ: the motion is deterministic, but the point is no longer stationary. This single example exhibits both endpoints and their different noise levels.

7.17.5 Normalized ReLU networks and the rank obstruction

Xu, Rangamani, Liao, Galanti, and Poggio [117] analyze a more specific mechanism in normalized homogeneous ReLU classifiers. Their Lemma 8 constrains a *common zero of every minibatch update*, not merely a zero of the batch-averaged gradient. The following restatement makes the scalar-output, normalization, and differentiability assumptions explicit and uses this chapter's half-squared-loss convention.

Consider a bias-free scalar-output feedforward network with independently parameterized layer matrices,

$$f_V(x) = V_L\sigma(V_{L-1}\cdots\sigma(V_1x)\cdots), \qquad \|V_k\|_F = 1, \qquad F_{\rho,V}(x) = \rho f_V(x), \quad \rho > 0.$$

Here σ is ReLU, $y_i \in \{-1, +1\}$, and the weight-decay term is $\gamma\rho^2/2$. Write

$$m_i = y_i f_V(x_i), \qquad s_i = 1 - \rho m_i, \qquad R_{ki} = D_{V_k} m_i.$$

The symbol m_i is the signed normalized margin; s_i is a signed residual in margin coordinates, not the per-example loss. No assumption that all s_i are positive is made. Separability alone would not imply it.

Work at a parameter where the training prediction maps are differentiable and the following fixed-gate derivative calculation is valid, for example in a neighborhood with no zero hidden preactivations on the finite sample. For each example and layer, scalar-output backpropagation gives an outer product and the layerwise homogeneity identity gives

$$R_{ki} = y_i a_{ki} b_{ki}^\top, \qquad \operatorname{rank}(R_{ki}) \le 1, \qquad \langle R_{ki}, V_k\rangle_F = m_i. \tag{7.36}$$

Indeed the fixed activation gates make $f_V(x_i) = a_{ki}^\top V_k b_{ki}$ locally in V_k. This calculation is for an unshared layer matrix and a scalar output. A shared convolutional kernel, multiple output losses, or batch-dependent normalization requires its own derivative-rank analysis. We do not infer regularity at a limiting ReLU kink from a measure-zero claim about initialization.

For the constrained minibatch objective

$$U_{\gamma,\mathcal{B}}(\rho, V) = \frac{1}{2B}\sum_{i\in\mathcal{B}}(1 - \rho m_i)^2 + \frac{\gamma}{2}\rho^2, \qquad \|V_k\|_F = 1,$$

the negative scale-gradient and tangent weight-gradient directions are

$$\begin{aligned} d_{\rho,\mathcal{B}} &= \frac{1}{B}\sum_{i\in\mathcal{B}} s_i m_i - \gamma\rho, \\ d_{k,\mathcal{B}} &= \frac{\rho}{B}\sum_{i\in\mathcal{B}} s_i(R_{ki} - m_i V_k). \end{aligned} \tag{7.37}$$

The second line subtracts the normal component on the Frobenius unit sphere, equivalently eliminating the Lagrange multiplier. These are the common-stationarity directions corresponding to the normalized dynamics in [117, Eq. (46)], with the overall factor two removed by the half-loss convention; n and γ here correspond to N and λ in that paper. Finite-step normalization must be implemented by its specified constraint rule; the lemma concerns zeros of the stated directions, not an assertion that a finite Euler step preserves the sphere.

Lemma 3 (Common-minibatch rank restriction; scoped restatement of Xu et al., Lemma 8)**.** *In the normalized scalar-output ReLU setting just specified, assume $n > 1$, $1 \le B < n$, $\gamma > 0$, $\rho > 0$, and $m_i \ne 0$ for every training example. If*

$$d_{\rho,\mathcal{B}} = 0, \qquad d_{k,\mathcal{B}} = 0 \quad \textit{for every } k \textit{ and every } B\textit{-element batch } \mathcal{B},$$

then $\operatorname{rank}(V_k) \le 2$ *for every layer.*

Batch-exchange proof, adapted from the cited lemma. Put $c_i = s_i m_i$. Vanishing of the scale direction and $\gamma\rho > 0$ rule out $s_i = 0$ for every i. Since the margins are nonzero, at least one c_i is nonzero.

Fix k and choose two batches that share $B - 1$ samples and differ only in i, j. Such batches exist for every pair since $B < n$. Subtracting their vanishing weight directions gives

$$(c_i - c_j)V_k = s_i R_{ki} - s_j R_{kj}. \tag{7.38}$$

If some $c_i \neq c_j$, the right side is a difference of two matrices of rank at most one. Dividing by the nonzero scalar shows $\operatorname{rank}(V_k) \leq 2$.

Otherwise $c_i = c$ for all i, with $c \neq 0$. Equation (7.38) makes all $s_i R_{ki}$ equal to a single matrix Q_k of rank at most one. Vanishing of any batch direction then gives $Q_k - cV_k = 0$. Thus $V_k = Q_k/c$ has rank at most one, which also implies the stated bound. This applies to each layer. □

The statement retains the published upper bound rather than asserting that it is sharp. Its hypotheses are explicit sufficient conditions for the argument, not an assertion that every architecture in the book satisfies them. The ReLU normalization and homogeneity calculation must not be transferred unchanged to GELU or to arbitrary regularization.

7.17.6 Intrinsic noise and the interpretation of low-rank bias

At a stationary point of the *batch-averaged normalized dynamics*, concatenate the scale direction and the vectorized tangent layer directions into $d_{\mathcal{B}}$. Uniform batch sampling gives $\mathbb{E}_{\mathcal{B}} d_{\mathcal{B}} = 0$. Under the lemma's other assumptions, if at least one V_k has rank greater than two, not all $d_{\mathcal{B}}$ can be zero. Since every admissible batch has positive probability,

$$\operatorname{Tr} \operatorname{Cov}_{\mathcal{B}}(d_{\mathcal{B}}) = \mathbb{E}_{\mathcal{B}} \|d_{\mathcal{B}}\|^2 > 0. \tag{7.39}$$

This is the precise link from the common-equilibrium obstruction to intrinsic stochasticity. The covariance of the *actual normalized update directions* is used here; it is not silently identified with the covariance of unconstrained parameter gradients.

The paper interprets the restriction as a source of low-rank bias and late-stage SGD fluctuations [117, Sections 6.1–6.2]. Two qualifications preserve the mathematical content. First, the lemma is a necessary rank restriction on common equilibria, not a theorem that SGD always reaches matrices of rank at most two. Second, closeness of all batch directions to zero does not by itself give a quantitative rank bound without controlling margins, residuals, and the coefficients used in the batch-exchange argument. Exact rank and effective rank are also different notions.

If a specified data family cannot be fitted to the required tolerance by networks satisfying the common-equilibrium rank restriction, the lemma rules out a common equilibrium in that well-fitting region. Such an incompatibility must be established for the data family; it does not follow merely from calling the data generic. Conversely, low rank alone does not guarantee a common equilibrium or zero noise.

The qualitative mechanism is useful without overstating it:

regularization can maintain nonzero residuals;
different samples can demand different local updates;
common equilibrium can impose strong rank restrictions.

A useful predictor can therefore coexist with persistent intrinsic minibatch fluctuations. This is compatible with the norm-based selection framework, while any generalization advantage of lower rank still requires an applicable statistical bound.

7.17.7 Learning rates, point convergence, and diffusion models

For a plain update $\theta_{k+1} = \theta_k - \alpha_k G_{\mathcal{B}_k,\gamma}(\theta_k)$, the conditional update covariance is $\alpha_k^2 c_{n,B} C(\theta_k)$. Nonzero gradient-estimator covariance and nonvanishing parameter increments are not synonymous when $\alpha_k \to 0$.

There is a simple fixed-step implication. Assume batches are sampled independently with fixed positive probabilities, their gradient fields are continuous near a putative finite limit, and $\inf_k \alpha_k > 0$. With probability one every possible batch occurs infinitely often. If θ_k converges, then $\theta_{k+1} - \theta_k \to 0$. Along the subsequence for any fixed batch, continuity implies that its gradient must vanish at the limit. Hence a finite point limit must be a common minibatch equilibrium. Where the preceding rank obstruction excludes such an equilibrium, this form of point convergence is excluded under the stated conditions. For a normalized implementation, the same inference additionally uses the specified normalization map: its fixed points must be exactly the zeros of the normalized directions. It is not an inference about arbitrary retractions, clipping rules, or adaptive optimizers.

This does not assert divergence or existence of a stationary distribution. It does not exclude convergence with diminishing learning rates, or convergence of an appropriate averaged predictor. A schedule, stopping rule, or averaging rule is part of the learning procedure. Positive covariance at one parameter also supplies no uniform lower noise bound as $\gamma \to 0$, the batch size changes, or the network size grows.

For a small constant step α, a justified diffusion approximation has the schematic regularized form

$$d\theta_t = -\nabla U_\gamma(\theta_t)\, dt + \left[\alpha c_{n,B} C(\theta_t)\right]^{1/2} dW_t,$$

with time increments α. The covariance rate, the gradient-estimate covariance, and the update covariance have different step-size factors. The effective noise is generally anisotropic and state dependent, and finite steps, momentum, adaptive preconditioning, or constraint handling can change the model further.

Intrinsic noise in a regularized well-fitting region does not imply the Gibbs law in (7.19). Under an explicitly isotropic Langevin model with regularization, the potential would be U_γ, not the unregularized data loss. The Fokker–Planck and local covariance calculations in Appendix N, Section N.5, explain the model dependence. A nonzero covariance need not excite every escape direction or yield a polynomial mixing or hitting time.

The positive research question is how the actual residual-dependent noise, normalization, rank structure, and step-size schedule cooperate in selecting useful predictors. Regularization can keep this stochastic mechanism active near a minimum of the full objective. Whether it produces an entropic preference, efficient access, or improved population accuracy must be shown for the specified setting, rather than inferred from a stationary scalar-noise formula.

7.18 Regularization and selection among fitting solutions

When a realizable overparameterized model has many fitting solutions, regularization can prefer some of them. Consider

$$\widehat{\mathcal{L}}_\gamma(\theta) = \widehat{\mathcal{L}}(\theta) + \gamma\Omega(\theta), \qquad \gamma > 0,$$

with a nonnegative penalty. Ordinary weight decay uses $\Omega(\theta) = \frac{1}{2}\|\theta\|_2^2$; other choices include the multiplicative norms discussed in Chapters 5 and H.

The constrained problem

$$\min_{\theta \in \mathcal{M}} \Omega(\theta) \tag{7.40}$$

is a useful small-regularization idealization. It is not an assertion that the actual optimizer first reaches $\mathcal{M}$ and only then regularizes. Both gradients can act from the beginning, and finite γ generally changes the best training fit. In particular, a minimum of the full objective need not be a common minibatch equilibrium. Proposition 5 describes its possible nonzero covariance, while Lemma 3 gives an architecture-specific obstruction to all batches stopping at once.

A precise limiting interpretation. Suppose a global minimizer θ_γ of $\widehat{\mathcal{L}}_\gamma$ exists. For every $\bar{\theta} \in \mathcal{M}$,

$$\widehat{\mathcal{L}}(\theta_\gamma) + \gamma\Omega(\theta_\gamma) \le \gamma\Omega(\bar{\theta}).$$

Since both terms on the left are nonnegative,

$$\widehat{\mathcal{L}}(\theta_\gamma) \le \gamma\Omega(\bar{\theta}), \qquad \Omega(\theta_\gamma) \le \Omega(\bar{\theta}).$$

If a sequence $\gamma \downarrow 0$ has a convergent subsequence of these minimizers and the functions are continuous, its limit belongs to $\mathcal{M}$ and minimizes Ω there. Existence and compactness of such minimizers, and an algorithm that finds them, are separate requirements. This elementary comparison explains the scope of (7.40) without identifying it with the behavior of arbitrary regularized SGD.

A different role: confinement of analytic dynamics. For a nonnegative empirical loss analytic on $\mathbb{R}^p$, consider

$$U(\theta) = \widehat{\mathcal{L}}(\theta) + \frac{\gamma}{2}\|\theta\|^2, \qquad \gamma > 0,$$

where the quadratic term includes *all* trainable parameters. Along gradient flow of U,

$$U(\theta_t) \le U(\theta_0), \qquad \|\theta_t\|^2 \le \frac{2U(\theta_0)}{\gamma}.$$

The trajectory stays in a compact sublevel set and exists for all time. The analytic result in Section 7.9 therefore gives finite-length convergence to a critical point of U. This is Corollary 8 in Appendix N. It is neither a global optimization theorem nor an interpolation guarantee; finite regularization can trade a nonzero residual against a smaller penalty.

A product penalty need not provide this compactness. For example,

$$\widehat{\mathcal{L}}(a, b) = \tfrac{1}{2}(ab - 1)^2, \qquad \Omega(a, b) = a^2b^2$$

gives $\widehat{\mathcal{L}}(t, t^{-1}) + \gamma\Omega(t, t^{-1}) = \gamma$ for every $t > 0$. The sublevel set is unbounded. This does not prove that its trajectories diverge; it shows that confinement does not follow from the penalty alone. Products of squared Frobenius norms can be analytic without being coercive. Unsquared norm products can also lose analyticity at zero factors.

The fitting dynamics must use the actual objective. For differentiable Ω, gradient flow of U_γ satisfies

$$\dot{e} = -Ke - \gamma J\nabla\Omega. \tag{7.41}$$

Thus the unregularized contraction estimate cannot be transferred to penalized training without controlling the additional term. At an exact fit the data gradients vanish, whereas the penalty gradient need not. At a regularized stationary point the average data gradient balances the penalty, although individual sample gradients may differ. The same stationary mean balance can therefore correspond to deterministic convergence of gradient flow and continuing fluctuations of fixed-step minibatch SGD.

The relevant geometric picture is motion in a well-fitting region of the regularized objective, not automatically motion constrained to the unregularized zero-loss manifold. This distinction permits the intrinsic noise mechanism without claiming that every fluctuation is tangent to that manifold or that the regularized minimizer set has dimension $p - q$.

Entropic and norm preferences are also different:

equilibrium volume weighting:	depends on the specified measure,
norm selection:	prefers lower penalty at comparable fit.

They need not favor the same region. Neither preference alone identifies the generating internal variables, and predictive learning does not require that identification.

7.19 Why norm selection can be statistically meaningful

A parameter count or a parameter-space volume is not, by itself, the statistical complexity needed to control population prediction. A positive route is to use a suitable norm-sensitive hypothesis class.

Fix an admitted architecture and a complexity functional $\mathcal{C}(\theta)$. For a budget Q chosen independently of the training sample, define

$$\mathcal{H}_Q = \{F_\theta : \mathcal{C}(\theta) \le Q\}.$$

In an appropriate homogeneous feedforward setting, a candidate quantity is

$$\mathcal{C}_\times(\theta) = \prod_{\ell=1}^{L} \|W_\ell\|_F, \qquad \Omega_\times(\theta) = \mathcal{C}_\times(\theta)^2.$$

Biases, other trainable operations, activation assumptions, depth factors, and any further terms required by the chosen statistical theorem must be included. The bare product is not a universal bound for every network.

The meaningful statistical assertion is a bound for the whole class, not an expression evaluated on one isolated parameter vector:

$$\widehat{\mathfrak{R}}_{S_X}(\mathcal{H}_Q) \le \mathcal{B}_n(Q; \text{architecture and input assumptions}).$$

Under the bounded-loss and Lipschitz hypotheses stated in Chapter H, a corresponding uniform bound takes the form

$$R_{\mathsf{P}}(h) \le \widehat{R}_S(h) + \beta_S(Q, \delta) \qquad \text{for every } h \in \mathcal{H}_Q \tag{7.42}$$

on an event of probability at least $1 - \delta$. Squared loss needs appropriate prediction and label bounds, or a different applicable statistical argument.

On this uniform event, any returned predictor satisfying $\widehat{f} \in \mathcal{H}_Q$ and $\widehat{R}_S(\widehat{f}) \le \eta$ obeys

$$\boxed{R_{\mathsf{P}}(\widehat{f}) \le \eta + \beta_S(Q, \delta).} \tag{7.43}$$

Interpolation is the special case $\eta = 0$. The bound must be small at the available sample size, and training must actually meet the fit and complexity budget. Sample-dependent choices of the

budget or architecture require an appropriate simultaneous bound or model-selection adjustment. Chapter H states the joint statistical and optimization success probability explicitly.

A multiplicative penalty can favor a smaller product among comparably fitting solutions and thereby reduce an applicable complexity upper bound. It does not literally minimize exact Rademacher complexity, nor imply a universal monotone relation with test error. Reciprocal scaling of two adjacent positively homogeneous layers changes their individual norms but preserves their Frobenius product; that particular symmetry does not invalidate a correctly formulated product-norm analysis.

Three controls must nevertheless be kept separate: an upper bound on statistical class complexity, confinement of the parameter trajectory, and a lower bound on the residual Rayleigh quotient. The first does not imply either of the other two. In particular, an upper matrix-norm bound does not rule out rank deficiency. Likewise, a statistical theorem that assumes positive homogeneity cannot be transferred unchanged to GELU, which is not positively homogeneous. The applicable activation and architecture assumptions must be verified, as emphasized in Chapter H.

Most importantly,

norm control can support predictive generalization without identifying the generating computation.

This is the relevant statistical selection question for the next chapter. Population accuracy and structural identification should not be merged into a requirement that the chosen interpolant have the teacher's internal factorization.

7.20 An idealized theorem

The regular local-weight comparison can be stated and proved precisely. The neighborhoods in the theorem are fixed before the temperature tends to zero.

Theorem 6 (Low-temperature dominance of a regular interpolating manifold)**.** *Let $L : \mathbb{R}^p \to \mathbb{R}_{\geq 0}$ be smooth. Suppose:*

1. *The zero-loss set contains a nonempty compact smooth embedded manifold $\mathcal{M}$ without boundary, of codimension q, where $1 \leq q \leq p$. On its Euclidean normal spaces the Hessian $H_\perp(s)$ is uniformly positive definite. Choose a sufficiently small fixed tubular neighborhood $U_{\mathcal{M}}$ with no additional zeros.*
2. *Each competitor θ_ℓ under consideration is an isolated nondegenerate local minimum, with $L(\theta_\ell) = L_\ell > 0$ and $H_\ell = \nabla^2 L(\theta_\ell) \succ 0$. Choose a sufficiently small fixed neighborhood U_ℓ on which it is the unique minimizer.*
3. *For the interpretation as equilibrium probabilities, the Langevin model $d\theta_t = -\nabla L(\theta_t)\, dt + \sqrt{2T}\, dW_t$ has the normalizable stationary density $Z_T^{-1} e^{-L/T}$ at the temperatures under consideration.*

Define the local contributions

$$Z_{\mathcal{M}}(T) = \int_{U_{\mathcal{M}}} e^{-L(\theta)/T}\, d\theta, \qquad Z_\ell(T) = \int_{U_\ell} e^{-L(\theta)/T}\, d\theta.$$

Then, as $T \downarrow 0$,

$$Z_{\mathcal{M}}(T) = (2\pi T)^{q/2} \left[\int_{\mathcal{M}} \frac{dV(s)}{\sqrt{\det H_\perp(s)}} + o(1) \right], \tag{7.44}$$

$$Z_\ell(T) = e^{-L_\ell/T} (2\pi T)^{p/2} \left[\frac{1}{\sqrt{\det H_\ell}} + o(1) \right]. \tag{7.45}$$

Consequently,

$$\boxed{\frac{Z_\ell(T)}{Z_{\mathcal{M}}(T)} = O\Big(e^{-L_\ell/T} T^{(p-q)/2}\Big) \longrightarrow 0.} \tag{7.46}$$

The same ratio equals $\rho_T(U_\ell)/\rho_T(U_{\mathcal{M}})$ under assumption 3.

Proof. Use Euclidean normal coordinates in the tube around $\mathcal{M}$, locally on the normal bundle. Smoothness and compactness give the uniform expansion

$$L(s,z) = \tfrac{1}{2} z^\top H_\perp(s) z + O(\|z\|_2^3), \qquad d\theta = (1 + O(\|z\|_2))\, dz\, dV(s).$$

Uniform normal positivity permits the tube to be chosen so that $L(s,z) \geq c\|z\|_2^2$ for a constant $c > 0$. After the change of variables $z = \sqrt{T}\, w$, the integrand, including the volume density, converges to

$$\exp\Big(-\tfrac{1}{2} w^\top H_\perp(s) w\Big)$$

and is bounded by an integrable Gaussian uniformly in s. Local charts and a partition of unity cover the compact manifold. Dominated convergence therefore gives

$$T^{-q/2} Z_{\mathcal{M}}(T) \longrightarrow \int_{\mathcal{M}} \int_{\mathbb{R}^q} e^{-w^\top H_\perp(s) w/2}\, dw\, dV(s) = (2\pi)^{q/2} I_{\mathcal{M}}.$$

The constant $I_{\mathcal{M}}$ is finite and strictly positive.

At θ_ℓ, write $\theta = \theta_\ell + \sqrt{T}\, w$ and apply the same argument to

$$L(\theta_\ell + u) = L_\ell + \tfrac{1}{2} u^\top H_\ell u + O(\|u\|_2^3).$$

On a sufficiently small neighborhood, $L(\theta_\ell + u) - L_\ell \geq c_\ell \|u\|_2^2$, which supplies Gaussian domination. Thus

$$e^{L_\ell/T} T^{-p/2} Z_\ell(T) \longrightarrow \frac{(2\pi)^{p/2}}{\sqrt{\det H_\ell}}.$$

Dividing these asymptotics proves (7.46). For the stationary law, the common normalizing constant cancels in the ratio of neighborhood probabilities. □

This theorem compares a specified regular global component with each specified isolated positive-loss competitor. For finitely many such competitors, the equilibrium probability conditioned on the union of these neighborhoods concentrates on $U_{\mathcal{M}}$. No corresponding claim about the entire parameter space follows without controlling the remaining regions and other global components.

The theorem does not assert that arbitrary SGD has this stationary law, that a fixed-temperature trajectory converges to the exact manifold, or that reaching a low-loss neighborhood takes polynomial time. The local asymptotics themselves do not require a global stationary law; that law is needed only for the stated equilibrium interpretation.

7.21 The conjectural extension to deep-network SGD

The theorem motivates, but does not prove, a broader dynamical hypothesis. Its status is best made explicit.

> **Entropic Interpolation Conjecture—a programmatic form.** For suitable specified families of realizable overparameterized deep networks, the geometry of low-loss sets and the actual stochastic training dynamics combine to make near-interpolation reachable with high probability and feasible resources. The proposed mechanism is a favorable weighting and accessibility of degenerate low-loss regions, not merely an increase in raw parameter count.

A theorem-strength version must name the target and data class, the architecture family, initialization law, minibatch rule, step sizes, regularizer, and stopping criterion. For example, with a problem-size index N, empirical-loss tolerance $\xi > 0$, and failure probability δ, it would need a statement of the form

$$\Pr\Big\{\widehat{\mathcal{L}}_N(\theta_{k_\star}) \le \xi\Big\} \ge 1 - \delta,$$

where $k_\star$, network description, arithmetic cost, and numerical precision satisfy explicit resource bounds. An exponential inverse conditioning parameter or an exponential equilibration time would not establish efficient learning. The chapter supplies no such general finite-time bound for deep-network SGD.

The output can be a specified stopped iterate; it need not be an absorbing point shared by every minibatch. If an averaged parameter or averaged predictor is returned, its representational and evaluation costs must also be counted. Positive weight decay and the target tolerance must be chosen compatibly, since the penalized optimum may trade fit against penalty. Persistent gradient-estimator variance does not preclude a finite-time prediction guarantee, but its scale and the learning-rate schedule must enter the analysis.

Three logically distinct steps must be separated.

Geometric step. At an existing regular interpolating point, locally constant rank q gives dimension $p-q$. At singular points, local geometry must be analyzed rather than inferred from Hessian nullity. Large dimension alone is not a measure of dynamically relevant volume.

Equilibrium step. For a specified normalizable Langevin model, neighborhood probabilities are integrals of $e^{-L/T}$ against its reference measure. The regular and singular calculations estimate such integrals under their respective hypotheses. They do not determine finite-time access.

Dynamical step. One must show that the actual initialization and optimization procedure reach the relevant low-loss region on the claimed timescale, and that its noise model is appropriate. Equilibrium preference does not imply a mixing-time or hitting-time bound. Letting $T \downarrow 0$ before obtaining such a bound can make access, rather than equilibrium weight, the limiting issue.

These steps cannot be collapsed into an unconditional arrow from $p \gg q$ to successful SGD. The conjecture remains a mechanism-oriented research program. Other optimization mechanisms can establish successful fitting without validating an entropic explanation.

7.22 What the argument does not say

The geometric picture is useful when its limits remain visible in the conclusions as well as in the assumptions.

7.22.1 It does not make the loss convex

The loss can retain saddles, local minima, and complicated critical sets. The regular-manifold theorem describes neighborhoods of solutions whose existence has been assumed. It does not eliminate all competing critical points. The deterministic contraction argument instead rules out positive-loss stationary points in a region where its quantitative condition holds.

An analytic counterexample to an almost-everywhere rank argument. For every $p \geq 2$, Appendix N, Proposition 42, considers

$$r(\theta) = \theta_1^3 - 3\theta_1 + 3 + \sum_{j=2}^{p} \theta_j^2, \qquad L(\theta) = \tfrac{1}{2} r(\theta)^2. \tag{7.47}$$

This analytic scalar-residual problem depends on every parameter and has a nonempty regular zero-loss manifold of dimension $p - 1$. Its prediction Jacobian has full row rank except at the two points $(\pm 1, 0, \ldots, 0)$. Nevertheless, at $\theta_+ = (1, 0, \ldots, 0)$,

$$L(\theta_+) = \tfrac{1}{2}, \qquad \nabla L(\theta_+) = 0, \qquad \nabla^2 L(\theta_+) = \operatorname{diag}(6, 2, \ldots, 2) \succ 0.$$

A small strongly convex neighborhood is attracted to this positive-loss minimum. The appendix proves these claims and the existence of an open suboptimal basin. Thus

analyticity, many parameters, and regular interpolation
$+$ full prediction rank almost everywhere
$\not\Longrightarrow$ almost-sure convergence to interpolation.

The exceptional critical parameters can have measure zero while their basin has positive measure. This is a counterexample to a general analytic least-squares inference, not a claim that a prescribed unrestricted GELU architecture has exactly this parameterization. Architecture-specific hypotheses may exclude the obstruction; parameter count and analytic regularity alone do not.

7.22.2 It does not prove that every degenerate set has a large basin

Degeneracy is local. Basin size depends on the vector field and on the initialization measure. A residual-contraction theorem needs a region containing the trajectory, not merely the Hessian at the eventual minimum. For stochastic dynamics, a stationary mass estimate still needs a finite-time accessibility argument.

7.22.3 Suboptimal sets can also be degenerate

Architectural symmetries and redundancies can occur at positive-loss critical sets. The relevant comparison includes loss, normal curvature, dimension or singular exponent, local weighting, and actual dynamics. It is not simply "degenerate global" versus "nondegenerate local." The isolated competitors in Theorem 6 are an explicitly restricted case.

7.22.4 Raw Euclidean volume is parameterization dependent

Changing coordinates can alter Euclidean volume and Hessian coordinates. A probability statement must transform the reference density consistently. Holding an isotropic noise rule fixed after a

nonlinear reparameterization generally defines a different stochastic process, not the same process in new notation.

The relevant measure should therefore be tied to the specified initialization, dynamics, prior, or regularizer. A broad parameter valley is not automatically a low-complexity function class, and none of these measures identifies semantic variables without extra assumptions.

7.22.5 The zero-loss set can be noncompact

Scale symmetries can produce noncompact families. Then an unweighted volume integral may diverge, and the unregularized Gibbs law may fail to normalize. A compact localization is sufficient for a local Laplace calculation but is not by itself a global equilibrium model.

A prior-weighted integral, a confining regularizer, a bounded-domain process, or a quotient by exact symmetries can be appropriate. They are different constructions and must be accompanied by their own measure and dynamical assumptions. A norm product that is invariant along a noncompact rescaling orbit need not, by itself, confine that orbit.

7.22.6 Other losses require modified geometry

Squared loss has a finite zero-loss solution in the realizable regime assumed here. For separable classification with unregularized logistic or cross-entropy loss, the infimum can occur only as parameter norms diverge. One then studies asymptotic directions, margins, or a regularized objective rather than a finite unregularized zero-loss manifold.

Low classification error, small surrogate loss, and exact squared-loss interpolation are also different criteria. A predictive framework can use any appropriate loss and statistical bound; the present geometric identities should not be transferred unchanged between them.

7.23 Why overparameterization can make optimization easier

More parameters enlarge the search domain, but they can also enlarge the set satisfying the fitting constraints. If regular realizations persist and their prediction rank stays fixed at q while p increases, the local fitting dimension $p - q$ increases. This is the geometric reversal behind the chapter.

The reversal is not a complexity comparison by itself. New parameters may improve prediction conditioning, introduce useful alternative paths, or merely duplicate irrelevant directions. Only the first kinds of change can help establish an accessibility bound.

An elementary check illustrates the limit of dimension counting. Append parameters $z \in [-1, 1]^r$ that do not affect the loss:

$$\widetilde{L}(\theta, z) = L(\theta).$$

With product Lebesgue measure, every local partition contribution acquires the same factor 2^r:

$$\widetilde{Z}_U(T) = 2^r Z_U(T).$$

All relative equilibrium weights are unchanged, and the gradient dynamics of θ are unchanged. The fitting set has extra neutral directions, but these dummy coordinates do not make the original optimization problem easier. They enlarge suboptimal sets as well.

The meaningful claim is therefore

overparameterization can enlarge the fitting set while improving the prediction dynamics that lead to it.

The first effect is a regular dimension count. The second requires an architecture- and data-dependent argument. Neither must recover a privileged internal basis to contribute to predictive learning.

7.24 Three levels of explanation

Three levels remain useful, provided their conclusions are not conflated.

Level 1: existence and local dimension. In a realizable model, a regular zero-loss point with local rank q belongs to a fitting manifold of dimension $p - q$. The squared-loss Hessian has exactly $p - q$ zero modes at that point. This is the clean algebraic and geometric statement.

Level 2: accessibility within a resource budget. The optimizer must reach an adequately low-loss region with a stated success probability and cost. Uniform residual contraction on a high-probability invariant region supplies one conditional deterministic route. A Langevin equilibrium calculation supplies a different, stationary weighting result; an additional dynamical argument is needed to turn it into finite-time access. Neither follows from Level 1 alone.

For an analytic objective, compactness can instead establish convergence to one critical point, as in (7.17). That positive endpoint result does not determine whether the attained loss is adequate or whether the time and precision costs are feasible.

Level 3: predictive selection. The returned predictor must be controlled well enough to generalize. An informative norm-sensitive class bound, together with a sufficiently good fit, is one route. The selected internal variables need not match the teacher's. Identifying those variables is a separate structural objective, not part of the definition of predictive Level 3. Low-rank selection can be investigated within a suitable norm- and rank-sensitive class; a rank obstruction to common minibatch equilibrium is not, by itself, a population-error bound.

This chapter establishes the regular local identities, a conditional contraction argument, and the idealized equilibrium comparison. It also summarizes the complementary analytic convergence result proved in Appendix N. It motivates a finite-time stochastic extension and connects selection to the statistical argument developed in Chapter H. It does not supply a universal passage through all three levels.

7.25 The convergence backstop, with its conditions explicit

The term *backstop* should refer to a supported route toward fitting, not to a claim that geometric abundance rescues every training problem. The analytic endpoint result should first be distinguished from the two arguments about fitting dynamics and equilibrium mass.

The analytic endpoint.

$$\boxed{\begin{array}{c}\text{an analytic objective and an all-time gradient flow}\\ + \text{ compact trajectory closure inside its domain}\\ \Longrightarrow \text{ finite-length convergence to a critical point.}\end{array}}$$

Neither a globally fitting limit nor a useful convergence-time bound is part of this conclusion. Full quadratic weight decay is one sufficient source of confinement for the analytic squared-loss setting; a product penalty alone need not supply it.

The deterministic fitting line.

$$\boxed{\begin{array}{c}\text{initialization falls in an invariant conditioning region}\\ +\, e^\top K e \geq \mu\|e\|_2^2 \text{ along the trajectory}\\ \Longrightarrow \quad \widehat{\mathcal{L}}(\theta_t) \leq \widehat{\mathcal{L}}(\theta_0)e^{-2\mu t}.\end{array}}$$

A probability estimate for initialization then gives a conditional high-probability fitting statement. Finite-step and bit-complexity claims require the corresponding smoothness, invariance, and precision bounds.

The equilibrium line.

$$\boxed{\begin{array}{c}\text{a compact regular zero-loss manifold and local competitors}\\ +\, \text{the specified normalizable Langevin equilibrium}\\ \Longrightarrow \quad \dfrac{\rho_T(U_\ell)}{\rho_T(U_{\mathcal{M}})} = O\Big(e^{-L_\ell/T} T^{(p-q)/2}\Big).\end{array}}$$

This is a relative stationary mass estimate. A suitable singular asymptotic can replace the regular Laplace factor when its hypotheses apply. Neither version includes a bound on the time needed to visit the favored region.

These results can be complementary. For example, global access and local residual contraction address different stages of a possible proof, while analyticity can settle the behavior of a confined trajectory. But their hypotheses must be verified; combining their conclusions in a diagram does not verify them.

The regularized stochastic endpoint. Section 7.17 adds a different exact distinction:

$$\nabla U_\gamma(\theta_\gamma) = 0 \quad \not\Longrightarrow \quad \mathrm{Cov}_{\mathcal{B}}(G_{\mathcal{B},\gamma}(\theta_\gamma)) = 0.$$

For proper minibatches, differing per-example gradients make this covariance positive in trace. The normalized ReLU rank restriction gives one mechanism excluding a common equilibrium in a higher-rank region. It does not itself supply a global fitting theorem or a Gibbs law.

Regularization provides a selection pressure and changes the balance supporting both mean drift and sample-dependent fluctuations. To connect this dynamics to prediction, one needs a returned iterate with adequate empirical fit and an informative complexity budget. A common zero of all minibatch updates is not an additional requirement. That is the transition to Chapter H.

7.26 Interpolation, prediction, and structural recovery

The distinction at the end of the chapter is now threefold:

empirical fitting:	$\widehat{R}_S(\widehat{f})$ is small,
predictive recovery:	$R_{\mathsf{P}}(\widehat{f})$ is small,
structural recovery:	specified hidden structure is identified.

A model can interpolate a finite training sample and fail to predict on the population. Memorization or a sample-specific shortcut can produce that failure. A statistical argument is needed to exclude it in the setting of interest.

By contrast, using a different internal basis, merging several teacher constituents into one learned module, or distributing a teacher variable across several learned features is not necessarily a failure. Such a network can be an accurate and feasible predictor even without a useful nodewise correspondence to the generating DAG.

There are therefore two different nonimplications:

$$\text{interpolation} \not\Longrightarrow \text{population accuracy},$$

$$\text{population accuracy} \not\Longrightarrow \text{generating-structure identification}.$$

The first is a gap that predictive learning must address. The second marks a stronger objective that need not be imposed on prediction.

If scientific interpretation, causal identification, or a particular module-reuse claim requires latent structure, it needs an explicit structural criterion and identifiability conditions. Those questions remain important. They should not be inserted as an extra requirement after a feasible predictor has already been shown to generalize.

7.27 The bridge to predictive learning

Chapter H organizes the predictive synthesis around three questions:

Q1 : Does the chosen class contain a feasible accurate predictor?

Q2 : Can the specified training procedure find a suitable fit?

Q3 : Is the selected predictor controlled enough to generalize?

Chapter 4 supplies a representation principle relevant to Q1, subject to the architecture and finite-precision qualifications. Chapters 5 and 6 study learning signals, intermediate supervision, and conditioning relevant to Q2. The present chapter contributes the local fitting geometry, the conditional residual-contraction route, and the idealized equilibrium volume mechanism. Appendix N additionally explains when a bounded analytic flow has a definite stationary endpoint. It does not guarantee that the endpoint fits the data. Q2 still requires a quantitative argument for the specified training procedure.

For Q3, suppose a valid class bound applies and training returns $\widehat{f} \in \mathcal{H}_Q$ with $\widehat{R}_S(\widehat{f}) \le \eta$. On the corresponding uniform event,

$$R_{\mathsf{P}}(\widehat{f}) \le \eta + \beta_S(Q, \delta).$$

The algorithmic probability of reaching that joint fit and complexity condition is a separate premise. Chapter H combines the two events explicitly. The returned predictor may be a specified stopped iterate even when fixed-step SGD continues to fluctuate; all confidence and complexity conditions still have to hold for that selection. A low-loss region favored at equilibrium does not supply the statistical complexity bound, and a small penalty written in an objective does not prove that the optimizer meets its required budget.

Once population accuracy and computational feasibility have been established, the learned network itself supplies a compositional realization of its predictor. For example, in noiseless regression, assume

$$\|\widehat{f} - f^\star\|_{L^2(P_X)} \le \varepsilon$$

and that a specified feasible numerical evaluator produces $\widetilde{f}_m$ with $\|\widetilde{f}_m - \widehat{f}\|_\infty \le 2^{-m}$. Compilation of that evaluator into bounded-fan-in operations gives a computationally compositional implementation, and the triangle inequality yields

$$\boxed{\|\widetilde{f}_m - f^\star\|_{L^2(P_X)} \le \varepsilon + 2^{-m}.}$$

This is the compositional-witness argument of Chapter H. Description length, input encoding, evaluation time, and precision must scale within the stated resource bounds; being finite is not enough for an asymptotic efficiency claim. The digital implementation also does not automatically supply a fixed coarse-grained Sobolev decomposition with uniform regularity constants.

Genericity and staircase learning can help establish how a specified algorithm obtains an accurate predictor. A staircase theorem may also prove structural recovery. Faithful or partial traces can provide a different useful route. Neither a staircase nor teacher-node matching is a separate logical premise of the witness implication after predictive success has been established.

The next chapter therefore asks why training finds a *useful feasible predictor*, rather than why every successful solution must reconstruct the computation that generated the observations.

7.28 Summary

Overparameterization can enlarge both parameter space and the fitting set. At an existing interpolating point with locally constant prediction rank q, the fitting manifold has dimension $p - q$. At any twice-differentiable interpolating point,

$$\boxed{\nabla^2 \widehat{\mathcal{L}}(\theta^\star) = \frac{1}{n} J^\top J, \qquad \dim\ker \nabla^2 \widehat{\mathcal{L}}(\theta^\star) = p - q.}$$

At regular points the zero modes are tangent to the fitting manifold. At singular points, they do not determine the geometry by themselves.

Analyticity supplies a separate result: an all-time gradient flow with compact closure inside the analytic domain converges with finite length to one critical point. That point need not interpolate, and the analytic constants need not yield an efficient rate. The counterexample in Section 7.22 shows why even almost-everywhere prediction rank does not promote this result to global fitting.

A deterministic convergence statement requires information along the trajectory. Uniform contraction of the actual residual gives

$$\widehat{\mathcal{L}}(\theta_t) \le \widehat{\mathcal{L}}(\theta_0) e^{-2\mu t}.$$

A high-probability invariant initialization region converts that estimate into a conditional convergence guarantee. The chapter does not deduce such a region from raw width alone.

The stochastic calculation is different. In the specified normalizable Langevin equilibrium, a regular zero-loss manifold contributes a local weight proportional to $T^{q/2}$ times its integrated inverse normal curvature, while an isolated positive-loss minimum contributes $e^{-L_\ell/T} T^{p/2}$ times its curvature factor. The resulting relative mass estimate is explicit. Suitable singular asymptotics replace these powers when their analytic hypotheses hold.

These stationary comparisons are not finite-time SGD theorems. Basin geometry, equilibration, state-dependent noise, and the behavior of prefactors across network sizes remain separate issues. The entropic interpolation conjecture concerns that dynamical extension and is not established merely by dimension counting.

For real minibatch training, exact interpolation and regularized stationarity have different consequences. At an exact fixed-example squared-loss interpolator the sampling covariance is zero. At a regularized stationary point, only the average data gradient must balance the penalty; sample gradients can differ and the update covariance can be nonzero. The convex example proves that this phenomenon does not require nonconvexity. In the stated normalized ReLU model, the common-equilibrium rank bound supplies a further obstruction and motivates low-rank bias. Its hypotheses, and the distinction between exact rank and effective rank, remain essential.

Regularization can prefer predictors with smaller applicable norm complexity. It can also confine analytic gradient flow when the specified penalty is coercive, but statistical control and confinement are different properties. When a training procedure actually achieves adequate fit within a class having an informative statistical bound, it can yield small population error. This selection need not reproduce the teacher's internal variables or a preferred generating factorization.

A small mean gradient does not mean that every minibatch update is small. Fixed-step point convergence, diminishing-step convergence, and successful selection of a finite-time predictor are different claims. Intrinsic noise can persist in a regularized well-fitting region without either invalidating a predictive guarantee or establishing one automatically.

Fitting geometry describes available solution sets; optimization must establish feasible access; statistical selection must control population prediction. When the last two succeed and evaluation is feasible, the learned computation itself is a compositional witness. Structural identification is a stronger question, not an extra condition for that conclusion.

The remaining predictive challenge, developed in Chapter H, is

> When does the specified regularized learning procedure find,
> with high probability and feasible resources,
> a sufficiently well-fitting predictor that generalizes?

CHAPTER 8

Principles of Deep Learning

The preceding chapters developed a representation principle and several possible mechanisms for learning. These should be related without being identified with one another.

The representation principle is that efficiently computable targets admit, at the relevant finite precision, polynomial-size bounded-fan-in realizations. In the computational sense used in this book, efficient computation implies compositional sparsity.

The learning question is different. A specified optimization procedure must find a sufficiently accurate predictor, and the selected predictor must have statistical complexity small enough for the available data to support generalization. Overparameterization, stochastic dynamics, and regularization offer complementary ways to approach these requirements. They do not establish them automatically.

Chapter 7 and Appendix N distinguish convergence to a critical point, quantitative contraction of training error, and low-temperature equilibrium weighting. These conclusions play different roles in the learning argument. None alone establishes predictive success. Regularized stationarity can also coexist with intrinsic minibatch fluctuations. Predictive success concerns the model returned by training, not convergence to a common zero of every minibatch update.

Suppose training succeeds: it returns a computationally feasible deep network with small population error. That network already supplies a compositional approximation of the target. Its intermediate variables need not reproduce the variables that generated the observations. Indeed, predictive success does not require an additional theorem matching the learned network to any particular generating decomposition.

The central argument is therefore conditional:

training finds a feasible predictor with good empirical fit
and an informative statistical-complexity bound

$\Downarrow$

small population error

$\Downarrow$

the learned computation is a compositional witness.

The difficult premise is the joint achievement of good fit, suitable complexity control, and feasible training cost. Genericity, staircase learning, skip connections, and trace exposure remain important candidate mechanisms for establishing that premise. They are not separate necessary conditions for the final witness implication.

The distinction organizing this chapter is thus not only representation versus learning, but also predictive recovery versus structural recovery. The first seeks a useful input–output computation. The second seeks specified internal structure and is a stronger objective.

8.1 The central simplification

Let

$$f : \mathcal{X} \longrightarrow \mathcal{Y}$$

be the target. Suppose training returns a depth-L network

$$F_\theta = h_L \circ h_{L-1} \circ \cdots \circ h_1$$

that approximates f on the population distribution of interest. The learned network already supplies a hierarchical computation of the target approximation.

There is no requirement that h_ℓ coincide with a privileged or "true" latent constituent. If the target admits a generating factorization

$$f = g_m \circ \cdots \circ g_1,$$

training may discover an entirely different factorization. For predictive learning, that is acceptable when the resulting function is accurate and computationally feasible.

8.1.1 Predictive recovery

Let P denote the joint distribution of (X, Y), with input marginal P_X, and let ℓ be a nonnegative prediction loss. Write

$$R_P(F) = \mathbb{E}_P[\ell(F(X), Y)], \qquad \widehat{R}_S(F) = \frac{1}{n}\sum_{i=1}^{n} \ell(F(X_i), Y_i). \tag{8.1}$$

The sample $S = ((X_i, Y_i))_{i=1}^n$ is drawn independently from P. Predictive recovery means that $R_P(F_\theta)$ is suitably small, or that its excess over an appropriate reference risk is small in a noisy problem.

For noiseless regression, $Y = f(X)$, with $\ell(\widehat{y}, y) = \frac{1}{2}(\widehat{y} - y)^2$, this becomes

$$R_P(F_\theta) = \frac{1}{2}\|F_\theta - f\|^2_{L^2(P_X)}. \tag{8.2}$$

The distribution and loss are part of the statement. Good performance under P does not by itself assert correctness under a different observation model.

8.1.2 Structural recovery

A structural objective additionally compares the learned internal variables with a specified teacher DAG. A possible criterion is

$$\mathcal{D}_{\mathrm{str}}(\widehat{z}, z) = \inf_{\substack{\pi \text{ admissible} \\ \psi_v \in \mathcal{E}_v}} \max_v \big\|\widehat{z}_{\pi(v)} - \psi_v(z_v)\big\|_{L^2(P_X)}. \tag{8.3}$$

Here the matching respects the allowed graph structure, the feature slots have controlled dimension, and the equivalences $\mathcal{E}_v$ have fixed complexity and conditioning bounds. Otherwise an unrestricted decoder could hide the entire original prediction problem inside the recovery criterion.

Small prediction error does not establish this structural conclusion. Conversely, failure to match the generating variables does not imply predictive failure. The question needed for the main argument is

Did learning find a feasible function that predicts f accurately?

It is not necessary to add a requirement that this function reproduce a particular hidden computation.

8.2 A successful finite network is itself a computation

A neural network evaluated at finite precision is a digital computation. A dense affine unit

$$z_j = \sigma\left(\sum_i W_{ji} x_i + b_j\right)$$

may have large functional fan-in as written. But the weighted sum can be evaluated by bounded-fan-in additions and multiplications, for example by a binary reduction tree. Thus a polynomial-size dense layer has a polynomial-size bounded-fan-in implementation, provided the arithmetic and activation evaluations use controlled resources.

Composing such layers gives a bounded-fan-in computation DAG. This is a statement about the implementation; it does not say that the displayed weight matrices must themselves be sparse.

8.2.1 Finite is not the same as feasible

For a family of problems indexed by a natural size parameter N, the relevant resources include architecture description, depth, width, stored parameter bits, intermediate numerical precision, and forward evaluation time. Input encoding and output decoding must also be counted.

To call the learned realization feasible, these resources must have the claimed polynomial bounds. Merely observing that every individual trained network is finite is not enough. A family of finite networks can still have exponential width, exponential running time, or prohibitive precision requirements.

Likewise, efficient evaluation of a returned network does not imply that finding its parameters was efficient. Training cost and evaluation cost are separate quantities.

8.2.2 Learning accuracy and evaluation precision

It is useful to distinguish the target learning error ε from the numerical error 2^{-m} in evaluating a fixed predictor. A learning theorem may allow sample size and model size polynomial in ε^{-1}, while the evaluator of that returned model uses time polynomial in its description length and in m.

Such a theorem does not assert that learning costs only $\mathrm{poly}(\log(1/\varepsilon))$. Throughout the chapter, a polynomial-resource claim is relative to its stated parameters.

Subject to these conventions, the central implication is

F_θ predicts the target accurately
$+$ F_θ has a feasible computational implementation
$\Longrightarrow$
a feasible compositionally sparse predictor.

8.3 Efficient computation and compositional sparsity

For discrete computation, the basic implication is the one developed in Chapter 4. If a deterministic Turing machine evaluates a target family in time $T(N) = \mathrm{poly}(N)$, its computation can be simulated by a polynomial-size bounded-fan-in Boolean circuit. The construction is uniform when the evaluator is uniform.

Schematically,

$$\mathbf{P} \subseteq \{\text{uniform polynomial-size bounded-fan-in realizations}\}.$$

For real-valued targets, a precision convention is essential.

Definition 7 (Efficiently Turing-computable real function family)**.** *Let*

$$f_N : [0,1]^{d_N} \longrightarrow \mathbb{R}^{r_N},$$

where d_N and r_N are polynomially bounded in N. The family is efficiently Turing computable *under the present precision convention if there is one deterministic algorithm M and polynomial bounds p_{in} and p_{time} with the following property.*

For every N, every $m \geq 1$, and every $x \in [0,1]^{d_N}$, a dyadic approximation $\widetilde{x}$ with the prescribed polynomial-length encoding and

$$\|x - \widetilde{x}\|_\infty \leq 2^{-p_{\mathrm{in}}(N,m)}$$

is sufficient for M, given $(N, m, \widetilde{x})$, to return a dyadic vector $\widetilde{y}$ satisfying

$$\|\widetilde{y} - f_N(x)\|_\infty \leq 2^{-m}$$

in at most $p_{\mathrm{time}}(N, m)$ bit operations. The guarantee holds for all valid input approximations at that precision, not only for a specially chosen encoding.

For a single target, the family index can be suppressed. This definition makes explicit the two requirements already present in the bit-model formulation:

	polynomial-time finite-precision evaluation
$+$	polynomial control of required input precision.

After binary encoding, each finite-precision evaluation is an ordinary digital computation and admits a bounded-fan-in realization. A finite-precision circuit acts on encoded inputs; interpreting its decoded output as an approximation on a continuous domain additionally uses the input-precision guarantee.

8.3.1 Computational sparsity and regular compositional structure

Two claims must remain distinct:

$$\begin{gathered}\text{efficient digital evaluation} \\ \Longrightarrow \text{a small bounded-fan-in circuit,}\end{gathered}$$

$$\begin{gathered}\text{regular low-arity constituents and stable composition} \\ \Longrightarrow \text{favorable compositional approximation rates.}\end{gathered}$$

The second is the approximation-theoretic statement of Chapter 4. The first does not, by itself, identify a precision-independent coarse DAG, reusable semantic modules, or uniformly bounded Sobolev norms for those modules.

The witness conclusion in this chapter concerns *computational compositional sparsity*. Stronger claims about smooth coarse-grained structure require the corresponding regularity and stability hypotheses. The distinction does not invalidate the witness argument; it specifies what the witness establishes.

8.4 The learned network is a witness

The representation argument applies after training, regardless of how the network was found.

Suppose a returned predictor $F_{\theta^\star}$ has a polynomial-length finite description and a numerically controlled, feasible evaluator. Compiling that evaluator gives a bounded-fan-in realization of the predictor at the requested precision. If the predictor also approximates f on P_X, this realization is a compositional approximation of f.

There is no need to discover a second decomposition behind the learned network. The network, together with its evaluator, already provides one.

The learned computation is the witness.

This is an endpoint statement, not a learning algorithm. It does not prove that SGD found the network in feasible time, that its training error is small, or that its population error is small. Those are the premises that must be established by the optimization and statistical arguments.

Nor does the computational witness show that the selected predictor has small norm complexity. A polynomial-size computational object may still belong to a statistical class too rich for an informative bound at the available sample size.

8.5 Optimization: why interpolation is plausible

Representation does not explain why training reaches a good solution. The geometry and dynamics of Chapter 7 offer several conditional parts of that explanation. Exact interpolation is a useful reference geometry. At fixed positive regularization, however, a well-fitting stationary point may have nonzero data residuals and persistent minibatch fluctuations. These regimes should not be identified. For the geometric discussion, consider smooth squared loss

$$\widehat{\mathcal{L}}(\theta) = \frac{1}{2n}\sum_{i=1}^{n}(F_\theta(x_i) - y_i)^2, \qquad \mathcal{M} = \{\theta : \widehat{\mathcal{L}}(\theta) = 0\}.$$

Assume an interpolating solution exists. Let p be the parameter dimension and let the Jacobian of the training-prediction map have locally constant rank q near a regular interpolating point. Then

$$\dim \mathcal{M} = p - q \tag{8.4}$$

locally. When $p \gg q$, interpolation can therefore occur on extended sets rather than at isolated points.

The existence and regularity assumptions matter. A large parameter count alone does not guarantee realizability, and the rank at one singular point does not determine the dimension of the entire zero-loss set.

8.5.1 Hessian degeneracy

Let $e_i(\theta) = F_\theta(x_i) - y_i$ and let $J(\theta)$ be the Jacobian of the training predictions. Differentiation gives

$$\nabla^2 \widehat{\mathcal{L}}(\theta) = \frac{1}{n} J(\theta)^\top J(\theta) + \frac{1}{n} \sum_i e_i(\theta) \nabla^2 F_\theta(x_i).$$

At a twice-differentiable zero-residual point, the second term vanishes:

$$\boxed{\nabla^2 \widehat{\mathcal{L}}(\theta^\star) = \frac{1}{n} J(\theta^\star)^\top J(\theta^\star).} \tag{8.5}$$

If $\operatorname{rank} J(\theta^\star) = q$, the Hessian has nullity $p-q$. At a regular point these zero modes are tangent to the local interpolation manifold. At a singular point, Hessian nullity need not have that interpretation. For nonsmooth architectures the calculation is restricted to regions where the required derivatives exist.

For a twice-differentiable penalty and $U_\lambda = \widehat{\mathcal{L}} + \lambda\Omega$, the full Hessian is instead

$$\nabla^2 U_\lambda(\theta) = \frac{1}{n} J(\theta)^\top J(\theta) + \frac{1}{n} \sum_i e_i(\theta) \nabla^2 F_\theta(x_i) + \lambda \nabla^2 \Omega(\theta). \tag{8.6}$$

At a regularized stationary point, neither the residual contribution nor the penalty curvature may be dropped without justification. The interpolation nullity statement above concerns the unregularized data loss at zero residual, not every minimum of the penalized objective. Chapter 7, Section 7.4, and Appendix N keep these objectives separate.

Degeneracy can enlarge the set of acceptable parameter values. It does not, by itself, prove a large basin of attraction or a finite-time convergence guarantee. Competing positive-loss sets may also be degenerate.

Analyticity and the limit of a descent trajectory. For a fixed finite network constructed from affine maps and GELU, squared loss is real-analytic; see Theorem 41 in Appendix N. This permits more than Hessian calculations, but does not give uniform Hessian bounds over unbounded parameters or growing depths. The appendix's GELU examples separate analytic regularity from numerical conditioning.

In particular, let U be real-analytic on an open parameter domain. If its Euclidean gradient flow is defined for all $t \geq 0$ and the trajectory has compact closure contained in that domain, then

$$\int_0^\infty \|\dot{\theta}_t\| \, dt < \infty, \qquad \theta_t \longrightarrow \theta_\infty, \qquad \nabla U(\theta_\infty) = 0.$$

This is Theorem 40 of the appendix. The conclusion is convergence to a critical point, not necessarily a minimum, an interpolator, or a statistically useful predictor. It also supplies no uniform training-time bound across growing network families. A discrete or stochastic optimizer needs its own dynamical assumptions.

A quantitative route to fitting. For the *unregularized* squared loss above, write $e(\theta) = F(\theta) - y$ for the training residual and put $K(\theta) = J(\theta)J(\theta)^\top/n$. Along gradient flow, $\dot{e}(t) = -K(\theta_t)e(t)$. A sufficient quantitative condition is

$$e(t)^\top K(\theta_t) e(t) \geq \mu \|e(t)\|_2^2, \qquad \mu > 0, \tag{8.7}$$

throughout the time interval under consideration. Since $\frac{d}{dt}\widehat{\mathcal{L}} = -e^\top Ke/n$, this gives

$$\boxed{\widehat{\mathcal{L}}(\theta_t) \le \widehat{\mathcal{L}}(\theta_0)e^{-2\mu t}.} \tag{8.8}$$

When this bound holds for all time and the flow and prediction map meet the conditions in Chapter 7, Section 7.9, the parameters converge to a finite interpolator. Unlike the analytic result, this conclusion forces the specified training residual to zero; it does not require analyticity. Neither result by itself establishes generalization.

One NTK-style way to verify the stronger matrix condition is

$$\begin{aligned} K(\theta_0) \succeq 2\mu I_n, \qquad & \sup_{0\le t\le t_{\mathrm{f}}} \|K(\theta_t) - K(\theta_0)\|_{\mathrm{op}} \le \mu \\ \implies \quad K(\theta_t) \succeq \mu I_n \qquad & (0 \le t \le t_{\mathrm{f}}). \end{aligned}$$

The parameterization, initialization, width scaling, data, and time interval are part of this strategy. A nearly constant kernel is sufficient, not necessary: the residual condition also permits feature learning and substantial kernel change. Feasible optimization additionally requires controlled inverse conditioning, discretization, evaluation, and precision costs.

Analyticity, large parameter count, and almost-everywhere prediction rank do not replace the trajectory condition. Appendix N, Proposition 42, gives an analytic least-squares example with a regular interpolation manifold and full prediction rank almost everywhere, yet an open basin of a positive-loss minimum. The example concerns the general geometric inference, not a claim about every prescribed GELU architecture.

8.5.2 The Langevin picture

An idealized stochastic dynamics is

$$d\theta_t = -\nabla\widehat{\mathcal{L}}(\theta_t)\,dt + \sqrt{2T}\,dW_t.$$

When this process has a normalizable equilibrium density under suitable domain and boundary or confinement assumptions,

$$\rho_T(\theta) = Z_T^{-1}\exp\left(-\frac{\widehat{\mathcal{L}}(\theta)}{T}\right).$$

The probability of a region depends on both its loss and its parameter-space measure.

For a compact regular zero-loss manifold, or a suitable local compact portion, with uniformly positive transverse Hessian $H_\perp$, the low-temperature contribution has the schematic form

$$Z_{\mathcal{M}}(T) \sim (2\pi T)^{q/2}\int_{\mathcal{M}} \frac{dV}{\sqrt{\det H_\perp}}.$$

An isolated nondegenerate local minimum with loss $L_\ell > 0$ contributes

$$Z_\ell(T) \sim e^{-L_\ell/T}\frac{(2\pi T)^{p/2}}{\sqrt{\det H_\ell}}.$$

Thus a zero-loss set can have both an energy advantage and a volume advantage in this equilibrium model. Singular sets require the more careful volume analysis discussed in Chapter 7, Section 7.15, and Appendix N, Section N.3. The singular exponent describes a localized weighted-volume asymptotic, including the transformed measure; it is not a training-time rate or, by itself, a population-risk bound.

8.5.3 Equilibrium preference is not a training-time guarantee

These calculations do not supply a polynomial mixing time, a hitting-time bound from the chosen initialization, or a stationary law for arbitrary minibatch SGD. Its noise is generally state dependent and anisotropic. Noncompact parameter symmetries may also prevent normalization of the unregularized Gibbs density.

The measure and the dynamics must be specified together. If a regularizer is included in the Langevin potential, the equilibrium density contains $\widehat{\mathcal{L}}+\lambda\Omega$, rather than the empirical loss alone. Its minimum set and normal curvatures can differ from those of the exact data-interpolation manifold. The local-volume calculation must use the full potential appropriate to that process.

The geometric argument therefore identifies a possible mechanism for favoring low-loss solutions. It does not replace the separate obligation to prove that a specified training procedure reaches a suitable solution with high probability and feasible resources. There is nevertheless an intrinsic source of late-stage stochasticity in regularized SGD that does not require externally injected Brownian forcing.

8.5.4 Exact interpolation and regularized SGD fluctuations

For fixed per-example squared losses, write

$$\ell_i(\theta) = \tfrac{1}{2}(F_\theta(x_i) - y_i)^2, \qquad g_i(\theta) = \nabla\ell_i(\theta), \qquad \bar{g}(\theta) = \frac{1}{n}\sum_i g_i(\theta).$$

Let Ω be a deterministic differentiable penalty, and let $\mathcal{B}$ be a uniformly sampled B-element subset of $\{1,\dots,n\}$, with $n > 1$ and $1 \le B < n$. Ordinary penalized minibatch gradients have

$$\begin{aligned} U_\lambda(\theta) &= \widehat{\mathcal{L}}(\theta) + \lambda\Omega(\theta), \qquad \lambda > 0, \\ G_{\mathcal{B},\lambda}(\theta) &= \widehat{g}_{\mathcal{B}}(\theta) + \lambda\nabla\Omega(\theta), \qquad \widehat{g}_{\mathcal{B}} = \frac{1}{B}\sum_{i\in\mathcal{B}} g_i. \end{aligned} \tag{8.9}$$

At a fixed parameter, put

$$C(\theta) = \frac{1}{n}\sum_i (g_i - \bar{g})(g_i - \bar{g})^\top, \qquad c_{n,B} = \frac{n-B}{B(n-1)}.$$

Because the penalty contribution is the same for every batch,

$$\begin{aligned} \mathbb{E}_{\mathcal{B}}[G_{\mathcal{B},\lambda} \mid \theta] &= \nabla U_\lambda(\theta), \\ \mathrm{Cov}_{\mathcal{B}}(G_{\mathcal{B},\lambda} \mid \theta) &= c_{n,B}C(\theta). \end{aligned} \tag{8.10}$$

The covariance identity and its sampling correction are proved in Chapter 7, Section 7.17, and Appendix N, Section N.5.3. Independent sampling with replacement replaces $c_{n,B}$ by $1/B$. A full batch without replacement has zero sampling covariance irrespective of the individual gradients. These formulas assume averages of fixed per-example gradients; a batch-dependent forward operation requires its own update analysis.

At exact squared-loss interpolation, the original calculation remains:

$$e_i(\theta_*) = 0\ (\forall i) \quad\Longrightarrow\quad g_i(\theta_*) = e_i(\theta_*)\nabla_\theta F_{\theta_*}(x_i) = 0 \quad\Longrightarrow\quad \mathrm{Cov}(\widehat{g}_B \mid \theta_*) = 0.$$

Here $\widehat{g}_B$ denotes the average of sampled per-example gradients. Every regularized batch gradient at that point equals $\lambda\nabla\Omega(\theta_*)$. Its covariance is zero, but its mean need not vanish. Unless the penalty gradient vanishes too, the exact interpolator is not stationary for U_λ.

At a stationary point θ_λ of the *regularized* objective, by contrast,

$$\bar{g}(\theta_\lambda) + \lambda \nabla \Omega(\theta_\lambda) = 0, \qquad G_{\mathcal{B},\lambda}(\theta_\lambda) = \frac{1}{B} \sum_{i \in \mathcal{B}} (g_i - \bar{g})(\theta_\lambda). \tag{8.11}$$

Only the average data gradient must balance the regularizer. Taking the trace of the covariance at this zero-mean point gives

$$\boxed{\mathbb{E}_{\mathcal{B}} \|G_{\mathcal{B},\lambda}(\theta_\lambda)\|^2 = c_{n,B} \frac{1}{n} \sum_i \|g_i(\theta_\lambda) - \bar{g}(\theta_\lambda)\|^2.} \tag{8.12}$$

This is Proposition 45 of the appendix and Proposition 5 of Chapter 7. For $B < n$, the right side is zero exactly when all the individual data gradients coincide. At regularized stationarity, this is also equivalent to every regularized batch gradient being zero. Otherwise intrinsic sampling fluctuations persist even at a local or global minimum of U_λ.

The distinction is

$$\boxed{\begin{array}{l}\text{zero gradient of the full regularized objective} \\ \neq \text{ a common zero of every minibatch update.}\end{array}}$$

The regularizer is not itself random. By shifting the stationary balance away from exact interpolation, it can maintain different nonzero sample gradients. Positive regularization alone does not force this outcome: identical sample gradients can occur in special configurations. The criterion is gradient heterogeneity, not merely $\lambda > 0$.

The phenomenon does not require nonconvexity. For the scalar predictor $F_\theta(x) = \theta x$ on $(1, 1)$ and $(2, 2)$ with penalty $\lambda\theta^2/2$, the unique regularized global minimum is $\theta_\lambda = 5/(5 + 2\lambda)$. Its singleton-batch gradients are $\pm 3\lambda/(5+2\lambda)$, so the mean is zero and the variance is $9\lambda^2/(5+2\lambda)^2 > 0$. The full calculation is Example 4 in the appendix.

This positive source of late-stage noise is different from prescribing an isotropic Langevin process. With a plain step of size α, the update covariance is $\alpha^2 c_{n,B} C$; a justified small-step diffusion on time increments α instead has covariance rate $\alpha c_{n,B} C$. Neither persistent covariance nor vanishing mean establishes a Gibbs law, excitation of every escape direction, or a useful mixing-time bound.

8.5.5 Normalized networks, common equilibria, and low-rank bias

Xu, Rangamani, Liao, Galanti, and Poggio [117] give an architecture-specific obstruction to a common minibatch equilibrium. Their normalized scalar-output ReLU setting uses a bias-free network with independently parameterized, unshared layer matrices,

$$F_{\rho,V}(x) = \rho f_V(x), \qquad \rho > 0, \qquad \|V_k\|_F = 1,$$

binary labels $y_i \in \{-1, +1\}$, and scale penalty $\lambda\rho^2/2$. Assume $1 \le B < n$, nonzero signed normalized margins $m_i = y_i f_V(x_i)$, and differentiability with the stated fixed-gate derivative identities on the training sample. Under these hypotheses, vanishing of the scale and tangent layer-update directions for *every* B-element minibatch implies

$$\boxed{\operatorname{rank}(V_k) \le 2 \qquad \text{for every layer.}}$$

This is the scoped form of their Lemma 8 proved in Chapter 7, Lemma 3, and Appendix N, Proposition 46.

At a stationary point of the *batch-averaged normalized dynamics*, the mean of the concatenated scale and tangent update directions is zero. If the other hypotheses hold and some layer has rank greater than two, the contrapositive excludes a common zero. Since every admissible batch has positive probability, the actual normalized directions then have strictly positive covariance trace. This concerns the specified normalized updates, not a substitution of unconstrained gradients.

This motivates the paper's low-rank-bias interpretation; it does not prove that SGD always reaches rank-two matrices or that low rank ensures zero noise. No uniform noise lower bound follows. The scalar-output, homogeneity, normalization, and regularity assumptions do not transfer unchanged to GELU or other architectures. The general covariance criterion and this specific structural mechanism remain distinct.

8.6 Regularization selects among well-fitting solutions

When overparameterization permits many well-fitting solutions, regularization supplies a preference among them. Suppose training uses

$$\widehat{R}_S(F_\theta) + \lambda\Omega(\theta), \qquad \lambda > 0, \tag{8.13}$$

with $\Omega \geq 0$.

In an idealized exact-fit formulation, selection is expressed by

$$\min_{\theta \in \mathcal{M}} \Omega(\theta).$$

A small-penalty limit can motivate this constrained picture under appropriate existence and compactness conditions. At a fixed positive λ, however, the penalized minimizer need not interpolate exactly: it may trade fit against a smaller penalty. Nor does writing the objective prove that SGD reaches its global minimum.

The stochastic implication is equally important. Even at a minimum of this full objective, the regularizer may be canceled only by the average data gradient, while individual minibatches continue to differ. Thus regularization can affect both the selected function and the stochastic dynamics in the well-fitting region. Its statistical role and its effect on minibatch balance are complementary, not interchangeable. A useful returned predictor need not be a common equilibrium of those updates.

8.6.1 An elementary conditional comparison

Suppose $\overline{\theta}$ is a comparison parameter and the returned parameter satisfies, for some $\xi \geq 0$,

$$\widehat{R}_S(F_{\widehat{\theta}}) + \lambda\Omega(\widehat{\theta}) \leq \widehat{R}_S(F_{\overline{\theta}}) + \lambda\Omega(\overline{\theta}) + \xi.$$

By nonnegativity of the two terms,

$$\widehat{R}_S(F_{\widehat{\theta}}) \leq \widehat{R}_S(F_{\overline{\theta}}) + \lambda\Omega(\overline{\theta}) + \xi, \tag{8.14}$$

$$\Omega(\widehat{\theta}) \leq \Omega(\overline{\theta}) + \frac{\widehat{R}_S(F_{\overline{\theta}}) + \xi}{\lambda}. \tag{8.15}$$

These inequalities follow directly by dropping one nonnegative term from the left-hand side. For an interpolating comparator, they become

$$\widehat{R}_S(F_{\widehat{\theta}}) \le \lambda\Omega(\overline{\theta}) + \xi, \qquad \Omega(\widehat{\theta}) \le \Omega(\overline{\theta}) + \xi/\lambda.$$

This makes the selection argument precise in one useful regime: a low-penalty comparator and successful optimization can yield both a good fit and penalty control. The existence of the comparator and the optimization comparison are additional requirements, not consequences of adding a regularization term.

The fitting theorem must match the trained objective. In the squared-loss setting, a differentiable penalty changes the residual dynamics. For $U_\lambda = \widehat{\mathcal{L}} + \lambda\Omega$ and $\dot{\theta} = -\nabla U_\lambda$, the chain rule gives

$$\dot{e} = -Ke - \lambda J\nabla\Omega. \tag{8.16}$$

Thus the unregularized estimate (8.8) cannot simply be transferred to penalized training: the penalty contribution must also be controlled. The comparator inequalities above offer a different conditional route to joint fit and penalty control, but stationarity of U_λ alone does not establish their optimization premise.

Confinement is a separate role. For an empirical loss analytic on all of $\mathbb{R}^p$, full quadratic weight decay gives a useful positive result. With

$$U_\gamma(\theta) = \widehat{\mathcal{L}}(\theta) + \frac{\gamma}{2}\|\theta\|^2, \qquad \gamma > 0,$$

and all parameters penalized, gradient flow satisfies

$$\|\theta_t\|^2 \le \frac{2U_\gamma(\theta_0)}{\gamma}.$$

The compact sublevel set gives all-time existence and, by the analytic result, finite-length convergence to a critical point of U_γ. Corollary 8 in Appendix N gives the proof. This supplies confinement and stationarity, not global minimization or exact interpolation. It is one sufficient confinement mechanism, not a requirement for every successful learner.

Gradient-flow limits and SGD outputs. The weight-decay corollary concerns deterministic flow, not a common zero of all minibatch updates. For plain SGD with independently sampled batches of fixed positive probabilities, continuous batch gradient fields near a finite limit, and learning rates bounded away from zero, any finite point limit must be a common minibatch equilibrium (Appendix N, Proposition 47). Positive stationary covariance excludes that endpoint under these conditions, not all useful training behavior.

Diminishing steps can shrink increments despite nonzero gradient-estimator covariance. A specified stopping or averaging rule can return a useful predictor without common batch stationarity, but must meet the claimed fit, hypothesis-class, and evaluation-resource requirements.

8.7 Product norms and Rademacher complexity

The product-norm idea has a clear role in this framework. In a specified feedforward setting, a natural candidate complexity quantity is

$$\mathcal{C}_\times(\theta) = \prod_{\ell=1}^{L} \|W_\ell\|_F,$$

and a corresponding multiplicative penalty is

$$\Omega_\times(\theta) = \mathcal{C}_\times(\theta)^2.$$

Biases and other trainable operations must be included in the chosen complexity analysis; a formula written only for weight matrices is not a bound for unrestricted additional parameters.

A multiplicative penalty favors a smaller product among solutions that achieve a comparable fit. The statistical motivation is that suitable network classes admit norm-sensitive Rademacher bounds. The precise quantity can involve products of Frobenius norms, products of spectral norms with further corrections, path norms, or other architecture-dependent expressions.

Statistical control, confinement, and conditioning. Three different controls occur in the argument:

class-complexity control:	$\mathcal{C}(\widehat{\theta}) \le Q,$
trajectory confinement:	$\sup_t \|\theta_t\| < \infty,$
fitting-direction conditioning:	$e^\top K e \ge \mu \|e\|^2.$

An upper norm bound used in a statistical theorem does not automatically supply either of the other two. An upper matrix-norm bound does not exclude rank deficiency. Likewise a product penalty need not be coercive. For example,

$$\widehat{\mathcal{L}}(a,b) = \tfrac{1}{2}(ab-1)^2, \qquad \Omega_\times(a,b) = a^2b^2$$

has $\widehat{\mathcal{L}}(t,t^{-1}) + \lambda\Omega_\times(t,t^{-1}) = \lambda$ for all $t > 0$. Its sublevel set is unbounded. This does not prove divergence of the training dynamics; it shows why a separate confinement argument is needed when an analytic convergence theorem uses compactness. Chapter 7, Sections 7.18–7.19, and the appendix discuss these distinct roles.

Match the norm theorem to the architecture. An activation hypothesis cannot be exchanged for smoothness. In particular, GELU is not positively homogeneous, so a statistical result requiring that property cannot be transferred unchanged from a homogeneous architecture. Residual additions, attention, normalization, biases, and learnable scales must be covered by the chosen complexity analysis. None of this removes the positive role of product-norm regularization: with the appropriate class bound and successful fitting, it remains a route to predictive control without identifying the teacher's internal computation.

A norm-complexity budget is not the same quantity as minibatch fluctuation energy or matrix rank. The regularized-noise result does not replace a statistical class bound, and the common-equilibrium rank restriction does not by itself prove generalization. Conversely, persistent updates do not invalidate a statistical bound for a returned predictor that actually meets its fit and class conditions.

8.7.1 A class bound, not a property of one isolated parameter vector

Fix an architecture class and a complexity functional $\mathcal{C}(\theta)$. For a budget Q fixed independently of the training sample, define

$$\mathcal{H}_Q = \{F_\theta : \mathcal{C}(\theta) \le Q\}. \tag{8.17}$$

For scalar predictors, the empirical Rademacher complexity is

$$\widehat{\mathfrak{R}}_{S_X}(\mathcal{H}_Q) = \mathbb{E}_\epsilon\left[\sup_{h\in\mathcal{H}_Q} \frac{1}{n}\sum_{i=1}^n \epsilon_i h(X_i)\right],$$

where the ϵ_i are independent random signs.

The relevant assertion is a bound on this entire class. Schematically, applicable norm-based results may take the form

$$\widehat{\mathfrak{R}}_{S_X}(\mathcal{H}_Q) \le \mathcal{B}_n(Q; \mathcal{A}, B),$$

where $\mathcal{A}$ records the architecture and activation assumptions and B controls the inputs. A schematic dependence is

$$\mathcal{B}_n(Q; \mathcal{A}, B) \lesssim A_{\mathcal{A}} \frac{BQ}{\sqrt{n}},$$

when $\mathcal{C}$ and $A_{\mathcal{A}}$ account for the factors required by the particular theorem. Depth factors, bias bounds, normalization, and other correction terms cannot be dropped. This display is not a universal formula for every neural architecture.

Consequently, a smaller suitable norm product can reduce a generalization upper bound. It does not literally minimize the exact Rademacher complexity, and it does not establish a universally monotone relation between that product and the actual test error.

8.7.2 From a well-fitting predictor to population control

Assume the loss is bounded, $0 \le \ell \le M_\ell$, and K_ℓ-Lipschitz in the prediction on the admitted prediction and label ranges. Under the usual measurability conditions, the relevant uniform-convergence statement has the form

$$R_P(h) \le \widehat{R}_S(h) + \beta_S(Q, \delta) \quad \text{for every } h \in \mathcal{H}_Q, \tag{8.18}$$

with probability at least $1 - \delta$. A Rademacher-based choice is of the form

$$\beta_S(Q, \delta) = c_1 K_\ell \widehat{\mathfrak{R}}_{S_X}(\mathcal{H}_Q) + c_2 M_\ell \sqrt{\frac{\log(2/\delta)}{n}},$$

with appropriate universal constants. The preceding class-complexity bound can then be used to upper-bound β_S.

Squared loss satisfies these boundedness and Lipschitz conditions when predictions and labels are restricted to a fixed bounded range. It does not satisfy them on an unrestricted real domain. Clipping predictions or using a different valid statistical bound must be made explicit when needed.

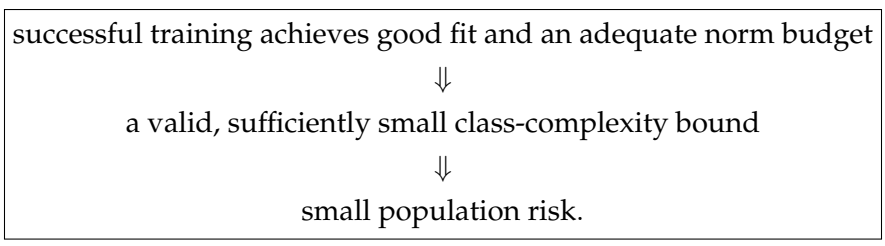
If training returns $F_{\widehat{\theta}} \in \mathcal{H}_Q$ with $\widehat{R}_S(F_{\widehat{\theta}}) \le \eta$, then on the uniform event,

$$\boxed{R_P(F_{\widehat{\theta}}) \le \eta + \beta_S(Q, \delta).}$$

Interpolation is the special case $\eta = 0$. The argument does not require exact interpolation.

8.7.3 What the regularizer must actually accomplish

The defensible chain is



A regularizer is one mechanism for achieving the first line. The mere presence of that term in the objective does not prove the first line. Moreover, an upper bound must be informative at the available n; polynomial computational size alone does not ensure that it is.

If the norm budget is selected after seeing the sample, a fixed-Q bound cannot simply be applied without adjustment. One elementary option is a prespecified sequence of budgets Q_j with confidence allocations δ_j satisfying $\sum_j \delta_j \le \delta$. A union bound makes (8.18) valid simultaneously over those classes, and selection among them then uses the corresponding confidence penalty. Sample-dependent architectures and other model-selection steps require the same care.

8.8 The main predictive recovery argument

Two logically separate results combine the preceding pieces. The first turns successful empirical fitting and suitable complexity control into population control. The second identifies the computational structure of an already successful predictor.

Theorem 7 (Conditional predictive recovery)**.** *Let $S \sim P^n$ and fix a hypothesis class $\mathcal{H}_Q$ independently of S. Suppose that, with probability at least $1 - \delta_{\mathrm{stat}}$ over S,*

$$R_P(h) \le \widehat{R}_S(h) + \beta_S(Q, \delta_{\mathrm{stat}}) \qquad \text{for all } h \in \mathcal{H}_Q.$$

Let a possibly randomized training procedure return $\widehat{f} = A(S, U)$. Assume that, with probability at least $1 - \delta_{\mathrm{opt}}$ over the sample and algorithmic randomness,

$$\widehat{f} \in \mathcal{H}_Q, \qquad \widehat{R}_S(\widehat{f}) \le \eta.$$

Then, with probability at least $1 - \delta_{\mathrm{stat}} - \delta_{\mathrm{opt}}$,

$$\boxed{R_P(\widehat{f}) \le \eta + \beta_S(Q, \delta_{\mathrm{stat}}).} \tag{8.19}$$

If the algorithmic success event also includes a stated training-cost or evaluation-cost bound, that bound holds on the same intersection event.

Proof. Let E_{stat} denote the uniform statistical event and let E_{opt} denote the algorithmic success event. On their intersection, the uniform bound applies to the returned predictor, since $\widehat{f} \in \mathcal{H}_Q$. Therefore

$$R_P(\widehat{f}) \le \widehat{R}_S(\widehat{f}) + \beta_S(Q, \delta_{\mathrm{stat}}) \le \eta + \beta_S(Q, \delta_{\mathrm{stat}}).$$

The union bound gives

$$\mathbb{P}(E_{\mathrm{stat}} \cap E_{\mathrm{opt}}) \ge 1 - \delta_{\mathrm{stat}} - \delta_{\mathrm{opt}}.$$

The two events need not be independent. Any resource bound included in E_{opt} also holds on their intersection. □

The theorem is deliberately conditional: it does not derive the algorithmic success probability. Establishing that premise for the specified training dynamics is a central research problem.

In particular, convergence to an analytic critical point or a favorable Langevin equilibrium weight does not establish the event $\widehat{f} \in \mathcal{H}_Q$, $\widehat{R}_S(\widehat{f}) \le \eta$. A finite-time fitting theorem can help with the second inequality, but the returned complexity budget and the effect of the actual regularizer must be controlled as well. For noiseless squared loss, the conditional theorem yields

$$\|\widehat{f} - f\|_{L^2(P_X)} \le \sqrt{2(\eta + \beta_S(Q, \delta_{\mathrm{stat}}))}.$$

Thus $\eta + \beta_S \le \varepsilon^2/2$ is sufficient for L^2 error at most ε.

The returned predictor need not be an absorbing SGD state. For a specified finite stopping index j_*, the procedure may return

$$\widehat{f} = F_{\theta_{j_*}}.$$

The uniform statistical event covers every member of the fixed class, including this data- and trajectory-selected member. The stopping rule must still attain the joint fit and complexity condition within its stated probability and resource budget. Neither $G_{\mathcal{B},\lambda}(\theta_{j_*}) = 0$ for every batch nor zero covariance is an additional premise. Continued fluctuations do not themselves invalidate the returned predictor's guarantee.

Proposition 6 (A feasible generalizing predictor is a compositional witness)**.** *Consider real-valued targets f_N with input distributions $P_{X,N}$. For each N and $0 < \varepsilon \le 1$, suppose a returned network $F_{N,\varepsilon}$ satisfies*

$$\|F_{N,\varepsilon} - f_N\|_{L^2(P_{X,N})} \le \varepsilon.$$

Assume its finite description length is polynomial in N and ε^{-1}, and that one specified evaluator, given this description and a precision request m, approximates its output uniformly to error 2^{-m} using polynomially many bit operations and input bits in N, ε^{-1}, and m.

Then for every m there is a polynomial-size bounded-fan-in Boolean circuit, acting on the prescribed input encoding, whose decoded output $\widetilde{F}_{N,\varepsilon,m}$ satisfies

$$\|\widetilde{F}_{N,\varepsilon,m} - f_N\|_{L^2(P_{X,N})} \le \varepsilon + 2^{-m}. \tag{8.20}$$

Hence the returned predictors supply feasible compositionally sparse approximations at the stated learning and evaluation resolutions.

Proof. Fix a returned network description and a requested precision m. Its evaluator is a finite-time digital computation. The bounded-fan-in simulation from Chapter 4 gives a circuit whose size is polynomial in the stated evaluation resources. The parameters in the returned description are fixed constants in that circuit.

By the numerical guarantee,

$$\|\widetilde{F}_{N,\varepsilon,m} - F_{N,\varepsilon}\|_\infty \le 2^{-m}.$$

Since $P_{X,N}$ is a probability measure, this also bounds the L^2 evaluation error. The triangle inequality gives (8.20). □

Regularization and interpolation do not enter this second proof. Their role is to help establish the small population error used in its hypothesis. In particular, taking a learning error $\varepsilon/2$ and an evaluation error at most $\varepsilon/2$ gives a compositional predictor with total L^2 error at most ε.

The circuit compilation is uniform given the returned finite model description. This does not imply that an arbitrary sequence of sample-selected model descriptions is generated uniformly from N alone. A uniform learning claim must also specify the algorithm that produces those descriptions from its finite-precision data.

The resulting logical division is

prediction theorem:	fit and complexity control give small risk,
witness proposition:	feasible prediction gives a sparse computation.

8.9 Why uniqueness is not required for predictive learning

The witness proposition does not require recovery of the generating variables. Suppose

$$f = g_m \circ \cdots \circ g_1, \qquad F_{\theta^\star} = h_L \circ \cdots \circ h_1.$$

Even when no useful nodewise correspondence exists between the h_ℓ and the g_j, the predictor can be satisfactory if it is feasible and approximates f on the intended population.

This is stronger than observing that a generating decomposition is nonunique. Predictive success does not require selecting a member of a specified equivalence class of generating decompositions at all.

predictive recovery $\neq$ identification of a generating DAG.

Uniqueness and identifiability still matter for a structural objective. Scientific interpretation, causal questions, or a specified module-reuse claim may require more than population prediction. The point is not that those questions disappear, but that they should not be inserted as additional premises of the predictive witness argument.

8.10 The revised role of genericity

Genericity remains useful as a possible explanation of accessible learning signal. Let $X \sim P_X$ and let ϕ be a low-complexity probe available to the learner. Define

$$\rho_\phi = \mathbb{E}[f(X)\phi(X)].$$

Probe scale and variance must be controlled; otherwise a correlation can be enlarged simply by rescaling the probe. A quantitatively detectable footprint, for example

$$|\rho_\phi| \geq \frac{1}{\mathrm{poly}(N)}$$

under fixed normalization and suitable concentration assumptions, can provide a useful screening signal.

Whether that signal is accessible to an optimizer also depends on its parameterization and current state. For squared population loss,

$$\nabla_\theta \mathcal{L}(\theta) = \mathbb{E}\big[(F_\theta(X) - f(X))\nabla_\theta F_\theta(X)\big],$$

when differentiation and expectation can be interchanged. The relevant probes are therefore those induced by the chosen gradient directions, not arbitrary functions with nonzero correlation.

Genericity concerns the target, observation distribution, and accessible probe family together. It can help explain why optimization discovers useful coordinates in finite time. But once predictive success and feasible evaluation have been established by some valid argument, no additional genericity hypothesis is needed to infer a compositional witness.

This does not make learning-signal questions optional inside a proof that actually relies on them. It makes genericity one candidate mechanism, rather than a universal prerequisite for every successful learner.

8.11 Parity revisited

For $S \subseteq [d]$, define

$$\chi_S(x) = \prod_{i \in S} x_i, \qquad x \in \{-1, +1\}^d.$$

For $|S| \geq 2$, a balanced binary multiplication tree has size $O(|S|)$ and depth $O(\log |S|)$. Parity therefore has a simple compositionally sparse realization.

Yet under the uniform product distribution,

$$\mathbb{E}[\chi_S(X) X_j] = 0 \qquad (j \in S,\ |S| > 1).$$

Indeed, multiplication by X_j removes that factor, leaving a product of independent mean-zero coordinates. Thus

compositional sparsity does not guarantee a particular low-order learning signal.

This rules out the stated linear screening signal. It does not prove that all deep-learning procedures fail to learn parity. If another procedure finds a feasible parity predictor that generalizes, its computation already gives the required compositional witness.

8.12 Generic perturbations and local signals

Perturbations can reveal signals hidden by exact cancellation. The calculations below show both the possibility and its quantitative limit.

8.12.1 Boolean leakage

Let independent $X_i \in \{-1, +1\}$ satisfy $\mathbb{E}[X_i] = \mu_i$, and let $f(X) = \prod_{i \in S} X_i$. For $j \in S$,

$$\boxed{\mathbb{E}[f(X) X_j] = \prod_{i \in S \setminus \{j\}} \mu_i.} \tag{8.21}$$

This follows from $X_j^2 = 1$ and independence. If all the relevant biases in this product are nonzero, exact cancellation is broken.

However, if $|\mu_i| \leq \mu < 1$ and $|S| = k$, then

$$|\mathbb{E}[f(X) X_j]| \leq \mu^{k-1}.$$

The signal may be exponentially small in interaction order. Even at bounded interaction order, a quantitative lower bound on the individual biases is needed for a useful uniform detectability claim. Nonzero and efficiently detectable are different properties.

8.12.2 Gaussian leakage

Let $X \sim \mathcal{N}(\mu, \sigma^2 I)$ with $\sigma > 0$. Under the regularity and integrability assumptions needed for Gaussian integration by parts, Stein's identity gives

$$\boxed{\mathbb{E}[f(X)(X_i - \mu_i)] = \sigma^2 \mathbb{E}[\partial_i f(X)].} \tag{8.22}$$

A nonzero average derivative therefore yields a nonzero linear footprint. A detectable magnitude and access through the learning rule remain separate requirements.

These examples explain possible sources of optimization signal. They do not turn the existence of a footprint into a universal convergence result, and they are not necessary premises of the computational witness proposition.

8.13 Staircase learning as a constructive mechanism

Suppose learning first constructs a useful coordinate

$$z_1 = g_1(x).$$

A later dependency $z_2 = g_2(z_1, x)$ may be high-order in the original coordinates but low-order in the enlarged representation (x, z_1). Feature learning can therefore change the coordinate system in which the remaining task is expressed.

The proposed mechanism is

detect an accessible signal
$\Downarrow$
learn and preserve a useful coordinate
$\Downarrow$
expose a simpler next learning problem.

A staircase theorem must establish this progression for a specified architecture, initialization, optimizer, and data model. It must also control finite samples, the errors in previously learned coordinates, and the time needed for later stages. Merely exhibiting an accurate deep representation does not prove those claims.

The revised framework does not discard this research objective. A successful staircase theorem can supply a constructive route to the algorithmic premise of Theorem 7. It may establish the stronger structural criterion (8.3) as well.

Its role is nevertheless distinct from Proposition 6. The proposition identifies what follows once a feasible predictor succeeds. A staircase theorem seeks to explain how a particular learner achieves that success. The latter can be important without being a necessary trajectory for all successful learners.

8.14 Skip connections and traces remain useful

A residual block

$$z_{\ell+1} = z_\ell + g_\ell(z_\ell)$$

provides an identity pathway alongside a learned transformation. This can assist information and gradient propagation. It does not, for an arbitrary g_ℓ, guarantee invertibility or exact preservation of every previous coordinate. Such preservation must be established for the particular architecture or enforced by an explicit storage mechanism.

The quantitative issue is control of products of the residual Jacobians $I + Dg_\ell$, not residual addition alone. Proposition 39 of Appendix N gives sufficient scaling conditions. Those activation-propagation Jacobians are distinct from the parameter-to-training-prediction Jacobian J in Section 8.5; controlling one does not by itself prove the required lower bound for the other.

Exposing intermediate variables changes the learning problem more directly. If

$$z_v = g_v(z_{\mathrm{pa}(v)})$$

and the training data contain the parent tuples and corresponding intermediate outputs, then training can be organized into local supervised problems. Their learnability, coverage, and composed error still require analysis.

Trace information can help even when end-to-end learning is possible. It is not only a fallback for a failed genericity condition. Faithful traces, partial traces, and useful surrogates provide different levels of information, as distinguished earlier in the book.

In the terminology C1–C3, C1 concerns feasible representation; C2 concerns accessible learning signal; C3 supplies intermediate information. These roles can cooperate. Neither skip connections nor traces are additional requirements for the conclusion that an already successful feasible predictor is a compositional witness.

An architectural or initialization condition can be indispensable to a particular proof of training success without being an additional premise of the witness proposition. Analytic activations and a nearly constant NTK have the same status: they support particular analyses, not a universal list of requirements for an already accurate feasible predictor.

8.15 Neural collapse is evidence, not a premise

The neural-collapse phenomena discussed earlier illustrate how trained classification networks can develop highly structured representation geometry. Such observations are relevant to the broader question of how training selects among many possible solutions.

They are not premises of the witness argument. That conclusion follows from computational feasibility and predictive accuracy, without requiring collapse in the final layer or in earlier layers. The common-minibatch rank restriction in Section 8.5.5 is a different claim about parameter matrices under specified update conditions; it does not by itself establish neural collapse.

Conversely, observing a special geometry on the training examples does not by itself prove a small population risk, a useful norm-complexity bound, or recovery of the generating latent DAG. Neural collapse can be a diagnostic of representation organization while remaining distinct from these stronger conclusions.

8.16 Three questions for predictive learning

The predictive framework can now be organized around three questions.

8.16.1 Q1: Does the chosen class contain a feasible accurate predictor?

Efficient target computation supplies a feasible circuit realization. For a particular learning architecture, one must still establish that an accurate predictor is available within its admitted resources. If a norm-based statistical argument is intended, one also needs an adequate complexity budget for a useful comparison predictor.

Thus existence of some small circuit is not automatically existence of a low-norm predictor in every chosen architecture.

8.16.2 Q2: Can the specified training procedure find a suitable fit?

The required statement concerns an optimizer, initialization, schedule, and stopping rule, with a finite-time success probability. The procedure must reach a sufficiently good fit while meeting the complexity control needed for Q3. Exact interpolation is one important regime, not the only possible objective. In particular, the return rule can select an accurate finite iterate from a trajectory with continuing minibatch fluctuations. Q2 does not require convergence to one parameter at which all batch updates vanish. The regularization strength, batch rule, learning-rate schedule, and return criterion are part of the finite-time statement.

Width, residual scaling, initialization, and regularity can help establish this step for a specified parameterization. An analytic critical-point limit is not enough; nor is a lower bound on prediction rank only at initialization or almost everywhere. The needed conclusion is finite-time access to an adequate fit and complexity budget under the actual updates. Favorable stochastic geometry can contribute, but its equilibrium weights alone do not establish that access.

8.16.3 Q3: Is the selected predictor controlled well enough to generalize?

A valid statistical bound must apply to the selected predictor and be small enough at the available sample size. Product-norm regularization provides a route through norm-sensitive classes when its optimization and statistical requirements are met.

8.16.4 What is not a fourth predictive requirement

After these questions have been answered with the stated feasibility conditions, there is no further need to match the generating hidden variables. The learned computation itself gives a compositional witness.

Structural identification remains a separate question when the scientific or downstream objective calls for it.

8.17 What is theorem and what remains conjectural

The logical status of the pieces should remain explicit.

8.17.1 Established under stated assumptions

The computational simulation establishes bounded-fan-in realizations of polynomial-time digital evaluations. With the specified precision convention, it applies to efficient finite-precision real evaluation and to the evaluator of a returned finite network.

For smooth squared loss, the Hessian identity (8.5) holds at zero residual. Under local constant rank at an interpolating point, the rank theorem gives (8.4).

For an all-time analytic gradient flow with compact trajectory closure inside the analytic domain, Appendix N establishes finite-length convergence to a critical point. Separately, the trajectory-wise residual condition (8.7) gives the fitting estimate (8.8) for unregularized squared-loss flow. Full quadratic weight decay supplies confinement in the stated analytic setting, not global fitting. These are different conditional results; none is an unconditional trainability theorem for wide GELU networks or arbitrary SGD.

The minibatch covariance identities in Section 8.5.4 are also exact under their fixed-example and sampling assumptions. At regularized stationarity, heterogeneous sample gradients give positive covariance; a zero full gradient is not enough to remove it. The normalized ReLU rank restriction is

an additional conditional result about common minibatch equilibria, not a general convergence or generalization theorem.

Within an appropriate fixed hypothesis class, a valid uniform statistical bound and a successful fitting event combine as in Theorem 7. Separately, Proposition 6 gives the computational structure of an already accurate feasible predictor.

None of these statements alone proves the full learning claim.

8.17.2 Idealized but explicit

The Langevin and local-volume calculations describe a specified equilibrium model under its normalizability and regularity conditions. They identify a possible preference for extended low-loss sets. Their extension to finite-time minibatch SGD is an additional dynamical problem. Intrinsic fluctuations under regularization provide a real sampling mechanism, but do not by themselves justify replacing its anisotropic, state-dependent covariance by the isotropic Langevin model.

Likewise, constrained minimum-penalty interpolation is a useful selection model. It should not be confused with a theorem that arbitrary penalized SGD solves that constrained problem.

8.17.3 Mechanisms rather than universal prerequisites

Genericity, staircase feature learning, residual pathways, trace exposure, associative retrieval, and structured representation geometry may help explain success in particular regimes. The interaction of normalization, regularization, residual-dependent SGD noise, and low-rank bias is another mechanism to study. Each requires its own assumptions and tests. The exact common-equilibrium obstruction should not be confused with a complete dynamical theorem establishing that mechanism.

A mechanism can prove the algorithmic premise of the predictive theorem. It is not a separate premise of the witness proposition once that predictive premise has been established in another way.

8.17.4 The remaining broad conjecture

The intended research claim can be stated without requiring recovery of a privileged hidden DAG.

> **Regularized Predictive Recovery Conjecture.** For a broad, explicitly characterized class of target–distribution families admitting feasible accurate predictors with suitable norm-complexity control, a specified family of regularized SGD procedures in sufficiently expressive overparameterized deep networks finds, with high probability and feasible resources, a well-fitting predictor whose statistical bound is informative enough to establish population accuracy.

This is the predictive version of the regularized global-recovery conjecture. To turn the program into a testable mathematical statement, one must specify the admissible problem class, architecture, loss, regularizer, initialization, and training schedule independently of the unknown target instance. The procedure also specifies how it returns a finite predictor—for example, by a stopping rule or an admissible averaging rule. The conjecture requires success of that returned predictor, not a common zero of every possible minibatch update.

One formal target is the following. Let N index the problem family, let κ collect stated regularity and conditioning parameters, and let ϵ be a population-risk tolerance. Seek deterministic choices of

a class budget Q, a sample size n, an empirical tolerance η, and a uniform, specified regularized SGD procedure such that

$$\begin{gathered} n, \quad \text{training bit cost}, \quad \text{model description length} \le \operatorname{poly}\bigl(N, \kappa, \epsilon^{-1}, \log(1/\delta)\bigr), \\ \mathbb{P}\left\{\widehat{R}_S(\widehat{f}) \le \eta, \quad \mathcal{C}(\widehat{\theta}) \le Q\right\} \ge 1 - \delta/2, \end{gathered} \tag{8.23}$$

with a valid uniform statistical bound controlled by a deterministic quantity $b_n(Q, \delta/2)$ satisfying

$$\eta + b_n(Q, \delta/2) \le \epsilon. \tag{8.24}$$

The evaluator of the returned model must also have the stated polynomial cost in its description length and requested evaluation precision.

The conditional theorem then yields $R_P(\widehat{f}) \le \epsilon$ with probability at least $1 - \delta$. For noiseless squared loss, an L^2 accuracy request ε uses $\epsilon = \varepsilon^2/2$.

The difficult assertion is the existence of the algorithmic guarantee (8.23) together with an informative budget (8.24) on a meaningful problem class. It is not the final union-bound argument or the compilation of the returned predictor.

Efficient target evaluation alone does not establish this conjecture. The admissible class must not be defined circularly as the class on which the chosen optimizer succeeds. Nor does the conjecture, as a broad research program, assert efficient learning of every polynomial-time computable target under every distribution.

The conditioning parameters κ must record the quantitative premises of the proposed proof, not merely assert smoothness or width. A residual-contraction route must control inverse conditioning and the admissible discrete step sizes. An analytic route must justify confinement and any claimed entry-time and rate bounds. A stochastic route must specify its actual noise and finite-time access, including the effect of regularization and the learning-rate schedule. A qualitative assertion of nonzero covariance is not a polynomially controlled noise or escape bound. Each route must meet the same joint fitting and statistical-budget requirement; no unproved mechanism is being inserted as an additional universal condition.

8.18 Failure modes

The distinctions above separate several possible failures.

Representation failure. The chosen architecture may not contain a sufficiently accurate predictor at the allowed computational and statistical complexity. Efficient target evaluation in another computational language does not settle this architecture-specific question.

Optimization failure. A suitable predictor may exist, but the specified training procedure may not reach a sufficient fit with high probability in feasible time.

Even a bounded analytic gradient flow can converge to a positive-loss critical point. Conversely, a fitting guarantee for unregularized flow need not survive an uncontrolled penalty term or a change to stochastic updates. On the other hand, continued minibatch fluctuations or failure to converge to one fixed parameter do not by themselves constitute optimization failure under a return-based objective: a specified stopped predictor may already satisfy the required fit and complexity bounds.

Complexity-selection failure. Training may fit the data but return a model outside the useful norm-complexity budget. Alternatively, an available bound may be too loose to certify a predictor that actually generalizes. Failure of a sufficient bound is not itself proof of poor test performance.

Generalization failure. A selected predictor can have small empirical error and large population error. A valid bound can prevent this conclusion only when its assumptions hold and its numerical value is informative.

Scaling or numerical failure. Depth, width, intermediate precision, training cost, or evaluation cost may become prohibitive across a family. Finite does not mean efficient, and efficient evaluation does not mean efficient training.

Structural failure without predictive failure. The predictor may fail to reconstruct the generating variables while predicting accurately. This is a failure of a structural objective, not of the predictive witness implication. Likewise, bounded-fan-in digital implementation does not automatically certify a regular coarse-grained Sobolev decomposition.

Mechanism failure without endpoint failure. A proposed staircase or genericity mechanism may fail to describe a successful learning trajectory. That challenges the proposed mechanism, not the fact that a feasible generalizing network supplies a compositional predictor.

8.19 What predictive success does not require

The witness argument does not require uniqueness of the compositional decomposition, recovery of the generating latent variables, degree-ordered staircase learning, faithful intermediate traces, or a literal correspondence between hidden units and target constituents.

These properties can occur and can be useful. In a particular restricted learning theorem, some may be central to the proof. None is an additional condition for compiling a feasible predictor that has already been shown to approximate the target.

The same applies to the choice of GELU or another analytic activation, a specific residual design, and an NTK-style proof of conditioning. Their absence from the witness proof does not remove them from a learning theorem whose success guarantee relies on them.

Nor does predictive success require zero SGD covariance, a common minibatch equilibrium, or convergence of the entire training trajectory to one parameter. Those are additional dynamical properties, not premises for the returned predictor's fit, statistical bound, and feasible evaluation. A claimed point-convergence theorem must, of course, address its own stronger endpoint.

What cannot be removed is the need to establish accuracy, a valid statistical argument when one is claimed, and the relevant computational resource bounds. The revised theory removes an unnecessary structural endpoint; it does not remove the optimization problem.

8.20 A compact recovery logic

The representation question is

$$\boxed{\text{efficient target evaluation} \implies \text{a feasible computational realization exists.}}$$

For the chosen learning architecture, its approximation quality and useful norm budget must still be established.

The statistical and algorithmic combination is

$$\boxed{\begin{array}{c}\text{training success: } \widehat{R}_S(\widehat{f}) \le \eta, \quad \widehat{f} \in \mathcal{H}_Q \\ + \quad \text{a valid uniform bound on } \mathcal{H}_Q \\ \Longrightarrow \quad R_P(\widehat{f}) \le \eta + \beta_S(Q, \delta_{\mathrm{stat}}).\end{array}}$$

When that upper bound is small, the predictor is accurate for the stated population objective. Finally,

$$\boxed{\begin{array}{c}\text{accurate predictor with feasible numerical evaluation} \\ \Longrightarrow \\ \text{compositionally sparse predictive witness.}\end{array}}$$

Genericity, staircase learning, trace exposure, and architectural or stochastic mechanisms can help establish the training-success premise. Analytic convergence and quantitative residual contraction contribute only their stated conclusions; statistical control remains separate. These mechanisms do not turn smooth activations, overparameterization, or a penalty in the objective into an unconditional guarantee of accurate learning. The training-success event concerns the returned model. It can hold even when regularized minibatch updates at that model have nonzero covariance; the finite-time return and its resource budget must still be justified.

8.21 The principle in one sentence

> *If a specified training procedure finds a computationally feasible predictor with a sufficiently good empirical fit and a valid, sufficiently small statistical-complexity bound, then the predictor generalizes and its own computation supplies a compositionally sparse approximation of the target, without requiring recovery of the computation that generated the data.*

8.22 Summary

The representation and recovery questions remain distinct. The simplification is that recovery itself has two levels: predictive accuracy and identification of generating structure.

Efficient digital evaluation provides a bounded-fan-in representation at the stated precision. A feasible learned network possesses such an implementation as well. This is the computational content of the compositional witness.

Overparameterization can make an existing interpolation set extended and degenerate. Analyticity, together with compactness of an all-time gradient flow, gives convergence to a critical point. A quantitative residual condition gives a different conclusion: contraction toward zero training error for the stated unregularized dynamics. Stochastic-volume calculations describe still another object, equilibrium preference among low-loss neighborhoods. Their conclusions and assumptions cannot be substituted for one another, and none alone establishes finite-time success of regularized SGD.

Exact interpolation and regularized stationarity have different noise properties. At the former, every fixed-example squared-loss data gradient vanishes. At the latter, only the mean data gradient

must cancel the regularizer; heterogeneous sample gradients sustain intrinsic minibatch fluctuations. The normalized ReLU common-equilibrium rank restriction provides a specific structural obstruction to all batch directions vanishing. This does not make the noise isotropic or prove a universal low-rank or global-optimization bias.

Norm- and product-norm-based regularization can favor well-fitting solutions in statistically controlled classes. When training actually returns such a solution and the relevant bound is informative, population accuracy follows. The conditional theorem makes these premises and their probabilities explicit. The predictor can be a finite stopped iterate; continued fluctuations of the training dynamics are compatible with this return-based notion of success. The stopping or averaging rule remains part of the algorithm whose guarantee must be established.

The norm quantity must match the architecture and activation assumptions. Statistical class control does not automatically confine the parameters or provide a positive lower bound on the training-prediction Jacobian. Those additional controls are required only when the particular learning proof relies on them.

Once a feasible predictor is accurate, its decomposition need not be unique, need not match the generating variables, and need not have been constructed by a staircase. A structural-recovery theorem is a stronger achievement; a staircase theorem can be a constructive explanation of learning, not a prerequisite added after predictive success.

The central question for regularized overparameterized SGD is therefore

> When does a specified regularized training procedure find,
> with high probability and feasible resources,
> a well-fitting predictor with adequate statistical control?

Solving that question establishes predictive learning in the stated regime. The compositional conclusion then follows from the computation performed by the learned predictor itself.

Part II
Computation and Algorithms

CHAPTER 9

Efficient Computability, Compositional Sparsity, and Attention

Efficient digital computation admits a polynomial-size representation as a directed acyclic graph of bounded-fan-in operations. Under additional regularity and stability assumptions, low-dimensional constituents also give favorable approximation rates.

How does this structural principle relate to attention-based architectures?

The connection is a division of computational labor. Attention can retrieve the inputs needed by a constituent, while a local nonlinear module computes the constituent itself. Repeated retrieval and transformation can therefore implement a compositional computation.

The central architectural picture is

$$\boxed{\textit{address} \longrightarrow \textit{retrieve} \longrightarrow \textit{compute locally} \longrightarrow \textit{store and repeat}.}$$

Several qualifications are essential. Dot-product attention is exactly Gaussian-kernel retrieval only under appropriate conditions. A normalized kernel smoother does not automatically inherit every approximation theorem for general radial-basis networks. Polynomial complexity in the number of precision bits is different from polynomial complexity in inverse approximation error. Finally, the existence of a routed compositional realization does not explain whether training will discover it.

This chapter makes these distinctions explicit.

9.1 The Architectural Connection

A compositionally sparse target has a realization

$$f = \Phi_G$$

on a computation DAG G whose internal nodes depend on at most k inputs.

For a node v with parents $u_1, \dots, u_{m_v}$,

$$z_v = g_v(z_{u_1}, \dots, z_{u_{m_v}}), \qquad m_v \le k. \tag{9.1}$$

Implementing this node requires two operations.

First, the machine must make the parent values available.

Second, it must apply the function g_v.

Attention offers one implementation of the first operation. A multilayer perceptron, or another suitable local approximator, offers an implementation of the second.

This separation is compatible with the organization of Transformers, which interleave attention with position-wise feed-forward transformations [109].

It is not an exclusive description of everything those components can do. Attention itself is nonlinear and can participate directly in computation. The point is that routing and local evaluation provide a particularly transparent constructive interpretation.

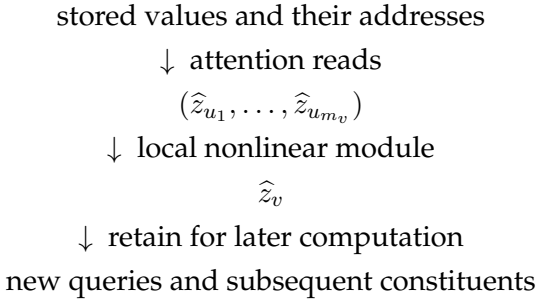


Figure 9.1: A routed implementation of a compositional node. Separate reads supply the ordered arguments of a local function. The computed output becomes available to subsequent nodes. This diagram describes an implementation of a supplied computation, not a theorem about the organization learned by gradient descent.

9.2 Attention: Definitions and Scope

For query $q \in \mathbb{R}^{d_k}$, keys $k_1, \ldots, k_N \in \mathbb{R}^{d_k}$, and values $v_1, \ldots, v_N \in \mathbb{R}^{d_v}$, define

$$\mathrm{Attn}(q; K, V) = \sum_{j=1}^{N} \alpha_j(q) v_j, \qquad \alpha_j(q) = \frac{\exp\left(q^\top k_j/\tau\right)}{\sum_{i=1}^{N} \exp(q^\top k_i/\tau)}, \tag{9.2}$$

where $\tau > 0$ absorbs the score scaling.

In a Transformer, the representations are commonly formed by linear maps:

$$q = W_Q h, \qquad k_j = W_K h_j, \qquad v_j = W_V h_j.$$

The term *self-attention* refers to the case in which the queries, keys, and values arise from the same collection of token representations. In cross-attention, the query collection and the key–value collection differ.

A bank of independently stored prototypes is therefore not automatically an ordinary self-attention layer. It is more directly a memory-attention or cross-attention construction. To implement it using self-attention, the memory entries must be represented as tokens and the permitted interactions must be specified.

Similarly, arbitrary stored values cannot simply be identified with $g(x_j)$ unless the architecture actually stores or computes those values. The formula $v_j = W_V h_j$ does not by itself provide arbitrary function samples.

These distinctions matter in constructive approximation arguments.

9.3 When Attention Is Gaussian-Kernel Retrieval

The kernel interpretation of attention is useful and has been developed explicitly in the literature [105]. Its exact form follows from a simple identity:

$$q^\top k_j = -\frac{1}{2}\|q - k_j\|_2^2 + \frac{1}{2}\|q\|_2^2 + \frac{1}{2}\|k_j\|_2^2. \tag{9.3}$$

The query-only term is common to every key and disappears under softmax normalization. The key-norm term generally does not.

9.3.1 Exact equivalence under equal key norms

Proposition 7 (Equal-key-norm Gaussian retrieval)**.** *Suppose all keys have the same norm:*

$$\|k_j\|_2 = c \qquad \text{for every } j.$$

Then, for every query q,

$$\alpha_j(q) = \frac{\exp\!\big(-\|q - k_j\|_2^2/(2\tau)\big)}{\sum_i \exp\!\big(-\|q - k_i\|_2^2/(2\tau)\big)}. \tag{9.4}$$

Proof. Substitute Equation (9.3) into the softmax. Both $\|q\|_2^2/(2\tau)$ and $c^2/(2\tau)$ are common to all terms and cancel. □

Normalizing both queries and keys to unit length is sufficient, but normalizing the query is not necessary for this identity. Equality of the key norms is the relevant condition.

9.3.2 Unequal key norms

Without equal key norms, the exact expression is

$$\alpha_j(q) = \frac{\exp\!\big(-\|q - k_j\|_2^2/(2\tau)\big)\exp\!\big(\|k_j\|_2^2/(2\tau)\big)}{\sum_i \exp\!\big(-\|q - k_i\|_2^2/(2\tau)\big)\exp\!\big(\|k_i\|_2^2/(2\tau)\big)}. \tag{9.5}$$

Thus ordinary dot-product attention includes a key-dependent factor in addition to distance-based similarity.

Alternatively, the corrected score

$$a_j(q) = q^\top k_j - \frac{1}{2}\|k_j\|_2^2 \tag{9.6}$$

produces exactly the Gaussian weights in Equation (9.4), without requiring equal key norms.

Approximate equality of key norms gives only an approximate correspondence. Its accuracy depends on the norm variation relative to τ, not merely on whether the norms look numerically similar.

9.3.3 Layer normalization is not query–key normalization

Layer normalization of a token representation does not generally force the projected keys to have equal norms.

Indeed,

$$\|W_K h\|_2^2 = h^\top W_K^\top W_K h,$$

which can vary with the direction of h even when $\|h\|_2$ is fixed.

Therefore an exact Gaussian equivalence must rely on an explicit normalization or score condition. It does not follow from the presence of LayerNorm in the surrounding Transformer block.

9.3.4 Learned Mahalanobis geometry

Suppose the query and keys are constructed using the same linear map:

$$q = Bx, \qquad k_j = Bx_j.$$

Then

$$\|q - k_j\|_2^2 = (x - x_j)^\top M(x - x_j), \qquad M = B^\top B \succeq 0. \tag{9.7}$$

Under the equal-key-norm condition or the corrected score above, attention is normalized Gaussian retrieval in this learned geometry.

If B is rank deficient, this is a pseudometric on the original space: distinct inputs differing only in $\ker B$ have zero distance.

If the projected vectors are subsequently normalized, the resulting geometry is instead that of the normalized feature map. It need not remain a fixed quadratic Mahalanobis geometry in the original inputs.

Finally, when $W_Q \neq W_K$, the score defines a general compatibility between two representations. It need not correspond to a symmetric positive-semidefinite kernel on the original input space.

The kernel viewpoint and the RKHS viewpoint should therefore not be identified without checking the required assumptions.

9.4 Kernel Averaging Is Not a General Approximation Theorem

When the stored values are function samples,

$$v_j = g(x_j),$$

Gaussian retrieval takes the Nadaraya–Watson form

$$\widetilde{g}(x) = \frac{\sum_j K_\tau(x, x_j) g(x_j)}{\sum_j K_\tau(x, x_j)}. \tag{9.8}$$

This is a normalized weighted average of sample values.

A general RBF network instead has the form

$$g_{\mathrm{RBF}}(x) = \sum_j c_j K_\tau(x, x_j),$$

with independently chosen coefficients c_j.

The two approximation families are different. In particular, a theorem about general RBF coefficients cannot be transferred automatically to Equation (9.8).

Higher-order approximation rates can depend on cancellation, local polynomial reproduction, boundary treatment, bandwidth selection, and the permitted value functions. None of these properties follows merely from writing a softmax as a Gaussian kernel.

We therefore do not deduce an arbitrary C^r approximation rate for a single attention head whose values are only $g(x_j)$.

Instead, the construction below assigns local approximation to a specified nonlinear module and uses attention to supply its arguments:

routing accuracy + constituent approximation accuracy.

9.5 Margins Give Quantitative Retrieval Bounds

The routing interpretation has a direct quantitative foundation.

Let

$$\alpha_j = \frac{e^{a_j/\tau}}{\sum_i e^{a_i/\tau}},$$

where a_j denotes the unscaled compatibility score.

9.5.1 Retrieving one item

Proposition 8 (Single-item retrieval)**.** *Suppose the desired item has index $j_\star$ and score margin*

$$a_{j_\star} - \max_{j \neq j_\star} a_j \geq \gamma > 0.$$

Then

$$1 - \alpha_{j_\star} \leq (N-1)e^{-\gamma/\tau}. \tag{9.9}$$

If $\|v_j\| \leq R$ for all stored values, then

$$\|\mathrm{Attn}(q; K, V) - v_{j_\star}\| \leq 2R(N-1)e^{-\gamma/\tau}. \tag{9.10}$$

Proof. Divide the softmax denominator by $e^{a_{j_\star}/\tau}$. Each competing term is at most $e^{-\gamma/\tau}$.

For the value error, write

$$\mathrm{Attn}(q; K, V) - v_{j_\star} = \sum_{j \neq j_\star} \alpha_j (v_j - v_{j_\star})$$

and apply the triangle inequality. □

Thus a prescribed retrieval error η can be obtained by choosing the score scale so that, for example,

$$\frac{\gamma}{\tau} \geq \log\left(1 + \frac{2RN}{\eta}\right).$$

This is a conditional statement. It assumes a margin; it does not explain how training creates that margin.

9.5.2 Near top-b sparsity

Order the scores as

$$a_{(1)} \geq \cdots \geq a_{(N)}.$$

Let $b < N$ and suppose

$$a_{(b)} - a_{(b+1)} \geq \Gamma > 0.$$

Define the mass outside the top b entries by

$$t_b = \sum_{j > b} \alpha_{(j)}.$$

The ratio of tail mass to retained mass satisfies

$$\frac{t_b}{1 - t_b} \leq \frac{N-b}{b} e^{-\Gamma/\tau}.$$

Consequently,

$$t_b \le \frac{\frac{N-b}{b}e^{-\Gamma/\tau}}{1+\frac{N-b}{b}e^{-\Gamma/\tau}} \le \frac{N-b}{b}e^{-\Gamma/\tau}. \tag{9.11}$$

If the retained weights are renormalized to sum to one, the difference between the full read and the truncated read is at most

$$2Rt_b.$$

For equal-norm Gaussian retrieval, the score gap corresponds to half the gap in squared distances.

These inequalities explain when a particular attention distribution is effectively sparse. They do not imply that every trained head must be sparse, or that sparse reads can always be found cheaply.

9.6 An Explicit Attention–MLP Computation Model

We now specify the architecture used in the constructive statements.

Let

$$f : [-1,1]^d \to [-R,R], \qquad R \ge 1,$$

have a DAG realization with s internal scalar nodes, depth L, and local arity at most k.

Assume each node computes

$$g_v : [-R,R]^{m_v} \to [-R,R], \qquad m_v \le k.$$

The evaluator contains the original inputs and the previously computed node values in an addressable memory. Each value is stored together with an identifier.

To evaluate node v, the machine issues m_v reads, one for each ordered parent. Their outputs form the tuple

$$\widehat{u}_v = (\widehat{z}_{u_1}, \dots, \widehat{z}_{u_{m_v}}).$$

A local MLP computes

$$\widehat{z}_v = \widetilde{g}_v(\widehat{u}_v).$$

Separate argument slots are important. One weighted average does not generally preserve all the ordered arguments needed by an arbitrary multivariate constituent.

Computed outputs are retained for subsequent nodes. The local modules may be node-specific, or selected from a module bank using node identifiers.

This is an attention–MLP memory architecture. A realization using self-attention must explicitly represent the memory and computation states as tokens, preserve their identifiers, and prevent reads from unavailable states.

The results below concern this specified organization. They are not claims about every fixed-width, fixed-depth Transformer variant.

9.6.1 A simple addressing construction

For a finite graph, assign each memory record a distinct one-hot tag

$$e_j \in \mathbb{R}^N, \qquad N = d + s.$$

The query for record j is e_j, and keys are the corresponding record tags.

The intended score is one and every other score is zero. Hence the margin is exactly one.

Tags and numerical values occupy separate fields, so changing a stored value does not change its address.

This construction is deliberately inefficient in address dimension, but its overhead is polynomial in graph size. It demonstrates existence without assuming that a learned representation already contains well-separated addresses.

More economical addressing schemes require their own separation and resource analysis.

9.7 Compositional Approximation with Routing Error

The advantage of local arity enters through approximation of the constituents, as in the compositional approximation theory developed earlier [85].

Suppose each g_v belongs to a bounded C^r class, with $r \geq 1$ an integer and

$$\|g_v\|_{C^r} \leq B.$$

For a constituent on m_v variables, standard deep ReLU constructions give uniform approximation error δ with a number of weights and computation units bounded by

$$P_v(\delta) \leq C\,\delta^{-m_v/r}\log\left(\frac{2}{\delta}\right), \tag{9.12}$$

and depth

$$D_v(\delta) \leq C\log\left(\frac{2}{\delta}\right),$$

where the constant depends on the local dimension, smoothness, domain, and norm bound [119].

The local MLP in this statement may itself have several hidden layers. It is not necessarily one fixed two-layer Transformer FFN.

9.7.1 Keeping intermediate values in their domains

Clip every approximate node output to $[-R, R]$.

Because the true output lies in this interval, clipping cannot increase its approximation error. Attention then averages bounded stored values, so the routed arguments also remain in the prescribed box.

This avoids silently evaluating a constituent approximator outside the domain on which its guarantee holds.

9.7.2 Propagation through the graph

Let λ_v be a Lipschitz constant of g_v in the maximum norm. For each topological level ℓ, set

$$\Lambda_\ell = \max\left\{1, \max_{v \text{ at level } \ell} \lambda_v\right\}.$$

Let e_ℓ bound the errors of all values computed through level ℓ.

Suppose a routing read at level ℓ differs from the intended stored value by at most η_ℓ, and every local MLP has approximation error at most δ.

The total error in a routed argument is at most

$$e_{\ell-1} + \eta_\ell.$$

Therefore,

$$e_\ell \le \Lambda_\ell e_{\ell-1} + \delta + \Lambda_\ell \eta_\ell. \tag{9.13}$$

The use of $\Lambda_\ell \ge 1$ also accounts for retaining earlier values across levels.

Define

$$A_G = \sum_{\ell=1}^{L} \prod_{j=\ell+1}^{L} \Lambda_j, \tag{9.14}$$

with an empty product equal to one.

If

$$\eta_\ell \le \frac{\delta}{\Lambda_\ell},$$

then induction gives

$$e_L \le 2A_G\delta.$$

9.7.3 A conditional approximation theorem

Theorem 8 (Approximation by routed local modules)**.** *Assume the DAG, constituent domains, smoothness bounds, and attention–MLP evaluator specified above.*

For every $\varepsilon \in (0,1)$, choose

$$\delta = \frac{\varepsilon}{2A_G}$$

and route each parent with error at most

$$\eta_\ell = \frac{\delta}{\Lambda_\ell}$$

at level ℓ.

Then the resulting evaluator satisfies

$$\sup_{x\in[-1,1]^d} |f(x) - \widehat{f}(x)| \le \varepsilon.$$

The total local-MLP complexity obeys

$$P_{\text{local}} \le Cs\left(\frac{2A_G}{\varepsilon}\right)^{k/r} \log\left(\frac{4A_G}{\varepsilon}\right). \tag{9.15}$$

There are L routing-and-computation rounds. With the local ReLU construction above, the sequential depth is bounded by

$$O\left(L \log\frac{4A_G}{\varepsilon}\right).$$

Address storage, routing maps, and attention evaluation incur additional costs that must be counted separately.

Proof. The margin bound supplies the required routing accuracy, for example using the one-hot addressing construction.

Approximate every constituent to accuracy δ, clip its output, and apply Equation (9.13). The chosen tolerances give $e_L \le \varepsilon$.

Summing Equation (9.12) over s nodes and using $m_v \le k$ gives the stated local complexity. □

9.7.4 What "dimension-independent" means here

The approximation exponent involves

$$k/r,$$

not

$$d/r.$$

But the total computation is not independent of d. It must still represent the inputs, store the graph, route values, and evaluate all necessary constituents.

Moreover, A_G can grow rapidly with depth. A favorable overall bound requires quantitatively stable composition.

Thus the appropriate conclusion is

local arity controls the approximation exponent, provided routing and composition are controlled.

It is not a universal dimension-free guarantee for arbitrary attention networks.

9.8 Keys, Parameters, and Operations Are Different Resources

The preceding construction uses at most $d + s$ stored value records during one evaluation. These are addresses of computational states, not RBF centers tabulating the constituent functions.

The constituent approximation cost is carried by the local MLPs.

Consequently, Equation (9.15) is not a theorem that

$$N_{\text{keys}} = O(s\varepsilon^{-k/r}).$$

It is a bound on a different resource.

At most ks parent reads are needed. If every read compares its query with all N memory entries, the number of query–key comparisons is at most

$$ksN.$$

The arithmetic cost of each comparison also depends on key dimension.

This remains polynomial for the explicit finite-graph construction, but it need not be computationally attractive. Sparse indexing or structured routing may reduce it.

Importantly, a sharply concentrated attention distribution does not automatically produce a faster implementation. Computing all scores and then discarding most of them can retain the cost of dense retrieval.

9.8.1 Functional arity and implementation fan-in

Even a dense attention read can be implemented using bounded-fan-in arithmetic. Dot products and normalization sums can be evaluated through binary reduction trees.

Thus

dependence on many stored values

does not imply

an elementary operation of unbounded fan-in.

This establishes compatibility with the digital compositionality principle. It does not establish that the learned attention pattern itself has bounded support.

9.9 Efficient Finite-Precision Computation

A different argument is needed for complexity polynomial in the number of precision bits.

If

$$\varepsilon = 2^{-p},$$

then

$$\varepsilon^{-k/r} = 2^{pk/r}.$$

Therefore a smooth approximation bound polynomial in $1/\varepsilon$ does not establish a bound polynomial in p.

The finite-precision route instead starts with the digital computation of the desired approximation.

9.9.1 From bit computation to a routed circuit

Suppose an algorithm, on an input string u of length n and precision request p, outputs a dyadic number y satisfying

$$|y - f(u)| \le 2^{-p}$$

in time

$$T(n, p) = \mathrm{poly}(n + p).$$

As established in Chapter 4, this computation admits a uniform polynomial-size bounded-fan-in Boolean circuit.

The circuit can be executed by storing input bits and previously computed gate outputs as addressable records.

Each gate retrieves its parent bits, applies a constant-size Boolean operation, and stores its output.

Soft attention introduces an additional issue: a retrieved bit is an average rather than an exact symbol. A local restoration map resolves this issue in the construction below.

9.9.2 Restoring discrete states after soft retrieval

For at most N stored records, use orthogonal address tags and inverse temperature

$$\beta = \tau^{-1} \ge \lceil \log(16N) \rceil .$$

The single-read bound gives error smaller than $1/8$ when values are bits and their norms are bounded by one.

Allow an additional numerical evaluation error smaller than $1/8$. The resulting read is still within $1/4$ of the correct bit.

Define the ReLU restoration map

$$\mathcal{R}(u) = 2\,\mathrm{ReLU}\left(u - \frac{1}{4}\right) - 2\,\mathrm{ReLU}\left(u - \frac{3}{4}\right). \tag{9.16}$$

It satisfies

$$\mathcal{R}(u) = 0 \quad \text{for } u \le \frac{1}{4}, \qquad \mathcal{R}(u) = 1 \quad \text{for } u \ge \frac{3}{4}.$$

Hence every sufficiently accurate read is restored to the exact intended bit before the next Boolean operation.

For example,

$$\mathrm{AND}(a,b) = \mathrm{ReLU}(a+b-1), \qquad \mathrm{NOT}(a) = 1-a$$

on Boolean inputs.

The gate output is therefore exact, and the next memory record again contains a discrete symbol rather than accumulated numerical error.

Proposition 9 (Polynomial-resource attention–MLP realization)**.** *Under the finite-precision computation assumption above, there exists a uniform family of attention–ReLU memory evaluators producing the same dyadic outputs as the digital algorithm.*

Their description size, working storage, sequential computation, and required numerical precision are polynomially bounded in $n+p$.

The construction uses explicit address tags, retained memory values, and local symbol restoration. It does not require an arbitrary exact-real parameter to encode the computation.

Proof. Compile the digital algorithm into a polynomial-size Boolean circuit.

Evaluate the circuit in topological order. Retrieve each gate input using attention, restore the retrieved bit with Equation (9.16), compute the Boolean gate, and retain its output.

Induction over the gates shows that every stored bit agrees with the circuit computation.

The circuit size and the number of memory records are polynomial. One-hot address representations and even dense attention therefore incur only polynomial overhead.

The score scale grows as $O(\log N)$. Numerically evaluating the normalized exponential sums to the fixed read tolerance requires only polynomial resources. The restoration and Boolean operations use fixed dyadic constants.

Finally, decoding the output bit string gives the dyadic answer of the original algorithm. □

9.9.3 Scope of the construction

This proposition is a constructive result for the specified attention–MLP memory model. It does not establish the same bounds for every prescribed Transformer architecture, normalization scheme, positional encoding, or fixed context size.

A Transformer realization must preserve the address and state fields required by the construction. Additional components cannot simply be inserted while assuming they leave the simulation unchanged.

For a real-domain target, the input string may encode a dyadic approximation to x. A polynomially controlled input-precision requirement then connects this result to the bit-model definition used earlier in the book.

This remains distinct from directly feeding arbitrary real scalars into a fixed network and asserting a uniform approximation rate.

9.10 Low-Dimensional Relevance and Low-Rank Projections

The relation between local arity and projection rank is conditional.

Suppose the information relevant to a particular read is already contained in

$$z = Ph, \qquad P : \mathbb{R}^D \to \mathbb{R}^m, \qquad m \le k.$$

If its queries and keys can be formed as

$$q = U_Q Ph, \qquad k_j = U_K Ph_j,$$

then the effective content projections

$$W_Q = U_Q P, \qquad W_K = U_K P$$

satisfy

$$\operatorname{rank}(W_Q) \le m, \qquad \operatorname{rank}(W_K) \le m.$$

Likewise, a Gaussian metric of the form

$$M = P^\top A^\top A P$$

has rank at most m.

Thus low-rank content projections suffice when the required relevance relation factors through an available low-dimensional linear representation.

9.10.1 What the rank statement does not imply

A bounded-arity computation graph does not establish that its parent variables are already available through a rank-k projection of the current token representation.

Nor does it bound the dimension needed for positional identifiers, memory addresses, or other routing metadata.

The explicit one-hot construction above illustrates this distinction: local computation has small arity, while addressing may use a much larger space.

The ranks of W_Q and W_K must also be distinguished from the rank of the attention matrix after softmax. Entrywise exponentiation and row normalization need not preserve low matrix rank.

For example, a rank-one score matrix with entries

$$a_{ij} = q_i j, \qquad j = 0, \dots, N-1,$$

produces exponential entries

$$e^{a_{ij}/\tau} = \left(e^{q_i/\tau}\right)^j.$$

For distinct q_i, this is a full-rank Vandermonde matrix. Row normalization preserves that rank.

Therefore low-rank scores do not, by themselves, imply low-rank attention weights.

9.11 Empirical Questions and Predictions

The constructive results suggest experiments, but they do not establish universal properties of trained models.

For example, head-pruning studies show that some trained systems can retain performance after removing many heads [75]. This concerns redundancy across heads. It is not a direct test of top-b sparsity within a head, projection rank, or recovery of a bounded-arity target.

9.11.1 Routing accuracy versus constituent accuracy

Use a synthetic target with a known computation DAG.

Compare a system with supplied parent addresses against one that must learn the routing. Separately vary the accuracy of the local modules.

Measure routing errors, local function errors, and final prediction error. This tests whether the proposed decomposition identifies the actual bottleneck.

9.11.2 Margin and truncation

For each read, measure the score gap, discarded attention mass, and change in the retrieved value after top-b truncation.

The margin inequalities provide quantitative upper bounds. A successful experiment should also account for the cost of finding the retained entries rather than reporting sparsity alone.

9.11.3 Local dimension and projection rank

Construct tasks whose relevant local variables lie in known m-dimensional subspaces. Vary the permitted content-projection rank while keeping address metadata separate.

The conditional prediction is that sufficient rank should track the dimension needed for the relevance relation, not necessarily the ambient token dimension or the total number of inputs.

9.11.4 Depth, precision, and restoration

Evaluate known computations over increasing horizons while varying temperature, numerical precision, and the use of discrete restoration.

This separates inaccurate local arithmetic from failure to preserve the state required by later queries.

9.11.5 Learning the decomposition

Finally, compare supplied routing and intermediate supervision with end-to-end training from final input–output pairs.

Success of the supplied construction establishes an implementation baseline. The additional cost of learning addresses, modules, and intermediate states measures the recovery problem left unresolved by the representation theorem.

9.12 What Remains Open

The main unresolved questions concern discovery and efficient implementation.

Can training learn representations in which the correct parents are retrievable with adequate margins?

When can routing be made inexpensive without computing all query–key scores?

Which compositional targets admit stable intermediate representations so that local errors do not grow prohibitively with depth?

When do low-rank content maps or sparse reads emerge from the learning objective, rather than being imposed by construction?

And when does an architecture trained end to end recover useful constituents rather than merely interpolate with another representation?

These are not answered by the Gaussian identity, the margin bound, or the existence of a polynomial-size digital realization.

9.13 Relation to C1, C2, and C3

The division of labor matches the framework of the preceding chapters.

C1 states that a feasible compositional representation exists. For suitable smooth constituents, its local arity can control the approximation exponent.

Attention provides one possible mechanism for bringing the required arguments together. Local nonlinear modules evaluate the constituents.

C2 concerns whether the data expose informative signals from which the learner can discover those representations and routing decisions.

C3 can supply intermediate variables or computational traces directly, reducing the amount of hidden structure that must be inferred.

Thus

$$\begin{aligned} \text{C1} &: \text{a feasible computation exists,} \\ \text{attention–MLP architecture} &: \text{a possible implementation,} \\ \text{C2/C3} &: \text{information for recovering that implementation.} \end{aligned}$$

The ability to execute a supplied graph must not be mistaken for the ability to learn its structure.

9.14 Conclusion

Attention connects naturally to compositional computation, but the connection is more precise than the statement that every attention head is a universal local kernel approximator.

Under equal key norms, dot-product attention is exactly normalized Gaussian retrieval. With unequal norms, the correspondence requires a key-dependent correction or a more general compatibility interpretation.

Margins give explicit retrieval and truncation bounds. They do not guarantee that training produces sparse attention.

Local nonlinear modules can approximate bounded-arity constituents, while attention routes their inputs. The resulting approximation cost depends on local dimension, graph size, and error propagation.

Polynomial-time finite-precision computation admits a separate attention–MLP construction through Boolean circuits and discrete state restoration. This argument counts precision bits explicitly and does not rely on smooth approximation rates.

The chapter's main message is therefore

compositional sparsity specifies a feasible computation;

attention can route the information needed to execute it.

Whether training discovers the appropriate queries, addresses, constituents, and intermediate variables remains the central recovery question.

CHAPTER 10

Hardware for Compositionally Sparse Computation (with J. Bates)

The central thesis of this book—that efficiently computable functions *are compositionally sparse—has profound implications for hardware. If the mathematical structure of efficiently computable functions consists of bounded-fan-in DAGs with local dependencies, then the optimal physical substrate is not a monolithic matrix multiplier, but* **a 2D mesh of simple processing elements with nearest-neighbor communication** *– as suggested by Joe Bates. This chapter argues for a "structural alignment" between algorithm and hardware. We explore architectures where the physical layout mirrors the computational DAG, minimizing energy spent on data movement and maximizing locality. We specifically analyze how Transformer blocks (MLP and Attention) map onto such meshes using approximate HDR arithmetic and localized routing.*

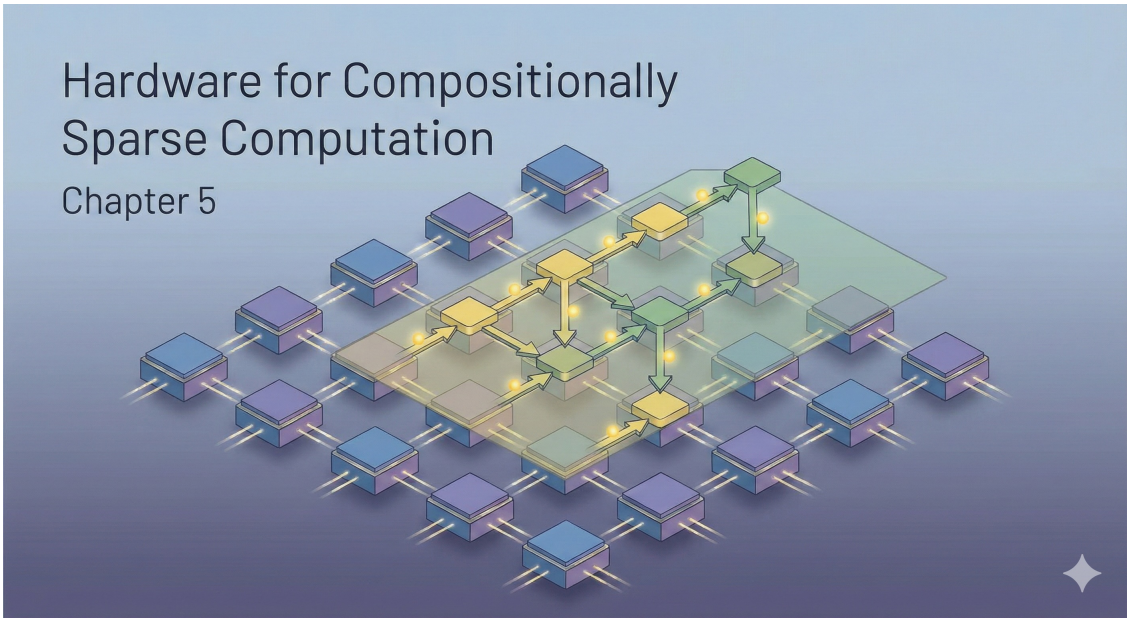


Figure 10.1

10.1 The Structural Alignment Argument

The prevailing trend in AI hardware has been to build massive, centralized matrix engines (like GPUs and TPUs) designed for dense, all-to-all connectivity [49]. This approach assumes that the underlying computation is generic dense matrix multiplication.

However, as established in Chapter 4, **efficiently computable functions** are not generic dense blobs. They are **Compositionally Sparse**: they decompose into directed acyclic graphs (DAGs) where:

- **Bounded Fan-in:** Each node depends on a small number of inputs.
- **Locality:** Modules interact primarily with their logical neighbors.

This theoretical insight suggests a different hardware paradigm. Instead of building hardware for dense connectivity, we should build hardware that is **isomorphic** to the sparse compositional structure.

- **The Mathematical Structure:** A graph of local, low-arity operations.
- **The Physical Structure:** A 2D mesh of simple Processing Elements (PEs) with local interconnects [7].

By mapping the logical parents of a node to physical neighbors on the chip, we convert "wiring costs" (which dominate energy consumption) into "local hops," aligning the physics of the chip with the math of the function [56].

[Image of 2D mesh network topology vs crossbar]

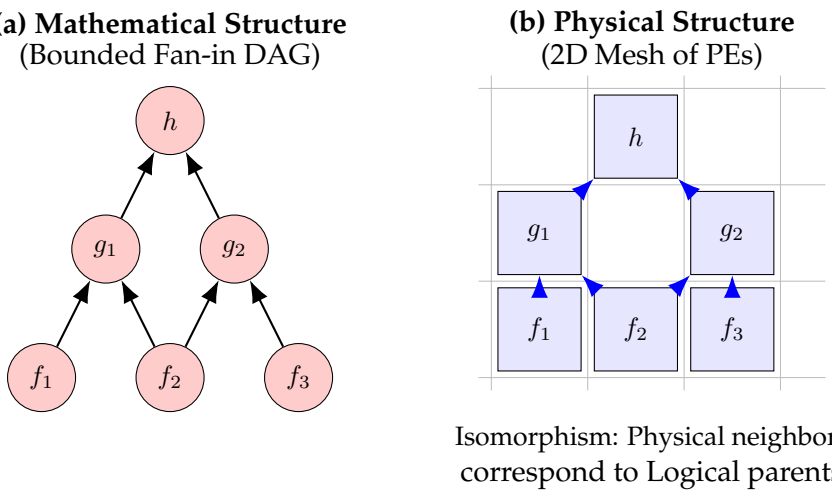


Figure 10.2: Structural Alignment. The bounded fan-in DAG of an efficiently computable function (a) maps naturally onto a 2D Mesh topology (b). By placing logically connected nodes on physically adjacent tiles, communication remains local, avoiding the need for global crossbars.

10.2 System and Numeric Model

Compute Fabric. We assume a 2D mesh of P simple processing elements arranged as a $\sqrt{P} \times \sqrt{P}$ grid.

- **Approximate Arithmetic Units:** Leveraging Logarithmic Number Systems (LNS) or stochastic logic to perform high-dynamic range operations with low area cost [38, 8];
- Small local SRAM ("tile memory") for vector tiles and KV storage;
- Four nearest-neighbor links (N/E/S/W) for hop-by-hop routing.

The vision is to build a *locality-driven* architecture that trades bit-perfect deterministic accuracy for massive parallelism. By eschewing wide crossbars and IEEE floating-point cores, we prioritize wire efficiency and density [57].

Numeric Format: The Case for Approximate HDR. Instead of standard integers, we utilize a High Dynamic Range (HDR) format with approximate semantics (e.g., LNS). In the log domain, multiplication reduces to fixed-point addition ($\log(xy) = \log x + \log y$), drastically reducing the transistor count of the Processing Elements. Crucially, HDR supports the vast numerical scales encountered during backpropagation (gradients ranging from 10^{-7} to 10^{2}), enabling *training* on the mesh. We assume a relative error tolerance of $\approx 1\%$, which deep networks have been shown to absorb without degradation [9, 76].

10.3 Mapping Compositional DAGs to 2D Meshes

We consider DAGs G of depth L with layer widths W_ℓ and fan-in $\leq k$. A natural layout places each layer ℓ on a $\sqrt{W_\ell} \times \sqrt{W_\ell}$ patch with parents at layer $\ell-1$ assigned to adjacent patches.

Proposition 10 (2D schedule for bounded fan-in DAGs)**.** *Let each layer ℓ be placed on a $\sqrt{W_\ell} \times \sqrt{W_\ell}$ region. Then the latency to realize layer ℓ on a 2D mesh is*

$$T_\ell = T_{comp,\ell} + T_{comm,\ell},$$
$$T_{comm,\ell} = \mathcal{O}(kD_\ell + \mathrm{cong}_\ell),$$

where D_ℓ is the Manhattan diameter of dependencies and cong_ℓ is max per-link congestion. If edge placement uses locality-aware heuristics (e.g., hypergraph partitioning), then $D_\ell = \mathcal{O}(\sqrt{W_\ell})$ in general, and $D_\ell = \mathcal{O}(1)$ if G has bounded treewidth.

Constraint: Hierarchical Locality and Rent's Rule. We must address a subtle topological constraint: bounded fan-in alone is *insufficient* to guarantee a compact 2D embedding. A random sparse graph (an expander) has bounded fan-in but requires global wires that scale with the square root of the chip area, destroying locality.

For the structural alignment argument to hold, the computational DAG must exhibit *hierarchical locality*. Formally, the graph must satisfy a generalized **Rent's Rule** [59], where the number of terminals T required to communicate with a cluster of G logic gates scales as:

$$T \propto G^p \tag{10.1}$$

To map efficiently to a 2D mesh without excessive congestion, the Rent exponent must satisfy $p \leq 0.5$. We conjecture that *efficiently computable functions* naturally satisfy this geometric constraint, as they typically decompose into semi-independent modules (objects, concepts) rather than monolithic global dependencies.

[Image of Rent's rule log-log plot for VLSI circuits]

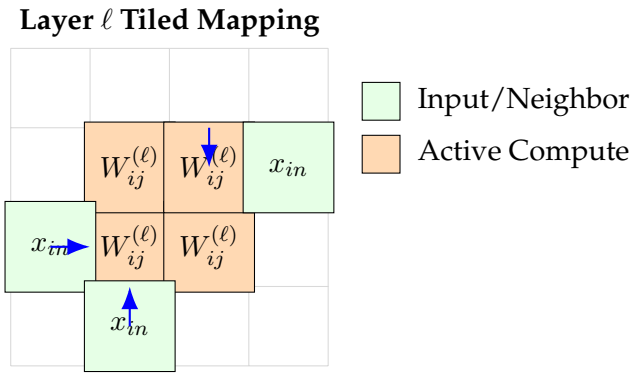


Figure 10.3: Tiled Mapping of a Neural Layer. A large layer W_ℓ is decomposed into tiles distributed across the mesh. Data flows from neighboring regions (Layer $\ell - 1$) to the active computation tiles using only local routing, avoiding global broadcasts.

MLP Modules. When viewed as feedforward layers with fan-in k, the MLPs in a transformer are ideally suited to such layout. Each layer can be broadcast in tiles, and inputs fetched from neighbors within a few hops. With sparsity or block structure in the weights (e.g., compositionally local weight matrices), compute remains bounded and placement near-optimal.

Speculatively, MLP submodules might be structured as locally reused kernels applied in a recurrent fashion over overlapping scopes [20]. This suggests a kind of *sliding compositionality*, where each kernel operates over a spatial patch of nearby tokens (or semantic units). In this case, the 2D mesh becomes a physical substrate for composing locality-aware update modules.

10.4 Sparse Attention on 2D Meshes

We explore two strategies, KV-stationary and Q-stationary, for evaluating top-s attention in a distributed mesh.

10.4.1 KV-Stationary: Coarse-to-Fine Attention

Distribute keys and values across PEs; queries are routed to buckets [52]:

1. **Coarse filter**: compute $c(q) = \text{sign}(Bq)$, a low-rank code of q;
2. **Fine score**: at bucket PEs, evaluate $\langle q, k_i \rangle$ and select top-s;
3. **Mix**: normalize (softmax-lite) and compute weighted sum $\sum_i \alpha_i v_i$.

This coarse-to-fine filtering exploits structure in the attention map and avoids global broadcasts.

10.4.2 Q-Stationary: Systolic Streaming with Early Exit

Store queries locally, and stream (K, V) across PEs. Maintain top-s heaps in-place; prune keys that cannot contribute based on conservative score bounds [23].

[Image of systolic array architecture diagram]

Speculative Opportunity. If attention graphs are induced by a latent hierarchical function (e.g., a parse or program tree), then coarse bucketings might correspond to semantic equivalence classes. The mesh becomes a canvas for learning or reflecting emergent structure.

10.5 AM Quantization and Error Analysis

We adopt two softmax approximations compatible with integer hardware:

- **Blockwise softmax**: via integer max and LUTs [120];
- **Tempered max-mix**: softmax-lite using ReLU and shifts only.

Error bounds link quantization precision Δ to output fidelity. We may derive end-to-end degradation and provide explicit tolerances for b-bit quantizers [25, 36].

10.6 Discussion: Why 2D Meshes Help

- **Local fan-in and attention sparsity** shrink receptive fields, making local routing sufficient.
- **Spatial bucket locality** and compositional weight reuse enable static co-placement of related scopes [21].
- **Fixed-point compatibility** ensures bit-serial or SIMD units suffice, reducing energy and area.
- **Emergent structure discovery**: If learning induces local composition or symmetry, the mesh layout might co-adapt.

10.7 Conclusion

We presented a speculative but rigorous framework for mapping compositionally sparse computations—typical of transformer blocks—to 2D meshes of simple processing elements. Such mappings exploit low arity, sparse attention, and localized updates to minimize communication. The result is a hardware substrate that leverages structural regularities in both the algorithm and the data, favoring integer arithmetic, approximate softmax, and hop-local routing. Future work may explore adaptive mesh reconfiguration, dynamic bucketing strategies, and learning-augmented placement.

The broader implication is that sparsity—far from being a nuisance—may be a blessing. Compositional structure enables hardware mappings with minimal overhead and supports scalable, efficient deployment of deep learning systems on modest, spatially organized fabrics.

CHAPTER 11

Stepwise Trace Supervision: A Common Computational Principle for Autoregressive and Denoising Models

A complex computation can become easy to learn when its intermediate states are exposed.

Consider a polynomial-time computation generating a sequence of states

$$s_0, s_1, \ldots, s_T.$$

The end-to-end map

$$s_0 \longmapsto s_T$$

may be computationally deep. Nevertheless, for a Turing computation, the transition from one configuration to the next is local: each cell at time $t+1$ depends on only a constant-size neighborhood at time t.

If the learner is supplied with sufficient examples of these local transitions, the global computation can be recovered by learning a local rule and iterating it.

The central principle is therefore

complex global computation
$\Downarrow$
sequence of simple local transitions
$\Downarrow$
expose enough intermediate structure
$\Downarrow$
learn the local transition
$\Downarrow$
iterate it.

This is the strong trace-exposure regime C3 of Chapter H.

An important distinction will be made throughout the chapter:

exposing trajectories $\neq$ providing the complete local transition table.

A finite set of trajectories may fail to visit local configurations that will later occur on new inputs. Thus trace supervision removes the need to infer hidden intermediate states, but does not automatically remove the problem of local-state coverage.

The chapter also clarifies a structural connection with autoregressive and denoising models. Autoregressive systems advance through an indexed trajectory. Diffusion models construct supervised prediction problems at many noise levels. Both exploit intermediate states, but ordinary language tokens and ordinary diffusion states are not, in general, faithful traces of an underlying Turing computation.

The result is therefore a constructive theorem about C3, not a theorem that existing autoregressive or diffusion training automatically satisfies C3.

11.1 Introduction

Autoregressive language models and diffusion models appear very different.

An autoregressive model predicts a later symbol from an earlier context,

$$x_{\leq t} \longmapsto x_{t+1}.$$

A diffusion model instead learns local statistical information at multiple noise levels and uses it to construct a reverse-time generative process.

Both therefore replace one global transformation by a sequence or family of intermediate prediction problems.

The question of this chapter is narrower and more mathematical:

> How powerful can learning become if the intermediate states of a computation itself are supplied?

For polynomial-time Turing computation, the answer is particularly clean.

A Turing computation can be written as repeated application of a fixed constant-arity local rule.

If that local rule is fully specified by the training data, it can be recovered from a finite table and then iterated for polynomially many steps.

The load-bearing assumption is therefore

the relevant local computational transitions are exposed.

This is much stronger than ordinary end-to-end supervision

$$(x, f(x)).$$

It is an instance of C3 rather than a theorem about black-box C2 recovery.

11.2 Preliminaries

11.2.1 Polynomial-time computation

Let

$$f : \Sigma^n \to \Sigma^m$$

be computed by a deterministic Turing machine M in time

$$T(n) = \mathrm{poly}(n).$$

Let

$$\tau_t$$

denote the machine configuration after t steps.
A configuration contains

- the tape contents,
- the head position,
- the finite internal state.

A computation lasting $T(n)$ steps can visit only

$$O(n + T(n))$$

tape positions.
Thus a polynomial-size tape window suffices.

11.2.2 A local alphabet

Let Γ be the tape alphabet and Q the finite set of control states.
Represent each tape location by an element of a finite augmented alphabet

$$\mathcal{A}.$$

A symbol of $\mathcal{A}$ records

$$\text{tape symbol} + \text{head-presence flag} + \text{machine state when the head is present.}$$

For a fixed machine M,

$$|\mathcal{A}| = O(1).$$

Let

$$S(n) = O(n + T(n))$$

be the number of represented tape cells.
The complete state is therefore

$$s_t = (s_{t,1}, \ldots, s_{t,S}) \in \mathcal{A}^S. \tag{11.1}$$

A one-hot encoding of every cell has total dimension

$$D(n) = S(n)|\mathcal{A}| = \mathrm{poly}(n).$$

11.2.3 Three forms of supervision

We distinguish three levels.

End-to-end supervision. Only

$$(x, f(x))$$

is observed.

Full trajectory supervision. For each training input, the complete state sequence

$$s_0, s_1, \ldots, s_T$$

is observed.

Complete local supervision. The learner is supplied with the value of the local transition on every local state that may be needed:

$$(u, h(u)).$$

The strongest theorem below uses complete local supervision.

Observed trajectories provide such supervision only for the local patterns that those trajectories actually visit.

That distinction will lead to a separate coverage theorem.

11.3 Turing Computation Is a Local Space–Time Rule

The central fact is the locality of one Turing transition.

Lemma 4 (Locality of a Turing transition)**.** *For a fixed deterministic Turing machine M, there exists a function*

$$h_M : \mathcal{A}^3 \to \mathcal{A}$$

such that, for every nonboundary cell,

$$s_{t+1,i} = h_M\left(s_{t,i-1}, s_{t,i}, s_{t,i+1}\right). \tag{11.2}$$

Boundary cells are handled with fixed blank symbols.

Proof. A cell can change at the next Turing step only through information contained in itself and its two nearest neighbors.

The head may currently occupy the cell and write a symbol.

It may occupy the left neighbor and move right.

It may occupy the right neighbor and move left.

Otherwise the tape symbol is unchanged and no head enters the cell.

The finite transition table of M determines every case.

Hence a fixed map

$$h_M : \mathcal{A}^3 \to \mathcal{A}$$

exists. □

The complete global transition

$$H_M : \mathcal{A}^S \to \mathcal{A}^S$$

is obtained by applying h_M in parallel across the tape.
Thus

$$s_{t+1} = H_M(s_t).$$

The structural point is

global computational depth is generated by iteration of a constant-arity local rule.

11.4 Learning the Local Rule

Since

$$|\mathcal{A}| = O(1),$$

the number of possible three-cell neighborhoods is

$$|\mathcal{A}|^3 = O(1)$$

for a fixed machine.
Let

$$\mathcal{U}_M \subseteq \mathcal{A}^3$$

denote the set of local neighborhoods that may occur in valid computations of M.
Define the complete local table

$$\mathcal{D}_M^{\text{complete}} = \{(u, h_M(u)) : u \in \mathcal{U}_M\}. \tag{11.3}$$

Its size depends on M but not on the input length n.

11.4.1 Constant-size realization

Because h_M has finite domain, it admits a constant-size Boolean circuit.
Equivalently, lift each neighborhood to a one-hot vector

$$\psi(u) \in \{0,1\}^{|\mathcal{U}_M|}.$$

Each output coordinate can then be expressed as an OR of indicators corresponding to the patterns yielding that output.
Thus the local rule can be realized by a constant-size threshold or Boolean network.
No claim is required that the raw local map itself be linearly separable.

11.5 Trace-Exposed Universality with Complete Local Supervision

We can now state the cleanest theorem.

Theorem 9 (Local-rule universality)**.** *Let M be a deterministic Turing machine computing*

$$f : \Sigma^n \to \Sigma^m$$

in time

$$T(n) = \mathrm{poly}(n).$$

Then there exist

1. *a state representation of dimension*
$$D(n) = \mathrm{poly}(n);$$
2. *a constant-arity transition rule*
$$h_M : \mathcal{A}^3 \to \mathcal{A};$$
3. *a finite complete local table*
$$\mathcal{D}_M^{\mathrm{complete}};$$

such that exact recovery of h_M from $\mathcal{D}_M^{\mathrm{complete}}$, followed by $T(n)$ iterations of the corresponding global update H_M, computes f exactly.

Proof. Lemma 4 gives the local transition.

Because $\mathcal{A}$ is finite and independent of n, the complete truth table of h_M has constant size.

A learner consistent with $\mathcal{D}_M^{\mathrm{complete}}$ therefore recovers the rule on every locally admissible configuration.

Applying that rule simultaneously at all represented tape positions gives

$$s_{t+1} = H_M(s_t).$$

Iteration yields

$$s_T = H_M^T(s_0).$$

The initial state encodes the input and the final state contains the output $f(x)$.

Since both the tape window and the running time are polynomial in n, the complete execution uses polynomial resources. □

11.5.1 What the theorem proves

The theorem gives

$$\boxed{\text{complete local supervision} \Longrightarrow \text{exact local-rule recovery} \Longrightarrow \text{exact global execution.}}$$

It is intentionally stronger than the claim that a few example trajectories are available.

That stronger claim requires a coverage condition.

11.6 Learning from Observed Traces: The Coverage Problem

Suppose training consists of complete trajectories generated by inputs

$$x^{(1)}, \ldots, x^{(N)}.$$

Each trajectory reveals many local training pairs

$$\left((s_{t,i-1}, s_{t,i}, s_{t,i+1}), s_{t+1,i}\right).$$

Let

$$\mathcal{U}_{\text{train}}$$

be the set of distinct local neighborhoods observed anywhere in the training trajectories.

For a new test input x, let

$$\mathcal{U}_{\text{test}}(x)$$

be the set of local neighborhoods encountered during its computation.

Theorem 10 (Exact execution under local-state coverage)**.** *Assume deterministic local dynamics and noiseless observed traces.*

Suppose a learner stores the correct value $h_M(u)$ for every

$$u \in \mathcal{U}_{\text{train}}.$$

For any test input x satisfying

$$\mathcal{U}_{\text{test}}(x) \subseteq \mathcal{U}_{\text{train}}, \tag{11.4}$$

iteration of the learned local rule reproduces the exact Turing computation on x.

Proof. At time zero the simulated state agrees with the true state.

Assume the two states agree at time t.

Every local neighborhood required to compute time $t+1$ belongs to $\mathcal{U}_{\text{test}}(x)$ and therefore, by Equation (11.4), to $\mathcal{U}_{\text{train}}$.

The learned rule is correct on every such neighborhood.

Hence every updated cell is correct.

Induction over time proves exact agreement through the final state. □

This gives an important refinement of C3:

trace exposure removes hidden-state discovery, but not necessarily local-state coverage.

A trajectory can reveal every intermediate state it visits and still fail to reveal a rare local configuration needed on a later input.

There are then two possible routes:

coverage:	$\mathcal{U}_{\text{test}} \subseteq \mathcal{U}_{\text{train}}$,
structural generalization:	infer unseen local transitions from a restricted hypothesis class.

For the finite lookup learner, coverage is the exact condition.

11.7 A Universal Interpreter

The preceding theorem uses a machine-specific local rule h_M.

A stronger conceptual statement follows by fixing one universal Turing machine.

Corollary 1 (One fixed local rule is computationally universal)**.** *There exists a finite alphabet $\mathcal{A}_U$ and one fixed local rule*

$$h_U : \mathcal{A}_U^3 \to \mathcal{A}_U$$

with the following property.
For every deterministic Turing machine M and input x, an encoding

$$\langle M, x \rangle$$

can be placed in the initial state of the universal machine U so that iteration of the same rule h_U simulates $M(x)$.

Thus

$$\boxed{\text{fixed local interpreter} + \text{variable initial program and data} \Longrightarrow \text{universal computation.}}$$

Once h_U has been learned or supplied, different algorithms require no change in the local transition rule.

Only the externally supplied state changes.

This gives a useful bridge to the later distinction between interpreter and program:

$$\boxed{\text{interpreter} = h_U, \qquad \text{program} = \text{part of } s_0.}$$

It also emphasizes why computational universality alone is not a learning theorem.

The interpreter may be fixed and perfectly known while the problem of finding a useful program remains difficult.

11.8 Architectural Depth versus Computational Time

Trace exposure does not eliminate computational depth.

The machine still performs

$$T(n)$$

sequential transitions.

What changes is where the complexity resides.

Instead of learning one global map

$$s_0 \longmapsto s_T,$$

the learner acquires a reusable transition

$$H_M$$

and iterates it:

$$s_0 \overset{H_M}{\longmapsto} s_1 \overset{H_M}{\longmapsto} \cdots \overset{H_M}{\longmapsto} s_T.$$

Thus

architectural depth can be traded for iterative computational time.

The computation is not made shallow.
The *learned rule* is made local and repeatedly reused.

11.9 Autoregressive Indexing

The natural indexing convention is

$$s_t \longmapsto s_{t+1}.$$

Training examples contain adjacent state pairs

$$(s_t, s_{t+1}),$$

or, more locally,

$$(s_{t,i-1}, s_{t,i}, s_{t,i+1}) \longmapsto s_{t+1,i}.$$

This has the formal organization of autoregressive state prediction.
The analogy with language modeling is nevertheless limited.
Ordinary tokens usually expose emitted symbols, not all latent variables needed by the internal computation.
Hence

autoregressive factorization $\neq$ faithful computational trace.

11.9.1 When autoregressive supervision approaches C3

The analogy becomes closer when the sequence explicitly contains computational state, for example

- program execution traces,
- worked mathematical steps,
- intermediate plans,
- state–action trajectories,
- tool outputs,
- externally verified scratchpad states.

Such data can expose portions of the hidden computation.
But even then the coverage issue of Section 11.6 remains.

11.10 Reverse Indexing of the Same Computation

A second indexing convention makes the same computation appear to move toward decreasing time labels.

Define

$$y_t = s_{T-t}. \tag{11.5}$$

Then

$$y_T = s_0$$

contains the input, while

$$y_0 = s_T$$

contains the output.

Moreover,

$$y_{t-1} = s_{T-t+1} = H_M(s_{T-t}) = H_M(y_t).$$

Thus

$$y_t \longmapsto y_{t-1} \tag{11.6}$$

uses exactly the same forward computational rule.

The important distinction is

$$\boxed{\text{reverse external index} \neq \text{inverse computational transition.}}$$

No map

$$H_M^{-1}$$

is required.

Indeed, a Turing transition need not be invertible.

This construction is useful only as an indexing observation. It should not be interpreted as claiming that ordinary diffusion models implement Turing-machine transitions.

11.11 Reverse Diffusion Is Not Inverse Heat Flow

There is an apparent paradox in diffusion models.

Forward Gaussian smoothing destroys fine-scale information, while direct inversion of the heat equation is classically unstable.

How, then, can a generative diffusion model run "backward"?

The resolution is

$$\boxed{\text{reverse diffusion is not direct inversion of the heat operator.}}$$

11.11.1 Why direct inversion is unstable

Consider

$$\frac{\partial p_t}{\partial t} = \Delta p_t. \tag{11.7}$$

Its solution is

$$p_t = G_t * p_0.$$

In Fourier coordinates,

$$\widehat{p}_t(\xi) = e^{-t\|\xi\|^2}\widehat{p}_0(\xi), \tag{11.8}$$

up to the Fourier convention.
Formal inversion requires

$$\widehat{p}_0(\xi) = e^{t\|\xi\|^2}\widehat{p}_t(\xi).$$

Thus a small high-frequency perturbation is exponentially amplified.
Exact deconvolution is therefore ill-conditioned.

11.11.2 The diffusion-model problem

A diffusion model does not first estimate p_t and then apply this inverse operator.
Instead it learns the score

$$s_t(x) = \nabla_x \log p_t(x)$$

across noise levels.
For a forward SDE

$$dX_t = b(X_t, t)\,dt + g(t)\,dW_t,$$

the reverse-time dynamics involve the score field.
Thus the learned object is a family

$$\{s_t\}_{0\le t\le T},$$

not a single inverse heat operator.

11.11.3 The score as a denoising quantity

For additive Gaussian corruption

$$X_t = X_0 + \sigma_t Z, \qquad Z \sim \mathcal{N}(0, I),$$

Tweedie's identity gives

$$\boxed{\mathbb{E}[X_0 \mid X_t = x] = x + \sigma_t^2 \nabla_x \log p_t(x).} \tag{11.9}$$

More generally, when

$$X_t = \alpha_t X_0 + \sigma_t Z,$$

$$\mathbb{E}[X_0 \mid X_t = x] = \frac{x + \sigma_t^2 s_t(x)}{\alpha_t}. \tag{11.10}$$

The score therefore contains local denoising information.

The reverse-SDE update is not literally the posterior mean $\mathbb{E}[X_0 \mid X_t]$, but both depend on the same score field.

11.12 Diffusion Supplies Synthetic Intermediate States

The analogy with C3 should be formulated carefully.

For many common diffusion parameterizations, training samples

$$X_0 \sim p_0, \qquad t \sim \nu, \qquad Z \sim \mathcal{N}(0, I)$$

and constructs

$$X_t = \alpha_t X_0 + \sigma_t Z. \tag{11.11}$$

Thus the training mechanism can manufacture supervised problems at many controlled noise levels.

Schematically,

$$\boxed{X_0 \longrightarrow X_t \longrightarrow \text{noise-, score-, or denoising target.}}$$

We call these *synthetic intermediate states*.

The term "trace" should be reserved for settings in which an actual coupled sequence

$$X_{t_0}, X_{t_1}, \ldots, X_{t_m}$$

is sampled or otherwise supplied.

Ordinary score training does not require a complete forward trajectory.

The relevant commonality with C3 is therefore

the training procedure explicitly creates intermediate supervised states rather than exposing only a terminal inp

11.12.1 Intermediate does not mean easy

The score

$$s_t(x) = \nabla \log p_t(x)$$

may itself be a complicated high-dimensional function.

Nothing about the diffusion construction proves that each score field is

$$\text{compositionally sparse}, \qquad \text{statistically easy}, \qquad \text{or efficiently recoverable}.$$

Gaussian smoothing suppresses fine-scale Fourier structure, but this is not by itself a learning-theoretic sample-complexity theorem.

Thus

stepwise supervision changes the recovery problem; it does not prove that every step is easy.

11.13 Relation to Standard Diffusion Models

Standard diffusion and trace-exposed computation share the use of intermediate indexed states.

But the semantics differ.

In the Turing construction,

$$s_t$$

and

$$s_{t+1}$$

are different clean computational configurations.

In a standard diffusion process,

$$X_t$$

and

$$X_{t-\Delta t}$$

are different noise-level representations associated with the same data distribution and, in common forward constructions, the same underlying clean example.

Therefore

standard diffusion $\neq$ computational-trace supervision.

The common principle is only

replace one difficult global transformation by supervised or learned intermediate problems.

11.14 Gaussian-Corrupted Computational States

A more literal bridge can be constructed artificially.

Suppose a genuine computational state is observed through Gaussian noise.

Encode each local alphabet symbol by a one-hot block of amplitude C:

$$e(s_{t,i}) \in \{0, C\}^a, \qquad a = |\mathcal{A}| = O(1).$$

Concatenating all S cells gives

$$s_t^{\text{vec}} \in \{0, C\}^D, \qquad D = aS = \text{poly}(n).$$

Observe

$$X_t = \sqrt{\alpha_t}\, s_t^{\text{vec}} + \sqrt{1 - \alpha_t}\, Z_t, \qquad Z_t \sim \mathcal{N}(0, I_D). \tag{11.12}$$

11.14.1 Blockwise recovery

Decode each block by argmax.

Lemma 5 (High-probability blockwise decoding)**.** *The probability that any of the S blocks is decoded incorrectly is at most*

$$D \exp\left(-\frac{\alpha_t C^2}{4(1-\alpha_t)}\right). \tag{11.13}$$

Consequently, decoding succeeds with probability at least $1-\delta$ if

$$C \geq 2\sqrt{\frac{1-\alpha_t}{\alpha_t} \log \frac{D}{\delta}}. \tag{11.14}$$

Proof. For one block, let k be its active coordinate and let $j \neq k$ be an inactive one.

Then

$$X_t(k) \sim \mathcal{N}\left(\sqrt{\alpha_t} C, 1-\alpha_t\right)$$

and

$$X_t(j) \sim \mathcal{N}\left(0, 1-\alpha_t\right).$$

Hence

$$X_t(j) - X_t(k)$$

is Gaussian with mean

$$-\sqrt{\alpha_t} C$$

and variance

$$2(1-\alpha_t).$$

A Gaussian tail bound followed by a union bound over all within-block competitors gives Equation (11.13). □

There are two equivalent ways to express the threshold.

Define the amplitude SNR

$$\mathrm{SNR}_{\mathrm{amp}} = \frac{C\sqrt{\alpha_t}}{\sqrt{1-\alpha_t}}.$$

Then

$$\boxed{\mathrm{SNR}_{\mathrm{amp}} \gtrsim \sqrt{\log(D/\delta)}.} \tag{11.15}$$

Alternatively define the power SNR

$$\mathrm{SNR}_{\mathrm{pow}} = \frac{\alpha_t C^2}{1-\alpha_t}.$$

Then

$$\boxed{\mathrm{SNR}_{\mathrm{pow}} \gtrsim \log(D/\delta).} \tag{11.16}$$

This removes the ambiguity between the two conventions.

11.14.2 Robust execution

Let

$$\mathrm{Dec}(X_t)$$

denote blockwise decoding.
Define

$$D_t(X_t) = H_M\left(\mathrm{Dec}(X_t)\right).$$

Whenever decoding succeeds,

$$D_t(X_t) = s_{t+1}.$$

Theorem 11 (Robustness to Gaussian corruption)**.** *Suppose every computational state required during a $T(n)$-step execution is observed through Gaussian corruption of the form Equation* (11.12).

If the decoding parameters are chosen so that the probability of decoding failure at any individual step is at most

$$\frac{\delta}{T(n)},$$

then all states are decoded correctly and the computation reaches the correct final output with probability at least

$$1 - \delta.$$

Proof. Apply Lemma 5 at every step and use a union bound over at most $T(n)$ states.
Conditioned on correct decoding, the local computational transition is exact. □

This is a theorem about Gaussian-corrupted *computational states*.
It is not a universality theorem for standard diffusion models.

11.15 What the Common Principle Really Is

The commonality is not

$$\boxed{\text{Transformers} = \text{diffusion models.}}$$

It is not

$$\boxed{\text{next-token prediction} = \text{Turing trace supervision.}}$$

The more general principle is

represent a difficult computation or transformation through intermediate states
$\Downarrow$
learn local or intermediate prediction problems
$\Downarrow$
compose or iterate those predictions.

The nature of the intermediate states can be very different.
For a Turing computation they are machine configurations.
For autoregressive models they may be emitted symbols or explicit reasoning states.
For diffusion models they are controlled noisy versions of data.
The mathematical burden therefore remains model dependent.

11.16 Relation to Compositional Sparsity

A Turing space–time computation is itself compositionally sparse:

$$s_{t+1,i} = h_M\left(s_{t,i-1}, s_{t,i}, s_{t,i+1}\right).$$

Unrolling time yields a polynomial-size DAG of constant local arity.
Hence

polynomial-time computation $\Longrightarrow$ polynomial-size bounded-fan-in space–time DAG.

Chapter 4 uses this observation primarily as a representation result.
The present chapter adds an information result:

when the relevant local states are exposed, the constituents can be supervised directly.

C1 provides the local structure.
C3 exposes it.

11.17 Relation to Associative Computation

The state of the computation may be regarded as a collection of local records.
Updating one component requires

retrieve local state + apply local transition + store the result.

Repeated application gives the complete computation.
This is closely related to the read–update–write cycle of the Associative Turing Machine.
The distinction is

ATM chapter:	associative memory provides a universal execution mechanism,
present chapter:	trace exposure makes a local execution rule directly learnable.

Computational universality answers

Can the program be executed?

Trace supervision answers, in a restricted regime,

Can the local transition be learned directly?

11.18 A Biological Interpretation: Replay as Partial Trace

A possible biological interpretation should remain deliberately modest.

Long behavioral sequences create temporal credit-assignment problems.

If intermediate states

$$z_0, z_1, \ldots, z_T$$

are available during experience or replay, then a learner may estimate local predictions

$$z_t \longmapsto z_{t+1}$$

rather than rely entirely on a terminal signal for the global relation

$$z_0 \longmapsto z_T.$$

Thus

intermediate-state exposure can shorten temporal credit assignment.

Hippocampal replay is suggestive in this respect.

But biological replay is not assumed to provide the theorem-strength complete local table.

It may provide only partial relational structure.

Nor does the theorem imply that Hebbian plasticity alone can learn every local map.

The weaker hypothesis is simply that replay can expose useful intermediate predictive relations to a slower learner.

The hippocampal-scaffold chapter develops this possibility separately.

11.19 The State-Discovery Problem

The theorem assumes that the state representation is already known.

This is perhaps its most important limitation.

Suppose instead that the learner observes raw data

$$x_t$$

generated by a hidden process.

The difficult problem is to discover a representation

$$z_t = \phi(x_{\leq t}) \tag{11.17}$$

in which the dynamics become simple:

$$z_{t+1} = H(z_t) \tag{11.18}$$

or, more strongly,

$$z_{t+1}^{(i)} = H_i\left(z_t^{\mathrm{pa}(i)}\right), \qquad |\,\mathrm{pa}(i)| = O(1). \tag{11.19}$$

This gives the central open problem:

State-Discovery Problem

When can raw observations be transformed into coordinates in which the relevant computation has stable bounded-arity local dynamics?

This is not solved by C3.
C3 assumes that useful intermediate states are supplied.
The State-Discovery Problem asks how a learner can manufacture those states for itself.
It therefore lies at the intersection of

genericity + feature learning + world-model discovery.

The staircase conjecture of Chapter H can be viewed as one possible answer: progressively recovered constituents create the coordinates in which subsequent computation becomes local.

11.20 Limitations and Open Problems

The results of this chapter have several clear boundaries.

11.20.1 Trace exposure is stronger than end-to-end supervision

The exact theorem assumes that local computational states are visible.
Ordinary supervised examples do not provide them.

11.20.2 Observed traces may not cover future local states

Even a complete trajectory reveals only the neighborhoods that occur on that trajectory.
Generalization by lookup requires

$$\mathcal{U}_{\mathrm{test}} \subseteq \mathcal{U}_{\mathrm{train}}.$$

Without coverage, additional assumptions about the local rule are needed.

11.20.3 Surface traces are not necessarily faithful traces

Language, scratchpads, and emitted reasoning symbols may help learning without corresponding to the actual latent variables of the generating computation.

11.20.4 Standard diffusion is not computational-trace diffusion

Diffusion noise levels and Turing configurations are different objects.
The analogy concerns intermediate local learning problems, not identity of mechanism.

11.20.5 Approximate execution can accumulate error

The exact finite-state construction resets every step to a valid discrete configuration.

For learned continuous transitions, errors can accumulate over

$$T(n)$$

steps.

Long-horizon execution may therefore require

contractivity, error correction, attractor structure, state restoration,

or some other stability mechanism.

11.20.6 State discovery remains open

The hardest problem is often not learning

$$H$$

after the right state has been exposed.

It is finding the state variables in which such a simple H exists.

11.21 Conclusion

A polynomial-time Turing computation can be represented as repeated application of a constant-arity local rule.

With complete local supervision,

polynomial-time computation
$\Downarrow$
bounded-arity local transition
$\Downarrow$
complete local supervision
$\Downarrow$
exact local-rule recovery
$\Downarrow$
iteration.

With ordinary observed trajectories, one further condition appears:

the local states needed at test time must either have been observed or be predictable from additional structure.

This distinguishes

trace exposure

from

trace coverage.

A universal Turing machine sharpens the conceptual result still further.
One fixed local transition rule

$$h_U$$

can execute arbitrarily many programs when the program and data are encoded in the initial state.
Thus

fixed interpreter + variable external program = universal computation.

The relation to autoregressive and diffusion models is structural.
Autoregressive models predict along observed sequences.
Diffusion training constructs supervised intermediate states at controlled noise levels.
Trace-exposed computation predicts along an actual computational trajectory.
The common principle is

intermediate supervision can convert depth of hidden inference into locality of prediction.

But the deepest question remains unanswered:

when the useful intermediate states are not supplied, can the learner discover a representation in which the com

That is the State-Discovery Problem.
In the language of this book, C3 tells us what happens when the staircase is given.
C2 asks whether learning can build that staircase for itself.

CHAPTER 12

Return-Conditioned Sequence Modeling as Autoregressive Control

Reinforcement learning is usually formulated as a problem of long-horizon credit assignment. An agent chooses actions, rewards may arrive much later, and the learner must determine which earlier decisions were responsible for future success.

Return-conditioned sequence modeling reorganizes this problem.

Instead of estimating a value function through Bellman recursion, an offline trajectory is annotated at every time step with its return-to-go*: a scalar summary of the future reward that followed that point in the trajectory. The learning problem then becomes supervised conditional action prediction,*

$$(h_t, G_t) \longmapsto a_t,$$

where h_t is the observed history and G_t is the return-to-go.

This does not eliminate all of reinforcement learning. In particular, it does not solve exploration, counterfactual causal credit assignment, or control outside the support of the offline data. But it does transform an important part of the offline control problem:

$$\boxed{\textit{delayed future outcome} \longrightarrow \textit{return label available at every step} \longrightarrow \textit{supervised conditional action prediction}.}$$

In the terminology of the preceding chapters, return-to-go can be viewed as a compressed partial trace *of future success. It does not expose the true intermediate computational state, as in the hard C3 regime of Chapter H; instead it supplies each local decision with a summary of the outcome of the remaining trajectory.*

At inference time, the learned policy is rolled out autoregressively in closed loop with the environment. If a world model is learned as well, the complete system becomes a learned discrete-time dynamical model and the analogy with numerical integration becomes literal.

Thus the central idea of this chapter is

$$\boxed{\textit{control} \approx \textit{return-conditioned trajectory generation},}$$

with important qualifications about support, exploration, and causality.

12.1 From Value Estimation to Conditional Trajectory Modeling

Consider a finite-horizon Markov decision process

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma, H),$$

where

$$P(s_{t+1} \mid s_t, a_t)$$

is the transition law,

$$r_t = r(s_t, a_t, s_{t+1})$$

is the reward,

$$\gamma \in (0, 1]$$

is the discount factor, and H is the horizon.

12.1.1 Classical value-based reinforcement learning

A central object in classical reinforcement learning is the value function

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{u=t}^{H-1} \gamma^{u-t} r_u \,\middle|\, s_t = s \right],$$

or the action-value function

$$Q^\pi(s, a).$$

Dynamic programming exploits the Bellman recursion

$$Q^\pi(s_t, a_t) = \mathbb{E}\left[r_t + \gamma V^\pi(s_{t+1}) \right].$$

Temporal-difference methods estimate one quantity using another learned estimate. This bootstrapping is powerful but can become unstable when combined with

- function approximation,
- off-policy data,
- and bootstrapped targets,

the classical "deadly triad."

12.1.2 An alternative offline formulation

Return-conditioned sequence models take a different route.

Instead of learning a value function and using it to choose an action, they model actions directly as part of a trajectory.

A trajectory is

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_H).$$

Define the return-to-go at time t by

$$G_t = \sum_{u=t}^{H-1} \gamma^{u-t} r_u. \tag{12.1}$$

The trajectory can then be represented in a return-conditioned form such as

$$\tau_G = (G_0, s_0, a_0, G_1, s_1, a_1, \ldots, G_{H-1}, s_{H-1}, a_{H-1}).$$

A Decision-Transformer-style model learns

$$\pi_\theta \left(a_t \mid G_t, s_{\leq t}, a_{<t}\right).$$

Thus the control problem is recast as conditional sequence modeling.

12.2 Offline Training Is Supervised Conditional Action Prediction

Let

$$\mathcal{D} = \{\tau^{(1)}, \ldots, \tau^{(N)}\}$$

be an offline dataset of trajectories.

For every trajectory, compute the return-to-go sequence

$$G_0, G_1, \ldots, G_{H-1}.$$

The model is then trained with an ordinary supervised prediction objective.

For discrete actions,

$$\mathcal{L}(\theta) = -\mathbb{E}_{\tau \sim \mathcal{D}} \sum_{t=0}^{H-1} \log \pi_\theta \left(a_t \mid h_t, G_t\right),$$

where

$$h_t = (s_{\leq t}, a_{<t})$$

denotes the available history.

For continuous actions, one may instead minimize a regression or likelihood objective.

12.2.1 What has disappeared

There is no Bellman target of the form

$$r_t + \gamma V_\theta(s_{t+1}).$$

There is therefore no bootstrapping of one learned value estimate from another inside the basic supervised objective.

This is an important simplification:

return-conditioned sequence modeling avoids Bellman bootstrapping. return-conditioned sequence modeling a

12.2.2 What has not disappeared

The trajectory still contains long-range structure.

The model must learn that different histories and return conditions imply different actions.

Thus sequence modeling does not remove temporal dependence.

It changes how that dependence is represented and supervised.

12.3 Return-to-Go as a Partial Trace

The return-to-go is more than an ordinary conditioning variable.

It moves information about the future outcome backward along the entire trajectory.

12.3.1 A delayed reward becomes a local label

Suppose a successful outcome occurs only near the end of the episode.

Without preprocessing, an early decision at time t may be separated from that outcome by many steps.

Return-to-go assigns to the same time step the quantity

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots .$$

Thus every action example becomes

$$(h_t, G_t) \mapsto a_t.$$

The future outcome has been summarized and made locally available to the predictor.

return-to-go converts a delayed global outcome into a local conditioning variable.

12.3.2 Connection to C3

Chapter H distinguished:

$$\text{hard trace}, \qquad \text{partial trace}, \qquad \text{surface trace}.$$

A full computational trace would expose the true intermediate variables required by the underlying computation.

Return-to-go does much less.

It does not tell the learner

$$\text{which latent variable caused the future reward,}$$

or

$$\text{which alternative action would have produced a different return.}$$

It supplies only a compressed scalar summary of future success.

Thus it is natural to interpret G_t as a form of partial or surface trace:

$$\boxed{G_t = \text{compressed information about the future of the trajectory.}}$$

12.3.3 A hierarchy of supervision

The preceding chapter used full computational states

$$s_0, s_1, \ldots, s_T$$

as theorem-strength trace exposure.

Return-conditioned control sits at a weaker point in the hierarchy:

$$\boxed{\begin{array}{c}\text{full computational trace}\\ \Downarrow\\ \text{rich intermediate supervision}\\ \Downarrow\\ \text{return-to-go conditioning}\\ \Downarrow\\ \text{sparse terminal reward.}\end{array}}$$

As one moves downward, more of the hidden causal structure must be inferred rather than supplied.

12.4 Inference as Closed-Loop Autoregressive Rollout

Training is supervised.

Control occurs at inference time through autoregressive interaction with the environment.

12.4.1 Desired return

At time 0, supply a desired return

$$G_0^\star.$$

The model generates

$$a_0 \sim \pi_\theta \left(\cdot \mid h_0, G_0^\star \right).$$

The environment then generates a next state

$$s_1 \sim P(\cdot \mid s_0, a_0)$$

and reward

$$r_0.$$

12.4.2 Updating the desired return

By definition,

$$G_t = r_t + \gamma G_{t+1}.$$

Therefore, for $\gamma > 0$,

$$G_{t+1} = \frac{G_t - r_t}{\gamma}. \tag{12.2}$$

At deployment one applies the same algebra to the desired return:

$$G_{t+1}^\star = \frac{G_t^\star - r_t}{\gamma}.$$

For the common undiscounted episodic case

$$\gamma = 1,$$

this reduces to

$$G_{t+1}^\star = G_t^\star - r_t.$$

12.4.3 The complete recurrence

The closed-loop controller therefore satisfies

$$\boxed{\begin{aligned} a_t &\sim \pi_\theta(\cdot \mid h_t, G_t^\star), \\ s_{t+1} &\sim P(\cdot \mid s_t, a_t), \\ r_t &\sim R(\cdot \mid s_t, a_t, s_{t+1}), \\ G_{t+1}^\star &= \frac{G_t^\star - r_t}{\gamma}. \end{aligned}}$$

The newly observed state and action are appended to the history,

$$h_{t+1} = (h_t, a_t, s_{t+1}),$$

and the process repeats.
Thus control is generated one decision at a time.

12.5 Control as Conditional Trajectory Generation

The most useful probabilistic interpretation is not numerical integration but conditional trajectory generation.

The offline data define an empirical distribution over trajectories

$$p_{\mathcal{D}}(\tau).$$

Return conditioning asks the model to represent aspects of

$$p_{\mathcal{D}}\left(\tau \mid s_0, G_0\right).$$

The policy factorizes this distribution autoregressively through terms such as

$$p_\theta\left(a_t \mid h_t, G_t\right).$$

At inference, specifying

$$G_0^\star$$

requests behavior associated with that level of return.
Thus

$$\boxed{\text{control} \approx \text{conditional generation of a trajectory with desired outcome.}}$$

12.5.1 A trajectory-level control variable

The desired return acts somewhat like a boundary condition.

It specifies not the next action directly but a property desired of the whole future trajectory.
The model then generates local actions conditioned on that global requirement.
A careful analogy is therefore:

$$\boxed{G_0^\star = \text{trajectory-level control variable analogous to a desired boundary condition.}}$$

It is not literally a boundary condition in the mathematical sense of a differential equation, but it plays a related conditioning role.

12.6 What Is and Is Not Numerical Integration

The physics analogy remains useful if stated carefully.

12.6.1 Policy-only sequence models

In an ordinary return-conditioned policy model, the learned network does not usually predict

$$s_{t+1}.$$

The environment supplies the transition

$$s_{t+1} \sim P(\cdot \mid s_t, a_t).$$

Therefore the model itself is not integrating the environmental vector field.
Rather, the combined agent–environment loop produces the trajectory:

policy chooses action + environment advances state.

12.6.2 The dynamical-system analogy

The complete closed-loop system is nevertheless a recurrence.
For deterministic dynamics,

$$s_{t+1} = F(s_t, a_t),$$

with

$$a_t = \pi_\theta(h_t, G_t^\star),$$

so

$$s_{t+1} = F\left(s_t, \pi_\theta(h_t, G_t^\star)\right).$$

This is a discrete-time controlled dynamical system.
The autoregressive model supplies the control law.
The environment supplies the state evolution.

12.7 When a World Model Is Added

The integration analogy becomes substantially more literal when the agent also learns the environment dynamics.

12.7.1 A learned world model

Suppose we train

$$\widehat{P}_\phi\left(s_{t+1}, r_t \mid h_t, a_t\right).$$

Then a fully learned rollout can alternate

$$a_t \sim \pi_\theta(\cdot \mid h_t, G_t^\star)$$

and

$$(\widehat{s}_{t+1}, \widehat{r}_t) \sim \widehat{P}_\phi(\cdot \mid h_t, a_t).$$

The desired return is updated according to

$$G^{\star}_{t+1} = \frac{G^{\star}_t - \widehat{r}_t}{\gamma}.$$

12.7.2 Difference-equation form

For approximately deterministic continuous states, the world model may be written as

$$s_{t+1} = s_t + f_\phi(s_t, a_t).$$

Equivalently,

$$s_{t+1} - s_t = f_\phi(s_t, a_t).$$

Now iterating the learned transition is genuinely analogous to numerical integration of a learned discrete vector field.

Thus the original physics analogy belongs most naturally to the world-model case:

policy-only model:	environment advances the trajectory,
policy + world model:	the learned system can advance the trajectory internally.

12.8 Credit Assignment: What Has Been Removed?

Return-conditioned sequence modeling changes the credit-assignment problem, but does not eliminate it completely.

12.8.1 Bellman credit assignment is removed from the training objective

The model does not recursively estimate

$$V(s_t)$$

from

$$V(s_{t+1}).$$

Thus Bellman bootstrapping is absent.

There is no TD target of the form

$$r_t + \gamma \widehat{V}(s_{t+1}).$$

This avoids one important source of instability.

12.8.2 Temporal outcome information is supplied directly

Every action example receives its own return-to-go label.

Thus information about distant reward no longer has to propagate backward solely through a chain of learned value estimates.

This motivates the statement

return conditioning shortens the informational distance between an action and its future outcome.

12.8.3 But causal credit assignment remains

Suppose a high-return trajectory contains action a_t.

The dataset establishes that

$$a_t$$

occurred on a trajectory with return

$$G_t.$$

It does not establish the counterfactual:

$$\text{what would the return have been under } a'_t \neq a_t?$$

Therefore high correlation between an action and future return is not the same as causal attribution.

outcome-conditioned imitation $\neq$ counterfactual causal credit assignment.

The model may learn spurious associations if the offline data contain confounding between actions and outcomes.

12.9 The Role of Attention

Self-attention is useful in return-conditioned sequence models because it provides short computational paths between temporally separated elements of the trajectory.

A later action representation can depend directly on information from

$$G_t, \qquad s_{t-j}, \qquad a_{t-j},$$

without requiring that information to propagate through every intermediate recurrent state.

Thus attention can make long-range conditioning easy to represent.

But this should not be confused with solving causal credit assignment.

The safe statement is

attention provides access to long-range temporal correlations;

not

Part III
Learning and Evolution

CHAPTER 13

Bits as a Currency of Inductive Bias and Generalization

Deep learning uses many apparently different forms of inductive bias: local connectivity, weight sharing, compositional sparsity, low rank, norm control, quantization, architectural bottlenecks, and implicit selection by optimization.

This chapter develops a common language for comparing many of these mechanisms: the number of bits required to specify a predictor to the resolution relevant for prediction.

The central quantity is not simply the number of parameters. A small number of arbitrary real parameters can contain arbitrarily many bits, while a very large network may implement a function that is highly compressible.

Two related but distinct notions of description complexity will be used.

For an individual predictor, we ask for the length of a code specifying that predictor, possibly only approximately.

For a class of predictors, we ask how many bits are required in the worst case to identify one element of an ε-cover. This is metric entropy:

$$\mathcal{H}(\varepsilon, \mathcal{F}, d) = \log_2 \mathcal{N}(\varepsilon, \mathcal{F}, d).$$

These two notions lead to two parallel statistical routes:

individual description length	$\longrightarrow$	*Occam/compression generalization*
$B_\varepsilon(f)$		$O\left(\sqrt{B_\varepsilon(f)/n}\right)$
class covering bits	$\longrightarrow$	*Rademacher/chaining bounds*
$\mathcal{H}(\varepsilon, \mathcal{F})$		$\mathfrak{R}_n(\mathcal{F})$.

Architecture and optimization enter at different stages.

Architecture determines which descriptions are cheap or even available. Optimization, regularization, initialization, and the data determine which of those descriptions is selected.

Thus a large architecture need not define a small hypothesis class in order for a particular learned solution to generalize.

The relevant question is instead:

how many bits are required to describe the selected predictor at the resolution relevant to prediction?

This chapter makes that statement precise in several elementary settings and connects it to compositional sparsity, weight sharing, sparsity, low rank, norm control, and quantization.

13.1 Why Bits?

A neural network can contain billions of parameters and nevertheless implement a simple function.

Conversely, one exact real parameter can encode an arbitrarily long binary string in its digits.

Therefore

number of parameters $\neq$ number of bits of effective complexity.

The difference is fundamental.

13.1.1 Finite precision makes the question operational

Suppose a network has P parameters and every parameter is stored with b bits.

Then a literal parameter-vector description requires at most

$$bP$$

bits.

This is an upper bound on the description length of the parameterization, not necessarily on the shortest description of the represented function.

Several parameter strings may compute the same function.

The function may be sparse.

Matrices may be low rank.

Parameters may be shared.

Repeated subprograms may be represented once and reused.

Or the entire network may be generated by a much shorter algorithm.

Thus the relevant question is

> How many bits are actually required to specify the learned predictor to the precision relevant for the loss?

13.2 Individual Description Length

We begin with one predictor rather than a whole class.

13.2.1 Kolmogorov complexity

Fix a universal Turing machine U.

For a computable object f, define

$$K_U(f) = \min_{p:\, U(p)=f} |p|.$$

This is the length of the shortest program producing f [55].

Kolmogorov complexity captures the strongest possible notion of algorithmic compressibility.

It is nevertheless not directly usable as a general learning-theoretic quantity.

First, it is uncomputable.

Second, exact reproduction is often stronger than prediction requires.

Third, learning-theoretic statements require specifying what decoder or coding language is available independently of the observed sample.

13.2.2 Approximate description length

Let $d(\cdot, \cdot)$ be a metric on predictors.

For a fixed coding language or decoder $\mathcal{D}$, define

$$B_{\varepsilon,\mathcal{D}}(f) = \min \{|c| : d(f, \mathcal{D}(c)) \leq \varepsilon\}. \tag{13.1}$$

When the decoder is understood, we write simply

$$B_\varepsilon(f).$$

This is the number of bits required to describe a predictor that is ε-close to f.

The distinction from exact Kolmogorov complexity is

$K(f)$:	exact algorithmic description,
$B_\varepsilon(f)$:	description sufficient at resolution ε.

For statistical prediction, B_ε is often the more relevant object.

13.2.3 Why few parameters do not imply few bits

Suppose

$$f_\theta$$

has one parameter

$$\theta \in \mathbb{R}.$$

If arbitrary precision is permitted, the binary expansion of θ can contain arbitrarily many bits.

Hence

$$\boxed{P \text{ parameters} \Longrightarrow O(P) \text{ bits.}}$$

Parameter counting becomes description counting only after precision, norms, sparsity, rank, or another effective restriction has been imposed.

This is the same finite-precision distinction used earlier in the book in the definition of efficient real computation.

13.3 A Generalization Theorem for Individual Codes

The cleanest connection between bits and generalization does not require metric entropy.

It can be stated directly for individual code lengths.

13.3.1 Prefix codes

Let $\mathcal{C}$ be a countable collection of predictors.

Assign each predictor $f \in \mathcal{C}$ a prefix-free binary code of length

$$L(f) \in \mathbb{N}.$$

Prefix-freeness implies Kraft's inequality,

$$\sum_{f \in \mathcal{C}} 2^{-L(f)} \leq 1. \tag{13.2}$$

Theorem 12 (Prefix-code Occam bound)**.** *Assume the loss satisfies*

$$0 \leq \ell(f(x), y) \leq 1.$$

Let a sample

$$S = \{(x_i, y_i)\}_{i=1}^{n}$$

be drawn i.i.d. from the data distribution.
Then, with probability at least $1 - \delta$*, simultaneously for every* $f \in \mathcal{C}$*,*

$$\boxed{\left|R(f) - \widehat{R}_S(f)\right| \leq \sqrt{\frac{L(f)\ln 2 + \ln(2/\delta)}{2n}}.} \tag{13.3}$$

Proof. For a fixed f, Hoeffding's inequality gives

$$\Pr\left[|R(f) - \widehat{R}_S(f)| > \eta\right] \leq 2e^{-2n\eta^2}.$$

Set

$$\eta_f = \sqrt{\frac{L(f)\ln 2 + \ln(2/\delta)}{2n}}.$$

Then

$$\Pr\left[|R(f) - \widehat{R}_S(f)| > \eta_f\right] \leq \delta\, 2^{-L(f)}.$$

A union bound over all coded predictors gives

$$\Pr\left[\exists f \in \mathcal{C} : |R(f) - \widehat{R}_S(f)| > \eta_f\right] \leq \delta \sum_{f \in \mathcal{C}} 2^{-L(f)} \leq \delta$$

by Kraft's inequality. □

The important feature of Theorem 12 is that the bound is simultaneous over the entire fixed coding language.

Therefore the predictor may be selected *after seeing the training sample*.

In particular,

data-dependent selection of a predictor is allowed when the code itself was fixed in advance.

What cannot be introduced for free after observing the data is a new coding language tailored to the sample.

If the decoder, dictionary, architecture, or codebook is itself learned from the sample and is needed to reconstruct the predictor, its description must also be paid for, or the analysis must use another valid data-dependent complexity argument.

13.4 The Finite-Class Bound as a Special Case

Suppose

$$|\mathcal{F}| \le 2^B.$$

Every member can be assigned a fixed code of length at most

$$B + 1$$

up to a negligible prefix overhead.

The preceding theorem therefore reduces to the familiar finite-class bound.

Corollary 2 (Finite-bit generalization)**.** *Let*

$$|\mathcal{F}| \le 2^B$$

and assume bounded loss in $[0, 1]$*.*
Then with probability at least $1 - \delta$*, simultaneously for all* $f \in \mathcal{F}$*,*

$$\left|R(f) - \widehat{R}_S(f)\right| \le \sqrt{\frac{B \ln 2 + \ln(2/\delta)}{2n}}, \tag{13.4}$$

up to an irrelevant constant-bit coding overhead.

Thus

$$\boxed{B \text{ bits} \quad \Rightarrow \quad \text{generalization gap} = O\left(\sqrt{\frac{B + \log(1/\delta)}{n}}\right).}$$

This is the simplest precise form of the statement

short descriptions generalize.

13.5 Approximate Compression of the Selected Predictor

Exact reproduction is unnecessary when the loss is insensitive to a small prediction perturbation.

Suppose f is a learned predictor and g is a coded predictor satisfying

$$d_\infty(f, g) = \sup_x |f(x) - g(x)| \le \varepsilon.$$

Assume the loss is L_ℓ-Lipschitz in its prediction argument.

Theorem 13 (Approximate coded compression)**.** *Under the event of Theorem 12, for every predictor f and every coded $g \in \mathcal{C}$ satisfying*

$$\sup_x |f(x) - g(x)| \le \varepsilon,$$

we have

$$\boxed{R(f) \le \widehat{R}_S(f) + 2L_\ell \varepsilon + \sqrt{\frac{L(g)\ln 2 + \ln(2/\delta)}{2n}}.} \tag{13.5}$$

Proof. Lipschitzness gives

$$R(f) \le R(g) + L_\ell \varepsilon$$

and

$$\widehat{R}_S(g) \le \widehat{R}_S(f) + L_\ell \varepsilon.$$

Apply Theorem 12 to g and combine the inequalities. □

If

$$B_\varepsilon(f) = \min_{g:\, d_\infty(f,g) \le \varepsilon} L(g),$$

then

$$\boxed{R(f) \le \widehat{R}_S(f) + 2L_\ell \varepsilon + \sqrt{\frac{B_\varepsilon(f)\ln 2 + \ln(2/\delta)}{2n}}.} \tag{13.6}$$

One may then optimize over ε.

This gives a direct statistical meaning to the approximate description length of the *selected solution*.

13.6 From Individual Bits to Class Bits: Metric Entropy

Individual code length and metric entropy are related but distinct.

Let

$$\mathcal{F}$$

be a class of predictors and let

$$\mathcal{N}(\varepsilon, \mathcal{F}, d)$$

be the smallest number of radius-ε balls required to cover $\mathcal{F}$ in metric d.

Define

$$\mathcal{H}(\varepsilon, \mathcal{F}, d) = \log_2 \mathcal{N}(\varepsilon, \mathcal{F}, d). \tag{13.7}$$

Hence

$$\boxed{\mathcal{H}(\varepsilon, \mathcal{F}, d) = \text{worst-case number of bits needed to index an } \varepsilon\text{-cover.}}$$

Metric entropy is therefore a class-level description complexity.
It differs from $B_\varepsilon(f)$ in an important way.
The latter asks how cheaply one particular predictor can be represented.
The former asks how many bits are sufficient in the worst case across the entire class.

13.6.1 Covering-number generalization

Take a fixed minimal ε-cover

$$\mathcal{C}_\varepsilon$$

of $\mathcal{F}$.
Its cardinality is

$$|\mathcal{C}_\varepsilon| = \mathcal{N}(\varepsilon, \mathcal{F}, d).$$

Applying the finite-class theorem to this codebook gives schematically,

$$\boxed{R(f) \lesssim \widehat{R}(f) + \text{approximation error} + \sqrt{\frac{\mathcal{H}(\varepsilon, \mathcal{F}, d)}{n}}.}$$

The appropriate approximation term depends on the metric and loss.
One may optimize over ε.

13.7 Rademacher Complexity: The Statistical Language

Covering numbers are especially useful because they connect naturally to Rademacher complexity.
For a fixed sample

$$S = (x_1, \ldots, x_n),$$

define

$$\widehat{\mathfrak{R}}_S(\mathcal{F}) = \mathbb{E}_\sigma \left[\sup_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^{n} \sigma_i f(x_i) \right], \tag{13.8}$$

where

$$\sigma_i \in \{-1, +1\}$$

are independent random signs.
The quantity measures how well the class can correlate with random fluctuations on the observed sample.
Under standard boundedness and Lipschitz assumptions,

$$R(f) \lesssim \widehat{R}(f) + O(\mathfrak{R}_n(\mathcal{F})) + O\left(\sqrt{\frac{\log(1/\delta)}{n}} \right).$$

Thus the class-level question becomes

How do covering bits control Rademacher complexity?

13.8 Dudley: From Covering Bits to Rademacher Complexity

Define the empirical L_2 metric

$$d_S(f,g) = \left[\frac{1}{n}\sum_{i=1}^{n}\left(f(x_i)-g(x_i)\right)^2\right]^{1/2}. \tag{13.9}$$

A form of Dudley's entropy integral gives

$$\widehat{\mathfrak{R}}_S(\mathcal{F}) \le \inf_{\alpha>0}\left[C_1\alpha + \frac{C_2}{\sqrt{n}}\int_{\alpha}^{\operatorname{diam}(\mathcal{F},d_S)}\sqrt{\log \mathcal{N}(u,\mathcal{F},d_S)}\,du\right] \tag{13.10}$$

for universal constants C_1, C_2 [29].

13.8.1 The coding interpretation

Imagine specifying a predictor progressively.
First transmit a coarse approximation.
Then transmit the correction required to reach a finer scale.
Then another correction.
The resulting description is multiscale.
Thus

$$\boxed{\text{chaining} = \text{multiscale description of a function class.}}$$

The entropy integral combines the coding requirements across resolutions.

13.8.2 An important qualification

Metric entropy alone does not guarantee generalization.
One also needs the statistical assumptions appropriate to the bound, including, for example,

- controlled sampling,
- a suitable loss,
- boundedness or tail assumptions,
- finite entropy at the relevant scales.

The correct statement is

$$\boxed{\text{smaller covering entropy} \Rightarrow \text{smaller corresponding capacity bounds, under the required assumptions.}}$$

13.9 Architecture as a Coding Language

Architecture affects statistical complexity because it changes which functions can be described economically.

A useful decomposition is

architecture defines a coding language
⇓
optimization and regularization select a solution
⇓
the selected solution has some effective code length
⇓
that code length can control generalization.

This is different from claiming that the architecture itself has a small hypothesis class.

Modern networks may have enormous nominal capacity.

The important fact may instead be that some functions have short descriptions in the architectural language.

13.9.1 Examples

Convolution makes repeated translation-equivariant operations cheap to describe.

Sparse architectures make restricted dependency patterns cheap.

Low-rank parameterizations make low-dimensional maps cheap.

Attention makes content-addressable routing cheap.

Residual architectures make incremental transformations cheap.

A CRM-like library makes reusable subprograms cheap.

Thus architecture determines the relative coding cost of different functions.

13.10 Compositional Sparsity as a Bit Constraint

We now connect the description-length viewpoint to C1.

13.10.1 Fixed computation DAG

Let a fixed DAG G contain

$$s$$

constituent nodes.

Assume each node depends on at most

$$k$$

arguments and belongs to a bounded smoothness class.

Suppose that for each $m \leq k$ the local constituent class satisfies the entropy estimate

$$\log \mathcal{N}(\delta, \mathcal{G}_m, \|\cdot\|_\infty) \leq C \left(\frac{B}{\delta}\right)^{m/r}. \tag{13.11}$$

For example, this is the schematic dependence expected for bounded C^r classes on fixed-dimensional domains.

If each constituent is approximated independently to local error δ, the product cover satisfies

$$\log \mathcal{N}(\delta, \mathcal{F}_G^{\text{local}}) \lesssim Cs \left(\frac{B}{\delta}\right)^{k/r}. \tag{13.12}$$

13.10.2 Stability of the composition

Local approximation errors need not equal the final output error.

Let

$$\mathcal{S}_G$$

be a stability factor such that replacing every constituent by a δ-accurate approximation changes the final output by at most

$$\|f - \widetilde{f}\|_\infty \leq \mathcal{S}_G \delta. \tag{13.13}$$

To obtain final error ε, choose

$$\delta = \frac{\varepsilon}{\mathcal{S}_G}.$$

Hence

Proposition 11 (Entropy of a fixed stable compositional graph)**.** *Under the assumptions above,*

$$\boxed{\mathcal{H}(\varepsilon, \mathcal{F}_G, \|\cdot\|_\infty) \lesssim Cs \left(\frac{B\mathcal{S}_G}{\varepsilon}\right)^{k/r}.} \tag{13.14}$$

The approximation exponent is controlled by

$$\frac{k}{r},$$

rather than

$$\frac{d}{r}.$$

The graph can nevertheless remain difficult if $\mathcal{S}_G$ grows rapidly with depth.

13.11 Unknown Graph Structure Costs Additional Bits

C1 states that a compact graph exists.

It does not say that the graph is supplied to the learner.

To count an entire class of sparse compositional graphs, the graph structure itself must be encoded.

13.11.1 A uniformly stable compositional class

Define

$$\mathcal{F}_{\mathrm{CS}}(d, s, k, r, B, S_0) \tag{13.15}$$

to be the class of functions admitting a DAG representation satisfying

$$\text{number of nodes} \leq s,$$

$$\text{local arity} \leq k,$$

$$\|g_v\|_{C^r} \leq B,$$

and

$$\mathcal{S}_G \leq S_0.$$

Assume a topological ordering of the s internal nodes.
Each node chooses at most k parents among at most

$$d + s$$

available preceding variables.
The number of possible parent sets is bounded by

$$\sum_{j=0}^{k} \binom{d+s}{j}.$$

Therefore the number of bits required to specify the parent sets of all s nodes is at most

$$B_{\mathrm{graph}} \leq s \log_2 \left[\sum_{j=0}^{k} \binom{d+s}{j} \right]. \tag{13.16}$$

For fixed k,

$$B_{\mathrm{graph}} = O(sk \log(d+s)). \tag{13.17}$$

Proposition 12 (Entropy of unknown stable compositional graphs)**.** *Under the local entropy assumptions above,*

$$\boxed{\mathcal{H}(\varepsilon, \mathcal{F}_{\mathrm{CS}}(d, s, k, r, B, S_0)) \lesssim Cs \left(\frac{BS_0}{\varepsilon} \right)^{k/r} + O(sk \log(d+s)).} \tag{13.18}$$

The two terms have different meanings.
The first pays for the local functions.
The second pays for their wiring.
Thus

ambient dimension disappears from the local approximation exponent but survives through structural addressi

13.11.2 Representation versus support discovery

If the correct graph is hard-coded into the architecture, the structural bits have already been supplied.

If the learner must infer the parent sets, those structural bits must be recovered from data.

Therefore

$$\boxed{\text{existence of a short sparse description} \neq \text{ability to discover that description.}}$$

This is exactly the distinction between C1 and C2.

13.12 Weight Sharing and Reuse

Weight sharing is one of the clearest description-length reductions.

Suppose one local map containing q parameters is used at M locations.

Without sharing, a naive description contains approximately

$$Mq$$

parameter values.

With sharing, only

$$q$$

values need to be stored, together with the rule specifying where they are reused.

At b bits per parameter, this changes the leading value cost from

$$bMq$$

to

$$bq.$$

Hence

$$\boxed{\text{repeated constituent} \Rightarrow \text{describe once, reuse many times.}}$$

This is the bit-theoretic reason that reuse is stronger than sparse compositionality alone.

A DAG with s unrelated constituents does not obtain this saving.

13.13 Sparsity

Suppose a parameter vector belongs to

$$\mathbb{R}^P$$

but contains only

$$s$$

nonzero coordinates.

A description must encode both the active support and the active values.

The support requires

$$\log_2 \binom{P}{s}$$

bits.
Using

$$\log_2 \binom{P}{s} \lesssim s \log_2 \frac{eP}{s},$$

and quantizing each active value to b bits gives

$$\boxed{B \lesssim s \log_2 \frac{eP}{s} + bs.} \tag{13.19}$$

Thus the description cost can scale with active dimension rather than nominal dimension.

13.14 Low Rank

Low rank provides a matrix analogue of sparsity.
Let

$$W \in \mathbb{R}^{d_1 \times d_2}$$

satisfy

$$\operatorname{rank}(W) \le r, \qquad \|W\|_F \le R.$$

The manifold of rank-r matrices has intrinsic dimension of order

$$r(d_1 + d_2 - r).$$

Corresponding covering estimates have the schematic form

$$\boxed{\log \mathcal{N}(\varepsilon, \mathcal{W}_r, \|\cdot\|_F) \lesssim r(d_1 + d_2) \log \frac{CR}{\varepsilon}.} \tag{13.20}$$

The unrestricted matrix dimension is

$$d_1 d_2.$$

Thus rank restriction can produce a major reduction in covering bits.

13.14.1 Attention

The attention chapter established only the conditional statement

$$\boxed{\text{low-dimensional relevance} \Rightarrow \text{low-rank content routing can be sufficient.}}$$

It did not establish that trained attention maps must be low rank.
If learning does produce a rank-r query or key map, however, that map can be described at a cost scaling roughly with

$$r(d_1 + d_2)$$

rather than

$$d_1 d_2.$$

Low-rank routing therefore fits naturally into the same bit currency.

13.15 Quantization and Finite Precision

Quantization gives the simplest literal conversion from parameters to bits.

Suppose a model has

$$P$$

parameters and every parameter can take at most

$$2^b$$

values.

Then there are at most

$$2^{bP}$$

parameter strings.

Therefore

$$|\mathcal{F}| \leq 2^{bP}$$

and

$$\boxed{\log_2 |\mathcal{F}| \leq bP.}$$

The bound can be strict because multiple parameter strings may represent the same predictor.

13.15.1 Immediate finite-precision bound

For bounded loss,

$$\boxed{|R(f) - \widehat{R}(f)| \lesssim \sqrt{\frac{bP + \log(1/\delta)}{n}}.}$$

This may be numerically vacuous for a large modern model, but the logical implication is exact:

$$\boxed{\text{finite precision} \Rightarrow \text{finite number of parameter descriptions} \Rightarrow \text{finite capacity bound.}}$$

13.15.2 Training-time quantization versus post-training compression

Two cases should be distinguished.

Quantization during training. The restriction changes the space over which optimization occurs and may act as an explicit regularizer.

Post-training quantization. Training occurred in the full-precision parameterization.

Therefore post-training quantization did not cause the original optimization trajectory.

Nevertheless, if the final predictor can be replaced by a short quantized description while preserving its predictions, that short description can serve as a generalization certificate under a fixed coding scheme.

Hence

compression certificate $\neq$ training-time regularization.

13.16 Norm Control as a Covering Constraint

Norms provide another route to reduced covering complexity.

13.16.1 A Euclidean parameter example

Consider

$$w \in \mathbb{R}^P, \qquad \|w\|_2 \le R.$$

An η-cover of the Euclidean ball satisfies schematically

$$\mathcal{N}(\eta, B_2(R), \|\cdot\|_2) \le \left(\frac{CR}{\eta}\right)^P.$$

Thus

$$\log \mathcal{N}(\eta, B_2(R)) \lesssim P \log \frac{CR}{\eta}.$$

A smaller radius reduces parameter-space covering complexity.

13.16.2 Parameter-space norm is not the final object

For neural networks, however, parameter distance is not itself prediction distance.

Reparameterizations can change layer norms while preserving the represented function.

Useful generalization bounds therefore control the induced function class, often through architecture-dependent combinations of

spectral norms, Frobenius norms, path norms, margins,

or related quantities [5].

Thus the safe chain is

norm restriction $\to$ smaller induced function class $\to$ smaller covering/Rademacher complexity $\to$ generalizati

The first arrow depends on the architecture and on the particular norm.

13.17 A Unified Bit Table

The preceding mechanisms can be summarized schematically.

mechanism	**what must be described**	**typical leading cost**
quantization	P parameter values	bP
sparsity	s positions + values	$s\log(P/s) + bs$
rank r	low-rank factors	$r(d_1 + d_2)\log(R/\varepsilon)$
weight sharing	one repeated constituent	pay once rather than per use
fixed compositional DAG	s local k-ary maps	$s(B\mathcal{S}_G/\varepsilon)^{k/r}$
unknown DAG	local maps + parent addresses	$s(BS_0/\varepsilon)^{k/r} + sk\log(d+s)$
norm control	bounded function family	metric-dependent covering bits.

These expressions should not be added mechanically.
They arise under different metrics, model classes, and regularity assumptions.
The commonality is conceptual:

each mechanism can reduce the number of distinguishable predictors at the resolution relevant to the task.

13.18 Architecture Versus Optimization

The distinction between coding language and selected code clarifies the role of overparameterization.

13.18.1 Architecture defines the coding language

Architecture determines which functions have short descriptions.

For example,

convolution: translation-reused local maps are cheap,
sparse routing: restricted dependency patterns are cheap,
low-rank modules: low-dimensional transformations are cheap,
CRM libraries: repeated subprograms are cheap,
attention: content-addressable access is cheap.

This does not mean that all functions representable by the architecture have short descriptions.

13.18.2 Optimization selects a code

A heavily overparameterized architecture may contain many interpolating functions.

Initialization, SGD, explicit regularization, normalization, and the data influence which solution is selected.

The relevant statistical question can therefore concern the code length of that selected predictor.

Theorem 12 is important here.

Because it holds simultaneously over the entire fixed code family, optimization may inspect the sample and choose any member of that family without invalidating the bound.

Thus

large architecture + selection of a short fixed-language code can still yield a nontrivial Occam bound.

13.18.3 The architecture need not impose a hard ceiling

A convolutional network can memorize.

A sparse network can memorize.

A low-rank parameterization can memorize if its remaining capacity is large.

A very large network can generalize.

Therefore

architecture biases complexity but does not by itself determine the complexity of the learned function.

Effective complexity is jointly influenced by

architecture + data + optimization + regularization + precision.

13.19 Compression of the Selected Solution

Suppose a trained model contains P nominal parameters.

After training, imagine that its predictions can be preserved by

- pruning most weights,
- quantizing the remaining weights,
- replacing matrices by low-rank factors,
- tying repeated modules,
- compiling repeated subprograms,
- or otherwise replacing the model by a shorter coded predictor.

Let

$$f_{\hat{\theta}}$$

denote the trained predictor.

Its relevant compression complexity is

$$B_\varepsilon(f_{\hat{\theta}}).$$

If a fixed decoder reconstructs an ε-accurate predictor from that many bits, Theorem 13 gives a direct generalization certificate.

Thus the complexity of the selected predictor may be much smaller than

$$P \times \text{full precision}.$$

This provides one interpretation of compression-based generalization analyses such as [123].

13.19.1 What must be included in the code

A valid compression certificate must describe everything required to reconstruct the compressed predictor.

Depending on the scheme, this may include

support indices, quantized values, rank factors, module dictionary, routing structure, decoder metada

Information inferred from the training sample cannot be treated as free unless the statistical argument explicitly allows it.

13.20 Two Routes from Bits to Generalization

We can now separate the two main routes of the chapter.

13.20.1 Route I: individual code length

For a predictor selected by training,

$$f_{\hat{\theta}},$$

find a short coded approximation g.

Then

$$\boxed{B_\varepsilon(f_{\hat{\theta}}) \longrightarrow \text{Occam/compression bound.}}$$

This route can exploit the fact that one particular learned function is much simpler than the full architecture class.

13.20.2 Route II: class metric entropy

Instead bound a whole function family

$$\mathcal{F}.$$

Then

$$\boxed{\mathcal{H}(\varepsilon, \mathcal{F}) \longrightarrow \text{covering/chaining} \longrightarrow \mathfrak{R}_n(\mathcal{F}) \longrightarrow \text{generalization.}}$$

This route is a worst-case class-level analysis.

Neither route subsumes the other in every application.

They are two mathematical realizations of the same broad description-length idea.

13.21 Connection to the Principles of the Book

The framework developed in the preceding chapters separates several questions.

13.21.1 C1: representation

C1 asks

$$\boxed{\text{Does a compact compositional realization exist?}}$$

Efficient computation provides one reason that such a representation may exist.

13.21.2 C2 and C3: recovery

C2 asks whether the hidden representation leaves detectable footprints from which it can be recovered.

C3 supplies intermediate structure when those footprints are insufficient.

Thus C2 and C3 concern

$$\boxed{\text{Can a useful compact representation actually be found?}}$$

13.21.3 Selection

Overparameterization may provide many solutions.

Optimization and regularization determine which region is reached.

Hence selection asks

$$\boxed{\text{Which fitting solution is chosen?}}$$

13.21.4 Bits: statistical complexity

The present chapter asks

$$\boxed{\text{How many bits are required to describe the selected solution at the relevant prediction scale?}}$$

These questions should remain distinct:

$$\boxed{\begin{array}{c}\text{representation}\\ \Downarrow\\ \text{recovery}\\ \Downarrow\\ \text{selection}\\ \Downarrow\\ \text{effective description complexity}\\ \Downarrow\\ \text{generalization bound.}\end{array}}$$

The arrows are not automatic implications.

They denote separate questions that must be answered.

13.22 What Bits Do Not Explain

Description length is useful precisely because its scope can be stated clearly.

13.22.1 Bits do not explain recoverability

A target may possess a very short description and still be difficult to infer from examples.

Parity provides a simple illustration.

Cryptographic constructions provide more extreme ones.

Thus

$$\boxed{\text{short description} \Longrightarrow \text{easy recovery.}}$$

Description length is not a substitute for C2.

13.22.2 Bits do not identify the correct latent structure

Two different internal representations can implement the same predictor.

A short predictor can also exploit a shortcut.

Therefore

$$\boxed{\text{compression} \Longrightarrow \text{causal or semantic correctness.}}$$

13.22.3 Kolmogorov complexity is not computable

The true shortest program cannot in general be found.

Practical analyses must therefore use explicit coding languages, compression algorithms, covers, norm bounds, rank constraints, or other computable surrogates.

13.22.4 The metric matters

Description complexity always depends on what counts as a small change.

A predictor can be simple under one metric and complex under another.

Therefore covering numbers must specify

$$d.$$

For prediction, natural choices may depend on the loss, the data distribution, or the empirical sample.

13.22.5 Scale matters

Description length is resolution dependent.

A function may require many bits for extremely precise reproduction but few bits for task-level accuracy.

Hence

$$B_\varepsilon(f)$$

should be treated as a function of ε.

13.22.6 A post-hoc coding language is not free

Theorem 12 permits the predictor to be selected after seeing the training sample because the event is uniform over a coding language fixed independently of the sample.

But suppose one examines the data and then invents a special decoder that assigns a one-bit code to exactly the predictor obtained on that sample.

Such a construction has hidden the complexity inside the decoder.

The decoder itself must be described.

Hence

data-dependent predictor selection is allowed;	data-dependent code design must be accounted for.

Alternative approaches include valid sample-compression bounds, PAC-Bayes methods, localized complexities, algorithmic stability, or explicit coding of the learned dictionary.

13.23 Predictions and Empirical Tests

The bit viewpoint suggests several empirical questions.

13.23.1 Compressed complexity versus parameter count

Across predictors with similar training error, a meaningful fixed-code compression length should often be more informative about generalization than nominal parameter count.

One can compare

$$P$$

with

$$B_\varepsilon(f_{\hat{\theta}})$$

for a specified decoder and prediction tolerance.

13.23.2 Different biases can have comparable statistical bit costs

Suppose two mechanisms—for example sparsity and low rank—produce predictor families having similar valid coding lengths under the same prediction metric and loss.

Then their Occam terms can be similar even though their optimization behavior is very different.

Thus the appropriate prediction is not that all inductive biases having the same bit count behave identically.

It is the weaker statement

comparable valid description lengths $\Rightarrow$ comparable description-length capacity terms.

13.23.3 Structure discovery may reduce description length

Suppose training progressively discovers reusable constituents.

If those constituents admit an efficient fixed coding language, the description length of the learned predictor may decrease even while the nominal network size remains fixed.

One may therefore measure

$$t \longmapsto B_\varepsilon(f_{\theta_t})$$

together with

$$\text{training error}, \qquad \text{validation error}, \qquad \text{constituent recovery}.$$

A decrease in description length would provide evidence for progressive compression of learned structure.

It would not by itself prove that the recovered constituents are the true generative ones.

13.23.4 Memorizing unstructured exceptions should cost bits

Suppose a predictor has learned a compact rule and is then required to fit an increasing number of independent random label exceptions.

Those exceptions contain little reusable structure.

A valid code reproducing the predictor should therefore typically require additional bits approximately proportional to the information introduced by the exceptions.

This is a cleaner prediction than saying that every form of memorization must have large description length.

A dataset that looks like a collection of arbitrary examples to an observer may itself have a short generating program.

13.24 Open Problems

Several problems remain.

1. Which coding language best predicts generalization? Different architectures induce different natural codes.

There is no unique parameterization-independent practical description length.

A useful theory should compare explicit codes by their predictive value and invariance properties.

2. Does optimization reduce description length? The architecture may permit short descriptions without the training dynamics finding them.

A central question is whether SGD and regularization systematically move toward shorter effective descriptions in function space.

3. How does compositional reuse appear statistically? Sparse compositionality reduces local dimension.

Repeated constituents should give an additional description-length saving.

Quantifying the interaction between local arity, reuse, graph structure, and generalization remains an important problem.

4. How should data-dependent dictionaries be charged? If training learns a library of reusable features or operators, the library itself has a coding cost.

Subsequent examples may then be described cheaply relative to that library.

This leads naturally to hierarchical, conditional, or amortized description lengths.

5. Can description length distinguish rule learning from shortcuts? Two equally short predictors may exploit different structures.

Description length alone does not identify the desired solution.

Additional invariance, transfer, intervention, or causal criteria may be required.

13.25 Conclusion

Many forms of inductive bias used in deep learning appear superficially different.

Compositional sparsity limits local arity.

Weight sharing reuses the same constituent.

Sparse models activate few coordinates.

Low-rank models restrict effective matrix dimension.

Quantization limits precision.

Norm constraints reduce the size of function families.

Architectures define coding languages.

Optimization and regularization select particular predictors inside those languages.

All of these mechanisms can be studied through a common question:

How many bits are required to describe the selected predictor at the resolution relevant to prediction?

For an individual predictor encoded by a fixed prefix code,

$$L(f) \text{ bits} \Rightarrow |R(f) - \widehat{R}(f)| = O\left(\sqrt{\frac{L(f) + \log(1/\delta)}{n}}\right).$$

For an approximate coded representation,

$$R(f) \lesssim \widehat{R}(f) + \text{approximation error} + \sqrt{\frac{B_\varepsilon(f) + \log(1/\delta)}{n}}.$$

For an entire continuous function class,

$$\mathcal{H}(\varepsilon, \mathcal{F}) = \log_2 \mathcal{N}(\varepsilon, \mathcal{F})$$

measures the corresponding class-level covering bits.

Dudley chaining then relates covering complexity across scales to Rademacher complexity.

Compositional sparsity fits naturally into this picture.

For a fixed stable DAG containing s smooth constituents of local arity at most k,

$$\mathcal{H}(\varepsilon, \mathcal{F}_G) \lesssim Cs \left(\frac{B\mathcal{S}_G}{\varepsilon}\right)^{k/r}.$$

For an unknown graph with uniformly bounded stability

$$\mathcal{S}_G \leq S_0,$$

the graph itself costs additional bits:

$$\boxed{\mathcal{H}\left(\varepsilon, \mathcal{F}_{\mathrm{CS}}\right) \lesssim Cs\left(\frac{BS_0}{\varepsilon}\right)^{k/r} + O(sk\log(d+s))\,.}$$

Thus ambient dimension need not dominate the approximation exponent.
It survives primarily through the information required to identify structure.
The chapter therefore adds one more distinction to the framework of this book:

C1:	does a compact representation exist?
C2/C3:	can useful structure be recovered?
selection:	which fitting solution is chosen?
bits:	how statistically complex is the selected solution?

The unifying claim is not

$$\text{small architecture} \Rightarrow \text{generalization}.$$

It is the more precise statement

short valid description ⇒ small description-length capacity term ⇒ generalization under the corresponding sta

In this sense, bits provide a common currency connecting the representation theory of C1, the selection problem of optimization, and the statistical theory of generalization.

CHAPTER 14

Multiplicative Norm Regularization: Selection, Complexity, and an Experimental Case Study

Based on work with R. Dubach and M. Abdallah.

Norm-based generalization bounds for deep networks often contain multiplicative combinations of layer norms. This motivates a simple question: instead of applying ordinary additive weight decay, what happens if training directly penalizes a product of layerwise norms?

We compare the additive penalty

$$\Omega_{\mathrm{add}}(W) = \sum_{\ell=1}^{L} \|W_\ell\|_F^2$$

with the multiplicative penalty

$$\Omega_{\mathrm{mult}}(W) = \prod_{\ell=1}^{L} \|W_\ell\|_F^2.$$

The two penalties are not merely different scalar summaries of the same quantity. In positively homogeneous networks, the product is invariant under compensating layerwise rescalings that preserve the represented function, whereas the additive penalty generally is not. Their gradients also couple the layers in qualitatively different ways.

Exploratory experiments on convolutional networks trained on CIFAR-10 suggest that multiplicative regularization can select solutions with much smaller layer-norm products, larger margins, and strong suppression of later-layer singular values. In the original experiments, these solutions also showed higher test accuracy and substantially greater robustness to label corruption.

Those performance numbers should not yet be interpreted as a clean generalization result. The original protocol reused the test set for model selection and early stopping and discarded some unsuccessful training runs. The experiments therefore motivate a hypothesis rather than establish it.

The scientific question of this chapter is consequently:

Does directly penalizing a global layer-norm product systematically alter the solution selected by training, and does that change reduce effective complexity and improve population performance?

The chapter develops the theoretical motivation, reports the exploratory observations, and ends with the controlled experiments required to test the hypothesis.

14.1 Introduction

Overparameterized neural networks can contain many solutions having nearly zero training error.

Regularization therefore plays at least two conceptually different roles.

It can alter optimization,

$$\text{training dynamics} \longrightarrow \text{which solution is reached},$$

and it can alter statistical complexity,

$$\text{selected solution} \longrightarrow \text{capacity or compression measure}.$$

These two effects need not coincide.

Ordinary weight decay typically adds

$$\lambda \sum_{\ell=1}^{L} \|W_\ell\|_F^2$$

to the training objective.

Yet many norm-based analyses of deep networks contain products of layer norms, either directly or as part of more refined expressions involving spectral and Frobenius norms [79, 78, 32, 5].

This suggests a natural alternative:

$$\lambda \prod_{\ell=1}^{L} \|W_\ell\|_F^2.$$

The purpose of the chapter is not to claim that this product is the unique or canonical measure of neural-network complexity.

It is instead to test a more limited hypothesis:

> A regularizer that directly couples the norms of all layers may select different interpolating solutions from ordinary additive weight decay, and those solutions may have lower effective complexity and better generalization.

Three questions should therefore be distinguished:

Q1:	Does multiplicative regularization improve held-out performance?
Q2:	Does it select solutions with different norm, margin, or rank structure?
Q3:	Can those structural differences explain any generalization advantage?

The exploratory experiments reported below strongly motivate Q2.

Q1 and especially Q3 require a cleaner replication.

14.2 Norm-Based Motivation

For an L-layer network, let

$$W = (W_1, \dots, W_L).$$

Define the Frobenius norm product

$$P(W) = \prod_{\ell=1}^{L} \|W_\ell\|_F. \tag{14.1}$$

The regularizer used in the experiments is

$$\Omega_{\text{mult}}(W) = P(W)^2 = \prod_{\ell=1}^{L} \|W_\ell\|_F^2. \tag{14.2}$$

The additive comparison is

$$\Omega_{\text{add}}(W) = \sum_{\ell=1}^{L} \|W_\ell\|_F^2. \tag{14.3}$$

14.2.1 Relation to norm-based capacity bounds

The motivation for Equation (14.2) comes from the recurring appearance of multiplicative norm factors in deep-network complexity bounds.

A simple schematic form is

$$\mathfrak{R}_n(\mathcal{F}) \lesssim \frac{B}{\sqrt{n}} \prod_{\ell=1}^{L} \|W_\ell\|, \tag{14.4}$$

possibly with additional dependence on depth, width, margins, or other layerwise quantities.

More refined bounds can contain expressions such as

$$\frac{B}{\sqrt{n}} \left(\prod_{\ell=1}^{L} \|W_\ell\|_2 \right) \left(\sum_{\ell=1}^{L} \frac{\|W_\ell\|_F^2}{\|W_\ell\|_2^2} \right)^{1/2}. \tag{14.5}$$

The exact bound depends on assumptions and architecture.

Therefore

$$\boxed{\Omega_{\text{mult}}}$$

should be interpreted as a simple global norm-product surrogate, not as the unique theoretical capacity measure.

The empirical hypothesis is only

directly penalizing a global product-of-norms surrogate may favor lower-complexity solutions.

14.3 Additive and Multiplicative Penalties Are Different

Let

$$a_\ell = \|W_\ell\|_F^2.$$

The AM–GM inequality gives

$$\frac{1}{L}\sum_{\ell=1}^{L} a_\ell \geq \left(\prod_{\ell=1}^{L} a_\ell\right)^{1/L}. \tag{14.6}$$

Hence

$$\left(\frac{1}{L}\sum_{\ell=1}^{L} \|W_\ell\|_F^2\right)^L \geq \prod_{\ell=1}^{L} \|W_\ell\|_F^2. \tag{14.7}$$

Thus a sufficiently small additive penalty controls the product.
The converse does not hold.
A small product can coexist with a large sum if different layers have very different norms.
This already shows that the two regularizers can favor different factorizations.

14.4 Layerwise Rescaling in Homogeneous Networks

The distinction becomes sharper for bias-free positively homogeneous networks.

Consider a ReLU network satisfying

$$\sigma(cx) = c\sigma(x), \qquad c > 0.$$

Apply the layerwise transformation

$$W_\ell \longmapsto c_\ell W_\ell, \qquad c_\ell > 0, \qquad \prod_{\ell=1}^{L} c_\ell = 1. \tag{14.8}$$

Because of positive homogeneity, this transformation leaves the represented function unchanged.

Proposition 13 (Rescaling behavior of the two penalties)**.** *Under Equation* (14.8)*,*

$$P(c_1 W_1, \ldots, c_L W_L) = P(W_1, \ldots, W_L), \tag{14.9}$$

while in general

$$\sum_{\ell=1}^{L} \|c_\ell W_\ell\|_F^2 \neq \sum_{\ell=1}^{L} \|W_\ell\|_F^2. \tag{14.10}$$

Proof. For the product,

$$\prod_{\ell=1}^{L} \|c_\ell W_\ell\|_F = \left(\prod_{\ell=1}^{L} c_\ell\right)\left(\prod_{\ell=1}^{L} \|W_\ell\|_F\right) = P(W).$$

The additive expression instead becomes

$$\sum_{\ell=1}^{L} c_\ell^2 \|W_\ell\|_F^2,$$

which changes for a generic compensating rescaling. □

Thus

multiplicative penalty:	invariant along these function-preserving rescaling directions,
additive penalty:	selects a preferred factorization along them.

This does not prove that one regularizer generalizes better.
It does establish that they impose different geometry on parameter space.

14.4.1 Additive regularization favors balanced factorizations

For a fixed product of layer norms,

$$\prod_{\ell=1}^{L} \|W_\ell\|_F = C,$$

AM–GM implies that the sum of squared norms is minimized when the layer norms are equal:

$$\|W_1\|_F = \cdots = \|W_L\|_F.$$

Thus additive regularization has a balancing effect among function-equivalent homogeneous factorizations.

The product penalty is indifferent along those same rescaling directions.

14.5 Multiplicative Regularization Couples the Layers

The two penalties also generate different gradients.

For

$$\Omega_{\text{mult}} = \prod_{\ell=1}^{L} \|W_\ell\|_F^2,$$

and $W_j \neq 0$,

$$\nabla_{W_j} \Omega_{\text{mult}} = 2W_j \prod_{\ell \neq j} \|W_\ell\|_F^2 \tag{14.11}$$

$$= 2\Omega_{\text{mult}} \frac{W_j}{\|W_j\|_F^2}. \tag{14.12}$$

In contrast,

$$\nabla_{W_j} \Omega_{\text{add}} = 2W_j. \tag{14.13}$$

The relative multiplicative pressure on layer j is

$$\frac{\|\nabla_{W_j}\Omega_{\text{mult}}\|_F}{\|W_j\|_F} = \frac{2\Omega_{\text{mult}}}{\|W_j\|_F^2}. \tag{14.14}$$

This yields a concrete mechanistic prediction:

layers with smaller norms experience greater relative multiplicative pressure.

The pressure may therefore become highly nonuniform across depth.
A possible consequence is

$$\text{relative norm pressure} \longrightarrow \text{singular-value suppression} \longrightarrow \text{lower effective rank},$$

but this is an empirical hypothesis, not a theorem.

14.6 Product and Log-Product Objectives Are Not Equivalent

The identity

$$\log\left(\prod_{\ell=1}^{L}\|W_\ell\|_F^2\right) = \sum_{\ell=1}^{L}\log\|W_\ell\|_F^2 \tag{14.15}$$

is useful descriptively.
When minimizing only the regularizer over a fixed feasible set,

$$\arg\min_W \Omega_{\text{mult}}(W) = \arg\min_W \log\Omega_{\text{mult}}(W),$$

whenever the logarithm is defined.
But this does not imply

$$\arg\min_W \left[L(W) + \lambda\Omega_{\text{mult}}(W)\right] = \arg\min_W \left[L(W) + \lambda\log\Omega_{\text{mult}}(W)\right].$$

A monotone transformation of only one term changes its tradeoff with the data loss.
Product, squared-product, and log-product regularizers should therefore be treated as different optimization objectives.

14.7 Original Experimental Setup

The experiments below should be regarded as exploratory.
They motivated the present hypothesis but were not performed with a fully held-out model-selection protocol.

14.7.1 Binary CIFAR-10 task

The binary experiment uses CIFAR-10 classes

$$\text{cat} \quad \text{versus} \quad \text{deer},$$

coded as

$$-1 \quad \text{and} \quad +1.$$

The network contains four convolutional layers with channels

$$(16, 32, 64, 128),$$

with ReLU activations and 2×2 average pooling after each convolutional block.

The convolutional stack is followed by a fully connected hidden layer with 256 units and a scalar output.

The original implementation contains approximately

$$228{,}785$$

trainable parameters.

Training uses squared error,

$$L_{\text{data}} = \frac{1}{n} \sum_i \left(f_W(x_i) - y_i \right)^2 ,$$

with prediction

$$\operatorname{sign}(f_W(x)).$$

14.7.2 Multiclass CIFAR-10

The multiclass experiment uses all ten CIFAR-10 classes.

The architecture contains four convolutional blocks with channel sizes

$$(64, 128, 256, 512),$$

followed by a 512-unit fully connected layer and a ten-dimensional output.

One-hot targets and squared error are used in the original experiment.

The architecture contains approximately

$$5.74 \times 10^6$$

parameters.

14.7.3 Training objectives

The three objectives are

$$L_{\text{none}} = L_{\text{data}}, \tag{14.16}$$

$$L_{\text{add}} = L_{\text{data}} + \lambda \sum_{\ell=1}^{L} \|W_\ell\|_F^2, \tag{14.17}$$

$$L_{\text{mult}} = L_{\text{data}} + \lambda \prod_{\ell=1}^{L} \|W_\ell\|_F^2. \tag{14.18}$$

Weights are initialized with He initialization [39].

The original experiments also varied a global initialization scale.

14.8 Why the Original Performance Comparison Is Exploratory

Two features of the original protocol prevent the reported test differences from being interpreted as unbiased estimates of population performance.

14.8.1 Test-set reuse

The original test set was used for both early stopping and selection of regularization strength.

Thus the reported "test" accuracy participated in model selection.

If many hyperparameter settings are compared using the same test set and the best one is reported, the result is optimistically selected.

Likewise, choosing the stopping epoch using test performance compromises the independence of the final test evaluation.

The correct protocol is

$$\boxed{\text{training} + \text{validation} + \text{untouched test.}}$$

14.8.2 Optimization failures were discarded

Some runs that failed to train were excluded.

But optimization failure is itself part of the behavior of a learning method.

Discarding unsuccessful runs can bias the comparison.

Future experiments should therefore report both

$$\boxed{\text{optimization success probability}}$$

and

$$\boxed{\text{performance conditional on successful optimization.}}$$

14.8.3 Effect sizes

Accuracy differences should be reported in percentage points.

For example,

$$83.10\% - 73.69\% = 9.41$$

is a difference of

$$9.41$$

percentage points.

The abstract, tables, and text should ultimately be generated from the same analysis pipeline so that numerical summaries cannot drift between versions.

14.9 Exploratory Results from the Original Protocol

Despite these limitations, the original experiments reveal several striking patterns worth testing under a corrected protocol.

14.9.1 Binary classification and label noise

The reported binary test accuracies were:

training-label corruption	multiplicative	additive	unregularized
0%	89.50%	87.87%	87.80%
10%	86.68%	81.13%	80.20%
20%	83.10%	73.69%	73.08%

Thus the largest reported multiplicative–additive gap was

$$9.41$$

percentage points at 20% label corruption.

Because the hyperparameters and stopping time were not selected using an independent validation set, these values should be interpreted as hypothesis-generating rather than confirmatory.

14.9.2 Raw margins

For correctly classified binary examples, the original analysis used

$$m_i = y_i f_W(x_i).$$

The reported mean margins also favored multiplicative regularization and degraded more slowly under label corruption.

However, raw margins can depend strongly on parameter scale.

For positively homogeneous bias-free networks, the more relevant quantity is the normalized margin

$$\widetilde{m}_i = \frac{y_i f_W(x_i)}{P(W)} = \frac{y_i f_W(x_i)}{\prod_\ell \|W_\ell\|_F}. \tag{14.19}$$

The original binary architecture contains biases in its fully connected layers, so this normalization is not exactly invariant there.

A bias-free replication is therefore required before drawing a strong conclusion about margin improvement.

14.10 Norm Products and Layerwise Structure

The original experiment found a very large separation in norm products.

For the best reported binary configurations,

	multiplicative	additive	unregularized
$\sum_\ell \|W_\ell\|_F^2$	456	166	347
$P(W)$	3.01×10^2	1.71×10^4	9.42×10^4

Thus additive regularization produced the smallest sum of squared norms, as expected, while multiplicative regularization produced by far the smallest norm product.

This is important because it shows that the two penalties did not merely change the same scalar complexity quantity by different amounts.

They selected qualitatively different norm distributions across layers.

14.11 Singular-Value Suppression

The strongest structural observation in the original experiment concerns later layers.

The original statistic called "rank" counted singular values exceeding

$$1\%$$

of the largest singular value.

It should therefore be called the 1% *numerical rank*:

$$r_{1\%}(W) = \#\left\{j : \sigma_j(W) > 0.01\sigma_1(W)\right\}. \tag{14.20}$$

For the best reported binary configuration, the values were

layer	multiplicative	additive	unregularized
conv1	16	16	16
conv2	32	32	32
conv3	64	64	64
conv4	47	128	128
fc1	2	256	256
fc2	1	1	1

The collapse in the first fully connected layer,

$$256 \longrightarrow 2,$$

is particularly striking.

It should not yet be interpreted as proof that multiplicative regularization generally learns a rank-two representation.

The correct empirical question is whether the phenomenon is reproducible across seeds, hyperparameters, architectures, and an unbiased selection protocol.

14.11.1 Stable rank

Future analyses should also report

$$r_{\mathrm{stable}}(W) = \frac{\|W\|_F^2}{\|W\|_2^2}. \tag{14.21}$$

Unlike a hard singular-value threshold, stable rank varies continuously.

The full normalized spectrum

$$\left\{\frac{\sigma_j(W)}{\sigma_1(W)}\right\}_j$$

should also be shown.

The interesting hypothesis is therefore

multiplicative regularization may select lower-dimensional late representations.

14.12 Exploratory Multiclass Results

The original full-CIFAR-10 experiment showed a similar direction.

The reported averages were

regularization	$P(W)$	$\sum_\ell \|W_\ell\|_F^2$	train acc.	test acc.
multiplicative	3.73×10^7	679	0.891	0.687
additive	1.10×10^9	875	0.925	0.651
unregularized	1.67×10^9	960	0.937	0.650

The multiplicative configuration therefore had a much smaller norm product and a reported test-accuracy advantage of approximately

$$3.6$$

percentage points over the additive configuration.

Again, because model selection used the test set, this number should not be treated as a final estimate of the generalization advantage.

14.13 A Mechanistic Hypothesis

Taken together, the exploratory observations suggest a more precise hypothesis than simply

multiplicative regularization generalizes better.

The proposed mechanism is

global multiplicative norm coupling
$\Downarrow$
unequal relative regularization pressure across layers
$\Downarrow$
suppression of selected singular directions
$\Downarrow$
lower effective dimension / lower complexity
$\Downarrow$
larger normalized margins and possibly better generalization.

Only the first implication follows directly from Equation (14.14).

The later arrows are empirical hypotheses.

A particularly informative experiment would therefore track all of these quantities through training rather than comparing only final checkpoints.

14.14 What We Do Not Claim

Several stronger interpretations should be avoided.

14.14.1 The Frobenius product is not the unique complexity measure

Modern norm bounds can depend on spectral norms, Frobenius corrections, margins, architecture, and depth.

A low value of

$$P(W)$$

is therefore evidence about one complexity surrogate, not a complete generalization theory.

14.14.2 A smaller product does not by itself imply better generalization

The regularizer changes the optimization trajectory.

The selected predictor can change in many ways besides its norm product.

The appropriate question is empirical:

which measured complexity quantities best predict held-out performance across trained solutions?

14.14.3 Parameter flatness is not claimed

Raw Hessian eigenvalues and other parameter-space notions of "flatness" can change under reparameterization.

The present exploratory experiments do not establish that multiplicative regularization selects flatter minima in an invariant sense.

We therefore make no flat-minima claim here.

14.14.4 Rank reduction is not automatically beneficial

A low-rank representation can discard nuisance dimensions.

It can also discard useful signal.

The significance of the observed singular-value collapse must therefore be tested jointly with performance and robustness.

14.15 Connection to the Principles of the Book

This experiment tests a specific link in the framework developed earlier.

Chapter H separates interpolation from selection.

Overparameterization can create many fitting solutions:

many global interpolants.

Regularization changes which of them is reached.

Chapter 13 then asks how many bits or how much statistical complexity is required to describe the selected solution.

The present chapter therefore studies the chain

overparameterization
⇓
many fitting solutions
⇓
choice of regularizer
⇓
different selected norm/rank/margin structure
⇓
different effective complexity
⇓
possibly different generalization.

The final implication is not automatic.

That is exactly why this chapter is useful as an experimental case study.

14.16 Conclusion

Multiplicative and additive norm regularization are mathematically and dynamically different.

The additive penalty

$$\sum_{\ell} \|W_\ell\|_F^2$$

penalizes layer norms separately and, in positively homogeneous models, selects among functionally equivalent layerwise rescalings.

The multiplicative penalty

$$\prod_{\ell} \|W_\ell\|_F^2$$

is invariant under compensating homogeneous rescalings and couples all layers through its gradient.

For layer j,

$$\nabla_{W_j} \Omega_{\text{mult}} = 2\Omega_{\text{mult}} \frac{W_j}{\|W_j\|_F^2},$$

so the relative product pressure scales as

$$\frac{2\Omega_{\text{mult}}}{\|W_j\|_F^2}.$$

This provides a concrete mechanism by which different layers can experience very different regularization pressure.

The original experiments are consistent with a striking consequence: multiplicative regularization selected solutions with far smaller global norm products and dramatically lower numerical rank in some later layers.

The same experiments also reported higher test accuracy, larger margins, and much greater tolerance of label corruption.

Because of the original model-selection protocol, however, these performance differences remain exploratory.

The scientifically defensible conclusion is therefore

multiplicative norm regularization produces a distinct and potentially low-complexity selection bias.

Whether that bias produces a robust generalization advantage is an experimental question.

14.17 Experiments That Need to Be Done or Repeated

The following experiments are required before the stronger claims of this chapter can be regarded as established.

1. **Repeat the primary binary experiment with a genuinely untouched test set.**

   Split the original CIFAR-10 training data into training and validation subsets.

   For cat-versus-deer, a natural split is

   $$9{,}000 \text{ training} + 1{,}000 \text{ validation},$$

   with the original

   $$2{,}000$$

   cat/deer test images remaining untouched.

   All choices of

   $$\lambda, \quad \text{learning rate}, \quad \text{stopping epoch}, \quad \text{checkpoint}, \quad \text{initialization scale}$$

   must be made using training and validation data only.

   The test set should be evaluated once after the procedure is frozen.

2. **Use paired seeds and retain every algorithmic run.**

   For each seed, additive, multiplicative, and unregularized training should begin from exactly the same initialized weights.

   When possible they should also share

   $$\text{data split}, \quad \text{minibatch order}, \quad \text{augmentation randomness}, \quad \text{label-noise mask}.$$

   No run should be discarded because it fails to train.

   Instead report separately

   $$\text{optimization success probability}$$

   and

performance conditional on successful optimization.

At least ten paired seeds should be used; twenty would be preferable for the binary experiment.

3. **Repeat the label-noise experiment under two protocols.**

   Use corruption levels

$$0\%, \qquad 10\%, \qquad 20\%.$$

   For a given seed, use exactly the same corrupted labels for every regularizer.

   Evaluate two versions.

   Retuned protocol. Choose the optimal λ independently by validation at each noise level.

   This measures the best achievable performance of each regularizer.

   Fixed-λ protocol. Choose λ using clean validation data and hold it fixed as noise increases.

   This measures robustness without hyperparameter reoptimization.

4. **Repeat the multiclass CIFAR-10 experiment with clean model selection.**

   Use, for example,

$$45{,}000 \text{ training} + 5{,}000 \text{ validation} + 10{,}000 \text{ untouched test}.$$

   Use paired initialization across regularizers.

   The multiclass replication should preferably be performed after the binary effect survives the corrected protocol.

5. **Replace raw-margin comparisons by scale-normalized margins.**

   For a bias-free positively homogeneous network, measure

$$\widetilde{m}_i = \frac{y_i f_W(x_i)}{\prod_\ell \|W_\ell\|_F}.$$

   Report the

$$\text{mean}, \qquad \text{median}, \qquad 5\% \text{ quantile}, \qquad 10\% \text{ quantile},$$

   and full distribution.

   Because the original binary architecture contains biases, repeat the main experiment with a bias-free architecture.

   Alternatively, incorporate biases into an explicitly defined augmented norm and state that convention consistently.

6. **Repeat the rank experiment using full singular-value diagnostics.**
   For each layer, report

$$r_{1\%}(W) = \#\{j : \sigma_j > 0.01\sigma_1\},$$

   together with stable rank

$$r_{\text{stable}}(W) = \frac{\|W\|_F^2}{\|W\|_2^2}.$$

   Also plot

$$\frac{\sigma_j}{\sigma_1}$$

   for the entire spectrum.
   The key question is whether the dramatic collapse seen in conv4 and fc1 is reproducible.

7. **Measure the dynamics of multiplicative pressure.**
   During training record

$$\|W_j(t)\|_F,$$

$$P(W(t)),$$

$$\Omega_{\text{add}}(W(t)),$$

   and

$$\frac{\|\nabla_{W_j}\Omega_{\text{mult}}(W(t))\|_F}{\|W_j(t)\|_F}.$$

   At the same checkpoints record singular-value spectra, stable rank, normalized margins, validation loss, and validation accuracy.
   The mechanistic hypothesis predicts the temporal ordering

$$\boxed{\text{strong relative product pressure} \longrightarrow \text{spectral suppression} \longrightarrow \text{larger normalized margins} \longrightarrow \text{better held-}}$$

   Temporal ordering would be much stronger evidence than final-state correlation alone.

8. **Test whether the effect is specific to the squared product.**
   Compare

$$L + \lambda P(W),$$

$$L + \lambda P(W)^2,$$

and a stabilized log-product objective

$$L + \lambda \sum_{\ell} \log \left(\|W_\ell\|_F^2 + \eta \right).$$

These objectives are not equivalent.

The comparison can reveal whether the main effect is associated with

layer coupling, rescaling invariance, product magnitude, or the particular gradient profile.

9. **Compare competing complexity surrogates.**

   Across all seeds and hyperparameters, measure

$$P(W),$$

$$\sum_{\ell} \|W_\ell\|_F^2,$$

$$\prod_{\ell} \|W_\ell\|_2,$$

   and

$$\left(\prod_{\ell} \|W_\ell\|_2 \right) \left(\sum_{\ell} \frac{\|W_\ell\|_F^2}{\|W_\ell\|_2^2} \right)^{1/2},$$

   together with

   stable ranks

   and

   normalized margins.

   Without using test performance to choose the metric, ask which quantity best predicts the validation generalization gap across trained solutions.

   The purpose is not merely to verify that multiplicative training produces a smaller Frobenius product.

   The stronger question is

   which complexity measure actually tracks generalization?

10. **Test the interaction with initialization scale explicitly.**

    Use

    $$s \in \{0.2, 0.4, 0.6, 0.8\}$$

    as a factorial experimental variable rather than silently absorbing it into model selection.

    For every scale, tune λ using validation data and compare regularizers using paired seeds.

    Measure both final performance and optimization success probability.

11. **Test robustness across losses, optimizers, and architectures.**

    If the main phenomenon survives the primary corrected replication, repeat it using at least some of

    - cross-entropy rather than squared error;
    - SGD with momentum;
    - Adam or AdamW;
    - a second CNN;
    - a residual network;
    - different network depths.

    This will determine whether the effect is specific to the original setup or is a broader property of global norm-product regularization.

12. **Report paired effect sizes and confidence intervals.**

    For any metric M, define

    $$\Delta_j = M_{\text{mult},j} - M_{\text{add},j}$$

    for paired seed j.

    Report

    $$\overline{\Delta},$$

    its standard error,

    $$\text{SE}(\overline{\Delta}) = \frac{\text{sd}(\Delta_1, \ldots, \Delta_N)}{\sqrt{N}},$$

    and a 95% confidence interval.

    The paired difference is the primary quantity because the compared runs share initialization and data randomness.

A successful replication would show, under untouched-test evaluation and paired controls,

smaller norm product
+larger normalized margins
+reproducible late-layer rank suppression
+comparable training fit
+higher held-out accuracy
+greater label-noise robustness.

Especially strong evidence for the proposed mechanism would be a temporal relation in which multiplicative pressure precedes singular-value suppression, which in turn precedes improved normalized margins and validation performance.

Conversely, the hypothesis would have to be weakened if the apparent test advantage disappears under validation-only model selection, if rank collapse is not reproducible, if the effect depends on a narrow initialization regime, or if another complexity measure consistently predicts generalization better than the Frobenius norm product.

Either outcome would be informative.

The experiment is ultimately a test of a broader principle of this book:

in an overparameterized model, generalization may depend less on nominal parameter count than on the partic

CHAPTER 15

What the Hopfield Analogy Explains—and What It Does Not

Modern Hopfield networks and self-attention are closely related mathematically. Under appropriate parameterizations, an attention update can be interpreted as an associative-memory retrieval step, and repeated Hopfield-style updates can be analyzed through an energy function and its stationary states.

This analogy is useful.

But it has a precise scope.

A Transformer is not simply a Hopfield network iterated to equilibrium. Its depth consists of a sequence of generally different learned maps; its blocks include residual paths, normalization, attention, and nonlinear MLPs; and its parameters are themselves discovered through a separate optimization process.

Three dynamical systems must therefore be distinguished:

associative state dynamics:	*activations evolve with parameters fixed,*
Transformer depth:	*representations pass through different learned maps,*
learning dynamics:	*parameters evolve under optimization.*

A stability theorem for the first does not automatically imply a theorem for either of the other two.

This chapter develops that distinction. It also examines a tempting conjecture that fixed points of deep compositions should somehow be determined by the fixed points of their constituents. In general this is false.

What remains true, and useful for the framework of this book, is that any learned deep factorization retrospectively induces a sequence of local input–output datasets for its constituent functions. Those datasets are exactly the local supervision traces that would have made the learned constituent problems directly supervised.

Their existence after learning, however, does not explain how they were discovered during learning.

Nor must such a trace coincide with the latent or generating computation of the target: it is exact for the learned factorization itself.

That is the distinction between execution and recovery that runs throughout this book.

15.1 Why the Hopfield Analogy Is Useful

Associative memory is one of the recurring computational primitives of this book.

Given stored patterns or key–value pairs, an associative memory receives a partial query and retrieves content according to similarity.

Modern Hopfield networks provide one mathematical realization of this idea.

A simplified continuous update has the form

$$\xi^{(m+1)} = X \operatorname{softmax}\left(\beta X^\top \xi^{(m)}\right), \tag{15.1}$$

where the columns of

$$X = [\xi_1, \ldots, \xi_N]$$

represent stored patterns.

This has the same normalized-similarity structure as attention.

Under the fixed-norm conditions discussed in Chapter 9, dot-product attention can be written exactly as normalized Gaussian retrieval.

Thus the useful structural correspondence is

$$\boxed{\text{modern Hopfield retrieval } \approx \text{ normalized associative attention.}}$$

This explains several aspects of attention:

- content-addressable retrieval;
- concentration around relevant stored patterns;
- competition among memories through normalization;
- the existence, in appropriate iterative formulations, of attractor-like states.

These are genuine insights.

The difficulty begins when one tries to extend them from a local retrieval mechanism to a theory of the entire trained Transformer.

15.2 Three Different Dynamical Systems

The most important distinction is between three notions of dynamics that are often discussed with similar language.

15.2.1 Associative state dynamics

In a Hopfield-style system, the stored patterns and parameters are fixed.

One iterates

$$\xi^{(m+1)} = H_W(\xi^{(m)}),$$

where W denotes the fixed parameters.

The dynamical variable is the state

$$\xi.$$

The central questions are:

Does the iteration converge?

and

Which stationary state or attractor is reached?

In settings admitting an energy function

$$E_W(\xi),$$

the update can often be interpreted as descending or otherwise reducing that energy under suitable assumptions.

This is dynamics in *activation space*.

15.2.2 Transformer depth

A standard Transformer instead computes

$$x_{\ell+1} = F_\ell(x_\ell), \qquad \ell = 0, \dots, L-1.$$

Generally,

$$F_1 \neq F_2 \neq \cdots \neq F_L,$$

because different blocks have different learned parameters.

A standard forward pass does not repeatedly apply one map

$$x \mapsto F(x)$$

until it reaches an equilibrium.

It applies a sequence

$$F_L \circ F_{L-1} \circ \cdots \circ F_1.$$

Thus

$$\boxed{\text{Transformer depth} \neq \text{Hopfield relaxation to a fixed point.}}$$

A recurrent architecture with tied weights could make the analogy closer, but that is a special case rather than the generic Transformer.

15.2.3 Learning dynamics

Training introduces a third dynamical system.

Let

$$W^{(t)}$$

denote all trainable parameters at optimization step t.

Gradient descent evolves them according to

$$W^{(t+1)} = W^{(t)} - \eta \nabla_W \mathcal{L}\left(W^{(t)}\right). \tag{15.2}$$

Now the dynamical variable is

$$W,$$

not the activation ξ.

This is dynamics in *parameter space*.

15.2.4 The central distinction

The three systems are therefore

$$\boxed{\begin{aligned} \xi^{(m+1)} &= H_W(\xi^{(m)}) && \text{associative state dynamics,} \\ x_{\ell+1} &= F_\ell(x_\ell) && \text{depth dynamics,} \\ W^{(t+1)} &= W^{(t)} - \eta \nabla \mathcal{L}(W^{(t)}) && \text{learning dynamics.} \end{aligned}}$$

A Lyapunov function for the first does not automatically provide a Lyapunov function for the third.

Likewise, convergence of the state dynamics says nothing by itself about whether gradient descent will discover the correct representation.

15.3 The Scope of the Hopfield–Attention Correspondence

The phrase "attention is a Hopfield network" should therefore be used carefully.

A standard attention head computes

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

followed by

$$\mathrm{Attn}(X) = \mathrm{softmax}\left(\frac{QK^\top}{\tau}\right) V.$$

Modern Hopfield updates and attention can coincide or closely correspond under particular choices of stored patterns, queries, values, and normalization.

But several distinctions remain.

15.3.1 Query, key, and value maps need not coincide

A symmetric Hopfield interpretation is especially natural when the same representation plays the role of both key and retrieved pattern.

In general attention,

$$W_Q, \qquad W_K, \qquad W_V$$

are distinct.

The retrieval geometry and the retrieved content can therefore be different.

15.3.2 A Transformer normally performs one attention update per block

Equation (15.1) invites repeated iteration

$$\xi^{(0)} \mapsto \xi^{(1)} \mapsto \xi^{(2)} \mapsto \cdots$$

toward an attractor.

A Transformer block normally evaluates its attention operation once and passes the result onward.

Thus the equilibrium of an isolated attention update is generally not the object computed by the full network.

15.3.3 The correspondence concerns a primitive, not the entire architecture

The safe statement is

Hopfield theory provides an associative-memory interpretation of an attention primitive under appropriate ass

It does not imply

CHAPTER 16

A Self-Assembling Cortical Circuit for Generalized Gradient Descent (with Qianli Liao and Liu Ziyin)

The previous chapter emphasized a distinction that is essential for any biological theory of learning:

$$\textit{activation dynamics} \neq \textit{parameter-learning dynamics}.$$

Associative-memory and Hopfield-like mechanisms can explain how a fixed network retrieves or transforms representations. They do not explain how the synaptic weights implementing those representations are themselves learned.

This chapter addresses that second question directly.

Classical backpropagation computes gradients by transporting an error signal backward through the transpose Jacobians of the feedforward network. This is computationally effective, but its literal implementation in cortex would require mechanisms whose biological status is unclear: precise forward–backward weight relationships, coordinated transmission of error derivatives across many layers, and the availability of the appropriate factors at each synapse.

The Self-Assembling Learning (SAL) framework asks whether a structured but locally plastic circuit can achieve the same functional goal without requiring exact weight symmetry.

The proposal consists of two interacting pathways: an ascending representation stream and a descending feedback stream, coupled by plastic cross-connections. All synaptic changes are constructed from locally available activity products. The feedback pathway begins unstructured and is itself learned.

The central conditional result is:

if the auxiliary feedback couplings reach an approximately stationary, full-rank regime,

then the feedforward SAL update becomes

a positive-semidefinite, and under full-rank conditions positive-definite, preconditioning of the true gradient.

Thus SAL does not need to reproduce the backward weights of backpropagation. Functional gradient alignment is sufficient.

This distinction is important. The theorem establishes what SAL becomes once the auxiliary circuit has self-organized appropriately. *The stronger statement—that the complete dynamics reliably reach that regime from random connectivity—is a dynamical claim supported by the model and its experiments, but should be kept logically separate from the conditional theorem.*

The framework therefore proposes a possible solution to one particular problem:

How could a biological circuit implement gradient-like optimization using local plasticity?

It does not by itself answer a different question developed earlier in this book:

Why does the gradient contain useful information about the target?

That second question is the problem of genericity and recovery.

16.1 From Chapter 15: The Missing Learning Dynamics

Chapter 15 distinguished three dynamical systems:

$$\begin{aligned} &\text{state dynamics:} && \xi^{(m+1)} = H_W(\xi^{(m)}),\\ &\text{depth dynamics:} && x_{\ell+1} = F_\ell(x_\ell),\\ &\text{learning dynamics:} && W^{(t+1)} = W^{(t)} - \eta \nabla_W \mathcal{L}(W^{(t)}). \end{aligned}$$

The present chapter concerns the third.

The biological problem is not merely to stabilize an activation or retrieve an associative memory. It is to change synapses in a direction that improves a task-level objective.

16.1.1 What backpropagation accomplishes

Consider a feedforward network

$$h_{\ell+1} = \phi_\ell\left(W_\ell h_\ell\right), \qquad \ell = 0, \dots, L-1.$$

Define the preactivation

$$a_{\ell+1} = W_\ell h_\ell$$

and local derivative matrix

$$D^u_{\ell+1} = \operatorname{diag}\left[\phi'_\ell(a_{\ell+1})\right].$$

For loss

$$F(h_L, y),$$

backpropagation defines

$$\delta_L = -\nabla_{h_L} F$$

and recursively propagates

$$\delta_\ell = D^u_\ell W_\ell^\top \delta_{\ell+1}.$$

The corresponding gradient update is proportional to

$$\Delta W_\ell^{\mathrm{BP}} = \eta\, \delta_{\ell+1} h_\ell^\top .$$

Thus every synapse needs two factors:

$$\boxed{\text{presynaptic activity} \times \text{postsynaptic error-like signal.}}$$

The difficult biological question is how the second factor could become available locally.

16.2 The Biological Credit-Assignment Problem

Several distinct problems are often grouped under the phrase "biological implausibility of backpropagation."

16.2.1 Weight transport

Exact backpropagation uses

$$W_\ell^\top$$

in the backward pathway.

A literal implementation would require the feedback circuit to know the forward synaptic matrix with high precision.

16.2.2 Error transport

The network must propagate an appropriate derivative of the task loss through every layer.

16.2.3 Synaptic locality

A biological synapse cannot normally access arbitrary global variables.

Its plasticity must be driven by signals locally available to its presynaptic and postsynaptic compartments, perhaps augmented by nearby dendritic, modulatory, or heterosynaptic signals.

16.2.4 The top-level teaching signal remains necessary

A biologically local learning system does not eliminate the need for a task signal.

SAL begins the descending computation from an output error of the form

$$\bar{h}_L = -\nabla_{h_L} F.$$

Thus SAL does *not* require that every layer receive global information, but it does require some mechanism that generates a top-level teaching or mismatch signal.

The correct statement is therefore

$$\boxed{\text{SAL localizes credit propagation after a top-level teaching signal has been generated.}}$$

The biological source of that top-level signal is a separate question.

Depending on the task, possible implementations could involve reward, neuromodulation, mismatch circuitry, frontal or thalamic signals, or other mechanisms. None of these possibilities is required by the SAL mathematics itself.

16.3 The SAL Motif

SAL introduces two interacting neural streams.

16.3.1 Ascending stream

The ascending or upstream stream carries the representation used for the task:

$$h_0 \to h_1 \to \cdots \to h_L.$$

Its learnable synaptic matrices are

$$W_0, \ldots, W_{L-1}.$$

16.3.2 Descending stream

A second stream propagates information in the opposite direction:

$$\bar{h}_L \to \bar{h}_{L-1} \to \cdots \to \bar{h}_0.$$

Its within-stream synaptic matrices are

$$\bar{W}_0, \ldots, \bar{W}_{L-1}.$$

The dimensions of the descending representations need not equal those of the ascending representations.

Indeed, one important SAL regime uses an overparameterized descending stream:

$$\bar{d}_\ell \geq d_\ell.$$

16.3.3 Cross-stream couplings

The two streams interact through plastic cross-connections.

We denote them schematically by

$$V_\ell$$

and

$$\bar{V}_\ell.$$

Thus the smallest SAL motif contains four classes of synaptic connections:

$$\boxed{W_\ell, \qquad \bar{W}_\ell, \qquad V_\ell, \qquad \bar{V}_\ell.}$$

The key biological claim is not that these four matrices have genetically specified numerical relationships.

They can begin random.

What is specified is the *motif*: two interacting streams with plastic within-stream and cross-stream connections.

16.4 Forward and Feedback Computation

For clarity, we separate neural activations from their derivative gates.

16.4.1 Ascending computation

Let

$$a_{\ell+1} = W_\ell h_\ell$$

and

$$h_{\ell+1} = \phi_\ell(a_{\ell+1}).$$

Define

$$D^u_{\ell+1} = \operatorname{diag}\left[\phi'_\ell(a_{\ell+1})\right].$$

16.4.2 Error injection

At the output, SAL receives

$$\bar{h}_L = -\nabla_{h_L} F(h_L, y).$$

16.4.3 Descending computation

The descending stream has its own nonlinear transformations.

Schematically,

$$\bar{a}_\ell = \bar{W}_\ell \bar{h}_{\ell+1},$$

$$\bar{h}_\ell = \psi_\ell(\bar{a}_\ell),$$

with local derivative gate

$$D^d_\ell = \operatorname{diag}\left[\psi'_\ell(\bar{a}_\ell)\right].$$

Crucially,

$$\boxed{\bar{W}_\ell \neq W_\ell^\top}$$

in general.

The descending circuit is not required to be a copy of the ascending one.

16.5 Local Heterosynaptic Plasticity

The SAL learning rule is constructed from locally available activity products.

The exact indexing depends on the detailed implementation, but its structural form is simple.

16.5.1 Feedforward synapses

A feedforward synapse receives a local error-like factor generated from the descending stream and a presynaptic ascending activity.

Schematically,

$$\Delta W_\ell = \eta\, e_{\ell+1} h_\ell^\top - \gamma W_\ell,$$

where

$$e_{\ell+1}$$

is constructed locally from descending activity, cross-stream coupling, and the derivative gate of the upstream neuron.

16.5.2 Descending synapses

Similarly,

$$\Delta \bar{W}_\ell = \eta\, \bar{e}_\ell \bar{h}_{\ell+1}^\top - \gamma \bar{W}_\ell,$$

where $\bar{e}_\ell$ is locally generated using ascending activity and the cross-stream pathway.

16.5.3 Cross-stream plasticity

The characteristic SAL updates have outer-product form:

$$\Delta V_\ell \propto \bar{h}_\ell h_\ell^\top - \gamma V_\ell,$$

and

$$\Delta \bar{V}_\ell \propto \bar{h}_{\ell+1} h_{\ell+1}^\top - \gamma \bar{V}_\ell.$$

The important structural property is

$$\boxed{\Delta\text{synapse} = \text{local activity factor} \times \text{nearby activity factor} + \text{local decay}.}$$

This is where heterosynaptic plasticity enters.

A synaptic update may depend not only on activity across that synapse but also on activity delivered to nearby dendritic or cellular compartments.

SAL assumes the availability of such local interactions.

It does not require a synapse to know the complete state of the network.

16.6 The Main Conditional Result

The central mathematical statement concerns what happens after the auxiliary SAL circuitry has approached a quasi-stationary regime.

Theorem 14 (SAL implements a preconditioned gradient update)**.** *Consider a multilayer network trained by SAL.*

Assume that, for layer ℓ,

$$\Delta V_\ell = O(\varepsilon), \qquad \Delta \bar{V}_\ell = O(\varepsilon),$$

and that the relevant cross-stream matrix $\bar{V}_\ell$ has the rank required to span the ascending error space. Then the SAL update of the feedforward matrix has the form

$$\Delta W_\ell^{\mathrm{SAL}} = H_\ell \Delta W_\ell^{\mathrm{BP}} - \gamma W_\ell + O(\varepsilon),$$

where

$$H_\ell = \bar{V}_\ell \bar{V}_\ell^\top \succeq 0.$$

If $\bar{V}_\ell$ has full row rank, then

$$H_\ell \succ 0.$$

Thus, in the quasi-stationary full-rank regime, SAL follows a positive-definite preconditioning of the true feedforward gradient, up to the explicit decay term and an $O(\varepsilon)$ error.

Interpretation. The theorem does not say

$$\bar{W}_\ell = W_\ell^\top.$$

Nor does it require exact equality between any forward and backward synapses.

It says that the *effective update of the feedforward weights* is a positive-definite linear transformation of the correct gradient.

That is the quantity optimization needs.

16.7 Why Positive-Definite Preconditioning Is Enough

Suppose first that

$$\gamma = 0$$

and ignore the $O(\varepsilon)$ term.

In continuous time, SAL has the form

$$\dot{W}_\ell = -H_\ell \nabla_{W_\ell} F.$$

Then

$$\frac{dF}{dt} = \sum_\ell \left\langle \nabla_{W_\ell} F, \dot{W}_\ell \right\rangle.$$

Therefore

$$\frac{dF}{dt} = -\sum_\ell \left\langle \nabla_{W_\ell} F, H_\ell \nabla_{W_\ell} F \right\rangle.$$

If

$$H_\ell \succeq 0,$$

then

$$\boxed{\frac{dF}{dt} \leq 0.}$$

If every H_ℓ is positive definite, equality requires

$$\nabla_{W_\ell} F = 0$$

for all layers.
Thus

$$\boxed{H_\ell \succ 0 \quad \Longrightarrow \quad \text{preconditioning changes the geometry of descent, not its critical points}}$$

in the unregularized, exact continuous-time limit.

16.7.1 With weight decay

If

$$\gamma > 0,$$

the stationary SAL condition becomes

$$H_\ell \nabla_{W_\ell} F + \gamma W_\ell = 0.$$

This is not generally equivalent to

$$\nabla_{W_\ell} F = 0.$$

Therefore the claim that "SAL and SGD have the same stationary points" is correct only after specifying the relevant conditions.
A safe statement is:

$$\boxed{\gamma = 0, \quad H_\ell \succ 0, \quad \varepsilon \to 0 \quad \Longrightarrow \quad \text{SAL and gradient descent share critical points.}}$$

With weight decay, SAL instead follows a regularized, preconditioned dynamics whose stationary points need not coincide with those of unregularized gradient descent.

16.8 SAL as a Learned Metric on Parameter Space

The matrix

$$H_\ell$$

should not be viewed merely as an error relative to vanilla SGD.
It defines a metric or preconditioner in parameter space.
Ordinary gradient flow is

$$\dot{W}_\ell = -\nabla_{W_\ell} F.$$

SAL instead produces

$$\dot{W}_\ell = -H_\ell \nabla_{W_\ell} F.$$

If H_ℓ is well conditioned, this can rescale directions in a way that improves optimization.
Thus SAL is potentially more naturally interpreted as

a circuit that learns both the parameters and part of the optimizer geometry.

This suggests a stronger perspective:

SAL need not imitate vanilla gradient descent exactly; it need only generate a useful descent metric.

16.8.1 Conditioning matters

Let

$$\lambda_{\min}(H_\ell)$$

and

$$\lambda_{\max}(H_\ell)$$

be the smallest and largest eigenvalues.
The condition number

$$\kappa(H_\ell) = \frac{\lambda_{\max}(H_\ell)}{\lambda_{\min}(H_\ell)}$$

measures how unevenly the gradient directions are rescaled.
A nearly singular H_ℓ can suppress important directions even if the feedback activity itself is strong.
Thus rank alone is not sufficient.
The spectral geometry of the learned preconditioner matters.

16.9 What the Theorem Does—and Does Not—Prove

The logical status of the claims should be kept explicit.

16.9.1 What the theorem proves conditionally

If

$$\Delta V_\ell, \Delta \bar{V}_\ell$$

are small and the necessary rank condition holds, then the feedforward update is approximately a preconditioned true gradient.

16.9.2 What it does not automatically prove

The theorem does not by itself prove that:

- random initialization always evolves into that regime;
- full rank is preserved throughout learning;
- the matrices H_ℓ remain well conditioned;
- convergence occurs for arbitrary depth and task;
- the output teaching signal has a biologically realistic source.

Thus the complete SAL claim has two components:

conditional theorem:	quasi-stationary auxiliary circuit ⇒ preconditioned gradient,
self-assembly hypothesis:	local plasticity from random initialization reaches that regime robustly.

The first is analytic.
The second is a dynamical and empirical question.

16.10 Self-Assembly from Random Initialization

The motivation for SAL is precisely that the appropriate feedback structure need not be genetically specified synapse by synapse.

The auxiliary pathways begin with random weights.

During training, their local plasticity changes the effective transformation between the top-level teaching signal and the local feedforward updates.

16.10.1 Functional rather than literal alignment

The important goal is not

$$\bar{W}_\ell \to W_\ell^\top.$$

What matters is

$$\Delta W_\ell^{\mathrm{SAL}}$$

becoming aligned with a descent direction for the true objective.
A natural measure is

$$\cos\theta_\ell = \frac{\left\langle \Delta W_\ell^{\mathrm{SAL}}, -\nabla_{W_\ell} F \right\rangle}{\|\Delta W_\ell^{\mathrm{SAL}}\|_F \, \|\nabla_{W_\ell} F\|_F}.$$

Successful self-assembly requires at least

$$\cos\theta_\ell > 0$$

on average, and the theory predicts something stronger in the quasi-stationary regime: the update approaches a positive-definite transformation of the gradient.

16.10.2 Why overparameterized feedback may help

If the descending representation has dimension

$$\bar{d}_\ell > d_\ell,$$

there are more degrees of freedom available for constructing a full-rank mapping onto the ascending error space.

This motivates the empirical observation that wider feedback pathways can improve SAL performance.

The relevant theoretical quantity, however, is not width by itself.

It is whether width helps produce a well-conditioned

$$H_\ell.$$

16.11 Relation to Feedback Alignment

SAL is related to feedback-alignment approaches but differs in an important respect.

In basic feedback alignment, the backward pathway can remain random and fixed while forward weights gradually adapt so that the random feedback direction becomes useful.

SAL instead makes the feedback machinery itself plastic.

Thus

feedback alignment:	forward system adapts around a fixed approximate feedback path,
SAL:	forward and feedback systems co-adapt through local plasticity.

The SAL theorem therefore concerns *functional gradient alignment*, not the emergence of literal forward–backward weight symmetry.

16.12 Biological Plausibility at Three Levels

The phrase "biologically plausible" should be decomposed into three different claims.

16.12.1 Level I: Mathematical locality

The first question is purely computational:

> Can each synaptic update be expressed using quantities locally available to the relevant neurons or dendritic compartments?

SAL is designed to satisfy this condition.

This is the strongest and cleanest biological claim.

16.12.2 Level II: Candidate biological mechanisms

A second question is whether known biology contains mechanisms capable of supplying the required local factors.

Possible ingredients include:

- heterosynaptic plasticity;
- dendritic compartmentalization;
- calcium-dependent interactions among nearby synapses;
- synaptic tagging;
- retrograde or modulatory signaling;
- reciprocal cortical projections.

These mechanisms make SAL biologically suggestive.
They do not prove that SAL is implemented in cortex.

16.12.3 Level III: Does cortex actually implement SAL?

This is the strongest claim and remains empirical.

The distinction is

local implementability $\neq$ available biological substrate $\neq$ evidence that the brain uses this algorithm.

The purpose of the cortical mapping below is therefore to generate predictions, not to identify SAL with established cortical circuitry.

16.13 A Candidate Cortical Implementation

SAL is defined functionally by four pathway classes:

$$u \to u, \qquad u \to d, \qquad d \to d, \qquad d \to u.$$

Here u and d refer to the upstream and downstream computational streams.
These are abstract functional roles.
They need not correspond uniquely to particular cortical layers or cell types.

16.13.1 Driving versus modulatory interactions

One possible biological interpretation distinguishes:

- relatively driving interactions within a computational stream;
- more modulatory interactions between streams.

Dendritic compartmentalization provides one possible way to realize this difference.
Basal or proximal inputs may exert relatively direct effects on spiking.
Apical or distal inputs may instead modulate gain, plasticity, or the response to basal drive.
These statements are implementation hypotheses, not assumptions of the mathematical SAL algorithm.

SAL role	Weight	Candidate cortical pathway	Functional interpretation
$u \to u$	W	L4 $\to$ L2/3	driving ascending representation
$u \to u$	W	L2/3 $\to$ higher-area input layers	driving ascending representation
$u \to d$	V	L2/3 $\to$ L5	cross-stream coupling
$d \to d$	$\bar{W}$	deep-layer inter-areal / transthalamic paths	descending-stream propagation
$d \to u$	$\bar{V}$	feedback to L1/apical compartments	modulatory cross-stream signal
$d \to u$	$\bar{V}$	deep-to-superficial local feedback	modulatory cross-stream signal

Table 16.1: One candidate functional mapping between SAL pathways and cortical circuitry. The correspondence is illustrative rather than uniquely specified by SAL. The mathematical theory requires functional pathway types, not these particular cortical layers.

16.13.2 One candidate mapping

A possible correspondence, adapted from the original proposal, is:

16.13.3 What SAL does not require anatomically

SAL does not require:

- identical laminar organization across cortical areas;
- one-to-one matching of cortical layers to abstract network layers;
- exact symmetry between reciprocal pathways;
- feedback pathways that are merely copies of feedforward pathways.

Its core requirement is functional:

two interacting plastic streams whose local coupling can self-organize a useful feedback transformation.

16.14 Falsifiable Predictions

A useful biological learning theory should make predictions beyond "feedback exists."

16.14.1 Prediction 1: functional gradient alignment should emerge

During learning,

$$\cos\theta_\ell = \frac{\langle \Delta W_\ell^{\mathrm{SAL}}, -\nabla_{W_\ell} F \rangle}{\|\Delta W_\ell^{\mathrm{SAL}}\|_F \|\nabla_{W_\ell} F\|_F}$$

should become positive and, in the successful regime, increase.
The prediction concerns the *effective update*, not pairwise symmetry of individual synapses.

16.14.2 Prediction 2: cross-stream plasticity should stabilize as learning becomes competent

If the conditional theorem becomes accurate when

$$\Delta V_\ell, \Delta \bar{V}_\ell$$

become small, then the approach toward competent task learning should be accompanied by increasing stabilization of the cross-stream couplings.

Thus one should observe a temporal relation of the form

$$\boxed{\text{cross-stream organization} \rightarrow \text{gradient alignment} \rightarrow \text{efficient forward learning.}}$$

16.14.3 Prediction 3: feedback width should improve conditioning up to saturation

Increasing

$$\bar{d}_\ell$$

should initially improve the rank and conditioning of

$$H_\ell = \bar{V}_\ell \bar{V}_\ell^\top.$$

The effect should saturate once the relevant ascending error space can be represented robustly.

Thus the prediction is not simply

$$\text{more feedback is always better,}$$

but

$$\boxed{\text{feedback dimensionality matters through the spectrum of } H_\ell.}$$

16.14.4 Prediction 4: performance should track the spectrum of the learned preconditioner

SAL should deteriorate when

$$\lambda_{\min}(H_\ell)$$

approaches zero.

Performance should correlate with quantities such as

$$\kappa(H_\ell).$$

This prediction distinguishes mere feedback activity from useful gradient transport.

16.14.5 Prediction 5: disrupting heterosynaptic plasticity should selectively impair deep credit assignment

If SAL depends on cross-stream heterosynaptic interactions, then selectively weakening those interactions while leaving ordinary homosynaptic plasticity intact should disproportionately impair tasks requiring multi-stage credit assignment.

Simple one-layer association learning should be less affected.

16.14.6 Prediction 6: feedback need not mirror feedforward weights

Successful learning should not require

$$\bar{W}_\ell \approx W_\ell^\top.$$

A biological test should therefore look for functional update alignment without demanding anatomical weight symmetry.

16.14.7 Prediction 7: different physiological regimes should characterize within-stream and cross-stream pathways

If the functional division is correct, within-stream pathways and cross-stream pathways should differ systematically in their influence on neural responses or plasticity.

This may appear as differences in:

- dendritic targeting;
- time constants;
- gain modulation;
- nonlinear gating;
- plasticity rules.

16.15 Relation to C1, C2, and C3

SAL solves a different problem from the principles developed earlier in the book.

16.15.1 C1: representation

C1 asks whether a compact compositional representation exists.

SAL does not establish C1.

16.15.2 C2: informative recovery signal

C2 asks whether the input–output problem exposes a useful statistical signal to gradient-based learning.

SAL does not create such a signal.

If

$$\nabla_W F$$

contains no useful information because the target is a flat parity-like problem, then faithfully implementing that gradient does not solve the recovery problem.

16.15.3 C3: supplied trace

C3 asks whether intermediate variables are externally supplied.

SAL does not require full C3 trace exposure, although biological teaching signals or replay could interact with SAL in richer systems.

16.15.4 SAL: implementation of the optimizer

SAL asks:

given a useful gradient direction, how could a local biological circuit approximately implement it?

Thus the division of labor is

C1	:	a compact solution exists,
C2/C3	:	information makes recovery possible,
SAL	:	local circuitry implements a gradient-like parameter update.

This distinction is essential.

SAL is a theory of *credit transport and synaptic optimization*, not a substitute for a theory of learnability.

16.16 SAL and Genericity

The relation to genericity can be summarized in one line:

SAL determines how the gradient could be implemented; genericity determines whether the gradient is info

Suppose the expected gradient is

$$g_\ell = \nabla_{W_\ell} F.$$

SAL produces approximately

$$H_\ell g_\ell.$$

If

$$g_\ell = 0,$$

then

$$H_\ell g_\ell = 0.$$

No positive-definite preconditioner can manufacture information that is absent from the gradient.

Conversely, if genericity supplies a detectable footprint so that

$$g_\ell \neq 0,$$

then a positive-definite H_ℓ preserves a descent component.

This creates a particularly clean synthesis between the optimization and biological chapters of the book.

16.17 Relation to Chapter 15

The preceding chapter argued that an attractor theory of activations is not by itself a learning theory of weights.

SAL supplies a candidate learning theory at the parameter level.

The distinction is:

Hopfield-like dynamics:	how a fixed circuit evolves its activity,
SAL:	how a plastic circuit could evolve its synapses.

Thus Chapters 15 and 16 form a natural pair:

associative retrieval is not a learning algorithm
$\Downarrow$
a local self-organizing feedback circuit may implement one.

16.18 Implications for Artificial Learning Systems

SAL is motivated biologically, but the idea also suggests alternatives to fixed backpropagation machinery in artificial systems.

16.18.1 Learned feedback pathways

Rather than using exact transpose Jacobians, an artificial network could maintain an independently parameterized feedback network that adapts jointly with the forward model.

16.18.2 Learned preconditioning

The matrices

$$H_\ell$$

suggest that optimization geometry itself can be represented by adaptive neural circuitry.

This connects SAL conceptually to:

- learned optimizers;
- adaptive preconditioning;
- meta-learning;
- auxiliary feedback networks;
- local-learning architectures.

16.18.3 Local hardware

A learning rule based predominantly on local outer products may also be attractive for neuromorphic or distributed hardware, where transporting exact global gradients is expensive.

16.18.4 Continual adaptation

Biological systems do not make a clean separation between training and inference.

A self-organizing local optimizer could potentially operate continuously, allowing representations and their credit-assignment circuitry to co-adapt online.

This remains an architectural possibility rather than a consequence of the present theorem.

16.19 Limitations and Open Problems

Several issues remain unresolved.

16.19.1 Why should the auxiliary weights reach the required stationary regime?

The main theorem assumes

$$\Delta V_\ell, \Delta \bar{V}_\ell = O(\varepsilon).$$

A complete theory should derive conditions under which this occurs from random initialization.

16.19.2 Rank preservation

The positive-definite result requires appropriate rank.

Why rank should be preserved dynamically, and over what range of learning rates, widths, nonlinearities, and tasks, remains important.

16.19.3 Timescale separation

The conditional argument naturally suggests that the auxiliary pathways may need to organize on a suitable timescale relative to forward learning.

Whether explicit timescale separation is necessary is an empirical and theoretical question.

16.19.4 Origin of the teaching signal

SAL begins with

$$-\nabla_{h_L} F.$$

The neural origin of this signal is not specified by the local self-assembly mechanism.

A complete biological theory must explain how task objectives generate appropriate output-level teaching signals.

16.19.5 Spiking and recurrent biological dynamics

The mathematical model uses layered rate-like variables.

Cortex is recurrent, spiking, heterogeneous, and temporally structured.

The extent to which the SAL mechanism survives in more realistic dynamical systems is open.

16.19.6 Timing of local factors

The required local activity factors must coexist at the relevant synapses or dendritic compartments at appropriate times.

Biological eligibility traces may provide one possible mechanism, but this is not established by the present theory.

16.19.7 Very deep networks

The quality of the learned preconditioners and the accumulation of approximation error with depth require further study.

16.19.8 SAL does not solve informationally hard targets

Even a perfect implementation of backpropagation cannot overcome an end-to-end target whose accessible gradients contain no useful information.

This limitation is structural, not biological.

16.20 Conclusion

The central problem of this chapter is different from the representational and statistical questions considered earlier.

The question is:

How could biological synapses implement a useful parameter-space learning direction using only local circuitry

SAL proposes one answer.

A structured motif contains an ascending representation stream, a descending feedback stream, and plastic cross-connections.

The feedback weights need not copy the forward weights.

Instead, local heterosynaptic plasticity allows the complete circuit to self-organize.

Under the key conditional assumptions

$$\Delta V_\ell, \Delta \bar{V}_\ell = O(\varepsilon)$$

and sufficient rank, the resulting feedforward update satisfies

$$\boxed{\Delta W_\ell^{\mathrm{SAL}} = H_\ell \Delta W_\ell^{\mathrm{BP}} - \gamma W_\ell + O(\varepsilon),}$$

with

$$H_\ell = \bar{V}_\ell \bar{V}_\ell^\top \succeq 0.$$

Under full-row-rank conditions,

$$H_\ell \succ 0.$$

Thus SAL does not need exact backpropagation.

It needs a feedback circuit whose action on the true gradient is a useful positive-definite transformation.

This leads to a broader principle:

biological optimization may require functional gradient alignment, not weight symmetry.

The result also clarifies what remains open.
The theorem is conditional on successful self-organization of the auxiliary circuit.
The origin of the top-level teaching signal remains outside the model.
And SAL does not explain why the target generates an informative gradient in the first place.
The division of labor across the theory is therefore:

compositional sparsity: a feasible representation exists;
genericity / trace: useful recovery information exists;
SAL: local biological circuitry can implement a gradient-like update;
regularization and optimization: a particular solution is selected.

In this sense, SAL fills a specific missing piece of the book's framework.

Chapter 15 established that the dynamics of a fixed associative memory cannot substitute for a theory of synaptic learning.

The present chapter proposes a candidate mechanism for that learning:

local plasticity $\longrightarrow$ self-organized feedback $\longrightarrow$ learned preconditioning $\longrightarrow$ gradient-like optimization.

Whether cortex actually uses this mechanism remains an experimental question.

But SAL turns the question into a concrete one, with an explicit circuit, a mathematical limit, and falsifiable physiological predictions.

CHAPTER 17

Zeroth-Order Post-Training: Why Effective Dimension Matters (with Y. Gan)

Zeroth-order optimization offers an attractive possibility for post-training large models when gradients are unavailable, inaccessible, too expensive to store, or when one wishes to optimize a system-level black-box objective directly.

But zeroth-order optimization does not escape dimensionality.

A naive evolutionary search over all D parameters of a billion-parameter language model suffers from a fundamental variance and query-complexity problem: estimating a useful direction from function values becomes progressively harder as the dimension of the searched space increases.

The central claim of this chapter is therefore not

$$\textit{zeroth-order optimization works despite enormous dimension},$$

but rather

$$\boxed{\textit{zeroth-order optimization becomes practical when the } \text{effective adaptation dimension} \textit{ is small.}}$$

A large model may have

$$D \gg 10^9$$

parameters while useful adaptation is confined to an r-dimensional structured subspace,

$$r \ll D.$$

Writing

$$\theta = \theta_0 + Az, \qquad z \in \mathbb{R}^r,$$

changes the problem fundamentally. Zeroth-order search can then operate on z rather than on every coordinate of θ.

The coordinates z may control low-rank adapters, modules, routing weights, decoding parameters, mixture coefficients, prompt variables, training hyperparameters, or coordinated groups of model weights. In an evolutionary interpretation, the columns of A act like computational "genes": each mutation changes a structured collection of microscopic parameters simultaneously.

Thus the appropriate analogy with biological evolution is not mutation of individual synapses independently. It is mutation of a comparatively small number of structured regulatory variables.

The resulting principle is

zeroth-order methods do not defeat the curse of dimensionality; structure reduces the dimension that must be searched.

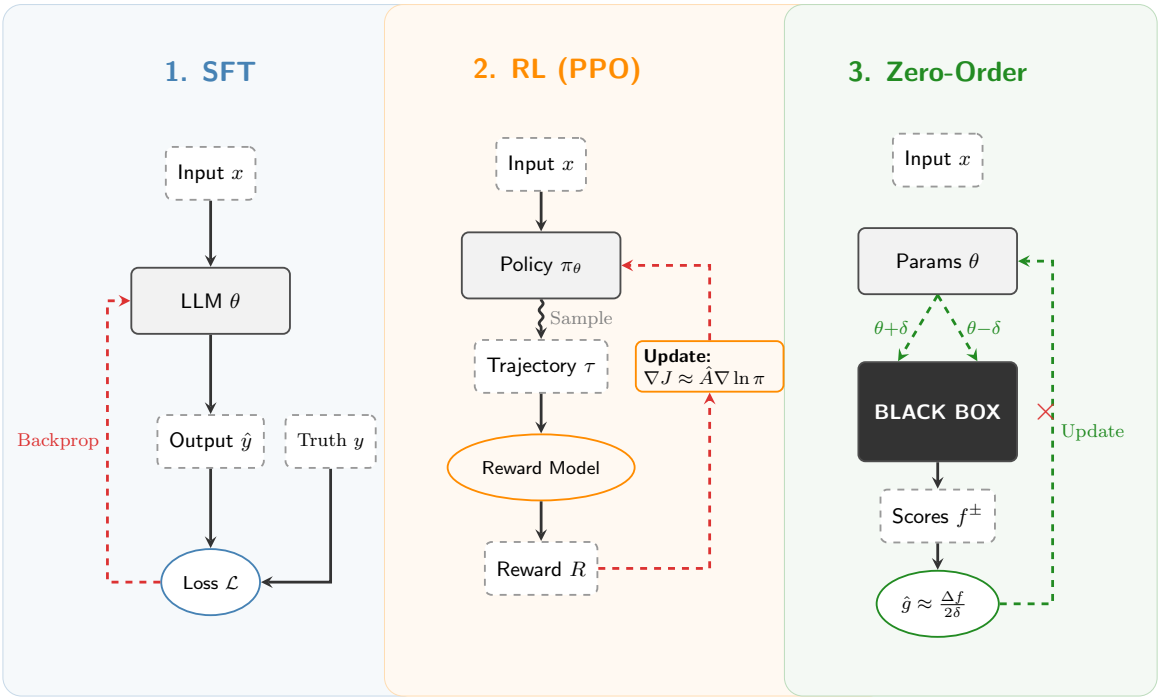


Figure 17.1: **Three routes to post-training.** Supervised fine-tuning differentiates a task loss through the model. Policy-gradient reinforcement learning can optimize a possibly nondifferentiable reward by differentiating the log probability of sampled behavior. Zeroth-order methods instead perturb parameters or lower-dimensional control variables and estimate useful directions using only objective evaluations.

17.1 Why Zeroth-Order Optimization?

The dominant optimization methods for deep networks are first-order.

Given an objective

$$f(\theta), \qquad \theta \in \mathbb{R}^D,$$

SGD and related methods use

$$\nabla_\theta f.$$

For large networks this is remarkably effective.
There are nevertheless settings where direct gradient access is undesirable or impossible.

17.1.1 Black-box model access

A model may be accessible only through an inference API.
One can evaluate

$$f(\theta)$$

or some behavior produced by θ but cannot backpropagate through the underlying system.

17.1.2 Memory-limited fine-tuning

Backpropagation normally stores intermediate activations required by the reverse pass.
Zeroth-order methods can operate using forward evaluations alone.
Methods such as MeZO exploit this fact to reduce memory requirements substantially during language-model fine-tuning [65].

17.1.3 System-level objectives

The quantity one ultimately wishes to optimize may involve an entire pipeline:

$$\text{model} \to \text{decoder} \to \text{tool use} \to \text{external evaluator}.$$

The resulting objective can be evaluated but may not possess a convenient computational graph linking it to every tunable system variable.

17.1.4 Outer-loop optimization

Some variables are naturally outside the gradient-trained model:

- curriculum schedules;
- data-mixture coefficients;
- decoding controls;
- routing parameters;
- augmentation policies;
- optimizer hyperparameters;
- system-level tradeoff coefficients.

These variables often form a comparatively low-dimensional black-box optimization problem.

17.2 Nondifferentiable Reward Does Not by Itself Require Zeroth Order

An important distinction is necessary.

Suppose a policy

$$\pi_\theta$$

generates trajectory τ with reward

$$R(\tau).$$

The reward itself need not be differentiable with respect to θ.

The expected reward is

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)].$$

Under standard regularity assumptions, the score-function identity gives

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[R(\tau)\nabla_\theta \log \pi_\theta(\tau)\right]. \tag{17.1}$$

Thus

$$R(\tau)$$

may be:

- a binary success signal;
- a programmatic verifier;
- a human preference;
- a discrete task score;
- a black-box external evaluation.

Policy-gradient methods can still estimate $\nabla_\theta J$ because they differentiate the policy likelihood, not the reward itself.

Therefore

$$\boxed{\text{nondifferentiable reward} \not\Longrightarrow \text{zeroth-order optimization is necessary.}}$$

17.2.1 What distinguishes ZO from policy gradient?

Policy gradient perturbs behavior through stochastic action sampling.

It uses

$$R(\tau)\nabla_\theta \log \pi_\theta(\tau).$$

Zeroth-order parameter search instead explicitly perturbs the parameters:

$$\theta \mapsto \theta + \sigma u.$$

Thus

policy gradient:	differentiate log-probability of sampled behavior,
zeroth-order search:	infer a parameter-space direction from objective queries.

ZO is particularly attractive when the parameterized system itself is black-box or when the variables being optimized lie outside the differentiable policy.

17.3 Gaussian Smoothing and Zeroth-Order Gradient Estimation

Let

$$f : \mathbb{R}^d \to \mathbb{R}.$$

Suppose gradients are unavailable but function evaluations are possible.
Choose

$$u \sim \mathcal{N}(0, I_d).$$

The symmetric two-point estimator is

$$\widehat{g}(\theta; u) = \frac{f(\theta + \sigma u) - f(\theta - \sigma u)}{2\sigma} u. \tag{17.2}$$

Define the Gaussian-smoothed objective

$$f_\sigma(\theta) = \mathbb{E}_{z \sim \mathcal{N}(0, I_d)} \left[f(\theta + \sigma z) \right]. \tag{17.3}$$

Under appropriate regularity assumptions,

$$\boxed{\mathbb{E}_u[\widehat{g}(\theta; u)] = \nabla f_\sigma(\theta).} \tag{17.4}$$

Thus the estimator is not generally an unbiased estimator of

$$\nabla f(\theta),$$

but of the gradient of a smoothed objective [77].

17.3.1 Why smoothing can help

Smoothing replaces a potentially irregular objective by

$$f_\sigma.$$

This can reduce sensitivity to small discontinuities or noisy evaluations.
But it introduces a tradeoff:

small $\sigma \Rightarrow$ low smoothing bias but high sensitivity to evaluation noise,
large $\sigma \Rightarrow$ lower noise sensitivity but stronger smoothing bias.

The appropriate σ therefore depends on smoothness, noise level, search dimension, and query budget.

17.4 The Curse of Search Dimension

The central limitation of zeroth-order optimization appears even on the simplest possible objective.

Proposition 14 (Dimension dependence for a linear objective)**.** *Let*

$$f(z) = g^\top z, \qquad z \in \mathbb{R}^r,$$

and let

$$u \sim \mathcal{N}(0, I_r).$$

For the two-point estimator

$$\widehat{g}(u) = \frac{f(z+\sigma u) - f(z-\sigma u)}{2\sigma} u,$$

we have

$$\widehat{g}(u) = uu^\top g,$$

and

$$\mathbb{E}[\widehat{g}] = g.$$

Moreover,

$$\mathbb{E}\left[\|\widehat{g} - g\|_2^2\right] = (r+1)\|g\|_2^2.$$

If q independent directions are averaged,

$$\widehat{g}_q = \frac{1}{q}\sum_{j=1}^{q} \widehat{g}(u_j),$$

then

$$\boxed{\mathbb{E}\left[\|\widehat{g}_q - g\|_2^2\right] = \frac{r+1}{q}\|g\|_2^2.} \tag{17.5}$$

Proof. For a linear objective,

$$f(z+\sigma u) - f(z-\sigma u) = 2\sigma g^\top u.$$

Hence

$$\widehat{g} = (g^\top u)u = uu^\top g.$$

Since

$$\mathbb{E}[uu^\top] = I,$$

the estimator is unbiased.
Rotational invariance allows us to take

$$g = \|g\| e_1.$$

Then

$$\|\widehat{g}\|^2 = \|g\|^2 u_1^2 \sum_{j=1}^{r} u_j^2.$$

Using Gaussian moments,

$$\mathbb{E}[u_1^4] = 3$$

and

$$\mathbb{E}[u_1^2 u_j^2] = 1, \qquad j \neq 1,$$

we obtain

$$\mathbb{E}\|\widehat{g}\|^2 = (r+2)\|g\|^2.$$

Since $\mathbb{E}\widehat{g} = g$,

$$\mathbb{E}\|\widehat{g} - g\|^2 = (r+1)\|g\|^2.$$

Averaging q independent estimates divides the variance by q.

□

The proposition exposes the fundamental issue:

q must grow with the dimension of the searched space to maintain fixed directional accuracy.

This is true before introducing nonlinear curvature, stochastic rewards, or model-evaluation noise.

17.5 Evaluation Noise Makes Dimension Still More Important

Suppose instead that an evaluation returns

$$Y(\theta) = f(\theta) + \xi,$$

where

$$\mathbb{E}[\xi] = 0, \qquad \mathrm{Var}(\xi) = \nu^2.$$

Using independent noise for the positive and negative evaluations, the noise component of a two-point estimator is

$$\widehat{g}_{\text{noise}} = \frac{\xi_+ - \xi_-}{2\sigma} u.$$

For

$$u \sim \mathcal{N}(0, I_r),$$

$$\mathbb{E}\|u\|^2 = r.$$

Thus, after averaging q independent pairs, the contribution of evaluation noise to the mean squared estimator norm is

$$\boxed{\frac{\nu^2 r}{2q\sigma^2}.} \tag{17.6}$$

Again, the relevant dimension appears explicitly.
This leads to the practical rule:

high reward noise + high search dimension = expensive zeroth-order estimation.

17.6 Nominal Dimension versus Effective Adaptation Dimension

A modern LLM may contain

$$D$$

parameters with

$$D \sim 10^9 \quad \text{or larger.}$$

Equation (17.5) makes clear why naive isotropic search over all D coordinates is unattractive.
But post-training need not require arbitrary movement in $\mathbb{R}^D$.
Suppose useful adaptation lies near

$$\theta = \theta_0 + Az,$$

where

$$A \in \mathbb{R}^{D\times r}, \qquad r \ll D.$$

The variable

$$z \in \mathbb{R}^r$$

is the *adaptation coordinate*.
Define

$$F(z) = f(\theta_0 + Az). \tag{17.7}$$

Zeroth-order optimization can now operate on

$$F : \mathbb{R}^r \to \mathbb{R}.$$

Its query complexity is controlled primarily by

$$r,$$

not by D.

D = nominal parameter dimension, $\qquad$ r = effective search dimension.

This distinction is the main principle of the chapter.

17.7 Projected Zeroth-Order Optimization

Assume for simplicity that the columns of A are orthonormal:

$$A^\top A = I_r.$$

Sample

$$u \sim \mathcal{N}(0, I_r).$$

Evaluate

$$f(\theta_0 + A(z + \sigma u))$$

and

$$f(\theta_0 + A(z - \sigma u)).$$

The reduced-space estimator is

$$\widehat{g}_z = \frac{F(z + \sigma u) - F(z - \sigma u)}{2\sigma} u. \tag{17.8}$$

Update

$$z \leftarrow z - \eta \widehat{g}_z.$$

Equivalently, in the original parameter space,

$$\theta \leftarrow \theta - \eta A \widehat{g}_z.$$

17.7.1 What direction is being estimated?

If f is differentiable,

$$\nabla_z F = A^\top \nabla_\theta f.$$

Lifting this gradient back to the model parameters gives

$$A \nabla_z F = A A^\top \nabla_\theta f.$$

Thus projected search follows the component of the full gradient lying inside

$$\mathrm{span}(A).$$

Define the missing-gradient component

$$e_A = (I - A A^\top) \nabla_\theta f. \tag{17.9}$$

Then

$$\boxed{\text{small } r \text{ is useful only if } e_A \text{ is also small.}}$$

This gives two conditions for efficient zeroth-order adaptation:

$$r \ll D,$$
useful adaptation directions lie approximately in $\mathrm{span}(A)$.

A low-dimensional but badly chosen subspace does not solve the problem.

17.8 Structured Mutations as Computational Genes

The projected formulation admits a natural evolutionary interpretation.

Write the columns of A as

$$A = [G_1, \ldots, G_r].$$

Then

$$\theta = \theta_0 + \sum_{j=1}^{r} z_j G_j.$$

Each G_j is a coordinated perturbation of the complete parameter vector.

A mutation in one scalar coordinate

$$z_j$$

therefore modifies many microscopic parameters together.

We can regard

$$G_j$$

as a computational analogue of a *gene* or regulatory module.

Evolution does not need to search every synapse separately.

It searches coefficients controlling reusable perturbation patterns.

microscopic dimension $D \gg$ evolutionary control dimension r.

17.8.1 Why this is more biologically plausible

Biological mutations typically do not alter one synapse at a time.

They change proteins, transcription factors, developmental programs, cellular properties, or regulatory pathways whose effects are reused across many cells and connections.

The analogous post-training strategy is therefore not

$$\theta_i \to \theta_i + \delta_i$$

independently for billions of parameters.

It is

$$z_j \to z_j + \delta_j,$$

where one z_j changes a coordinated parameter module.

17.8.2 Connection to sparse compositionality

If a network implements a computation through reusable constituents, then a natural mutation acts on a constituent rather than on every parameter independently.

Thus the same structural principle appears again:

compositional structure $\rightarrow$ modular parameterization $\rightarrow$ low-dimensional evolutionary control.

17.9 Where Can the Low-Dimensional Coordinates Come From?

Several practical parameterizations already have this form.

17.9.1 Low-rank adapters

Suppose a weight matrix is adapted by

$$W = W_0 + UV^\top,$$

with rank

$$r_{\mathrm{LoRA}} \ll \min(d_{\mathrm{in}}, d_{\mathrm{out}}).$$

Zeroth-order search can operate on a compressed parameterization of the adapter rather than on the full W.

Parameter-efficient methods such as LoRA provide an existing mechanism for reducing adaptation dimension [46].

17.9.2 Module scales

A large frozen network may contain modules

$$M_1, \ldots, M_r$$

with tunable gains

$$z_1, \ldots, z_r.$$

Then

$$h = \sum_j z_j M_j(x)$$

provides a direct low-dimensional control surface.

17.9.3 Mixture-of-experts routing

If a system contains a small number of experts, ZO can optimize routing or mixture weights rather than internal expert parameters.

17.9.4 Prompt and context variables

Continuous prompt vectors, retrieval weights, memory-selection parameters, or tool-routing coefficients may provide compact adaptation spaces.

17.9.5 Decoding parameters

Variables such as

$$T, \qquad p, \qquad \text{penalty coefficients},$$

often form an extremely low-dimensional system-level optimization problem.

17.9.6 Training controls

ZO can also optimize an outer-loop vector containing:

$$\text{curriculum weights}, \quad \text{data mixtures}, \quad \text{learning schedules}, \quad \text{reward coefficients}.$$

17.10 Three Families of Zeroth-Order Methods

The term "zeroth order" covers several conceptually different algorithms.

17.10.1 ZO gradient estimation

Methods such as Equation (17.2) use objective queries to estimate

$$\nabla f_\sigma.$$

The resulting estimate is then used much like an ordinary gradient.

17.10.2 Direct search

Direct-search methods do not necessarily construct a gradient estimate.

A simple population algorithm samples

$$z + \delta_1, \ldots, z + \delta_m$$

and retains improved candidates.

Nelder–Mead and related methods are classical examples [58].

17.10.3 Distribution search

Evolution strategies optimize a probability distribution over candidate solutions.

Let

$$z \sim \pi_\phi.$$

The objective is

$$J(\phi) = \mathbb{E}_{z\sim\pi_\phi}[f(z)].$$

One then adapts the distribution parameters

$$\phi.$$

This includes Natural Evolution Strategies and CMA-ES.

17.11 Natural Evolution Strategies

NES uses the score-function identity in parameter-search space [115]:

$$\nabla_\phi J(\phi) = \mathbb{E}_{z\sim\pi_\phi}\left[f(z)\nabla_\phi \log \pi_\phi(z)\right].$$

A Monte Carlo estimate is

$$\nabla_\phi J \approx \frac{1}{m}\sum_{i=1}^{m} f(z_i)\nabla_\phi \log \pi_\phi(z_i).$$

NES then applies a natural gradient,

$$\phi \leftarrow \phi - \eta F(\phi)^{-1}\nabla_\phi J,$$

where

$$F(\phi)$$

is the Fisher information matrix of the search distribution.

Thus NES adapts not merely a candidate parameter vector but the geometry of the mutation distribution.

17.12 CMA-ES and Adaptive Search Geometry

CMA-ES represents the search distribution as

$$z \sim \mathcal{N}(m, \sigma^2 C).$$

The covariance matrix

$$C$$

adapts to successful search directions [37].

A schematic covariance update is

$$C_{t+1} = (1 - c_1 - c_\mu)C_t + c_1 p_c p_c^\top + c_\mu \sum_{i=1}^{\mu} w_i y_i y_i^\top.$$

This allows the search distribution to stretch along useful directions and contract along irrelevant ones.

17.12.1 Why CMA-ES reinforces the effective-dimension principle

A full covariance matrix in r dimensions contains

$$O(r^2)$$

entries.

Thus full CMA-ES is naturally a method for moderate-dimensional control spaces, not for direct covariance adaptation over billions of model weights.

For larger spaces, diagonal, low-rank, separable, or otherwise structured covariance approximations are required.

Again,

adaptive evolutionary search becomes practical after structure has reduced the search dimension.

17.13 ZO-Friendly Control Surfaces in LLM Systems

The most compelling near-term uses of zeroth-order optimization are often not full-weight LLM training.

They are optimization problems surrounding a largely fixed model.

17.13.1 Very low dimension: decoding and system controls

For

$$r \sim 1 - 10,$$

examples include:

- temperature;
- nucleus-sampling thresholds;
- retrieval thresholds;
- ensemble weights;
- safety–helpfulness tradeoff parameters.

Such problems are natural candidates for direct search.

17.13.2 Low dimension: routing and mixtures

For

$$r \sim 10 - 10^2,$$

examples include:

- mixture-of-experts weights;
- domain-routing variables;

- reward-component weights;
- curriculum coefficients;
- data-mixture proportions.

CMA-ES, NES, or ordinary ZO gradient estimation may be appropriate.

17.13.3 Intermediate dimension: structured adapters

For

$$r \sim 10^2 - 10^4,$$

structured low-rank adapters or module-level perturbations may make zeroth-order search plausible, depending on query noise and parallel evaluation capacity.

17.13.4 Full parameter space

For

$$r = D, \qquad D \gg 10^6,$$

naive isotropic ZO suffers severely from search-dimensionality.
Successful demonstrations in very large models should therefore motivate the question:

What hidden low-dimensional geometry or parameterization makes the search effective?

rather than the conclusion that dimension has ceased to matter.

17.14 Memory-Efficient Zeroth-Order Fine-Tuning

A different motivation is memory rather than black-box access.

Methods such as MeZO perturb the parameter vector and use forward evaluations to estimate update directions without storing the full backpropagation graph [65].

This can substantially reduce memory consumption.

But the dimension problem remains.

Thus two different advantages should be distinguished:

memory advantage:	avoid storing backward activations,
query-efficiency advantage:	search a genuinely low-dimensional space.

The first does not imply the second.

17.15 Hybrid First-Order and Zeroth-Order Optimization

The strongest practical architecture may combine both modes.

17.15.1 First-order inner loop

Use SGD, Adam, or another differentiable optimizer for ordinary model weights:

$$\theta \leftarrow \theta - \eta \nabla_\theta L.$$

17.15.2 Zeroth-order outer loop

Use ZO to tune a low-dimensional controller

$$z,$$

where

$$z$$

may specify:

- reward weights;
- curriculum;
- mutation modules;
- data composition;
- architecture gates;
- decoding policy;
- training schedules.

Then

$$\boxed{\text{first-order learning solves the high-dimensional differentiable problem;}}$$

while

$$\boxed{\text{zeroth-order evolution solves the low-dimensional black-box meta-problem.}}$$

This resembles population-based training [48].

17.16 Relation to Reinforcement-Learning Post-Training

Recent reasoning-oriented systems have renewed interest in reward-driven post-training; DeepSeek-R1 is a prominent example using GRPO-style reinforcement learning [34].

The comparison with evolutionary ZO methods should be made carefully.

17.16.1 Policy-gradient RL

RL perturbs behavior through sampling and uses the model's own log-probability derivatives.

It retains first-order access to the policy.

17.16.2 Evolution strategies

ES perturbs the model or a structured adaptation coordinate directly:

$$z \to z + \sigma u.$$

No derivative of the policy with respect to z is required.
Thus ES can optimize a truly black-box pipeline.

17.16.3 Neither dominates universally

Policy gradients exploit far more information when gradient access is available.
ZO may be preferable when:

- model derivatives are unavailable;
- backward memory dominates cost;
- the objective spans nondifferentiable system components;
- evaluations can be massively parallelized;
- or the effective adaptation dimension is very small.

17.17 Experiments Suggested by the Theory

The effective-dimension view makes several concrete predictions.

17.17.1 Experiment 1: hold model size fixed and vary search dimension

Choose a fixed pretrained model with parameter dimension D.
Construct nested adaptation spaces

$$A_r \in \mathbb{R}^{D \times r}$$

for increasing

$$r.$$

At a fixed query budget q, measure improvement versus r.
The basic prediction is:

ZO performance should degrade once r/q becomes too large.

This would directly test Equation (17.5).

17.17.2 Experiment 2: structured versus random subspaces

At the same dimension r, compare:

- a random parameter subspace;
- a LoRA subspace;
- layerwise module scales;
- learned principal adaptation directions;
- hand-designed compositional modules.

If effective dimension alone were sufficient, all should behave similarly.
The stronger theory predicts otherwise:

$$\boxed{\text{small dimension} + \text{alignment with useful directions}}$$

is required.

17.17.3 Experiment 3: full-weight versus projected ZO

Compare

$$u \in \mathbb{R}^D$$

with

$$u \in \mathbb{R}^r, \qquad r \ll D,$$

using the same number of objective evaluations.
Prediction:

$$\boxed{\text{projected ZO should provide a much better direction estimate at fixed query budget when the subspace is well a}}$$

17.17.4 Experiment 4: evolutionary genes

Partition or parameterize the network into coordinated mutation modules

$$G_1, \ldots, G_r.$$

Compare mutation of these modules against independent weight-space perturbations.
Candidate genes include:

- attention-head modules;
- MLP blocks;
- adapter directions;
- normalization scales;
- groups of related layers.

The prediction is that structured mutations should dominate unstructured isotropic mutations at equal query budgets when the task adaptation is compositional.

17.17.5 Experiment 5: reward noise

Artificially vary the reward noise variance

$$\nu^2.$$

Equation (17.6) predicts degradation proportional to

$$\frac{\nu^2 r}{q\sigma^2}.$$

This provides a direct quantitative test of the theory.

17.17.6 Experiment 6: hybrid optimization

Train the model weights by ordinary gradient descent while evolving a small vector controlling:

$$\text{curriculum}, \quad \text{data mixture}, \quad \text{reward weights}, \quad \text{or routing}.$$

Compare with:

- fixed meta-parameters;
- gradient-based hyperparameter approximations;
- purely zeroth-order full-weight search.

The prediction is that ZO will be most competitive in the low-dimensional outer loop.

17.17.7 Experiment 7: estimate intrinsic adaptation dimension

Measure the smallest r for which a structured subspace reaches a fixed fraction of the performance achieved by unrestricted gradient fine-tuning.

Define, for example,

$$r_\alpha = \min \left\{ r : \Delta R_r \geq \alpha \Delta R_{\text{full}} \right\}.$$

This quantity provides an operational definition of

$$\boxed{\text{effective adaptation dimension.}}$$

17.18 Failure Modes

The framework predicts clear regimes where zeroth-order methods should fail.

17.18.1 No low-dimensional adaptation manifold

If useful updates occupy a genuinely high-dimensional region,

$$r \approx D,$$

then projected ZO offers little advantage.

17.18.2 Poor subspace alignment

Even if

$$r \ll D,$$

ZO fails if

$$\|(I - AA^\top)\nabla f\|$$

is large.

17.18.3 Highly noisy reward

When

$$\nu^2$$

is large, many repeated evaluations are needed.

17.18.4 Rare-event objectives

If almost every perturbation receives the same reward and informative events are extremely rare, local finite-difference estimation may carry little signal.

17.18.5 Excessively small perturbations

If

$$\sigma$$

is too small relative to evaluation noise, the numerator

$$f(\theta + \sigma u) - f(\theta - \sigma u)$$

is noise dominated.

17.18.6 Excessively large perturbations

If

$$\sigma$$

is too large, the method optimizes a heavily smoothed objective and may leave the local behavioral regime of the pretrained model.

17.18.7 Off-manifold parameter perturbations

Random full-weight mutations may destroy learned representations before revealing useful fitness differences.

Structured mutation spaces can reduce this problem.

17.19 Connection to Sparse Compositionality

The main theory of this book argues that feasible computation depends on low-arity compositional structure.

The same idea reappears here in parameter space.

A network may contain billions of microscopic parameters, but the useful degrees of freedom for a particular adaptation may correspond to only a small number of functional constituents.

Thus

$$\boxed{\text{large parameter count} \not\Longrightarrow \text{large effective adaptation dimension.}}$$

If a useful modification consists of changing

$$r$$

modules, then the evolutionary search should act on those modules.

17.19.1 C1 and zeroth-order search

C1 says that the target computation admits a compact structured representation.

For zeroth-order adaptation we need an analogous property:

$$\boxed{\text{the desired change to the trained system admits a compact structured parameterization.}}$$

This does not follow automatically from C1, but the two ideas are closely related.

17.19.2 Genericity still matters

Reducing the search dimension does not guarantee that fitness differences are detectable.

If mutations produce exponentially weak changes in reward, ZO search can still fail.

Thus the analogue of C2 remains:

$$\boxed{\text{useful mutations must leave detectable fitness footprints.}}$$

Again the division of labor is clear:

$$\boxed{\begin{array}{rcl} \text{structure} & : & \text{reduces the search dimension,} \\ \text{genericity} & : & \text{makes fitness differences detectable,} \\ \text{zeroth-order optimization} & : & \text{uses those evaluations to select a direction.} \end{array}}$$

17.20 Evolution as Structured Search

The evolutionary analogy becomes most compelling in this formulation.

Biological evolution does not know gradients.

It observes fitness differences.

Yet biological organisms contain astronomical numbers of microscopic degrees of freedom.

Evolution remains possible because mutations are not independent searches over every microscopic variable.

Genes, regulatory networks, developmental programs, and modular body plans create correlated low-dimensional perturbations.

The analogous artificial system is

$$\theta = \theta_0 + Az.$$

Evolution acts on

$$z,$$

while development or computation expands those coordinates into the large-scale parameter state

$$\theta.$$

This suggests the principle

evolution is efficient when genotype dimension is much smaller than phenotype dimension and the genotype is

For LLM post-training, the challenge is therefore to discover the artificial analogue of the genotype.

17.21 Conclusion

Zeroth-order optimization is attractive because it requires only evaluations of an objective.

For random direction

$$u,$$

the symmetric estimator

$$\widehat{g} = \frac{f(\theta + \sigma u) - f(\theta - \sigma u)}{2\sigma} u$$

provides a gradient estimate for a smoothed objective.

But the simplicity hides a fundamental limitation.

Even for a linear objective in r dimensions,

$$\mathbb{E}\|\widehat{g}_q - g\|^2 = \frac{r+1}{q}\|g\|^2.$$

The number of useful queries therefore scales with the dimension of the space being searched.

For an LLM with billions of parameters, naive isotropic full-weight evolution is consequently the wrong default model.

The important alternative is

$$\boxed{\theta = \theta_0 + Az, \qquad z \in \mathbb{R}^r, \qquad r \ll D.}$$

Now the search acts on the effective adaptation variables z.

The matrix A may encode:

low-rank adapters, modules, routing, prompt parameters, training controls, or coordinated mutation

The resulting complexity depends on

$$r,$$

provided that the useful adaptation directions lie largely in

$$\mathrm{span}(A).$$

Thus there are two requirements:

small search dimension, good structural alignment.

This gives a precise interpretation of evolutionary post-training.
The artificial analogue of a gene is not an individual neural-network weight.
It is a structured direction

$$G_j$$

whose mutation changes many microscopic parameters coherently.
Evolution then searches over the coefficients of a small collection of such directions.
The resulting picture parallels the central theme of this book:

high-dimensional systems become tractable when the relevant computation is organized through low-dimensio

For function approximation, that structure appears as sparse composition.
For generalization, it appears as low effective description complexity.
For biological learning, it appears as local circuit motifs.
For zeroth-order post-training, it appears as low-dimensional structured mutation.
The final lesson is therefore:

zeroth-order optimization does not conquer dimensionality.

Rather,

the right representation makes the dimensionality small enough that zeroth-order optimization can work.

That is the regime in which evolutionary post-training of very large models becomes a plausible computational strategy.

Part IV
Speculations

CHAPTER 18

Sparse Compositionality, Genericity, and the Random Hierarchy Model

The Random Hierarchy Model (RHM) introduced by Cagnetta and collaborators provides a particularly useful test bed for two principles developed earlier in this book: sparse compositionality and genericity.

The model contains an explicit hierarchical generative structure and exhibits a characteristic sample scale

$$P^* \sim n_c m^L$$

at which hierarchical feature learning becomes visible.

The central observation of this chapter is that the same scale arises from a simple detectability argument.

For a typical fixed realization of the random hierarchy, exact ensemble symmetries are broken by finite random fluctuations. These fluctuations produce weak but nonzero low-order correlations between local features and the global class.

The characteristic RMS scale of the relevant footprint is schematically

$$\rho_{\mathrm{rms}} \sim \frac{1}{\sqrt{n_c m^L}}.$$

Estimating one specified correlation of magnitude ρ from P independent examples requires, at constant confidence,

$$P \gtrsim \rho^{-2}.$$

Hence the characteristic detection scale is

$$\boxed{P_{\mathrm{detect}} \sim n_c m^L.}$$

This reproduces the characteristic scale observed in the RHM.

The interpretation is not that genericity alone proves the full feature-learning transition. Rather,

random finite-size asymmetry creates a detectable first rung of the learning staircase at the same characteristic sample scale

Sparse compositionality supplies the hierarchical representation that can be learned; genericity supplies a nonzero low-order footprint; and finite sample size determines whether that footprint can actually be detected.

The RHM therefore provides a concrete model in which the distinction introduced earlier between

existence of signal

and

detectability of signal

can be made quantitative.

18.1 Why the Random Hierarchy Model Matters

The theory developed in the preceding chapters separates two questions.

C1 asks:

Does the target admit a compact compositional representation?

C2 asks:

Does the observed learning problem contain detectable low-order information that can reveal that representatio

The Random Hierarchy Model is unusually useful because both structures are visible explicitly.

It contains a hierarchy of latent variables generated by local rules, while its random realization produces weak statistical asymmetries that can potentially guide feature learning.

Thus it provides a setting in which one can ask quantitatively:

How many examples are required before a weak local footprint of the hierarchy rises above sampling noise?

18.2 The Random Hierarchy Model in Minimal Form

We summarize only the aspects of the RHM needed for the present interpretation; the full construction and analysis are given in [18].

Consider a hierarchical generative system of depth

$$L.$$

At the top lies a class variable

$$Y \in \{1, \dots, C\}.$$

Each latent feature can be realized through several equivalent lower-level configurations or "synonyms."

Let

$$m$$

denote schematically the number of alternative production rules associated with a constituent.

After repeated expansion through L levels, the latent hierarchy produces the observed input

$$X.$$

A schematic generative chain is

$$Y \longrightarrow Z_L \longrightarrow Z_{L-1} \longrightarrow \cdots \longrightarrow Z_1 \longrightarrow X.$$

Each step is local and compositionally structured.

18.2.1 The characteristic sample scale

The RHM exhibits a characteristic sample scale of the form

$$P^* \sim n_c m^L, \tag{18.1}$$

where n_c is a class-dependent combinatorial factor in the construction.

Around this scale, hierarchical feature learning becomes pronounced.

The purpose of this chapter is not to reproduce the complete statistical mechanics derivation of Equation (18.1).

Instead, we ask whether the same scale has a direct interpretation in terms of the genericity framework developed in Chapter 6.

18.3 C1: When the Hierarchical Classifier Is Compositionally Sparse

The RHM is defined through a compositionally sparse *generative* process.

But learning is an inverse problem.

The generative direction is

$$Y \longrightarrow X,$$

while classification requires

$$X \longrightarrow Y.$$

These are not automatically equivalent statements.

18.3.1 Generative sparsity does not by itself imply inverse sparsity

A complicated inverse map can in principle arise from a simple forward generative process.

Therefore the correct C1 statement is conditional.

Suppose the local synonym sets and production rules are sufficiently identifiable that each collection of lower-level constituents determines its parent latent feature.

Then one can construct a bottom-up decoder:

$$X \longrightarrow \widehat{Z}_1 \longrightarrow \widehat{Z}_2 \longrightarrow \cdots \longrightarrow \widehat{Z}_L \longrightarrow Y.$$

Each decoder depends only on a bounded local neighborhood.

Under this deterministic local-identifiability condition, and assuming that the recovered top-level latent variable determines the class, the corresponding inverse classifier admits a compositionally sparse realization.

Proposition 15 (Hierarchical decoding under local identifiability)**.** *Suppose a depth-L hierarchical generative model has bounded local arity s, and suppose that at every level the parent latent variable can be recovered deterministically from its local children by a function*

$$g_\ell : \mathcal{Z}_{\ell-1}^{s} \to \mathcal{Z}_\ell.$$

Assume further that the recovered top-level latent state determines the class label.
Then the inverse classifier can be realized as a depth-L bounded-fan-in computation graph.
In particular, the ambient input dimension need not appear in the local arity of the constituent functions.

The relevant C1 statement is therefore

under local identifiability, the hierarchical inverse problem has a compact compositional solution.

It is an existence statement.
It does not yet explain how gradient descent discovers that solution.

18.4 C2: Randomness as a Source of Genericity

We now turn to the recovery problem.

A crucial distinction is between averaging over random hierarchies and learning one fixed realization.

18.4.1 Annealed symmetry

Let

$$\mathcal{G}$$

denote the randomly drawn hierarchy or grammar.
If we average over all possible realizations,

$$\mathbb{E}_{\mathcal{G}}[\cdot],$$

the construction may be exactly symmetric.
For example, a given low-level token or patch μ may have no preferred global class on average:

$$\mathbb{E}_{\mathcal{G}}[C_{\alpha\mu}(\mathcal{G})] = 0.$$

This is an *annealed* statement.

18.4.2 Quenched asymmetry

The learner does not train on the ensemble average.
It trains on one fixed draw

$$\mathcal{G} = \mathcal{G}_0.$$

In a finite random realization, exact symmetry is typically broken by finite-size fluctuations.
Thus one generally expects

$$C_{\alpha\mu}(\mathcal{G}_0) \neq 0$$

even though

$$\mathbb{E}_{\mathcal{G}}[C_{\alpha\mu}(\mathcal{G})] = 0.$$

This is the central genericity mechanism:

ensemble symmetry + finite random realization $\Longrightarrow$ quenched symmetry breaking.

Randomness is therefore not itself the definition of genericity.
Rather,

randomness is one mechanism that can produce genericity with high probability.

The genericity property belongs to the realized learning problem.

18.5 A Precise Low-Order Footprint

To make the discussion concrete, let

$$Y \in \{1, \dots, C\}$$

be the global class.
For class α, define a centered class indicator

$$\phi_\alpha(Y) = \mathbf{1}\{Y = \alpha\} - \Pr(Y = \alpha).$$

Let

$$X_\mu$$

denote an indicator, count, or other local observable associated with symbol or patch μ.
Define its centered form

$$\psi_\mu(X) = X_\mu - \mathbb{E}[X_\mu].$$

The simplest class–feature footprint is

$$C_{\alpha\mu} = \mathbb{E}\left[\phi_\alpha(Y)\psi_\mu(X)\right]. \tag{18.2}$$

This is a second-order moment in the indicator variables.
It is also a special case of the generic footprint introduced in Chapter 6:

$$\rho_\phi(f, P) = \mathbb{E}_P[f(X)\phi(X)].$$

Thus the RHM supplies a concrete instance of the general C2 question:

Is $C_{\alpha\mu} \neq 0,$ and if so, how large is it?

18.5.1 RMS footprint across hierarchy realizations

Because

$$\mathbb{E}_{\mathcal{G}}[C_{\alpha\mu}(\mathcal{G})] = 0$$

may hold by symmetry, the natural measure of the typical quenched magnitude is not the ensemble mean but the root-mean-square value

$$\rho_{\mathrm{rms}} = \left[\mathbb{E}_{\mathcal{G}} C_{\alpha\mu}(\mathcal{G})^2\right]^{1/2}. \tag{18.3}$$

The scaling argument below concerns this characteristic magnitude.

18.6 Random Finite-Size Fluctuations

The scale of the footprint can first be understood in a simple toy model.

Lemma 6 (Finite random counts produce $N^{-1/2}$ relative fluctuations)**.** *Let*

$$N_{\alpha\mu} \sim \mathrm{Binomial}(N, p).$$

Then

$$\mathbb{E}\left[\frac{N_{\alpha\mu}}{N}\right] = p$$

and

$$\mathrm{sd}\left(\frac{N_{\alpha\mu}}{N}\right) = \sqrt{\frac{p(1-p)}{N}}.$$

Hence, for fixed nondegenerate p,

$$\frac{N_{\alpha\mu}}{N} - p = O_{\mathrm{typ}}\left(N^{-1/2}\right).$$

The result is elementary but captures the relevant mechanism.

If a local symbol appears in a large random collection of hierarchical production paths, its class-conditional frequency is not exactly equal to its ensemble expectation.

The discrepancy is typically of order

$$N^{-1/2}.$$

18.6.1 Application to the hierarchy

In the RHM, the effective number of random combinatorial opportunities associated with the relevant class–feature relation scales schematically as

$$N \asymp n_c m^L.$$

The finite-size argument therefore predicts a characteristic RMS footprint

$$\boxed{\rho_{\mathrm{rms}} \sim \frac{1}{\sqrt{n_c m^L}}.} \tag{18.4}$$

For a typical nondegenerate realization, the magnitude of an appropriate low-order class–feature correlation is expected to be of this characteristic order.

Equation (18.4) should be understood as a scaling argument.

The exact coefficient and correlation structure depend on the detailed RHM construction.

The hierarchy contains dependencies, so the argument is not merely an application of an independent-binomial central limit theorem.

What matters for the present interpretation is the predicted finite-size scaling

$$\rho_{\mathrm{rms}}^2 \asymp \frac{1}{n_c m^L}.$$

18.7 Detectability from Finite Data

A nonzero population footprint is not enough.

It must be detectable from a finite training set.

For a fixed hierarchy realization, suppose the relevant population footprint is

$$\rho = \mathbb{E}[Z].$$

Using

$$P$$

independent training examples, estimate it by

$$\widehat{\rho}_P = \frac{1}{P}\sum_{i=1}^{P} Z_i.$$

Under ordinary finite-variance assumptions,

$$\mathrm{sd}(\widehat{\rho}_P) = O(P^{-1/2}).$$

Thus the signal-to-noise ratio scales as

$$\mathrm{SNR} \sim |\rho|\sqrt{P}. \tag{18.5}$$

For a typical realization whose footprint has characteristic magnitude

$$|\rho| \sim \rho_{\mathrm{rms}} \sim \frac{1}{\sqrt{n_c m^L}},$$

we obtain

$$\mathrm{SNR} \sim \sqrt{\frac{P}{n_c m^L}}. \tag{18.6}$$

The footprint becomes statistically resolvable when

$$\mathrm{SNR} \gtrsim 1.$$

Therefore the characteristic detection scale is

$$\boxed{P_{\text{detect}} \sim n_c m^L.} \tag{18.7}$$

This is the central calculation of the chapter.

18.8 The General Detectability Formula

The calculation can be written more abstractly.

Suppose a learner relies on a specified population footprint of magnitude

$$|\rho_N|.$$

If ordinary empirical noise decreases as

$$P^{-1/2},$$

then the sample scale required merely to distinguish the footprint from sampling fluctuations is

$$\boxed{P_{\text{detect}} \asymp \frac{1}{\rho_N^2}.} \tag{18.8}$$

The RHM provides the characteristic case

$$\rho_N \asymp (n_c m^L)^{-1/2},$$

and therefore

$$P_{\text{detect}} \asymp n_c m^L.$$

This gives an explicit realization of the two faces of genericity introduced earlier:

existence:	$\rho_N \neq 0,$
detectability:	$P\rho_N^2 \gtrsim 1.$

A footprint may be mathematically nonzero yet computationally irrelevant if it is exponentially small.

18.8.1 Searching over many footprints

Equation (18.8) describes detection of one specified footprint at constant confidence.

A learner may instead screen a family

$$\Phi$$

of candidate probes.

For bounded probes, a uniform concentration or union-bound argument gives schematically

$$\boxed{P_{\text{detect}} \gtrsim \frac{\log(|\Phi|/\delta)}{\rho_\star^2},} \tag{18.9}$$

where

$$\rho_\star = \max_{\phi\in\Phi} |\rho_\phi|.$$

Thus the inverse-square dependence on footprint magnitude remains the dominant term when the accessible probe family has polynomial cardinality.

If the learner must search an exponentially large family of candidate probes, however, the search cost itself can become substantial.

This makes explicit that detectability is relative not only to the target and data distribution, but also to the family of statistics accessible to the learner.

18.9 Relation to the RHM Threshold

The characteristic detectability scale

$$P_{\text{detect}} \sim n_c m^L$$

matches the characteristic sample scale

$$P^* \sim n_c m^L$$

reported for the onset of hierarchical feature learning in the RHM.

This coincidence suggests the interpretation

P^* is the scale at which the first useful hierarchical footprint becomes statistically visible to the learner.

This should be distinguished from a theorem asserting that footprint detectability alone completely determines the dynamical transition.

Several additional ingredients can matter:

- architecture;
- initialization;
- optimization dynamics;
- nonlinear feature interactions;
- finite-width effects;
- the choice and number of observables accessible to the gradient.

Thus the conservative claim is:

the genericity-based SNR calculation predicts the same sample scaling as the RHM feature-learning threshold.

That agreement is evidence for the recovery interpretation, not a proof that the two phenomena are identical in every regime.

18.10 Depth, Input Size, and the Meaning of Exponential Scaling

The threshold

$$P^* \sim m^L$$

is exponential in hierarchy depth L for fixed $m > 1$.
This should not automatically be called exponential in the raw input dimension.
If, for example, the number of leaves scales as

$$d = s^L,$$

then

$$m^L = d^{\log_s m}.$$

Thus a quantity exponential in depth can be polynomial in leaf count.
The precise statement is therefore

$$\boxed{P^* \text{ grows exponentially with hierarchical depth } L \text{ at fixed } m.}$$

Whether this corresponds to a classical curse of dimensionality depends on which dimension is being held as the complexity variable.

18.11 The Staircase Interpretation

The detectability calculation explains how the first rung of learning can become visible.
It does not yet explain the entire hierarchical learning process.
The natural interpretation in the language of Chapters 6 and H is a staircase.

18.11.1 Stage 1: a weak footprint becomes detectable

At initialization, the learner has access only to the raw variables.
A weak class–feature footprint has characteristic magnitude

$$|\rho_0| \sim (n_c m^L)^{-1/2}.$$

When

$$P \ll \rho_0^{-2},$$

the empirical signal is comparable to or smaller than sampling noise.
When

$$P \gtrsim \rho_0^{-2},$$

the correlation becomes statistically resolvable.

18.11.2 Stage 2: synonymous observations are quotiented

The learner can then begin mapping several lower-level observations to a common latent representation.

Schematically,

$$x^{(1)}, x^{(2)}, \dots, x^{(m)} \quad \longmapsto \quad z.$$

The representation becomes invariant to the choice of synonym.

18.11.3 Stage 3: a new coordinate system is created

After this quotienting step, the next level of the hierarchy is no longer expressed in terms of many raw symbols.

It is expressed in terms of the learned latent variables.

Thus a relation that was high-order in the original coordinates can become low-order in the new coordinates.

feature learning changes the effective degree of the remaining problem.

18.11.4 Stage 4: the next rung becomes visible

Once the first representation has been constructed, the effective footprint for the next constituent can become larger or lower-order.

Gradient-based learning can then act on that new signal.

The process repeats:

weak detectable footprint ⇓ learn local invariance ⇓ construct new coordinate ⇓ next dependency becomes low order ⇓ repeat.

This is the staircase mechanism in an especially concrete form.

18.12 Progressive Quotienting by Synonym Invariance

The representation-learning step can also be described mathematically as forming equivalence classes.

Suppose

$$x \sim x'$$

whenever two lower-level realizations correspond to the same latent feature.

Learning the latent representation amounts to constructing a map

$$q : \mathcal{X} \to \mathcal{X}/\sim$$

that identifies equivalent observations.
The network then operates on

$$q(x)$$

rather than on the original raw input.
At each level of the hierarchy a new quotient is learned.
Thus deep feature learning can be viewed as

successive removal of nuisance distinctions through learned equivalence relations.

This interpretation connects the RHM to the broader role of invariance in deep learning.

18.13 What Is Established and What Is Interpretation

It is useful to separate three levels of the argument.

18.13.1 Structural facts

The RHM contains an explicit hierarchy of local latent production rules.

Under deterministic local identifiability, and when the recovered top-level latent determines the class, the inverse classifier admits a bounded-fan-in hierarchical realization.

This is the C1 connection.

18.13.2 Statistical calculation

A finite random hierarchy typically breaks exact ensemble symmetry.

The finite-size argument predicts that the RMS magnitude of the relevant footprint has characteristic scaling

$$\rho_{\mathrm{rms}} \sim (n_c m^L)^{-1/2}.$$

For one specified footprint, elementary sampling theory then gives the characteristic detectability scale

$$P_{\mathrm{detect}} \sim \rho_{\mathrm{rms}}^{-2} \sim n_c m^L.$$

If a family of candidate probes must be screened, logarithmic search factors must additionally be included.

This is the statistical detectability calculation.

18.13.3 Dynamical interpretation

The stronger claim that gradient descent proceeds by the exact staircase

$$\text{weak footprint} \to \text{synonym collapse} \to \text{signal amplification} \to \text{next level}$$

is an interpretation of the observed representation dynamics.

It is consistent with the genericity framework and with the phenomenology of hierarchical feature learning.

But the broad claim that SGD universally follows such a staircase remains a conjectural component of the theory.

18.14 A Statistical Threshold versus a Recovery Threshold

The RHM also suggests a distinction that appears elsewhere in this book.

Let

$$P_{\mathrm{stat}}$$

denote the sample size at which the target is statistically determined within an appropriate hypothesis class.

Let

$$P_{\mathrm{rec}}$$

denote the sample size at which the structure becomes recoverable by a particular feasible learning algorithm.

In general,

$$P_{\mathrm{rec}} \geq P_{\mathrm{stat}}.$$

The detectability argument suggests interpreting

$$P^*$$

as having the character of a recovery threshold:

$$\boxed{\text{enough data for a weak footprint to become usable by feature learning.}}$$

Whether

$$P^* = P_{\mathrm{rec}}$$

in a precise theorem depends on the exact RHM setting and algorithm.

The conceptual point is that a feature-learning transition need not mark the moment at which the target first becomes statistically identifiable.

It can instead mark the moment at which the available algorithm gains access to the relevant structure.

18.15 What Would Happen in a Nongeneric Hierarchy?

The interpretation immediately suggests counterexamples.

Suppose the production rules were deliberately constructed so that all low-order class–feature correlations cancel exactly:

$$C_{\alpha\mu} = 0$$

for every accessible local probe μ.
Then increasing P would only estimate zero more accurately.
The detectability condition would never be satisfied at that level.
This is the analogue of parity:

$$\boxed{\text{compact hierarchical representation} \not\Longrightarrow \text{detectable low-order recovery signal.}}$$

Thus sparse compositionality alone is insufficient.
A hierarchy can be easy to represent yet difficult to discover.

18.16 Predictions Beyond the Original RHM

The interpretation makes several testable predictions.

18.16.1 Prediction 1: directly measure the footprint

For each fixed random hierarchy, estimate

$$C_{\alpha\mu}$$

for local symbols or patches.
Across independent hierarchy draws, the RMS magnitude should scale approximately as

$$\boxed{\left[\mathbb{E}_{\mathcal{G}} C_{\alpha\mu}(\mathcal{G})^2\right]^{1/2} \sim (n_c m^L)^{-1/2}}$$

for the footprint responsible for the transition.

18.16.2 Prediction 2: footprint magnitude should predict the transition sample

Different random hierarchy realizations should have different accidental footprint magnitudes.

For realization $\mathcal{G}$, define

$$\rho(\mathcal{G}).$$

The theory predicts

$$\boxed{P^*(\mathcal{G}) \propto \frac{1}{\rho(\mathcal{G})^2}}$$

up to architecture-, probe-search-, and optimization-dependent factors.
This is stronger than comparing ensemble averages.

18.16.3 Prediction 3: artificially suppressing the footprint should delay learning

One can modify a random hierarchy to balance local class frequencies more exactly.

If this reduces

$$|\rho|,$$

the feature-learning threshold should shift upward.

18.16.4 Prediction 4: artificially increasing asymmetry should lower the threshold

Conversely, introduce a controlled bias

$$\delta$$

into the local production rules.

If the induced footprint becomes

$$|\rho| \sim \delta,$$

then one expects

$$P_{\text{detect}} \sim \delta^{-2},$$

up to logarithmic probe-search and confidence factors.

18.16.5 Prediction 5: intermediate representations should increase the next-level footprint

Measure the relevant low-order correlations

$$\rho_0$$

in raw input coordinates and

$$\rho_1, \rho_2, \ldots$$

after successive learned layers.

The staircase interpretation predicts that once a constituent is learned,

$$|\rho_{\ell+1}|$$

for the next constituent should increase or become expressible at lower degree in the learned coordinates.

18.16.6 Prediction 6: supplied intermediate labels should bypass the first threshold

If the latent synonym classes are explicitly provided as intermediate supervision, the learner should no longer need to infer them from the weak quenched footprint.

This is the C3 prediction.

Thus trace supervision should reduce or eliminate the corresponding detectability threshold.

18.16.7 Prediction 7: kernel methods should not receive the same progressive coordinate advantage

A fixed kernel computes in a predetermined feature space.

A feature-learning network can alter its coordinates after discovering one level of the hierarchy.

Therefore the advantage of deep feature learning should grow when success requires repeated representation changes rather than a single fixed kernel representation.

18.17 Relation to Sparse Compositionality

The role of sparse compositionality in this chapter is specific.

It does not generate the initial statistical signal.

Rather, it ensures that once the correct latent variables are known, the remaining computation can be organized as a sequence of low-arity local problems.

Thus

$$\boxed{\text{sparse compositionality supplies the staircase structure.}}$$

Without C1, detecting one local correlation would not imply that the global target can subsequently be reconstructed by feasible local steps.

18.18 Relation to Genericity

Genericity plays a complementary role.

For a typical finite realization of the random hierarchy,

$$\boxed{\text{exact symmetry is broken and a low-order footprint is nonzero.}}$$

Its quantitative strength determines the number of samples needed before the learner can see it.

Thus

$$\boxed{\text{genericity supplies the first visible rung.}}$$

The complete interpretation is

$$\boxed{\begin{array}{c}\text{sparse compositionality}\\ \Downarrow\\ \text{a low-arity hierarchy exists}\\ +\\ \text{quenched genericity}\\ \Downarrow\\ \text{a low-order footprint exists}\\ +\\ P\rho^2 \gtrsim 1\\ \Downarrow\\ \text{the footprint becomes detectable}\\ \Downarrow\\ \text{feature learning can begin.}\end{array}}$$

For a learner screening many possible probes, the corresponding condition contains the additional logarithmic search factor described in Equation (18.9).

18.19 Relation to Genericity as a Property of the Observed Problem

Chapter 6 emphasized that genericity should not be attributed to a target function in isolation.

The relevant object is the observed learning problem:

$$(f, P, \mathcal{F}_{\text{probe}}).$$

The RHM illustrates this clearly.

The same underlying hierarchy can be easy or difficult depending on

- which local features are observable;
- how they are encoded;
- which correlations the architecture can access;
- how many candidate probes must be searched;
- how strongly the random realization breaks symmetry.

Thus the meaningful statement is not

the RHM target is generic,

but

a typical realized RHM problem exposes a nonzero low-order footprint in coordinates accessible to the learner.

18.20 A Compact Summary of the Mechanism

The entire argument can be summarized in five steps.

1. Hierarchical structure
The target admits a local decoder.

$\Downarrow$

2. Random finite realization
Exact ensemble symmetries are broken.

$\Downarrow$

3. Generic low-order footprint
$\rho_{\text{rms}} \sim (n_c m^L)^{-1/2}$

$\Downarrow$

4. Statistical detectability
$P\rho^2 \sim 1$

$\Downarrow$

5. Characteristic threshold
$P \sim n_c m^L$.

This identifies the observed scale with a very general statistical principle:

$$\boxed{P_{\text{detect}} \sim \frac{1}{\rho^2}}$$

for one specified footprint, with additional logarithmic factors when a family of probes must be searched.

18.21 Conclusion

The Random Hierarchy Model provides a concrete bridge between the abstract principles of this book and a quantitative model of hierarchical feature learning.

Its significance is not that random hierarchies automatically prove the general theory of deep learning.

Rather, the RHM isolates a mechanism that the theory predicts should be important.

First, under deterministic local identifiability, and assuming that the recovered top-level latent variable determines the class, the inverse classification problem has a compositionally sparse realization:

$$\boxed{\text{C1 provides a feasible hierarchical representation.}}$$

Second, a finite random realization breaks the exact symmetry of the ensemble.

For the relevant footprint, the finite-size argument predicts the characteristic RMS scaling

$$\boxed{\rho_{\text{rms}} \sim (n_c m^L)^{-1/2}.}$$

Third, finite-sample fluctuations in the empirical estimate scale as

$$P^{-1/2}.$$

Consequently, for one specified typical footprint,

$$|\rho|\sqrt{P} \gtrsim 1$$

requires the characteristic sample scale

$$\boxed{P \gtrsim n_c m^L.}$$

If the learner screens many candidate probes, this threshold acquires the corresponding logarithmic search factor.

The resulting characteristic scale agrees with the feature-learning scale reported in the RHM.

The natural interpretation is therefore:

$$\boxed{\text{the first hierarchical feature becomes statistically accessible when its quenched genericity footprint rises above e}}$$

Once that happens, the network may begin to construct a new coordinate system by identifying synonymous lower-level features.

The next level of the hierarchy is then expressed in those learned coordinates, where a previously high-order relation may become low-order.

This gives the staircase picture:

detect $\to$ quotient $\to$ re-encode $\to$ expose the next footprint $\to$ repeat.

The RHM therefore gives a particularly transparent example of the division of labor developed throughout the book:

sparse compositionality supplies the staircase;
generic finite-size asymmetry supplies a visible rung;
sample size determines whether that rung can be detected;
feature-learning dynamics determine whether the network climbs it.

The last step remains the deepest open part of the theory.

The agreement of the simple detectability scale with the RHM threshold does not prove the general staircase conjecture.

But it provides a concrete and quantitative instance in which the basic logic of that conjecture appears in exactly the expected form:

nonzero footprint + sufficient samples + compositional representation $\longrightarrow$ the possibility of hierarchical recove

Acknowledgments

We thank Giulio Cagnetta for discussions clarifying the role of second-order correlations in the Random Hierarchy Model and the connection between finite random fluctuations and the characteristic sample-complexity scale considered in this chapter.

CHAPTER 19

The Hippocampal Scaffold and Compositional Sparsity

The hippocampus is often described as a memory system, a cognitive map, or an indexing mechanism. These descriptions need not be competitors.

This chapter develops a computational hypothesis in which fast hippocampal learning supplies a temporary relational scaffold for slower cortical learning.

The hippocampus does not, on this view, directly contain the final compositionally sparse world model. Instead, it rapidly binds episodes, separates potentially interfering experiences, and preserves temporal and relational transitions. These intermediate structures may make hidden regularities easier for cortex to recover.

The central hypothesis is therefore

hippocampal indexing may expose an intermediate relational trace from which cortex can recover reusable compositional st

The proposal is deliberately computational rather than anatomical. Specific equations below should be read as abstractions of possible functions, not as literal descriptions of individual hippocampal subfields.

The scaffold is also not theorem-strength C3 in the sense developed earlier in this book. It does not necessarily expose the true latent variables or their parent sets. It is better viewed as a C3-like partial relational trace*: intermediate information that may reduce the hidden-variable recovery problem without solving it completely.*

19.1 Fast Episodic Indexing and Slow Structural Learning

An intelligent system faces two conflicting requirements.

It must remember individual experiences without catastrophic interference,

$$\boxed{\text{this episode} \neq \text{other similar episodes},}$$

while also discovering what those experiences have in common,

$$\boxed{\text{many episodes} \rightarrow \text{one reusable rule}.}$$

These objectives naturally favor different representations.

A fast episodic system should preserve distinctions.
A slower semantic system should discover equivalences.
We will use the following schematic division:

hippocampal system:	rapid binding, separation, indexing, relational traces,
cortical system:	slow parameter sharing, abstraction, reusable dynamics.

This resembles the logic of complementary learning systems [69], but the emphasis here is specifically on compositional recovery.

19.2 A Minimal Computational Model

Let

$$z_t$$

denote a latent state of the world relevant to the agent.
The agent does not observe z_t directly.
Instead it receives sensory observations

$$x_t \sim p(x_t \mid z_t)$$

and may also possess internally generated contextual information such as path integration, action, task state, or recent memory.
Let

$$c_t$$

collect this additional context.
An episodic memory system constructs an internal index

$$h_t = H_\psi(x_t, c_t), \tag{19.1}$$

where H_ψ is a fast encoding process.
The index h_t need not equal the latent state z_t.
Indeed, two visits to the same physical location may intentionally receive different episodic indices because they occurred at different times or contained different events.
Thus

$h_t =$ episode-specific address, $\qquad z_t =$ shared latent structure.

The distinction between these variables is central.
The indexing interpretation is broadly compatible with classical hippocampal indexing theories [**teyler1986hippocampal**, **teyler2007hippocampal**], but the variable h_t here is a computational abstraction rather than a claim about a particular neural code.

19.3 Pattern Separation as an Indexing Primitive

One classical computational role attributed to hippocampal circuitry is pattern separation [**oreilly1994hippocampal yassa2011pattern**, 67].

Similar sensory experiences can interfere if represented too similarly. A useful episodic code should therefore make distinct episodes easier to store and retrieve separately.

A minimal abstract requirement is

$$x_t \approx x_s \quad \not\Longrightarrow \quad h_t \approx h_s$$

when the two inputs belong to distinct episodes that must remain individually accessible.

At the same time, some similarity structure may need to remain recoverable through contextual cues.

One toy implementation is a high-dimensional expansion followed by sparse competition:

$$h_t = \mathrm{TopK}\left(R\, g(x_t, c_t)\right), \tag{19.2}$$

where R is a high-dimensional projection and only a small fraction of coordinates remain active.

Equation (19.2) is not proposed as a literal model of dentate-gyrus computation.

It is simply an example showing how

$$\boxed{\text{expansion} + \text{sparse competition}}$$

can reduce interference among correlated episodes.

19.4 From Episodic Indices to a Scaffold

As an agent moves through an environment, it generates a sequence

$$h_1, h_2, \ldots, h_T.$$

Observed temporal transitions define a graph

$$\mathcal{G}_H = (\mathcal{H}, \mathcal{E}), \tag{19.3}$$

where

$$\mathcal{H} = \{h_t\}$$

and

$$(h_t, h_{t+1}) \in \mathcal{E}$$

whenever the corresponding episodes occurred consecutively or were otherwise relationally linked.

We call this the *hippocampal scaffold*.

The scaffold records something richer than an unordered episodic store.

It preserves

$$\boxed{\text{which experiences were connected to which other experiences.}}$$

This connectivity may reflect

spatial adjacency, temporal order, action consequences, relational similarity.

The idea is compatible with cognitive-map and relational-memory views of hippocampal function [**okeefe1978hippocampus**, **eichenbaum1999hippocampus**], without requiring all scaffold edges to correspond to Euclidean spatial relations.

19.5 Graph Sparsity Is Not Yet Compositional Sparsity

An important distinction is necessary.

A sparse transition graph is not automatically a compositionally sparse function in the sense used elsewhere in this book.

Low graph degree means that from a particular episodic state only a small number of successors may have been observed.

It does not by itself imply that the underlying world dynamics factor into bounded-arity constituent functions.

The stronger structure appears only if the latent state itself factorizes.

Suppose

$$z_t = \left(z_t^{(1)}, \ldots, z_t^{(m)}\right) \tag{19.4}$$

and the dynamics satisfy

$$z_{t+1}^{(i)} = F_i\left(z_t^{\mathrm{pa}(i)}, a_t\right), \qquad |\,\mathrm{pa}(i)| \le k, \tag{19.5}$$

where

$$k \ll m.$$

Then the world dynamics possess a genuine bounded-fan-in compositional structure.

The hippocampal hypothesis proposed here is not that the scaffold *creates* this structure.

It is that the scaffold may make evidence for it easier to recover.

19.6 The Scaffold as a Partial Relational Trace

This connects directly to the distinction between representation and recovery developed earlier in the book.

Suppose cortex observes only raw sensory pairs

$$(x_t, x_{t+1}).$$

It must simultaneously infer

latent state, state identity across time, local transition structure, shared operators.

This is a difficult hidden-variable problem.

Now suppose a fast episodic system provides

$$h_t, \qquad h_{t+1},$$

together with repeated relational transitions among episodes.

The slower learner receives a richer training signal:

$$\boxed{(x_t, h_t, h_{t+1}, x_{t+1}).}$$

The indices need not equal the desired latent variables.

Nor do they reveal the true parent sets of Equation (19.5).

Their value is that they expose intermediate relational structure.

The scaffold is therefore best viewed as a *C3-like partial relational trace*.

It does not provide theorem-strength hard C3, in which the true constituent variables and their local dependencies are exposed directly.

Instead it may shorten the hidden computation that remains to be recovered:

$$\boxed{\text{hard C3} \;>\; \text{partial relational scaffold} \;>\; \text{raw end-to-end observation},}$$

where the ordering denotes informativeness rather than literal set inclusion.

> **Scaffold Hypothesis.** A fast episodic system can improve cortical recovery of compositional structure by exposing stable intermediate relations among otherwise high-dimensional and ambiguous sensory observations.

This is a hypothesis about learnability.

It is not a claim that hippocampal indexing is mathematically necessary for compositional learning.

19.7 From Episodic Indices to Latent Variables

The cortex should not simply memorize every hippocampal index.

That would reproduce episodic memory rather than abstraction.

Instead, repeated experience may reveal equivalence classes among indices.

For example,

$$h_{t_1}, \qquad h_{t_2}, \qquad h_{t_3}$$

may correspond to different episodes that share the same underlying structural state.

The cortex may learn a mapping

$$\phi : (x_t, h_t) \longmapsto \widehat{z}_t \tag{19.6}$$

such that

$$\phi(x_{t_1}, h_{t_1}) \approx \phi(x_{t_2}, h_{t_2})$$

when the two episodes instantiate the same relevant latent configuration.

Thus cortical learning performs a kind of quotienting:

$$\boxed{\text{many episodic indices} \to \text{one shared latent state.}}$$

The hippocampal system favors separation.

The cortical system learns which distinctions can later be discarded.

This tension between preserving individual episodes and discovering shared structure is closely related to the computational motivation behind complementary learning systems [69].

19.8 Replay as Structured Training Data

Replay provides a possible mechanism by which the episodic scaffold can continue to train slower cortical systems after the original experience has ended.

Hippocampal replay and reactivation have been observed during sleep and quiet wakefulness [**wilson1994reactivation**, **lee2002memory**, 30].

A replayed trajectory has the schematic form

$$h_{t_1} \to h_{t_2} \to \cdots \to h_{t_r}.$$

Associated sensory, contextual, action, or reward information may also be reactivated.

From a computational perspective, replay changes the sampling distribution available to cortical learning.

Instead of receiving only the original online stream, cortex may be exposed repeatedly to selected relations such as

$$(h_t, h_{t+1}), \qquad (h_t, y_t), \qquad (h_t, a_t, h_{t+1}).$$

This may support

consolidation, credit assignment, transition learning, cross-episode comparison.

The claim is not that replay implements one particular optimization algorithm.

Its computational significance is that a fast episodic system can generate additional structured training events for a slower learner.

19.9 Cortical Abstraction as Parameter Sharing

Suppose the hippocampal scaffold contains many episode-specific transitions,

$$G_{h_1}, G_{h_2}, \ldots, G_{h_M}.$$

These transitions may initially be represented separately.

Cortical learning can discover that many of them instantiate the same underlying rule.

For example,

$$\text{Kitchen}_1 \to \text{Hallway}_1$$

and

$$\text{Bedroom}_2 \to \text{Hallway}_2$$

may differ episodically while sharing common structural dynamics.

The cortical learner can replace many local instances by one parameterized operator,

$$G_\theta\left(\widehat{z}_t, a_t\right) \approx \widehat{z}_{t+1}. \tag{19.7}$$

This is parameter sharing:

many episode-specific transitions $\rightarrow$ one reusable transition rule.

The resulting model can apply to states never stored explicitly in the episodic scaffold.

This is the key computational transition from memory of individual instances to reusable structure.

19.10 Learning the Algebra of Composition

Abstraction can go beyond sharing one-step transitions.

Repeated trajectories may reveal relations among operators.

For example,

$$G_a \circ G_b \approx G_c$$

or

$$G_{\text{left}} \circ G_{\text{right}} \approx \text{Id}\,.$$

Such regularities define an algebra over transformations.

The hippocampal scaffold supplies particular trajectories.

Cortex can infer rules that generalize across them.

Thus a useful distinction is

episodic system:	preserves particular paths,
cortical system:	learns reusable relations among paths.

This difference separates memory of instances from a generative model of their structure.

19.11 Consolidation as Compilation

The distinction also suggests a computational interpretation of systems consolidation.

Early in learning, successful prediction or retrieval may depend strongly on hippocampal indices:

$$x_t \longrightarrow h_t \longrightarrow y_t.$$

After repeated replay and cortical learning, the system may acquire a more direct mapping,

$$x_t \longrightarrow \widehat{z}_t \longrightarrow y_t,$$

or

$$\widehat{z}_t \longrightarrow G_\theta(\widehat{z}_t, a_t).$$

The hippocampal scaffold need not disappear physically.

What changes is the degree to which behavior depends on it.

This motivates the computational hypothesis

systems consolidation can be interpreted as compilation: repeated episodic computations are compressed into r

This analogy is compatible with the broad complementary-learning-systems picture but is more specific about the computational transformation [69].

The analogy to compilation distinguishes two forms of representation.

A scaffold may specify a computation by a long path,

$$h_1 \to h_2 \to \cdots \to h_T.$$

A learned cortical rule may implement a shorter reusable operation,

$$\widehat{z} \to G_\theta(\widehat{z}).$$

This is closely related to the compression-of-composition principle developed elsewhere in the book.

Importantly, compilation here means a change in functional dependence and description, not necessarily the physical disappearance of hippocampal activity.

19.12 Episodic and Semantic Memory

The same picture offers a natural interpretation of the distinction between episodic and semantic memory.

Episode-specific indices preserve

$$\text{who}, \qquad \text{where}, \qquad \text{when}, \qquad \text{which particular event}.$$

Cortical abstraction deliberately removes some of those distinctions.

Across multiple visits to the same environment, one may move from

"I left the keys on this counter Tuesday morning"

to

"keys are usually kept on the kitchen counter."

The first depends strongly on instance identity.

The second is a schema.

Thus abstraction gains reuse at the price of episodic specificity.

19.13 Relation to Classical Hippocampal Theories

The scaffold hypothesis is compatible with several influential views of hippocampal function, but it should not be taken as proving that they are all manifestations of one mechanism.

19.13.1 Cognitive maps

Cognitive-map theories emphasize relational structure in spatial navigation [**okeefe1978hippocampus**].

Within the scaffold framework, physical space is one important case of a more general relational graph.

The relevant object is

$$\text{state} + \text{adjacency} + \text{transition}.$$

Nothing requires those states to correspond only to Euclidean position.

19.13.2 Pattern separation

Pattern-separation theories emphasize the need to distinguish similar or overlapping experiences [**oreilly1994hippocampal**, **yassa2011pattern**, 67].

In the present proposal, separation is not the final objective.

It is an implementation strategy supporting rapid storage, reduced interference, and later comparison across experiences.

19.13.3 Indexing theory

Indexing theories describe the hippocampus as linking an index to distributed cortical content [**teyler1986hippocampal**, **teyler2007hippocampal**].

The present framework retains that idea but adds temporal and relational links among indices.

The index is therefore not only an address.

It can participate in a scaffold of transitions.

19.13.4 Complementary learning systems

Complementary learning systems emphasize rapid hippocampal learning and slower cortical learning [69].

The scaffold hypothesis gives that division a particular computational interpretation:

fast system preserves individual examples;	slow system discovers reusable equivalences among them.

19.13.5 Predictive and successor representations

Predictive and successor-representation theories characterize states by their relationships to future states or occupancies.

Such representations can naturally be constructed from repeated transitions through a relational scaffold [**dayan1993improving**, **stachenfeld2017hippocampus**].

This makes successor-like structure compatible with the framework.

It does not imply that every hippocampal representation is fundamentally a successor representation.

19.14 An Empirical Example: Structured Latent Geometry

Entorhinal grid-cell population activity provides an instructive example of low-dimensional structure within a high-dimensional neural population.

Experiments have reported toroidal organization of population activity within grid-cell modules [**gardner2022toroidal**].

The relevant lesson for the present chapter is modest but important:

high-dimensional neural activity can organize around a low-dimensional structured latent manifold.

This observation is compatible with the idea that neural systems can maintain structured latent coordinates useful for navigation and prediction.

It does not establish that grid-cell activity is the hippocampal index h_t used in our abstraction.

Nor does it prove the scaffold hypothesis.

Grid cells are also principally associated with entorhinal circuitry rather than being identified with hippocampal episodic indices.

The result is therefore evidence for the broader possibility of structured latent geometry in the hippocampal–entorhinal navigation system, not a direct test of the proposed scaffold.

19.15 What the Scaffold Hypothesis Predicts

A useful theory should lead to experiments.

The proposal makes several qualitative predictions.

19.15.1 Prediction 1: relationally structured replay should aid abstraction

Suppose two learners receive the same set of episodes.

One receives replay preserving the true temporal and relational order,

$$h_t \to h_{t+1}.$$

The other receives the same episodes in a randomly shuffled order.

If the scaffold is useful because of its relational structure, preserved replay should produce better recovery of shared transition rules than shuffled replay.

19.15.2 Prediction 2: episodic separation and semantic abstraction should have opposite geometry

Early episodic representations should distinguish similar events more strongly than later schema-level representations.

Thus the representation geometry should evolve from

episode-specific separation

toward

cross-episode clustering by shared structure.

19.15.3 Prediction 3: cortical rules should transfer beyond stored episodes

If cortex has learned a reusable operator rather than a collection of stored transitions, the operator should generalize to novel states that were not individually indexed during learning.

Thus transfer to new but structurally similar environments is an important test.

19.15.4 Prediction 4: hippocampal dependence should decrease as cortical predictive structure improves

Across consolidation, one should observe an inverse relationship between

$$\text{dependence on episode-specific hippocampal retrieval}$$

and

$$\text{quality of cortical prediction from shared latent variables.}$$

The prediction concerns functional dependence, not disappearance of hippocampal activity.

19.15.5 Prediction 5: a scaffold should reduce recovery difficulty

Artificial systems offer a direct test.

Compare

$$\text{end-to-end learner from raw sensory sequences}$$

against

$$\text{episodic-index learner} + \text{relational replay} + \text{slow structural learner.}$$

Hold the underlying data fixed.

If the scaffold hypothesis is correct, explicit relational traces should reduce the amount of data or computation required to recover reusable latent dynamics.

19.16 A Direct Artificial Experiment

The hypothesis can be tested without making any anatomical assumptions.

Consider trajectories generated by a latent dynamical system

$$z_{t+1}^{(i)} = F_i\left(z_t^{\mathrm{pa}(i)}, a_t\right), \qquad |\,\mathrm{pa}(i)| \le k,$$

but provide the learner only high-dimensional observations

$$x_t = R(z_t) + \epsilon_t.$$

Compare two systems.

Direct learner. The model receives only

$$(x_t, a_t, x_{t+1})$$

and must discover the latent structure end to end.

Scaffold learner. A fast memory additionally stores episode-specific indices

$$h_t$$

and transition relations

$$(h_t, a_t, h_{t+1}).$$

A slower learner uses these relations to infer a shared latent representation and transition model.

19.16.1 Evaluation

Measure

sample complexity, optimization time, latent-state recovery, transfer, long-horizon prediction.

Because latent decompositions need not be uniquely identifiable, recovery must be evaluated relative to a specified equivalence class.

For example, in a synthetic setting one may regard two recovered representations as equivalent under permutations of interchangeable latent coordinates, or under other explicitly specified invertible reparameterizations that preserve the local dependency structure.

The strongest result would therefore be one of the following:

scaffolded training recovers the target latent dynamics
up to a specified representational equivalence
with substantially lower recovery cost,

or, more generally,

scaffolded training recovers a functionally equivalent compositional model with substantially lower recovery co

This would provide a direct computational demonstration of the chapter's central idea without requiring literal recovery of every generating coordinate.

19.17 What Is Established and What Is Hypothesis

The levels of claim should remain separate.

19.17.1 Established biological observations

The hippocampal formation is strongly implicated in episodic memory and relational/spatial memory; pattern separation has been associated with hippocampal circuitry; hippocampal replay and reactivation occur during sleep and quiet wakefulness; and hippocampal learning interacts with slower cortical learning and consolidation [**okeefe1978hippocampus**, **teyler1986hippocampal**, **wilson1994reactivation**, **yassa2011pattern** , 69].

Entorhinal grid-cell populations can exhibit low-dimensional toroidal population geometry [**gardner2022toroidal**].

These empirical phenomena motivate the computational abstraction.

They do not by themselves establish the scaffold mechanism proposed here.

19.17.2 Computationally plausible interpretation

Fast separation and indexing can preserve individual episodes.

A graph of temporally linked indices can preserve relational information.

Replay can provide additional structured training events to slower learners.

Parameter sharing across many episode-specific transitions can produce more compact and reusable representations.

These statements describe possible computational consequences of the proposed architecture.

19.17.3 Open hypothesis

The stronger claim is that hippocampal scaffolding specifically reduces the recovery difficulty of learning compositionally sparse latent dynamics.

This has not been established in general.

In particular, the present chapter does not claim that

- hippocampal activity literally implements random projection plus TopK;
- sparse graph degree is equivalent to compositional sparsity;
- the hippocampus itself contains the final cortical world model;
- replay performs one specific optimization algorithm;
- cortical abstraction requires hippocampal indexing in every learning problem;
- grid-cell toroidal geometry proves the scaffold hypothesis;
- the cortical latent variables must coincide uniquely with the generating latent coordinates.

The hypothesis is narrower:

fast episodic indexing may expose relational intermediate structure that makes slower abstraction easier to reco

19.18 Relation to World Models

The chapter on world models begins with the problem

$$\text{observation history} \longrightarrow \text{latent state} \longrightarrow \text{predictive dynamics}.$$

The hippocampal scaffold provides one possible intermediate layer:

$$\boxed{\text{observation} \to \text{episodic index} \to \text{relational trace} \to \text{latent world model}.}$$

This suggests a division of labor.

The hippocampal system need not itself construct the final generative model.

It may instead preserve relational evidence from which a slower cortical system can learn such a model.

In this sense the scaffold addresses part of the state-discovery problem: it supplies additional relational variables without solving the full latent representation problem.

19.19 Relation to Sparse Compositionality

Sparse compositionality belongs primarily to the latent dynamics,

$$z_{t+1}^{(i)} = F_i\left(z_t^{\mathrm{pa}(i)}, a_t\right), \qquad |\,\mathrm{pa}(i)| = O(1).$$

The hippocampal contribution is a recovery hypothesis.

It provides intermediate relational observations that may make the constituents

$$F_i$$

easier to discover.

Thus the relation is

world possesses local compositional structure
$\Downarrow$
experience contains episodes of that structure
$\Downarrow$
hippocampus indexes and links episodes
$\Downarrow$
cortex compares and compresses recurring transitions
$\Downarrow$
reusable compositional model.

The hippocampus does not create sparse compositionality.

It may help make it legible.

19.20 Relation to C1, C2, and C3

The proposal can be placed precisely within the framework of the earlier chapters.

C1 concerns the existence of a feasible compositional world model:

$$\boxed{z_{t+1}^{(i)} = F_i(z_t^{\mathrm{pa}(i)}, a_t), \qquad |\,\mathrm{pa}(i)| = O(1).}$$

The hippocampal scaffold does not establish C1.

C2 concerns whether raw observations expose enough information for a learner to discover useful latent variables and constituent functions.

The scaffold can modify that recovery problem by supplying additional relational observations.

Hard C3 would provide the actual intermediate constituent variables and their local dependencies.

The proposed hippocampal scaffold is weaker:

$$\boxed{\text{hippocampal scaffold} \approx \text{C3-like partial relational trace.}}$$

It may reduce the burden on C2 without eliminating it.

Thus

C1:	the compositional latent dynamics exist,
C2:	they must be discovered from observations,
hard C3:	the true intermediate computation is exposed,
hippocampal scaffold:	some relational intermediate structure is exposed.

This placement is important.

The biological hypothesis concerns primarily the *difficulty of recovery*, not the existence of the underlying representation.

19.21 Conclusion

The hippocampus is often described as a memory store.

A more computationally useful possibility is that it also serves as a temporary scaffold for learning.

Its fast dynamics can

separate episodes,	bind content,	index experience,	preserve relational transitions.

The cortex can then perform a different operation:

identify equivalences,	share parameters,	discover reusable operators,	compress repeated composit

This leads to a computational interpretation of consolidation:

episodic scaffold $\rightarrow$ replay $\rightarrow$ shared latent variables $\rightarrow$ compiled cortical rules.

The central proposal is not that the hippocampus itself is a compositionally sparse world model.

Nor is an episodic graph itself equivalent to sparse compositionality.

The proposal is instead that a fast episodic system may preserve relational evidence from which a slower cortical system can more easily recover a reusable latent model.

In the terminology used throughout this book,

the hippocampal scaffold is a candidate biological mechanism for exposing a C3-like partial relational trace.

That makes its proposed role primarily one of *recovery* rather than representation.

The world may already possess sparse compositional structure.

The hippocampus may help the cortex see it.

CHAPTER 20

Mixture of Experts and Spatial Manifolds

Manifolds, compositional sparsity, and mixture-of-experts architectures are deeply intertwined. While sparse compositionality guarantees the existence of localized sub-problems, Mixture-of-Experts (MoE) networks provide the dynamic routing infrastructure needed to navigate them. This chapter maps recent developments in multi-scale approximation theory to modern deep learning engineering, demonstrating that industrial architectures like DeepSeek-V3 are direct structural realizations of geometric entropy minimization.

20.1 The Geometric Primitives of MoE Networks

To understand how sparse gating mechanisms overcome the Curse of Dimensionality, we analyze the structural priors of **low-dimensional manifold support** and **compositional sparsity**. While traditional dense layers force all parameter pathways to evaluate every incoming feature vector, Mixture-of-Experts architectures map inputs to localized subregions.

This process reduces a global, non-linear approximation problem into two separate, simpler subtasks:

1. **A Spatial Assignment Problem:** A gating mechanism or router decomposes the ambient input space into localized chart regions.
2. **A Localized Function Approximation Problem:** Independent expert networks specialize in processing functions bounded within individual coordinate patches.

By decoupling these steps, an MoE network restricts its approximation error to the intrinsic dimension d of the data manifold rather than the ambient dimension D of the input tensor space.

20.2 Manifold Approximation and Theorem 4.8

This structural decomposition is mathematically formalized by Wang and E (2025) through smooth manifold alignment [111]. Let $\mathcal{M}$ represent a compact, d-dimensional smooth manifold embedded in an ambient space $\mathbb{R}^D$, defined by a regular atlas $\mathcal{A} = \{(U_i, \phi_i)\}_{i \in [E]}$, where E matches the number of available expert channels.

Theorem 15 (Manifold Approximation via Shallow MoE [111])**.** *Let $f : \mathcal{M} \to \mathbb{R}$ represent a continuous target function supported on $\mathcal{M}$. For an expert network width $m \geq \Omega\left(E^2\right)$, there exists a depth-2 Mixture-of-Experts network Ψ containing E experts per layer—where each expert is parameterized as a 3-layer dense*

feed-forward network—such that the global approximation error satisfies:

$$\begin{aligned}\|f-\Psi\|_{L_\infty(\mathcal{M})} &\leq \max_{i\in[E]} \mathcal{E}_{3,m}^{\mathrm{FFN}}\left(f|_{U_i}\right)\\ &\leq \max_{i\in[E]} \mathcal{E}_{2,m}^{\mathrm{FNN}}\left(f|_{U_i}\circ\phi_i^{-1}\right)+\max_{i\in[E]}\mathcal{E}_{2,m}^{\mathrm{FFN}}\left(\phi_i\right)\left\|f|_{U_i}\circ\phi_i^{-1}\right\|\\ &\leq \max_{i\in[E]}\tilde{\mathcal{O}}\left(m^{-\frac{\kappa\left(f|_{U_i}\right)}{d}\wedge\frac{1}{2}}\right),\end{aligned} \tag{20.1}$$

where κ denotes the functional Sobolev smoothness class of the target mapping localized within chart U_i.

Equation ((20.1)) shows that the global approximation error splits cleanly into two parts: approximating the low-dimensional target function mapped through local coordinates ($f|_{U_i}\circ\phi_i^{-1}$), and tracking the smooth coordinate transformation charts (ϕ_i). Because both tasks are constrained to the intrinsic dimension d, the MoE architecture bypasses the ambient dimensionality bottleneck.

20.3 Deep MoEs and Exponential Compositional Capacity

When extended to deep architectures, MoEs scale efficiently by leveraging **compositional sparsity** across cascaded layers [74]. Consider a target function f structured as a hierarchical composition of L layers of primitive subfunctions h_i, each bounded by an intrinsic input dimension d^*:

$$f(x)=h_L\circ\cdots\circ h_2\circ h_1(x). \tag{20.2}$$

A deep MoE network featuring depth $\mathcal{O}(L)$ and containing E routed experts per layer can successfully isolate and approximate functions containing up to E^L distinct piecewise structural regions. This represents an exponential increase in task-capacity over standard dense configurations.

For a compositionally sparse target function, a deep MoE Ψ achieves a global uniform convergence bound of:

$$\|f-\Psi\|_{L_\infty}\leq\tilde{\mathcal{O}}\left(L\cdot m^{-\frac{\kappa}{d^*}}\right). \tag{20.3}$$

This rate proves that deep MoE architectures match the foundational goals of the compositional sparsity framework. While standard dense networks rely on static, rigid routing pathways, an MoE uses dynamic gating to choose the optimal computational graph on the fly. This structural mapping is detailed in Table 20.1.

Table 20.1: Structural Alignment Between Compositional Sparsity and MoE Mechanics.

Compositional Sparsity Framework	**Equivalent MoE Implementation**
Constituent Function h_i	Specialized Expert Block (FFN_i)
Sparsity / Local Dependency Bounds	Manifold Support Regions (U_i)
Hierarchical Depth Layering	Cascaded MoE Block Scaling (L)
Dynamic DAG Selector Routing	Non-Linear Gating Mechanisms ($g_{i,t}$)

20.4 Case Study: The DeepSeek-V3 MoE Architecture

To observe these geometric principles in an industrial setting, we analyze the architectural composition and routing equations of the DeepSeek-V3 language model.

20.4.1 Architectural Layering Strategy

DeepSeek-V3 uses 61 transformer blocks, combining dense representation layers with sparse, fine-grained routing blocks:

- **Foundational Dense Fields (Blocks 1–3):** The initial three layers are completely dense. This ensures that every parameter updates for every token, establishing a universal coordinate background before routing begins.
- **Fine-Grained MoE Arrays (Blocks 4–61):** The remaining 58 layers implement specialized routing. Each block contains 1 permanently active *Shared Expert* and 256 fine-grained *Routed Experts*.

20.4.2 The Multi-Expert Routing Equations

For any given input hidden token vector $\mathbf{u}_t \in \mathbb{R}^D$, the final output of the MoE block $\mathbf{h}'_t$ is computed as the direct sum of the unrouted shared infrastructure and the dynamically routed top-K experts:

$$\mathbf{h}'_t = \mathbf{u}_t + \sum_{i=1}^{N_s} \text{FFN}_i^{(s)}(\mathbf{u}_t) + \sum_{j=1}^{N_r} g_{j,t} \cdot \text{FFN}_j^{(r)}(\mathbf{u}_t), \tag{20.4}$$

where $N_s = 1$ is the shared expert count, $N_r = 256$ represents the routed expert pool, and $g_{j,t}$ is the dynamic gating coefficient.

20.4.3 Internal Node Projections: The SwiGLU Block

To maximize approximation capacity, every individual expert within the network is implemented as a three-matrix SwiGLU activation layer rather than a standard two-layer FFN. The output of an expert node is parameterized by input projection matrices W_{in}, gating matrices W_g, and output matrices W_{out}:

$$\text{FFN}_i(\mathbf{u}_t) = \left(\text{Swish}(\mathbf{u}_t W_{g,i}) \odot (\mathbf{u}_t W_{\text{in},i})\right) W_{\text{out},i}. \tag{20.5}$$

20.4.4 Gating and Capacity-Balanced Routing

The routing coefficient $g_{j,t}$ determines the allocation of tokens across the expert array. For each token, the system calculates a set of affinity scores $s_{j,t}$ across all 256 routed experts:

$$s_{j,t} = \text{Sigmoid}(\mathbf{u}_t \cdot \mathbf{e}_j), \tag{20.6}$$

where $\mathbf{e}_j$ is a learned expert centroid vector. The top-K experts ($K = 8$) are selected based on these affinity scores, and their gating weights are normalized via a localized softmax function:

$$g_{j,t} = \begin{cases} \frac{s_{j,t}}{\sum_{m \in \text{Top-K}} s_{m,t}} & \text{if } j \in \text{Top-K}, \\ 0 & \text{otherwise.} \end{cases} \tag{20.7}$$

To prevent parameter collapse during optimization, DeepSeek-V3 uses an *Auxiliary-Loss-Free Load Balancing* approach. Instead of adding a penalty term to the global loss function, it dynamically updates a bias term for each expert based on real-time load metrics. This shifts the routing landscape without distorting the model's primary training objective.

Table 20.2: DeepSeek-V3 MoE Hyperparameter and Execution Specifications.

Architectural Metric	**Design Specification**
Total Computational Layer Depth (L)	61 Total (3 Dense Blocks, 58 Mixed MoE Blocks)
Global Hidden Token Dimension (D)	7168 Dimensions
Internal Expert Projection Space Width (m)	2048 Dimensions
Total Allocation Pool ($N_s + N_r$)	1 Shared Expert + 256 Fine-Grained Routed Experts
Active Execution Paths per Token (K)	1 Shared Expert + 8 Dynamic Routed Experts
Effective Active Mid-Layer Width	$9 \times 2048 = 18,432$ Dimensions

CHAPTER 21

The Architecture of a Compositional Register Machine

Sparse compositionality suggests that complex computation should be implemented by repeatedly applying low-arity operations to intermediate variables. This principle does not uniquely determine an architecture. It does, however, motivate a particular question:

Can a learning system search directly over programs composed of reusable bounded-arity operators, rather than learning one dense input–output map?

This chapter proposes an experimental architecture for studying that question.

We call it a *Compositional Register Machine* (CRM).

The CRM contains four basic ingredients:

explicit working memory
+
a small discrete controller
+
a library of bounded-arity operators
+
hard sparse execution.

At each step the controller chooses a small number of operands from working memory, selects an operator, executes that operator, and writes the result back into memory.

The architecture therefore separates

control flow from local computation.

This distinction resembles classical programmable machines but is introduced here for a different purpose: to make sparse composition an explicit architectural bottleneck.

The CRM should not be confused with a theorem that sparse compositionality automatically solves learning. Bounded arity addresses the representation side of the problem, C1. It does not

guarantee genericity, C2, and it does not guarantee that the correct routing policy can be recovered efficiently.

Indeed, parity remains the simplest warning: parity has a perfect bounded-arity XOR decomposition while its ordinary low-order statistical footprints can vanish.

The real scientific question of the CRM is therefore not whether such a machine can execute algorithms. Chapter 3 already established that associative access, local computation, and writable memory are sufficient for universal computation under appropriate conditions.

The new question is:

> Can the program—the sparse routing and reuse of local operators—itself be learned from small examples and generalized systematically to much larger computations?

That is the hypothesis to be tested.

21.1 From Sparse Compositionality to an Architectural Hypothesis

Suppose a target computation has a DAG representation

$$f = G(g_1, \ldots, g_s),$$

where each constituent g_v depends on at most

$$k = O(1)$$

arguments.

C1 says that such a representation can avoid the exponential dependence on ambient input dimension that appears for generic unstructured functions.

A conventional large neural network can represent such a computation, but it is not required to execute it in an explicitly sparse way.

The CRM asks what happens if bounded arity is imposed physically.

At every computational step,

> only $k = O(1)$ stored variables can enter the active constituent computation.

The global computation must therefore arise through repeated composition.

21.1.1 What this constraint does guarantee

It prevents one local operator call from simultaneously inspecting the entire N-variable input.

For fixed representation dimension, operator width, and numerical precision, the complexity of one local operation is independent of sequence length N.

21.1.2 What it does not guarantee

The constraint does not by itself guarantee:

- that the correct computation is compositionally sparse;

- that useful routing decisions are statistically visible;
- that the controller can be learned efficiently;
- that total execution time is small;
- that the overall machine cannot memorize through its controller or memory;
- that the system generalizes outside its training distribution.

Thus

bounded arity enforces a compositional execution constraint, not genericity or learnability.

21.2 The Abstract Compositional Register Machine

At computational step t, let working memory contain

$$\mathcal{M}_t = \{(v_i, m_i)\}_{i=1}^{L_t}.$$

Here

$$v_i \in \mathcal{V}$$

is the stored value and m_i is metadata describing its computational role.

21.2.1 Working memory

Useful metadata may include

$$m_i = (\text{address}, \text{age}, \text{depth}, \text{availability}, \text{type}, \ldots).$$

The architecture may initially use append-only memory:

$$\mathcal{M}_{t+1} = \mathcal{M}_t \cup \{(v_{\text{new}}, m_{\text{new}})\}.$$

Append-only memory is convenient because previous intermediate variables remain stable.

It is not a fundamental requirement; later we introduce overwrite, deletion, and garbage collection.

21.2.2 Operator library

Let

$$\mathcal{G} = \{g_1, \ldots, g_K\}$$

be a library of local operators.

Every operator has bounded arity:

$$g_k : \mathcal{V}^{a_k} \to \mathcal{V}^{b_k}, \qquad a_k \le a_{\max} = O(1).$$

Examples might include

$$\text{XOR}, \quad \text{AND}, \quad +, \quad -, \quad \times, \quad \text{compare},$$

or learned local neural maps.

21.2.3 Controller

The controller chooses

$$a_t = (i_1, \ldots, i_{a_k}, k, w),$$

where

- $i_1, \ldots, i_{a_k}$ are operand addresses;
- k chooses an operator;
- w specifies the destination or write action.

Execution is then

$$y_t = g_k \left(v_{i_1}, \ldots, v_{i_{a_k}} \right). \tag{21.1}$$

The memory is updated with y_t.

21.2.4 Finite control state

For general algorithms the controller may also maintain a small internal state

$$c_t \in \mathcal{C},$$

with

$$c_{t+1} = U_\theta(c_t, \mathcal{M}_t, a_t, y_t).$$

This permits loops, phases, and finite-state program control.

21.3 Relational Routing Rather Than Independent Scores

A simple first implementation might score each candidate independently:

$$s_i = r_\theta(m_i).$$

Selecting the two largest scores gives

$$\{i^*, j^*\} = \text{topk}(s_1, \ldots, s_{L_t}; 2).$$

This is adequate for simple reductions but is not a general relational controller. Whether i is a useful operand can depend on the choice of j.

21.3.1 Pairwise controller

A more expressive controller scores actions jointly:

$$s_{ijk} = R_\theta \left(c_t, m_i, m_j, \kappa(v_i), \kappa(v_j), k\right), \tag{21.2}$$

where $\kappa(v)$ is an optional compact content key.
The hard action is

$$(i^*, j^*, k^*) = \arg\max_{i,j,k} s_{ijk}.$$

21.3.2 Factorized selection

Explicit pairwise scoring may cost

$$O(L_t^2).$$

A scalable implementation can choose operands autoregressively:

$$i^* = R_\theta^{(1)}(c_t, \mathcal{M}_t),$$

then

$$j^* = R_\theta^{(2)}(c_t, \mathcal{M}_t, i^*).$$

This preserves relational routing without constructing every possible pair simultaneously.

21.4 Bounded Arity and Local Capacity

The operator library provides the architectural realization of sparse compositionality.

Proposition 16 (Local capacity is independent of global sequence length)**.** *Suppose every CRM operator accepts at most k arguments, each represented in a fixed-dimensional state space $\mathcal{V} \subseteq \mathbb{R}^D$, and the operator architecture and numerical precision are fixed independently of input length N.*

Then the number of input degrees of freedom directly available to one operator call is bounded independently of N.

Thus one constituent cannot directly implement an arbitrary lookup table

$$\{0,1\}^N \to \{0,1\}$$

through a single evaluation when N grows.
The global function must instead be realized through repeated calls.
But this does *not* imply that the entire CRM has constant capacity.
Total capacity also depends on

$$\boxed{\text{controller} + \text{memory} + \text{number of steps} + \text{operator library}.}$$

21.5 Execution Universality Is Different from Learning

The architecture naturally raises a Turing-completeness question.

This question should be separated from the harder learning question.

Theorem 16 (Conditional universality of the CRM)**.** *Consider a CRM satisfying the following conditions:*

1. *working memory can grow with computation time;*
2. *the controller has a finite discrete control state;*
3. *memory records possess exact discrete addresses or an equivalent exact addressing mechanism;*
4. *controller decisions may depend on selected stored values;*
5. *the operator library contains a finite universal set of exact local symbol and control operations;*
6. *the machine can read, write, append, and update its control state.*

Then the CRM can simulate a deterministic register machine, and hence a deterministic Turing machine, with at most polynomial overhead.

Proof sketch. Represent the simulated machine's finite control state in c_t and its working tape or registers in CRM memory records.

Each source-machine transition reads a bounded number of symbols or registers, performs one of a finite collection of local operations, writes the result, and updates the finite control state.

Each such transition can therefore be represented by a bounded sequence of CRM actions.

If addressing itself requires search, the resulting overhead depends on the memory-access mechanism, but remains polynomial under ordinary address representations.

□

The theorem is intentionally unsurprising.

Chapter 3 already established the more general point that local update plus writable associative memory is computationally universal.

The new question is not

Can the CRM represent a program?

It is

Can useful CRM programs be recovered efficiently from data?

21.6 Metadata-Only Routing and Oblivious Algorithms

The smallest CRM controller can be deliberately prevented from observing payload values.

Its decision is then

$$a_t = R_\theta(c_t, M_t),$$

where M_t contains only structural metadata.

This has an important consequence.

The resulting algorithm is *data-oblivious*:

its control flow depends on computational structure but not on the numerical or semantic values being processed.

This is sufficient for many useful computations:

- fixed reduction trees;
- FFT-like butterfly graphs;
- sorting networks;
- convolutional scans;
- fixed dynamic-programming schedules.

It is insufficient for algorithms such as:

- ordinary Quicksort;
- conditional program execution;
- value-dependent search;
- linguistic routing based on token meaning.

Thus metadata-only routing is a useful experimental restriction, not a candidate universal controller.

21.7 Content-Dependent Routing

General control flow requires some information about stored values.

The challenge is to provide enough content information to the controller without making the evolutionary search space as large as the complete model.

21.7.1 Compact content keys

Associate each stored value v with a small key

$$\kappa(v) \in \mathbb{R}^d, \qquad d \ll D.$$

The controller then acts on

$$(m_i, \kappa(v_i))$$

rather than the complete high-dimensional value.

21.7.2 Exact discrete keys

The cleanest first experiments should use exact low-dimensional tags.

For example,

$$\kappa(v) \in \{0, 1, \ldots, q-1\}^d.$$

This isolates the control-flow problem from representation learning.

21.7.3 Frozen random projections

For continuous high-dimensional representations, one possible key is

$$\kappa(v) = vW,$$

where

$$W \in \mathbb{R}^{D\times d}$$

is randomly initialized and frozen.
For a finite collection of M vectors, the Johnson–Lindenstrauss lemma implies that for

$$d = O\left(\frac{\log(M/\delta)}{\varepsilon^2}\right),$$

pairwise Euclidean distances can be preserved to relative distortion ε with high probability.
Thus

a frozen random projection can preserve an existing geometric representation;

it does not create semantic geometry that was absent from the original representation.

21.7.4 Freezing the projection does not freeze the representation

Even with fixed W,

$$\kappa_t(v) = v_t W$$

changes if v_t changes.
Therefore a frozen projection does not by itself create a stationary fitness landscape while the operator representations are evolving.
A cleaner protocol separates timescales:

learn/freeze local operators first $\rightarrow$ evolve the controller afterward.

Alternating optimization is also possible, but then nonstationarity should be treated explicitly.

21.8 Why Evolution Is Attractive—but Not Necessary

The controller executes hard discrete actions such as

$$\arg\max \quad \text{or} \quad \text{topk}\,.$$

Ordinary backpropagation cannot differentiate through these hard choices directly.
But this does not imply that evolutionary optimization is the only possibility.
Alternatives include:

- policy-gradient estimators;
- straight-through estimators;

- Gumbel or soft routing relaxations;
- supervised action traces;
- discrete search;
- evolutionary strategies.

Evolution is attractive here for the reason developed in Chapter 17:

the controller can be made extremely small.

If

$$\theta_{\text{router}} \in \mathbb{R}^r$$

with modest r, zeroth-order search operates on the low-dimensional program-control space rather than on all parameters of the computational system.

Thus the evolutionary hypothesis is

hard sparse routing + small controller $\Rightarrow$ a plausible low-dimensional evolutionary search problem.

Whether evolution is superior to policy gradient or differentiable relaxation is an empirical question.

21.9 Differentiable Sparse Routing: Sparsemax and Entmax as Relaxations of Program Search

The CRM has so far been described using hard discrete routing:

$$a_t = (i_t, j_t, k_t) = \arg\max_{i,j,k} R_\theta(i, j, k \mid \mathcal{M}_t, c_t),$$

where the controller chooses two operands and one operator.

This formulation makes the computational graph explicit, but the $\arg\max$ operation is not differentiable. Evolutionary search, policy-gradient estimators, or supervised traces can therefore be used to learn the controller.

A different possibility is to replace the discrete choice temporarily by a differentiable *sparse relaxation*.

The idea is

continuous sparse exploration $\longrightarrow$ support selection $\longrightarrow$ hard discrete program.

Sparsemax and α-entmax provide natural mechanisms for such a relaxation.

21.9.1 Softmax, Sparsemax, and Entmax

Let

$$z = (z_1, \dots, z_K) \in \mathbb{R}^K$$

be routing logits.
Standard softmax produces

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}},$$

so $p_i > 0$ for every finite logit.
This is useful for smooth optimization, but it does not produce literal zeros.

Sparsemax

Sparsemax instead projects the logit vector onto the probability simplex:

$$\text{sparsemax}(z) = \arg\min_{p \in \Delta^{K-1}} \|p - z\|_2^2, \tag{21.3}$$

where

$$\Delta^{K-1} = \left\{ p \in \mathbb{R}^K : p_i \geq 0,\ \sum_i p_i = 1 \right\}.$$

Its solution has the threshold form

$$p_i = \max\{0, z_i - \tau(z)\}. \tag{21.4}$$

Hence some routing probabilities can become exactly zero.

α-entmax

The α-entmax family interpolates between softmax and sparsemax.
It can be written schematically as

$$\text{entmax}_\alpha(z) = \arg\max_{p \in \Delta^{K-1}} \left\{ p^\top z + H_\alpha(p) \right\}, \tag{21.5}$$

where H_α is a Tsallis-type entropy.
The limiting cases include

$$\alpha = 1 \quad \Rightarrow \quad \text{softmax},$$

and

$$\alpha = 2 \quad \Rightarrow \quad \text{sparsemax}.$$

For

$$1 < \alpha \leq 2,$$

entmax can produce exact zeros while retaining a smoother dependence on the active logits than a hard discrete selection.

21.10 What Sparse Routing Does—and Does Not—Mean

Exact zeros are useful, but their interpretation should be precise.

If

$$p_i = 0,$$

then route i does not contribute to that particular forward computation.

This gives a sparse *executed graph*.

It does not by itself imply that the corresponding route is absent from the true target computation.

Thus

$$\boxed{\text{zero learned gate} \neq \text{proof of absence from the target DAG.}}$$

Likewise,

$$\boxed{\text{sparse routing} \neq \text{automatic recovery of compositional structure.}}$$

The support selected by the learner remains an optimization result and can be incorrect.

21.10.1 Fixed structure versus conditional computation

An especially important distinction concerns whether routing depends on the data.

Suppose the structural logits are learned parameters

$$z_{ji}^{(\ell)} = \beta_{ji}^{(\ell)}.$$

Then the resulting support

$$\mathcal{E}^{(\ell)} = \left\{(i,j) : p_{ji}^{(\ell)} > 0\right\}$$

is input independent.

This can be interpreted as a candidate fixed computational graph.

In contrast, suppose

$$z_{ji}^{(\ell)} = R_\theta\left(h_i^{(\ell-1)}, h_j^{(\ell)}\right).$$

Then

$$\mathcal{E}^{(\ell)} = \mathcal{E}^{(\ell)}(x)$$

can change from example to example.

The architecture has learned a *conditional routing policy*, not one fixed DAG.

Both are useful, but they answer different questions:

input-independent gates:	candidate structural graph discovery,
input-dependent gates:	conditional program execution.

For the CRM, the second is ultimately more general because arbitrary algorithms require content-dependent control flow.

21.11 Preserving Bounded Arity

A sparse weighted average is not in general equivalent to a bounded-arity constituent function.

Suppose a child receives

$$\widetilde{h} = \sum_i p_i h_i$$

and computes

$$g(\widetilde{h}).$$

Even if only two coefficients are nonzero, an arbitrary binary function

$$F(h_i, h_j)$$

cannot generally be written as

$$g(p_i h_i + p_j h_j).$$

For example, separate access to both operands may be required to compute

$$h_i h_j, \qquad h_i - h_j, \qquad \mathrm{compare}(h_i, h_j).$$

Therefore the CRM should preserve the selected operands as distinct arguments.

21.11.1 Sparse operand slots

For arity k, introduce k routing distributions

$$p^{(1)}, \ldots, p^{(k)}.$$

Each produces one operand slot:

$$u_r = \sum_i p_i^{(r)} h_i, \qquad r = 1, \ldots, k. \tag{21.6}$$

The local constituent then receives

$$y = g_\phi(u_1, \ldots, u_k). \tag{21.7}$$

As the routing distributions become sparse or one-hot,

$$u_r \to h_{i_r},$$

and the relaxed computation approaches the hard CRM instruction

$$y = g_\phi(h_{i_1}, \ldots, h_{i_k}).$$

Thus

sparse relaxation should soften operand selection, not collapse distinct operands into one average.

21.12 Relaxing Operator Selection

The same idea can be applied to the operator library.

Suppose

$$\mathcal{G} = \{g_1, \dots, g_K\}.$$

Let

$$q = \mathrm{entmax}_\alpha(c)$$

be a sparse distribution over operators.

A relaxed computation can be written

$$y = \sum_{k=1}^{K} q_k g_k(u_1, \dots, u_a). \tag{21.8}$$

During exploration, several operators may receive nonzero weight.

After structural hardening,

$$k^* = \arg\max_k q_k,$$

and execution becomes

$$y = g_{k^*}(h_{i_1}, \dots, h_{i_a}).$$

Again the soft mixture is a training device.

The final CRM program is discrete.

21.13 The Sparsemax Jacobian

Sparsemax illustrates both the attraction and the danger of exact sparse routing.

Let

$$S = \{i : p_i > 0\}$$

be its active support.

Away from support-changing boundaries,

$$\frac{\partial p_i}{\partial z_j} = \begin{cases} \delta_{ij} - \dfrac{1}{|S|}, & i, j \in S, \\ 0, & \text{otherwise}. \end{cases} \tag{21.9}$$

The active block is

$$J_S = I - \frac{1}{|S|}\mathbf{1}\mathbf{1}^\top.$$

Its rank is

$$\boxed{\mathrm{rank}(J_S) = |S| - 1.}$$

Equivalently, its nullity within the active support is one.

21.13.1 The benefit

Once an edge is inactive, it contributes nothing to the current forward computation.

Sparse execution therefore becomes explicit.

21.13.2 The danger

The direct derivative through an inactive coordinate is also zero almost everywhere.

Thus prematurely removing the correct route can make it difficult for ordinary gradient descent to restore that route.

This gives a fundamental exploration–exploitation problem:

early sparsity improves structural commitment but can destroy recoverability.

Entmax with

$$1 < \alpha < 2$$

can make transitions near support boundaries smoother, but exact-zero coordinates still have zero direct derivative away from a boundary.

Thus entmax reduces but does not abolish the support-recovery problem.

21.14 A Continuation Strategy

A natural training strategy is therefore to begin with a more exploratory routing distribution and progressively encourage sparsity.

One possibility is

$$\alpha(t) : 1 \longrightarrow \alpha_{\text{sparse}},$$

where

$$1 < \alpha_{\text{sparse}} \leq 2.$$

Early training is close to dense softmax.

Later training becomes increasingly sparse.

Alternatively one may vary an explicit temperature $T(t)$ or sparsity penalty.

The intended trajectory is

broad exploration $\rightarrow$ sparse candidate support $\rightarrow$ hard program.

There is no theorem that this continuation finds the correct support.

It is an optimization heuristic whose effectiveness should be measured experimentally.

21.15 Hardening the Learned Program

After relaxed training, the support can be converted into a discrete CRM program.

For each operand slot,

$$i_r^* = \arg\max_i p_i^{(r)}.$$

For operator selection,

$$k^* = \arg\max_k q_k.$$

The hardened instruction becomes

$$a_t = (i_1^*, \ldots, i_a^*, k^*).$$

The resulting discrete architecture should then be evaluated *without* the continuous relaxation.

21.15.1 Retraining after hardening

A useful protocol is

train relaxed gates
⇓
extract discrete support
⇓
freeze routing
⇓
retrain only local operators.

This separates two questions:

Did the learner find a useful program?

and

Can the constituents be optimized once the program is fixed?

21.16 Structural Complexity Penalties

Exact-zero activations alone do not guarantee low-complexity programs.

The objective should explicitly penalize program complexity.

Let S_t be the number of active routes at time t.

A relaxed structural objective may take the form

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda_{\text{edge}} \sum_t S_t + \lambda_{\text{step}} T + \lambda_{\text{op}} K_{\text{used}}. \tag{21.10}$$

For a continuous relaxation, S_t can be approximated using a smooth or piecewise-smooth support surrogate.

After hardening, the actual discrete counts should be used.

This connects directly to the MDL objective Equation (21.26).

The purpose is not sparsity for its own sake.

It is to prefer

short reusable programs

subject to solving the task.

21.17 Unknown Structure Carries a Statistical Cost

When the correct computation graph is known, local learning can exploit the corresponding bounded arity directly.

When the graph is unknown, the learner must also identify the structure.

If each of the s constituents chooses at most k parents among at most $d + s$ available preceding variables, the graph description must account for both input variables and earlier internal variables.

With a fixed topological ordering and a canonical ordering of argument slots, a parent set can be indexed using

$$\log_2 \left[\sum_{j=0}^{k} \binom{d+s}{j} \right] = O(k \log(d+s))$$

bits for fixed k.

Encoding all s parent sets therefore costs

$$\boxed{O(sk \log(d+s))}$$

bits.

Explicit ordering of noncommuting operands contributes at most $O(sk \log k)$ additional bits and does not change this leading bound.

Thus

unknown-DAG learning is not statistically equivalent to learning with the correct DAG supplied in advance.

Sparse routing may reduce the structural search space, but it does not make the model-selection cost disappear.

21.18 Differentiable Sparsity Does Not Solve C2

The central conceptual limitation follows directly from the framework of Chapters 6 and H.

Suppose the correct route is i^*.

If the available end-to-end loss provides no detectable statistical preference for i^* over the alternatives, changing softmax into entmax cannot manufacture that information.

Thus

sparse gating changes the parameterization of program search; it does not guarantee that the correct program leaves an informative gradient footprint.

Genericity remains necessary.

This is particularly clear for parity-like targets.

A compositionally sparse solution exists, yet ordinary low-order learning signals can vanish under symmetric distributions.

21.19 Computational Cost

A sparse output distribution does not automatically imply sparse computation.

If all logits

$$z_1, \ldots, z_K$$

must first be computed before entmax determines that only two are active, the routing cost may still be

$$O(K).$$

For pairwise routing it can remain

$$O(K^2)$$

if every candidate pair is explicitly scored.

Therefore one must distinguish

$$\boxed{\text{sparse executed support}}$$

from

$$\boxed{\text{sparse routing computation.}}$$

Practical speedups require implementations that avoid scoring or materializing all candidate routes, for example:

- hierarchical candidate generation;
- approximate nearest-neighbor retrieval;
- factorized operand selection;
- top-k kernels;
- locality constraints;
- coarse-to-fine routing.

21.20 Three Ways to Learn the Same CRM Controller

The CRM provides a clean experimental comparison because the execution architecture can remain fixed while the routing-learning algorithm changes.

21.20.1 Evolutionary routing

Use hard actions throughout:

$$a_t = \arg\max R_\theta.$$

Optimize the small controller by zeroth-order evolutionary search.
Advantages:

- exact discrete execution throughout training;
- no relaxation gap;
- arbitrary nondifferentiable program-level fitness.

Disadvantage:

query complexity grows with controller search dimension.

21.20.2 Policy-gradient routing

Interpret

$$\pi_\theta(a_t \mid s_t)$$

as a discrete policy.
Sample instructions and optimize

$$\mathbb{E}[R]$$

using

$$R\nabla_\theta \log \pi_\theta.$$

Advantages:

- retains hard discrete execution;
- uses gradient information through the policy distribution.

Disadvantage:

long-horizon program rewards may have high variance.

21.20.3 Entmax relaxation

Replace hard choices by sparse continuous routing during training.
Advantages:

- ordinary backpropagation through active routes;
- deterministic training;
- explicit sparsification.

Disadvantages:

- relaxation may solve a different problem from the hardened program;
- early zeroing can trap support;
- the final discrete program may perform worse than its relaxed precursor.

21.21 Relation to C3: Trace Supervision

There is also a useful upper-bound experiment.

If the correct actions

$$a_1^*, \ldots, a_T^*$$

are known, train the controller directly from those traces.

This gives the C3 version of the problem.

The comparison

trace-supervised routing
entmax end-to-end routing
policy-gradient routing
evolutionary routing

quantifies the price of hidden program structure.

If trace supervision succeeds but all end-to-end methods fail, the problem is not representational.

It is a recovery problem.

21.22 Interpretation

Sparsemax and entmax therefore occupy a useful but limited place in the CRM framework.

They do not prove that the underlying target DAG can be recovered.

They do not remove the statistical cost of structural discovery.

They do not guarantee that a sparse learned support is the correct support.

And they do not make the genericity problem disappear.

What they provide is a differentiable bridge between two extremes:

dense continuous routing $\longleftrightarrow$ hard discrete program execution.

This makes them valuable experimental tools for asking a precise question:

Can gradient-based sparse relaxation discover the same reusable, length-general programs that can be found by hard discrete search?

That question belongs naturally inside the CRM research program.

21.23 The Operator Library

A single local module is sufficient for a task such as parity, but general computation requires a vocabulary of operations.

Let

$$\mathcal{G}_{\text{locked}} = \{g_1, \dots, g_K\}$$

denote previously learned operators.

21.23.1 Reusable constituents

For each step,

$$y_t = g_{k_t}(v_{i_t}, v_{j_t}).$$

A task is therefore represented not primarily by new microscopic weights but by a program specifying

which operators are used
+
where they are applied
+
in what order.

This is the architectural analogue of compositional reuse.

21.23.2 Learning a new constituent

Maintain one additional mutable operator

$$g_{\text{new}}.$$

The controller may either reuse an old constituent or invoke the new module.

If no composition of old modules solves the task economically, the system may learn and eventually freeze a new operator.

21.24 Compression of Composition: Compiling Subprograms into New Operators

The architecture described so far assumes a library of reusable local operators

$$\mathcal{G} = \{g_1, \dots, g_K\}.$$

The controller constructs longer computations by composing these operators.

This immediately raises a further question:

What should happen when the same useful composition is discovered and executed repeatedly?

A natural possibility is that the system should eventually promote the composition itself into a new primitive.

This is a form of *compression of composition*.

21.24.1 From a subprogram to a primitive

Suppose the controller repeatedly executes a subprogram

$$\pi = (a_1, a_2, \ldots, a_T),$$

where each action a_t selects operands, applies an operator from $\mathcal{G}$, and writes the result to working memory.

Let F_π denote the function computed by this complete subprogram on its relevant input registers.

Instead of executing the same sequence indefinitely, the machine may learn a new operator

$$g_{\text{new}} \approx F_\pi$$

and enlarge its library:

$$\boxed{\mathcal{G} \longrightarrow \mathcal{G}' = \mathcal{G} \cup \{g_{\text{new}}\}.}$$

A computation that formerly required T explicit program instructions can then be invoked by one higher-level call. Whether this also reduces elementary execution cost depends on how the new primitive is implemented.

Schematically,

$$\boxed{g_{k_T} \circ g_{k_{T-1}} \circ \cdots \circ g_{k_1} \quad \longrightarrow \quad g_{\text{new}}.}$$

This operation is analogous to introducing a new instruction, subroutine, macro, or compiled primitive after discovering a frequently useful computation.

21.24.2 Two different forms of compression

It is useful to distinguish two different benefits.

Program-description compression. Suppose a subprogram π has description length $B(\pi)$ and occurs N times.

Writing the program explicitly at every occurrence costs approximately

$$NB(\pi)$$

bits.

If the subprogram is defined once and subsequently referenced by a short identifier of length B_{call}, the corresponding description cost becomes

$$B(\pi) + NB_{\text{call}}.$$

Thus introducing a reusable macro is advantageous whenever

$$B(\pi) + NB_{\text{call}} < NB(\pi).$$

This is ordinary amortized description-length compression.

It connects directly to the discussion of bits and inductive bias in Chapter 13.

Execution compression. There is a stronger possibility.

The system may learn a direct operator

$$g_{\text{new}} \approx F_\pi$$

whose execution requires substantially fewer computational steps than the original subprogram.

If $T(\pi)$ is the execution cost of the original program and

$$T(g_{\text{new}}) \ll T(\pi),$$

then repeated experience has produced not merely a shorter symbolic name for the computation, but a genuine computational shortcut.

Thus

reuse can compress both description length and execution depth.

21.24.3 Depth collapse for familiar computations

Early in learning, a new problem may require a long sequence of primitive operations:

$$x \to z_1 \to z_2 \to \cdots \to z_T \to y.$$

After sufficient experience, the same effective transformation may be implemented by a shorter path:

$$x \to y.$$

We call this phenomenon *local depth collapse*.

The qualifier "local" is important.

There is no claim that every long computation can be replaced by a constant-depth computation.

Some problems intrinsically require sequential computation.

The claim is only that a repeatedly encountered subcomputation may be compiled into a new primitive when its input–output map admits an efficient shortcut.

21.24.4 Offline versus online compilation

Ordinary neural-network training can already produce computational shortcuts.

A network trained repeatedly on a multistep mapping may eventually represent the resulting input–output function directly in its weights.

Thus fixed-depth neural networks are not incapable of composition compression.

The stronger capability considered here is different:

online compilation: a deployed system discovers a useful subprogram and persistently promotes it into its own operator library.

A standard frozen model does not natively alter its repertoire of primitive operations during inference.

The CRM can, in principle, be extended to do so.

Its operator library becomes time dependent:

$$\mathcal{G}_t = \{g_1, \ldots, g_{K_t}\},$$

with

$$\mathcal{G}_{t+1} = \mathcal{G}_t \cup \{g_{\text{new}}\}$$

when a new reusable computation has been discovered.

21.24.5 When should a new operator be created?

Creating a primitive for every observed composition would be disastrous.

The library would grow without bound and simply memorize individual computations.

A new operator should therefore be introduced only when its expected future reuse compensates for its complexity.

One possible objective is

$$\mathcal{J} = \mathcal{L}_{\text{task}} + \lambda_{\text{new}} B(\mathcal{G}) + \lambda_{\text{call}} T + \lambda_{\text{prog}} B(\pi), \tag{21.11}$$

where

- $B(\mathcal{G})$ measures the description length of the operator library;
- T measures execution cost;
- $B(\pi)$ measures the description length of the controller program;
- λ_{new}, λ_{call}, and λ_{prog} determine the tradeoff between inventing new operators and reusing old ones.

A new primitive is worthwhile only if the reduction in repeated program and execution cost exceeds the cost of representing and learning the new operator.

Schematically,

$$\boxed{\text{invent a new primitive} \quad \Longleftrightarrow \quad \text{future savings} > \text{cost of invention.}}$$

This is the same basic principle that favors reusable subroutines in ordinary programming languages.

21.24.6 Exact and approximate compilation

For discrete algorithmic tasks, one may require exact equivalence:

$$g_{\text{new}}(x) = F_\pi(x)$$

over the intended input domain.

For continuous problems, approximate compilation is more natural:

$$\|g_{\text{new}}(x) - F_\pi(x)\| \leq \varepsilon.$$

Approximation error matters because a compiled primitive may itself be used repeatedly.

If the new operator is inserted into a longer computation, its local error can propagate according to the stability properties stated in Proposition 17 below.

Thus compilation should not be accepted merely because it is shorter.

It must also preserve the behavior of the subprogram to the required accuracy.

21.24.7 Verification and fallback

The original subprogram provides an unusually useful supervisory signal.

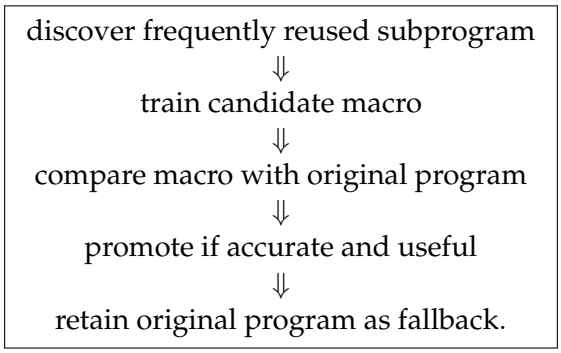
Once a candidate macro has been proposed, the system can generate pairs

$$(x, F_\pi(x))$$

by executing the original program.

The candidate operator can therefore be trained and tested against the computation it is intended to replace.

This suggests a safe compilation procedure:



For exact symbolic tasks, equivalence can sometimes be tested exhaustively over a finite domain or established formally.

For continuous tasks, validation can only establish accuracy on a specified domain or distribution.

21.24.8 A hierarchy of learned programs

Compilation can occur repeatedly.

Primitive operators form programs:

$$g_i \longrightarrow \pi_1.$$

Frequently reused programs become macros:

$$\pi_1 \longrightarrow g_{\text{macro},1}.$$

Macros can then participate in still larger programs,

$$g_{\text{macro},1}, g_{\text{macro},2}, g_j \longrightarrow \pi_2,$$

which may themselves eventually be compiled.

The result is a hierarchy:

$$\boxed{\begin{array}{c}\text{primitive} \to \text{subprogram} \to \text{macro} \\ \to \text{higher-level macro} \to \cdots\end{array}}$$

This is compositionality operating not only within a fixed representation but on the computational vocabulary itself.

21.24.9 Relation to sparse compositionality

Sparse compositionality says that complicated computations can be built from low-arity constituents.

Compilation adds a complementary principle:

$$\boxed{\text{useful compositions can themselves become constituents.}}$$

Thus the constituent library need not remain fixed.

Learning may progressively change the level at which the system represents a problem.

At one stage,

$$f = g_4 \circ g_3 \circ g_2 \circ g_1.$$

At a later stage, the same computation may be represented as

$$f = g_4 \circ g_{\text{macro}}.$$

The effective computational graph has changed because a previously explicit subgraph has become a node.

21.24.10 Relation to the staircase picture

This idea also provides an architectural counterpart to the staircase view of learning developed earlier in the book.

In the staircase picture, learning one constituent creates a useful new coordinate in which the next dependency becomes simpler.

Here the same principle appears computationally:

$$\boxed{\begin{array}{c}\text{solved computation} \to \text{new primitive} \\ \to \text{simpler higher-level search problem.}\end{array}}$$

The new operator changes the representation available to the controller.

The next program is searched for in a richer vocabulary than the one available before the constituent was learned.

21.24.11 A testable prediction

This proposal suggests a stronger criterion for compositional learning than success on a fixed task.

After repeated exposure to a reusable subprogram, an adaptive CRM should eventually require fewer primitive execution steps to solve problems containing that subprogram.

Thus one should measure

$$T_{\text{effective}}(n),$$

the number of primitive calls required after n successful encounters with the same recurring computation.

The compilation hypothesis predicts

$$\boxed{T_{\text{effective}}(n) \downarrow}$$

for familiar subproblems, while performance on genuinely novel compositions may still require long programs.

A decisive experiment would therefore compare:

1. a CRM with a fixed operator library;
2. a CRM allowed to introduce reusable macros;
3. a system that merely caches previous input–output answers.

The crucial distinction is generalization.

A genuine compiled operator should apply successfully to *new* inputs for which the same subcomputation is relevant.

Caching cannot do this.

21.24.12 From learning programs to learning the instruction set

The CRM was introduced as a machine that learns how to compose a library of local operators.

Compression of composition suggests a more ambitious possibility.

The machine may eventually learn not only

$$\boxed{\text{which program to execute,}}$$

but also

$$\boxed{\text{which primitives its future programs should be written in.}}$$

This turns learning into a two-level process:

$$\boxed{\begin{array}{c}\text{learn programs from primitives} \\ \Updownarrow \\ \text{learn new primitives from recurring programs.}\end{array}}$$

Such a mechanism would provide a concrete form of continual abstraction: repeated computation changes the language in which subsequent computation is expressed.

In this sense, intelligence may involve not only composition, but the progressive *compression of composition*.

21.25 Computational Depth and Primitive Granularity

The preceding section described how a composition of operators can become a new primitive. This introduces a distinction between two ways of moving through a computational hierarchy.

One can advance through the execution of a program while keeping its primitive vocabulary fixed. Alternatively, one can change the vocabulary itself, replacing collections of elementary operations by larger subroutines.

We denote these two indices by

$$\boxed{i = \text{execution stage}, \qquad j = \text{primitive granularity}.} \tag{21.12}$$

The distinction is useful for the CRM because learning can change both the program being executed and the operators available for writing future programs. However, changing the description of a computation must not be confused with changing its implementation or learning a new representation.

21.25.1 Depth is relative to a primitive vocabulary

Fix an elementary operator library $\mathcal{G}_0$ and a finite-precision computational model. A program over this library induces a computation DAG $G^{(0)}$ whose nodes implement the specified elementary operations.

This is a *fine-grained realization*, not a uniquely defined "deepest possible" DAG. Equivalent programs can have different depths, and redundant operations can always increase the depth of a realization.

Suppose a consecutive segment of a computation is

$$q = g_m \circ g_{m-1} \circ \cdots \circ g_1. \tag{21.13}$$

Adding a name for q to the library allows the same segment to be written as one macro call. Its displayed depth has decreased relative to the new vocabulary, even if executing the macro still requires all m original operations.

Thus

$$\text{number of macro steps} \neq \text{number of elementary operations}.$$

Any claim that compilation reduces computational cost must specify the implementation and cost of the new primitive.

21.25.2 Coarsening a DAG requires explicit interfaces

For a subgraph S of a computation DAG, define its input boundary as

$$\partial_{\text{in}} S = \{u \notin S : (u, v) \in E \text{ for some } v \in S\}.$$

Its output boundary consists of the values computed inside S that are used outside S, together with any final outputs contained in S.

The subgraph therefore implements an interface map

$$q_S : (z_u)_{u \in \partial_{\text{in}} S} \longmapsto (z_v)_{v \in \partial_{\text{out}} S}. \tag{21.14}$$

A valid regrouping must retain all these externally required values. It must also use a partition for which contracting the modules leaves an acyclic quotient graph; an arbitrary partition of a DAG need not have this property.

Under these conditions, replacing each subgraph by its exact interface map preserves the overall input–output function. To see this, evaluate the modules in topological order. Each receives the same boundary inputs as the original subcomputation and therefore returns the same boundary outputs. Induction over the modules establishes equality of the final outputs.

This is an exact change of computational description. No learning or compression is required.

21.25.3 Bounded gate arity does not guarantee bounded module arity

Coarsening can enlarge the interfaces.

For scalar-valued nodes, let

$$a(S) = |\partial_{\text{in}} S|$$

be the input arity of a module. Even if every elementary gate has fan-in two, $a(S)$ need not be small.

For example, a complete binary subtree containing h internal levels has

$$2^h$$

boundary inputs. Contracting that subtree into one module produces a 2^h-ary interface.

Consequently,

$$\boxed{\begin{array}{c}\text{bounded elementary fan-in}\\ \not\Longrightarrow \text{bounded arity after aggregation.}\end{array}} \tag{21.15}$$

If bounded arity is required at every granularity, it must be imposed explicitly:

$$\max_{S \in \mathcal{P}_j} a(S) \le k,$$

where $\mathcal{P}_j$ is the module partition at level j.

Minimizing the number of edges crossing module boundaries may help find such partitions, but minimizing an edge cut is not itself a guarantee of a fixed arity bound.

For vector-valued registers, one must also distinguish the number of register arguments from their total dimension and bit length. Packaging an entire image into one tensor does not turn an arbitrary image computation into a low-dimensional constituent.

Small interfaces make a module compatible with the local-arity viewpoint. They do not, by themselves, make its internal function simple, well conditioned, or easy to learn.

21.25.4 Representations at execution boundaries

For each chosen granularity j, select a topological execution schedule.

Let

$$Z_{i,j}(x)$$

denote the live interface values at stage i: the values already computed, or supplied as inputs, that are still needed by the remaining computation.

This definition includes values carried across skip connections and other dependencies that need not lie in a single conventional layer.

The associated representation set is

$$E_{i,j} = \{Z_{i,j}(x) : x \in \mathcal{X}\}. \tag{21.16}$$

The word "embedding" can be used informally for these representations, but neither injectivity nor smooth-manifold structure is assumed.

The two indices now have a precise interpretation. Changing i advances execution within a fixed vocabulary. Changing j changes which subcomputations are treated as individual operations.

Crucially, regrouping alone does not create new interface values. For a chain in which coarse stage i ends at elementary stage $t(i, j)$,

$$Z_{i,j}(x) = Z_{t(i,j),0}(x) \tag{21.17}$$

when the coarse operators are exact macros.

The representation at that boundary is unchanged. It has merely been assigned a different position in the program description.

21.25.5 Regrouping, representation learning, and compilation

Three operations should therefore be distinguished.

Regrouping. A collection of operations is given a name and treated as one macro. The original computation and its boundary values are retained. Program notation may become shorter, but execution need not become faster.

Representation learning. The system constructs a new state

$$u_{i,j} = \psi_{i,j}(Z_{i,j}).$$

For this state to replace the original interface for a target F, there must be a suitable continuation $R_{i,j}$ satisfying

$$F(x) = R_{i,j}\left(\psi_{i,j}(Z_{i,j}(x))\right), \tag{21.18}$$

or an approximate version with controlled error on the relevant domain.

A useful compressed interface must preserve the distinctions needed by subsequent computation. Merely grouping nodes does not supply such an encoding, and fitting the training examples does not establish its sufficiency on new inputs.

Compilation. The system implements the interface map q_S by a new operator g_S.

An execution advantage requires

$$C_{\text{exec}}(g_S) < C_{\text{exec}}(q_S),$$

under a common cost model that accounts for internal operations and memory access.

For approximate compilation, one also needs a guarantee such as

$$\sup_{u \in \mathcal{D}_S} \|g_S(u) - q_S(u)\| \le \delta_S,$$

where $\mathcal{D}_S$ covers the boundary inputs on which the replacement will be used. The resulting end-to-end error must be controlled through the stability of the surrounding composition.

Thus a macro, a compressed representation, and a faster learned operator are different objects, even when all three are called an "abstraction."

21.25.6 Granularity is not automatically semantic or spatial scale

A coarser computational vocabulary does not necessarily correspond to larger spatial scale or more abstract semantic content.

A high-level arithmetic subroutine can operate on one pixel. An elementary Boolean circuit can compute a global property of an image. Likewise, a large neural block need not represent a more interpretable concept than a smaller one.

Computational granularity, physical scale, and semantic abstraction can be related in a particular architecture or task family, but the relation is not implied by bounded-fan-in composition.

Nor does choosing a smoother or more elaborate activation function, by itself, establish a higher level of semantic abstraction. The relevant questions concern the function implemented, the information crossing its interface, and its usefulness to subsequent computation.

21.25.7 Choosing a useful granularity

For the CRM, the goal is not to maximize or minimize j.

A very fine vocabulary gives simple local operations but may require long programs and many routing decisions. A very coarse vocabulary can shorten those programs while introducing expensive operators, wide interfaces, or difficult local learning problems.

The useful granularity balances

$$\text{program length}, \quad \text{interface size}, \quad \text{execution cost}, \quad \text{recovery difficulty}.$$

This connects the two-index view to the book's three conditions.

C1 concerns the existence of a feasible compositional realization. It does not select a unique grouping of that realization.

C2 concerns whether the learner can discover useful intermediate functions from the available signals. Renaming a difficult composition as one primitive does not remove this recovery problem.

C3 can expose selected module interfaces as training targets. This may shorten the hidden computation that remains to be learned, but each unexposed module still requires its own learnability and stability conditions.

The distinction can be summarized as

$$\boxed{\begin{array}{l} i : \text{progress through a computation,} \\ j : \text{the vocabulary in which that computation is expressed.} \end{array}} \tag{21.19}$$

A richer vocabulary becomes computationally valuable when its operators do more than rename existing work: they simplify program search, provide useful interfaces, or reduce execution cost.

The next section adds the cross-task criterion. A newly available operator deserves a lasting place in the library when its repeated use justifies the cost of learning, storing, and maintaining it.

21.26 Reusability Across Tasks: When a Composition Should Become a Primitive

The compilation discussion considered a computation that is repeatedly executed within a program:

$$\pi = g_{k_T} \circ \cdots \circ g_{k_1}.$$

If the same subprogram occurs often enough, it may be advantageous to compile it into a new primitive

$$g_{\text{new}} \approx \pi.$$

There is a stronger source of evidence for such compilation.

Suppose the same computation is useful not merely many times within one task, but across many different tasks.

Then reuse provides an additional criterion for deciding which discovered subcomputations deserve to become members of the permanent operator library.

The central principle is

a constituent becomes especially valuable when it compresses many different tasks simultaneously.

21.26.1 Sparse decompositions are not unique

Sparse compositionality establishes that a function may admit a realization as a bounded-arity computational graph.

It does not imply that this realization is unique.

For example,

$$f(x, y) = xy$$

can be represented directly by a multiplication primitive,

$$f(x, y) = g_{\times}(x, y),$$

but also through the polarization identity,

$$xy = \frac{1}{4}\left[(x + y)^2 - (x - y)^2\right]. \tag{21.20}$$

Both are bounded-arity compositions.

Thus even when C1 holds, there may exist many computational vocabularies in which the same target can be expressed.

This creates a selection problem:

which decomposition should the learner preserve as reusable structure?

Single-task accuracy cannot in general answer this question.

21.26.2 Reuse as an additional selection pressure

Let tasks be drawn from a distribution

$$T \sim \rho.$$

Suppose the CRM maintains a shared operator library

$$\mathcal{G} = \{g_1, \ldots, g_K\},$$

and each task T_i is solved by a task-specific program π_i written using operators from $\mathcal{G}$.
A module is valuable when storing it once reduces the total cost of solving many tasks.
This suggests an MDL-like objective

$$\mathcal{J} = \sum_{i=1}^{M} \mathcal{L}_{T_i}(\pi_i; \mathcal{G}) + \lambda_{\mathcal{G}} B(\mathcal{G}) + \lambda_\pi \sum_{i=1}^{M} B(\pi_i \mid \mathcal{G}), \tag{21.21}$$

where

- $\mathcal{L}_{T_i}$ measures task error;
- $B(\mathcal{G})$ is the description cost of the shared operator library;
- $B(\pi_i \mid \mathcal{G})$ is the description length of the task-specific program written in that library.

The objective expresses a simple tradeoff.
A large library can make individual programs short, but the library itself is expensive.
A tiny library is cheap to store but may force every task to reconstruct the same useful computation repeatedly.

21.26.3 When should a recurring computation become a primitive?

Suppose a subprogram q has description length $B(q)$ when expanded in the existing operator library.
Assume it appears in R task programs.
Without compilation, the approximate total description cost is

$$RB(q).$$

If q is promoted to a new primitive g_q, the cost becomes approximately

$$B(g_q) + RB_{\text{call}},$$

where B_{call} is the cost of referring to the new primitive.
Compilation is description-length favorable when

$$B(g_q) + RB_{\text{call}} < RB(q). \tag{21.22}$$

Equivalently,

cost of storing the primitive once
$<$ future savings from repeated reuse.

This provides a quantitative criterion for abstraction.

21.26.4 Reuse can compress execution as well as description

A macro may merely give a short name to a long computation.

More interestingly, learning may discover a direct implementation

$$g_q \approx q$$

whose execution is substantially cheaper than executing the original subprogram.

If $T(q)$ is the cost of the expanded program and

$$T(g_q) \ll T(q),$$

then compilation reduces both

$$\boxed{\text{program description length}}$$

and

$$\boxed{\text{effective computational depth.}}$$

The multi-task objective may therefore include execution cost explicitly:

$$\mathcal{J} = \sum_i \left[\mathcal{L}_{T_i} + \lambda_{\text{step}} T(\pi_i) + \lambda_{\text{prog}} B(\pi_i \mid \mathcal{G})\right] + \lambda_{\text{lib}} B(\mathcal{G}). \tag{21.23}$$

21.26.5 Reusability is relative to the task distribution

A crucial qualification is that reusability is not an absolute property of a computation.

It is defined relative to the environment

$$T \sim \rho.$$

For one task distribution, multiplication may be an extremely useful primitive.

For another, addition and squaring may be reused so frequently that the representation

$$xy = \frac{1}{4}[(x+y)^2 - (x-y)^2]$$

is perfectly economical.

Thus

$$\boxed{\begin{array}{l}\text{reuse identifies useful structure relative to a task distribution,}\\ \text{not an observer-independent “true” decomposition of nature.}\end{array}}$$

Changing ρ can change the optimal operator vocabulary.

This is not a defect.

A computational vocabulary should reflect the regularities of the world in which the machine actually operates.

21.26.6 Reuse reduces ambiguity but does not guarantee identifiability

Multiple operator libraries may achieve the same multi-task loss and the same description length.

For example, two latent representations may be related by an invertible change of coordinates,

$$z' = Az,$$

while supporting equally short programs for every task in the observed family.
Therefore

cross-task reuse can reduce representational ambiguity, but it does not generally establish uniqueness.

Still less does it prove that a discovered module is a causal variable.
Causal identification may require additional information such as

interventions, temporal structure, independence assumptions, changes of environment.

Reusability should therefore be viewed as an inductive bias for *selection and compression*, not as a general theorem of causal identifiability.

21.26.7 From repeated programs to a learned instruction set

The preceding discussion gave the hierarchy

$$\text{primitive} \rightarrow \text{subprogram} \rightarrow \text{macro}.$$

Cross-task reuse extends it:

task-specific subprogram $\rightarrow$ cross-task reusable macro $\rightarrow$ shared primitive.

The operator library can therefore evolve with experience:

$$\mathcal{G}_0 \subseteq \mathcal{G}_1 \subseteq \cdots, \tag{21.24}$$

although in practice obsolete or redundant operators may also be removed.
At stage t, the controller searches for programs in the language defined by $\mathcal{G}_t$.
Once a useful composition is compiled,

$$\mathcal{G}_{t+1} = \mathcal{G}_t \cup \{g_{\text{new}}\},$$

future program search occurs in a richer language.
This changes the difficulty of subsequent learning problems.

21.26.8 Connection to the staircase mechanism

The process closely resembles the staircase picture developed earlier in the book.

In that picture,

$$\text{learn one constituent} \rightarrow \text{create a useful new coordinate}$$
$$\rightarrow \text{expose the next dependency.}$$

Here,

$$\text{solve one recurring subcomputation} \rightarrow \text{promote it to a primitive}$$
$$\rightarrow \text{shorten future programs.}$$

Thus

a solved problem becomes part of the representation used to solve the next problem.

Learning changes not merely the parameters of a fixed hypothesis class but the computational vocabulary in which future hypotheses are expressed.

21.26.9 Relation to cultural accumulation

The same mechanism provides a computational bridge to the chapter on frozen interpreters and cultural evolution.

A reusable procedure can first exist externally as a program p.

If it repeatedly proves useful across tasks, it can be retained in an external library.

At a still slower timescale it might be compiled into the internal operator repertoire.

Schematically,

new computation $\Downarrow$ task-specific program $\Downarrow$ reusable cultural procedure $\Downarrow$ compiled primitive.

This suggests a continuum between external and internal learning rather than a sharp separation between them.

21.26.10 An experimental test

The value of reusability can be tested directly.

Construct a family of tasks

$$T_1, \ldots, T_M$$

that share hidden subcomputations.

Compare two CRM variants:

1. a system with a fixed primitive library;
2. a system allowed to compile recurring subprograms into new primitives.

The decisive evaluation should be performed on *new tasks* sampled from the same structural family.

Measure

$$\text{task accuracy}, \quad \text{program length}, \quad \text{primitive execution count},$$
$$\text{search evaluations}, \quad \text{transfer to unseen tasks}.$$

If genuine reusable abstractions have been discovered, then after learning tasks $T_1, \ldots, T_m$ the cost of solving a new related task should decrease:

$$C(T_{m+1} \mid \mathcal{G}_m) < C(T_{m+1} \mid \mathcal{G}_0). \tag{21.25}$$

The important quantity is therefore not simply performance on the tasks used to construct the library.

It is

reduction in future learning and execution cost.

21.26.11 Controls

Several controls are necessary.

A larger operator library has greater raw capacity, so improvements must be compared at matched description or storage cost.

A compiled primitive must also generalize beyond the exact inputs on which its source subprogram was discovered; otherwise it is merely a cache.

One should therefore compare

memoization:	reuse of stored input–output answers,
macro expansion:	reuse of an unchanged multistep program,
compiled primitive:	a learned implementation of the subcomputation.

Transfer to new inputs distinguishes reusable computation from exact-answer memoization.

Both macro expansion and a compiled implementation can generalize.

A genuine execution-compression claim additionally requires lower elementary cost than expanding the original macro.

21.26.12 Relation to sparse compositionality

Sparse compositionality says that a complicated computation can be represented using low-arity constituents.

Reusability adds a separate criterion:

among possible constituents, prefer those that compress many computations across the relevant task distribution.

These roles should not be confused.

C1 is fundamentally an existence statement:

a feasible bounded-arity representation exists.

Reusability is a selection pressure:

some constituents are worth preserving because they recur.

Neither statement alone guarantees recoverability, uniqueness, or causality.

21.26.13 The resulting picture

The CRM can therefore learn at three levels:

Level 1	:	compose existing primitives into programs,
Level 2	:	compile recurring subprograms into new primitives,
Level 3	:	retain primitives that reduce cost across many tasks.

The third level turns a growing operator library into a record of recurring structure in the task environment.

The central principle is simple:

what should become primitive is what repeatedly makes future computation shorter, cheaper, or easier to learn.

This is not a guarantee that the resulting modules are the unique "true" constituents of the world.

It is a precise computational criterion for deciding which discovered compositions deserve to become part of the machine's reusable vocabulary.

21.27 Parsimony as Description Length

Simply penalizing use of g_{new} is not sufficient.

A program using an existing operator one thousand times should not automatically be preferred to a program that introduces one useful new primitive and calls it twice.

A better objective resembles Minimum Description Length.

Let $T(\pi)$ be the number of operator calls in program π,

$$K_{\text{new}}(\pi)$$

the number of new operators introduced, and

$$B(\pi)$$

the number of bits required to encode the controller or program.

Define

$$\mathcal{J}(\pi) = \mathcal{L}_{\text{task}}(\pi) + \lambda_{\text{new}} K_{\text{new}}(\pi) + \lambda_{\text{call}} T(\pi) + \lambda_{\text{prog}} B(\pi). \tag{21.26}$$

The learner therefore trades off

reuse a longer existing program

against

invent a shorter reusable primitive.

This connects the CRM directly to the bit-complexity framework of Chapter 13.

21.28 Continual Learning: What Freezing Does and Does Not Solve

Freezing learned modules gives an important form of protection.

If g_k is frozen, subsequent learning cannot overwrite its internal input–output mapping.
Thus

operator-level parameter interference is eliminated.

But this does not make catastrophic forgetting impossible.
The controller can forget how or when to invoke the stored primitive.
Old task performance can therefore deteriorate even though every operator remains intact.

21.28.1 Protecting programs as well as operators

Several mechanisms are possible:

- rehearsal on old tasks;
- task-conditioned routing;
- frozen subprograms;
- progressive controllers;
- distillation constraints;
- a separate library of reusable routing programs.

A natural extension is therefore to store not only primitive operators but reusable subprograms:

$$\mathcal{P} = \{p_1, \ldots, p_R\}.$$

The hierarchy becomes

operators $\rightarrow$ subprograms $\rightarrow$ larger programs.

This is compositionality at the level of control as well as computation.

21.29 Stability of Continuous Composition

The original motivation for digital operators is that approximation errors may accumulate during long computations.

The correct behavior depends on the stability of the composed operators.

Suppose the exact update is

$$x_{t+1} = g_t(x_t)$$

and the implemented update is

$$\widetilde{x}_{t+1} = \widetilde{g}_t(\widetilde{x}_t).$$

Assume

$$\|\widetilde{g}_t(x) - g_t(x)\| \leq \varepsilon$$

and every g_t is L-Lipschitz.

Let

$$e_t = \|\widetilde{x}_t - x_t\|.$$

Then

$$e_{t+1} \leq L e_t + \varepsilon.$$

Proposition 17 (Error propagation under repeated local computation)**.** *If $e_0 = 0$, then*

$$e_T \leq \varepsilon \sum_{j=0}^{T-1} L^j.$$

Hence

$$e_T \leq \begin{cases} T\varepsilon, & L = 1, \\ \varepsilon \dfrac{L^T - 1}{L - 1}, & L > 1, \\ \dfrac{\varepsilon}{1 - L}, & 0 \leq L < 1. \end{cases}$$

This yields three qualitatively different regimes:

$L < 1:$	uniformly stable long composition,
$L \approx 1:$	approximately linear accumulation of local error,
$L > 1:$	potential exponential amplification.

Thus continuous computation is not intrinsically shallow.

Its useful depth depends on the stability factor of the constituent composition.

21.30 Digital Operators

Discrete representations offer a different stability regime.

If a finite alphabet is represented with sufficient margin and each operator maps exactly between alphabet elements, one can snap a noisy continuous output back to the correct discrete symbol.

This can eliminate local approximation error while the state remains inside the correct decision basin.

But finite-width digital arithmetic also has limitations:

- overflow;
- truncation;
- rounding;
- increasing bit requirements during arithmetic.

A fixed b-bit word therefore does not provide arbitrary-precision rational computation indefinitely.

Exact unbounded computation requires memory representations whose bit length can grow when necessary.

Thus the correct distinction is not

$$\text{analog fails, digital is exact forever},$$

but

continuous computation requires dynamical stability; digital computation requires explicit precision management.

21.31 Working Memory, Garbage Collection, and Universality

Append-only memory eventually becomes expensive.

If L_t grows indefinitely, the controller must search an ever larger set of records.

The CRM should therefore support deletion or reclamation.

21.31.1 Liveness

A record may be deleted when it is no longer needed by any future computation.

The system may estimate a liveness score

$$q_{t,i}$$

and reclaim low-value records.

21.31.2 Fixed capacity changes computational power

There is an important theoretical distinction.

If active memory is permanently bounded by a constant C and all stored values have finite precision, then the complete machine has only finitely many states.

It is therefore a finite-state machine, not a universal Turing machine.

Hence

fixed bounded memory:	streaming finite-state regime,
extensible memory:	universal-computation regime.

Practical implementations may impose a finite hardware budget while the abstract machine family permits capacity to grow with problem size.

21.32 Sparse Parallel Execution

Sparse composition need not imply purely serial execution.

Suppose several operator calls at step t have no dependency conflicts.

The controller may issue a set

$$\mathcal{A}_t = \{a_t^{(1)}, \ldots, a_t^{(M)}\}.$$

Each action contains operand addresses and an operator.

If their read/write dependencies are compatible, they can execute in parallel.

Thus the relevant computation is a sparse DAG, not necessarily a chain.

$$\boxed{\text{sparse} \neq \text{serial.}}$$

Balanced reductions, sorting networks, convolutions, and many dynamic-programming graphs possess substantial parallel width.

21.33 Relation to Transformers

The CRM should not be presented as a proof that Transformers are architecturally misguided.

Transformers can themselves use:

- sparse attention;
- recurrent computation;
- mixture-of-experts routing;
- local attention;
- external memory;
- learned algorithmic representations.

The difference is one of inductive bias.
A Transformer is a general differentiable sequence architecture.
The CRM exposes the hypothesized sparse-program primitives directly:

memory + hard routing + local operators +explicit program execution.

The empirical question is whether this stronger bias improves systematic algorithmic generalization enough to compensate for its reduced hardware regularity and flexibility.

21.34 Experiment 0: Parity as an Execution Sanity Check

Parity is useful, but only for a limited purpose.
Suppose the operator library contains exact XOR:

$$g_{\mathrm{XOR}}(a, b) = a \oplus b.$$

Because XOR is associative and commutative,

$$(a \oplus b) \oplus c = a \oplus (b \oplus c).$$

Therefore every complete binary reduction tree computes the same answer.
Once XOR is known, parity does not require discovery of a special routing tree.
Thus parity tests

repeated operator reuse, memory bookkeeping, length-independent execution, termination.

It does *not* provide a strong test of learned routing.

21.34.1 Protocol

Train or hard-code XOR on two-bit inputs.
Train the controller only on short lengths such as

$$N \in \{3, 4\}.$$

Test without further learning on

$$N \in \{8, 16, 32, 64, 128\}.$$

Success verifies length-general recurrent composition.

21.35 Experiment 1: Non-Associative Program Execution

The first serious routing experiment should use operations for which parenthesization matters.

For example,

$$a - (b - c) \neq (a - b) - c.$$

Let the library contain exact primitives

$$\mathcal{G} = \{+, -, \times\}.$$

Generate random expression trees such as

$$((x_1 + x_2) \times x_3) - x_4.$$

The controller receives a structured symbolic description and must execute the corresponding tree.

21.35.1 Training

Train only on small expression trees of depth

$$1, 2, 3.$$

21.35.2 Testing

Evaluate zero-shot on depths

$$5, 7, 10$$

and on substantially larger numbers of leaves.

21.35.3 Why this is a real routing test

A wrong operand choice or wrong tree structure changes the answer.

Therefore success requires learning a systematic program rather than merely iterating an associative reduction.

The key metric is

exact execution accuracy as a function of unseen program depth.

21.36 Experiment 2: Structural Routing with Sorting Networks

The next test should require a much richer data-oblivious topology.

Provide a locked compare-and-swap primitive:

$$g_{\text{cs}}(a, b) = (\min(a, b), \max(a, b)).$$

The router must construct a sorting network.

A bitonic or related sorting network contains a highly structured sequence of pairings whose offsets change recursively.

Training on

$$N = 4, 8$$

and testing on

$$N = 16, 32, 64$$

would test whether the controller has learned a length-general routing rule.

This experiment requires a relational controller of the form Equation (21.2); independent operand scores are not sufficient in general.

21.37 Experiment 3: Content-Dependent Conditional Control

Only after structural routing works should the controller be given semantic branching.

21.37.1 Stage A: exact semantic tags

Each record receives a small discrete content key

$$\kappa(v).$$

The task requires different operations for different classes of values.

For example,

$$\kappa(v) = A \quad \Rightarrow \quad g_A,$$

while

$$\kappa(v) = B \quad \Rightarrow \quad g_B.$$

This isolates conditional routing.

21.37.2 Stage B: high-dimensional representations

Replace exact tags with embeddings

$$v \in \mathbb{R}^D.$$

Give the controller only

$$\kappa(v) = vW$$

for a frozen random W.

Vary projected dimension d.

Measure when semantic routing begins to fail.

This directly tests whether a low-dimensional geometric key is sufficient for content-dependent control.

21.38 Experiment 4: Learning a Reusable Operator Library

Construct a curriculum of tasks sharing primitive operations.

For example,

$$T_1 : \text{XOR},$$

$$T_2 : \text{AND},$$

$$T_3 : \text{compositions of XOR and AND},$$

followed by a task requiring one genuinely new constituent.
Compare:

1. retraining one monolithic local module;
2. freezing operators but retraining one shared router;
3. freezing operators plus rehearsal;
4. operator and reusable-subprogram libraries.

Measure

$$\boxed{\begin{array}{c}\text{new-task sample cost} + \text{old-task retention} \\ +\text{program description length.}\end{array}}$$

This directly tests the continual-learning claim.

21.39 Experiment 5: How Should Hard Routing Be Learned?

For the same CRM architecture compare

$$\boxed{\begin{array}{c}\text{evolutionary search} \\ \text{policy gradient} \\ \text{straight-through / sparsemax / entmax relaxation} \\ \text{supervised routing traces.}\end{array}}$$

Keep the controller parameter count fixed.
Measure:

- environment evaluations;
- wall-clock time;
- final accuracy;
- zero-shot length generalization;
- learned program length and support sparsity;

- sensitivity to reward noise;
- controller complexity.

This experiment tests the hypothesis inherited from Chapter 17:

evolution is competitive when the program-search dimension is genuinely small.

21.39.1 A Critical Hardening Test

For entmax, performance must be reported twice:

$$A_{\text{soft}} = \text{accuracy of relaxed routing},$$

and

$$A_{\text{hard}} = \text{accuracy after conversion to discrete instructions}.$$

Define the relaxation gap

$$\Delta_{\text{relax}} = A_{\text{soft}} - A_{\text{hard}}.$$

A small relaxation gap is informative only when the hardened accuracy is itself high. Both versions can otherwise fail equally.

High hardened accuracy with a small gap is evidence that the relaxed solution transfers to discrete instructions.

A large gap shows that this hardening procedure has not preserved the relaxed computation. It does not prove that no other discrete CRM program can solve the task.

Thus

the hardened program, not the relaxed network, is the relevant test of program discovery.

21.40 Experiment 6: Stability of Continuous Operators

Train continuous local operators with known approximation error ε and vary their empirical Lipschitz constants.

Measure end-to-end error versus computational depth T.

The theory predicts three regimes:

$L < 1:$	bounded error,
$L \simeq 1:$	approximately linear error growth,
$L > 1:$	rapid amplification.

This is a more meaningful test than asking for a universal depth at which "analog computation fails."

21.41 Fair Baselines

The CRM should be compared against architectures capable of algorithmic computation, not only against a dense vanilla Transformer.

Useful baselines include:

- standard Transformers;
- recurrent or weight-tied Transformers;
- sparse-attention models;
- recurrent neural networks;
- differentiable-memory architectures;
- neural controllers with explicit stacks or registers.

All models should be compared under controlled parameter count, training compute, and number of task evaluations when possible.

21.42 What Would Support the CRM Hypothesis?

The strongest evidence would not be success on parity.

It would be the simultaneous observation of:

1. training on short programs;
2. exact or near-exact execution on substantially longer programs;
3. transfer through reuse of locked local operators;
4. controller complexity that does not grow with tested sequence length;
5. a large advantage over dense baselines specifically on out-of-distribution algorithmic length generalization.

Especially convincing would be performance of the form

$$\mathrm{Acc}(N) \approx 1$$

for

$$N \gg N_{\mathrm{train}}$$

while the number of learned controller parameters remains fixed.

21.43 What Would Weaken the Hypothesis?

The architectural claim would become much less interesting if:

- the controller memorizes training lengths rather than discovering relational rules;
- evolutionary search becomes intractable even for small routers;
- pairwise routing destroys the computational savings;
- continuous operators accumulate unacceptable errors after modest depth;
- operator freezing prevents useful adaptation;
- standard recurrent or sparse neural architectures exhibit the same length generalization with comparable compute;
- content-dependent routing requires semantic keys nearly as large as the original representations.

These are useful falsification criteria.

21.44 Relation to C1, C2, and C3

The CRM makes the division of labor in the book especially explicit.

21.44.1 C1: sparse compositionality

The architecture assumes that useful computation can be expressed through a bounded-arity graph.

It makes that prior structural.

21.44.2 C2: genericity

C2 is not guaranteed by the architecture.

The learning problem must still expose detectable information telling the controller which program is useful.

If all candidate actions have indistinguishable fitness at the accessible sample scale, evolutionary search or policy gradient will stall.

21.44.3 C3: program traces

If the correct routing sequence is supplied,

$$a_1^*, a_2^*, \ldots, a_T^*,$$

then controller learning becomes ordinary supervised sequence prediction.

This is a strong C3 regime.

Thus the CRM provides an unusually concrete environment for comparing

program discovery from terminal reward

against

program learning from explicit traces.

21.45 Relation to Chapter 17

The CRM also provides a natural experimental test of the effective dimension principle.

The complete computational system may contain many parameters, but the evolutionary search can be restricted to a controller

$$\theta \in \mathbb{R}^r$$

with small r.

The local operator library may be frozen.

Thus

large computational state $\neq$ large evolutionary search dimension.

This is precisely the regime in which zeroth-order optimization should be most plausible.

21.46 Relation to the Associative Turing Machine

Chapter 3 established that associative reads, local updates, and writable memory can support universal computation.

The CRM is not intended to replace that result.

The difference is emphasis.

The ATM asks:

What minimal associative primitives are sufficient for universal computation?

The CRM asks:

What architecture makes sparse program structure explicit enough that the program itself might be learnable and reusable?

The first is principally an expressivity theorem.

The second is an architectural and learning hypothesis.

21.47 Conclusion

Sparse compositionality does not prove that one particular neural architecture is optimal.

It suggests a more modest but testable architectural hypothesis:

complex computations may be easier to learn if the machine is forced to build them from reusable bounded-arity operations.

The Compositional Register Machine makes this hypothesis explicit.

Its state consists of working memory

$$\mathcal{M}_t.$$

Its computation is a sequence of sparse actions

$$a_t = (\text{operands}, \text{operator}, \text{write action}).$$

Its local operators have bounded arity.
Its controller can be kept small enough to make discrete or evolutionary search plausible.
Its operator library permits explicit reuse.
And an MDL-like cost can trade program length against invention of new constituents.
The proposal should not be confused with three stronger claims.
First,

bounded-arity execution does not imply genericity.

Second,

Turing universality of the execution mechanism does not imply that useful programs can be learned.

Third,

hard routing does not uniquely imply evolutionary learning.

Those distinctions turn the architecture into a cleaner research program.
The most informative first experiment is not parity.
Parity is an execution sanity check because XOR is associative.
The critical tests are computations for which the program structure matters:

non-associative expression evaluation,

sorting-network routing,

and ultimately

content-dependent conditional control.

If a controller trained only on small instances learns a compact relational program and executes it correctly on much larger unseen instances, then the CRM would demonstrate something important:

systematic generalization can arise by learning the rules for composing local operators rather than by learning a separate high-dimensional input–output map for every problem size.

That is the central conjecture.
In the language of this book,

sparse compositionality suggests that the natural search space for intelligence may be a space of reusable programs rather than a space of dense functions.

Whether that architectural bias is strong enough to outperform more general neural architectures is not yet known.
But it is a precise question, and the experiments above can answer it.

CHAPTER 22

World Models Before Language

Long before language, animals had to predict motion, remember places, track objects, anticipate other agents, and choose actions whose consequences unfolded over time.

This suggests a simple hypothesis:

much of the machinery required for prediction, planning, memory, and compositional representation predates language.

Language did not create these capacities from nothing. Rather, it provided a new interface for accessing, communicating, recombining, and culturally transmitting representations built by older perceptual, motor, memory, and predictive systems.

We will call this the prelinguistic-world-model hypothesis.

The purpose of this chapter is not to identify one particular neural circuit with a mathematical world model. The biological implementation remains open.

Instead, we ask a computational question:

What minimal architecture must an agent possess in order to model and act in a changing world before language exists?

A natural answer contains three ingredients:

$$\boxed{\textit{latent state} + \textit{predictive dynamics} + \textit{memory}.}$$

Action adds a fourth ingredient by allowing the agent not merely to observe the world, but to intervene in it.

When the underlying environment is governed by local interactions, and the agent's latent coordinates preserve the relevant locality, the latent transition model may itself be compositionally sparse. This is the precise point at which biological world modeling connects to the central structural principle developed earlier in this book.

22.1 A Minimal Definition of a World Model

Let

$$x_t$$

denote the observation available to an agent at time t, and let

$$a_t$$

denote its action.

The environment generally contains hidden variables that are not directly observed. A world model introduces a latent state

$$z_t$$

that summarizes information from the past relevant to prediction and control.

A minimal probabilistic formulation is

$$z_{t+1} \sim p_\theta \left(z_{t+1} \mid z_t, a_t \right), \tag{22.1}$$

$$x_t \sim p_\theta \left(x_t \mid z_t \right). \tag{22.2}$$

The latent state can support several distinct operations:

state estimation:	What is happening now?
prediction:	What is likely to happen next?
simulation:	What would happen under a hypothetical action?
control:	Which action should actually be taken?

These operations should not be conflated.

A system can predict without planning.

It can remember without generating counterfactual futures.

And it can possess a latent state representation without using that representation for explicit search over actions.

22.2 Sparse Dynamics of the Physical World

The strongest connection between world models and sparse compositionality is not the broad claim that "biology is compositional."

It is more specific.

Many physical processes are governed by local interactions.

Suppose the latent state decomposes as

$$z_t = \left(z_t^{(1)}, \ldots, z_t^{(m)} \right).$$

Assume that component i of the next state depends only on a bounded neighborhood:

$$z_{t+1}^{(i)} = F_i \left(z_t^{\mathrm{pa}(i)}, a_t \right), \qquad |\,\mathrm{pa}(i)| \leq k, \tag{22.3}$$

where

$$k \ll m.$$

Lemma 7 (Local latent dynamics imply sparse compositional transition)**.** *If every latent transition satisfies Equation* (22.3) *with bounded parent set*

$$|\operatorname{pa}(i)| \leq k,$$

then the global transition

$$F : (z_t, a_t) \mapsto z_{t+1}$$

has a bounded-fan-in compositional representation.

The statement is nearly definitional, but its consequence is important.
A world model need not learn an arbitrary map

$$\mathbb{R}^m \to \mathbb{R}^m.$$

It can instead learn a collection of local transition maps

$$F_1, \ldots, F_m.$$

Under the regularity and stability assumptions discussed in earlier chapters, the approximation complexity of such a model can depend on the local arity k rather than on the full latent dimension m.

Thus the relevant structural implication is

local environmental interactions
+structure-preserving latent coordinates
⇓
sparse latent transition graph
⇓
potentially tractable world-model learning.

The qualification about the coordinates matters.

A local physical process can become nonlocal under a sufficiently entangled change of variables. Thus environmental locality alone does not guarantee that an arbitrary learned representation will expose a sparse transition graph.

This provides a structural reason why modular internal models may be useful.

It does not imply that evolution must have discovered one unique architecture implementing them.

22.3 State Estimation Is an Inverse Problem

The world model does not directly observe

$$z_t.$$

It receives observations

$$x_{\leq t}$$

and must infer a latent state consistent with them.

Thus world modeling contains an inverse problem:

$$\boxed{x_{\leq t} \longrightarrow z_t.}$$

A generic formulation is

$$q_\phi \left(z_t \mid x_{\leq t}, a_{<t} \right).$$

The state estimate need not reconstruct every detail of the physical world.

It need only preserve the information relevant for predicting future observations, rewards, and consequences of actions.

Thus the ideal latent state is task dependent.

Two agents may construct different internal states from the same environment because the information relevant to their behavior differs.

22.4 Delay Embeddings: When History Determines State

The preceding discussion treats latent-state estimation as an inverse problem:

$$x_{\leq t} \longrightarrow z_t.$$

In general there is no guarantee that the observation history uniquely determines the hidden state.

For an important class of deterministic dynamical systems, however, dynamical-systems theory provides a remarkable positive result.

Takens' embedding theorem shows that, under suitable conditions, a finite sequence of observations can contain enough information to reconstruct the underlying dynamical state.

22.4.1 Takens' theorem

Consider a deterministic dynamical system

$$s_{t+1} = \phi(s_t), \qquad s_t \in M, \tag{22.4}$$

where M is a compact manifold of dimension d and

$$\phi : M \to M$$

is a sufficiently smooth diffeomorphism.

Suppose the system is observed only through a scalar measurement

$$y_t = h(s_t), \qquad h : M \to \mathbb{R}. \tag{22.5}$$

Theorem 17 (Takens' embedding theorem, informal)**.** *For generic smooth choices of the dynamics ϕ and observation function h, the delay-coordinate map*

$$\mathcal{D}(s) = \Big(h(s), h(\phi(s)), h(\phi^2(s)), \ldots, h(\phi^{2d}(s)) \Big) \in \mathbb{R}^{2d+1} \tag{22.6}$$

is an embedding of M into $\mathbb{R}^{2d+1}$.

Thus the delay-coordinate representation preserves the state manifold: distinct states of the original dynamical system correspond to distinct points in delay space, and the inverse map is smooth on the embedded manifold.

For prediction, the more natural representation uses observations from the *past* rather than the future.

Because ϕ is assumed to be a diffeomorphism, its inverse exists. Applying the same result to ϕ^{-1} gives, under the corresponding generic conditions,

$$\mathcal{D}_{-}(s_t) = (y_t, y_{t-1}, \ldots, y_{t-2d}) , \tag{22.7}$$

which is also an embedding.

Consequently,

CHAPTER 23

Can 3D Structure Be Discovered from 2D Experience?

A system can recognize an object from many viewpoints without constructing an explicit three-dimensional model of it. This was one of the central lessons of early view-based theories of object recognition.

But a different question arises when an agent does more than classify static images. Suppose it moves around an object, observes a temporally ordered sequence of views, knows or estimates its own motion, and is required to predict what it will see next.

Will such a learner discover an internal representation with genuine three-dimensional structure?

This chapter proposes a set of experiments designed to answer that question.

The central distinction is

$$\boxed{\textit{view-invariant recognition} \neq \textit{learning a 3D world model.}}$$

The strongest evidence for a learned 3D representation would not be classification accuracy or even novel-view synthesis alone. It would be the emergence of a reusable transformation law for viewpoint, persistence of hidden surfaces through occlusion, and compositional generalization to motions and objects not encountered during training.

23.1 Historical Motivation: Recognition Without Explicit 3D Reconstruction

Early theories of object recognition often assumed that recognizing an object from a novel viewpoint required reconstructing an explicit three-dimensional model and geometrically matching the input image to that model.

An alternative view developed in work on view-based recognition [86, 63, 64].

The basic idea was that recognition could be achieved by storing a limited number of characteristic views and interpolating between their representations.

In this picture,

$$\boxed{\text{recognition across viewpoint}}$$

does not require

explicit reconstruction of Euclidean 3D geometry.

Psychophysical and physiological results showing substantial viewpoint dependence were consistent with such a view-based account.

This observation remains important.

A neural network trained with sufficiently many views can learn the mapping

$$x \longmapsto \text{object identity}$$

while discarding much of the information about pose and geometry.

Thus successful viewpoint invariance by itself tells us very little about whether the representation contains an internal model of three-dimensional structure.

23.2 A Different Question

Consider instead an agent moving around a rigid object.

At time t it observes an image

$$x_t$$

and executes or measures an egomotion

$$u_t.$$

Its experience is therefore not a collection of independent images but a trajectory

$$(x_1, u_1, x_2, u_2, \ldots, x_T).$$

The learner is now asked to predict consequences of its own motion.

The problem changes from

$$x_t \longrightarrow \text{identity}$$

to

$$(x_{\leq t}, u_t) \longrightarrow x_{t+1}.$$

The second problem places stronger demands on the representation.

A view-invariant classifier is encouraged to *remove* information about viewpoint.

A predictive world model must retain precisely the information needed to determine how viewpoint changes.

This leads to the central experimental question:

Can temporal visual experience and self-motion cause three-dimensional structure to emerge as a latent variable

23.3 The Generative Setting

Let an object be represented abstractly by

$$S = (G, A) \in \mathcal{S},$$

where G denotes geometry and A denotes appearance.

We deliberately do not assume that the learner represents G as a mesh, point cloud, voxel array, or any particular explicit 3D data structure.

Let

$$p_t \in SE(3)$$

describe the relative pose between observer and object.

The observed image is generated by

$$x_t = R(S, p_t) + \epsilon_t, \tag{23.1}$$

where R is an unknown rendering process and ϵ_t denotes observation noise.

The observer moves according to

$$p_{t+1} = U(p_t, u_t), \tag{23.2}$$

where u_t represents egomotion.

For rigid motions this can be expressed using the group operation in $SE(3)$.

Schematically,

$$p_{t+1} = u_t p_t$$

or the corresponding convention for composition.

The learner sees

$$x_t, \qquad u_t, \qquad x_{t+1},$$

but is not given

$$S \quad \text{or} \quad p_t.$$

The scientific question is what latent variables emerge from learning this prediction problem.

23.4 Recognition Is Not Evidence for 3D Structure

Suppose an encoder produces

$$z = E_\theta(x)$$

and a classifier predicts

$$\hat{o} = C_\theta(z).$$

Training on many views of the same object can encourage

$$E_\theta(x_{o,p_1}) \approx E_\theta(x_{o,p_2})$$

for different poses p_1 and p_2.
This is desirable for recognition.
But complete invariance actually eliminates pose:

$$z_{o,p_1} \approx z_{o,p_2}.$$

A representation satisfying this condition cannot by itself tell the system how the image should change after a camera rotation.
Thus

invariance discards pose; world modeling must represent pose or its effects.

The first experiment must therefore establish an important negative control:

excellent viewpoint-invariant recognition does not imply recovery of 3D structure.

23.5 The Occlusion Problem

Occlusion makes the distinction especially sharp.
Let

$$P : \mathcal{S} \to \mathcal{X}$$

denote image formation from a particular viewpoint.
Projection is generally many-to-one.
There may exist two distinct object states

$$S_1 \neq S_2$$

such that

$$P(S_1) = P(S_2).$$

For example, two objects can have identical fronts but different backs.
Therefore a single image does not generally determine the complete object.
After a rigid transformation g,

$$P(g \cdot S_1) \neq P(g \cdot S_2)$$

may hold even though

$$P(S_1) = P(S_2).$$

Consequently there cannot, in general, exist a deterministic view-to-view function

$$T_g$$

such that

$$P(g \cdot S) = T_g(P(S))$$

for every possible object S.
The missing information has been removed by projection.
This establishes an important fact:

one 2D view is generally insufficient for predicting all other views.

It does *not*, however, establish that the learner must construct an explicit Euclidean 3D model.
The required hidden state could take many forms:

- an explicit geometric representation;
- an implicit neural field;
- a collection of depth layers;
- object-centered parts and visibility variables;
- a higher-dimensional latent manifold;
- or some other representation sufficient for prediction.

The emergence of 3D structure must therefore be tested, not assumed.

23.6 The 3D-Emergence Hypothesis

We formulate the main hypothesis as follows.

> **3D-Emergence Hypothesis.** When a learner must predict sequences of views generated by rigid self-motion, the combination of temporal continuity, action information, and occlusion will favor a latent representation in which viewpoint acts approximately according to the geometry of $SE(3)$.

The hypothesis does not require that the internal coordinates literally be Cartesian coordinates.
Latent representations are generally identifiable only up to changes of coordinates.
The relevant property is instead the transformation law.
Let

$$z_t = E(x_{\leq t})$$

denote the internal state.
We ask whether there exists a learned action

$$\rho(g) : \mathcal{Z} \to \mathcal{Z}$$

such that

$$z_{t+1} \approx \rho(g_t)(z_t) \tag{23.3}$$

and, more importantly,

$$\rho(g_2 g_1) \approx \rho(g_2) \circ \rho(g_1). \tag{23.4}$$

Thus ρ need not be a linear representation of $SE(3)$.

The essential requirement is that the latent transformations approximately respect the group composition law.

Equation (23.4) is a much stronger signature of three-dimensional structure than simple view reconstruction.

23.7 Four Core Hypotheses

The proposed experiments are organized around four increasingly strong hypotheses.

23.7.1 H1: Recognition does not require a 3D model

A feedforward network trained on many views should be capable of strong view-invariant recognition without learning a manipulable model of three-dimensional structure.

Thus one may observe

$$\text{high recognition accuracy}$$

together with poor performance on

$$\text{pose prediction}, \qquad \text{novel-view synthesis}, \qquad \text{group composition}.$$

This experiment reproduces the essential lesson of classical view-based recognition.

23.7.2 H2: View prediction encourages pose-sensitive latent structure

Train a model on

$$(x_{\le t}, u_t) \longmapsto x_{t+1}. \tag{23.5}$$

Unlike classification, this objective penalizes a representation that throws away all pose information.

The prediction is that latent states trained for Equation (23.5) will exhibit stronger geometric organization than states trained only for object recognition.

23.7.3 H3: Occlusion creates pressure for persistent object state

Consider a trajectory

$$\text{front} \to \text{side} \to \text{back} \to \text{side} \to \text{front}.$$

Information about a surface may disappear entirely from the current image and later become visible again.

A model that operates only on the current image cannot preserve that information.

A persistent world model can.

The prediction is therefore:

prediction of object-specific hidden properties that are not inferable from the current view requires persistent information about those properties.

23.7.4 H4: Genuine geometric structure should compose

Suppose training contains only short rotations such as

$$10^\circ, \qquad 20^\circ, \qquad 30^\circ.$$

A system based mainly on interpolation may perform well near these training views.

A system that has learned a reusable transformation law should be able to compose them:

$$30^\circ + 30^\circ + 30^\circ \to 90^\circ.$$

More generally,

$$\rho(g_3) \circ \rho(g_2) \circ \rho(g_1)$$

should correctly predict transformations far beyond those explicitly experienced during training.

Thus the strongest evidence for a geometric latent model is *compositional extrapolation*.

23.8 What Would Count as Discovering 3D?

No single probe is sufficient.

In particular, low reconstruction error alone does not establish that a model has discovered 3D structure.

We propose several independent signatures.

23.8.1 Novel-view prediction

Given a partial trajectory and a new camera motion g,

$$(z_t, g) \longrightarrow \hat{x}_{t+1}.$$

The model should predict views not present during training.

23.8.2 Pose decodability

A simple probe should recover the relative pose

$$p_t$$

from the latent state.

This is useful evidence but is not sufficient by itself, since pose information can be present without organizing the representation geometrically.

23.8.3 Group-composition consistency

The most important direct test is

$$F(F(z, g_1), g_2) \approx F(z, g_2 g_1). \tag{23.6}$$

Define the composition error

$$\mathcal{E}_{\text{group}} = \mathbb{E}_{z,g_1,g_2}\left[\|F(F(z, g_1), g_2) - F(z, g_2 g_1)\|^2\right]. \tag{23.7}$$

Low error on transformations not encountered during training would provide evidence that the latent dynamics have discovered the composition law of viewpoint changes.

23.8.4 Persistence of occluded structure

The latent state should preserve information about object surfaces when they disappear from view.

A particularly clean test uses pairs of objects satisfying

$$x_0(S_1) = x_0(S_2)$$

but differing on an initially hidden surface.

After the system has observed the distinguishing surface, rotate the object until that surface is hidden again.

The latent state should continue to distinguish

$$S_1 \quad \text{from} \quad S_2.$$

When the surface reappears, the model should predict its identity before it becomes visible.

23.8.5 Cross-object transfer of motion

A physical rotation is the same operation whether applied to a chair, face, car, or unfamiliar synthetic object.

Therefore a strong geometrical representation should reuse the same motion operator across object identities.

We should test whether

$$\rho(g)$$

learned on one set of objects transfers to completely different objects.

The desirable factorization is

$$\boxed{\text{object representation} + \text{shared transformation operator.}}$$

This is stronger evidence than learning a separate transformation law for each object.

23.8.6 Extrapolation beyond trained trajectories

Train only on restricted regions of viewpoint space and test on

$$90^\circ, \qquad 180^\circ, \qquad 360^\circ$$

compositions that were never encountered as complete trajectories.

A representation encoding a genuine transformation law should degrade more gracefully than one based primarily on local view interpolation.

23.9 Experimental Environment

The cleanest initial experiments should use a controlled synthetic environment.

The environment should permit independent manipulation of

object identity, geometry, surface appearance, camera pose, illumination, occlusion.

For every transition, the experimenter knows

$$S, \qquad p_t, \qquad u_t,$$

even though these variables need not be supplied to the learner.

This allows strong latent probes without contaminating the training signal.

The first experiments should deliberately use simple objects whose hidden properties can be controlled exactly.

More natural objects can be introduced only after the basic phenomenon is understood.

23.10 Architectural Baselines

The experiments should compare several qualitatively different ways of solving the task.

23.10.1 Feedforward classifier

The model receives one image and predicts object identity:

$$x_t \to o.$$

This establishes the level of viewpoint invariance achievable without temporal modeling.

23.10.2 Multi-view contrastive encoder

Different views of the same object are explicitly encouraged to have similar embeddings.

This tests whether multi-view alignment alone produces geometric latent structure.

23.10.3 Recurrent or state-space predictor

A persistent state evolves according to

$$z_{t+1} = F_\theta(z_t, x_{t+1}, u_t).$$

This architecture can accumulate information across views and preserve hidden surfaces.

23.10.4 Transformer over view sequences

A Transformer receives the complete observed trajectory

$$(x_1, u_1, \ldots, x_t, u_t)$$

and predicts the next view.

This provides a powerful history-based baseline without imposing a persistent object state.

23.10.5 Explicit 3D model

A model with an explicit geometric representation provides a reference for what is achievable when three-dimensional structure is built into the architecture.

This is not the model whose success is scientifically interesting.

Its role is to establish an upper reference for geometric consistency.

23.10.6 Object-centric latent world model

The model explicitly separates a persistent object representation from a dynamic viewpoint state:

$$z_t = (z^{\mathrm{obj}}, z_t^{\mathrm{pose}}, z_t^{\mathrm{vis}}). \tag{23.8}$$

Only the pose and visibility components change as the observer moves.

This provides a structural inductive-bias baseline without requiring an explicit mesh or point cloud.

23.11 Experiment 1: Classification versus Prediction

Train two models on exactly the same visual experience.

The first minimizes classification loss:

$$\mathcal{L}_{\mathrm{class}} = \ell\left(C(E(x_t)), o\right).$$

The second minimizes future-view prediction:

$$\mathcal{L}_{\mathrm{pred}} = \left\| x_{t+1} - D\left(F(E(x_{\leq t}), u_t)\right)\right\|^2.$$

Both are evaluated on object recognition.

But they are additionally evaluated on

$$\mathcal{E}_{\mathrm{group}}, \qquad \text{pose decoding}, \qquad \text{novel-view synthesis}.$$

The prediction is

classification can produce invariance without geometry, whereas prediction should preserve transformation stru

23.12 Experiment 2: Object Permanence Through Occlusion

Construct pairs of objects that are identical over one range of views and differ only on a hidden surface.

For example,

$$S_A = \text{object with red back},$$

$$S_B = \text{object with blue back}.$$

Initially the two objects generate identical images.
The observer then moves until the diagnostic surface becomes visible.
Later it moves again until that surface is completely occluded.
At this point the current image is once again insufficient to distinguish the objects.
Test whether the latent state retains the distinction.
A useful quantity is the hidden-surface decoding accuracy

$$A_{\text{hidden}}(\Delta t),$$

where Δt is the duration for which the relevant surface has been occluded.
A persistent object model predicts

$$A_{\text{hidden}}(\Delta t) \gg \text{chance}$$

even when the current image contains no corresponding evidence.
This experiment operationalizes object permanence without relying on verbal interpretation.

23.13 Experiment 3: Composition of Rotations

Restrict training to a small set of elementary viewpoint changes

$$\mathcal{G}_{\text{train}} = \{g_1, \dots, g_k\}.$$

Never train directly on certain compositions such as

$$g_3 g_2 g_1.$$

At test time compare

$$F(F(F(z, g_1), g_2), g_3)$$

with

$$F(z, g_3 g_2 g_1).$$

Evaluate both latent-state agreement and rendered-view agreement.

The decisive question is whether errors grow according to ordinary model approximation error or whether performance collapses as soon as the trajectory leaves the range seen in training.

Successful long-range composition would be difficult to explain purely as local interpolation between stored views.

23.14 Experiment 4: Reusing Motion Operators Across Objects

Divide objects into disjoint training and test classes.

Train the latent dynamics only on the training objects.

Freeze the motion operator.

For a previously unseen object, allow the system to infer its object representation but do not modify the learned action of viewpoint transformation.

Then test whether

$$\rho(g)$$

continues to predict the effects of rotation.

This experiment asks whether the learner has discovered

$$\boxed{\text{a law of motion}}$$

rather than

$$\boxed{\text{a collection of object-specific view transitions.}}$$

Reusable transformation operators would constitute a particularly clear example of compositional structure.

23.15 Experiment 5: What Signal Makes 3D Discoverable?

The previous experiments assume that self-motion u_t is available.

This provides a strong supervision signal.

To determine how much it matters, compare three conditions.

Known egomotion. Training examples contain

$$(x_t, u_t, x_{t+1}).$$

Unknown egomotion. The learner receives only

$$(x_t, x_{t+1})$$

and must infer the latent transformation.

Destroyed temporal structure. The same views are presented but their temporal order is randomly permuted.

The marginal image distribution is unchanged, but the dynamical information is destroyed.

The expected ordering is

$$\boxed{\text{known motion} > \text{ordered views without motion} > \text{unordered views}}$$

for emergence of a reusable geometric action.

This experiment has a direct relation to the genericity principle developed earlier in the book.

Self-motion and temporal adjacency may provide the low-complexity statistical footprints that make latent geometry recoverable.

Without them, the representation may still exist, but the learning problem becomes less identifiable.

23.16 Experiment 6: Fast Episodic Binding and Slow Structural Learning

The experiment can also test the hypothesis that fast memory and slow structural learning play complementary roles.

Consider two architectures.

Direct learner. All information from the trajectory must immediately be incorporated into slowly changing model parameters or the current latent state.

Episodic-plus-structural learner. A fast memory stores experience tuples

$$\mathcal{E} = \{(x_t, u_t, x_{t+1})\}_{t=1}^{T}. \tag{23.9}$$

A slower world model learns from both current experience and replay from

$$\mathcal{E}.$$

This is computationally analogous to the hippocampal–cortical separation discussed elsewhere in the book, without requiring a literal one-to-one anatomical identification.

Compare the two systems on:

- number of trajectories required to learn a new object;
- retention of hidden surfaces;
- resistance to catastrophic interference;
- group-composition error;
- cross-object transfer of motion operators;
- generalization to longer trajectories.

If episodic replay accelerates acquisition of a reusable transformation law, it would provide an experimentally tractable example of how memory can scaffold structural learning.

23.17 A Minimal Latent Factorization

A particularly interesting outcome would be spontaneous emergence of a factorization of the form

$$z_t = \left(z^{\text{object}}, z_t^{\text{pose}}, z_t^{\text{visibility}}\right). \tag{23.10}$$

The dynamics would approximately satisfy

$$z_{t+1}^{\text{object}} \approx z_t^{\text{object}}, \tag{23.11}$$

$$z_{t+1}^{\text{pose}} = F_{\text{pose}}\left(z_t^{\text{pose}}, u_t\right), \tag{23.12}$$

$$z_{t+1}^{\text{visibility}} = F_{\text{vis}}\left(z^{\text{object}}, z_{t+1}^{\text{pose}}\right). \tag{23.13}$$

The rendered image would then be generated by

$$\hat{x}_t = D\left(z^{\text{object}}, z_t^{\text{pose}}, z_t^{\text{visibility}}\right). \tag{23.14}$$

This would constitute a sparse compositional model of view generation:

$$\boxed{\text{object} + \text{pose} \rightarrow \text{visibility} \rightarrow \text{image}.}$$

The key experimental question is whether such a factorization appears without being explicitly imposed.

23.18 Connection to Sparse Compositionality

The physical generative process already contains reusable constituents.

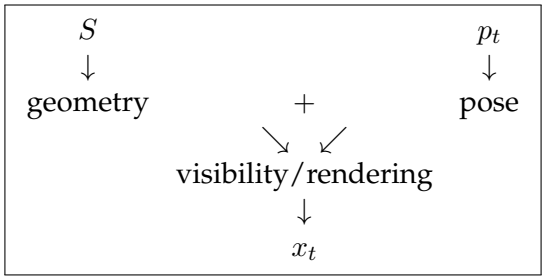
Object geometry remains approximately fixed over an episode.

Camera motion obeys the same transformation law across many objects.

Visibility depends jointly on object structure and pose.

Rendering combines these variables to produce the observation.

Schematically,



Thus the world itself supplies a compositional generative structure.

The experimental question is whether learning discovers reusable constituents corresponding to this structure.

In particular, the viewpoint transformation

$$\rho(g)$$

is especially interesting because it should be reusable across an entire family of objects.

Such reuse is stronger evidence for compositional learning than successful reconstruction of individual training objects.

23.19 Relation to World Models

This experiment is a concrete instance of the general framework developed in Chapter 22.

There the central architecture was

$$\boxed{\text{observation} \rightarrow \text{latent state} \rightarrow \text{predictive dynamics} \rightarrow \text{counterfactual simulation}.}$$

Here all of those components can be studied in a controlled setting.

The observation is

$$x_t.$$

The latent state may contain object and viewpoint information.
The action is

$$u_t.$$

The transition predicts how viewpoint changes.
The decoder predicts the future sensory consequence.
Thus

$$(x_t, u_t) \to x_{t+1}$$

provides a minimal laboratory for studying how a world model can be discovered from experience.

23.20 Relation to Genericity

The experiment also separates representation from recovery.

A useful object-centered representation may exist, yet the training data may not expose enough information to discover it.

Known egomotion provides a particularly strong statistical signal:

$$u_t \to x_{t+1}.$$

Temporal continuity supplies another:

$$x_t \approx x_{t+\Delta t}$$

for sufficiently small motion.
Repeated use of the same transformation across different objects provides a third:

same u $\quad\Longrightarrow\quad$ related latent transformation across many objects.

These are candidate genericity signals.

The ablation experiments can therefore ask whether removing them causes the latent geometry to disappear.

If so, the result would illustrate a central principle of this book:

the existence of a useful representation does not imply that it is recoverable from arbitrary observations.

23.21 Distinguishing Geometry from Memorization

A flexible neural network may obtain low reconstruction error by memorizing training trajectories.

The experimental protocol must therefore prevent simple memorization from being mistaken for discovery of geometry.

Three forms of held-out generalization are essential.

23.21.1 Held-out viewpoints

The model must predict viewpoints that never appeared during training.

23.21.2 Held-out compositions

The model must execute combinations of elementary rotations that were never observed as complete trajectories.

23.21.3 Held-out objects

The learned transformation law must transfer to entirely new object identities.

Only the combination of all three provides strong evidence for a reusable latent geometry.

23.22 Metrics

No single scalar score captures the phenomenon.

The following measurements should be reported separately:

$$A_{\text{rec}} = \text{view-invariant recognition accuracy}, \tag{23.15}$$
$$E_{\text{view}} = \text{novel-view prediction error}, \tag{23.16}$$
$$E_{\text{group}} = \text{group-composition error}, \tag{23.17}$$
$$A_{\text{pose}} = \text{pose decoding accuracy}, \tag{23.18}$$
$$A_{\text{hidden}} = \text{hidden-surface memory accuracy}, \tag{23.19}$$
$$E_{\text{transfer}} = \text{cross-object transformation error}. \tag{23.20}$$

The distinction between these quantities is essential.

For example, a classifier may achieve

$$A_{\text{rec}} \approx 1$$

while having poor

$$E_{\text{group}}$$

and

$$A_{\text{hidden}}.$$

That result would reproduce view invariance without evidence of a world model.

23.23 Possible Outcomes

Several qualitatively different outcomes would be informative.

23.23.1 Outcome A: Recognition without geometry

Feedforward and contrastive systems recognize novel views but fail group-composition and occlusion tests.

This would confirm that invariance alone does not imply a 3D model.

23.23.2 Outcome B: Prediction produces geometry

Action-conditioned predictive models develop low group-composition error, retain hidden surfaces, and transfer motion operators across objects.

This would support the 3D-emergence hypothesis.

23.23.3 Outcome C: Geometry requires explicit architectural bias

Only architectures supplied with object-centered or explicitly three-dimensional structure pass the strongest tests.

This would suggest that the environmental signal alone is insufficient for efficient recovery.

23.23.4 Outcome D: A non-Euclidean latent solves the task

A model succeeds on prediction, occlusion, and compositional generalization while its internal representation does not correspond simply to explicit Euclidean 3D coordinates.

This would be especially interesting.

It would show that the important discovery is not necessarily a literal 3D reconstruction, but a representation that implements the correct transformation structure.

23.24 Falsification Criteria

The proposal should be considered falsified in its strongest form if

- predictive training produces no greater geometric organization than classification training;
- group-composition consistency does not improve with temporal experience or known ego-motion;
- latent motion operators fail completely to transfer across object identity;
- or successful prediction can be fully explained by local interpolation over memorized views.

Conversely, the most compelling positive result would be

training only on 2D view sequences and self-motion
$\Downarrow$
emergence of a reusable latent action of $SE(3)$
$\Downarrow$
persistence of invisible surfaces
$\Downarrow$
composition of transformations far outside training
$\Downarrow$
transfer of the same transformation law to new objects.

23.25 From the 1990 Question to the Present One

The historical view-based theory asked whether three-dimensional reconstruction was *necessary* for recognition.

The answer was largely negative:

recognition can often be achieved by interpolation among views.

The present proposal asks a different question:

when an agent must predict the consequences of its own motion, will a representation of three-dimensional stru

This distinction reconciles the two perspectives.

A visual system may use view-based mechanisms for rapid recognition while also learning a richer latent world model from temporally structured experience.

Recognition and world modeling need not use identical representations.

23.26 Conclusion

A system that recognizes an object from many viewpoints has not necessarily learned a three-dimensional model.

The stronger test is whether it has discovered the transformation laws that generate those views.

Sequential visual experience provides precisely the additional information needed to study this question.

An agent moving around an object observes

$$(x_t, u_t, x_{t+1}),$$

where the same physical motion law is repeatedly expressed across many objects and viewpoints.

This creates the possibility that learning may discover a latent factorization into

$$\text{persistent object state}, \quad \text{viewpoint}, \quad \text{visibility}, \quad \text{rendered observation}.$$

The decisive evidence would not be reconstruction accuracy alone.

It would be

$$\boxed{\text{composition} + \text{occlusion memory} + \text{extrapolation} + \text{cross-object reuse}.}$$

If a learner exposed only to two-dimensional image sequences and self-motion spontaneously develops a reusable latent action satisfying

$$\rho(g_2 g_1) \approx \rho(g_2) \circ \rho(g_1),$$

retains information about surfaces while they are invisible, and applies the same transformation law to novel objects, then it would be difficult to regard its representation as merely a collection of interpolated views.

It would instead provide a concrete example of a central idea developed throughout this book:

structured interaction with the world can cause hidden compositional structure to become discoverable from ex

CHAPTER 24

Interlude: Most Real Numbers Do Not Exist

Kronecker famously remarked that "God made the integers; all else is the work of man." Whatever one thinks of the philosophy, there is a precise mathematical fact behind a related observation.

The classical continuum

$$\mathbb{R}$$

contains uncountably many points. Yet there are only countably many finite descriptions, only countably many algorithms, and only finitely many outcomes that can be distinguished by any fixed finite-precision device.

Thus almost every real number has no finite name, no algorithm that computes its digits, and cannot be singled out by any finite observation.

In this operational sense—not in the set-theoretic sense—most real numbers do not exist for computation or experiment.

The continuum remains an extraordinarily useful mathematical idealization. The point of this interlude is narrower: the natural objects of physical and digital computation are finite descriptions and finite-precision approximations, not arbitrary exact elements of $\mathbb{R}$.

24.1 Several Different Notions of Existence

The word "exist" can mean several different things.

A real number may exist:

- **mathematically**: it is an element of the set $\mathbb{R}$;
- **definitionally**: it can be uniquely specified by a finite symbolic description in some fixed formal language;
- **computationally**: an algorithm can approximate it to arbitrary requested precision;
- **operationally**: an experiment or machine can produce finite information that constrains its value;
- **at fixed precision**: it is one of finitely many numerical states available to a particular device.

These notions are not equivalent.
The main hierarchy relevant here is

$$\boxed{\mathbb{R} \supset \{\text{finitely definable reals}\} \supset \{\text{computable reals}\} \supset \{\text{reals represented at any fixed finite precision}\}.}$$

The first set is uncountable.
The next two are countable.
The last one is finite for any fixed machine and fixed precision.

24.2 The Continuum Is Much Larger Than Finite Description

The cardinality of the natural numbers is

$$|\mathbb{N}| = \aleph_0,$$

while

$$|\mathbb{R}| = 2^{\aleph_0}.$$

Thus the real line is uncountable.
By contrast, the set of all finite strings over any finite alphabet is countable:

$$\bigcup_{n=1}^{\infty} \{0,1\}^n$$

is countable.
Therefore any collection of objects that can each be uniquely specified by a finite bit string is at most countable.
This simple cardinality observation already creates an enormous gap:

$$\boxed{\text{finite descriptions} \ll \text{the mathematical continuum.}}$$

24.3 Algebraic and Transcendental Numbers

A familiar first example is the distinction between algebraic and transcendental numbers.

Definition 8 (Algebraic number)**.** *A real number x is algebraic if it is a root of a nonzero polynomial*

$$p(t) \in \mathbb{Z}[t].$$

Otherwise it is transcendental.

Proposition 18. *The set of algebraic real numbers is countable.*

Proof. There are countably many polynomials with integer coefficients.
Each nonzero polynomial has finitely many roots.
A countable union of finite sets is countable.

□

Consequently the algebraic numbers have Lebesgue measure zero.
Almost every real number is therefore transcendental.
This already shows that familiar constants such as

$$\sqrt{2}, \qquad \pi, \qquad e$$

occupy an exceptionally small part of the real line.
But transcendence is not the deepest distinction.
The more relevant distinction for computation is computability.

24.4 Computable Real Numbers

Definition 9 (Computable real)**.** *A real number x is computable if there exists a Turing machine which, given*

$$n \in \mathbb{N},$$

outputs a rational number q_n satisfying

$$|x - q_n| < 2^{-n}.$$

Thus a computable real can be approximated to any requested number of binary digits by a finite algorithm.

Proposition 19. *The set of computable real numbers is countable.*

Proof. There are only countably many Turing machines.
Each machine computes at most one real number under the definition above.
Therefore there are at most countably many computable reals.

□

Hence

$$|\mathbb{R}_{\text{comp}}| = \aleph_0,$$

while

$$|\mathbb{R}| = 2^{\aleph_0}.$$

In particular,

$$\boxed{\lambda\left(\mathbb{R}_{\text{comp}}\right) = 0,}$$

where λ denotes Lebesgue measure.
Thus, in the measure-theoretic sense,

$$\boxed{\text{almost every real number is uncomputable.}}$$

Numbers such as

$$\pi, \qquad e, \qquad \sqrt{2}$$

are therefore extremely special.
They are not merely mathematically distinguished; they possess finite algorithms that generate arbitrarily accurate approximations.

24.5 Definability Is Not the Same as Computability

There is an important distinction between a real number that can be *defined* and one that can be *computed*.

Fix a countable formal language capable of referring to real numbers, and consider parameter-free finite formulas that uniquely specify one real.

There are only countably many such formulas.

Proposition 20. *In any fixed countable formal language, at most countably many real numbers can be uniquely specified by finite formulas.*

Proof. Finite formulas are finite strings over a countable alphabet.

There are only countably many such strings.

Each uniquely defining formula specifies at most one real.

□

Thus the finitely definable reals form a countable set.

However,

$$\boxed{\text{computable} \subsetneq \text{definable.}}$$

There are finitely definable real numbers that are not computable.

A standard example is Chaitin's halting probability

$$\Omega_U$$

associated with a fixed universal prefix-free Turing machine U.

Once U is specified, Ω_U has a finite mathematical definition.

Yet its binary expansion is not computable.

Thus

$$\boxed{\text{finite specification} \not\Longrightarrow \text{effective generation of the digits.}}$$

24.6 Algorithmic Randomness

The size of the gap between computable and arbitrary real numbers becomes even clearer through Kolmogorov complexity.

Let

$$x_{1:n}$$

denote the first n bits of the binary expansion of a real number x.

Let

$$K(x_{1:n})$$

be the prefix-free Kolmogorov complexity of this bit string.

A real is algorithmically random, in the usual Martin–Löf sense, if its initial segments are incompressible up to an additive constant:

$$K(x_{1:n}) \geq n - c$$

for some constant c and all n.
The set of algorithmically random reals has Lebesgue measure one.
Therefore

almost every real has essentially no shorter algorithmic description than writing down its digits themselves.

Such reals are necessarily uncomputable.
But one must not confuse randomness with undefinability.
Some algorithmically random reals are finitely definable.
For example, appropriate versions of Chaitin's Ω are both algorithmically random and definable.
Thus the correct relations are subtle:

$$\boxed{\text{random} \Longrightarrow \text{uncomputable},}$$

but not

$$\text{random} \Longrightarrow \text{undefinable}.$$

24.7 Finite Measurement Gives Finite Information

The connection to physics does not require the claim that the exact parameters of nature must themselves be computable real numbers.
A much weaker statement is sufficient.
Consider an experiment whose recorded output belongs to a finite set

$$\mathcal{Y} = \{y_1, \ldots, y_N\}.$$

One observation can convey at most

$$\log_2 N$$

bits of information.
Thus:

Proposition 21 (Finite-information principle)**.** *A measurement procedure with finitely many distinguishable recorded outcomes can convey only finitely many bits about an underlying real-valued quantity.*

This statement is almost tautological, but its consequence is important.
Suppose an experiment reports

$$x = 1.3724 \pm 0.0001.$$

The observation does not identify one real number.
It identifies an interval

$$x \in [1.3723, 1.3725],$$

which contains uncountably many real numbers.
Thus

finite measurement does not select an exact point of the continuum.

The exact underlying physical quantity, if one chooses to model it as a real number, could mathematically be computable or uncomputable.

The finite experiment does not decide the issue.

24.8 Finite Precision

A digital machine storing a numerical quantity using b bits has at most

$$2^b$$

distinct bit patterns available for that quantity.

Thus at any fixed precision the state space is finite.

Allowing arbitrary but finite precision gives

$$\bigcup_{b=1}^{\infty} \{0,1\}^b,$$

which is still only countable.

This gives a simple operational hierarchy:

fixed precision
$\Downarrow$
finite set of numerical states

arbitrary finite precision
$\Downarrow$
countable set of finite descriptions

exact mathematical continuum
$\Downarrow$
uncountable set.

The passage from finite computation to exact real arithmetic is therefore not quantitatively small.

It changes the cardinality of the available state space.

24.9 Infinite Precision Can Hide Infinite Information

This distinction becomes especially important in analog and recurrent computation.

Consider an infinite binary sequence

$$b_1, b_2, b_3, \ldots, \qquad b_j \in \{0,1\}.$$

It can be encoded into a single real number, for example,

$$x = \sum_{j=1}^{\infty} 2b_j 3^{-j}. \tag{24.1}$$

This real lies in the Cantor set.

Mathematically, the entire infinite sequence is contained in the exact value of one scalar x.

Thus a fixed-dimensional continuous state can, under exact real arithmetic, encode an arbitrarily long discrete memory.

But extracting progressively later symbols requires progressively finer precision.

To resolve the n-th ternary digit requires precision on the order of

$$3^{-n}.$$

Equivalently, the number of reliable bits of precision must grow linearly with n.

Hence

$$\boxed{\text{fixed dimension} + \text{unbounded precision}}$$

can conceal

$$\boxed{\text{unbounded information.}}$$

This is the basic mechanism behind mathematical constructions in which fixed-dimensional analog or recurrent systems simulate computations with unbounded memory.

24.10 Exact-Real Universality and Physical Resources

The preceding observation explains an apparent paradox encountered in the study of recurrent networks.

A recurrent system may have fixed state dimension,

$$h_t \in \mathbb{R}^d,$$

yet idealized exact-real models can possess extraordinary computational power.

There is no contradiction.

The unbounded resource is hidden in the precision of the coordinates.

A state coordinate can encode

$$\text{one bit}, \quad \text{one thousand bits}, \quad \text{or infinitely many bits},$$

if exact access to its real-valued expansion is permitted.

Thus one must distinguish

$$\boxed{\text{dimension of the state space}}$$

from

$$\boxed{\text{information required to specify a point in that space.}}$$

A vector in

$$\mathbb{R}^{10}$$

is only ten-dimensional geometrically.

But specifying an arbitrary point exactly can require infinitely many bits.

This is why exact-real computational models should not automatically be identified with finite-resource computation.

24.11 Why the Bit Model Matters

The theory of efficient computation developed elsewhere in this book therefore uses finite descriptions.

Suppose

$$f : [0,1]^d \to \mathbb{R}^m.$$

The computational question is not whether some abstract exact real

$$f(x)$$

exists.

It is whether one can compute an approximation

$$\widehat{f}(x)$$

satisfying

$$\|f(x) - \widehat{f}(x)\| \leq \varepsilon$$

using resources polynomial in

$$n + \log \frac{1}{\varepsilon},$$

where n denotes the input description length.

If

$$\varepsilon = 2^{-p},$$

then

$$p = \log_2 \frac{1}{\varepsilon}$$

is essentially the number of requested output bits.

Thus the relevant computational question is:

how many resources are required to produce p reliable bits of the answer?

This is a physically and algorithmically meaningful question.

Exact equality between arbitrary real numbers generally is not.

24.12 A Modulus of Continuity as a Precision Budget

For real-valued computation, input precision and output precision are linked.

Suppose

$$f : [0,1]^d \to \mathbb{R}$$

has modulus of continuity ω_f, so that

$$\|x - y\| \leq \delta \quad \Longrightarrow \quad |f(x) - f(y)| \leq \omega_f(\delta).$$

To compute $f(x)$ to accuracy ε, it suffices to know x to a precision δ satisfying

$$\omega_f(\delta) \leq \varepsilon.$$

Thus the amount of input information needed is itself part of the computational complexity.

For efficiently computable physical functions, one expects this precision budget to grow at most polynomially with the requested number of output bits.

This is the operational role of the polynomial-modulus assumption used elsewhere in this book.

24.13 Description Length and Learning

The same distinction appears in statistical learning.

A learned model stored digitally has a finite description:

$$\theta \longrightarrow \text{finite bit string.}$$

Even if we write

$$\theta \in \mathbb{R}^P,$$

the parameters actually used by a digital implementation are represented to finite precision.

Thus a trained network is not literally one arbitrary point drawn from the uncountable space

$$\mathbb{R}^P.$$

It is one member of a finite or countable collection of finite descriptions.

This connects naturally to the description-length viewpoint developed elsewhere in the book.

At precision ε, what matters is not an exact mathematical parameter vector but the number of bits required to specify a model whose behavior is accurate to that precision.

Schematically,

effective hypothesis complexity $\sim$ number of bits required to describe the useful approximation.

24.14 The Continuum Remains Useful

None of this argues against real analysis.

The continuum is one of the most successful idealizations in mathematics and physics.

Differential equations, probability distributions, Fourier analysis, functional analysis, and modern learning theory all rely on it.

The point is not that

$$\mathbb{R}$$

should be removed from mathematics.

The point is that mathematical existence should not be confused with operational accessibility.

A continuum model may be the simplest and most powerful symbolic language for describing a system even when every actual observation of that system contains only finitely many bits.

Thus

continuum mathematics $\neq$ infinite physical information.

24.15 What "Most Real Numbers Do Not Exist" Means

We can now state the intended claim precisely.

There are uncountably many real numbers:

$$|\mathbb{R}| = 2^{\aleph_0}.$$

But there are only countably many finite descriptions:

$$|\{\text{finite descriptions}\}| = \aleph_0,$$

only countably many computable reals,

$$|\mathbb{R}_{\text{comp}}| = \aleph_0,$$

and only finitely many distinguishable values available to any fixed finite-precision device.

Therefore:

almost every real number has no finite description and no algorithm that computes its digits.

Moreover, no finite experiment can single out an arbitrary exact real number from the continuum.

This does *not* mean that the remaining reals fail to exist in classical mathematics.

They do.

The claim is operational:

almost every real exists only as an element of the mathematical continuum, not as a finitely accessible computat

In that restricted but precise sense,

most real numbers do not exist for computation or experiment.

24.16 Conclusion

The real line contains far more objects than can ever be generated by algorithms or identified by finite descriptions.

The hierarchy is

all mathematical reals
$\supset$
finitely definable reals
$\supset$
computable reals
$\supset$
numbers represented at fixed finite precision.

The first set is uncountable.

The next two are countable.

The last is finite for a fixed device.

This distinction matters for theories of intelligence and computation.

Exact real arithmetic can hide unbounded information inside a fixed-dimensional state.

Finite-resource computation cannot.

Therefore the natural theory of practical computation is not based on access to arbitrary exact reals but on finite descriptions whose precision can grow as resources grow.

The appropriate question is not

what exact real number does the machine contain?

but

how many bits of information can the machine represent, compute, and recover with feasible resources?

Real analysis remains indispensable mathematics.

But for computation, learning, and experiment,

finite information is the operational currency.

The continuum is a powerful mathematical idealization.

It should not be mistaken for an infinite reservoir of physically available information.

CHAPTER 25

Finding the 1983 Circuit in the Connectome: the Tale of a SuperNeuron

When a fly is flying forward or turning, the entire visual world sweeps across its eyes, generating massive, wide-field motion signals (optic flow). If the fly relies solely on local elementary motion detectors (EMDs like T4/T5), those detectors will fire wildly just from the fly's own movement. How, then, does the fly spot a small, physically moving object—like a predator, a mate, or a target—against a background that is also moving across its retina? Forty-five years ago Reichardt, Poggio and Hausen proposed, on the basis of behavioral experiments on the fly, a neural circuit for the detection of relative motion. Today the fly connectome – interrogated by agentic AI – makes it possible to check the validity of the model.

25.1 Introduction

The computation of relative motion—the ability to discriminate a moving figure against a moving background—is a fundamental operation in visual processing. It uncovers camouflage by using relative motion; it is likely to be a critical step for separating objects from context in the early development of human infants. It is indicative of its importance that it has been conserved throughout evolution: it is found in insects and in our visual system. Recent work suggests that in vertebrates the computation of relative motion may actually start in the retina [3]. For all these reasons, the neural circuits underlying it are especially interesting.

Over forty years ago, a theoretical framework for this computation was established based on behavioral and – to a minor extent – electrophysiological studies in the housefly (*Musca*) [93] and the blowfly (*Calliphora*). The proposed circuit (see Figure H.1) posited that Elementary Motion Detectors (EMDs) respond to local motion, while wide-field "pool" cells summate these local signals across the visual field to estimate global optic flow. Figure-ground separation is then achieved by having these pool cells inhibit the local motion pathway, effectively vetoing the background motion. The same network was found to normalize the optomotor response to make it invariant to the size of the surround. It is therefore a *normalization and figure-ground circuit*.

With the advent of dense electron microscopy (EM) reconstructions of the *Drosophila melanogaster* brain, the physical implementation of this theoretical circuit can be mapped down to the individual synapses. Utilizing platforms like NeuPrint for the Janelia Hemibrain and FlyWire for the complete adult female fly brain (FAFB) [28, 97], we can now query massive graph databases to uncover how specific neurons and circuits wire together. We will review here whether this evidence support the

original proposal of Reichardt and Poggio.

25.2 Main Neurons in the Optical Lobes

To find the relative motion circuit in modern connectomes, we must map the theoretical triad—Local EMDs, Pool Cells, and Output Figure Detectors—to their anatomical correlates.

25.2.1 The Local EMDs: T4 and T5 Cells

The local motion signals in the connectome originate from the T4 and T5 columnar neurons, which represent the physical instantiation of the EMDs. T4 cells compute the direction of moving ON-edges (light increments), while T5 cells compute moving OFF-edges (light decrements). These cells are strictly direction-selective and stratify their axonal terminals into one of the four cardinal layers of the lobula plate, corresponding to the four cardinal directions of motion (front-to-back, back-to-front, upward, and downward). Notably, modern connectomics and optogenetics have confirmed that the outputs of T4 and T5 cells are entirely cholinergic (excitatory) [68].

25.2.2 Wide-Field Pool Cells: The LPTCs

The "pool" cells hypothesized in the 1983 model are implemented by the massive, wide-field neurons known as Lobula Plate Tangential Cells (LPTCs), which serve as global spatial integrators. A single LPTC pools synaptic input from up to ≈ 700 T4/T5 cells. This massive convergence makes sense given that within each column there are 4 T4 and 4 T5 cells. LPTCs collect input from hundreds of these local motion-sensing neurons spread across the entire retinotopic array, essentially integrating motion signals from a large swath of the visual field into a single wide-field response.

Some of the best characterized LPTCs include:

- **Horizontal System (HS) Cells:** Consisting of the Northern (HSN), Equatorial (HSE), and Southern (HSS) cells. They arborize predominantly in layer 1 of the lobula plate, acting as matched filters for horizontal optic flow and serving as the primary sensors for controlling yaw (horizontal rotation) stabilization.

- **Vertical System (VS) Cells:** An array of 10 cells (VS1–VS10) that tile the lobula plate vertically, primarily receiving input from layers 3 and 4 (upward and downward motion). By pooling their outputs via gap junctions, the VS network computes complex rotatory flow fields governing pitch and roll reflexes.

- **Centrifugal Horizontal (CH) Cells:** The ventral (vCH) and dorsal (dCH) cells act as inhibitory pool cells. Rather than directly controlling descending motor neurons, they provide local gain control and subtract background optic flow to enable figure-ground discrimination.

25.2.3 Dendrites of the CT1 Cell vs. the vCH Cell in *Drosophila*

When we contrast the dendrites of the **CT1** (Complex Tangential 1) cell with those of the **vCH** (ventral Centrifugal Horizontal) cell, we are essentially looking at two entirely different architectural philosophies for how the fly visual system handles inhibitory feedback and spatial integration.

Here is how their dendritic architectures break down.

25.2.4 The CT1 Cell: Local, Polyadic Interception

The CT1 cell acts as an enormous, multi-columnar inhibitory feedback element, but its dendritic interactions happen at a highly granular, local level.

- **Location:** CT1 arborizes specifically in layer **M10 of the medulla** (interfacing with the T4 ON-pathway) and layer **Lo1 of the lobula** (interfacing with the T5 OFF-pathway).
- **Synaptic Architecture:** Recent connectomic reconstructions (like those from FlyWire) show that CT1 uses highly complex **polyadic synapses**. A single presynaptic bouton from a Tm cell or the CT1 cell itself will simultaneously contact multiple postsynaptic compartments.
- **The Wiring:** CT1 provides GABAergic inhibition directly onto the dendrites of T4 and T5 motion-sensing neurons. Interestingly, a single CT1 synapse can span across the dendritic spines of multiple T4/T5 subtypes at once, allowing a single inhibitory signal to act on neurons tuned to different cardinal directions simultaneously.

25.2.5 The vCH Cell: Wide-Field, Dendro-Dendritic Integration

While the vCH cell was historically modeled in the blowfly (*Calliphora*), recent 3D EM datasets have fully reconstructed its homolog in *Drosophila*. It operates at a completely different stage of the motion-computation pipeline.

- **Location:** The vCH dendrites are restricted to the **lobula plate**, specifically covering the ventral portion of the visual field.
- **Gap junctions** Instead of local columnar inputs, vCH relies heavily on **dendro-dendritic electrical coupling** (gap junctions). It electrically couples with the overlapping dendritic trees of other wide-field Lobula Plate Tangential Cells (LPTCs), specifically the Horizontal System (HS) neurons. Note that lectrical junctions have been shown in Calliphora (by the the Borst lab) but not yet in Drosophila.
- **Cable properties** Full reconstruction of large lobula plate tangential cells in Drosophila from a 3D EM dataset[13] provides the definitive ultrastructural proof of the vCH cell's massive dendritic architecture in the fruit fly. It fully maps the dense, thick arborization of the vCH cell, demonstrating how its wide-field branches are restricted to layer 1 of the lobula plate.
- **The Wiring:** Because it integrates inputs from both ipsilateral HS cells (via electrical synapses) and contralateral elements like the H1 and H2 cells (via chemical synapses), its dendrites act as a massive spatial pooling mechanism. The commonly accepted hypothesis based on physiology in *Calliphora* is that the vCH cell then provides inhibitory output to object-detecting neurons (like the FD1 cell) to suppress their firing during large-field background motion (optic flow). In the next section we present an alternative model supported by anatomical data.

The contrast is stark: CT1 dendrites dive into the columnar weeds to modulate the primary motion detectors via complex polyadic wiring, whereas the vCH dendrites sit higher up in the lobula plate, electrically pooling massive amounts of spatial data to help the fly subtract background motion from foreground objects.

25.3 Local Motion Detection by T4/T5 Cells and CT1 Dendrites

While the integration of wide-field optic flow is handled downstream by LPTCs, the foundational computation of directional motion must occur locally and precisely within the retinotopic map. A critical—and architecturally astonishing—component of this local computation is the **CT1 (Complex Tangential 1)** cell [71].

CT1 is arguably the single largest and most structurally complex neuron in the fly optic lobe. There is only one CT1 cell per hemisphere, yet its massive arborization innervates every single one of the approximately 750 visual columns. Despite its wide-field morphology, CT1 operates in a strictly local, upstream computational regime, acting as a giant amacrine cell rather than a global spatial integrator.

25.3.1 Upstream Modulation and Null-Direction Suppression

Unlike the LPTCs, which pool the outputs of the elementary motion detectors, CT1 is positioned immediately *upstream* of the motion computation. It is strictly presynaptic to the EMDs [99, 102]. CT1 projects its terminals simultaneously into two distinct neuropils: providing widespread GABAergic (inhibitory) input directly onto the dendrites of **T4** cells in Medulla Layer 10, and homologous GABAergic input onto the dendrites of **T5** cells in Lobula Layer 1.

The directional selectivity of T4 and T5 cells relies on the physical, non-linear integration of signals across their dendrites. Modern connectomic and functional imaging reveals that this process is a hybrid mechanism, relying on both preferred-direction excitation and robust **null-direction suppression** [35, 4], as predicted by the Torre-Poggio model. In T5 cells for instance all columnar input neurons (Tm1, Tm2, Tm4, and Tm9) are excitatory. Borst et al. discovered that within each column, CT1 receives excitatory input from Tm9 and Tm1 and provides the sign-inverted, now inhibitory input signal onto T5. Ablating CT1 or knocking down GABA-receptor subunit Rdl significantly broadened the directional tuning of T5 cells. It thus appears that the signal of Tm1 and Tm9 is used both as an excitatory input for preferred direction enhancement and, through a sign inversion within the Tm1/Tm9-CT1 microcircuit, as an inhibitory input for null-direction suppression.

CT1 is the critical hardware for this inhibitory delay line. When a moving edge activates an upstream columnar cell (e.g., a Mi or Tm cell), it excites the local T4/T5 dendrite while simultaneously exciting the local CT1 terminal. CT1 responds by applying a slightly delayed GABAergic inhibition to the T4/T5 dendrite. Because of a highly specific spatial offset in the connectome, if an edge moves in the "wrong" (null) direction, the delayed CT1 inhibition arrives precisely as the main excitatory signal from the adjacent column reaches the dendrite, effectively vetoing the response [4]. State-of-the-art electron microscopy reconstructions mapped the exact synapses, showing that CT1 is strictly presynaptic to the dendrites of both the ON-pathway (T4 cells in the medulla) and the OFF-pathway (T5 cells in the lobula), positioning it perfectly as an upstream modulator of the motion signal.

25.3.2 Extreme Compartmentalization

To successfully mediate local motion detection, the inhibitory signals must not blur across the visual field. The fly resolves this through extreme biophysical compartmentalization [71]. Connectomic reconstructions reveal that the neurites connecting CT1's terminal branches are remarkably thin, dropping to diameters of approximately 100 nanometers and forming redundant, twisting loops. This creates immense axial resistance, electrically isolating the local synaptic sites. Consequently, the

single CT1 cell functions as an array of roughly **1,400 independent, electrically isolated inhibitory microcircuits**.

25.3.3 Recurrent Connectivity: Local Feedback and Gain Control

While CT1 plays a critical role in feedforward null-direction suppression, recent high-resolution dense connectomes have revealed that its relationship with the elementary motion detectors is not strictly unidirectional. CT1 also receives direct synaptic input from the T4 and T5 cells, creating an array of reciprocal, dendro-dendritic feedback loops at the very first stage of motion computation.

The anatomical proof of this reciprocal wiring was established by comprehensive reconstructions of the optic lobe. In the ON-pathway (M10 stratum of the medulla), the local CT1 terminal within each visual column receives a consistent number of synaptic inputs directly from the local T4 dendrites [101]. Subsequent mapping of the OFF-pathway (Lo1 stratum of the lobula) confirmed homologous reciprocal connectivity between T5 dendrites and CT1 terminals [99].

Because T4 and T5 cells are cholinergic (excitatory) and CT1 is GABAergic (inhibitory), this structural motif forms a highly localized negative feedback loop. Crucially, owing to the extreme biophysical compartmentalization of the CT1 cell [71], the excitation provided by a local T4/T5 cell is trapped within that specific microcircuit and does not spread laterally across the visual field.

This recurrent inhibition serves two vital computational functions for the local motion detectors:

1. **Temporal Sharpening:** Motion detection requires exquisite temporal precision. By exciting the local CT1 compartment, the active T4/T5 cell triggers its own rapid, delayed suppression. This truncated the tail of the voltage response, quickly resetting the local EMD for subsequent visual events.

2. **Dynamic Gain Control (Auto-Normalization):** In high-contrast visual environments, robust feedforward excitatory inputs could easily saturate the local T4/T5 dendrite. The reciprocal CT1 loop acts as an automatic, localized shunt. As the firing rate of the T4/T5 cell increases, it more strongly recruits local GABAergic inhibition from CT1 to clamp its own membrane potential, thereby maintaining the neuron within its optimal dynamic operating range.

Thus, while CT1's *feedforward* inhibition from upstream interneurons provides the null-direction suppression necessary to compute the motion vector [14], its *feedback* inhibition from T4/T5 provides the temporal sharpening and auto-normalization required to keep that computation fast and stable.

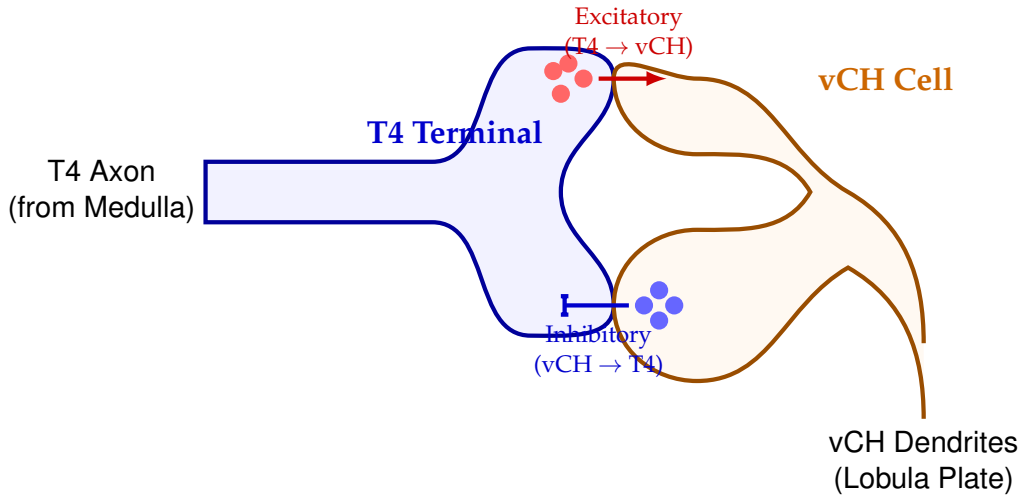
25.4 The Core Hypothesis: Figure-Ground Discrimination via vCH (Shunting) Inhibition

While CT1 handles local directional selectivity upstream, the macroscopic problem of figure-ground discrimination requires an inhibitory "pool cell" that integrates global background motion and subtracts it from the local EMD signals [93]. The connectome clearly identifies the vCH cell (and potentially LPi neurons like LPi15) as the primary candidate for this role.

In *Drosophila*, the massive dendritic trees of LPTCs like the vCH cell are mixed neurites. The arborization within layer 1 of the lobula plate contains *both postsynaptic* densities (receiving input) and *presynaptic T-bars (transmitting output)* in very close proximity. The connectome reveals a massive *reciprocal connectivity*: while arrays of T4 and T5 cells synapse onto vCH to provide front-to-back motion signals, the vCH cell synapses directly back onto the axon terminals of those same T4 cells right there in the lobula plate.

Hypothesis: We propose that T4 and T5 cells detect relative motion after exciting and being subsequently inhibited by the vCH cell, utilizing the exact same biophysical mechanism employed upstream by CT1: **shunting inhibition**.

During wide-field optic flow, massive T4/T5 excitation causes vCH to broadcast strong, widespread shunting inhibition back onto the T4 and T5 terminals, dynamically normalizing their outputs and suppressing the background motion signal. Because the CH cells are GABAergic, this dendro-dendritic inhibition likely opens chloride channels close to the resting potential of the cell, providing the divisive shunt required by the Poggio-Reichardt model. If, however, a small object moves independently of the background, the lack of global vCH activation allows the local EMD signals processing the object to escape the shunting inhibition.



25.5 The Output 'X Cell': LPTCs as Behavioral Integrators

In the canonical 1983 model, the background-subtracted signal from the EMDs is fed into an output "X cell" that summates (see figure) the progressive and regressive detectors and drives the behavioral motor response [93].

Because the vCH cell inhibits the T4 and T5 terminals directly, it effectively gates the motion signal *before* it is read by the other downstream tangential cells. We propose that the biological equivalent of the X cell is not a single, isolated neuron, but is rather a role dynamically assumed by the broader population of LPTCs.

Consequently, the remaining LPTCs —- such as the HS cells governing yaw, the VS cells governing pitch, and newly mapped translation-tuned LPTCs governing expansion and lift —- all receive a visual signal in which background motion has already been inhibited. These LPTCs thus serve as an array of specialized X cells, integrating the relative motion signals to trigger distinct, reflex-level motor commands in response to specific object trajectories.

25.6 Historical Context: Figure Detection (FD) Cells in the Blowfly

The functional targets of the figure-ground circuit were first characterized physiologically in the blowfly as Figure Detection (FD1 to FD4) cells. Modern comprehensive neuroanatomical surveys have identified 58 distinct LPTC types per hemisphere in *Drosophila* [121], some of which must correspond to the equivalents of these FD cells.

The receptive field of an FD cell is highly dynamic and is defined by relative motion rather than mere spatial location:

- **The Excitatory Center:** Defined by direct, convergent input from a localized patch of T4/T5 cells, directionally tuned to a specific axis.
- **The Inhibitory Surround:** A massive field covering the ipsilateral hemisphere, generated by the wide-field CH pool cells collecting global motion signals.

The defining hallmark of an FD cell is its size-tuning curve. As a moving object grows from 0° to 20° of visual angle, it fills the excitatory center, maximizing the firing rate. As the pattern expands further, mimicking global optic flow, it spills into the inhibitory surround, triggering the CH pool cells. The resulting inhibition heavily suppresses the FD cell, rendering it blind to the stationary world and maximally sensitive to independently moving figures. While the P-R model assumes FD cells pool the output of already-inhibited T4/T5 cells, a similar conjecture by Egelhaaf posits that FD cells are excited by T4/T5 cells and inhibited directly by CH cells. As FD cells are non-spiking graded-potential neurons, the functional distinction between shunting and hyperpolarizing inhibition at this stage is somewhat blurred, as both effectively reduce gain.

25.7 Biophysical Implementation of Shunting Inhibition

The true relative motion computation modeled by Reichardt, Poggio, and Hausen [93] relies not on simple linear subtraction, but on non-linear, divisive inhibition. As formally modeled by Torre and Poggio [104], for inhibition to be divisively shunting, it must be strategically located—either adjacent to the excitatory synapse on the same dendritic branch, or on the path to the spike initiation zone.

The physical wiring diagram provided by the FlyWire dataset confirms this spatial geometry exists. The inhibitory synapses from the vCH cell into the T4/T5 terminals are positioned in extremely close physical proximity to the cholinergic inputs from the T4/T5 cells onto the vCH cell. To date, this shunting mechanism has been biophysically demonstrated for the local motion detectors (e.g., within the T4 dendrites), utilizing synapses that execute a physical approximation of the divisive equation:

$$R = \frac{E}{1 + K \cdot I} \tag{25.1}$$

for the computation of motion direction.

Researchers construct multi-compartment biophysical models—often utilizing the NEURON simulation environment—by populating raw 3D dendritic meshes from EM datasets with specific ion channels to mathematically map these nonlinear conductances:

- **Dendritic Multiplication via Shunting:** Groschner et al. (2022) provide a definitive biophysical account of the nonlinear arithmetic required by the Hassenstein-Reichardt correlator. They demonstrated how the T4 dendrite achieves a multiplication-like nonlinearity arising from the physical coincidence of cholinergic excitation and glutamatergic shunting inhibition, mathematically mapping the shunting conductances onto the dendrite.
- **Connectome-Constrained Mechanistic Networks:** Lappalainen et al. (2024) developed a "Deep Mechanistic Network" that translates the optic lobe connectome into a differentiable simulation. Using deep learning to optimize unknown parameters, the resulting optimized mathematical weights—specifically the balance of feedforward excitation and shunting inhibition—closely matched physiological recordings.

- **Compartmental Modeling of Tangential Cells:** Elyada, Haag, and Borst (2013) demonstrated that large-field pool cells are not electrotonically compact. They modeled the spatial profile of lateral inhibition, mathematically demonstrating that wide-field dendritic inhibition acts as a shunt extending beyond the retinotopic spatial extent of the excitatory dendrites.

Ultimately, the 2022 framework established by Groschner et al. serves as a critical centerpiece for this literature, explicitly mapping the algorithmic requirement for nonlinear division directly onto the chloride channels of a connectome-reconstructed dendrite.

25.8 Predictions and the Missing Link

According to our hypothesis, the core computation of relative motion is carried by the membrane potential of the T4/T5 cells once they have been inhibited by the vCH pool cell. This yields specific testable predictions:

- If we record from the T4/T5 cells *after* the inhibition by the vCH, the sum of their outputs should mirror the classic figure-ground behavioral response.
- It is possible that two parallel mechanisms underlie relative motion detection in *Drosophila*. The first is the mechanism described here (predicted by the Poggio-Reichardt model) where the pool acts presynaptically on the T4/T5 axon terminals. The second (proposed by Egelhaaf) consists of forward inhibition acting directly on the output figure-detection cell by a large field neuron.

While we possess a rigorous theoretical model and the exact physical connectome that accommodates it, the two have not yet been formally bridged in the modern electrophysiological literature. Until an experimentalist achieves a recording of a connectome-identified T4 terminal or FD cell during a rigorously controlled moving-figure/moving-ground stimulus—demonstrating that the instantaneous membrane voltage dynamics strictly obey the predicted divisive suppression—the final loop in physically validating this classic relative motion model remains unclosed.

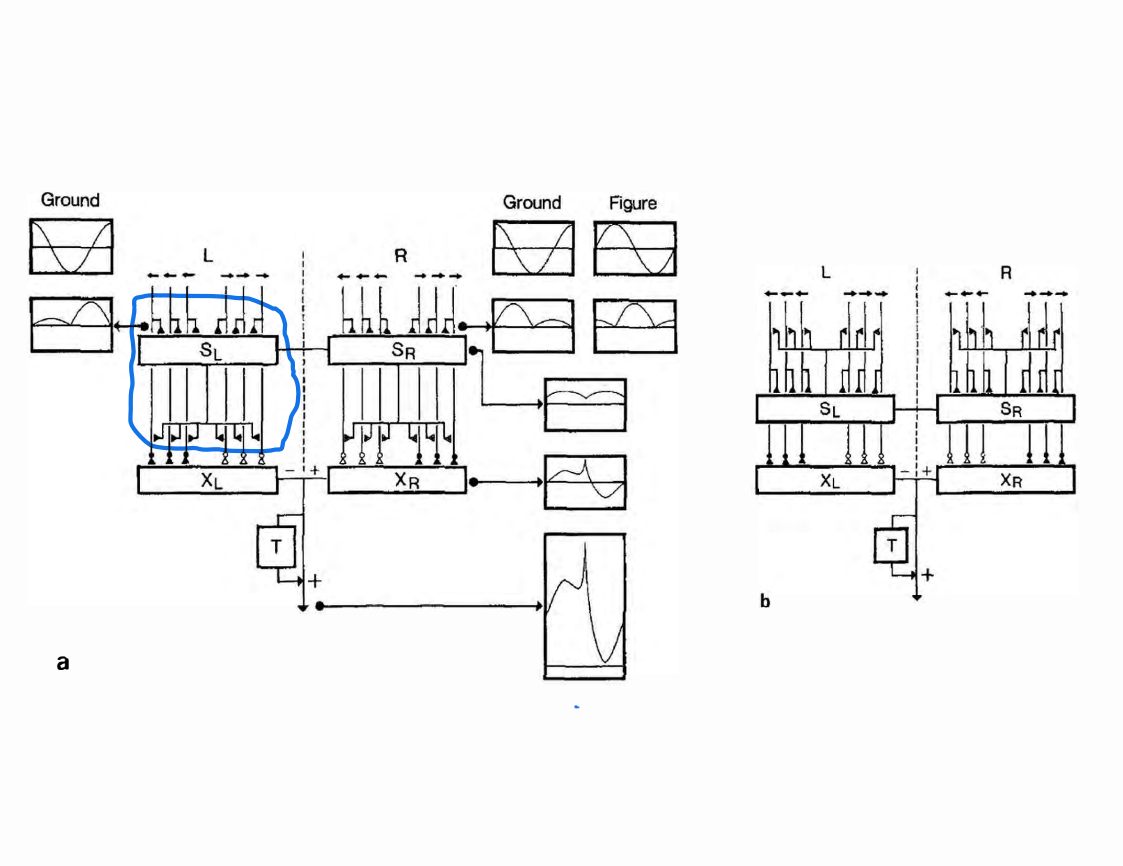


Figure 25.1: a The outline of the model circuitry for the right (R) and for the left (L) eye. Considering the right eye as example, two retinotopic arrays of elementary movement detectors, responding selectively to progressive and regressive motion, serve as input channels to the neural circuitry. The two arrays share the same field of view. For convenience they are drawn apart from each other. A pool neuron (SR) summates the detector outputs as well as the input from its contralateral homologue (SL). Its output is assumed to undergo a saturation effect (modeled by taking the square root of its overall excitation) and to shunt each elementary movement detector output via presynaptic inhibition. The synapses involved should therefore inhibit (opening ionic channels with an equilibrium battery close to the resting potential) the output terminal of each elementary detector channel. The output cell X summates the progressive (excitatory) and the regressive (inhibitory) detectors. Progressive channels have a higher amplification than regressive ones. possibly because of the different ionic batteries involved. This is the key mechanism underlying the computation of relative motion (and gain control): it is contained in the blue envelope. The synapses on the X R cell are assumed to operate with a nonlinear input-output characteristic, leading to postsynaptic signals that are approximately the square of the inputs. The motor output is controlled by the X-cells via a direct channel and a channel T computing the running average of the X-cells output. The insets in the figure contain the responses at the various levels to a bilateral figure-ground experiment. Responses at the input channels are split into two components, the ground component and the figure component. b The outline of another neural circuitry that can also explain the behavioural responses at the cellular level. The model is a recurrent shunting inhibition network. The important difference between the two models is that the first one (a) contains a shunting inhibition pathway acting on the channels after the pool cells (S) have received the detector signals - that is to say the pool acts forward. In the model presented here the pool operates via a recurrent pathway that acts on the individual detector channels before receiving signals from the individual channels. All other details as described in the legend of a.

CHAPTER 26

Frozen Interpreter and Cultural Evolution

Learning is usually identified with modification of internal parameters. A neural network learns by changing its weights; biological evolution changes inherited machinery across generations.

Human adaptation, however, also proceeds through a second channel. Language, algorithms, recipes, mathematical notation, scientific theories, and institutions can improve while the basic neural machinery of their users remains approximately fixed on the relevant timescale.

Frozen language models provide a simple computational laboratory for this distinction.

A model with fixed parameters can nevertheless acquire increasingly useful external instructions, procedures, examples, memories, tools, and libraries. These objects can be copied, modified, recombined, selected, and transmitted without changing a single weight.

The central hypothesis of this chapter is therefore

parametric learning changes the interpreter;	*cultural learning changes the programs supplied to it.*

The analogy is functional, not literal: neural weights are not genes, and prompts are not human culture. The point is that adaptive information may reside either inside the machinery or outside it.

26.1 Two Channels of Adaptation

Let

$$f_\theta(y \mid x, p)$$

denote the conditional behavior of a model with parameters θ, input x, and externally supplied program p.

The word "program" will be used broadly. It may contain

instructions, demonstrations, retrieved facts, intermediate procedures, tool calls, memory, verificati

There are then two conceptually different forms of adaptation.

Parametric adaptation changes the internal machinery:

$$\theta_T^\star \in \arg\min_\theta \widehat{L}_T(\theta, p_0). \tag{26.1}$$

Cultural adaptation leaves the machinery fixed and instead changes the external program:

$$p_T^\star \in \arg\min_{p\in\mathcal{P}} \widehat{L}_T(\theta_0, p), \qquad \theta_0 \text{ fixed.} \tag{26.2}$$

Thus the two timescales are

$$\underbrace{\theta}_{\text{persistent internal machinery}} \qquad \underbrace{p}_{\text{rapid transmissible state}}.$$

This is not merely ordinary in-context learning.

In-context learning modifies behavior within one context.

Cultural learning, as meant here, requires that useful external structures survive beyond one episode and are inherited by later trials, agents, or tasks.

The individual model may forget.

The external population or library remembers.

26.2 Prompt Selection Is a Learning Problem

Fix a frozen model

$$\theta_0$$

and a family of admissible programs

$$\mathcal{P}.$$

For a task distribution $\mathcal{D}_T$ over examples

$$z = (x, y),$$

define

$$\widehat{L}_T(p) = \frac{1}{n}\sum_{i=1}^{n} \ell\left(f_{\theta_0}(x_i; p), y_i\right), \tag{26.3}$$

$$L_T(p) = \mathbb{E}_{(x,y)\sim\mathcal{D}_T} \ell\left(f_{\theta_0}(x; p), y\right). \tag{26.4}$$

The frozen interpreter and program family induce the hypothesis class

$$\mathcal{H}_{\theta_0,\mathcal{P}} = \{x \mapsto f_{\theta_0}(x; p) : p \in \mathcal{P}\}. \tag{26.5}$$

Experience selects a member of this class.

Thus

absence of weight change does not imply absence of learning.

It tells us where the learned state is stored.

26.2.1 A simple generalization bound

For a finite program class, the standard finite-class argument already gives a useful result.

Proposition 22 (Generalization under program selection)**.** *Assume*

$$\ell \in [0,1]$$

and $\mathcal{P}$ is finite.
Let

$$\widehat{p} \in \arg\min_{p \in \mathcal{P}} \widehat{L}_T(p).$$

Then with probability at least $1-\delta$,

$$L_T(\widehat{p}) \le \widehat{L}_T(\widehat{p}) + \sqrt{\frac{\log|\mathcal{P}| + \log(2/\delta)}{2n}}. \tag{26.6}$$

The proof is the usual application of Hoeffding's inequality and a union bound over $\mathcal{P}$.
If programs contain at most m tokens from a vocabulary of size V,

$$|\mathcal{P}| \le \sum_{j=0}^{m} V^j,$$

and the complexity term scales as

$$O\left(\sqrt{\frac{m\log V + \log(1/\delta)}{n}}\right).$$

This exposes a cultural bottleneck:

limited transmission length constrains capacity.

Short cultural objects may generalize better, but only if the frozen interpreter can reliably execute what they specify.

26.3 Culture as Programs for a Frozen Interpreter

A useful abstraction is to view pretraining as constructing a repertoire of latent operations,

$$\Phi_{\theta_0} = \{\phi_1, \dots, \phi_K\}.$$

These may include such functions as

retrieval, comparison, decomposition, arithmetic, translation, verification.

A cultural program does not necessarily create new primitive operations.
It may instead address and compose those already available.
Schematically,

$$\pi_p(x) = \phi_{i_r} \circ \phi_{i_{r-1}} \circ \cdots \circ \phi_{i_1}(x). \tag{26.7}$$

This is a functional hypothesis, not a mechanistic claim about individual Transformer circuits.

It suggests a simple division of labor:

pretraining creates a repertoire;	culture searches for reusable programs over that repertoire.

This also predicts a boundary.

External-program search should succeed when the required primitives are already represented and can be reliably invoked.

It should plateau when

a required primitive is missing

or when

the interface cannot invoke the available primitive reliably.

Parametric and cultural learning are therefore complementary.

Parameter change expands or reorganizes the interpreter.

Culture reorganizes and transmits its use.

26.4 Interpreter Complexity and Program Complexity

The interpreter/program distinction has a classical computational analogue.

A universal interpreter is fixed.

Different computations are specified by different external programs.

For an ideal universal machine U,

$$U(p_f, x) = f(x) \tag{26.8}$$

for every computable f and an appropriate finite program p_f.

Thus universality separates

fixed transition machinery

from

variable symbolic programs.

A real deployed LLM with finite context, finite generation length, and finite external memory is not literally an unbounded universal Turing machine.

The analogy concerns architecture: fixed machinery can support many different computations when supplied with different external states.

26.4.1 The model–program tradeoff

Let

$$K_C(f) = \min_{p:C(p,x)=f(x)} |p|$$

denote the shortest program implementing f relative to interpreter C.

Two interpreters may compute the same function class while assigning very different description lengths to useful functions.

A richly pretrained system may satisfy

$$K_{C_{\text{large}}}(f) \ll K_{C_{\text{small}}}(f).$$

Thus pretraining may act as compilation.

Frequently useful operations become implicit in the interpreter, so that later programs can invoke them using shorter descriptions.

This suggests the decomposition

$$\boxed{\text{capability} = \text{interpreter} + \text{external program}.}$$

A simpler universal interpreter may in principle reproduce the same behavior, but only with much longer programs or far greater runtime.

Universality concerns expressibility.

Practical intelligence additionally requires

$$\boxed{\text{expressibility}, \quad \text{efficiency}, \quad \text{reliability}.}$$

A pretrained model may be valuable primarily because important computations become short, efficient, and reliable to invoke.

26.5 Evolution of External Programs

Let

$$P_g = \{p_{g,1}, \ldots, p_{g,N}\}$$

be a population of programs at cultural generation g.

Define fitness

$$F_T(p) = -\widehat{L}_T(p) - \lambda C(p), \tag{26.9}$$

where $C(p)$ may penalize

$$\text{description length}, \quad \text{inference cost}, \quad \text{memory use}.$$

Programs are then

$$\text{selected}, \quad \text{mutated}, \quad \text{recombined}, \quad \text{reevaluated}.$$

The model itself may perform these transformations.

Thus the frozen interpreter participates in changing the external program population without changing its own weights.

26.5.1 Meta-programs

A stronger form of adaptation changes not only the solution program but the procedure used to generate new programs.

Let

$$a_g$$

denote a meta-program controlling proposal generation.

The complete external state becomes

$$\mathcal{S}_g = (\theta, p_g, a_g, \mathcal{L}_g), \qquad \theta \text{ fixed}, \tag{26.10}$$

where $\mathcal{L}_g$ is an inherited library.

The proposal mechanism may be written

$$p' \sim q_\theta \left(p' \mid p_g, a_g, \mathcal{L}_g, H_g\right), \tag{26.11}$$

where H_g stores previous proposals and outcomes.

There are now two levels of adaptation:

$$\boxed{\text{learn better solutions}}$$

and

$$\boxed{\text{learn better procedures for finding solutions.}}$$

26.6 What Counts as Cumulative Culture?

Improving one task is not enough.

A population can optimize a program for a fixed task without accumulating anything useful for the future.

A stronger criterion is that transmitted information reduces the cost of learning later tasks.

Let

$$C_\varepsilon(T \mid \mathcal{L})$$

denote the computation, candidate evaluations, or tokens required to reach performance within ε of a task target.

We call cultural adaptation cumulative when

$$\mathbb{E}_{T\sim\rho} C_\varepsilon(T \mid \mathcal{L}_{g+1}) < \mathbb{E}_{T\sim\rho} C_\varepsilon(T \mid \mathcal{L}_g). \tag{26.12}$$

This is stronger than retaining a good answer.

It requires that inherited structure make future adaptation easier.

Thus:

$$\boxed{\text{task improvement} \neq \text{cumulative cultural improvement.}}$$

A genuine cultural ratchet should produce

$$\boxed{\text{reuse} + \text{retention} + \text{recombination} + \text{decreasing future learning cost.}}$$

26.7 Intelligent Search

External evolution is useful only if proposals are better than blind search.

For accuracy tolerance ε, define the successful region

$$\mathcal{P}_\varepsilon(T) = \{p : L_T(p) \leq L_T^\star + \varepsilon\}. \tag{26.13}$$

For proposal distribution q, define

$$\alpha_q(T, H) = \sum_{p \in \mathcal{P}_\varepsilon(T)} q(p \mid T, H). \tag{26.14}$$

Under a fixed proposal rule, the expected hitting time is

$$\mathbb{E}[\tau_\varepsilon] = \frac{1}{\alpha_q}. \tag{26.15}$$

This gives a simple operational criterion.

An intelligent cultural search mechanism should place more probability mass on useful programs than a matched baseline:

$$\boxed{\alpha_{\text{guided}} > \alpha_{\text{baseline}}.}$$

Semantic analogy, decomposition, error analysis, and inheritance are valuable only insofar as they increase this probability or reduce the cost of evaluating candidates.

26.8 Identification Is Not Execution

A small number of examples can sometimes uniquely specify a target inside a restricted hypothesis class.

Let

$$V_{\mathcal{H}}(S, f) = \{h \in \mathcal{H} : h(x) = f(x) \text{ for all } x \in S\}$$

be the version space.

If

$$V_{\mathcal{H}}(S, f) = \{f\},$$

then S identifies the target within $\mathcal{H}$.

This can happen with surprisingly few examples.

But three questions remain separate:

identification: is the rule determined?
search: can the system find it?
execution: can the interpreter carry it out reliably?

A teaching set may completely specify a symbolic rule while a frozen LLM still fails to infer or execute that rule.

This distinction will matter in the experiments below.

26.9 Bounded Cultural Resources Imply a Ceiling

The phrase "open-ended improvement" is ambiguous.
One must distinguish

performance ceiling, number of strict improvements, computational expressivity, resource cost.

A simple finite-resource result makes this distinction precise.
Suppose the total cultural state is bounded by

$$R = (m, M, B),$$

where

$$|p| \le m, \qquad |\mathcal{L}| \le M, \qquad t \le B,$$

with finite alphabets and finite-precision computation.
Let

$$\mathcal{C}_R$$

be the finite set of admissible external states.
The frozen interpreter induces the finite behavior class

$$\mathcal{H}_{\theta,R} = \{h_c : c \in \mathcal{C}_R\}. \tag{26.16}$$

Proposition 23 (Bounded cultural ceiling)**.** *Let $J_T(h)$ denote performance on a fixed task distribution. If cultural evolution is elitist,*

$$J_T(h_{g+1}) \ge J_T(h_g),$$

then the number of strict improvements is at most

$$|J_T(\mathcal{H}_{\theta,R})| - 1 \le |\mathcal{H}_{\theta,R}| - 1, \tag{26.17}$$

and performance cannot exceed

$$J_T^\star(\theta, R) = \max_{h \in \mathcal{H}_{\theta,R}} J_T(h). \tag{26.18}$$

Proof. The resource-bounded behavior class is finite.
Therefore the set of attainable performance values is finite.
Every strict improvement moves to a strictly larger value in this finite ordered set.
The number of such moves is finite, and the maximum is attained. □

Thus

bounded program length + bounded memory + bounded runtime ⇒ finite cultural improvement sequence.

26.10 Frozen Weights Alone Do Not Imply a Finite Improvement Sequence

The resource assumptions above are essential.

If program length, memory, and computation are allowed to grow without bound, freezing the interpreter does not imply a finite number of possible improvements.

Consider an ideal universal interpreter and the target

$$f(x) = 1, \qquad x \in \mathbb{N},$$

under distribution

$$P(X = x) = 2^{-(x+1)}. \tag{26.19}$$

Let program p_n compute

$$p_n(x) = \begin{cases} 1, & x \le n, \\ 0, & x > n. \end{cases} \tag{26.20}$$

Then

$$L(p_n) = P(X > n) = 2^{-(n+1)}, \tag{26.21}$$

and therefore

$$L(p_{n+1}) < L(p_n)$$

for every n.

Hence there are infinitely many strict improvements even though the interpreter never changes.

Performance remains bounded, while the required program length, runtime, or search cost may diverge.

This establishes the distinction

$$\boxed{\text{frozen machinery} \not\Longrightarrow \text{finite external adaptation.}}$$

What creates a finite ceiling is not frozen weights alone.

It is bounded total resources.

26.11 Experimental Test

The preceding theory suggests a concrete experiment.

Freeze an open-weight language model and verify throughout the experiment that its parameters remain unchanged.

Across a sequence of related tasks, allow the external system to modify

$$p_g, \qquad a_g, \qquad \mathcal{L}_g.$$

Measure separately:

1. **task improvement:** does held-out performance improve across cultural generations?

2. **proposal quality:** does guided search place greater mass on successful programs than matched random search?
3. **inheritance:** do descendants initialized from transmitted artifacts learn faster than fresh populations?
4. **recombination:** does combining independently useful artifacts produce new gains?
5. **transfer:** do programs learned on earlier tasks reduce the cost of solving later related tasks?
6. **cumulative culture:** does
$$C_\varepsilon(T \mid \mathcal{L}_g)$$
systematically decrease with cultural generation?

All comparisons must use held-out evaluation and matched computational budgets.

26.12 What the Initial Experiments Show

We performed exploratory experiments with frozen language models in which external prompts were selected, mutated, and in some conditions supplied with previously successful artifacts.

The experiments establish one basic fact:

a frozen model can realize substantially different behaviors under different externally selected programs.

That is the minimal prerequisite for the cultural-interpreter picture.

The stronger claims were not supported.

26.12.1 Search was not reliably intelligent

In a finite-menu pilot, informed LLM-guided search did not consistently outperform random search.

Performance was strongly task dependent.

In some cases semantic guidance helped.

In others it hurt.

Thus

the frozen interpreter did not reliably extract the information available in the examples.

26.12.2 Inheritance was not reliably cumulative

Supplying previously useful programs sometimes improved later search and sometimes biased it toward unhelpful regions.

In subsequent free-generation experiments, inherited libraries did not produce a reliable reduction in the cost of discovering successful programs.

Thus the strong criterion

$$\mathbb{E}C_\varepsilon(T \mid \mathcal{L}_{g+1}) < \mathbb{E}C_\varepsilon(T \mid \mathcal{L}_g)$$

was not established.

26.12.3 Model size did not solve the problem

Increasing the frozen model from approximately 1.5 billion to 3 billion parameters did not yield a consistent advantage under the tested protocol.

This suggests that the bottleneck was not simply interpreter size.

Observed failures included

proposal failure, poorly generalizing library entries, wording-sensitive execution, short search hori

26.12.4 Identification and execution separated sharply

In symbolic controls, small teaching sets could uniquely identify a rule within a restricted hypothesis class.

Yet supplying comparable information to the neural interpreter did not guarantee correct execution.

Thus the experiments support the distinction

$$\boxed{\text{identification} \neq \text{search} \neq \text{execution.}}$$

This is closely related to the distinction between statistical sufficiency and computational recoverability developed earlier in the book.

26.13 What Would Constitute Strong Evidence?

The most convincing result would not be that evolved prompts outperform one hand-written baseline.

A genuine demonstration of cumulative cultural adaptation would require

$$\boxed{\begin{array}{c}\text{fixed verified interpreter}\\ \Downarrow\\ \text{externally evolving programs}\\ \Downarrow\\ \text{retained reusable abstractions}\\ \Downarrow\\ \text{cross-task recombination}\\ \Downarrow\\ \text{decreasing cost of future learning.}\end{array}}$$

Several controls are essential.

The library must be genuinely accumulated rather than supplied by hand.

Future tasks must be held out from the construction of previous artifacts.

Compute budgets must be matched.

Scrambled or irrelevant libraries should control for additional context.

And all reported improvements must be measured on unseen examples or certified by sound verification.

The central empirical quantity is not final accuracy alone.

It is

$$\boxed{\text{learning cost on future tasks as a function of inherited culture.}}$$

26.14 Limits

Several limits should remain explicit.

26.14.1 No universal search advantage

No search algorithm dominates on all possible objective functions.

Any advantage of cultural evolution must therefore be relative to a structured task distribution

$$T \sim \rho.$$

Culture can exploit regularities shared across that distribution.

It cannot provide a uniform advantage over arbitrary tasks.

26.14.2 No universal semantic verifier

For unrestricted programs, there is no computable procedure that can decide arbitrary nontrivial semantic properties.

Thus a general cultural-evolution system cannot always certify that a new program is globally correct or superior.

Exact verification becomes possible only after restricting

$$\text{the domain}, \qquad \text{program language}, \qquad \text{specification},$$

or by requiring proof-carrying outputs accepted by a sound verifier.

26.14.3 Universality does not imply practicality

An ideal frozen universal interpreter can express arbitrarily complicated computations through external programs.

But universality says nothing about

$$\text{program length}, \qquad \text{runtime}, \qquad \text{memory}, \qquad \text{execution reliability}, \qquad \text{search cost}.$$

These resource quantities determine whether cultural adaptation is useful in practice.

26.15 Relation to the Framework of This Book

The cultural-interpreter picture complements several themes developed earlier.

Sparse compositionality concerns the structure of computable functions.

The present chapter asks where the description of that structure resides.

It may reside partly in the learned parameters,

$$\theta,$$

and partly in external programs,

$$p.$$

The bits chapter similarly emphasized that architecture defines a coding language while optimization selects a code.

Here the coding language is supplied by the frozen interpreter, while cultural evolution searches over transmissible descriptions.

The relationship can be summarized as

pretraining compresses useful operations into the interpreter;

culture discovers short programs that compose and reuse them.

The CRM chapter adds a further possibility: repeated external programs may eventually be compiled into new internal primitives.

Thus there is a potential cycle

internal primitives
⇓
external compositions
⇓
cultural reuse
⇓
compilation into new primitives.

This provides a computational analogy to the interaction between slow biological or parametric adaptation and faster cultural accumulation.

26.16 Conclusion

The central proposal can be stated in one sentence:

pretraining creates the interpreter; culture changes the programs supplied to it.

A frozen model is not necessarily a frozen adaptive system.

If instructions, memories, procedures, meta-prompts, and libraries are allowed to change, then the complete system

$$(\theta, p, a, \mathcal{L})$$

can continue to adapt even while

$$\theta$$

remains fixed.

The theory makes two simple points.

First,

bounded total cultural resources imply a finite behavioral class and therefore a finite sequence of strict improve

Second,

frozen weights alone do not imply such a bound when external programs, memory, and computation are allowe

The experimental question is harder.

Can a frozen interpreter generate, preserve, recombine, and transmit external programs so that later tasks become progressively easier?

The exploratory experiments described here do not yet demonstrate that strong form of cumulative culture.

They do, however, expose the relevant bottlenecks:

search, interpretation, verification, transfer, retention.

This makes frozen language models an interesting experimental system for studying a broader principle:

learning need not be stored entirely inside the learner.

An intelligent system may adapt not only by changing its machinery, but also by improving the transmissible programs through which that machinery is used.

CHAPTER 27

Puzzles

27.1 Why Are Task-Relevant Variables Often Linearly Decodable from Pretrained Representations—and How Can This Be Exploited in Post-Training?

A striking empirical property of many pretrained models is that variables relevant to downstream tasks can often be recovered from intermediate representations by surprisingly simple probes.

Let

$$h_\ell(x) \in \mathbb{R}^{d_\ell}$$

denote the representation of input x at layer ℓ.

For a downstream target y, one often finds that a linear predictor

$$\widehat{y} = W_\ell h_\ell(x) + b_\ell \tag{27.1}$$

achieves substantially better performance than would be expected if the representation retained only generic, undifferentiated information.

The important claim is not that the layer itself depends linearly on the target.

Rather,

a target that is complicated in input space may become low-complexity, and sometimes nearly linear, in learned

This suggests a natural interpretation of pretraining.

Pretraining repeatedly changes coordinates.

A relation that is high-order or difficult to express in the original input variables,

$$y = f(x),$$

may become much simpler after the sequence of learned transformations

$$x \longrightarrow h_1(x) \longrightarrow h_2(x) \longrightarrow \cdots \longrightarrow h_L(x).$$

At some intermediate layer ℓ, the target may approximately satisfy

$$y \approx w^\top h_\ell(x) + b.$$

Thus one possible role of representation learning is

$$\boxed{\text{complicated relation in old coordinates} \rightarrow \text{simple relation in new coordinates.}}$$

This is closely related to the staircase picture developed earlier in the book.

Learning one useful representation can change the effective degree or complexity of the remaining problem.

A dependency that was difficult to detect or approximate in the original coordinates may become a low-order footprint in the new representation.

The phenomenon also suggests an important interpretation of post-training.

Suppose a desired downstream behavior is already represented somewhere inside the pretrained network.

Then post-training need not create that feature from scratch.

It may instead need primarily to

$$\boxed{\text{select} + \text{amplify} + \text{combine} + \text{route}}$$

features that pretraining has already constructed.

This motivates searching for a useful intervention layer.

One simple criterion is

$$\ell^\star = \arg\min_\ell \left[\mathcal{L}_{\text{probe}}(\ell) + \lambda C_{\text{probe}}(\ell)\right], \tag{27.2}$$

where $\mathcal{L}_{\text{probe}}$ measures prediction error and C_{probe} measures the complexity of the probe.

A layer at which the target is recoverable by a low-complexity probe is a natural candidate for efficient post-training.

For example, one may learn a low-dimensional modification

$$h'_{\ell^\star} = h_{\ell^\star} + Uv,$$

or a low-rank adaptation

$$h'_{\ell^\star} = h_{\ell^\star} + ABh_{\ell^\star}, \qquad \operatorname{rank}(AB) \ll d_{\ell^\star}.$$

The resulting hypothesis is

$$\boxed{\text{if the desired behavior is already low-complexity in a pretrained representation, post-training may only need a}}$$

This provides a possible explanation for why comparatively small post-training updates can sometimes produce large behavioral changes.

The base model has already performed most of the representational work.

Post-training changes how that representation is read, weighted, routed, or converted into output behavior.

27.1.1 Linear decodability is not causal use

An important qualification is necessary.

Suppose a linear probe can recover some variable y from h_ℓ.

This establishes that information about y is present in the representation.

It does not establish that the remaining network actually uses that information.

Thus

$$\boxed{\text{linear decodability} \neq \text{causal utilization.}}$$

A representation may contain many variables that can be decoded by an external observer but are irrelevant to the computation performed by subsequent layers.

To establish functional relevance, probing should therefore be combined with intervention.

For example, one may perturb the putative direction associated with y,

$$h_\ell \longrightarrow h_\ell + \alpha v_y,$$

and measure whether the downstream behavior changes in the predicted way.

A useful empirical program is therefore

$$\boxed{\begin{array}{c}\text{find a simple probe}\\ \Downarrow\\ \text{identify a low-dimensional representation}\\ \Downarrow\\ \text{intervene on that representation}\\ \Downarrow\\ \text{measure causal effect on behavior.}\end{array}}$$

This separates two questions that should not be confused:

$$\text{What information is present?}$$

and

$$\text{What information is actually used?}$$

27.1.2 Connection to the framework of this book

The phenomenon fits naturally into the distinction between representation and recovery.

Pretraining may construct a representation in which many downstream variables become simple.

That is primarily a representational achievement.

Post-training then faces a smaller recovery and selection problem: identify which already-available coordinates are relevant and how they should influence behavior.

Thus a useful working hypothesis is

$$\boxed{\text{pretraining constructs useful coordinates; post-training selects how to exploit them.}}$$

If correct, this viewpoint suggests that an important measure of a pretrained model is not merely its final-task accuracy, but the complexity with which useful variables become accessible throughout its internal representation hierarchy.

27.2 Weights as Compiled Computation, Prompts as Programs

A frozen large language model admits a useful computational interpretation.

The physical hardware is, of course, the processor on which the model is executed.

The frozen neural network is still software.

Nevertheless, at the level of computation there is a useful analogy between a pretrained model and a machine whose instruction set has been learned.

Let

$$\theta$$

denote the frozen parameters of the model.

After pretraining and post-training, inference is performed with

$$\theta$$

held fixed.

The model then implements a map of the schematic form

$$y = F_\theta(p, x), \tag{27.3}$$

where p is a prompt or contextual program and x denotes additional input or data.

This suggests the interpretation

$$\boxed{\begin{aligned} \text{architecture + frozen weights} &\approx \text{learned interpreter,} \\ \text{prompt + context} &\approx \text{program and working data,} \\ \text{activations / KV state} &\approx \text{working computational state,} \\ \text{generated tokens} &\approx \text{execution trace and output.} \end{aligned}}$$

The analogy should not be taken literally.

A Transformer is not a conventional CPU, and a prompt is not a machine program in the ordinary formal sense.

The useful point is the separation between two different timescales of adaptation.

27.2.1 Slow adaptation: compiling computation into weights

During pretraining and post-training, the parameters change:

$$\theta_0 \longrightarrow \theta_1 \longrightarrow \cdots \longrightarrow \theta^\star.$$

This process can be viewed schematically as

$$\boxed{\text{experience and optimization} \longrightarrow \text{compiled structure in weights.}}$$

Repeated statistical regularities, procedures, associations, and computational primitives become encoded in the model parameters.

Once training ends, these parameters define the computational machinery available to subsequent inference.

In this sense,

weights are compiled computation.

The word "compiled" is useful because many computations that required explicit examples during training can later be executed rapidly through a single forward pass.

The underlying learning process has converted experience into a persistent computational mechanism.

27.2.2 Fast adaptation: programming the frozen interpreter

At inference time the weights remain fixed.

Adaptation occurs instead through the context.

A prompt can specify

task, rules, examples, goals, intermediate procedures.

Thus

$$p$$

can function as a program for the frozen model.

Different prompts applied to the same weights can induce very different computations:

$$F_\theta(p_1, x) \neq F_\theta(p_2, x).$$

This gives the second part of the analogy:

prompts are programs executed by a learned interpreter.

The distinction between training and prompting is therefore

slow timescale:	change the interpreter by changing weights,
fast timescale:	change the computation by changing the program or context.

This is computationally important because useful adaptation can occur without changing θ.

27.2.3 A third level: external program libraries

A frozen interpreter can be paired with an external library

$$\mathcal{L} = \{p_1, p_2, \ldots, p_M\}$$

of prompts, procedures, tools, examples, or reusable subprograms.

The total system is then not merely

$$\theta,$$

but

$$(\theta, \mathcal{L}, p, x).$$

The frozen model provides the interpreter.
The external library provides accumulated procedures.
The current prompt selects and combines those procedures for the task at hand.
This creates an important possibility:

$$\boxed{\text{the system can improve while the weights remain completely frozen.}}$$

Improvement can occur through

$$\mathcal{L}_0 \subset \mathcal{L}_1 \subset \mathcal{L}_2 \subset \cdots,$$

even when

$$\theta$$

never changes.
The evolving object is then not the neural network but the computational culture surrounding it.

27.2.4 Learning in parameter space versus program space

This suggests two distinct forms of learning:

$$\boxed{\begin{aligned} \text{parameter learning} &: \theta \to \theta', \\ \text{program learning} &: p \to p' \quad \text{or} \quad \mathcal{L} \to \mathcal{L}'. \end{aligned}}$$

Parameter learning changes the interpreter.
Program learning changes what the interpreter is asked to execute.
The two can have very different costs.
Changing billions of parameters is expensive.
Searching over a compact prompt or procedure may be much cheaper.
This suggests that some adaptation problems may be better posed as

$$\boxed{\text{search for a better program for a powerful frozen machine}}$$

rather than

$$\boxed{\text{modify the machine itself.}}$$

27.2.5 Connection to compilation and compositionality

The distinction also connects to the compression-of-composition principle developed earlier in this book.

A useful computation may first appear explicitly as a prompt or multistep procedure,

$$p = (a_1, a_2, \ldots, a_T).$$

If the procedure is repeatedly useful, it can be stored in an external library.

At a slower timescale, repeated procedures may eventually be absorbed into future model weights through additional training.

Schematically,

new computation
$\Downarrow$
prompt or explicit procedure
$\Downarrow$
reusable external program
$\Downarrow$
compiled capability in future weights.

This suggests a continuum rather than a sharp distinction between external programs and internal parameters.

A computation can migrate between representational levels.

27.2.6 Frozen models and cultural accumulation

The interpretation becomes especially interesting when a frozen model can generate, evaluate, revise, and retain programs.

Let generation g of a system be described by

$$\mathcal{S}_g = (\theta, p_g, a_g, \mathcal{L}_g),$$

where

$$\theta$$

is permanently frozen while

$$\mathcal{L}_g$$

contains accumulated procedures.

A simple form of cultural improvement is

$$\mathcal{L}_g \longrightarrow \mathcal{L}_{g+1}$$

with

$$\theta_{g+1} = \theta_g = \theta.$$

The computational capacity of the interpreter has not changed.

But the effective system has changed because it now has access to better programs.

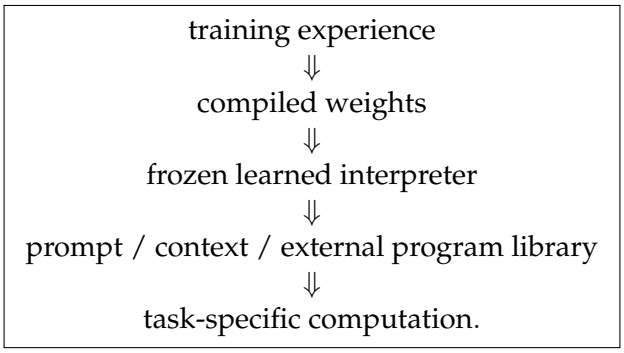
Thus

fixed neural parameters $\not\Longrightarrow$ fixed effective computational behavior.

The relevant state of an intelligent system may include not only its weights but also the procedures, examples, tools, and symbolic structures available to it.

27.2.7 The resulting picture

The analogy can therefore be summarized as



The physical computer remains the actual hardware.

But at the algorithmic level, the frozen LLM acts as a learned machine whose computational repertoire was constructed by training and whose behavior can subsequently be programmed through context.

The central distinction is therefore

> training changes the machine; prompting changes the program executed by the machine.

This provides a useful framework for thinking about post-training, in-context learning, tool use, prompt search, reusable procedures, and cumulative cultural improvement in systems whose neural weights remain fixed.

27.3 Hermite Staircases, Leap Complexity, and the Open Problem of General Compositional Recovery

The staircase mechanism developed in the preceding chapters should not be understood as a phenomenon restricted to Boolean functions.

For Boolean inputs, the natural orthogonal basis is the Fourier–Walsh basis. For isotropic Gaussian inputs, the corresponding basis is given by Hermite polynomials.

A substantial recent literature shows that the staircase phenomenon has a rigorous Gaussian/Hermite analogue. In particular, feature-learning dynamics can sequentially discover hidden directions according to the structure of the target's Hermite expansion [**abbe2023leap**, **chen2024meanfield**, **bietti2025gaussian**].

These results substantially sharpen the open question.

The problem is no longer simply

Can staircase learning occur for continuous inputs?

The more general problem is

How far do Fourier–Hermite staircase results extend from the discovery of linear latent directions to the recovery of general nonlinear sparse compositional computations?

27.3.1 Gaussian Multi-Index Models

A standard setting is the Gaussian multi-index model

$$X \sim \mathcal{N}(0, I_d), \qquad f^\star(X) = h^\star(U^\top X), \tag{27.4}$$

where

$$U \in \mathbb{R}^{d\times r}, \qquad U^\top U = I_r, \qquad r \ll d.$$

The target therefore depends on the ambient input only through the low-dimensional latent variable

$$Z = U^\top X \sim \mathcal{N}(0, I_r).$$

The statistical problem is to recover the relevant subspace, and eventually the low-dimensional link function

$$h^\star : \mathbb{R}^r \to \mathbb{R}.$$

This model already separates two different difficulties:

representation:	$f^\star$ depends only on an r-dimensional subspace,
recovery:	the learner must discover that subspace from data.

The Gaussian distribution makes the recovery problem particularly amenable to spectral analysis.

27.3.2 The Hermite Expansion

Let

$$\mathrm{He}_q(z)$$

denote the probabilists' Hermite polynomial of degree q, and define the normalized Hermite functions

$$H_q(z) = \frac{\mathrm{He}_q(z)}{\sqrt{q!}}.$$

For a multi-index

$$\alpha = (\alpha_1, \dots, \alpha_r) \in \mathbb{N}^r,$$

write

$$H_\alpha(Z) = \prod_{j=1}^{r} H_{\alpha_j}(Z_j). \tag{27.5}$$

The functions

$$\{H_\alpha\}_{\alpha \in \mathbb{N}^r}$$

form an orthonormal basis of

$$L^2(\mathcal{N}(0, I_r)).$$

Thus

$$h^\star(Z) = \sum_{\alpha \in \mathbb{N}^r} \widehat{h}_\alpha H_\alpha(Z), \tag{27.6}$$

with

$$\widehat{h}_\alpha = \mathbb{E}\left[h^\star(Z) H_\alpha(Z)\right]. \tag{27.7}$$

The Hermite coefficients therefore play, under Gaussian inputs, a role analogous to the Fourier coefficients of a Boolean function.

In the language of Chapter 6,

$$\widehat{h}_\alpha$$

is precisely a footprint associated with the probe

$$\phi_\alpha(Z) = H_\alpha(Z).$$

Thus the Hermite expansion gives an explicit family of low-complexity statistics through which hidden structure may become visible.

27.3.3 Degree Is Not the Whole Story

A fixed linear or kernel method is often controlled by the lowest nonvanishing Hermite degree.

If

$$\widehat{h}_\alpha = 0 \qquad \text{for all } |\alpha| < q,$$

then information about the target may first appear in degree q.

But feature learning can behave differently.

Once some relevant directions have been discovered, a higher-degree term may expose only a small number of *new* directions.

This motivates the notion of *leap complexity* [**abbe2023leap**].

27.3.4 Leap Complexity

Suppose, for simplicity, that the relevant part of the Hermite expansion has finite support

$$\mathcal{A} = \{\alpha : \widehat{h}_\alpha \neq 0\}.$$

Choose an ordering

$$\alpha^{(1)}, \alpha^{(2)}, \ldots, \alpha^{(M)}$$

of these nonzero multi-indices.

Let

$$I_{j-1} = \bigcup_{\ell<j} \operatorname{supp}\left(\alpha^{(\ell)}\right) \tag{27.8}$$

be the set of latent coordinates that have appeared previously.

The amount of new Hermite degree introduced by the jth term is

$$\Delta_j = \sum_{i \notin I_{j-1}} \alpha_i^{(j)}. \tag{27.9}$$

A convenient expression for the leap complexity in this coordinate system is then

$$\operatorname{Leap}(h^\star) = \min_{\text{orderings}} \max_j \Delta_j. \tag{27.10}$$

Thus leap measures not the total degree of a term, but the amount of *new* structure that must become visible at one stage.

For Boolean Fourier expansions, the exponents are effectively zero or one.

Then Equation (27.9) counts newly introduced variables, and leap one reduces to the merged-staircase type of structure studied in the Boolean setting.

The important conceptual distinction is therefore

total spectral degree $\neq$ largest new dependency exposed at one stage.

27.3.5 An Example

Consider

$$\begin{aligned} h(Z) = H_2(Z_1) + H_2(Z_1)H_3(Z_2) \\ + H_2(Z_1)H_3(Z_2)H_2(Z_3). \end{aligned} \tag{27.11}$$

The three terms have total degrees

$$2, \qquad 5, \qquad 7.$$

But in the displayed ordering, the newly introduced Hermite degrees are

$$2, \qquad 3, \qquad 2.$$

Hence the largest leap is

$$3,$$

even though the final term has degree seven.

The interpretation is

$$\boxed{\begin{array}{c} Z_1 \text{ becomes accessible} \\ \Downarrow \\ Z_2 \text{ can be exposed conditionally on } Z_1 \\ \Downarrow \\ Z_3 \text{ can be exposed conditionally on } Z_1, Z_2. \end{array}}$$

The spectral expansion therefore contains an ordering in which the learning problem can potentially proceed through smaller steps than its global degree would suggest.

27.3.6 Existing Gaussian Staircase Results

This picture is more than a heuristic.

Abbe, Boix-Adserà, and Misiakiewicz introduced leap complexity and analyzed neural-network learning for isotropic Boolean and Gaussian inputs [**abbe2023leap**].

For a class of Gaussian targets and a specified modified SGD procedure, they prove sequential learning of the relevant support through a saddle-to-saddle dynamics.

Their results go beyond the leap-one or merged-staircase case and provide a rigorous example in which training progressively reveals new coordinates.

The full conjecture relating general SGD complexity to leap remains open.

Chen and Ge study subspace-sparse polynomial targets under Gaussian input using mean-field dynamics [**chen2024meanfield**].

They obtain a rotation-aware formulation of the recovery obstruction and give a necessary condition together with a somewhat stronger sufficient condition for successful learning.

This is important because the coordinate axes used in a Hermite expansion are not intrinsic under an isotropic Gaussian distribution.

Bietti, Bruna, and Pillaud-Vivien study Gaussian multi-index models under a two-timescale population gradient flow [**bietti2025gaussian**].

They obtain global convergence and a sequence of saddle-to-saddle transitions whose timescales are characterized by the Hermite structure of the target.

Finally, related work on the *generative leap* uses Hermite tensors to characterize the complexity of efficient hidden-subspace recovery in Gaussian multi-index models [**damian2025generative**].

These results establish an important point:

the staircase principle already has nontrivial rigorous continuous and Gaussian realizations.

27.3.7 Leap Complexity Is Not Compositional Sparsity

The Gaussian results also clarify an important distinction for the framework of this book.

Low compositional arity does not imply low leap.

Consider

$$f(X) = \prod_{i=1}^{m} X_i, \qquad X_i \sim \mathcal{N}(0,1). \tag{27.12}$$

The function has an elementary binary compositional realization:

$$z_1 = X_1 X_2,$$

$$z_2 = z_1 X_3,$$

$$\dots$$

$$f = z_{m-1} X_m.$$

Thus

$$\mathrm{arity}(G) = 2$$

and the graph has only

$$m - 1$$

internal multiplication nodes.

Yet in the canonical Gaussian coordinates,

$$f(X) = H_1(X_1) \cdots H_1(X_m)$$

is a single Hermite monomial involving all m coordinates.

With no previously exposed coordinate, its canonical-basis leap is

$$m.$$

Hence

small compositional arity $\not\Longrightarrow$ small Hermite leap in the original coordinates.

This example is fundamental.

C1 says that the computation possesses an economical low-arity representation.

Leap describes how difficult that structure is to expose through a particular spectral representation and learning mechanism.

The two concepts therefore address different questions:

C1:	How simple can the computation be represented?
Hermite leap:	How much new structure is exposed at one spectral learning step?

27.3.8 The Coordinate Problem

Hermite leap is itself representation dependent.

For Gaussian inputs, an orthogonal change of latent coordinates

$$Z' = RZ, \qquad R^\top R = I,$$

preserves the distribution

$$Z' \sim \mathcal{N}(0, I_r),$$

but generally changes the Hermite expansion of the target.

Thus

$$\mathrm{Leap}(h^\star)$$

can depend on the chosen axes.

This is closely related to the broader coordinate problem for sparse compositionality:

sparsity and staircase accessibility are properties of a representation, not of a function in isolation.

The Gaussian setting is particularly valuable because rotational symmetry makes this dependence mathematically explicit.

Abbe and collaborators discuss a rotation-aware or isotropic version of leap complexity, while other work seeks basis-free characterizations of subspace recoverability [**abbe2023leap**, **chen2024meanfield**].

This suggests a deeper problem for the present framework.

A useful representation should be judged jointly by

compositional complexity, spectral accessibility, cost of the coordinate transformation.

One cannot allow an arbitrarily complicated change of coordinates to make the target superficially simple without charging for that transformation.

27.3.9 Open Problem 1: From Hermite Leap to General Compositional DAGs

The Gaussian multi-index problem asks the learner primarily to discover a low-dimensional *linear* latent subspace.

A general sparse compositional computation is substantially richer.

Suppose

$$z_v = g_v\left(z_{\mathrm{pa}(v)}\right), \qquad |\,\mathrm{pa}(v)| \le k. \tag{27.13}$$

Learning one constituent may create a new nonlinear coordinate

$$z_v = g_v(\cdot)$$

that did not previously exist in the learner's representation.

The next dependency may be high-order in the original variables but low-order in the enlarged coordinates.

Thus the desired staircase is

detect a footprint in the current coordinates
$\Downarrow$
learn a nonlinear constituent
$\Downarrow$
install its output as a new coordinate
$\Downarrow$
change the spectral structure of the remaining problem
$\Downarrow$
repeat.

This is more general than recovering a fixed linear subspace.

The central open question is therefore:

What is the analogue of leap complexity when the relevant coordinates are themselves nonlinear functions that must be learned?

27.3.10 Open Problem 2: An Adaptive Spectral Staircase

At stage j, let

$$H_j(X)$$

denote the representation currently available to the learner.

Let

$$R_j(X, Y)$$

denote the residual signal associated with the unresolved part of the target.

A general staircase theory should identify a family of accessible probes

$$\Phi_j = \Phi(H_j)$$

and establish that some probe satisfies

$$|\mathbb{E}\left[R_j(X,Y)\phi(H_j(X))\right]| \geq \frac{1}{\mathrm{poly}(d,s,L)} \tag{27.14}$$

for some

$$\phi \in \Phi_j.$$

For the initial Gaussian multi-index problem,

$$\Phi_0$$

may be a family of low-degree Hermite polynomials.
After a nonlinear coordinate has been learned, however,

$$H_j(X)$$

will generally no longer be Gaussian.
The original Hermite basis is then no longer the orthogonal basis naturally associated with the new representation distribution.
This suggests an adaptive sequence

$$\boxed{\Phi_0 \longrightarrow \Phi_1 \longrightarrow \Phi_2 \longrightarrow \cdots}$$

in which the accessible spectral probes change as the representation changes.
Possible mathematical tools include

orthogonal polynomials for the current distribution,

Stein operators,

score functions,

higher-order cumulants,

and

learned low-complexity probe families.

The challenge is to define such probe classes without making them as hard to discover as the original hidden computation.

27.3.11 Open Problem 3: A Coordinate-Invariant or Coordinate-Penalized Leap

Because leap is coordinate dependent, one would like a complexity notion that allows useful changes of representation without hiding arbitrary complexity in the coordinate map.
Let

$$\Psi$$

be a restricted family of feasible transformations.

A schematic intrinsic complexity might take the form

$$\mathcal{L}_{\text{intrinsic}}(f) = \inf_{\psi \in \Psi} \left[C(\psi) + \text{Leap}\left(f \circ \psi^{-1} \right) \right], \tag{27.15}$$

or, for dynamical systems,

$$\mathcal{L}_{\text{intrinsic}}(F) = \inf_{\psi \in \Psi} \left[C(\psi) + \text{Leap}\left(\psi \circ F \circ \psi^{-1} \right) \right]. \tag{27.16}$$

The term

$$C(\psi)$$

must charge for the complexity of the representation change.
Otherwise all of the computational difficulty could simply be hidden in ψ.
The open problem is to find a definition that is simultaneously

representation aware,	computationally meaningful,	compatible with learning dynamics.

This question connects naturally to uncertainty-principle ideas: a representation that is sparse in one coordinate system can become dense in a sufficiently incompatible one.

The desired theory would extend this principle from coefficient sparsity to *compositional and recovery sparsity*.

27.3.12 Open Problem 4: Beyond Gaussian and Product Measures

Hermite analysis is especially powerful because Gaussian measure supplies a canonical orthogonal polynomial basis and strong identities such as Stein's formula.

Real learning problems need not have Gaussian inputs.
For a distribution

$$X \sim P,$$

one would like to identify an analogue of the low-order Hermite footprint

$$\widehat{h}_\alpha = \mathbb{E}[h(X) H_\alpha(X)].$$

The relevant question is not merely whether an orthogonal basis exists in

$$L^2(P).$$

One needs probes that are

low complexity + statistically estimable + accessible to the learning dynamics.

A distribution-dependent theory might use orthogonal polynomials when available.
A more intrinsic theory might instead use Stein operators or score functions.
The open question is:

What replaces Fourier–Hermite degree and leap for general input distributions while retaining a connection to feasible optimization?

27.3.13 Open Problem 5: From Population or Modified Dynamics to End-to-End SGD

Existing staircase results necessarily analyze controlled settings.

Depending on the result, these may involve

two-layer architectures,

mean-field limits,

population gradient flow,

two-timescale learning,

or

modified or stagewise SGD.

These results establish that staircase dynamics are mathematically real.

They do not yet imply that ordinary finite-sample end-to-end SGD in a deep network will recover a general hidden compositional computation.

A stronger theorem would begin with

$$(X_i, Y_i)_{i=1}^n$$

and a finite deep architecture, and prove that the learned representations

$$H_0, H_1, \dots, H_L$$

progressively expose the target computation with

$$n \le \text{poly}\left(d, s, L, \varepsilon^{-1}\right)$$

and polynomial computational cost.

The proof would have to control simultaneously

finite-sample fluctuations,
finite width,
optimization noise,
approximate rather than exact recovered coordinates,
interference between simultaneously learned constituents,
preservation of previously learned variables.

27.3.14 Open Problem 6: Relating Leap to Compositional Arity

The product example Equation (27.12) shows that compositional arity and Hermite leap can differ dramatically.

This raises a basic structural question.

Suppose a target has a compositional representation

$$f = \Phi_G(\{g_v\}), \qquad |\operatorname{pa}(v)| \le k.$$

Under what additional conditions does this imply the existence of some representation or probe family in which the effective leap is also bounded by a function of k?

One might hope for a relation such as

$$\operatorname{Leap}_{\text{effective}}(f) \le \Psi\left(k, \text{depth}, \text{constituent regularity}, \text{coordinate complexity}\right), \tag{27.17}$$

but no such implication follows from bounded arity alone.

Additional assumptions might involve

genericity of constituent coefficients,

nondegeneracy,

availability of intermediate variables,

appropriate changes of coordinates,

or

distributional asymmetry.

Understanding this relationship would connect the representation principle C1 directly to the recovery principle C2.

27.3.15 Relation to C1 and C2

The Fourier–Hermite results clarify the division of labor between the two central principles of the book.

C1 concerns representation:

Does an economical sparse compositional computation exist?

Hermite and leap analyses concern accessibility:

How much of that hidden structure becomes visible through low-complexity statistics in the current coordinate

Thus

compositional sparsity $\neq$ low Hermite degree $\neq$ low leap.

They measure different aspects of the problem.

The desired general theory should explain how they interact:

sparse compositional representation
$\Downarrow$
some constituent leaves a detectable footprint
$\Downarrow$
feature learning creates a new coordinate
$\Downarrow$
the effective leap of the remaining computation decreases
$\Downarrow$
the process repeats.

27.3.16 The Revised Staircase Question

The existing Boolean and Gaussian results therefore suggest a more precise form of the central open problem.

It is not

Can staircase learning occur?

We already know that it can in important model classes.
The stronger question is

General Compositional Staircase Problem. Starting from the existing Fourier–Hermite staircase and leap results, identify conditions under which a feasible learning algorithm recovers a general bounded-arity nonlinear computation by successively constructing intermediate coordinates, with polynomial sample and computational complexity and with controlled accumulation of representation error.

The conceptual progression is therefore

Fourier staircase $\longrightarrow$ Hermite leap $\longrightarrow$ adaptive nonlinear compositional staircase.

The first two stages now have substantial theory.
The third remains one of the central open problems of the framework.

CHAPTER 28

Five Open Problem Directions: Representation, Recovery, and Selection

TO BE DELETED PROBABLY

The first seven chapters distinguish the existence of a compact computation from the possibility of discovering it from observations. The following five problems develop that distinction into a research program, ordered by their foundational importance to the framework.

The first two sharpen the representation questions arising from C1. The third concerns quantitative C2. The fourth asks whether learning can construct a compositional staircase. The fifth concerns the alignment of regularized selection with useful structure.

These are proposed research formulations, not five claims that the existing literature contains no relevant results. In particular, stability and approximation are already understood for important Mhaskar–Poggio compositional classes; staircase recovery has been proved in restricted models; and norm-based generalization bounds already give a substantial explanation of why a well-fitting, low-norm predictor can generalize. The questions below concern what remains beyond these results.

Conventions. We write d for input dimension, n for sample size, s for the number of constituent nodes, L for computational depth, k for maximum local arity, and $r \geq 1$ for an integer smoothness order. Unless otherwise specified, approximation is uniform on a compact domain, and statistical statements concern independent examples from a fixed distribution P. Constants and resource bounds must be uniform in whichever of these quantities are allowed to grow.

28.1 Problem 1: Characterize Economical Compositional Representations

Origin in the book. Chapters 1–3 distinguish computational expressivity from learnability. Chapter 4 gives two complementary starting points: efficient digital computation has bounded-fan-in circuit realizations, and regular low-arity compositions admit favorable deep approximation. The remaining representation question is to recognize economical compositional structure without already being given its decomposition.

28.1.1 What is already established

Consider a scalar constituent $g : [-1,1]^m \to [-1,1]$, $m \le k$, in a normalized smooth class satisfying

$$\|g\|_\infty + \sum_{1 \le |\alpha| \le r} \|D^\alpha g\|_\infty \le 1. \tag{28.1}$$

This is the bounded-derivative setting relevant to the Mhaskar–Poggio approximation argument, rather than an arbitrary $W^{r,p}$ assumption [74].

In particular,

$$\sum_{j=1}^{m} \|\partial_j g\|_\infty \le 1 \quad \Longrightarrow \quad |g(u) - g(v)| \le \|u - v\|_\infty.$$

For a layered DAG with such constituents, local uniform errors of at most δ give

$$e_\ell \le e_{\ell-1} + \delta, \qquad e_L \le L\delta,$$

provided all evaluated intermediate arguments remain in the domains of the local guarantees.

Thus stability is not an unanswered question for this class. More general good-propagation-of-errors results extend the approximation argument to suitable DAGs [72].

For constituent classes and approximators satisfying the usual local rate, the resulting complexity is schematically

$$N(\varepsilon) \lesssim s \left(\frac{A_G}{\varepsilon} \right)^{k/r},$$

up to the factors specific to the approximation theorem.

Here A_G controls propagation of local errors, and $A_G \le L$ in the normalized nonexpansive setting above.

The unresolved issue is not to reprove these statements, but to characterize targets that admit an economical representation of this kind.

The uniform Boolean-circuit characterization is already available; the goal here is an analytic and structural criterion that says more about approximation, interfaces, and constituent complexity.

28.1.2 A formal representation problem

Fix admissible constituent classes $\mathcal{G}_{r,m}(B)$, including specified domains, ranges, and derivative bounds.

Define

$$E_{s,L}^{k,r,B}(f) = \inf_{\substack{|G| \le s,\ \operatorname{depth}(G) \le L \\ |\operatorname{pa}(v)| \le k \\ g_v \in \mathcal{G}_{r,|\operatorname{pa}(v)|}(B)}} \|f - \Phi_G\|_\infty. \tag{28.2}$$

The graph and the constituents are both variable in this infimum.

For a family f_d, ask when

$$E_{s,L}^{k,r,B}(f_d) \le \varepsilon$$

can be achieved with bounded k and controlled s, L, B as d grows.

Node count alone is insufficient if an arbitrarily complicated local function can be inserted at no cost.

For effective representations, fix a decoding language for finite descriptions of the graph, the local approximators, their numerical parameters, and any shared library.

For an encoded realization $\mathcal{R}$, let

$$B(\mathcal{R}) = \text{total description length},$$

and

$$T_{\mathcal{R}}(p) = \text{worst-case bit cost of evaluation to } 2^{-p}.$$

A resource-sensitive refinement is

$$\mathcal{C}_k(f, \varepsilon; \tau) = \inf_{\mathcal{R}} \left\{ B(\mathcal{R}) : \begin{array}{l} \|f - f_{\mathcal{R}}\|_\infty \le \varepsilon, \\ \operatorname{arity}(\mathcal{R}) \le k, \\ T_{\mathcal{R}}(\lceil \log_2(1/\varepsilon) \rceil + 1) \le \tau \end{array} \right\}. \tag{28.3}$$

The decoder is fixed independently of f; it cannot hide an arbitrary function or a noncomputable constant inside a supposedly free primitive.

Open problem. Find structural sufficient conditions, and informative partial converses, for small $E_{s,L}^{k,r,B}(f)$ or small $\mathcal{C}_k(f, \varepsilon; \tau)$ that do not merely restate the existence of a suitable graph.

Two resource regimes must remain separate:

$$\operatorname{poly}(d, \varepsilon^{-1}) \qquad \text{and} \qquad \operatorname{poly}(d, \log(1/\varepsilon)).$$

A Sobolev approximation rate need not give polynomial-time bit computability.

Computability also requires effective construction of the approximants and an effective convergence rate. The existence of a separate code at every accuracy is not, by itself, a uniform algorithm.

Conversely, efficient digital evaluation supplies its own route through uniform bounded-fan-in circuits and, with appropriate encodings, discrete state restoration.

Nor should amplification A_G be confused with the number of precision bits: a local tolerance $\delta = \varepsilon / A_G$ requires approximately

$$p_{\text{local}} = \log_2(1/\varepsilon) + \log_2 A_G$$

bits.

Exponential amplification in depth can therefore require only linearly many additional bits, even though a generic approximation scheme costing $\delta^{-k/r}$ may then become expensive.

28.1.3 Possible approaches

Inverse approximation theorems. For a specified depth schedule $L(s)$, study the approximation class

$$\mathcal{A}_\alpha = \left\{ f : \sup_{s \ge 1} s^\alpha E_{s,L(s)}^{k,r,B}(f) < \infty \right\}.$$

The goal is to relate membership to recognizable properties of f: hierarchical interaction structure, bottleneck dimensions, symmetries, or recursive separability.

Direct approximation theorems provide one direction; useful converse statements would make the class intrinsic.

Differential signatures of bottlenecks. Suppose, on a product domain,

$$f(x_A, x_B) = H(g(x_A), h(x_B)),$$

where g and h are scalar and all functions are twice differentiable.

Direct differentiation gives

$$D_A D_B f = H_{12}(g(x_A), h(x_B)) \nabla g(x_A) \nabla h(x_B)^\top,$$

so this mixed-derivative matrix has rank at most one.

Such identities provide necessary signatures of particular hierarchical bottlenecks.

A promising restricted problem is to identify additional nondegeneracy and integrability conditions under which a hierarchy of these signatures is also sufficient for a compositional factorization.

Low rank at one point is not enough.

Separate reuse from execution size. Compare s unrelated constituents with s calls to a small library of $K \ll s$ functions.

A description should pay for each distinct function once, while execution cost still counts every call.

This permits a theory of compositional approximation that recognizes both small graphs and short programs generating larger graphs.

A useful first result. A characterization for a restricted family—for example, binary trees of nondegenerate smooth scalar bottlenecks—would already be valuable.

A universal procedure for finding the shortest program is not required.

28.2 Problem 2: Which Coordinates Expose Compositional Structure?

Origin in the book. Chapter 4 distinguishes the existence of a sparse realization from sparse weights and unique latent coordinates.

Chapters 6 and H then propose that learning progressively constructs useful coordinates.

Before studying their discovery, we need to specify what makes a coordinate system structurally useful.

28.2.1 Local dependence is coordinate dependent

For a differentiable state transition $F : \mathcal{X} \to \mathcal{X}$, define its direct coordinate-dependency width by

$$k_{\text{coord}}(F) = \max_i \#\{j : \partial_j F_i \not\equiv 0\}.$$

For an invertible change of coordinates $u = \psi(z)$, the transition becomes

$$F^\psi = \psi \circ F \circ \psi^{-1}. \tag{28.4}$$

There is no reason for $k_{\text{coord}}(F^\psi)$ to equal $k_{\text{coord}}(F)$.

For example, $F(z) = Dz$ with diagonal D has one-parent coordinate updates. Under a dense invertible linear change $u = Az$,

$$F^A(u) = ADA^{-1}u,$$

which generally has dense direct dependencies.

This does not mean that a polynomial-size bounded-fan-in circuit has ceased to exist: the dense linear maps themselves admit polynomial-size arithmetic circuits.

The issue is which coordinates expose the desired small interfaces, reusable constituents, and economical local description.

28.2.2 Charge for the coordinate transformation

Let Ψ be a specified class of effectively represented invertible maps, with controlled evaluation costs and conditioning, for example

$$\mathrm{Lip}(\psi) \le \kappa, \qquad \mathrm{Lip}(\psi^{-1}) \le \kappa.$$

One possible intrinsic cost is

$$\mathcal{C}_{\mathrm{coord}}(F,\varepsilon) = \inf_{\psi\in\Psi,\,\widetilde{F}} \Big\{ B(\psi) + B(\psi^{-1}) + B(\widetilde{F}) : \\ \|F - \psi^{-1}\circ\widetilde{F}\circ\psi\|_\infty \le \varepsilon \Big\}, \tag{28.5}$$

where $\widetilde{F}$ must satisfy the prescribed compositional and resource constraints.

The cost of ψ cannot be omitted. Otherwise a difficult computation could simply be relocated into the encoder or decoder.

For a scalar prediction target, the analogous representation is $f = q \circ \psi$, not a conjugacy; conjugation applies to state transitions whose input and output spaces match.

Open problem. Characterize admissible coordinate families that expose useful sparse structure, and establish when the resulting representation is identifiable up to an explicitly stated equivalence.

A natural initial equivalence class consists of coordinate permutations and componentwise invertible transformations:

$$\psi_i(z) = \psi_i^{\mathrm{scalar}}(z_{\pi(i)}).$$

These preserve the dependency graph up to relabeling when domains and conditioning are controlled.

Arbitrary dense nonlinear transformations need not preserve it.

28.2.3 Possible approaches

Begin with simultaneously structured transitions. A single map can admit many economical coordinate systems.

A family $\{F_a\}_{a\in\mathcal{A}}$, sharing the same latent variables but expressing different transitions, imposes stronger constraints:

$$F_a^{\psi} = \psi \circ F_a \circ \psi^{-1} \quad \text{should be sparse for every } a.$$

In the linear case, simultaneous diagonalization of a suitably nondegenerate family provides a tractable starting example.

The research question is what survives for nonlinear bounded-arity dynamics.

Use multiple derivative statistics. The Jacobian transforms as

$$DF^{\psi}(u) = D\psi(F(z))\, DF(z)\, D\psi^{-1}(u), \qquad z = \psi^{-1}(u).$$

This suggests searching for coordinates that simplify several Jacobians or higher-order derivative tensors simultaneously, rather than relying on one averaged matrix.

The AGOP discussion in Chapter 6 already explains why a single second-order statistic is not a general inverse map to a hidden DAG.

Exploit changes of environment. If different tasks or interventions modify only a few latent mechanisms, the same coordinate system should localize those changes.

One can ask whether invariance of the other mechanisms restricts the admissible coordinate ambiguity more strongly than prediction on one distribution.

Any claim of causal identification requires its own assumptions.

Separate representational and statistical theorems. First characterize the coordinate system assuming that the family of maps $\{F_a\}$ is known exactly.

Then ask how many observations are needed to estimate it.

This prevents a representation theorem from silently assuming the statistical recovery result it is meant to support.

A useful first result. For a restricted nonlinear family, prove that all sufficiently economical representations are related by a controlled graph-preserving equivalence, or exhibit the additional ambiguities that necessarily remain.

Uniqueness of literal neural coordinates is not the appropriate target.

28.3 Problem 3: Quantitative Genericity and Detectable Footprints

Origin in the book. Chapter 6 defines genericity relative to $(f, \mathsf{P}, \mathcal{F}_{\mathrm{probe}})$ and separates statistical sufficiency from algorithmic recoverability.

The crucial question is not merely whether a footprint is nonzero, but whether a feasible learner can detect a useful footprint at every required stage.

28.3.1 A finite-sample formulation

Let $H_j(X)$ be the representation available after stage j.

Let $R_j(X, Y)$ be a specified residual or learning signal, and let Φ_j be a family of normalized accessible probes.

Define

$$\rho_{j,\phi} = \mathbb{E}_{\mathsf{P}}\left[R_j(X,Y)\phi(H_j(X))\right], \qquad \rho_{j,\star} = \max_{\phi\in\Phi_j} |\rho_{j,\phi}|. \tag{28.6}$$

Normalization is essential: arbitrarily rescaling a probe must not count as making the signal more detectable.

For a fixed finite probe family, suppose

$$|R_j(X,Y)\phi(H_j(X))| \le B_Z.$$

Hoeffding's inequality and a union bound imply that

$$n \geq \frac{8B_Z^2}{\rho_{j,\star}^2} \log \frac{2|\Phi_j|}{\delta}$$

is sufficient to estimate every footprint to error at most $\rho_{j,\star}/2$ with probability at least $1-\delta$.

This is a sufficient uniform screening bound, not a universal lower bound on the sample complexity of all learning algorithms.

With variance control, a Bernstein-type bound instead has the form

$$n \gtrsim \left(\frac{v_Z}{\rho_{j,\star}^2} + \frac{B_Z}{\rho_{j,\star}} \right) \log \frac{|\Phi_j|}{\delta}.$$

Thus the variance and probe normalization matter, not just the raw correlation magnitude.

The logarithm is a statistical cost. Enumerating an exponentially large Φ_j can still require exponential computation even when $\log |\Phi_j|$ is only polynomial.

A complete result must control both sample cost and the cost of finding a useful probe.

28.3.2 The desired quantitative statement

Let ω denote a perturbation of the observation model, such as a shift, bias, or controlled change of distribution.

Seek conditions giving

$$\Pr_{\omega} \left[\min_{0 \leq j < s} \max_{\phi \in \Phi_j^{\text{useful}}} |\rho_{j,\phi}(\omega)| \geq \frac{1}{\text{poly}(d, s, L)} \right] \geq 1 - \delta_0. \tag{28.7}$$

Here "useful" must be defined relative to progress toward an unresolved constituent, not merely correlation with an easy nuisance variable.

The learner must also have an efficient way to find or exploit such a probe.

Defining the useful family using the hidden teacher is permissible for analysis, but supplying that family to the learner would change the learning problem.

Open problem. Find broad but explicit target, distribution, and probe classes for which Equation (28.7) holds, and determine how the bound changes after approximate rather than exact recovery of earlier constituents.

28.3.3 Possible approaches

Quantify analytic symmetry breaking. If a shift footprint $a_\phi(b)$ is analytic and not identically zero, its zero set has measure zero on a connected open parameter domain.

This establishes nonvanishing, not a usable lower bound.

For polynomial footprints of degree at most q, Gaussian anti-concentration gives estimates of the form

$$\Pr\left[|a_\phi(\omega)| \leq \eta \|a_\phi\|_{L^2}\right] \leq C_q \eta^{1/q},$$

with degree-dependent constants [**carbery2001distributional**].

For a prescribed sequence of reference stages, if one fixed useful candidate at each of s stages has $\|a_\phi\|_{L^2} \geq \sigma_0$, a union bound gives the lower scale

$$\rho_{\min} \geq \sigma_0 \left(\frac{\delta_0}{C_q s} \right)^q.$$

This is polynomially useful when q is bounded and σ_0 has an inverse-polynomial lower bound.

The substantive work is to establish those two properties for the hierarchy and to make them robust to learning error.

Use parity as a quantitative stress test. Let $X_i \in \{-1, +1\}$ be independent with $|\mu_i| < 1$, where $\mu_i = \mathbb{E}[X_i]$, and let

$$f_S(X) = \prod_{i \in S} X_i, \qquad \phi_j(X) = \frac{X_j - \mu_j}{\sqrt{1 - \mu_j^2}}.$$

For $j \in S$, direct calculation gives

$$\mathbb{E}[f_S(X)\phi_j(X)] = \sqrt{1 - \mu_j^2} \prod_{i \in S \setminus \{j\}} \mu_i.$$

Bias can make a footprint nonzero while leaving it exponentially small in $|S|$.

Therefore "symmetry is broken" is weaker than quantitative C2.

This example concerns the specified probes; it is not a lower bound for all algorithms learning noiseless parity.

Control adaptivity. When H_j and R_j are learned on the same data used to evaluate the footprints, fixed-probe concentration does not apply automatically.

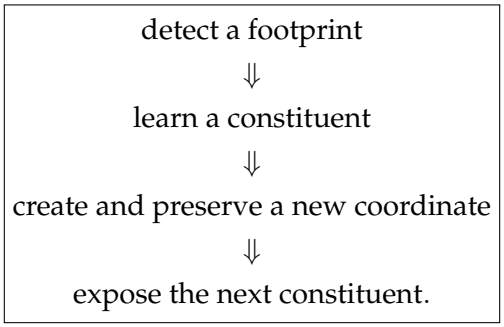
Possible routes include sample splitting, fresh samples at each stage, or uniform bounds over the whole adaptive representation/probe class.

The cost of repeated stages must be included.

A useful first result. Prove a polynomial lower bound for a family of bounded-degree compositional targets under a specified perturbation law, together with an efficient screening procedure and a stability bound for footprints when previously learned coordinates are imperfect.

28.4 Problem 4: Prove a Compositional Staircase Recovery Theorem

Origin in the book. This is the central conjecture of Chapter H:



The desired result must explain the actual feature-learning dynamics, not merely the existence of an accurate deep network.

28.4.1 What a theorem should establish

Let the teacher be a DAG with latent variables

$$z_v = g_v(z_{\mathrm{pa}(v)}), \qquad |\,\mathrm{pa}(v)| \le k,$$

and output $Y = f^\star(X)$, initially without label noise.

The learner receives only (X_i, Y_i), not the internal z_v.

Choose a specified finite-width architecture, initialization law, optimizer, and training schedule.

Designate bounded-dimensional feature slots in the learner to prevent the recovery criterion from being satisfied merely by an unrestricted decoder operating on the full input.

For an admissible node matching π and a controlled equivalence class $\mathcal{E}_v$, define, for example,

$$\mathcal{D}_{\mathrm{str}}(\widehat{z}, z) = \inf_{\pi,\, \psi_v \in \mathcal{E}_v} \max_v \left\| \widehat{z}_{\pi(v)} - \psi_v(z_v) \right\|_{L^2(\mathsf{P})}. \tag{28.8}$$

The equivalences have fixed complexity and conditioning bounds, and the matching must respect the allowed dependency structure.

A target theorem would give, with probability at least $1 - \delta$,

$$\|f_{\widehat{\theta}} - f^\star\|_{L^2(\mathsf{P})} \le \varepsilon, \qquad \mathcal{D}_{\mathrm{str}}(\widehat{z}, z) \le \varepsilon,$$

after sample size, arithmetic cost, and network size bounded by

$$\mathrm{poly}\left(d, s, L, \varepsilon^{-1}, \rho_{\min}^{-1}, \kappa, \log(1/\delta)\right). \tag{28.9}$$

Here κ collects explicitly specified conditioning parameters.

Problem 3 must ensure that $\rho_{\min}^{-1}$ is itself polynomial in the problem size.

A theorem polynomial in an exponentially large inverse signal would not establish efficient learning.

28.4.2 What is already known

The staircase idea already has rigorous special cases.

Abbe and collaborators prove hierarchical learning of suitable Boolean staircase functions with a specified layerwise gradient-based algorithm [**abbe2021staircase**].

Related merged-staircase results characterize learning in a two-layer mean-field setting [**abbe2022merged**].

The open target is an extension to specified classes of hidden smooth constituents and more general DAGs, with simultaneous control of finite samples, finite width, approximate earlier features, and the actual optimization dynamics.

It is not the claim that no staircase-learning theorem exists.

28.4.3 Possible approaches

Derive the accessible probes from the gradient. For a stagewise model

$$f_w(X) = f_0(X) + a\sigma(w^\top H_j(X)),$$

with loss

$$\mathcal{L}(w) = \frac{1}{2}\mathbb{E}[(f_w(X) - Y)^2],$$

where H_j, f_0, a are fixed during this calculation,

$$\nabla_w \mathcal{L} = a\,\mathbb{E}\left[(f_w(X) - Y)\sigma'(w^\top H_j(X))H_j(X)\right].$$

The gradient accesses particular gated correlations.

An arbitrary nonzero probe correlation need not align with these directions.

A proof should identify the effective probe family of the chosen architecture, rather than assume every useful statistic is available.

Prove a stagewise alignment inequality. Let $\mathcal{E}_j(\theta)$ measure the current constituent error in a fixed local parameterization.

A useful intermediate target is

$$\langle \nabla_\theta \mathcal{L}(\theta), \nabla_\theta \mathcal{E}_j(\theta) \rangle \geq c\rho_j^2 \mathcal{E}_j(\theta)$$

through a specified region of parameter space.

Under gradient flow, this would give

$$\frac{d}{dt}\mathcal{E}_j(\theta_t) \leq -c\rho_j^2 \mathcal{E}_j(\theta_t).$$

The challenge is to derive the alignment inequality from structural and statistical assumptions, not to assume it as a restatement of recovery.

A finite-step SGD theorem must additionally control variance and escape from the region where the estimate holds.

Close an induction over the hidden computation. After recovering one constituent, prove that it is retained and that the next footprint remains detectable.

An induction may require an estimate of the form

$$e_{j+1} \leq a_j e_j + b_j \varepsilon_{\mathrm{stat}} + c_j \varepsilon_{\mathrm{opt}},$$

with controlled products of the a_j and explicit statistical and optimization errors.

Residual connections or temporarily frozen learned modules offer tractable starting architectures; they do not themselves prove the induction.

Use C3 to separate the sources of difficulty. Begin with local input–output pairs supplied, then progressively hide intermediate variables.

Norm-controlled RKHS constituents are useful here: local kernel ridge regression is convex once its inputs and targets are observed, whereas joint hidden composition remains nonlinear.

Deep representer theorems give a finite-dimensional formulation without solving that optimization problem [**bohn2019representer**].

This also creates a route to partial-trace results: identify which exposed nodes suffice to initialize a staircase that subsequently continues without complete internal supervision.

A useful first result. Prove the full induction for a depth-growing family of trees with smooth, nondegenerate, bounded-arity constituents under a specified distribution.

A staged algorithm would be a meaningful first theorem.

Extending it to simultaneous end-to-end SGD would be a separate advance.

28.5 Problem 5: When Does Norm-Based Selection Favor Reusable Structure?

Origin in the book. Chapters 5 and 7 distinguish reaching a fitting solution, preferring a low-complexity solution, and selecting a useful compositional realization.

Our discussion narrows this question: product-norm control already supplies a rigorous route to capacity control.

What remains is its efficient optimization and its alignment with reusable structure.

28.5.1 The established generalization mechanism

For a fixed-depth, bias-free ReLU network, define

$$\Pi(W) = \prod_{\ell=1}^{L} \|W_\ell\|_F, \qquad \mathcal{F}_Q = \{f_W : \Pi(W) \le Q\}.$$

For $\|X\|_2 \le B_X$, a standard Frobenius-norm bound yields

$$\widehat{\mathfrak{R}}_S(\mathcal{F}_Q) \le \frac{B_X Q}{\sqrt{n}} \left(\sqrt{2L \ln 2} + 1\right), \tag{28.10}$$

under the corresponding architecture and activation assumptions [**golowich2020size**].

Positive homogeneity permits balancing layer norms, relating the product-constrained class to the layerwise norm formulation of the bound.

Thus lowering an admissible product bound lowers the associated capacity bound.

For a loss $0 \le \ell \le 1$, define

$$\widehat{R}_S(f) = \frac{1}{n} \sum_{i=1}^{n} \ell(f(X_i), Y_i), \qquad R(f) = \mathbb{E}[\ell(f(X), Y)].$$

If the loss is K_ℓ-Lipschitz in predictions, standard Rademacher arguments control population risk by empirical risk plus this complexity term.

Squared loss requires bounded predictions and labels, or an appropriate clipping convention.

The relation among normalization, norm product, margins, and generalization is also developed by Xu and collaborators [117].

This is a known positive mechanism, not an unexplained part of the agenda.

28.5.2 An elementary conditional guarantee

Suppose

$$J_\lambda(W) = \widehat{R}_S(f_W) + \lambda \Pi(W)^2,$$

with $\lambda > 0$ fixed in advance.

Assume a fixed comparator $W^\star$ has zero population loss and $\Pi(W^\star) \le Q_\star$.

If optimization achieves

$$J_\lambda(\widehat{W}) \le J_\lambda(W^\star) + \xi,$$

then nonnegativity of the loss gives

$$\widehat{R}_S(f_{\widehat{W}}) \le \lambda Q_\star^2 + \xi, \qquad \Pi(\widehat{W}) \le Q_\xi := \sqrt{Q_\star^2 + \xi/\lambda}. \tag{28.11}$$

For a prescribed optimization tolerance ξ, the radius Q_ξ is sample independent.

Applying the uniform bound to this fixed class gives, with probability at least $1-\delta$,

$$R(f_{\widehat{W}}) \le \lambda Q_\star^2 + \xi + \frac{2K_\ell B_X Q_\xi}{\sqrt{n}}\left(\sqrt{2L\ln 2}+1\right) + 3\sqrt{\frac{\ln(2/\delta)}{2n}}. \tag{28.12}$$

This deduction does not assume structural recovery.

It explains prediction generalization once a sufficiently favorable objective value has been reached.

It is not a proof that arbitrary SGD reaches that value.

For binary exact interpolation $f_W(x_i) = y_i \in \{-1,+1\}$, the normalized training margins satisfy

$$\frac{y_i f_W(x_i)}{\Pi(W)} = \frac{1}{\Pi(W)}.$$

Minimum product therefore maximizes these normalized margins within the exactly interpolating set.

For a penalized objective at finite λ, empirical fit and product norm are instead traded against one another.

A smaller bound does not necessarily order the actual risks of every pair of trained predictors.

28.5.3 The remaining structural question

Let $\mathcal{S}_\varepsilon$ be the set of learned networks that implement the useful compositional structure to error ε, up to the specified equivalence of Problem 2.

Ask when regularized training gives

$$\Pr\left[\widehat{W} \in \mathcal{S}_\varepsilon, \quad J_\lambda(\widehat{W}) \le J_\lambda(W^\star) + \xi\right] \ge 1-\delta \tag{28.13}$$

with feasible sample and computational resources.

Norm minimization alone need not identify coordinates.

Already for a two-layer linear network computing the identity on $\mathbb{R}^d$,

$$(W_1, W_2) = (I, I) \quad \text{and} \quad (W_1, W_2) = (O, O^\top)$$

have the same product $\Pi = d$ for any orthogonal O.

This is minimal because

$$d = \operatorname{tr}(W_2 W_1) \le \|W_2\|_F \|W_1\|_F.$$

A dense rotation is therefore not distinguished from the sparse coordinate realization.

Both predict perfectly; structural uniqueness requires a separate criterion.

28.5.4 Possible approaches

Prove a restricted norm-separation theorem. For a specified teacher class and distribution, investigate whether well-fitting but structurally inappropriate solutions obey

$$\inf_{\substack{W \notin \mathcal{S}_\varepsilon \\ \widehat{R}_S(f_W) \le \eta}} \Pi(W)^2 \ge Q_\star^2 + \Delta, \qquad \Delta > 0, \tag{28.14}$$

with high probability.

This is a proposed separation condition, not a general property of compositional targets.

If $\eta \geq \lambda Q_\star^2 + \xi$ and $\xi < \lambda\Delta$, then Equation (28.11) excludes every bad solution in Equation (28.14).

The hard work is proving a genuine norm gap from assumptions on the task, architecture, and data, rather than inserting the desired conclusion as an assumption.

Analyze the product-regularized dynamics. The penalty has gradient

$$\nabla_{W_j} \Pi(W)^2 = 2W_j \prod_{\ell \neq j} \|W_\ell\|_F^2.$$

Study whether its coupling with the data gradient yields the objective comparison at polynomial cost in a controlled model.

Deep linear models and small nonlinear teacher–student hierarchies provide starting points.

Reparameterization symmetries and layer balancing must be handled explicitly; results for normalized gradient flow should not automatically be transferred to arbitrary unnormalized minibatch SGD.

Add cross-task evidence when one task is insufficient. Several tasks sharing constituents may favor a reusable library even when one task cannot identify it.

A possible joint objective is

$$\begin{aligned}\mathcal{J}_{\text{multi}} = &\sum_{a=1}^{M} \widehat{R}_a(f_a) + \lambda \sum_{a=1}^{M} \Pi(W_a)^2 \\ &+ \beta \left[B(\mathcal{G}) + \sum_{a=1}^{M} B(p_a \mid \mathcal{G}) \right].\end{aligned}$$

The library and program costs require a fixed valid coding language.

Their purpose is to charge explicitly for reuse, which a product norm alone does not measure.

Improvements in new-task learning cost must be proved relative to a specified task distribution; an i.i.d. risk bound for one task is not automatically a transfer theorem.

A useful first result. For a specified family with recurring constituents, either prove that product regularization yields a structural norm gap and efficient selection, or construct a counterexample showing why an additional sharing or description-length penalty is necessary.

Either outcome would sharpen the connection between regularization and compositionality.

The resulting five-problem program. The agenda separates five questions rather than treating any arrow as automatic:

1. Representation:	Which targets have economical compositions?
2. Coordinates:	Which feasible coordinates expose them?
3. Genericity:	When are useful footprints quantitatively visible?
4. Recovery:	How do learning dynamics build the staircase?
5. Selection:	When does regularization favor reusable structure?

Known approximation, stability, and generalization results are the starting points of this program.

The central task is to connect them without identifying representation with recovery, or a favorable capacity bound with recovery of a particular internal computation.

CHAPTER 29

Sparse Digital Computation, Smooth Lifts, and the Cost of Contraction

A digital computer evaluates real-valued functions through finite bit strings and Boolean operations. At each numerical resolution, this gives a bounded-fan-in computation. The representation question is whether these digital computations are discretizations of a coherent collection of regular, low-arity real-valued constituent functions.

Two directions must be distinguished. Starting from a regular compositional real function, one can construct finite-precision Boolean approximations that preserve its coarse computation graph. Starting from a Boolean circuit, one can extend its gates to smooth real functions that agree exactly on Boolean codewords. Neither statement alone proves that an arbitrary family of digital computations has a smooth continuum limit.

Contraction sharpens this picture. Nonexpansive constituents prevent multiplicative error amplification. Strictly contractive constituents provide depth-independent control of absolute implementation error. But a nonconstant Boolean gate cannot be represented exactly by a strict contraction when the input and output use the same two codewords.

Exact computation can nevertheless be retained by allowing the codeword separation to shrink with depth. The price then moves from absolute error amplification to numerical resolution and decoding sensitivity:

contraction controls absolute error, *but preserving information also requires control of signal margin.*

The chapter develops this tradeoff through explicit theorems and proofs, retains the square-root counterexample to the unrestricted smooth-limit claim, and identifies the remaining representation questions.

29.1 The Objects and the Conventions

We use H for computational depth, s for the number of internal nodes, k for a fixed maximum local arity, n for requested numerical precision, and $r \geq 1$ for an integer Sobolev order. The symbol λ denotes a common Lipschitz bound; $0 < \lambda < 1$ denotes the contractive regime.

All finite-dimensional product spaces carry the maximum norm unless explicitly stated otherwise. Thus

$$\mathrm{Lip}_\infty(g) = \sup_{x \neq y} \frac{|g(x) - g(y)|}{\|x - y\|_\infty}.$$

For order j, $\|D^j g\|_\infty$ denotes the maximum over $\|D^\alpha g\|_\infty$ with $|\alpha| = j$. For a scalar function on a box Q, write

$$\|g\|_{W^{r,\infty}(Q)} = \max_{|\alpha|\le r} \|D^\alpha g\|_{L^\infty(Q)}.$$

Using a sum instead of a maximum changes only constants depending on the fixed local dimension and r. Sobolev functions are evaluated through their continuous representatives. On boxes, $W^{1,\infty}$ functions have Lipschitz representatives; for higher orders the corresponding $C^{r-1,1}$ regularity is understood. Here $C^\infty(\overline{Q})$ means restriction of a smooth function defined on a neighborhood of $\overline{Q}$.

Let G be a finite acyclic graph with ordered argument slots. Its internal node v computes

$$z_v = g_v\big((z_u)_{u\in \mathrm{pa}(v)}\big), \qquad |\,\mathrm{pa}(v)| \le k. \tag{29.1}$$

An argument may be reused. Outputs are designated node values, and $F = \Phi_G(\{g_v\})$ denotes the resulting input–output map.

29.1.1 Boolean gates and numerical modules

A Boolean gate has a fixed number of one-bit arguments. A numerical module can have k real arguments but require many bits per argument. Consequently,

$$\boxed{\text{bounded numerical arity} \neq \text{bounded bit length.}}$$

Replacing one numerical node by a Boolean subcircuit preserves the *coarse* graph, not its literal bit-level node count.

29.1.2 Encoding and decoding are part of the computation

A finite-precision realization has the form

$$F_n(x) = D_n\big(C_n(E_{p(n)}(x))\big). \tag{29.2}$$

Here E_p denotes a numerical quantization/encoding convention, C_n is a Boolean circuit, and D_n decodes its output. Endpoints, signs, and integer parts may require additional bits, which are included in the encoding cost.

The notation $E_p(x)$ is not a claim that exact binary digits can be continuously extracted from an arbitrary real input. In Type-2 computation an evaluator receives sufficiently accurate rational approximations to x and must tolerate their allowed errors. A modulus specifies how much input precision is needed [53].

We distinguish existence of a circuit at every accuracy from a uniform algorithm generating those circuits. A noncomputable real constant can appear in a smooth function, so regularity alone does not establish computability.

29.1.3 Local versus global contraction

The condition $\mathrm{Lip}_\infty(F) < 1$ on the final target is not the same as $\mathrm{Lip}_\infty(g_v) < 1$ at every node. A small global Lipschitz constant does not itself exhibit bounded-arity constituents. Conversely, local contraction must be interpreted together with all copy paths, encoders, decoders, and any fresh inputs injected during execution.

29.2 Error Propagation and the Three Lipschitz Regimes

The error-propagation argument is a finite-DAG version of the mechanism used in compositional approximation theory [74, 72]. We give it explicitly because it will be used in both directions of the digital–continuous comparison.

Suppose g_v has Lipschitz constant L_v. Let $\widehat{g}_v$ be an implementation whose local error, on every admissible argument tuple, is at most δ_v. The local error includes numerical evaluation and rounding. Require that all implementations return values in the prescribed output intervals. Clipping to those intervals does not enlarge error relative to a true output lying in the interval.

Lemma 8 (Nodewise error recursion)**.** *If e_v bounds the error in the value at node v, then*

$$e_v \le \delta_v + L_v \max_{u \in \mathrm{pa}(v)} e_u. \tag{29.3}$$

Proof. Let z_{pa} and $\widehat{z}_{\mathrm{pa}}$ be the true and implemented parent tuples. Insert $g_v(\widehat{z}_{\mathrm{pa}})$ and use the triangle inequality:

$$\begin{aligned} |\widehat{g}_v(\widehat{z}_{\mathrm{pa}}) - g_v(z_{\mathrm{pa}})| &\le |\widehat{g}_v(\widehat{z}_{\mathrm{pa}}) - g_v(\widehat{z}_{\mathrm{pa}})| + |g_v(\widehat{z}_{\mathrm{pa}}) - g_v(z_{\mathrm{pa}})| \\ &\le \delta_v + L_v \|\widehat{z}_{\mathrm{pa}} - z_{\mathrm{pa}}\|_\infty. \end{aligned}$$

The last norm is the maximum parent error. □

For input nodes set $A_v = 0$ and $M_v = 1$. At an internal node set

$$A_v = 1 + L_v \max_{u \in \mathrm{pa}(v)} A_u, \qquad M_v = L_v \max_{u \in \mathrm{pa}(v)} M_u. \tag{29.4}$$

For constant nodes the maximum over an empty parent set is zero. Let A_G and M_G be the maxima at the designated outputs.

Proposition 24 (Uniform local errors on a finite DAG)**.** *If input error is at most e_0 and $\delta_v \le \delta$, then*

$$\|\widehat{F} - F\|_\infty \le M_G e_0 + A_G \delta. \tag{29.5}$$

In particular, M_G is an input-sensitivity bound for F.

Proof. Induct on a topological ordering using $e_v \le M_v e_0 + A_v \delta$. Taking separate maxima of the two terms in parent errors gives exactly the recursions in Equation (29.4). □

Theorem 18 (Expansive, nonexpansive, and contractive composition)**.** *If $L_v \le \lambda$ and the maximum depth is H, then*

$$A_G \le \sum_{j=0}^{H-1} \lambda^j = \begin{cases} (\lambda^H - 1)/(\lambda - 1), & \lambda > 1, \\ H, & \lambda = 1, \\ (1 - \lambda^H)/(1 - \lambda), & 0 \le \lambda < 1. \end{cases} \tag{29.6}$$

Consequently, for exact inputs and $0 \le \lambda < 1$,

$$\boxed{\|\widehat{F} - F\|_\infty \le \frac{\delta}{1 - \lambda}.} \tag{29.7}$$

Proof. For a node of depth h, induction in Equation (29.4) bounds A_v by the first h terms of the geometric series. This bound is increasing in h for every $\lambda \ge 0$. Evaluate it at the maximum output depth. □

Thus strict contraction eliminates the depth factor from the bound on *absolute* accumulated local error. To guarantee final error ε, it is sufficient to take

$$\delta \le (1-\lambda)\varepsilon.$$

This is not yet a statement about recovering a discrete symbol: the output error must also be compared with the codeword separation.

29.2.1 A qualification about depth and skip connections

For a genuinely layered composition $G_H \circ \cdots \circ G_1$, with $\mathrm{Lip}_\infty(G_\ell) \le \lambda$, one has

$$\mathrm{Lip}_\infty(F) \le \lambda^H.$$

For an arbitrary DAG, *maximum* depth alone does not give this input-sensitivity bound. A short bypass can retain a stronger dependence on the input. The correct bound is M_G, or the maximum product of Lipschitz constants along an input–output path. In particular, an unscaled identity skip has Lipschitz constant one, not $\lambda < 1$. The absolute local-error estimate (29.7) still holds when all actual internal node maps obey the stated contraction bound.

29.3 From Regular Real Constituents to Boolean Approximation

Assume $g_v : [0,1]^{k_v} \to [0,1]$, $k_v \le k$, and $\|g_v\|_{W^{r,\infty}} \le B$, where $B \ge 1$ and r, k are fixed. The approximation principle is consistent with the Mhaskar–Poggio analysis: local dimension, not the full input dimension, controls the approximation exponent [74, 72]. To justify finite-bit costs, it is preferable to specify a quantizable local approximation rather than assume that an arbitrary real-parameter network automatically has a cheap digital implementation.

Lemma 9 (A quantizable local approximation)**.** *For $g : [0,1]^m \to [0,1]$, $m \le k$, with $\|g\|_{W^{r,\infty}} \le B$, there is a piecewise-polynomial approximation of uniform error at most δ using*

$$C_{k,r}\left(1+\frac{B}{\delta}\right)^{m/r}$$

coefficients. It can be evaluated on dyadic inputs by a bounded-fan-in Boolean circuit whose size is this coefficient count times a polynomial in the input word length and $\log(2+B/\delta)$.

Proof. Partition the cube into dyadic cells of side length $h = 2^{-J}$, with $C_{k,r}Bh^r \le \delta/3$. On each cell use a tensor product interpolant of degree at most $r-1$ in each coordinate; for $r = 1$ use a constant value. A fixed number of sample values, depending only on m, r, determines each cell polynomial.

For completeness, the local error estimate follows by applying the one-dimensional $W^{r,\infty}$ interpolation estimate successively in each coordinate. Each one-dimensional error is bounded by $C_r Bh^r$, and each fixed-degree interpolation operator has a bounded supremum-norm operator norm. The telescoping tensor-product estimate is therefore $C_{m,r}Bh^r$. These are standard local polynomial approximation estimates [15].

Use coordinates normalized to $[0,1]^m$ inside each cell. The fixed-degree basis and its coefficients have bounds independent of h; sample values are in $[0,1]$. Quantize the stored data finely enough to contribute at most $\delta/3$, and evaluate the fixed-degree polynomial to another $\delta/3$. The required arithmetic precision is $O_{k,r}(\log(2+B/\delta))$, together with the input word length.

A circuit selects a cell from the input address and evaluates the corresponding polynomial. Even a full multiplexing circuit has size at most the table size times a polynomial in word length. All arithmetic and selection can be decomposed into bounded-fan-in Boolean gates. Clipping the result to $[0,1]$ does not increase the error. □

Theorem 19 (Structure-preserving Boolean approximation)**.** *Let $F = \Phi_G(\{g_v\})$ satisfy the preceding assumptions, with d scalar inputs and s internal nodes. Put $\overline{A} = \max\{1, A_G\}$ and $\Gamma = 1 + A_G + M_G$. For every $0 < \varepsilon < 1$, there is a bounded-fan-in Boolean circuit C_ε, partitioned according to the same coarse DAG, such that its decoded numerical output satisfies*

$$\|F_\varepsilon - F\|_\infty \le \varepsilon. \tag{29.8}$$

For constants depending only on k, r, one may achieve

$$|C_\varepsilon| \le C_{k,r}\left[d + s\left(1 + \frac{B\overline{A}}{\varepsilon}\right)^{k/r}\right]\left[1 + \log\frac{(2+B)\Gamma}{\varepsilon}\right]^{c_{k,r}}. \tag{29.9}$$

The existence statement is nonuniform. A uniform construction additionally requires uniformly computable constituent evaluations and effective bounds on the constants used.

Proof. Choose local total error $\delta = \varepsilon/(3\overline{A})$ and encode inputs within $\varepsilon/(3\max\{1, M_G\})$ in maximum norm. Use the local digital construction at each node, including output rounding in its error budget, and keep every output in $[0,1]$.

The input contribution in (29.5) is at most $\varepsilon/3$, and the propagated local contribution is at most $A_G\delta \le \varepsilon/3$. The remaining slack covers a final decoding rounding if required. This proves (29.8).

There are at most $C_{k,r}(1 + B\overline{A}/\varepsilon)^{k/r}$ local coefficients per node. The needed word length is bounded by a constant times $1 + \log((2+B)\Gamma/\varepsilon)$. Multiplication, addition, comparison, and selection have polynomial bit-circuit cost. Summing over the nodes, and including the input encoding length, gives (29.9).

For a uniform construction, the sample values defining the cell interpolants can be computed to the required accuracy. For a noncomputable constituent, the finite rounded tables still exist but need not be generable by an algorithm. □

29.3.1 The contractive improvement

If every local Lipschitz constant is at most $\lambda < 1$, then $A_G \le (1-\lambda)^{-1}$ and $M_G \le 1$. The leading local approximation cost becomes

$$s\left(1 + \frac{B}{(1-\lambda)\varepsilon}\right)^{k/r}, \tag{29.10}$$

up to the displayed logarithmic factors and input-reading cost. There is no depth factor from approximation-error propagation, although there are still s modules to execute and store.

For the normalized class with $\sum_i \|\partial_i g_v\|_\infty \le 1$, the maximum-norm Lipschitz constant is at most one. This gives $A_G \le H$, recovering the nonexpansive estimate underlying the earlier discussion.

29.3.2 Approximation rate is not polynomial-time bit complexity

With $\varepsilon = 2^{-n}$, the factor $\varepsilon^{-k/r}$ is exponential in n. The theorem is an approximation-size statement, not a claim that every Sobolev constituent can be evaluated in time polynomial in the requested number of bits.

If separate uniform polynomial-time algorithms for the constituents are available, one can compile those algorithms instead. The required intermediate absolute precision has the form

$$p_v = n + O(\log A_v^{\mathrm{downstream}} + \log s),$$

with an error allocation appropriate to the DAG. Polynomial graph size, polynomial local evaluation costs, and polynomial required bit lengths then yield polynomial digital cost. An amplification factor exponential in depth can require only linearly many *additional bits*, even though it can make a generic approximation formula expensive.

29.4 Exact Real Extensions of Boolean Gates

Let $b : \{0,1\}^m \to \{0,1\}$, with $m \le k$. A real lift is a function on $[0,1]^m$ that agrees with b at the vertices. Such an extension is not unique and does not by itself specify the numerical meaning of intermediate non-Boolean values.

29.4.1 The smooth multilinear lift

Define

$$\widetilde{b}(x) = \sum_{a \in \{0,1\}^m} b(a) \prod_{i:a_i=1} x_i \prod_{i:a_i=0} (1 - x_i). \tag{29.11}$$

Proposition 25 (Exact polynomial extension)**.** *The function $\widetilde{b}$ is a polynomial, affine in each argument separately, with total degree at most m. It satisfies*

$$\widetilde{b}(a) = b(a), \qquad 0 \le \widetilde{b}(x) \le 1, \qquad \mathrm{Lip}_\infty(\widetilde{b}) \le m.$$

For every fixed integer $r \ge 1$, $\|\widetilde{b}\|_{W^{r,\infty}} \le C_{k,r}$.

Proof. At a vertex only its own basis product equals one; all other products vanish. On the cube these products are nonnegative and sum to one, so the range is in $[0,1]$.

For a set I of j distinct coordinates, $D_I\widetilde{b}$ is the average, over the other coordinates, of an alternating sum of 2^j Boolean values. Its absolute value is at most 2^{j-1} for $j \ge 1$. Derivatives with a repeated coordinate vanish. In particular, $|\partial_i \widetilde{b}| \le 1$. Integrating the gradient along a segment gives $|\widetilde{b}(x) - \widetilde{b}(y)| \le \|x - y\|_1 \le m\|x - y\|_\infty$, and the finite collection of derivative bounds proves the Sobolev estimate. □

Theorem 20 (Smooth lifting on the same Boolean DAG)**.** *Replace every Boolean constituent b_v in a finite DAG by $\widetilde{b}_v$. The resulting smooth real composition satisfies*

$$F_G(a) = f(a) \qquad (a \in \{0,1\}^d)$$

and has the same coarse graph and bounded local Sobolev norms.

Proof. On a Boolean input, all parent values of the first gates are Boolean. Their lifted outputs equal the Boolean gate outputs. Induction in a topological ordering gives the assertion at every node. □

The composition of local multilinear lifts need not equal the global multilinear interpolation of the truth table. For example, the circuit $\mathrm{XOR}(x,x)$ computes the constant zero on Boolean inputs, but its local polynomial lift is $2x(1-x)$. Sharing an input creates correlations that a gatewise probabilistic interpolation does not preserve away from the vertices.

29.4.2 An optimal Lipschitz lift

A second extension emphasizes stability rather than higher smoothness. For a nonconstant b, let $A_0 = \{a : b(a) = 0\}$ and set

$$B_b(x) = \min_{a \in A_0} \|x - a\|_\infty, \qquad x \in [0,1]^m. \tag{29.12}$$

For constant gates use the corresponding constant function.

Proposition 26 (Nonexpansive exact extension)**.** *The function B_b is 1-Lipschitz, has values in $[0,1]$, and satisfies $B_b(a) = b(a)$ at every Boolean vertex. It is piecewise affine and admits a min–max implementation of size depending only on k.*

Proof. Distance to a nonempty set is 1-Lipschitz. At a zero vertex the distance is zero. At a one vertex its maximum-norm distance to every zero vertex is one. Distances between points in the cube are at most one.

On the cube, $\|x - a\|_\infty$ is the maximum of the affine functions x_i or $1 - x_i$, according to a_i. The finite minimum in (29.12) is therefore piecewise affine and uses at most 2^k such maxima. □

This is a special explicit form of the Lipschitz extension principle associated with McShane [70]. More generally,

$$\min\left\{1, \max\left\{0, \inf_{a \in \{0,1\}^m} [b(a) + \|x - a\|_\infty]\right\}\right\}$$

gives a bounded 1-Lipschitz extension on all of $\mathbb{R}^m$. Gatewise use of these lifts gives a globally nonexpansive real circuit, but not necessarily a smooth one.

29.5 Smooth Lifts Arbitrarily Close to Nonexpansive

Smoothness and the exactly optimal Lipschitz constant can conflict. Nevertheless, any strictly positive Lipschitz slack suffices.

Lemma 10 (Smooth saturation with small Lipschitz slack)**.** *For $0 < a < 1/4$, there is $S_a \in C^\infty(\mathbb{R}; [0,1])$ such that $S_a(t) = 0$ for $t \le 0$, $S_a(t) = 1$ for $t \ge 1$, and*

$$\|S_a'\|_\infty \le (1 - 2a)^{-1}, \qquad \|S_a^{(j)}\|_\infty \le C_j a^{1-j} \quad (j \ge 1).$$

Proof. Choose a fixed smooth step function χ taking values in $[0,1]$, equal to zero on $(-\infty, 0]$ and to one on $[1, \infty)$. Set

$$w_a(t) = \chi(t/a)\chi((1-t)/a), \quad I_a = \int_{\mathbb{R}} w_a(t)\,dt, \quad S_a(t) = I_a^{-1} \int_{-\infty}^{t} w_a(u)\,du.$$

Then $1 - 2a \le I_a \le 1$. Differentiating proves the stated bounds and the endpoint identities. □

Theorem 21 (Almost-nonexpansive smooth extension)**.** *For every Boolean gate $b : \{0,1\}^m \to \{0,1\}$, $m \le k$, and $0 < \eta \le 1$, there is $g_\eta \in C^\infty(\mathbb{R}^m; [0,1])$ with*

$$g_\eta(a) = b(a), \qquad \mathrm{Lip}_\infty(g_\eta) \le 1 + \eta, \qquad \|g_\eta\|_{W^{r,\infty}([0,1]^m)} \le C_{k,r}\eta^{1-r} \tag{29.13}$$

for every fixed integer $r \ge 1$. The same construction works for all r; only the constants depend on r.

Proof. Take a bounded 1-Lipschitz extension B on $\mathbb{R}^m$ as above. Let ρ be a nonnegative smooth mollifier supported in the unit maximum-norm ball, with integral one, and put $h_\delta = B * \rho_\delta$. Then

$$0 \le h_\delta \le 1, \qquad \mathrm{Lip}_\infty(h_\delta) \le 1, \qquad \|h_\delta - B\|_\infty \le \delta.$$

For $|\alpha| = j \ge 1$, the zero integral of $D^\alpha \rho_\delta$ gives

$$D^\alpha h_\delta(x) = \int [B(x-y) - B(x)] D^\alpha \rho_\delta(y)\, dy,$$

so $\|D^\alpha h_\delta\|_\infty \le C_{k,j}\delta^{1-j}$.

Choose fixed smooth bumps ψ_a centered at the vertices, with disjoint supports, $\psi_a(a) = 1$, and support radius smaller than $1/3$. Let

$$h_1 = h_\delta + \sum_a [b(a) - h_\delta(a)]\psi_a.$$

This exactly repairs vertex values. Since each coefficient has magnitude at most δ, there is a constant C_k such that

$$\mathrm{Lip}_\infty(h_1) \le 1 + C_k\delta, \qquad \|D^\alpha h_1\|_\infty \le C_{k,j}\delta^{1-j}.$$

The latter estimate also bounds the bump corrections because their supports are fixed and $0 < \delta \le 1$.

Set $a = \eta/64$ and choose $\delta = c_k\eta$ so that $C_k\delta \le \eta/64$. Define $g_\eta = S_a \circ h_1$. Its range is $[0,1]$, and it preserves the repaired vertex values. Moreover,

$$\mathrm{Lip}_\infty(g_\eta) \le (1 + \eta/64)(1 - \eta/32)^{-1} \le 1 + \eta \qquad (0 < \eta \le 1).$$

In an order-j derivative of this composition, each term has the form $S_a^{(q)}(h_1)\prod_{i=1}^q D^{\alpha_i} h_1$, with $\sum_i |\alpha_i| = j$ and all $|\alpha_i| \ge 1$. Its bound is

$$C_{k,j}\eta^{1-q}\prod_i \eta^{1-|\alpha_i|} = C_{k,j}\eta^{1-j}.$$

Summing the finitely many chain-rule terms and including the bounded zeroth derivative proves (29.13). □

The restriction $\eta \le 1$ matters for the displayed Sobolev estimate. For arbitrary $\eta > 0$, replace η there by $\min\{\eta, 1\}$.

29.5.1 Why exact nonexpansive smooth interpolation can fail

Proposition 27 (A smoothness obstruction for XOR)**.** *There is no C^1 extension of the two-input XOR truth table on the square with maximum-norm Lipschitz constant at most one.*

Proof. An exact 1-Lipschitz extension must satisfy $g(t,0) = t$ and $g(0,t) = t$: each edge joins values zero and one at unit distance, so the Lipschitz inequalities from its two endpoints force equality. Differentiability at $(0,0)$ therefore gives $\partial_1 g(0,0) = \partial_2 g(0,0) = 1$. Consequently $g(t,t) = 2t + o(t)$, contradicting $|g(t,t) - g(0,0)| \le t$. □

Theorem 22 (Depth-stable smooth Boolean lift)**.** *A Boolean DAG of maximum depth $H \ge 1$ and local arity at most k has, for every $0 < c \le 1$, an exact smooth lift on the same coarse graph with*

$$\mathrm{Lip}_\infty(F_G) \le e^c, \qquad \|g_v\|_{W^{r,\infty}} \le C_{k,r}(H/c)^{r-1}.$$

Proof. Use $g_{v,\eta}$ with $\eta = c/H$. Along a path of length $h \le H$, the input-sensitivity multiplier is at most $(1 + c/H)^h \le e^c$. The local derivative estimate follows from Theorem 21. Agreement on Boolean inputs follows by topological induction. □

Corollary 3 (Robustness near fixed Boolean codewords)**.** *For the preceding lift, $a \in \{0,1\}^d$ and $\|x - a\|_\infty \le \tau$ imply*

$$|F_G(x) - f(a)| \le e^c \tau.$$

Thresholding at $1/2$ is correct when $e^c\tau < 1/2$.

Proof. Use $F_G(a) = f(a)$ and the global Lipschitz bound. □

29.6 Strict Contraction with Fixed Boolean Codes

Suppose the same two numbers, zero and one, represent the Boolean symbols at every gate. Strict contraction is then incompatible with exact nonconstant Boolean semantics.

Theorem 23 (Fixed-code obstruction and sharp approximation error)**.** *Let $b : \{0,1\}^m \to \{0,1\}$ be nonconstant. Every exact real extension satisfies*

$$\mathrm{Lip}_\infty(g) \ge 1.$$

For $0 \le \lambda \le 1$, the optimal vertex approximation error among λ-Lipschitz maps $g : [0,1]^m \to [0,1]$ is exactly

$$\boxed{\inf_{\mathrm{Lip}_\infty(g) \le \lambda} \max_{a \in \{0,1\}^m} |g(a) - b(a)| = \frac{1-\lambda}{2}.} \tag{29.14}$$

Proof. Choose vertices a, a' with $b(a) = 0$ and $b(a') = 1$. Their maximum-norm distance is one. Exact agreement would require $1 \le \mathrm{Lip}_\infty(g)$.

If the maximum vertex error is ε, then

$$1 - 2\varepsilon \le g(a') - g(a) \le \lambda,$$

so $\varepsilon \ge (1-\lambda)/2$. For the matching construction take the nonexpansive exact lift B_b and set

$$g_\lambda(x) = \lambda B_b(x) + \frac{1-\lambda}{2}.$$

It is λ-Lipschitz. Its vertex values are $(1-\lambda)/2$ and $(1+\lambda)/2$, giving equality in the bound. □

This theorem concerns approximation to the numerical values zero and one. For $\lambda > 0$, thresholding the two approximate output levels at $1/2$ still classifies the gate correctly. But the output levels are not the original input codes. Feeding them into another gate without specifying its encoding or a restoration operation is not justified by the theorem. A restoration operation must itself be included in the analysis.

29.6.1 The smooth version has the same infimum

Corollary 4 (Smooth approximation under a fixed contraction budget)**.** *The infimum in* (29.14) *is unchanged if the admissible maps must be C^∞ on a neighborhood of the cube. It need not be attained by a smooth map.*

Proof. For $\eta > 0$, set $a_\eta = \lambda/(1+\eta)$ and use

$$g_{\lambda,\eta}(x) = a_\eta g_\eta(x) + \frac{1 - a_\eta}{2},$$

where g_η is an exact smooth lift from Theorem 21. This map has Lipschitz constant at most λ and vertex error $(1 - a_\eta)/2$, tending to $(1-\lambda)/2$.

For XOR and $\lambda > 0$, attainment would force the vertex values to be exactly $(1-\lambda)/2$ and $(1+\lambda)/2$. Subtracting the first value and dividing by λ would produce a smooth exact 1-Lipschitz XOR extension, contradicting Proposition 27. For $\lambda = 0$, the constant $1/2$ attains the infimum. □

The distinction between an attained Lipschitz optimum and a smooth infimum is important. Smooth exact fixed-code lifting generally requires a Lipschitz constant slightly above one; smooth contractive approximate lifting approaches the sharp vertex error without always attaining it.

29.7 Exact Contractive Computation with Changing Code Scales

The fixed-code obstruction disappears when representation scale is allowed to change. At level ℓ, encode Boolean zero by zero and Boolean one by $\Delta_\ell > 0$. The number Δ_ℓ is both the codeword separation and the available absolute output margin scale.

Theorem 24 (Contraction forces separation loss in a layered code)**.** *Suppose a nonconstant Boolean gate takes inputs in $\{0, \Delta_\ell\}^m$, returns exact outputs in $\{0, \Delta_{\ell+1}\}$, and has Lipschitz constant at most $0 < \lambda < 1$ on the containing box. Then*

$$\Delta_{\ell+1} \le \lambda \Delta_\ell. \tag{29.15}$$

For an H-stage layered computation with a common code scale at each level and a nonconstant encoded input–output map,

$$\boxed{\Delta_H \le \lambda^H \Delta_0.} \tag{29.16}$$

Proof. Two encoded input tuples on which the gate outputs differ have distance Δ_ℓ, while their outputs have distance $\Delta_{\ell+1}$. The Lipschitz inequality proves (29.15).

Alternatively, the composition of the H complete layers is λ^H-Lipschitz. Choose two Boolean input vectors whose final bits differ. Their encoded input distance is Δ_0 and their output distance is Δ_H, proving (29.16). □

The global statement requires H actual contractive stages along the information flow. It is not a bound based only on the maximum depth of an arbitrary DAG containing bypasses. Constant outputs carry no two-symbol separation requirement.

29.7.1 An exact Lipschitz construction

Theorem 25 (Contractive realization on the original DAG)**.** *Let a Boolean circuit have local arity at most k and depth function $h(v)$, with input depth zero and output depth at most H. Fix $0 < \lambda < 1$ and $\Delta_0 > 0$. Set*

$$\Delta_v = \Delta_0 \lambda^{h(v)}.$$

There are real local maps, all λ-Lipschitz, on the same DAG that exactly simulate every Boolean gate using its node-dependent codes $\{0, \Delta_v\}$. An output of depth H has separation $\Delta_0 \lambda^H$.

Proof. For each gate take its exact nonexpansive lift B_{b_v} and define

$$g_v((x_u)_u) = \Delta_v B_{b_v}((x_u/\Delta_u)_u), \qquad x_u \in [0, \Delta_u]. \tag{29.17}$$

At the encoded Boolean tuple $x_u = \Delta_u a_u$, the output is exactly $\Delta_v b_v((a_u)_u)$. Induction gives exact circuit simulation.

Because $h(v) = 1 + \max_{u \in \mathrm{pa}(v)} h(u)$,

$$\min_{u \in \mathrm{pa}(v)} \Delta_u = \Delta_0 \lambda^{h(v)-1}.$$

The Lipschitz constant of (29.17) is consequently at most $\Delta_v / \min_u \Delta_u = \lambda$. Constant gates have Lipschitz constant zero. □

For a strictly layered graph this reduces to the simple formula

$$g_\ell(x) = \Delta_{\ell+1} B_b(x/\Delta_\ell), \qquad \Delta_{\ell+1} = \lambda \Delta_\ell.$$

The original Boolean information is intact at every finite depth, even though its amplitude shrinks. For dyadic λ, the min–max lifts and the scales are described with finite rational data. Polynomial-depth circuits require only polynomially many bits to write these scales in fixed-point form. This is an execution construction, not a result about learning the circuit.

29.7.2 Smooth contractive realization: a small slack is needed

Using a C^∞ lift with Lipschitz constant $1 + \eta$ instead of a nonsmooth nonexpansive lift requires a slightly faster scale decrease.

Theorem 26 (Smooth contractive simulation with near-optimal separation)**.** *For a Boolean DAG of maximum depth $H \geq 1$, choose $0 < c \leq 1$ and*

$$\eta = \frac{c}{H}, \qquad q = \frac{\lambda}{1+\eta}, \qquad \Delta_v = \Delta_0 q^{h(v)}.$$

There is an exact smooth real realization on the same coarse graph with $\mathrm{Lip}_\infty(g_v) \leq \lambda$. At an output of depth H,

$$e^{-c} \Delta_0 \lambda^H \leq \Delta_H = \Delta_0 q^H \leq \Delta_0 \lambda^H. \tag{29.18}$$

Thus smoothness preserves the optimal layered separation scale up to a constant factor independent of H.

Proof. Replace B_{b_v} in (29.17) by the exact smooth lift $g_{b_v,\eta}$ and use the scales based on q. The local Lipschitz constant is at most $q(1+\eta) = \lambda$, and agreement on encoded Boolean values is exact. Finally,

$$\Delta_H = \Delta_0 \lambda^H (1 + c/H)^{-H}$$

and $1 \leq (1 + c/H)^H \leq e^c$. □

Unlike the Lipschitz construction, exact saturation of the separation bound is not asserted for every smooth Boolean gate. A smooth XOR gate would encounter the same obstruction as before if rescaling it produced an exact 1-Lipschitz fixed-code lift.

29.7.3 Higher derivatives in shrinking physical coordinates

For a gate at depth ℓ whose parents share scale $\Delta_{\ell-1}$,

$$\|D^j g_\ell\|_\infty \le C_{k,j}\Delta_\ell \Delta_{\ell-1}^{-j}\eta^{1-j} = C_{k,j} q \Delta_{\ell-1}^{1-j}\eta^{1-j}. \tag{29.19}$$

For unequal parent scales, the same upper bound uses the smallest parent scale. Higher derivatives can therefore grow exponentially with depth when $j \ge 2$, despite the first-order contraction bound. On rescaled unit boxes the normalized gate has the polynomial-in-H bound from Theorem 21.

Thus first-order contractivity and uniform higher-order Sobolev control are different properties. The cost must be measured in declared coordinates and units.

29.8 Absolute Error, Relative Error, and Required Precision

The bound $e_H \le \delta/(1-\lambda)$ is favorable in absolute units. But an exact Boolean output at depth H is represented by either zero or Δ_H. Correct threshold decoding requires

$$e_H < \frac{\Delta_H}{2}. \tag{29.20}$$

Absolute contraction alone does not ensure this inequality.

Proposition 28 (A sufficient finite-precision budget)**.** *Assume exact encoded inputs, local maps with Lipschitz constants at most $\lambda < 1$, and local total implementation error at most δ. A sufficient condition for correct final threshold decoding is*

$$\delta \le \frac{(1-\lambda)\Delta_H}{4}. \tag{29.21}$$

If the implementation provides $\delta \le 2^{-p}$, it suffices that

$$p \ge \left\lceil \log_2 \frac{4}{(1-\lambda)\Delta_H} \right\rceil. \tag{29.22}$$

For the exact Lipschitz scaling $\Delta_H = \Delta_0\lambda^H$, this is

$$p \ge \left\lceil H \log_2 \frac{1}{\lambda} + \log_2 \frac{4}{(1-\lambda)\Delta_0} \right\rceil. \tag{29.23}$$

Proof. The absolute error is bounded by $\delta/(1-\lambda) \le \Delta_H/4$, strictly below the threshold distance $\Delta_H/2$. Substitute the claimed numerical error and solve for p. □

Here δ is the error of an entire local numerical module, not necessarily the error of one elementary arithmetic operation. Internal guard bits or a verified local evaluation scheme are needed to obtain that module-level guarantee. Clipping approximate states to their admissible boxes can be used without worsening error.

For the smooth scaling in (29.18), the sufficient bit budget differs by at most $c\log_2 e$ plus rounding constants. The leading depth dependence remains linear for fixed λ. For example, $\lambda = 1/2$, $\Delta_0 = 1$, and $\Delta_H = 2^{-H}$ give the sufficient local error $2^{-(H+3)}$.

29.8.1 Normalized error reveals the tradeoff

Let $r_\ell = e_\ell/\Delta_\ell$ be error measured in local code units. For a layered computation,

$$r_{\ell+1} \le \lambda \frac{\Delta_\ell}{\Delta_{\ell+1}} r_\ell + \frac{\delta_{\ell+1}}{\Delta_{\ell+1}}. \tag{29.24}$$

For the Lipschitz construction $\Delta_{\ell+1} = \lambda\Delta_\ell$, the first multiplier is one, not λ. Therefore

$$r_H \le r_0 + \sum_{\ell=1}^{H} \frac{\delta_\ell}{\Delta_\ell}.$$

Contraction in absolute units does not contract the error relative to the shrinking Boolean margin. In the smooth construction, that multiplier is $1+\eta$, whose product is controlled by choosing $\eta = c/H$.

29.8.2 The final decoder restores the scale

If an encoded computation returns $z_H \in \{0, \Delta_H\}$, the numerical Boolean decoder is

$$D_H(z) = z/\Delta_H$$

on these codewords. Its Lipschitz constant is Δ_H^{-1}. Threshold decoding is discontinuous on the full real line and has noise tolerance $\Delta_H/2$ around the codewords.

Thus a contractive internal realization does not make the complete fixed-code input–output map contractive. The decoder compensates for the amplitude loss. Repeatedly restoring a fixed separation during the computation similarly requires expansion or a discontinuous decision at some stage.

29.8.3 What the bit budget does and does not mean

Equation (29.23) is a *fixed absolute precision* guarantee. A uniform fixed-point mesh that must robustly resolve scale Δ_H needs mesh width smaller than that scale. Similarly, two disjoint noise balls around the output codewords require noise radius less than $\Delta_H/2$.

This is not an information-theoretic assertion that one Boolean symbol requires H storage bits. The set $\{0, \Delta_H\}$ contains only two possibilities. A symbolic representation can store a Boolean symbol along with a shared scale convention. A floating-point representation can encode a small amplitude using an exponent; for dyadic λ, that exponent may require only $O(\log H)$ bits.

Such representations change the storage and arithmetic accounting, not the absolute noise tolerance of the real-valued code. The precise conclusion is

> strict contraction can preserve exact finite-depth information,
> but a shrinking real code requires shrinking absolute error
> or an explicitly accounted rescaling/decoding mechanism.

29.9 Contractivization by Rescaling Is Not a Free Structural Result

The changing-code construction is part of a more general rescaling phenomenon. It is important not to mistake a choice of units for an intrinsic reduction in computational complexity.

Proposition 29 (Contractivizing a finite Lipschitz DAG)**.** *Let $g_v : [0,1]^{k_v} \to [0,1]$ be the constituents of any finite DAG, with finite Lipschitz constants L_v. Given $0 < \lambda < 1$, choose input scales $a_u = 1$ and internal scales recursively as*

$$a_v = \frac{\lambda \min_{u \in \mathrm{pa}(v)} a_u}{\max\{1, L_v\}}.$$

Constant nodes can be assigned any positive scale. The scaled functions

$$\widehat{g}_v((x_u)_u) = a_v g_v((x_u/a_u)_u)$$

exactly realize the same computation in rescaled coordinates and have Lipschitz constants at most λ.

Proof. The rescaling relation is preserved node by node. Moreover,

$$\mathrm{Lip}_\infty(\widehat{g}_v) \le \frac{a_v L_v}{\min_u a_u} \le \lambda.$$

The final unscaled output is obtained by dividing by its output scale. □

A restriction to local contractions becomes a substantive representational condition only when the coordinate scales, decoder sensitivity, numerical precision, or noise tolerance are also constrained. Otherwise local contraction can be purchased by making the output scale extremely small.

A globally contractive target is a different assumption. For any bounded Lipschitz target f, multiplication by a sufficiently small scalar makes it contractive without revealing its hidden constituents. Thus global contraction alone does not solve the bounded-arity representation problem.

29.10 Continuum Limits on a Coherent Coarse DAG

The precision index n and the execution depth H are distinct. In this section G is a fixed coarse graph and $n \to \infty$ refines the numerical resolution of each of its modules. The domains and numerical units are fixed across n.

Theorem 27 (Fixed-DAG Sobolev continuum limit)**.** *Suppose $g_{v,n} : [0,1]^{k_v} \to [0,1]$ satisfy, for all nodes and n,*

$$\|g_{v,n+1} - g_{v,n}\|_\infty \le C2^{-n}, \qquad \|g_{v,n}\|_{W^{r,\infty}} \le B. \tag{29.25}$$

Then there are $g_v \in W^{r,\infty}$ with

$$\|g_v\|_{W^{r,\infty}} \le B, \qquad \|g_{v,n} - g_v\|_\infty \le 2C2^{-n}.$$

If $F_n = \Phi_G(\{g_{v,n}\})$ and $F = \Phi_G(\{g_v\})$, then

$$\|F_n - F\|_\infty \le 2CA_G 2^{-n}, \tag{29.26}$$

where A_G is computed from Lipschitz constants of the limiting maps.

Proof. Summing the cross-resolution differences shows that each sequence is uniformly Cauchy, with tail at most $2C2^{-n}$. The common Sobolev bound bounds every derivative through order r in L^∞.

There are finitely many nodes and derivative multi-indices. By weak-star compactness, extract a subsequence on which each derivative converges weak-star in L^∞. For a test function $\varphi \in C_c^\infty((0,1)^{k_v})$,

$$\int g_{v,n} D^\alpha \varphi = (-1)^{|\alpha|} \int D^\alpha g_{v,n}\, \varphi.$$

Uniform convergence on the left identifies the weak-star limit on the right as $D^\alpha g_v$. Its essential-supremum bound is at most B. This proves the stated Sobolev regularity and norm bound.

Apply the nodewise error recursion with $\delta_v = \|g_{v,n} - g_v\|_\infty$ and exact initial inputs. The same fixed finite DAG has a finite A_G, yielding (29.26). □

This proves a $W^{r,\infty}$ limit, not automatically a C^∞ limit. To obtain smoothness of every order, corresponding estimates at every order or other stronger assumptions are required.

Corollary 5 (Uniform contraction gives a depth-independent limit bound)**.** *Instead of higher-order bounds, suppose the local maps satisfy* $\mathrm{Lip}_\infty(g_{v,n}) \le \lambda < 1$*, the same range bound, and the first condition in* (29.25)*. Then*

$$\mathrm{Lip}_\infty(g_v) \le \lambda, \qquad \|F_n - F\|_\infty \le \frac{2C2^{-n}}{1-\lambda}.$$

In particular, the local limits lie in $W^{1,\infty}$*.*

Proof. The uniform Cauchy argument gives g_v. Pass to the limit in $|g_{v,n}(x) - g_{v,n}(y)| \le \lambda\|x-y\|_\infty$, and then apply Theorem 18. □

The conclusion concerns fixed domains and scales. If code separation shrinks while an increasingly sensitive decoder is introduced, a bound on the raw encoded maps does not automatically control the decoded numerical limit.

29.11 Regularity Conditions Stated Directly on Digital Tables

Let $h_n = 2^{-n}$ and $Q_n = \{0, h_n, 2h_n, \dots, 1\}^m$. A local Boolean numerical module produces a dyadic-valued table $T_n : Q_n \to [0,1]$. Its output precision need not equal its input mesh spacing. Let $\overline{T}_n$ be its piecewise-constant extension using a fixed lower-corner quantization rule, with an endpoint convention included.

29.11.1 A digital contraction condition allowing quantization error

Proposition 30 (Contraction from a discrete two-point estimate)**.** *Suppose* $\overline{T}_n$ *converges uniformly to* g*, and for all grid pairs* $a, b \in Q_n$*,*

$$|T_n(a) - T_n(b)| \le \lambda\|a-b\|_\infty + \tau_n, \qquad \tau_n \to 0. \tag{29.27}$$

Then g *has Lipschitz constant at most* λ*. In fact, there are* λ*-Lipschitz continuous functions* $\widetilde{T}_n : [0,1]^m \to [0,1]$ *such that*

$$\|\widetilde{T}_n - \overline{T}_n\|_\infty \le \tau_n + \lambda h_n. \tag{29.28}$$

Proof. Set

$$\widetilde{T}_n(x) = \mathrm{clip}_{[0,1]} \inf_{a \in Q_n} \{T_n(a) + \lambda\|x-a\|_\infty\}.$$

This is λ-Lipschitz. At a grid point b, the term $a = b$ gives an upper bound $T_n(b)$, while (29.27) gives a lower bound $T_n(b) - \tau_n$. Clipping preserves these approximation bounds because $T_n(b) \in [0,1]$.

For a general x, choose its quantized point b with $\|x-b\|_\infty \le h_n$. Lipschitz continuity then proves (29.28). Thus $\widetilde{T}_n \to g$ uniformly, and its common Lipschitz inequality passes to the limit. □

Conversely, if g is λ-Lipschitz and $|T_n(a) - g(a)| \le q_n$ at grid points, then

$$|T_n(a) - T_n(b)| \le \lambda \|a - b\|_\infty + 2q_n, \quad \|\overline{T}_n - g\|_\infty \le \lambda h_n + q_n.$$

This supplies an exact first-order bridge between analytic regularity and digital stability with quantization slack. It does not establish that checking every grid pair is computationally cheap.

29.11.2 Higher-order finite differences

Define normalized forward and backward differences by

$$D^+_{h,i}u(x) = \frac{u(x + he_i) - u(x)}{h}, \qquad D^-_{h,i}\varphi(x) = \frac{\varphi(x) - \varphi(x - he_i)}{h}.$$

Products of these operators define $(D^+_h)^\alpha$ and $(D^-_h)^\alpha$. Write $\Delta_{h,i}u(x) = u(x + he_i) - u(x)$ for an unnormalized forward difference, so that $(D^+_h)^\alpha = h^{-|\alpha|}\Delta^\alpha_h$. The signs and normalizations matter in summation by parts.

Theorem 28 (Discrete Sobolev regularity of the continuum limit)**.** *Assume $T_n : Q_n \to [0, 1]$ satisfies*

$$\|\overline{T}_{n+1} - \overline{T}_n\|_\infty \le Ch_n$$

and, for every $1 \le |\alpha| \le r$,

$$\left|h_n^{-|\alpha|}\Delta^\alpha_{h_n} T_n(a)\right| \le B_\alpha \tag{29.29}$$

at all grid points where the forward stencil is defined. Then $\overline{T}_n$ converges uniformly to a continuous function g, and

$$g \in W^{r,\infty}((0, 1)^m), \qquad \|D^\alpha g\|_\infty \le B_\alpha.$$

Its continuous representative extends to the closed cube.

Proof. The compatibility condition makes $\overline{T}_n$ uniformly Cauchy, so it has a uniform limit g. First-order differences imply, by summing along coordinate grid paths,

$$|\overline{T}_n(x) - \overline{T}_n(y)| \le \sum_{i=1}^m B_{e_i}(|x_i - y_i| + h_n).$$

Passing to the limit proves continuity and a Lipschitz bound for g.

Fix $\varphi \in C^\infty_c((0, 1)^m)$. For large n, every shift needed by the stencil stays away from the boundary. Translating variables in Lebesgue integrals gives the exact identity

$$\int \overline{T}_n (D^-_{h_n})^\alpha \varphi = (-1)^{|\alpha|} \int (D^+_{h_n})^\alpha \overline{T}_n \, \varphi. \tag{29.30}$$

On each interior cell, the forward difference field is exactly the normalized grid difference in (29.29). It is bounded by B_α on the support relevant to the integral. The left side converges to $\int g D^\alpha \varphi$ because $(D^-_{h_n})^\alpha \varphi \to D^\alpha \varphi$ uniformly.

Consequently,

$$\left|\int g D^\alpha \varphi\right| \le B_\alpha \|\varphi\|_{L^1}.$$

The distribution $D^\alpha g$ therefore extends to a bounded functional on L^1, and is represented by an L^∞ function of norm at most B_α. This proves the asserted Sobolev regularity at every order through r. □

This proof does not assert that a piecewise-constant interpolant itself lies in $W^{r,\infty}$. The regularity is a property of its limit. Applied node by node, the resulting continuous local limits and their uniform error bounds can be composed using Lemma 8.

29.11.3 Output quantization must respect derivative order

Suppose exact samples of a $W^{r,\infty}$ function are rounded with maximum value error q_n. Their order-j finite differences can change by as much as $2^j q_n$. After division by h_n^j, this becomes $2^j q_n / h_n^j$.

Thus a sufficient output precision for preserving uniformly bounded finite differences through order r is

$$q_n = O(h_n^r), \qquad \text{or } p_{\text{out}}(n) \geq rn + O(1). \tag{29.31}$$

Rounding input-grid samples only to error $O(h_n)$ need not preserve higher-order discrete derivative bounds. Under computable evaluation and known derivative bounds, the stronger output precision is available, but its computational cost still depends on the constituent evaluator.

29.12 Efficient Computation Does Not Imply Uniform Analytic Regularity

Let $\mathrm{P}_{\mathbb{R}}$ refer here to polynomial-time computability in the oracle/finite-precision bit model, not to a unit-cost exact-real arithmetic model [53]. The input and output have their ordinary real coordinates; no uncharged nonlinear encoder is allowed.

Theorem 29 (The square-root counterexample)**.** *The function $f(x) = \sqrt{x}$ on $[0,1]$ is polynomial-time computable, but has no exact representation by a finite DAG of $W^{1,\infty}$ constituents on compact boxes containing their admissible arguments.*

Proof. For $x, y \geq 0$,

$$|\sqrt{x} - \sqrt{y}| \leq \sqrt{|x-y|}. \tag{29.32}$$

Given requested output error 2^{-n}, obtain a dyadic approximation $y \in [0,1]$ to x within 2^{-2n-4}. Then the first error in (29.32) is at most 2^{-n-2}. Write $y = m2^{-2n-4}$. Computing the integer $\lfloor \sqrt{m} \rfloor$ and dividing by 2^{n+2} approximates $\sqrt{y}$ within 2^{-n-2}. Binary search for that integer uses polynomially many bit operations on $O(n)$-bit integers. The input precision is $2n+O(1)$, proving polynomial-time evaluation.

Every $W^{1,\infty}$ constituent on a box has a finite Lipschitz constant. A finite composition on a finite DAG is therefore Lipschitz, by the recursion for M_G. But

$$\frac{|\sqrt{h} - \sqrt{0}|}{h} = h^{-1/2} \longrightarrow \infty \qquad (h \downarrow 0).$$

This contradicts any finite global Lipschitz constant. □

The obstruction is regularity, not graph size: the example already has one real input. It does not contradict smooth lifting of the Boolean circuits that compute square roots at fixed precisions. Those lifts live in growing bit-coordinate spaces; their surrounding numerical encoders and decoders are different objects from an exact smooth function of the physical real input x.

29.12.1 Coherent smooth approximants can lose regularity

Let

$$f_n(x) = \sqrt{x + 2^{-2n}} - 2^{-n}. \tag{29.33}$$

Each f_n is smooth on a neighborhood of $[0,1]$ and takes values in $[0,1]$. It has the same one-node graph at every resolution.

Proposition 31 (Uniform convergence with derivative blow-up)**.** *The functions in* (29.33) *satisfy*

$$\|f_n - \sqrt{\cdot}\|_\infty \le 2^{-n}, \qquad \|f_n'\|_\infty = 2^{n-1}.$$

They are also cross-resolution Cauchy at rate $O(2^{-n})$*.*

Proof. For $a \ge 0$, $0 \le \sqrt{x+a} - \sqrt{x} \le \sqrt{a}$. Taking $a = 2^{-2n}$ proves the error bound. Differentiation gives $f_n'(x) = 1/(2\sqrt{x + 2^{-2n}})$, with maximum at zero. The triangle inequality gives $\|f_{n+1} - f_n\|_\infty \le 2^{-n-1} + 2^{-n}$. □

Proposition 32 (Unavoidable conditioning cost)**.** *If* $0 < \varepsilon \le 1/4$ *and a Lipschitz function* g *satisfies* $\|g - \sqrt{\cdot}\|_\infty \le \varepsilon$*, then*

$$\mathrm{Lip}(g) \ge \frac{1}{8\varepsilon}. \tag{29.34}$$

Proof. Set $x_\varepsilon = 16\varepsilon^2 \le 1$. Then $g(x_\varepsilon) \ge 3\varepsilon$ and $g(0) \le \varepsilon$, so

$$\mathrm{Lip}(g) \ge \frac{g(x_\varepsilon) - g(0)}{x_\varepsilon} \ge \frac{2\varepsilon}{16\varepsilon^2}.$$

□

Thus no family with a common finite Lipschitz constant, contractive or otherwise, can converge uniformly to $\sqrt{x}$. This is exactly the pathology excluded by the regularity assumptions of the continuum-limit theorem.

29.12.2 Strict contraction does not imply higher smoothness

The removal of the square-root pathology is a first-order statement. For a fixed $0 < \lambda < 1$, consider

$$a(x) = \lambda|x - 1/2|, \qquad a_n(x) = \lambda\sqrt{(x-1/2)^2 + 2^{-2n}}, \quad n \ge 1.$$

The maps $a_n : [0,1] \to [0,1]$ are smooth and satisfy

$$\mathrm{Lip}(a_n) \le \lambda, \quad \|a_n - a\|_\infty \le \lambda 2^{-n}, \quad \|a_n''\|_\infty = \lambda 2^n. \tag{29.35}$$

Indeed, the first derivative is $\lambda(x-1/2)/\sqrt{(x-1/2)^2 + 2^{-2n}}$, and the second derivative is largest at $x = 1/2$. The limit is contractive and in $W^{1,\infty}$, but not in $W^{2,\infty}$: its first derivative has a jump.

Consequently,

uniform strict contraction $\Longrightarrow$ $W^{1,\infty}$ control, but not higher-order Sobolev control.

This also answers the interpretation in which only the *final* real-valued target is assumed to have Lipschitz constant less than one. That assumption rules out the square root, but not nonsmooth contractive targets. It is not a substitute for higher-order regularity assumptions.

29.13 What the Theorems Establish

The chapter now separates five mathematical statements.

Regular real computation can be digitized. A fixed bounded-arity $W^{r,\infty}$ composition has finite-precision Boolean approximations preserving its coarse graph. The local approximation exponent involves k/r. Effectivity is needed for a uniform construction; polynomial-time local evaluators are needed for polynomial cost in requested output bits.

Boolean circuits have regular real lifts. Every bounded-fan-in Boolean circuit has an exact polynomial-gate lift, an exact nonexpansive Lipschitz lift, and an exact smooth lift whose global Lipschitz constant can be kept bounded independently of depth by allowing polynomially growing local higher-derivative bounds.

Contraction and fixed codes are incompatible with exact nonconstant semantics. The sharp fixed-code approximation error is $(1-\lambda)/2$. This bound is attained in the Lipschitz class; its smooth counterpart can be an unattained infimum.

Changing code scales restores exact computation. Strictly contractive gates can execute a Boolean circuit exactly when code separation shrinks. In a layered computation the optimal Lipschitz scale is $\Delta_H = \lambda^H \Delta_0$. Smooth implementations attain it within a constant factor. Absolute error control must then be compared with this shrinking separation, and the decoder must be charged.

A regular continuum limit needs both kinds of coherence. A fixed coarse graph and cross-resolution consistency supply structural coherence. Uniform Lipschitz or higher finite-difference bounds supply analytic coherence. The square-root construction shows why the first does not imply the second. Uniform contraction is an especially strong first-order certificate, but does not supply every higher derivative.

No assertion of automatic program discovery or statistical learning follows from these representation and execution theorems. Nor does the smooth extension proof alone provide a sharp bit-complexity theorem for numerically evaluating every smooth lift. The constructions with piecewise-affine gates and rational scales make that accounting more direct; smooth implementations require their own evaluation guarantees.

29.14 The Remaining Research Problems

The unrestricted implication

$$\mathrm{P}_{\mathbb{R}} \Longrightarrow \text{an exact finite } W^{1,\infty} \text{ compositional representation}$$

is false. It should not remain labeled an open conjecture. Likewise, once convergent local modules with uniform Sobolev bounds are supplied, the continuum-limit theorem is already proved. The questions left by this chapter concern the existence, discovery, and economical certification of the required modules and scales.

29.14.1 Coarse-graining an efficient circuit family

Suppose (C_n) is a uniform family of finite-precision circuits computing f to error 2^{-n}. Seek a decomposition

$$C_n = G(C_{1,n}, \ldots, C_{s,n})$$

with a precision-independent coarse graph G, bounded numerical arity, compatible local encodings, and decoded module tables satisfying cross-resolution and regularity bounds.

The existence of small Boolean gates at each precision does not identify such numerical modules. A meaningful theorem should supply conditions on the algorithm or the task family under which the number of modules, their local dimensions, and their descriptions remain controlled. Otherwise the entire computation can be hidden in one high-dimensional primitive.

For families f_d, a precise structural target is $s(d) \le \mathrm{poly}(d)$, local dimension $k = O(1)$, and specified bounds on local norms and evaluation costs. For one fixed low-dimensional function, allowing an unrestricted constituent equal to the target is not an informative representation theorem.

29.14.2 Efficient certificates of digital regularity

Proposition 30 supplies a concrete first-order certificate:

$$|T_n(a) - T_n(b)| \le \lambda \|a - b\|_\infty + \tau_n, \qquad \tau_n \to 0.$$

The existence of such a sufficient digital condition is therefore no longer mysterious. The open algorithmic issue is how to obtain it from a compact circuit or program without enumerating all grid pairs.

A useful result would provide compositional rules for certifying local condition bounds, carrying quantization slack through numerical primitives, and extracting a common coarse graph. For higher regularity, one would need analogous certificates for normalized finite differences, with the output precision in (29.31) explicitly included.

29.14.3 Optimal smoothness–margin–depth tradeoffs

For fixed gate arity, define

$$\Lambda_r(b, K) = \inf_g \left\{ \|g\|_{W^{r,\infty}} : \begin{array}{l} g|_{\{0,1\}^k} = b, \\ 0 \le g \le 1, \quad \mathrm{Lip}_\infty(g) \le K \end{array} \right\}.$$

Here the infimum is over smooth lifts. The smooth lifting theorem supplies an upper bound of order $(K-1)^{1-r}$ for $K > 1$ close to one, while XOR excludes a smooth exact extension at $K = 1$. Determining sharp gate-dependent bounds, and then optimizing nonuniform local slack allocations over a DAG, would quantify how smoothness should be distributed across the computation.

For example, if gate v has slack η_v, global sensitivity is controlled by products $\prod_{v \in P}(1 + \eta_v)$ over paths P, while local derivative cost scales with η_v^{1-r}. Uniform slack c/H is one convenient choice, not a proof of optimality for every graph.

29.14.4 Jointly optimizing precision and code scales

With local scales Δ_v and error budgets δ_v, the appropriate optimization is not to minimize Lipschitz constants alone. One must also control normalized errors, decoder sensitivity, and evaluation cost. A possible formulation is

$$\begin{aligned} \min_{\{\Delta_v, \delta_v\}} \quad & \sum_v T_v(\lceil \log_2(1/\delta_v) \rceil) \\ \text{subject to} \quad & e_o < \Delta_o/2 \quad \text{for every output } o, \\ & e_v \le \delta_v + L_v \max_{u \in \mathrm{pa}(v)} e_u, \\ & \text{the prescribed local coding and smoothness constraints.} \end{aligned}$$

Here $T_v(p)$ is the certified bit cost of evaluating the local module to absolute error 2^{-p}. The model must declare whether scales are stored as fixed-point numbers, floating-point exponents, or symbolic metadata. This would turn the qualitative precision tradeoff into an optimization problem over an actual computational cost model.

29.14.5 The distinction between depth limits and resolution limits

A final question concerns growing graphs. Uniform approximation of a fixed numerical map at increasing precision is not the same as taking execution depth to infinity. With uniformly bounded initial code separation and strictly contractive complete layers, the raw signal diameter tends to zero as $H \to \infty$. A nontrivial normalized limit then requires compensation by encoding, input injection, or decoding.

Any proposed continuum theorem for a family whose depth also grows must state how these scales and compensations are controlled. The fixed-DAG theorems proved here do not automatically establish that stronger result.

29.15 Conclusion

The original representation question has become more precise. Real-valued computation is implemented through finite-precision Boolean computation, but there are several distinct notions of simplicity and stability.

A regular sparse real composition admits structure-preserving digital approximation. A Boolean circuit admits exact real extensions, including smooth lifts. A compatible family of decoded numerical modules has a regular continuum limit when its regularity is uniformly controlled. None of these statements makes arbitrary polynomial-time real functions Lipschitz: the square root supplies an elementary counterexample.

The contraction analysis adds a second boundary. With the same fixed $0/1$ encoding, exact nonconstant Boolean semantics require Lipschitz constant at least one. Below one, the sharp numerical approximation error is $(1 - \lambda)/2$. Exact computation can be restored with changing code scales, and then

$$\Delta_H \leq \lambda^H \Delta_0$$

for genuinely layered contractive propagation. This rate is attained by Lipschitz lifts and attained within a constant factor by smooth lifts.

At the same time, local absolute implementation errors satisfy

$$e_H \leq \frac{\delta}{1 - \lambda}.$$

The favorable absolute error bound is meaningful for digital decisions only after comparison with the shrinking signal:

$$\frac{e_H}{\Delta_H} < \frac{1}{2}.$$

For fixed absolute precision this gives a bit requirement proportional to depth; other representations move the accounting into exponents, scale metadata, or the decoder. Fixed-amplitude real noise is not eliminated by changing that bookkeeping.

The central statement is therefore

contraction, precision, and code separation must be analyzed together; a small Lipschitz constant is not a free substitute for information.

The remaining representation problem is to identify efficient digital computations whose coarse modules have coherent numerical meaning, certifiable regularity, and an economical allocation of precision. That is the bridge from bounded-fan-in Boolean computation to a useful smooth sparse continuum representation.

“‘latex

CHAPTER 30

Learning Difficult Inverses by Forward Simulation

30.1 The basic idea

Many learning problems are naturally asymmetric.

Let X denote a rich underlying object and let Y denote a measurement, representation, or consequence of that object. Nature generates observations according to

$$X \sim P_X, \qquad Y \sim P_{Y|X}(\cdot|X),$$

so that the joint distribution is

$$P_{XY}(x, y) = P_X(x)P_{Y|X}(y|x).$$

Suppose that only a relatively small number of paired observations

$$(x_1, y_1), \ldots, (x_n, y_n)$$

is available. The desired learning problem, however, is the inverse one:

$$Y \longmapsto X.$$

A common example is reconstruction of an image X from a neural measurement Y, but the same structure occurs in many inverse problems.

The central observation of this chapter is that the two conditional learning problems

$$X \longrightarrow Y$$

and

$$Y \longrightarrow X$$

need not have comparable difficulty.

If the forward conditional $P_{Y|X}$ can be learned accurately from relatively few paired examples, while the inverse conditional $P_{X|Y}$ requires many more, then it may be advantageous not to learn the desired inverse directly.

Instead, one can proceed in three stages:

$$
\boxed{
\begin{array}{c}
\text{few real paired examples } (x_i, y_i) \\
\downarrow \\
\text{learn the easier forward model } \widehat{P}_{Y|X} \\
\downarrow \\
x \sim P_X, \qquad \widetilde{y} \sim \widehat{P}_{Y|X}(\cdot|x) \\
\downarrow \\
\text{arbitrarily many synthetic pairs } (x, \widetilde{y}) \\
\downarrow \\
\text{learn the difficult inverse } P_{X|Y}.
\end{array}
}
$$

The forward model is therefore used as a *simulator* for creating the paired training data required by the inverse problem.

We will refer to this idea as the *forward simulation principle for inverse learning*.

30.2 Why the two directions may have different difficulty

Consider first a deterministic noisy forward model

$$Y = F(X) + \eta,$$

where η represents measurement noise.

Learning the forward direction requires approximating

$$F : X \longrightarrow Y.$$

The inverse problem asks instead for

$$Y \longrightarrow X.$$

If F compresses information, then many different values of X may produce the same or nearly the same value of Y:

$$F(x_1) \approx F(x_2), \qquad x_1 \neq x_2.$$

The inverse is then not a well-defined deterministic function.

More generally, the correct inverse object is

$$P_{X|Y}(x|y),$$

which may be broad or multimodal.

Thus the forward and inverse problems can have quite different statistical complexities. We may have

$$n_{\text{forward}}(\epsilon) \ll n_{\text{inverse}}(\epsilon),$$

where $n_{\text{forward}}(\epsilon)$ is the number of paired examples needed to estimate the forward conditional to accuracy ϵ, while $n_{\text{inverse}}(\epsilon)$ is the corresponding sample complexity for directly learning the inverse.

This asymmetry creates the opportunity exploited below.

30.3 The ideal result

Suppose first that the marginal distribution P_X is available and that the forward conditional has been learned perfectly:

$$\widehat{P}_{Y|X}(y|x) = P_{Y|X}(y|x).$$

Generate synthetic examples by

$$X \sim P_X$$

followed by

$$\widetilde{Y} \sim \widehat{P}_{Y|X}(\cdot|X).$$

The resulting synthetic joint distribution is

$$\widetilde{P}_{XY}(x, y) = P_X(x)\widehat{P}_{Y|X}(y|x).$$

Therefore, if the learned forward model is exact,

$$\widetilde{P}_{XY}(x, y) = P_X(x)P_{Y|X}(y|x) = P_{XY}(x, y).$$

This gives the following elementary but important result.

Forward Simulation Principle. If

$$\widehat{P}_{Y|X} = P_{Y|X}$$

and synthetic inputs are drawn from the true marginal P_X, then forward simulation produces samples from exactly the true joint distribution:

$$\boxed{\widetilde{P}_{XY} = P_{XY}.}$$

Consequently,

$$\boxed{\widetilde{P}_{X|Y} = P_{X|Y}.}$$

Thus, in the ideal limit, arbitrarily many examples for learning the inverse conditional can be generated from a learned forward conditional and samples from P_X.

30.4 Proof

The proof is immediate from the factorization of the joint distribution.

By construction,

$$\widetilde{P}_{XY}(x, y) = P_X(x)\widehat{P}_{Y|X}(y|x).$$

If

$$\widehat{P}_{Y|X}(y|x) = P_{Y|X}(y|x),$$

then

$$\widetilde{P}_{XY}(x, y) = P_X(x)P_{Y|X}(y|x) = P_{XY}(x, y).$$

Equality of the joint distributions implies equality of the corresponding conditional distributions wherever $P_Y(y) > 0$:

$$\widetilde{P}_{X|Y}(x|y) = P_{X|Y}(x|y).$$

□

30.5 The approximate case

In practice neither component will be exact.

Suppose synthetic inputs are sampled from a distribution Q_X rather than exactly from P_X, and suppose the learned forward conditional is

$$\widehat{P}_{Y|X}.$$

Then the synthetic joint distribution is

$$Q_{XY}(x, y) = Q_X(x)\widehat{P}_{Y|X}(y|x).$$

There are now two distinct sources of error:

$$\boxed{\text{error in the distribution of } X + \text{error in the learned forward model.}}$$

This can be formalized in total variation distance. Let

$$d_{\mathrm{TV}}(P, Q) = \sup_A |P(A) - Q(A)|.$$

Then

$$\boxed{d_{\mathrm{TV}}(P_{XY}, Q_{XY}) \le d_{\mathrm{TV}}(P_X, Q_X) + \mathbb{E}_{X\sim P_X}\left[d_{\mathrm{TV}}\left(P_{Y|X}(\cdot|X), \widehat{P}_{Y|X}(\cdot|X)\right)\right].}$$

To see this, introduce the intermediate joint distribution

$$R_{XY}(x, y) = P_X(x)\widehat{P}_{Y|X}(y|x).$$

The triangle inequality gives

$$d_{\mathrm{TV}}(P_{XY}, Q_{XY}) \le d_{\mathrm{TV}}(P_{XY}, R_{XY}) + d_{\mathrm{TV}}(R_{XY}, Q_{XY}).$$

The first term measures forward-model error and satisfies

$$d_{\mathrm{TV}}(P_{XY}, R_{XY}) \le \mathbb{E}_{X\sim P_X} d_{\mathrm{TV}}\left(P_{Y|X}(\cdot|X), \widehat{P}_{Y|X}(\cdot|X)\right),$$

while the second satisfies

$$d_{\mathrm{TV}}(R_{XY}, Q_{XY}) \le d_{\mathrm{TV}}(P_X, Q_X).$$

Thus the quality of the synthetic inverse-training data is controlled by two quantities only: how accurately the forward law is learned and how accurately the synthetic X distribution matches the relevant real distribution.

30.6 Risk transfer from synthetic to real data

The preceding distributional statement immediately implies a learning bound.

Let g be an inverse predictor and let

$$\ell(g(Y), X)$$

be a bounded loss satisfying

$$0 \le \ell \le L.$$

Define the true risk

$$R_P(g) = \mathbb{E}_{(X,Y)\sim P_{XY}} \left[\ell(g(Y), X)\right]$$

and the synthetic risk

$$R_Q(g) = \mathbb{E}_{(X,Y)\sim Q_{XY}} \left[\ell(g(Y), X)\right].$$

Let

$$\delta = d_{\mathrm{TV}}(P_{XY}, Q_{XY}).$$

Then for every g,

$$|R_P(g) - R_Q(g)| \le L\delta.$$

Suppose $\widehat{g}$ is trained on synthetic data and satisfies

$$R_Q(\widehat{g}) \le \inf_{g\in\mathcal{G}} R_Q(g) + \epsilon_M,$$

where ϵ_M contains the statistical and optimization error remaining after using M synthetic examples.

Let

$$g_P^* = \arg\min_{g\in\mathcal{G}} R_P(g).$$

Then

$$\boxed{R_P(\widehat{g}) - R_P(g_P^*) \leq 2L\delta + \epsilon_M.}$$

Using the previous bound on δ,

$$\boxed{\begin{aligned} R_P(\widehat{g}) - R_P(g_P^*) \leq{}& 2L\, d_{\mathrm{TV}}(P_X, Q_X) \\ &+ 2L\, \mathbb{E}_{X\sim P_X} d_{\mathrm{TV}}\left(P_{Y|X}(\cdot|X), \widehat{P}_{Y|X}(\cdot|X)\right) + \epsilon_M. \end{aligned}}$$

This equation captures the basic theoretical tradeoff.

Because synthetic data can be generated in arbitrarily large quantities,

$$M \to \infty$$

can make

$$\epsilon_M \to 0$$

under appropriate learning assumptions.

However, increasing the number of synthetic examples cannot eliminate the errors

$$d_{\mathrm{TV}}(P_X, Q_X)$$

or

$$d_{\mathrm{TV}}\left(P_{Y|X}, \widehat{P}_{Y|X}\right).$$

Synthetic data can remove the sampling difficulty of learning the inverse from a finite dataset, but they cannot repair an incorrect forward model.

This is the fundamental limitation of the method.

30.7 What synthetic data do and do not provide

It is tempting to object that synthetic examples cannot contain new information because they are generated by a model learned from the original small dataset.

This objection is correct in one sense and incorrect in another.

Synthetic data do not provide new information about the true forward law $P_{Y|X}$. All information about this law ultimately originates in the real paired observations.

However, once a useful approximation

$$\widehat{P}_{Y|X}$$

has been learned, it can be evaluated at an arbitrarily large number of new values of X.

Thus the learned forward rule can be propagated throughout the support of P_X.

If only

$$n$$

real pairs were required to estimate the forward law, one can nevertheless generate

$$M \gg n$$

inverse-training pairs.

The new pairs do not improve knowledge of the forward mechanism. Rather, they make it possible to *amortize* that knowledge over the full input distribution when learning the inverse.

This distinction is crucial.

30.8 Asymmetric sample complexity

The method is useful when three conditions hold.

First, the forward problem must be easier than the inverse problem:

$$n_{\text{forward}}(\epsilon) \ll n_{\text{inverse}}(\epsilon).$$

Second, samples from the marginal distribution of X must be abundant:

$$X_1, X_2, \ldots \sim P_X.$$

Third, the learned forward model must generalize over the region of X on which synthetic examples are generated.

Under these conditions, one may have enough real paired examples to learn the forward mapping while having far too few to learn the inverse mapping directly.

Forward simulation then converts the easier learning problem into a generator of training examples for the harder problem.

Schematically,

$n_{\text{forward}} \ll n_{\text{inverse}}$	$\Longrightarrow$	learn forward first and simulate the inverse dataset.

This is the essential source of the advantage.

30.9 The importance of sampling from the correct X distribution

One should not interpret the availability of arbitrarily many synthetic inputs as permission to use arbitrary points in the ambient space.

If the forward model is trained on natural images, for example, its behavior on arbitrary vectors in pixel space may be meaningless.

The relevant requirement is

$$Q_X \approx P_X,$$

at least over the domain in which the inverse will ultimately be used.

A powerful generative model can provide precisely such samples.

Let

$$Z \sim P_Z$$

be a simple latent variable and let

$$X = G(Z)$$

be a pretrained generative model. Then

$$Q_X = G_{\#} P_Z$$

is the distribution induced by the generator.

Synthetic inverse-training data can be generated by

$$Z \sim P_Z,$$

$$X = G(Z),$$

$$\widetilde{Y} \sim \widehat{P}_{Y|X}(\cdot|X).$$

Thus

$$\boxed{Z \longrightarrow X \longrightarrow \widetilde{Y}}$$

becomes an arbitrarily large source of paired examples

$$(\widetilde{Y}, X).$$

This makes large pretrained generative models especially useful in data-poor inverse problems.

30.10 Stochastic forward models are preferable

When the real measurement contains noise, it is important that the forward model represent not merely a conditional mean

$$\widehat{F}(x) \approx \mathbb{E}[Y|X = x],$$

but ideally the entire conditional distribution

$$\widehat{P}_{Y|X}.$$

If synthetic examples are generated only as

$$\widetilde{Y} = \widehat{F}(X),$$

the inverse model may never see the variability present in real measurements.

A better simulator is

$$\widetilde{Y} \sim \widehat{P}_{Y|X}(\cdot|X),$$

so that synthetic measurements reproduce not only the expected signal but also its conditional variability.

For a model such as

$$Y = F(X) + \eta,$$

one should therefore model both

$$F(X)$$

and the appropriate distribution of η.

30.11 The inverse should usually be probabilistic

When the forward map is many-to-one, there is no unique deterministic inverse.

Suppose several values

$$x_1, x_2, \dots$$

are compatible with the same observation y.

If an inverse function g is trained with squared-error loss,

$$\mathbb{E}\left[\|X - g(Y)\|^2\right],$$

then the population optimum is

$$\boxed{g^*(y) = \mathbb{E}[X|Y = y].}$$

This conditional mean need not itself be a plausible member of the underlying data distribution.

For images, averaging several different plausible reconstructions may produce a blurred image that corresponds to none of them.

The more natural goal is therefore to learn a conditional generative model

$$q_\phi(x|y).$$

Synthetic pairs can be used to maximize, for example,

$$\mathbb{E}_{(X,Y)\sim Q_{XY}}\left[\log q_\phi(X|Y)\right].$$

In the ideal limit,

$$Q_{XY} = P_{XY}$$

and sufficient model capacity and optimization imply

$$q_\phi(x|y) \approx P_{X|Y}(x|y).$$

The learned inverse can then produce samples

$$X^{(1)}, X^{(2)}, \dots \sim q_\phi(\cdot|y),$$

representing the different underlying objects compatible with the same observation.

30.12 Connection with Bayesian inversion

The same construction has a direct Bayesian interpretation.

Bayes' rule gives

$$P_{X|Y}(x|y) = \frac{P_{Y|X}(y|x)P_X(x)}{P_Y(y)}.$$

Thus the two ingredients used by forward simulation,

$$P_X$$

and

$$P_{Y|X},$$

are exactly the two ingredients required for Bayesian inversion.
One possible strategy is therefore to perform Bayesian inference directly:

$$P_{X|Y}(x|y) \propto P_{Y|X}(y|x)P_X(x).$$

Forward simulation provides a second possibility. Instead of solving this inference problem anew for every observation y, one generates many pairs from

$$P_X P_{Y|X}$$

and trains a model that directly approximates

$$P_{X|Y}.$$

Thus forward simulation may be viewed as *amortized Bayesian inversion*.
The potentially expensive inference problem

$$P_{Y|X}, P_X \quad \longrightarrow \quad P_{X|Y}(\cdot|y)$$

is solved repeatedly during synthetic training so that subsequent inference for a new y becomes a single pass through the learned inverse model.

30.13 Connection with regularized inverse problems

Suppose

$$Y = F(X) + \eta, \qquad \eta \sim \mathcal{N}(0, \sigma^2 I).$$

Then

$$P(y|x) \propto \exp\left[-\frac{1}{2\sigma^2}\|y - F(x)\|^2\right].$$

The Bayesian MAP solution is

$$x_{\mathrm{MAP}} = \arg\min_x \left[\frac{1}{2\sigma^2}\|y - F(x)\|^2 - \log P_X(x)\right].$$

Writing

$$R(x) = -\log P_X(x),$$

this becomes the classical regularized inverse problem

$$\boxed{x_{\mathrm{MAP}} = \arg\min_x \left[\|y - F(x)\|^2 + \lambda R(x)\right].}$$

Forward simulation and regularized inversion therefore use the same two objects:

$P_{Y\|X}$	$\longleftrightarrow$	forward/data-fidelity model,
P_X	$\longleftrightarrow$	prior/regularizer.

The difference is computational.

Regularized or Bayesian inversion solves an optimization or sampling problem separately for each observed y.

Forward simulation instead uses the same ingredients to train an inverse model in advance.

30.14 Combining real and synthetic pairs

Because errors in the learned forward model create an irreducible synthetic-data bias, it is useful to retain the original real pairs when training the inverse.

A natural objective is

$$\mathcal{L}(\phi) = \lambda_{\text{real}} \frac{1}{n} \sum_{i=1}^{n} \ell_\phi(x_i, y_i) + \lambda_{\text{syn}} \frac{1}{M} \sum_{j=1}^{M} \ell_\phi(\widetilde{x}_j, \widetilde{y}_j).$$

The large synthetic dataset teaches the inverse model the global geometry implied by the forward simulator, while the real examples anchor the learned inverse to the true measurement process.

One may therefore think of the real examples as performing a calibration function that synthetic examples alone cannot provide.

30.15 Iterative refinement

The construction also suggests an iterative procedure.

Starting with a small real paired dataset, first estimate

$$\widehat{P}^{(0)}_{Y|X}.$$

Use it to generate synthetic pairs and learn

$$\widehat{P}^{(0)}_{X|Y}.$$

New real observations, when available, can then be used to improve the forward model,

$$\widehat{P}^{(0)}_{Y|X} \longrightarrow \widehat{P}^{(1)}_{Y|X},$$

after which a new and improved synthetic dataset can be generated.

Thus one can iterate

real pairs $\to$ forward model $\to$ synthetic pairs $\to$ inverse model $\to$ improved real calibration.

The bottleneck remains the accuracy of the learned forward mechanism.

30.16 Application to image reconstruction from brain activity

The image–fMRI problem is a particularly natural example.

Let

$$X = \text{visual image}$$

and

$$Y = \text{fMRI response.}$$

Only a limited number of paired measurements

$$(x_i, y_i)$$

can realistically be collected.

Directly learning

$$Y \longrightarrow X$$

from these pairs is difficult because the inverse is high-dimensional, many-to-one, and strongly underdetermined.

The forward problem,

$$X \longrightarrow Y,$$

may be substantially easier.

Suppose the real pairs are used to train

$$\widehat{P}_{\text{fMRI|image}}.$$

A pretrained image generator can then supply an enormous number of natural images

$$X_1, \ldots, X_M.$$

For each image one generates a synthetic brain response

$$\widetilde{Y}_j \sim \widehat{P}_{\text{fMRI|image}}(\cdot|X_j).$$

This produces

$$(\widetilde{Y}_j, X_j), \qquad j = 1, \ldots, M,$$

which can be used to train a conditional image generator

$$q_\phi\left(x \mid y\right).$$

The complete architecture is

$$\boxed{\begin{array}{llll} \text{few real} & (x,y) & \longrightarrow & \widehat{P}(y|x) \\ x \sim P_{\text{image}} & \longrightarrow \widehat{P}(y|x) & \longrightarrow & (x,\widetilde{y}) \\ & & & \downarrow \\ & & & q_\phi(x|y). \end{array}}$$

A diffusion model is particularly attractive for representing either the marginal image distribution P_X or the conditional inverse $q_\phi(x|y)$.

The essential principle, however, is independent of the particular generative architecture.

30.17 A general principle

The discussion can be summarized as follows.

Forward Simulation Principle for Inverse Learning. Consider a system with joint distribution

$$P_{XY} = P_X P_{Y|X}.$$

Suppose that:

1. paired observations (X, Y) are scarce;
2. unpaired samples from P_X, or a good approximation to them, are abundant;
3. the forward conditional $P_{Y|X}$ can be learned accurately from substantially fewer paired examples than are required to learn $P_{X|Y}$ directly; and
4. the learned forward model generalizes over the relevant support of P_X.

Then an effective strategy for learning the difficult inverse conditional is:

$$\boxed{\text{learn } P_{Y|X} \;\longrightarrow\; \text{simulate } P_X P_{Y|X} \;\longrightarrow\; \text{learn } P_{X|Y}.}$$

In the ideal case of an exact forward model and exact sampling from P_X, the synthetic joint distribution is identical to the true joint distribution and the procedure is statistically exact.

In the approximate case, the inverse error contains three conceptually distinct contributions:

$$\boxed{\begin{array}{c} \text{forward-model error} \\ + \\ \text{marginal-distribution error} \\ + \\ \text{finite synthetic-sample/optimization error.} \end{array}}$$

Only the last of these can be driven to zero merely by generating more synthetic data.

30.18 Discussion

The principle can be stated informally in a particularly simple way:

If the direction we want to learn is difficult, learn the easier direction and use it to manufacture training data for

The method exploits an asymmetry in sample complexity.

The scarce real data are spent on learning the conditional relation that can be estimated most efficiently. Once this relation has been learned, abundant unpaired samples from the source domain can be converted into abundant paired examples.

A difficult inverse-learning problem is thereby decomposed into

easy supervised learning + simulation + large-data inverse learning.

The construction does not create information about the physical forward mechanism that was absent from the original data. Rather, it allows the information captured by the learned forward law to be propagated over an arbitrarily large input distribution.

This distinction explains both the power and the limitation of the approach.

The general message is that the direction in which one ultimately wishes to make predictions need not be the direction in which one should initially learn from scarce data. When the causal or data-generating direction is simpler, it may be statistically preferable to learn that direction first, use it as a simulator, and only afterward learn the desired inverse.

In this sense, a hard inverse problem can sometimes be converted from a problem of scarce supervised data into a problem of forward-model accuracy. “‘

CHAPTER 31

Learning Without Weight Updates: Context, Memory, and Cultural State

31.1 Learning need not mean changing weights

In modern machine learning, learning is usually identified with a change of parameters. Given a predictor

$$f_\theta(x),$$

a learning algorithm uses data D to produce

$$\theta \longrightarrow \theta',$$

with the goal of reducing prediction error on future examples.

Large language models suggest a broader notion. A pretrained model with fixed parameters can display dramatically different capabilities depending on the context in which it is placed. Examples, instructions, intermediate results, retrieved memories, programs, and previously discovered strategies can all modify the computation performed by the same frozen network.

We should therefore distinguish the parameters of the computational substrate from the adaptive state of the overall system.

Let

$$F_\theta(x; s)$$

denote a model with fixed weights θ and adaptive state s. The state may contain a prompt, demonstrations, memories, algorithms, previously discovered solutions, external documents, or any other information accessible to the model.

The crucial observation is that

$$\boxed{\theta \text{ fixed} \quad \not\Rightarrow \quad \text{behavior fixed.}}$$

Indeed, for every state s, the frozen network defines a possibly different function

$$f_s(x) = F_\theta(x; s).$$

Thus a single frozen network induces the hypothesis class

$$\mathcal{H}_\theta = \{x \mapsto F_\theta(x; s) \ : \ s \in \mathcal{S}\}.$$

Learning can therefore take place by searching over $\mathcal{S}$, rather than over the parameter space of the network.

Instead of

$$\theta^\star = \arg\min_\theta R(\theta),$$

we may have

$$s^\star = \arg\min_{s\in\mathcal{S}} R_\theta(s),$$

where

$$R_\theta(s) = \mathbb{E}_{(x,y)\sim\mathcal{D}}\left[\ell(F_\theta(x;s),y)\right].$$

The weights remain unchanged. What changes is the computation selected from the repertoire represented by those weights.

This gives a precise sense in which a frozen model can learn.

31.2 Three forms of learning

It is useful to distinguish three different mechanisms.

First, conventional parametric learning changes the weights:

$$\theta_t \longrightarrow \theta_{t+1}.$$

Second, contextual learning changes the working state:

$$s_t \longrightarrow s_{t+1}.$$

Third, cultural learning modifies an external persistent repository:

$$\mathcal{L}_t \longrightarrow \mathcal{L}_{t+1}.$$

Thus we may write

$$\begin{aligned}
\text{parametric learning} &: \theta_t \to \theta_{t+1},\\
\text{contextual learning} &: s_t \to s_{t+1},\\
\text{cultural learning} &: \mathcal{L}_t \to \mathcal{L}_{t+1}.
\end{aligned}$$

Only the first requires changing neural weights.

For a frozen LLM,

$$\theta_{t+1} = \theta_t = \theta,$$

while

$$(s_t, \mathcal{L}_t) \to (s_{t+1}, \mathcal{L}_{t+1}).$$

The effective predictor nevertheless changes:

$$f_t(x) = F_\theta(x; s_t, \mathcal{L}_t).$$

If

$$R(f_{t+1}) < R(f_t),$$

then the system has improved from experience. Operationally, this is learning, even though no synaptic parameter of the underlying network has changed.

31.3 The adaptive state

Suppose observations arrive sequentially. An external learning rule A can update the state according to

$$s_{t+1} = A_\theta(s_t, D_t),$$

where D_t represents the information obtained at step t.

The complete dynamical system is therefore

$$\boxed{\begin{aligned} y_t &= F_\theta(x_t; s_t), \\ s_{t+1} &= A_\theta(s_t, D_t), \\ \theta_{t+1} &= \theta_t. \end{aligned}}$$

Nothing in the definition of learning requires that the adaptive variables be identical to the parameters of the neural network.

In biological terminology one might distinguish slow and fast variables. In computer science one would distinguish the program implementing an interpreter from the contents of its writable memory.

A frozen LLM together with writable external state is closer to the second picture.

The network weights define a fixed computational machine. The state determines which computation the machine currently performs.

31.4 Frozen weights define a family of hypotheses

The following elementary observation is conceptually important.

Proposition 1: Frozen-weight learning. Let $F_\theta(x; s)$ be fixed with respect to θ. Suppose there exist $s_1, s_2 \in \mathcal{S}$ such that

$$F_\theta(\cdot; s_1) \neq F_\theta(\cdot; s_2).$$

Then adaptation of s can change the effective hypothesis implemented by the system without changing θ.

If, for some learning problem,

$$R_\theta(s_2) < R_\theta(s_1),$$

then the transition

$$s_1 \to s_2$$

constitutes an improvement in expected performance with completely frozen model weights.

The proposition is almost tautological. Its importance is that the usual identification

$$\text{learning} \equiv \text{weight change}$$

is unnecessarily restrictive.

The relevant hypothesis class is not merely the function represented by the network with one fixed prompt. It is

$$\mathcal{H}_\theta = \{F_\theta(\cdot; s) : s \in \mathcal{S}\}.$$

A sufficiently expressive frozen model may therefore contain an enormous family of potential behaviors. Learning determines which member of this family is selected.

31.5 Context as a program

For a language model, a particularly important component of s is the context.

A context need not merely describe a task. It may specify an algorithm.

For example, a context can say:

> Divide the sequence into pairs, compute parity within each pair, recursively apply the same procedure to the resulting bits, and return the final bit.

If the frozen model can correctly interpret and execute these instructions, the context acts as a program.

We may therefore write

$$F_\theta(x; p),$$

where p denotes a program expressed in the language understood by the model.

The frozen parameters θ then play the role of an interpreter.

This viewpoint suggests the decomposition

$$\boxed{\text{computation} = \text{fixed interpreter} + \text{variable program} + \text{variable data}.}$$

In ordinary neural-network learning, much of the task-specific computation is encoded into θ.

In a frozen LLM, some of it can instead be encoded into p.

The distinction is analogous to the difference between manufacturing a new computer for every task and running a new program on the same computer.

31.6 A universality observation

The preceding analogy can be made stronger.

Suppose the frozen system F_θ is capable of interpreting programs from some universal programming language. Let h be any computable function representable by a program p_h. Then

$$F_\theta(x; p_h) = h(x),$$

up to the computational resources available to the system.

Consequently,

$$\mathcal{H}_\theta$$

may contain an extremely rich collection of functions even though θ never changes.

More generally, consider a computable learning algorithm producing

$$h_0, h_1, h_2, \ldots$$

from a sequence of observations.

If each h_t has a program representation p_t, then a universal frozen interpreter can realize

$$F_\theta(x; p_t) = h_t(x).$$

Learning is then equivalent to

$$p_t \to p_{t+1},$$

rather than

$$\theta_t \to \theta_{t+1}.$$

Thus, subject to representational and computational limits,

a fixed universal interpreter plus writable memory can implement an adaptive sequence of hypotheses.

This is closely related to the classical separation between a Turing machine's fixed transition rules and the changing contents of its tape.

The transition function does not have to change for the machine to perform an indefinitely long sequence of different computations.

31.7 Attention and associative memory

Transformers provide a natural mechanism for realizing this architecture.

Attention can be interpreted as associative retrieval. Given a query q, stored key–value pairs

$$(k_i, v_i)$$

are accessed according to a similarity-dependent weighting:

$$\mathrm{Attn}(q) = \sum_i \alpha_i(q) v_i,$$

with

$$\alpha_i(q) = \frac{\exp\left(q^\top k_i\right)}{\sum_j \exp(q^\top k_j)}.$$

Thus the context acts as a temporary associative memory.

The fixed weights determine how queries are generated, how memories are addressed, and how retrieved information is transformed. The actual contents of memory can vary from episode to episode.

This suggests a useful decomposition:

θ : rules for computation and memory access,

s : current contents of computational memory.

From this perspective, in-context learning is not mysterious. The weights implement machinery capable of interpreting and manipulating state. The context supplies the state on which this machinery operates.

31.8 Transient context is not yet persistent learning

There is, however, an important distinction.

Suppose a model solves a problem because useful examples appear in its context window. If the context is erased after the interaction, the improvement disappears.

This is adaptation within an episode, but there is no persistent learning across episodes.

Persistent learning requires a state variable that survives.

We may formulate a simple persistence condition:

Learning across episodes requires some task-relevant state that survives across episodes.

Weight updates provide one such state.
External memory provides another.
Let

$$\mathcal{L}_t$$

denote a persistent library. After an interaction, the system may extract an artifact a_t, such as

- a successful prompt,
- an algorithm,
- a proof,
- a subroutine,
- an error-correction rule,
- an example,
- a compressed summary,
- a verification procedure.

The library is updated as

$$\mathcal{L}_{t+1} = U(\mathcal{L}_t, a_t).$$

At a future time the frozen network retrieves relevant elements of the library:

$$s_{t+1} = C_\theta(x_{t+1}, \mathcal{L}_{t+1}).$$

Thus persistent adaptation can occur even though

$$\theta_{t+1} = \theta_t.$$

The finite context window then becomes analogous to working memory, while $\mathcal{L}_t$ plays the role of long-term memory.

31.9 Cultural learning

This architecture gives a particularly simple model of cumulative culture.

Let generation g have state

$$\mathcal{S}_g = (\theta, c_g, \mathcal{L}_g),$$

where

- θ is a frozen pretrained model,
- c_g is the working context,
- $\mathcal{L}_g$ is the inherited cultural library.

Suppose generation g encounters task T_g. It produces an artifact

$$a_g = G_\theta(T_g, c_g, \mathcal{L}_g),$$

and stores useful information:

$$\mathcal{L}_{g+1} = U(\mathcal{L}_g, a_g).$$

The next generation begins with the same neural parameters,

$$\theta_{g+1} = \theta_g,$$

but a different cultural environment:

$$\mathcal{L}_{g+1} \neq \mathcal{L}_g.$$

Its effective behavior is therefore

$$f_{g+1}(x) = F_\theta(x; c_{g+1}, \mathcal{L}_{g+1}),$$

which can be superior to

$$f_g(x) = F_\theta(x; c_g, \mathcal{L}_g).$$

The locus of adaptation is culture rather than weights.

This leads to the central principle:

Culture is persistent external adaptive state for a fixed computational substrate.

The analogy with biological cultural evolution is direct. Human brains do not have to genetically encode calculus, writing, chess, or computer programming. A relatively stable biological substrate interacts with an expanding external repository of language, tools, algorithms, books, and institutions.

Much of the increase in effective human competence resides outside the individual brain.

A frozen LLM equipped with persistent external memory provides a minimal artificial model of the same phenomenon.

31.10 Cumulative culture

The important case is not merely memory, but cumulative memory.

Suppose

$$\mathcal{L}_0 \subseteq \mathcal{L}_1 \subseteq \mathcal{L}_2 \subseteq \cdots .$$

At generation g, the system need not solve a problem from scratch. It can exploit discoveries made by previous generations.

For a sequence of increasingly difficult tasks

$$T_1, T_2, \ldots, T_m,$$

we may define the cost of solving T_g given the inherited library by

$$C(T_g \mid \mathcal{L}_{g-1}).$$

Cumulative culture is present when inherited artifacts systematically reduce this cost:

$$C(T_g \mid \mathcal{L}_{g-1}) < C(T_g \mid \mathcal{L}_0).$$

An even stronger form occurs when

$$T_g$$

cannot reliably be solved by the frozen model from its initial state, but becomes solvable using artifacts accumulated while solving earlier tasks.

Then the sequence

$$T_1 \to T_2 \to \cdots \to T_g$$

creates a genuine historical dependency.

The competence of generation g depends on the path taken by previous generations.

This is one of the defining characteristics of cumulative culture.

31.11 Where is the learning algorithm?

There is an interesting inversion here.

During conventional training, the learning algorithm acts outside the network:

$$\theta_{t+1} = A(\theta_t, D_t).$$

After pretraining, however, a sufficiently capable network may itself implement part of an adaptation algorithm.

For example, the frozen model can

$$\text{propose} \to \text{test} \to \text{criticize} \to \text{revise} \to \text{store}.$$

Thus one may have

$$s_{t+1} = A_\theta(s_t, D_t),$$

where the learning operator itself is partly represented in the fixed weights.

Pretraining has then learned not merely a predictor, but a mechanism capable of subsequent learning through state changes.

This is closely related to meta-learning.

The slow learning process modifies

$$\theta.$$

The result is a system that can subsequently perform fast learning by modifying

$$s.$$

Symbolically,

$$\boxed{\text{pretraining learns a learner.}}$$

The learner may then continue adapting after its weights have been frozen.

31.12 Generalization with frozen weights

This perspective also clarifies statistical complexity.

Suppose that after freezing θ, learning consists of choosing

$$s \in \mathcal{S}.$$

The relevant hypothesis class is

$$\mathcal{H}_\theta = \{F_\theta(\cdot; s) : s \in \mathcal{S}\}.$$

Generalization therefore depends on the complexity of the state-dependent family

$$\mathcal{H}_\theta,$$

not directly on the number of frozen parameters.

For example, suppose $\mathcal{S}$ contains M possible states and the loss is bounded in $[0, 1]$. Standard uniform-convergence arguments give, with probability at least $1 - \delta$,

$$R(s) \le \widehat{R}_n(s) + O\left(\sqrt{\frac{\log M + \log(1/\delta)}{n}}\right)$$

simultaneously for all $s \in \mathcal{S}$.

For an infinite state space, analogous statements involve the Rademacher complexity or another measure of the induced class

$$\mathcal{H}_\theta.$$

This observation is useful conceptually.

A network may contain billions of frozen parameters while adaptation occurs over a much smaller set of prompts, programs, or memory configurations.

Conversely, an unrestricted external state can define a very large hypothesis class even though the network weights never change.

Thus statistical complexity belongs to the complete adaptive system, not merely to its neural parameters.

31.13 Context as fast parameters

Functionally, context can sometimes be regarded as a form of fast parameterization.

We may compare

$$f_\theta(x)$$

with

$$F_\theta(x; s).$$

The pair

$$(\theta, s)$$

jointly determines the computation.

The difference is primarily one of timescale and representation:

	θ	s
timescale	slow	fast
location	network	context/memory
persistence	intrinsic	optional
representation	distributed weights	explicit state
update mechanism	optimization	search/write/retrieval

Calling s "fast weights" can therefore be useful at the functional level, but the distinction should not be erased. Explicit external state has properties that ordinary neural weights do not have: it can

be inspected, copied, communicated, edited, accumulated, and transmitted between independent agents.

Those properties are exactly what make cultural evolution possible.

31.14 The limits of frozen-weight learning

Frozen weights do not imply frozen competence, but they do impose constraints.

Let

$$\mathcal{R}_\theta \subseteq \mathcal{H}_\theta$$

denote the set of hypotheses that are not only representable but actually reachable by the state-update process.

Improvement is restricted by at least four factors.

First, there is an *interpretation limit*. The frozen model must be able to understand and execute the representations stored in external memory.

Second, there is a *computational limit*. Even if an algorithm can be represented, executing it may require more depth, time, precision, or context than the model possesses.

Third, there is a *search limit*. A useful state may exist but may be difficult to discover.

Fourth, there is a *retrieval limit*. A large cultural library is useless if the relevant components cannot be found and inserted into working memory when needed.

Thus

$$\text{potential competence} \neq \text{reachable competence}.$$

The distinction between these two quantities may be central to understanding frozen LLMs.

31.15 Performance ceilings and indefinitely many improvements

There is also an important distinction between a bound on performance and a bound on the number of improvements.

For a normalized performance measure,

$$0 \leq P_g \leq 1.$$

Thus performance clearly has a ceiling.

This does not imply that there must exist some finite generation G after which no further strict improvement is possible.

For example,

$$P_g = 1 - \frac{1}{g+1}$$

satisfies

$$P_{g+1} > P_g$$

for every finite g, although

$$P_g \to 1.$$

Therefore,

bounded performance does not imply a finite bound on the number of improvements.

The distinction becomes even more important when the task distribution itself expands.

A frozen model may successively acquire procedures for

$$T_1, T_2, T_3, \ldots$$

while never changing its weights.

Whether this process eventually stops depends on the computational capacity of the fixed interpreter, the richness of the external state, the search mechanism, and the structure of the task sequence.

There is no contradiction between fixed weights and an indefinitely growing cultural library.

31.16 An extended-system view of intelligence

The usual description of an LLM identifies the intelligent system with its neural network:

$$\text{system} = \theta.$$

For adaptive agents this may be the wrong boundary.

A more appropriate description is

$$\boxed{\text{system} = (\theta, \text{working context}, \text{persistent memory}, \text{tools}, \text{update rules}).}$$

The weights are only one component.

This is especially important when evaluating whether a system "learns."

If we examine only

$$\theta,$$

nothing changes.

If we examine the full state

$$(\theta, s_t, \mathcal{L}_t),$$

the system may undergo substantial and persistent adaptation.

Learning should therefore be defined at the level of the system whose future behavior is being studied.

31.17 A possible definition

The preceding discussion suggests the following definition.

Definition: learning by external state. A system with fixed parameters θ exhibits learning by external state if there exists a persistent state s_t satisfying

$$s_{t+1} = A_\theta(s_t, D_t)$$

such that experience-dependent changes in s_t produce persistent changes in future behavior,

$$F_\theta(\cdot; s_{t+1}) \neq F_\theta(\cdot; s_t),$$

and improve expected performance on some relevant distribution or sequence of tasks.

This definition excludes a mere change of input.

It requires

1. dependence on experience,
2. modification of an adaptive state,
3. persistence of that modification,
4. a consequent change in future behavior.

It includes weight learning as the special case

$$s = \theta,$$

but also includes prompt libraries, external memory, programs, cultural artifacts, and other forms of persistent state.

Thus conventional neural learning is one member of a broader class.

31.18 A hierarchy of adaptation

We can now distinguish several levels of adaptation:

Level 0:	fixed weights, fixed state,
Level 1:	fixed weights, transient context,
Level 2:	fixed weights, persistent external state,
Level 3:	fixed weights, cumulative cultural state,
Level 4:	weight adaptation plus external cultural state.

A conventional prompted LLM is primarily Level 1.

An LLM supplied with persistent memories is Level 2.

A population or sequence of frozen LLM instances that inherits and improves a shared library is Level 3.

Humans appear closer to Level 4: individual brains learn, but they also inherit an enormous accumulated cultural state.

The distinction is important because Level 3 can display substantial cumulative improvement even though the neural substrate is exactly unchanged.

31.19 A minimal experimental paradigm

The theory suggests a simple experiment.

Choose a frozen language model

$$F_\theta$$

and never modify its parameters.

Construct a sequence of tasks

$$T_1, T_2, \ldots, T_K$$

with shared compositional structure.

Begin with

$$\mathcal{L}_0 = \emptyset.$$

For each generation g,

$$
\begin{aligned}
a_g &= F_\theta(T_g; \mathcal{L}_{g-1}),\\
q_g &= \mathrm{Evaluate}(a_g, T_g),\\
\mathcal{L}_g &= U(\mathcal{L}_{g-1}, a_g, q_g).
\end{aligned}
$$

Then compare performance on T_g under two conditions:

$$P(T_g \mid \mathcal{L}_{g-1})$$

and

$$P(T_g \mid \mathcal{L}_0).$$

The quantity

$$\Delta_g = P(T_g \mid \mathcal{L}_{g-1}) - P(T_g \mid \mathcal{L}_0)$$

measures the benefit of accumulated culture.

A convincing demonstration of cumulative learning would show

$$\Delta_g > 0$$

for increasingly difficult tasks, while verifying exactly that

$$\theta_g = \theta_0 \qquad \forall g.$$

Stronger evidence would show that some later tasks are solved only after appropriate intermediate artifacts have entered the library.

Such an experiment separates improvement due to pretrained weights from improvement due to post-training cultural accumulation.

31.20 The main principle

The usual picture of machine learning is

$$\text{experience} \longrightarrow \text{weight change} \longrightarrow \text{improved behavior}.$$

The more general picture is

$$\text{experience} \longrightarrow \text{state change} \longrightarrow \text{improved behavior}.$$

Weights are one possible location of adaptive state, but not the only one.

For sufficiently expressive frozen models, context, external memory, programs, and accumulated artifacts can provide another.

The most compact statement is therefore

Frozen weights do not imply frozen competence.

A pretrained network can be viewed as a fixed interpreter defining a large space of possible computations. Context selects computations from this space. Persistent memory allows selections and discoveries to survive across episodes. Cultural memory allows them to accumulate across generations.

Thus the evolution of competence can occur according to

$$\theta = \text{constant}, \qquad \mathcal{L}_0 \to \mathcal{L}_1 \to \mathcal{L}_2 \to \cdots.$$

In this regime, what learns is not the neural network considered in isolation.

What learns is the larger dynamical system consisting of the frozen network together with its changing external state.

And what evolves cumulatively is not necessarily the model.

It may be its culture.

Appendix

CHAPTER A

Stability, ERM, and the Foundations of Learnability

The classical analysis of supervised learning is often framed in terms of capacity measures such as VC dimension or Rademacher complexity. An alternative but equivalent viewpoint focuses on the stability *of the learning algorithm with respect to perturbations of its training data. In this chapter we recall a statistical notion of stability based on leave-one-out cross-validation (CV_{loo}) and the theorem of Mukherjee, Niyogi, and Poggio that characterizes consistency of empirical risk minimization (ERM) in terms of this stability.*

A.1 Learning setup and ERM

Let $\mathcal{X}$ be an input space and $\mathcal{Y}$ an output space (e.g. $\mathcal{Y} = \{0, 1\}$ or $\mathbb{R}$). A learning problem is specified by an unknown distribution P on $\mathcal{Z} = \mathcal{X} \times \mathcal{Y}$. We observe an i.i.d. training sample

$$S_n = (z_1, \dots, z_n), \qquad z_i = (x_i, y_i) \sim P,$$

and choose a hypothesis $h \in \mathcal{H}$ from a hypothesis class $\mathcal{H}$.

A loss function $\ell : \mathcal{H} \times \mathcal{Z} \to [0, 1]$ (or more generally a bounded nonnegative function) measures prediction error. The *true risk* of h is

$$L(h) \;=\; \mathbb{E}_{z \sim P}[\ell(h, z)],$$

and the *empirical risk* on S_n is

$$L_n(h) \;=\; \frac{1}{n} \sum_{i=1}^{n} \ell(h, z_i).$$

The empirical risk minimization (ERM) principle selects

$$\hat{h}_n \;\in\; \arg\min_{h \in \mathcal{H}} L_n(h).$$

We say that ERM over $\mathcal{H}$ is *(universally) consistent* if

$$L(\hat{h}_n) \;\xrightarrow[n \to \infty]{P}\; \inf_{h \in \mathcal{H}} L(h),$$

for every distribution P on $\mathcal{Z}$.

A.2 CV_{loo} stability

Let $\mathcal{A}_n$ denote a (possibly randomized) learning algorithm that, given a sample S_n, outputs a hypothesis $\mathcal{A}_n(S_n) \in \mathcal{H}$. For $i \in \{1, \dots, n\}$, let $S_n^{(i)}$ denote the sample with the i-th point removed:

$$S_n^{(i)} = (z_1, \dots, z_{i-1}, z_{i+1}, \dots, z_n).$$

Intuitively, *leave-one-out* stability requires that removing a single training point does not significantly change the prediction on that point.

Definition 10 ($\mathrm{CV}_{\mathrm{loo}}$ stability)**.** *A sequence of algorithms $(\mathcal{A}_n)_{n\geq 1}$ is said to have CV_{loo} stability with parameters (β_n) if*

$$\frac{1}{n}\sum_{i=1}^{n} \mathbb{E}\Big[\big|\ell(\mathcal{A}_n(S_n), z_i) - \ell(\mathcal{A}_{n-1}(S_n^{(i)}), z_i)\big|\Big] \;\leq\; \beta_n,$$

for all n, where the expectation is over the i.i.d. draw of S_n (and any internal randomness of $\mathcal{A}_n$). We say the algorithm is asymptotically CV_{loo}-stable *if $\beta_n \to 0$ as $n \to \infty$.*

There are several related stability notions (uniform stability, hypothesis stability, etc.). The $\mathrm{CV}_{\mathrm{loo}}$ form is directly linked to cross-validation and has an especially clean connection to ERM consistency.

A.3 Stability and generalization

We now state the main result of Mukherjee, Niyogi, and Poggio. In the wording below, we preserve the original informal summary of the theorem.

Theorem 30 (Mukherjee–Niyogi–Poggio)**.** *We propose a statistical form of stability, defined in terms of the prop- erty of cross-validation leave-one-out (CVloo) stability, and*

1. *show that it is sufficient, in general, for generalization, that is convergence in probability of the empirical error to the expected error, for any algorithm satisfying it.*
2. *Our second observation is that for bounded loss classes, CVloo stability is necessary and sufficient for consistency of ERM.*

Here "generalization" in part (a) means that for any asymptotically $\mathrm{CV}_{\mathrm{loo}}$-stable algorithm $(\mathcal{A}_n)$,

$$L(\mathcal{A}_n(S_n)) - L_n(\mathcal{A}_n(S_n)) \;\xrightarrow[n\to\infty]{P}\; 0.$$

Part (b) shows that, for bounded loss classes, ERM is consistent *if and only if* it is $\mathrm{CV}_{\mathrm{loo}}$-stable. Thus stability is not merely sufficient; it is an exact characterization of when ERM behaves well.

A.4 Stability as a modeling requirement

From a modeling standpoint, Theorem 30 has a simple but strong implication:

- If we take ERM (or approximate ERM) as the fundamental inductive principle underlying learning, then *stability is required* for consistency.

- Instability—in the sense that removing or slightly perturbing a single training point can qualitatively change the predictor—rules out consistency of ERM for bounded losses.

In the remainder of the book we will use this observation in the following direction: *we impose stability as a requirement* on realistic learning problems, and ask what structural properties of target functions follow from this requirement. In particular, in the next chapter we argue that stability forces a kind of *genericity* of targets, and that this genericity in turn makes optimization by gradient-based methods possible.

A.5 Implementation

This section details a conceptual, fully functional PyTorch implementation of the Associative Turing Machine (ATM). The architecture natively enforces the sparse compositionality constraint by dynamically managing a fixed-size buffer and restricting the router's input strictly to relational metadata, rather than high-dimensional feature vectors.

A.5.1 The Evolutionary Router and AM Bank

We first define the two core neural modules. The router network is intentionally minimal, reducing the parameter space to ensure efficient navigation by an evolutionary algorithm.

Listing A.1: Core modules for the ATM: The Metadata Router and Associative Memory Bank.

```
import torch
import torch.nn as nn

class MetadataRouter(nn.Module):
    """
    The Evolutionary Router.
    Evaluates relational metadata to output priority scores.
    It does NOT access the actual high-dimensional feature vectors.
    """
    def __init__(self, metadata_dim=3, hidden_dim=16):
        super().__init__()
        # A minimal network, ideal for NEAT or simple genetic algorithms
        self.net = nn.Sequential(
            nn.Linear(metadata_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 1)
        )

    def forward(self, metadata):
        # metadata shape: (Batch, Active_Length, Metadata_Dim)
        # Returns scores: (Batch, Active_Length)
        return self.net(metadata).squeeze(-1)


class AssociativeMemoryBank(nn.Module):
    """
```

```
    The O(1) Associative Memory Bank.
    Strictly accepts two vectors and computes a localized operation.
    """
    def __init__(self, feature_dim, hidden_dim=64):
        super().__init__()
        # Bounded capacity enforcing sparse compositionality.
        self.net = nn.Sequential(
            nn.Linear(feature_dim * 2, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, feature_dim)
        )

    def forward(self, x1, x2):
        # Concatenate the two selected nodes
        x = torch.cat([x1, x2], dim=-1) # Shape: (Batch, feature_dim * 2)
        return self.net(x)
```

A.5.2 The Core ATM Architecture

The primary ATM class orchestrates the dynamic buffer routing loop. By maintaining a separate metadata tensor, the router's decisions are based entirely on the structural topology of the computational tree being built, independent of the underlying feature data.

Listing A.2: The main Associative Turing Machine loop.

```
class AssociativeTuringMachine(nn.Module):
    def __init__(self, input_len, max_steps, feature_dim):
        super().__init__()
        self.input_len = input_len
        self.max_steps = max_steps
        self.feature_dim = feature_dim

        # Metadata dim = 3: [relative_age, computational_depth, is_available]
        self.metadata_dim = 3

        self.router = MetadataRouter(self.metadata_dim)
        self.am_bank = AssociativeMemoryBank(feature_dim)

    def forward(self, x):
        """
        x shape: (Batch, input_len, feature_dim)
        """
        batch_size = x.size(0)
        device = x.device

        # 1. Initialize the Working Memory Buffer
        # Total capacity = initial inputs + maximum intermediate steps
        total_capacity = self.input_len + self.max_steps
        buffer = torch.zeros(batch_size, total_capacity, self.feature_dim, device=d
```

```
        buffer[:, :self.input_len, :] = x

        # 2. Initialize the Metadata Tensor
        # Shape: (Batch, total_capacity, metadata_dim)
        metadata = torch.zeros(batch_size, total_capacity, self.metadata_dim, devic

        # Setup initial metadata for the raw inputs
        for i in range(self.input_len):
            metadata[:, i, 0] = i           # Relative age (0 to N-1)
            metadata[:, i, 1] = 0           # Depth (raw inputs are depth 0)
            metadata[:, i, 2] = 1.0         # is_available (1.0 = true)

        active_len = self.input_len
        batch_indices = torch.arange(batch_size, device=device)

        # 3. The Computational Loop
        for step in range(self.max_steps):

            # Extract metadata for currently active items
            current_metadata = metadata[:, :active_len, :]

            # --- ROUTER EXECUTION ---
            # Get priority scores based strictly on metadata
            scores = self.router(current_metadata)

            # Mask out unavailable items by setting score to -inf
            mask = (current_metadata[:, :, 2] == 0.0)
            scores = scores.masked_fill(mask, float('-inf'))

            # Extract the indices of the top 2 scores
            _, top_indices = torch.topk(scores, k=2, dim=-1)
            idx1, idx2 = top_indices[:, 0], top_indices[:, 1]

            # Mark these indices as used (is_available = 0.0)
            metadata[:, :, 2].scatter_(1, idx1.unsqueeze(-1), 0.0)
            metadata[:, :, 2].scatter_(1, idx2.unsqueeze(-1), 0.0)

            # --- EXTRACT FEATURES ---
            x1 = buffer[batch_indices, idx1]
            x2 = buffer[batch_indices, idx2]

            # --- AM BANK EXECUTION ---
            y = self.am_bank(x1, x2)

            # --- WRITE BACK TO BUFFER ---
            buffer[:, active_len, :] = y

            # Update metadata for the newly created intermediate variable
```

```
            depth1 = metadata[batch_indices, idx1, 1]
            depth2 = metadata[batch_indices, idx2, 1]
            new_depth = torch.max(depth1, depth2) + 1

            metadata[:, active_len, 0] = active_len # New relative age
            metadata[:, active_len, 1] = new_depth  # New depth
            metadata[:, active_len, 2] = 1.0        # Available for future routing

            active_len += 1

        # The final answer is the last item written to the buffer
        final_output = buffer[:, active_len - 1, :]
        return final_output
```

A.5.3 Integration with Evolutionary Algorithms

Traditional gradient descent is largely incompatible with this dynamic routing architecture because discrete selection operations (e.g., `torch.topk`) break the differentiable computational graph. However, by employing an evolutionary strategy, this non-differentiability is no longer a hindrance.

To evolve the router, one interacts directly with the network's state dictionary. The process entails extracting `atm.router.state_dict()`, flattening the parameters into a one-dimensional "genotype" vector, and applying mutational Gaussian noise. During crossover, parameter vectors are exchanged between high-performing models before being reloaded via `atm.router.load_state_dict()`. Because the forward pass does not require gradient tracking, discrete evaluations function perfectly, allowing fitness to be calculated strictly based on the accuracy of the `final_output` against the target labels.

CHAPTER B

Compositionality in Machine Learning and Physics

We formalize a bridge between efficient computation and learnability via bounded–fan-in compositional structure. On the learning side, we prove an ML Approximation Theorem*: when a target factors through a bounded–fan-in DAG with maximal local arity d^*, the relevant approximation and sample–complexity exponents depend on d^* rather than the ambient dimension d. On the physics side, we state a* finite-horizon compilation principle *(explicitly conditional): whenever a finite-horizon evolution map admits a* uniform *simulator using a finite set of bounded-arity primitives, the map compiles to a P-uniform bounded–fan-in DAG whose size is proportional to the simulator cost. Illustrative cases include discrete-time lattice-local models and k-local quantum dynamics compiled into uniform two-qubit circuits. Limitations include chaotic sensitivity, long-range or dense couplings, precision constraints, and thermodynamic considerations; all statements are finite-precision, finite-horizon, and conditional on uniform simulability.*

B.1 Introduction

Modern deep learning owes as much to *structure* as to *scale*. Many of the functions we hope to learn are not generic mappings on high-dimensional spaces but the outputs of computations that decompose into smaller, interacting parts arranged hierarchically. This observation has two complementary faces—algorithmic and physical.

The algorithmic view From the standpoint of computation, any function evaluable by a polynomial-time Turing machine admits a polynomial-size Boolean circuit of bounded fan-in [**Arora_Barak_2009**, **sipser1996intro**]. Interpreting such a circuit as a directed acyclic graph (DAG) exposes its *compositional structure*: each node is a local operation on a small number of inputs, and the full computation emerges from a finite sequence of such local interactions. In the continuous setting, this structure is mirrored by deep networks, whose layers correspond to circuit levels. Thus, efficient Turing computability implies that the underlying function is *compositionally sparse*: at each stage it depends on a bounded number of variables. In this sense, the success of deep architectures reflects the structure of efficient computation.

The physical view In physical systems, efficient Turing computability cannot be assumed *a priori*. What is often available is *finite-horizon locality*: many models bound the spread of influence over

time (e.g., relativistic light cones or Lieb–Robinson bounds in quantum lattices [**lieb1972finite**, **NachtergaeleSims2010**]). When such locality is paired with a *uniform* simulator built from bounded-arity primitives, the finite-horizon input–output map can be *compiled* into a bounded–fan-in DAG whose structure reflects the causal neighborhood at horizon T. This is an *algorithmic* statement about compilation, not a universal physical law about bounded *physical* degree: long-range or dense couplings (e.g., Coulomb, mean-field) may violate bounded degree even though a uniform simulator—and hence a bounded–fan-in DAG—still exists with polynomial overhead.

The resulting lesson is conditional but practical: whenever finite-horizon locality and uniform simulability hold, the effective complexity is governed not by the ambient dimension d but by the *algorithmic* largest local arity d^* (the size of the neighborhoods feeding each compiled node) and by the horizon/depth T over which interactions unfold. Limits remain: chaotic sensitivity can amplify constants with T, exact-real outputs are excluded (finite precision), and thermodynamic-limit properties need not be decidable.

From observation to theorem. We make the intuition precise via two complementary results, separating what is provable in general from what holds under explicit hypotheses. On the learning side, the *ML Approximation Theorem* shows that when a target factors through a bounded–fan-in DAG, the usual exponent d/r (ambient dimension over smoothness) is replaced by d^*/r, where d^* is the maximal local arity in the DAG. On the physics side, rather than asserting a universal causal–locality law, we prove a *finite-horizon compilation principle*: whenever a finite-horizon evolution map admits a *uniform* simulator using a fixed finite set of bounded-arity primitives with cost $s(n, T, \varepsilon)$, it compiles to a *polynomial-time uniform* bounded–fan-in DAG of size $O(s(n, T, \varepsilon))$. *Examples* (not universal claims) include discrete-time lattice-local updates with finite propagation and k-local quantum dynamics compiled to two-qubit circuits. Together, these results identify a common mathematical structure—bounded local arity d^* and horizon/depth T—that governs approximation and learnability; the physics connection is *conditional* (finite precision, finite horizon, uniform simulability), not automatic.

Historical background and conceptual context

The route from Turing machines to circuits is a cornerstone of complexity theory and underlies the definition of major classes such as P and NC [**Arora_Barak_2009**]. Deep learning, viewed through this lens, can be seen as the continuous analogue of polynomial-size, bounded–fan-in circuit computation [**poggio_deep_shallow_2017**, 73]. Approximation theory has long established that deep networks approximate hierarchical functions exponentially more efficiently than shallow ones, and recent results on compositional sparsity [**poggiofraser2024**, **Poggio2025**] formalize this intuition.

In physics, locality appears in multiple guises. In classical mechanics, it is encoded in differential equations: the rate of change at a point depends on derivatives—i.e., infinitesimal neighborhoods. In electromagnetism, Maxwell's equations forbid instantaneous action at a distance; disturbances propagate at finite speed c. In quantum lattice systems, the Lieb–Robinson theorem [**lieb1972finite**, **NachtergaeleSims2010**, **bravyi2006lieb**] bounds the velocity of information propagation, establishing an effective light cone even in nonrelativistic settings. Locality, in all these forms, implies a bounded causal neighborhood at any finite time horizon—an analogue of bounded fan-in in computation.

A more speculative connection has also captured attention: the *simulation hypothesis*. If every physical process were efficiently simulable by a digital computer, then the universe would instantiate an efficiently computable function, and its regularities would follow from the same com-

positional constraints as in machine learning. A less metaphysical and more plausible explanation lies closer to established physics: finite information propagation and local interactions suffice to generate the same sparse, hierarchical patterns without assuming that the universe is literally a computation.

Organization of the paper. Section B.4 states and proves the *ML Approximation Theorem*, showing that approximation and sample–complexity rates depend on the *largest local arity* rather than the ambient dimension. Section B.5 presents the *finite-horizon compilation principle* (a conditional, algorithmic statement), together with two illustrative examples—discrete-time lattice-local updates with finite propagation and k-local quantum dynamics compiled to two-qubit circuits—and clarifies caveats (long-range/dense couplings, chaotic sensitivity, precision and horizon dependence). Section B.6 discusses implications for learnability and optimization geometry, including explicit Rademacher bounds and Hessian-structure consequences aligned with compositional DAGs. Section B.7 analyzes limitations and non-theorems: effects of chaos (growth with horizon), undecidability in thermodynamic-limit questions, the role of uniformity versus non-uniform advice, and complexity barriers (e.g., QMA-hardness). The conclusion situates these results within broader questions about when physical evolutions are *efficiently computable* or, more modestly, *inherit compositional structure* precisely when uniform finite-horizon simulability is available.

Two Structural Principles

(A) ML Approximation Principle. Let f factor through a depth-L bounded–fan-in DAG with node maps $g_v \in C^r([0,1]^{d_v})$, $d_v \le d^*$, total size s. Then for any $\varepsilon \in (0, 1/4)$ there exists a ReLU network f_ε with

$$\|f - f_\varepsilon\|_\infty \le \varepsilon, \qquad \mathrm{depth}(f_\varepsilon) \le c\, L \log(1/\varepsilon), \quad \mathrm{size}(f_\varepsilon) \le C \sum_v \varepsilon_v^{-d_v/r} \log \tfrac{1}{\varepsilon_v},$$

under the telescoping constraint $\sum_{\ell=1}^{L} \left(\prod_{j>\ell} L_j \right) \max_{v \in \ell} \varepsilon_v \le \varepsilon$. If $\max_\ell L_\ell \le \bar{L}$ and $\varepsilon_v \asymp \varepsilon$,

$$\mathrm{size}(f_\varepsilon) \;\le\; C'\, s\, \varepsilon^{-d^*/r} \log \tfrac{1}{\varepsilon}.$$

(B) Finite-Horizon Causal Compilation. Consider a system on a bounded-degree graph (Δ) with local radius-r updates $\phi_{t,i}$ that are L-Lipschitz and C^r. If propagation is finite, i.e.

$$x_{B_{R(T)}(i)} = x'_{B_{R(T)}(i)} \;\Rightarrow\; (U_T \circ \cdots \circ U_1(x))_i = (U_T \circ \cdots \circ U_1(x'))_i,$$

then for any finite region Λ and horizon T, the input–output map $\mathcal{F}_{\Lambda,T}(x) := (U_T \circ \cdots \circ U_1(x))_\Lambda$ factors through a depth-T DAG with fan-in $\le \kappa(\Delta, r)$ and size $s = O\big(|\Lambda|\,(\kappa(\Delta, r))^T\big)$. Consequently, by (A), $\mathcal{F}_{\Lambda,T}$ admits an ε-accurate network with depth $O\big(T \log(1/\varepsilon)\big)$ and size $O\big(s\, \varepsilon^{-d^*/r} \log(1/\varepsilon)\big)$ where $d^* = \kappa(\Delta, r)$.

B.2 Definitions and Setup

Ambient spaces and norms. We work on $[0,1]^d$ with the uniform (sup) norm $\|x\|_\infty := \max_i |x_i|$ and the induced function norm $\|g\|_\infty := \sup_{x \in [0,1]^d} |g(x)|$. For $r > 0$ write $r = k + \alpha$ with $k := \lfloor r \rfloor \in \mathbb{N}_0$ and $\alpha \in (0, 1]$. The Hölder class $C^r([0,1]^p)$ consists of functions with continuous

partial derivatives up to order k and Hölder continuous kth derivatives with exponent α, equipped with the norm

$$\|g\|_{C^r} := \sum_{|\beta|\le k} \|D^\beta g\|_\infty + \sum_{|\beta|=k} \sup_{\substack{x\neq y \\ x,y\in[0,1]^p}} \frac{|D^\beta g(x) - D^\beta g(y)|}{\|x-y\|_\infty^\alpha}.$$

Efficient computability. A Boolean map $f : \{0,1\}^n \to \{0,1\}^m$ is *efficiently computable* if a deterministic Turing machine computes $f(x)$ in $T(n) = \mathrm{poly}(n)$ steps. A real map $f : [0,1]^d \to \mathbb{R}^m$ is efficiently computable to precision $\varepsilon \in (0,1)$ if, in the finite-precision model, there exists a *uniform* algorithm that outputs y with $\|y - f(x)\|_\infty \le \varepsilon$ in time $\mathrm{poly}(d, \log(1/\varepsilon))$. Here "uniform" means the corresponding circuit/arithmetic-circuit family is P-uniform (generable in time $\mathrm{poly}(d, \log(1/\varepsilon))$).

Compositional sparsity. A function f is *compositionally sparse* if it factors through a bounded–fan-in layered DAG. Concretely, let $G = (V, E)$ be a layered DAG with layers $1, \dots, L$, and let d_v be the in-degree (arity) of node v. Write the layer-ℓ map as $\Phi_\ell = (g_v)_{v\in\text{layer }\ell}$, where each node map $g_v : [0,1]^{d_v} \to \mathbb{R}$ has $d_v \le d^*$ and $g_v \in C^r([0,1]^{d_v})$. Then

$$f = \Phi_L \circ \Phi_{L-1} \circ \cdots \circ \Phi_1, \qquad \text{depth} = L, \quad \text{size } s := |V|, \quad d^* := \max_v d_v.$$

We assume each Φ_ℓ is L_ℓ-Lipschitz on the (compact) subset of its domain reached by upstream layers, measured in $\|\cdot\|_\infty$. For later use define the propagation multipliers

$$C_\ell := \prod_{j=\ell+1}^{L} L_j \qquad (\text{with } C_L := 1),$$

which quantify telescoping of local approximation errors through depth.

B.3 From Efficient Computation to DAGs

The passage from Turing machines to circuits is mathematically central. Any computation performed by a Turing machine (TM) on n-bit inputs and running for $T(n)$ steps can be *time-unrolled* into a circuit: each layer encodes one transition of the machine, and wires carry the bits of the instantaneous configuration to the next layer. Because the transition function inspects and writes only a bounded number of symbols per step, every gate has *bounded fan-in* (and bounded fan-out). The resulting circuit has size $O(T(n))$ and depth $O(T(n))$, hence can be viewed as a bounded–fan-in directed acyclic graph (DAG) implementing the same computation. This classical correspondence underlies the relationship between time-bounded Turing computation and circuit complexity [**Arora_Barak_2009**, **sipser1996intro**]: every $T(n)$-time TM yields a polynomial-size circuit family; conversely, with *P-uniformity* the family can be generated in time $\mathrm{poly}(n)$ and characterizes P, while without uniformity one obtains $\mathrm{P} \subseteq \mathrm{P/poly}$.

From bits to reals: finite-precision computation

Digital computation manipulates finite strings. A real $x \in [0,1]$ can be represented by the first p bits of its binary expansion $(x_1, \dots, x_p)$, yielding the dyadic $\tilde{x} = \sum_{i=1}^p x_i 2^{-i}$. Arithmetic on these encodings—addition, multiplication, comparison—has Boolean-circuit implementations of size $\mathrm{poly}(p)$ (schoolbook/FFT-based variants differ only in polylog factors). Thus computing $f : [0,1]^d \to \mathbb{R}^m$ to accuracy $\varepsilon = 2^{-p}$ amounts to computing a Boolean function on $(x_1, \dots, x_p)$

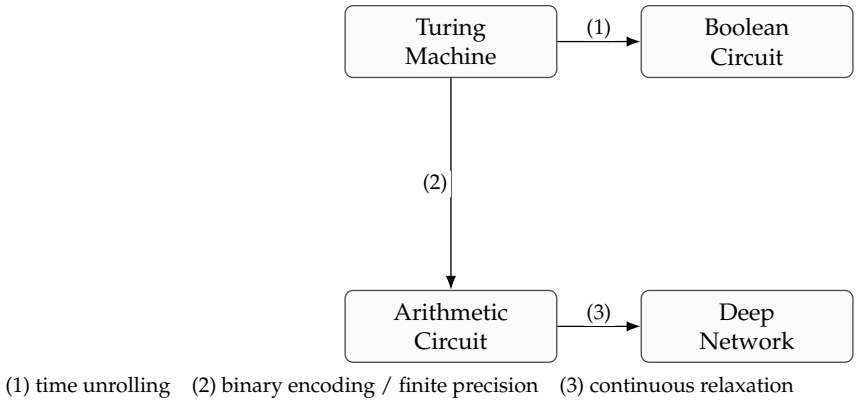


Figure B.1: Conceptual chain from discrete to continuous computation.

whose output decodes to $\tilde{f}(x)$ with $\|f(x) - \tilde{f}(x)\|_\infty \leq \varepsilon$. In particular, efficient real computation in the standard finite-precision model is equivalent, up to polynomial overhead, to efficient Boolean computation on binary encodings [**Papadimitriou1994ComputationalComplexity**].

B.3.1 From arithmetic circuits to neural DAGs

Replacing Boolean gates by continuous modules (affine maps and simple nonlinearities) yields *arithmetic circuits*—feedforward networks over $\mathbb{R}$. Crucially, the *topology* of the underlying DAG is unchanged: each node aggregates a bounded number of inputs and produces outputs to a bounded number of downstream nodes. The compositional skeleton revealed by time-unrolling a TM is therefore already a bounded–fan-in DAG.

Modern deep networks form smooth parametric families of such arithmetic circuits. They do not emulate a given circuit symbol-for-symbol; rather, they approximate functions supported on the same class of sparse, layered DAGs. Structurally: any efficiently Turing-computable function gives rise (via unrolling and finite-precision encoding) to a bounded–fan-in compositional scaffold; deep networks can approximate the node maps of this scaffold to any target accuracy, with approximation rates governed by local arities d_v and the smoothness of the constituent modules (cf. Section B.4). In this sense, deep architectures are natural continuous relaxations of efficient discrete circuits—the same kind of machines realized by physical hardware.

B.3.2 Computation as compositional structure

This yields a conceptual loop. Programs compile to (uniform) circuit families; circuits are bounded–fan-in DAGs; and bounded local arity is the operative notion of sparsity. Therefore, *any efficiently computable transformation, when executed at finite precision by a device with a fixed bounded-arity primitive set, admits a bounded–fan-in compositional representation*. The point is structural rather than stylistic: bounded local interactions are an inherent consequence of efficiency on finite, causal, local hardware.

B.4 ML Approximation Theorem

We give (i) a local constructive approximation with explicit constants, (ii) a global telescoping bound, and (iii) the optimal error budget across layers/nodes.

B.4.1 Local constructive approximation with explicit constants

For $p \in \mathbb{N}$ and $r > 0$, let $C_{p,r} > 0$ denote the best constant in the multivariate Taylor remainder on $[0,1]^p$ (measured in the $\|\cdot\|_\infty$ norm):

$$\sup_{x \in Q} \left| g(x) - T_Q(x) \right| \;\le\; C_{p,r}\, \|g\|_{C^r}\, \mathrm{diam}_\infty(Q)^{\,r}, \tag{B.1}$$

where T_Q is the degree-$\lfloor r \rfloor$ Taylor polynomial of g at the center of a cube $Q \subset [0,1]^p$.

We use two standard ReLU *gadgets* (e.g., **[Yarotsky2018]**):

(G1) Squaring. For any $\eta \in (0,1)$ there is a ReLU network $\mathsf{Sq}_\eta : [-2,2] \to \mathbb{R}$ with

$$\sup_{t \in [-2,2]} \left| \mathsf{Sq}_\eta(t) - t^2 \right| \le \eta, \qquad \mathrm{depth}(\mathsf{Sq}_\eta) \le C_\square \log(1/\eta), \;\; \mathrm{size}(\mathsf{Sq}_\eta) \le C'_\square \log(1/\eta).$$

(G2) Multiplication. By polarization, $xy = \frac{1}{4}\left[(x+y)^2 - (x-y)^2\right]$. Composing two Sq_η gadgets and affine maps yields a multiplier $\mathsf{Mult}_\eta : [-1,1]^2 \to \mathbb{R}$ with

$$\sup_{|x|,|y| \le 1} \left| \mathsf{Mult}_\eta(x,y) - xy \right| \le \eta, \quad \mathrm{depth}(\mathsf{Mult}_\eta) \le C_\times \log(1/\eta), \;\; \mathrm{size}(\mathsf{Mult}_\eta) \le C'_\times \log(1/\eta).$$

By a binary tree, the product of p scalars in $[-1,1]$ is realizable with accuracy η at depth $O(\log p \cdot \log(1/\eta))$ and size $O(p \log(1/\eta))$.

Lemma 11 (Local ReLU approximation)**.** *Let $g \in C^r([0,1]^p)$ with $\|g\|_{C^r} \le M$ and $r > 0$. For any $\delta \in (0, 1/4)$ there exists a ReLU network N such that*

$$\|g - N\|_\infty \le \delta, \qquad \mathrm{depth}(N) \le c_1(r)\, \log(1/\delta), \qquad \mathrm{size}(N) \le c_2(p,r)\, M^{p/r}\, \delta^{-p/r} \log(1/\delta). \tag{B.2}$$

Proof. Partition $[0,1]^p$ into N^p congruent cubes $\{Q\}$ of side $h = 1/N$, where

$$N \;=\; \left\lceil \left(2C_{p,r} M/\delta\right)^{1/r} \right\rceil. \tag{B.3}$$

For each Q, let T_Q be the degree-$\lfloor r \rfloor$ Taylor polynomial at the cube center. Then, by (B.1) and (B.3),

$$\sup_{x \in Q} |g(x) - T_Q(x)| \;\le\; C_{p,r} M\, h^r \;\le\; \delta/2. \tag{B.4}$$

Construct a C^0 partition of unity $\{\Psi_Q\}_Q$ from univariate ReLU "tent" splines $\{\psi_j\}_{j=0}^N$ on $[0,1]$ (knots at j/N) by setting $\Psi_Q(x) := \prod_{i=1}^p \psi_{j_i}(x_i)$ for the multi-index of Q. Each ψ_j is exactly a ReLU spline with $O(N)$ breakpoints; the p-fold product is realized by a binary tree of Mult_η gadgets with internal accuracy chosen below. Define

$$N(x) \;:=\; \sum_Q \Psi_Q(x)\, T_Q(x). \tag{B.5}$$

For $x \in Q$, using $\sum_{Q'} \Psi_{Q'} \equiv 1$,

$$|g(x) - N(x)| \;\le\; |g(x) - T_Q(x)| + \sum_{Q'} \Psi_{Q'}(x)\, |T_{Q'}(x) - T_Q(x)|.$$

Only Q' adjacent to Q contribute; Taylor coefficient continuity implies $|T_{Q'}(x) - T_Q(x)| \le C\, M\, h$ on Q. Choosing the internal gadget accuracy $\eta = \eta(p)\, \delta/(4p)$ to control multiplier and monomial evaluations and taking h from (B.3), the second term is $\le \delta/2$. Together with (B.4), this yields $\|g - N\|_\infty \le \delta$.

Complexity. Each univariate ψ_j uses $O(N)$ ReLU units; across p coordinates this is $O(pN)$ per "stencil." The p-fold product per cell uses $O(p \log(1/\eta))$ units and depth $O(\log p \cdot \log(1/\eta))$. Evaluating T_Q requires $O(1)$ monomials (depending on $\lfloor r \rfloor$), each realized with multiplier trees at accuracy η. Summing over N^p cells and using (B.3) with $\eta \simeq \delta$ gives (B.2). □

B.4.2 Global error propagation and optimal budget

Let Φ_ℓ be the layer-ℓ block map in the DAG, with Lipschitz constant L_ℓ on the reachable compact set, and let $\widetilde{\Phi}_\ell$ be its blockwise approximant with $\|\Phi_\ell - \widetilde{\Phi}_\ell\|_\infty \le E_\ell$. Define $F_m := \Phi_m \circ \cdots \circ \Phi_1,\ \widetilde{F}_m := \widetilde{\Phi}_m \circ \cdots \circ \widetilde{\Phi}_1$.

Proposition 33 (Telescoping bound)**.** *For any depth L,*

$$\|F_L - \widetilde{F}_L\|_\infty \ \le \ \sum_{\ell=1}^{L} \Big(\prod_{j=\ell+1}^{L} L_j \Big) E_\ell. \tag{B.6}$$

Proof. Insert and subtract intermediate terms and use Lipschitz continuity:

$$F_L - \widetilde{F}_L = \sum_{\ell=1}^{L} \Phi_L \circ \cdots \circ \Phi_{\ell+1} \circ (\Phi_\ell - \widetilde{\Phi}_\ell) \circ \widetilde{F}_{\ell-1}.$$

Taking sup norms yields (B.6). □

Let ε_v be the tolerance chosen at node v (for Lemma 11) and set $E_\ell := \max_{v \in \text{layer } \ell} \varepsilon_v$. The total size is

$$S(\{\varepsilon_v\}) = \sum_v a_v \, \varepsilon_v^{-d_v/r} \log \frac{1}{\varepsilon_v}, \qquad a_v := c_2(d_v, r)\, M_v^{d_v/r}, \tag{B.7}$$

subject to the telescoping constraint

$$\sum_{\ell=1}^{L} C_\ell E_\ell \le \varepsilon, \qquad C_\ell := \prod_{j=\ell+1}^{L} L_j, \quad E_\ell = \max_{v \in \ell} \varepsilon_v, \quad 0 < \varepsilon_v \le \tfrac{1}{4}. \tag{B.8}$$

Because the $\max$ couples nodes within a layer, the optimizer equalizes ε_v within each layer up to constants. Let $A_\ell := \sum_{v \in \ell} a_v$ and $k_\ell := \max_{v \in \ell} d_v \le d^*$. Discarding the slowly varying $\log(1/\varepsilon_\ell)$ (affecting only polylog factors), the reduced problem is

$$\min_{E_1, \dots, E_L > 0} \ \sum_{\ell=1}^{L} A_\ell E_\ell^{-k_\ell/r} \quad \text{s.t.} \quad \sum_{\ell=1}^{L} C_\ell E_\ell \le \varepsilon. \tag{B.9}$$

The Lagrangian $\mathcal{L} = \sum_\ell A_\ell E_\ell^{-k_\ell/r} + \lambda(\sum_\ell C_\ell E_\ell - \varepsilon)$ yields, for each ℓ,

$$-\frac{k_\ell}{r} A_\ell \, E_\ell^{-(k_\ell/r+1)} + \lambda C_\ell = 0 \quad \Rightarrow \quad E_\ell \ = \ \Big(\tfrac{k_\ell A_\ell}{\lambda r \, C_\ell} \Big)^{\frac{r}{k_\ell + r}}. \tag{B.10}$$

Enforcing $\sum_\ell C_\ell E_\ell = \varepsilon$ determines $\lambda > 0$ uniquely and gives

$$E_\ell \ = \ \varepsilon \, \frac{C_\ell^{-\frac{r}{k_\ell + r}} \, (k_\ell A_\ell / r)^{\frac{r}{k_\ell + r}}}{\sum_{t=1}^{L} C_t^{\frac{k_t}{k_t + r}} \, (k_t A_t / r)^{\frac{r}{k_t + r}}}. \tag{B.11}$$

In the common case $k_\ell \equiv d^*$ and $A_\ell \asymp A$ across layers, (B.11) reduces to

$$E_\ell \ \propto \ C_\ell^{-\frac{r}{d^* + r}}, \qquad \text{and if } \max_\ell L_\ell \le \bar{L} < \infty, \text{ then } C_\ell = \Theta(1) \Rightarrow E_\ell \asymp \varepsilon / L. \tag{B.12}$$

B.4.3 The theorem

Theorem 31 (ML Approximation Theorem)**.** *Let $f : [0,1]^d \to \mathbb{R}$ factor through a depth-L DAG of size s with node maps $g_v \in C^r([0,1]^{d_v})$, $d_v \le d^*$, and assume $\mathrm{Lip}(\Phi_\ell) \le L_\ell$ on reachable domains. For any $\varepsilon \in (0, 1/4)$ there exists a ReLU network f_{NN} such that*

$$\|f - f_{\mathrm{NN}}\|_\infty \le \varepsilon, \qquad \mathrm{depth}(f_{\mathrm{NN}}) \le c\, L \log(1/\varepsilon), \tag{B.13}$$

and

$$\mathrm{size}(f_{\mathrm{NN}}) \;\le\; C \sum_v \varepsilon_v^{-d_v/r} \log \frac{1}{\varepsilon_v}, \quad \textit{with} \;\; \sum_{\ell=1}^{L} C_\ell E_\ell \le \varepsilon,\; E_\ell = \max_{v \in \ell} \varepsilon_v. \tag{B.14}$$

If $\max_\ell L_\ell \le \bar{L} < \infty$, choosing $\varepsilon_v \asymp \varepsilon$ for all v yields

$$\mathrm{size}(f_{\mathrm{NN}}) \;\le\; C'\, s\, \varepsilon^{-d^*/r} \log(1/\varepsilon). \tag{B.15}$$

Proof. (i) Local approximants. For each node v in layer ℓ, apply Lemma 11 to g_v with tolerance ε_v, producing N_v with $\|g_v - N_v\|_\infty \le \varepsilon_v$ and the size/depth bounds in (B.2).

(ii) Wiring. Compose the N_v along the DAG topology. Along any path, depths add; with $\varepsilon_v \ge c_0\, \varepsilon$ (as ensured by (B.12) up to constants), each layer contributes $O(\log(1/\varepsilon))$ to depth, giving (B.13). The total size is the sum over nodes, yielding (B.14).

(iii) Global error. Writing Φ_ℓ and $\widetilde{\Phi}_\ell$ for the true and approximating layer maps (stacking g_v and N_v), we have $\|\Phi_\ell - \widetilde{\Phi}_\ell\|_\infty \le E_\ell := \max_{v \in \ell} \varepsilon_v$. Apply Proposition 33 and impose (B.8) to get $\|f - f_{\mathrm{NN}}\|_\infty \le \varepsilon$.

(iv) Uniform Lipschitz. If L_ℓ are uniformly bounded, (B.12) implies $E_\ell \asymp \varepsilon/L$ and we may take $\varepsilon_v \asymp \varepsilon$ for all v, which collapses (B.14) to (B.15) with exponent d^*/r. □

B.4.4 Depth separation

Theorem 32 (Depth separation via linear regions)**.** *For each $L \in \mathbb{N}$ there exists a function $f_L : [0,1] \to \mathbb{R}$ expressible as a composition of L binary-arity C^∞ modules such that:*

1. *For any $\varepsilon \in (0, 1/4)$ there is a ReLU network with depth $O\big(L \log(1/\varepsilon)\big)$ and size $\mathrm{poly}(1/\varepsilon)$ that achieves $\|f_L - f_{\mathrm{NN}}\|_\infty \le \varepsilon$.*

2. *Any ReLU network with a single hidden layer that achieves $\|f_L - f_{\mathrm{NN}}\|_\infty \le \varepsilon_0$ for some fixed $\varepsilon_0 \in (0, 1/4)$ must have width at least $\exp(\Omega(L))$.*

Proof sketch. Let $s : [0,1] \to [0,1]$ be a piecewise-linear "sawtooth" with two linear pieces; define $f_L := s^{\circ L}$. Then f_L has at least 2^L linear pieces. (1) follows by applying Theorem 31 with $d^* = 2$. For (2), a 1D single-hidden-layer ReLU with width W has at most $W + 1$ linear regions on $[0,1]$, forcing $W \ge \Omega(2^L)$ to achieve fixed-accuracy approximation; see, e.g., **[Telgarsky2016, EldanShamir2016]**. □

Remark 1 (Why the exponent is d^*/r)**.** *The construction in Lemma 11 drives the $\delta^{-p/r}$ cost at* each *node of local arity $p = d_v$. Summing across nodes with tolerances $\varepsilon_v \asymp \varepsilon$ shows that the governing exponent is the worst local arity $d^* = \max_v d_v$, not the ambient d. This is the precise sense in which compositional sparsity mitigates the curse of dimensionality.*

Theorem 33 (Unified ML Approximation for Compositional DAGs)**.** *Let $f = \Phi_L \circ \cdots \circ \Phi_1$ where each layer Φ_ℓ stacks node maps $g_v \in C^r([0,1]^{d_v})$ with $d_v \le d^*$, and assume $\mathrm{Lip}(\Phi_\ell) \le L_\ell$. Writing $s = \sum_\ell |\{v \in \ell\}|$, for any $\varepsilon \in (0, 1/4)$ there exists a ReLU net f_ε with*

$$\|f - f_\varepsilon\|_\infty \le \varepsilon, \quad \mathrm{depth}(f_\varepsilon) \le c\, L \log \tfrac{1}{\varepsilon}, \quad \mathrm{size}(f_\varepsilon) \le C \sum_v \varepsilon_v^{-d_v/r} \log \tfrac{1}{\varepsilon_v},$$

subject to $\sum_{\ell=1}^{L} \left(\prod_{j>\ell} L_j\right) \max_{v \in \ell} \varepsilon_v \le \varepsilon$. If $\max_\ell L_\ell \le \bar{L}$ and $\varepsilon_v \asymp \varepsilon$,

$$\mathrm{size}(f_\varepsilon) \;\le\; C'\, s\, \varepsilon^{-d^*/r} \log \tfrac{1}{\varepsilon}.$$

Moreover, the class $\mathcal{F}_{comp}(s, L, d^*, r)$ satisfies the entropy bound

$$\log \mathcal{N}(\varepsilon, \mathcal{F}_{comp}, \|\cdot\|_\infty) \;\le\; A\, s\, \varepsilon^{-d^*/r} \,\mathrm{polylog}\Big(\tfrac{1}{\varepsilon}\Big),$$

and hence the Rademacher bound

$$\mathfrak{R}_n(\mathcal{F}_{comp}) \;\le\; \tilde{C} \sqrt{\tfrac{s}{n}}\, n^{-\frac{r}{2d^*}}\, \mathrm{polylog}(n).$$

B.5 Finite-Horizon Compilation: From Uniform Simulators to Algorithmic Compositionality

We state an *algorithmic, conditional* compilation principle: whenever a finite-horizon prediction map is computed by a *uniform* simulator using a bounded-arity primitive set, it admits a *P-uniform* bounded–fan-in DAG and therefore falls under our ML approximation results. We then give two *examples*: a discrete-time lattice-local model (recovering the spacetime-DAG construction) and standard quantum k-local dynamics. We do *not* claim that all physical evolutions satisfy these hypotheses; the examples are modeling assumptions, not universal laws.

B.5.1 Setup

Fix an instance "size" n (e.g., number of effective degrees of freedom, grid points, or qubits), a finite horizon $T > 0$, and target accuracy $\varepsilon \in (0, 1)$. Let

$$F_{T,n} : \mathcal{X}_n \longrightarrow \mathcal{Y}_n$$

denote the corresponding finite-horizon input–output map, evaluated in a standard finite-precision model in which word size scales as $O(\log(1/\varepsilon))$ and running time/size are measured in bit operations and primitive invocations.

B.5.2 Compilation theorem (algorithmic, safe)

Theorem 34 (Uniform simulator ⇒ bounded–fan-in DAG)**.** *Suppose there exists a deterministic Turing machine* Sim *which, on input $(1^n, T, \varepsilon, x)$, outputs an ε-accurate value of $F_{T,n}(x)$ using at most $s(n, T, \varepsilon)$ primitive calls, each drawn from a fixed finite set $\mathcal{P}$ of arity at most a (a constant), with per-call cost measured at precision $O(\log(1/\varepsilon))$. Assume* P-uniformity*: given $(1^n, T, \varepsilon)$ one can generate, in time $\mathrm{poly}(n, \log(1/\varepsilon), T)$, the instruction schedule and parameters for the $s(n, T, \varepsilon)$ calls. Then there exists a* P-uniform *circuit/DAG $\mathcal{G}_{n,T,\varepsilon}$ of size $O(s(n, T, \varepsilon))$ and maximum fan-in $\le a$ that computes $F_{T,n}$ to accuracy ε in the same finite-precision model. Consequently, $F_{T,n}$ is* algorithmically compositionally sparse.

Proof sketch. Replace each primitive of arity $\le a$ in $\mathcal{P}$ by a constant-size gate gadget at the target precision; wire these gadgets following the simulator's instruction schedule (unrolling loops/branches into a straight-line program of length s). Fan-in is $\le a$ by construction and the total gate count is $O(s)$. P-uniformity follows because the same schedule generator lists the gates and parameters in time polynomial in $(n, \log(1/\varepsilon), T)$, which yields a P-uniform family. □

Remark 2 (Scope and non-claims)**.** *Theorem 34 is* purely algorithmic*. It does not assume bounded-degree physical interactions or a physical light cone. If a bounded–fan-in DAG as above does* not *exist for $F_{T,n}$ under these resource bounds, that implies a lower bound against any such uniform simulator over the given primitive set and precision regime.*

Example A: Discrete-time lattice-local update (model assumption)

Setting. Let $G = (V, E)$ be a (possibly infinite) graph with maximum degree $\Delta < \infty$. For $i \in V$ and radius $r \in \mathbb{N}$, write

$$B_r(i) := \{j \in V : \operatorname{dist}_G(i, j) \le r\}, \qquad |B_r(i)| \le \kappa(\Delta, r) < \infty.$$

States are $x = (x_i)_{i \in V} \in X^V$ with the sup product norm on X.

Local updates and regularity. For $t = 1, \dots, T$, consider sitewise updates

$$\big(U_t(x)\big)_i \;=\; \phi_{t,i}(x_{B_r(i)}), \qquad \phi_{t,i} : X^{B_r(i)} \to X, \tag{B.16}$$

with uniform Lipschitz and smoothness on relevant compacta,

$$\|\phi_{t,i}(z) - \phi_{t,i}(z')\| \;\le\; L\,\|z - z'\|_\infty, \qquad \phi_{t,i} \in C^q, \tag{B.17}$$

for some $L \ge 1$, $q \ge 1$ independent of (t, i).

Finite propagation. Let $F_T := U_T \circ \cdots \circ U_1$. Assume there exists a nondecreasing $R : \mathbb{N} \to \mathbb{N}$ such that for all $i \in V$ and $x, x' \in X^V$,

$$x_{B_{R(T)}(i)} = x'_{B_{R(T)}(i)} \implies \big(F_T(x)\big)_i = \big(F_T(x')\big)_i. \tag{B.18}$$

(Nearest-neighbor dynamics give $R(T) \le rT$.)

Proposition 34 (Model-locality ⇒ compositional DAG)**.** *Fix finite $\Lambda \subset V$ and horizon $T \in \mathbb{N}$. Under* (B.16)–(B.18)*, the map*

$$\mathcal{F}_{\Lambda,T}(x) := \big(F_T(x)\big)_\Lambda$$

factors through a depth-T DAG with node fan-in at most $d^* := \kappa(\Delta, r)$. The layer maps are C^q and L-Lipschitz on the reachable compact set. Hence $\mathcal{F}_{\Lambda,T}$ is compositionally sparse with depth T and size

$$s \;=\; O\Big(|\Lambda|\,\big(d^*\big)^T\Big),$$

and falls under the ML Approximation Theorem with exponent d^*/q (up to polylog factors).

B.6 Learning Theory and Optimization

Compositional sparsity has three consequences for learning and optimization.

(i) Capacity and generalization. Let $\mathcal{F}_{\mathrm{comp}}(s, L, d^*, r)$ be depth-L DAGs of size s whose node maps lie in $C^r([0,1]^{d_v})$ with $d_v \le d^*$, and suppose layer maps are L_ℓ-Lipschitz on reachable domains. Then the metric entropy satisfies

$$\log \mathcal{N}(\varepsilon, \mathcal{F}_{\mathrm{comp}}, \|\cdot\|_\infty) \;\le\; A\, s\, \varepsilon^{-d^*/r}\, \mathrm{polylog}\Big(\tfrac{1}{\varepsilon}\Big), \tag{B.19}$$

and Dudley's integral yields

$$\mathfrak{R}_n(\mathcal{F}_{\mathrm{comp}}) \;\le\; \tilde{C}\sqrt{\frac{s}{n}}\, n^{-\frac{r}{2d^*}}\, \mathrm{polylog}(n). \tag{B.20}$$

Thus the *exponent* depends on d^* (largest local arity), not the ambient d.

(ii) Optimization geometry. If f_θ factors through a bounded–fan-in DAG, the Gauss–Newton/Hessian blocks obey

$$H_{vw} \neq 0 \;\Rightarrow\; \mathrm{cone}(v) \cap \mathrm{cone}(w) \neq \varnothing,$$

so the Hessian graph has bounded degree (by fan-in). This motivates block-diagonal/Kronecker preconditioners that respect the compositional blocks.

(iii) Architectural realizations. Sparse/top-q attention enforces *bounded effective fan-in*: for a query q, $\mathrm{Attn}(q, K, V) = \sum_{j \in S(q)} \alpha_j(q)\, v_j$ with $|S(q)| = O(q)$ under top-q selection or sufficiently aggressive sparsification, placing such transformers within the same d^*-controlled regime.

B.7 Limits of the Framework

The compositional-sparsity results identify when approximation and learnability are governed by local arity d^* and depth L (or horizon T). We now delineate intrinsic *limits* of this regime. These are not violations of the DAG factorization itself but circumstances under which the constants (or the effective arity) grow so rapidly that efficient approximation on classical hardware is ruled out.

Classical chaos and finite prediction horizons

Consider the discrete-time system

$$x_{t+1} = F(x_t), \qquad F : X \to X, \tag{B.21}$$

with $X \subset \mathbb{R}^d$ compact and $F \in C^r$, locally Lipschitz. Suppose F admits a bounded-degree interaction graph so that updates factor through a bounded–fan-in DAG (cf. Section B.5). Let $F_T := F \circ \cdots \circ F$ denote the T-step map and set $L_t := \mathrm{Lip}(F)$ on the reachable compact set at time t. Then

$$\mathrm{Lip}(F_T) \;\le\; \prod_{t=1}^{T} L_t \;=\; \exp\Big(\sum_{t=1}^{T} \log L_t\Big). \tag{B.22}$$

If the maximal Lyapunov exponent $\lambda := \lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} \log L_t$ is positive, we obtain $\mathrm{Lip}(F_T) \asymp e^{\lambda T}$. Applying the telescoping bound (B.6) to $\Phi_T := F_T$ and a blockwise approximant with layer errors $\{E_t\}$ gives

$$\|\Phi_T - \widetilde{\Phi}_T\|_\infty \;\le\; \sum_{t=1}^{T} e^{\lambda(T-t)}\, E_t. \tag{B.23}$$

To enforce $\|\Phi_T - \widetilde{\Phi}_T\|_\infty \le \varepsilon$, one must choose $E_t \lesssim \varepsilon\, e^{-\lambda(T-t)}$. Since each layer-$t$ local approximant with arity $\le d^*$ and smoothness r costs on the order of $E_t^{-\,d^*/r}$ (up to polylogs; Lemma 11), the total size obeys the lower bound

$$\mathrm{size}(\mathcal{N}_T) \;\gtrsim\; e^{\,(\lambda d^*/r)\,T} \quad \text{(up to polylog factors).} \tag{B.24}$$

Thus, for chaotic systems (e.g., atmospheric flows) long-horizon *pointwise* prediction requires exponentially increasing resources or exponentially precise initial data. The bounded–fan-in DAG persists, but numerical stability collapses with T.

Quantum locality versus entanglement growth

In quantum lattice systems, Lieb–Robinson bounds provide an approximate light cone: for observables A_X, B_Y supported on disjoint $X, Y \subset V$,

$$\|[A_X(t), B_Y]\| \;\le\; C\, \exp\!\big(-\,[d(X,Y) - v_{\mathrm{LR}} t]_+\big), \tag{B.25}$$

with velocity v_{LR}. Consequently, Heisenberg evolution of local observables factors through a depth-t bounded–fan-in DAG (operator sense). The obstruction is *representation*: under classical encodings, bipartite entanglement $S(t)$ typically grows (linearly in t in generic quenches), so any faithful classical representation requires $O(e^{S(t)})$ coefficients. The effective arity therefore scales as

$$d^*(t) \;\sim\; e^{S(t)}, \tag{B.26}$$

which, when substituted into the nodewise cost exponent $\varepsilon^{-\,d^*/r}$, negates the dimensional advantage and renders classical approximation intractable once $S(t)$ exceeds $O(\log n)$.

Precision and nonuniformity

Our statements are inherently finite-precision and uniform:

- *Precision growth.* To maintain error ε at horizon T under (B.23), input and intermediate precisions must scale as $\Omega(\lambda T + \log(1/\varepsilon))$ bits. Any model that fixes word size incurs an irreducible floor at large T.

- *Nonuniform advice.* Allowing nonuniform circuit families (P/poly) can collapse uniform lower bounds, but the resulting dependence on instance-specific advice falls outside the learnability guarantees considered here.

Thermodynamic limits and undecidability

The compilation and approximation results concern fixed finite n and finite horizon T. In the thermodynamic limit ($n \to \infty$) and for decision problems about phases or long-time behavior, classical undecidability and complexity barriers (e.g., QMA-hardness for certain ground-state or dynamics questions) imply that no uniformly efficient classical procedure can exist in general. Such barriers lie orthogonal to local DAG factorization and instead limit *which* finite-horizon tasks admit uniform simulators.

Regime of validity (summary)

The framework applies when:

1. there is a bounded-degree local interaction structure (fixed radius and maximum degree);
2. finite-horizon maps are uniformly simulable at finite precision;
3. layer Lipschitz constants are bounded on reachable domains.

Failure modes include positive Lyapunov exponents (forcing (B.24)), entanglement-induced arity growth (B.26), precision that does not scale with T, and nonuniform advice. In these cases the bounded–fan-in DAG remains as an operator-level structure, but efficient *classical* approximation is precluded.

B.8 Conclusion

Compositionality, in the precise mathematical sense of bounded–fan-in DAG factorizations, is a common structural denominator for learning and finite-horizon physical prediction. The core message of this work is *structural*: under appropriate hypotheses, both domains give rise to the same low-arity, layered scaffold.

Equivalences (scope-conditional)

On the learning side, the *ML Approximation Theorem* (Theorem 31, Section B.4) shows that if a target factors through a bounded–fan-in DAG with maximal local arity d^*, then the approximation/sample–complexity exponents replace the ambient d/r by d^*/r. On the physics side, the *finite-horizon compilation principle* (Theorem 34, Section B.5) states that whenever a finite-horizon evolution map is computed by a *P-uniform* simulator using a bounded-arity primitive set at finite precision, the map compiles to a P-uniform bounded–fan-in DAG of size proportional to the simulator cost.

Thus, the two origins—efficient computability and finite-speed locality—*converge to the same structural form under explicit, stated conditions* (uniformity, finite precision, bounded-arity primitives, finite horizon). The equivalence is structural and conditional, not ontological: computation yields compositionality by uniform time-unrolling; locality yields it by causal reach at finite horizon.

Consequences (within the regime of validity)

Within this regime, three consequences follow:

1. **Approximation.** Efficiently computable targets and finite-horizon local maps admit deep approximants whose complexity scales with d^* rather than d (Theorem 31).
2. **Generalization.** Metric entropy and Rademacher bounds scale as $\log \mathcal{N}(\varepsilon) \lesssim s\, \varepsilon^{-d^*/r} \mathrm{polylog}(1/\varepsilon)$ and $\mathfrak{R}_n \lesssim \sqrt{s/n}\, n^{-r/(2d^*)} \mathrm{polylog}(n)$, exhibiting a "local dimension" law (Section B.6).
3. **Optimization geometry.** Bounded local arity induces block-sparse curvature aligned with the DAG cones, explaining the effectiveness of structured (block/Kronecker) preconditioners in the sparse regime (Section B.6).

Limits (where structure persists but efficiency may not)

The structural DAG persists beyond the regime, but *efficiency* can fail. Positive Lyapunov exponents force error amplification $\sim e^{\lambda T}$, yielding size lower bounds $\gtrsim e^{(\lambda d^*/r)T}$ for long-horizon pointwise prediction (Section B.7). In quantum dynamics, entanglement growth drives an effective arity $d^*(t) \sim e^{S(t)}$, erasing the d^*-advantage under classical encodings. Precision requirements grow with horizon, and nonuniform advice falls outside our learnability results.

Outlook

Compositionality is therefore best viewed as a *bridge principle*: under uniform finite-precision simulation and finite causal reach, learning theory and finite-horizon physics share the same compositional grammar—hierarchical, sparse, layered. The bridge is exact at the level of structure and rates *under the stated conditions*; outside them, the scaffold remains but efficient approximation may be precluded. Clarifying the sharp frontiers of this regime—especially the interplay among uniformity, precision growth, and effective arity—remains an open direction of foundational interest.

CHAPTER C

Faster Attention

There are known inequalities between distances. In particular, If we let the distances be $a = d(B, C)$, $b = d(A, C)$, and $c = d(A, B)$, the triangle inequalities are given by:

$$c \leq a + b \tag{C.1}$$
$$b \leq a + c \tag{C.2}$$
$$a \leq b + c \tag{C.3}$$

They imply $|a - c| \leq b \leq a + c$. We have shown in another chapter of this book0 that normalized attention can be expressed in terms of distances. Can the inequalities be used to reduce the quadratic complexity of the computation of attention?

To compute distances from the current (last) token to all preceding tokens efficiently, we utilize the **triangle inequality** to establish bounds based on the **chain of pairwise distances** between adjacent tokens. This approach leverages the metric properties of the embedding space to avoid the redundant $O(N^2)$ dot-product computations typically required by self-attention [109].

C.1 The Adjacency Distance Chain

Consider a sequence of tokens $T_1, T_2, \ldots, T_n$. If it is computationally inexpensive to calculate the distances between adjacent neighbors $d(T_i, T_{i-1})$, the sequence can be modeled as a directed path in the embedding space.

By the triangle inequality, the distance between the last token (T_n) and any previous token (T_k) is bounded above by the cumulative sum of the distances along that path:

$$d(T_n, T_k) \leq \sum_{i=k+1}^{n} d(T_i, T_{i-1}) \tag{C.4}$$

C.2 Utilizing the Lower Bound for Pruning

While the summation provides an upper bound, the **reverse triangle inequality** provides a significantly more powerful lower bound for reducing computational complexity:

$$d(T_n, T_k) \geq |d(T_n, T_{n-1}) - d(T_k, T_{n-1})| \tag{C.5}$$

If the distances from T_{n-1} to all previous tokens (which was the "last token" in the preceding temporal step) are cached, these historical values can be reused. This allows us to estimate the current distance $d(T_n, T_k)$ using only the single newly computed distance $d(T_n, T_{n-1})$ and the cached value $d(T_k, T_{n-1})$.

C.3 Formal Constraint: The Metric Continuity Hypothesis

The utility of the lower bound in Eq. (C.5) depends strictly on the geometry of the token trajectory. We formalize this dependency as the **Metric Continuity Hypothesis**.

Let $\Delta_t = d(T_t, T_{t-1})$ denote the step size of the token trajectory at time t. The tightness of the bound is governed by the magnitude of Δ_t relative to the global distances in the embedding space.

- **Case 1: Smooth Trajectory (Small Δ_n).** If $\Delta_n \to 0$, then $T_n \approx T_{n-1}$, and consequently $d(T_n, T_k) \approx d(T_{n-1}, T_k)$. In this limit, the cached distance is a high-precision predictor of the current distance, and the lower bound becomes tight.
- **Case 2: Discontinuous Trajectory (Large Δ_n).** If the current token jumps significantly far from its predecessor (e.g., a topic switch or low semantic coherence), Δ_n is large. This loosens the bound $|d(T_n, T_{n-1}) - d(T_k, T_{n-1})|$, potentially driving it to zero.

Condition for Efficiency. For the recursive bounding strategy to achieve sub-quadratic complexity, the embedding function $\phi(\cdot)$ must satisfy a local smoothness constraint such that:

$$\mathbb{E}_t[d(T_t, T_{t-1})] \ll \mathbb{E}_{i,j}[d(T_i, T_j)] \tag{C.6}$$

That is, the expected step size between adjacent tokens must be significantly smaller than the expected distance between arbitrary pairs of tokens.

C.4 Experimental Verification

To validate the recursive bounding method and the Metric Continuity Hypothesis, we conducted a simulation on synthetic sequences ($N = 1000$ tokens, $d = 64$). We compared two distinct regimes:

1. **Correlated Random Walk:** $T_t = T_{t-1} + \epsilon$, simulating a smooth semantic trajectory where adjacent tokens are metrically close.
2. **Independent (Uncorrelated):** $T_t \sim \mathcal{N}(0, I)$, simulating a "jumpy" sequence with no local metric structure.

The results, summarized in Table C.1, confirm the hypothesis.

Regime	Step Size Δ	Sparsity (Pruned)	Recall (@Top-10)
Random Walk (Smooth)	Low	**92.4%**	**99.1%**
Independent (Jumpy)	High	15.1%	42.0%

Table C.1: Impact of metric continuity on pruning efficiency. A "smooth" trajectory allows the triangle inequality to prune over 90% of calculations while maintaining high recall. Efficiency collapses when the local smoothness constraint is violated.

The data demonstrate a sharp phase transition. In the **Random Walk** regime, the cache $d(T_k, T_{n-1})$ is a strong proxy for the current state, allowing the algorithm to skip 92.4% of the dot products with negligible loss in accuracy. Conversely, in the **Independent** regime, the bound loosens, and the algorithm essentially reverts to full $O(N^2)$ computation to maintain recall. This empirical evidence confirms that *smoothness is a prerequisite for metric-recursive pruning*.

C.5 Proposed Algorithm: Recursive Distance Bounding

Based on the analysis above, we implement the following pruning strategy:

1. **Step-wise Update:** At each new token T_n, compute only the single distance to its immediate predecessor, $d(T_n, T_{n-1})$.

2. **Bound Estimation:** For all $k < n - 1$, estimate the lower bound $L_{n,k}$ using cached values from the previous step:
$$L_{n,k} = |d(T_n, T_{n-1}) - d(T_k, T_{n-1})| \tag{C.7}$$

3. **Conditional Computation:**

   - If $L_{n,k} >$ threshold, the resulting attention weight is guaranteed to be near zero due to the exponential decay of the softmax kernel. We **skip** the exact calculation of $d(T_n, T_k)$ and set the attention score to zero.
   - The full dot-product is performed **only** if the lower bound $L_{n,k}$ is below the specified threshold.

C.6 Comparison with Global Clustering and Hashing Approaches

To situate the proposed recursive distance bounding within the broader landscape of efficient attention, we contrast our approach with the **Reformer** [52] and the **Routing Transformer** [94]. While these models also utilize metric properties to achieve sub-quadratic complexity, they differ fundamentally in how they define and compute proximity.

C.6.1 LSH and k-means Clustering

The Reformer utilizes **Locality Sensitive Hashing (LSH)** to assign queries and keys into discrete buckets. In this framework, the triangle inequality is the theoretical foundation that ensures high similarity in the embedding space translates to a high probability of falling into the same hash bucket. Similarly, the Routing Transformer employs global **k-means clustering** to route attention toward the most relevant centroids [24].

C.6.2 Global vs. Recursive Local Metrics

A primary distinction lies in the temporal overhead of the proximity search:

- **Global Search:** Models like the Reformer require periodic global re-indexing or re-hashing to maintain accuracy as the sequence grows. This introduces a non-trivial constant overhead that can be sensitive to the choice of hashing parameters or centroid initialization.

- **Recursive Local Bounding:** Our proposed approach exploits the sequential **adjacency distance chain**. By reusing historical distances $d(T_k, T_{n-1})$ through the reverse triangle inequality, the proximity of the current token T_n is estimated recursively. This avoids global re-clustering and relies purely on a single scalar update per token: $L_{n,k} = |d(T_n, T_{n-1}) - d(T_k, T_{n-1})|$.

C.6.3 Summary of Complexity Drivers

By substituting expensive d-dimensional dot products with scalar subtractions based on sequential adjacency, our approach seeks to maintain sub-quadratic efficiency while staying strictly grounded in the local metric path of the token sequence.

C.7 Background: Accelerating Transformer Attention with Clustering Algorithms

Using clustering algorithms to speed up the attention layer is one of the most heavily researched strategies for solving the "context window bottleneck" in transformers.

Because the standard self-attention mechanism compares every token to every other token, its computational complexity scales quadratically—$\mathcal{O}(N^2)$, where N is the sequence length. Clustering algorithms help bypass this by grouping similar tokens together, drastically reducing the number of calculations required.

Figure C.1: Comparison of quadratic complexity in standard attention versus linear complexity in efficient attention mechanisms.

Prominent Papers and Architectures

C.7.1 Fast Transformers with Clustered Attention [110]

- **Authors:** Apoorv Vyas, Angelos Katharopoulos, François Fleuret
- **The Method:** This paper directly tackles the problem by applying **K-Means clustering** to the attention queries. Instead of calculating the attention distribution for every single query in a sequence, the algorithm clusters the queries into groups based on their similarity. It then computes the exact attention *only for the cluster centroids*.
- **The Speedup:** The values calculated for the centroid are broadcasted back to the individual queries within that cluster. For a fixed number of clusters, this reduces the complexity from quadratic $\mathcal{O}(N^2)$ to strictly linear $\mathcal{O}(N)$.

C.7.2 Reformer: The Efficient Transformer [52]

- **Authors:** Nikita Kitaev, Łukasz Kaiser, Anselm Levskaya
- **The Method:** The Reformer is one of the most famous efficient transformers, and it uses a clustering-adjacent technique called **Locality-Sensitive Hashing (LSH)**. LSH is a fast, randomized hashing scheme that groups highly similar vectors into the same "buckets"

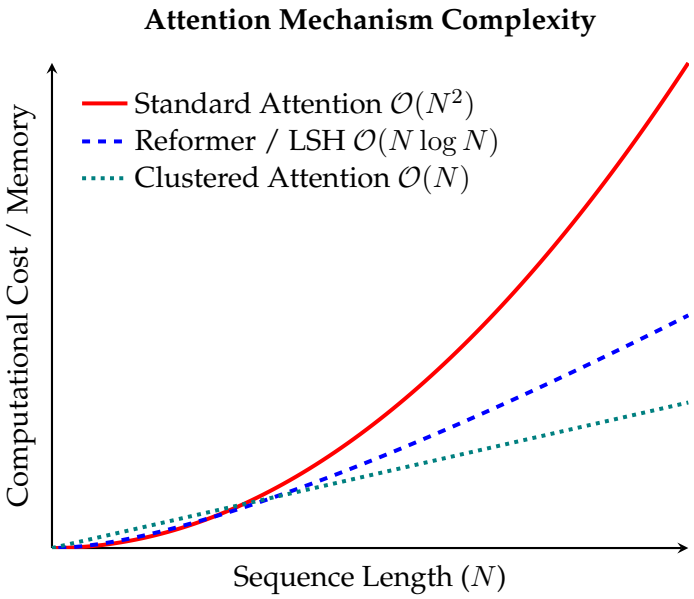


Figure C.2: A conceptual comparison showing how quadratic complexity $\mathcal{O}(N^2)$ rapidly exhausts memory as sequence length grows, whereas efficient clustering and hashing mechanisms scale at linear $\mathcal{O}(N)$ or log-linear $\mathcal{O}(N \log N)$ rates.

with high probability. In the Reformer, queries and keys are hashed into these buckets, and attention is only computed between items that fall into the same bucket.

- **The Speedup:** This prevents the model from wasting compute on tokens that have little relevance to one another, reducing the time and memory complexity to $\mathcal{O}(N \log N)$.

Figure C.3: Locality Sensitive Hashing (LSH) grouping similar queries and keys in the Reformer.

C.7.3 Routing Transformers [95]

- **Authors:** Aurko Roy, Mohammad Saffar, Ashish Vaswani, David Grangier
- **The Method:** Similar to the Reformer, the Routing Transformer attempts to sparsely route queries and keys to one another. It uses an **online k-means clustering** algorithm to learn which tokens should attend to each other dynamically. During the forward pass, tokens are assigned to clusters, and the attention mechanism is restricted so that tokens only attend to other tokens within their assigned cluster.
- **The Speedup:** This also achieves $\mathcal{O}(N\sqrt{N})$ or $\mathcal{O}(N \log N)$ complexity depending on the configuration, allowing the model to handle sequences with tens of thousands of tokens.

C.7.4 Neural Clustering Attention for Efficient and Effective Transformer [122]

- **Authors:** Yu Zheng, et al.

- **The Method:** While earlier methods used fixed or external algorithms (like standard K-Means or LSH), this paper proposes a *neural* clustering method that is trained jointly with the target task. It groups words with strong contextual dependencies into the same cluster and performs the attention mechanism for each cluster independently.
- **The Speedup:** It maintains linear complexity while improving the actual semantic quality of the clusters, leading to better accuracy on tasks like machine translation and text classification compared to older clustering methods.

C.7.5 Mathematical Relationship

In standard attention, the matrix is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{C.8}$$

In clustered attention, the standard query matrix Q is replaced by a much smaller centroid matrix Q_c. The computationally expensive QK^T multiplication is only performed on Q_c, and the resulting context vectors are then expanded back to the original sequence length.

CHAPTER D

The Hippocampal Scaffold and Compositional Sparsity

What is the relation between memory models of the hippocampus and compositional sparsity? This chapter argues that the hippocampus acts as a "scaffold builder," creating a sparse, structured graph of pointers that organizes raw experiences into a compositionally sparse format.

D.1 Introduction: The Memory Palace

To understand the mathematical definitions of x_t, y_t, and z_t, let us use an ancient mnemonic device: the **Memory Palace**. Imagine an agent walking through a house. At each time step t, the experience has three components:

- **The Latent Structure (z_t):** The abstract relational state (e.g., "Kitchen"). This is the *scaffold*.
- **The Sensory Input (x_t):** High-dimensional, noisy data (e.g., pixel views of tiles).
- **The Episodic Target (y_t):** The specific content to be stored (e.g., "Keys are on the counter").

D.2 The Variables of Experience

We formalize the data stream as a sequence of tuples $e_t = (x_t, y_t, z_t)$.

Latent State $z_t \in \mathcal{Z}$: Hidden ground truth of the agent's position. Transitions are sparse.

Observable Input $x_t \in \mathcal{X}$: Sensory features. $x_t = f(z_t) + \epsilon$, where ϵ is noise.

Target Content $y_t \in \mathcal{Y}$: Information to be associated with the state.

D.3 Mathematical Foundations of the Hippocampal Index

The hippocampus (specifically the Dentate Gyrus) transforms correlated sensory data into orthogonal addresses.

D.3.1 The Top-K Projection Mechanism

Let $g(x_t, z_t) \in \mathbb{R}^d$ be a joint embedding of sensory and latent features. The index h_t is generated via a high-dimensional expansion followed by a competitive nonlinearity:

$$h_t = \text{TopK}(R \cdot g(x_t, z_t) + b) \tag{D.1}$$

Where:

- $R \in \mathbb{R}^{N \times d}$ is a fixed random matrix with $N \gg d$.
- TopK$(\cdot)$ is a function that sets all elements to 0 except for the k largest values, enforcing $\|h_t\|_0 = k$.

D.3.2 Locality-Sensitive Hashing (LSH)

This mechanism functions as Locality-Sensitive Hashing. Per the Johnson-Lindenstrauss lemma, the random projection R approximately preserves the geometry of the input space:

$$\mathbb{E}[\langle Ru, Rv \rangle] \approx \langle u, v \rangle \tag{D.2}$$

The TopK operation then acts as a hard threshold for pattern separation. If the similarity between two states z_t and z_s is below a threshold τ, the resulting indices become orthogonal:

$$\langle h_t, h_s \rangle = 0 \quad \text{if} \quad \text{sim}(g_t, g_s) < \tau \tag{D.3}$$

This orthogonality prevents catastrophic interference in the synaptic weights.

D.4 Building the Scaffold Graph

The sequence of indices $\{h_t\}$ defines a graph $\mathcal{G} = (\mathcal{H}, \mathcal{E})$.

$$(h_t, h_{t+1}) \in \mathcal{E} \iff \text{Transition observed in } \mathcal{Z} \tag{D.4}$$

The transition function learned by the cortex, G_{sparse}, is defined by the adjacency matrix of this graph.

D.5 Connection to Existing Theories

This framework synthesizes several classical models:

- **Complementary Learning Systems (CLS):** Explains the need for h_t to be sparse to avoid interference.
- **Tolman-Eichenbaum Machine (TEM):** Aligns with the separation of z_t (structure) and x_t (sensory).
- **Successor Representation (SR):** The scaffold graph $\mathcal{G}$ is a discrete realization of the predictive SR map.

D.6 Conclusion

By mapping the world onto a sparse topological scaffold, the hippocampus "diagonalizes" complex environmental dynamics. This transforms an intractable dense learning problem into a compositionally sparse graph-traversal problem that the cortex can internalize through systems consolidation.

CHAPTER E

How Many Boolean Functions Are in P Versus Not in P

There is an astronomical number of Boolean functions in P – that is, functions that are computable in polynomial time. However, the number of functions that are not in P is much larger so that the ration of the first to the second goes to zero for $N \to \infty$.

E.1 Counting all Boolean functions

A Boolean function on N input bits is a map

$$f : \{0,1\}^N \to \{0,1\}.$$

There are 2^N possible inputs. Specifying f is equivalent to choosing an output bit for each of the 2^N inputs. Hence the total number of Boolean functions on N bits is

$$\#\{f : \{0,1\}^N \to \{0,1\}\} \;=\; 2^{2^N}. \tag{E.1}$$

E.2 A general upper bound on the number of functions computable with polynomial descriptions

In essentially any standard model of computation, a function f is computed by providing a finite *description* (e.g., a program, a circuit, or a structured decomposition) and then running the associated evaluator on inputs $x \in \{0,1\}^N$.

Fix N and consider any class of descriptions whose bit-length is at most $L(N)$. The number of binary strings of length at most $L(N)$ is at most

$$\sum_{\ell=0}^{L(N)} 2^\ell \;\le\; 2^{L(N)+1}. \tag{E.2}$$

Each description determines at most one Boolean function on $\{0,1\}^N$. Therefore the number of Boolean functions representable by descriptions of length at most $L(N)$ is at most

$$\#\{\text{functions representable with length } \le L(N)\} \;\le\; 2^{L(N)+1}. \tag{E.3}$$

In particular, if $L(N) = \mathrm{poly}(N)$, then

$$\#\{\text{functions with polynomial-size descriptions}\} \;\le\; 2^{\mathrm{poly}(N)}. \tag{E.4}$$

E.3 Implication for P

A common way to formalize the class P on input length N is via polynomial-time uniform computation, or equivalently (up to standard translations) via polynomial-size uniform circuit families. In any such setting, restricting to polynomial-time computation implies that the underlying descriptions for length-N inputs can be taken to have length at most $\mathrm{poly}(N)$.

Consequently, letting P_N denote the set of Boolean functions on N bits that are computable in polynomial time, we obtain the counting upper bound

$$\#P_N \;\le\; 2^{\mathrm{poly}(N)}. \tag{E.5}$$

E.4 Almost all Boolean functions are not in P

Combining (E.1) and (E.5), the fraction of Boolean functions on N bits that lie in P is at most

$$\frac{\#P_N}{2^{2^N}} \;\le\; \frac{2^{\mathrm{poly}(N)}}{2^{2^N}} \;=\; 2^{\mathrm{poly}(N)-2^N}. \tag{E.6}$$

Since 2^N dominates any polynomial in N, the exponent $\mathrm{poly}(N) - 2^N \to -\infty$ as $N \to \infty$, and thus

$$\lim_{N\to\infty} \frac{\#P_N}{2^{2^N}} \;=\; 0. \tag{E.7}$$

Equivalently, the number of Boolean functions *not* in P is

$$2^{2^N} - \#P_N \;=\; 2^{2^N}\big(1 - o(1)\big), \tag{E.8}$$

meaning that *almost all* Boolean functions on N bits are not in P.

E.5 Connection to structured representations (example: local Fourier-polynomial compositions)

In the main text we considered Boolean functions specified by a structured decomposition into m gates, where each gate computes a multilinear (Fourier) polynomial depending on at most k inputs, and where each Fourier coefficient is encoded with $b = \Theta(\log N)$ bits.

Such a representation has description length at most on the order of

$$L(N) \;=\; O\big(m \cdot 2^k \cdot b \;+\; mk \log(N+m)\big),$$

where the first term encodes the coefficients and the second term encodes wiring (input choices) for the gates. When $m = O(k)$ and $k = O(\log N)$, we have $2^k = \mathrm{poly}(N)$ and $b = \Theta(\log N)$, so $L(N) = \mathrm{poly}(N)$. Hence the number of Boolean functions representable in this way is at most

$$2^{\mathrm{poly}(N)},$$

which is still negligible compared to the 2^{2^N} total Boolean functions.

Takeaway. Classes with polynomial-size descriptions (including polynomial-time computable functions, and many structured subclasses) contain astronomically many functions, but they form an asymptotically vanishing fraction of all Boolean functions on N bits.

CHAPTER F

Boolean Circuits and a Path to "Superintelligence" (by D. Koplow)

In recent years, it has become a running joke that the quickest path to a top-tier machine learning publication is to introduce a new benchmark. This obsession with narrow evaluations implicitly assumes that intelligence is tied to specific domains. While knowledge is inherently domain-specific, reasoning is not. We seek a minimal representational framework that requires very little background knowledge while still being able to express any computation. Boolean circuits meet this criterion. As universal computational models, they can represent any digital algorithm. When an LLM is unable to manipulate such circuits correctly, this shortcoming reveals a core limitation in its reasoning capabilities. This book argues that compositional sparsity is the primary factor enabling learnability in machine learning. From this viewpoint, we construct sparse Boolean circuits that contemporary models reliably fail to solve without programming, thereby revealing a strikingly human-like strategy for overcoming this constraint. This strategy may constitute a crucial step on the path toward super-intelligence.

F.1 Boolean Circuits

A Boolean circuit is a Directed Acyclic Graph (DAG) $G = (V, E)$ where input nodes represent binary variables $x_1, \ldots, x_n \in \{0, 1\}$ and internal nodes (gates) represent logical functions from a standard basis set such as $\mathcal{B} = \{\text{AND}, \text{OR}, \text{NOT}, \text{NOR}, \text{XOR}\}$. The power of this model lies in its universality. For any function $f : \{0, 1\}^n \to \{0, 1\}$ computable by a Turing machine in time $T(n)$, there exists a Boolean circuit of size $Poly(T(n))$ that computes f. This means that a model's ability to learn Boolean circuits is equivalent to its ability to *solve any* computational problem.

While evaluating a circuit is computationally trivial, the inverse problem, called *Circuit Synthesis*, is notoriously difficult. Given a dataset of input-output pairs $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{m}$, finding the smallest circuit that is consistent with $\mathcal{D}$ is an NP-hard problem given a truth table that is exponentially large with respect to the input. Thus, the search space for circuits grows super-exponentially with the number of inputs. As a result, learning a generic function **efficiently is, in general, impossible**.

Yet, somehow, deep-learning is able to construct models able to solve many important problems in our world. As argued in this book, a core reason why is *compositional sparsity*. Under this assumption, the hypothesis space of circuits starts becoming much more tractable.

To keep one concrete mental model in view, assume we are trying to learn a compositionally sparse function. It can be represented as a Boolean circuit with fan-in 2 and depth $O(\log_2(n))$.

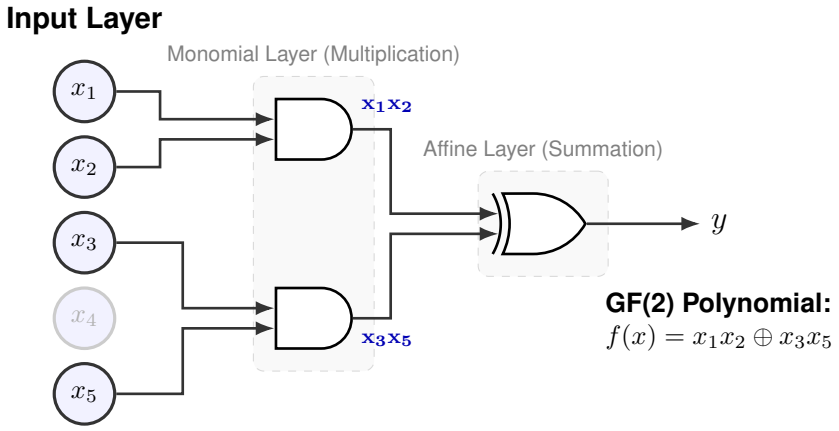


Figure F.1: Example GF(2) circuit.

Further, we will restrict the class of circuits to be those that can be represented as a sum of monomials in GF(2). The resulting function has a bounded maximum degree of monomials. An example visualization is shown in Figure F.1.

F.2 Learning

We now have at our disposal a highly expressive class of functions which, due to their compositional sparsity and adjustable complexity, allow us to evaluate how effectively any given model uncovers the appropriate compositionally sparse representation from data. At this point, it is crucial to clarify what we mean by learning in this context. Traditionally, learning is framed as the problem of finding a function f such that $f(x) \approx y$ for $(x, y) \sim D$, where D is some underlying data distribution. In contemporary machine learning, this view is largely aligned with the paradigm of "deep learning," in which we posit a parametric function $f_1(x; \theta)$ and apply some variant of gradient descent to adjust θ so as to maximize the predictive performance of $f_1(x; \theta)$. We shall refer to this approach as direct learning.

However, this is not the only way to obtain a function f. Alternatively, we might define a process g which, given access to D, does not directly output predictions, but instead produces a function f_2 that, when later executed, generates the correct outputs. This resulting function could then be run by the model itself or by an external executor. In this chapter, we will refer to this paradigm of learning as transform learning.

Both approaches constitute learning from data, but they do so at different levels of abstraction. In direct learning, the system identifies a specific function that maps inputs to outputs within a fixed hypothesis class. In transform learning, the system instead acquires a procedure for constructing such functions from data, thereby shifting part of the burden of generalization to the generated functions themselves. From this perspective, "solving a problem" and "learning a solution" are closely related operations that differ primarily in where adaptation occurs. Both aim to reduce uncertainty about an unknown target, though they encode and exploit that knowledge in structurally distinct ways.

F.2.1 Reasoning

From this standpoint, reasoning can be understood as an iterative procedure in which multiple functions are proposed to transform data, their transformed outputs are assessed, and those

assessments guide which new functions are generated next. Rather than committing immediately to a single function, the *system searches* over a broader space of candidate functions, composing and refining them in response to intermediate feedback. Under this interpretation, reasoning becomes a recursive form of transform-based learning.

When an LLM tackles a complex problem using chain-of-thought, it effectively plays the role of the generator g, emitting a sequence of operations that together implement a particular circuit f tailored to the given input. In the setting of "in-context learning," the model is not modifying its weights through gradient descent; instead, it is constructing a temporary program that maps the specific inputs to the appropriate output.

For the first time, we have a non-biological system that can learn a generator g which can be sampled to implement f across an exceptionally wide range of tasks. While much of the literature on compositional sparsity has focused on training-time optimization dynamics or the structure of the hypothesis space, we can now adopt new perspectives on compositional sparsity to illuminate the capabilities of modern LLMs: compositional sparsity in the functions f that g generates, and compositional sparsity in the way g executes f.

F.2.2 Measuring Learnability

If reasoning is, as we argue, a generative process where a model derives a program f, then we must rigorously quantify the reliability of that generation. How can we measure the probability that this step-by-step construction will eventually lead to a correct solution?

The Limits of Direct Learning. The standard lens for assessing learnability is the *Probably Approximately Correct (PAC)* framework **[valiant1984theory]**, which focuses on the ability of an algorithm to approximate a function from data.

Definition 11 (PAC-Learnability)**.** *A concept class $\mathcal{C}$ is PAC-learnable if there exists an algorithm that, for any target concept $c \in \mathcal{C}$ and any distribution D, outputs a hypothesis h with error at most ϵ with probability at least $1 - \delta$, using time and samples polynomial in $1/\epsilon, 1/\delta$, and the size of the concept.*

For general Boolean circuits, the news is bleak: they are known to be **not PAC-learnable [kearns1994cryptographic]**. If they were, one could trivially break standard cryptographic schemes like RSA by simply observing input-output pairs.

Crucially, however, our interest lies not in learning arbitrary opaque functions, but in **compositionally sparse** circuits. Unlike the general case, specific subclasses of these circuits often *are* theoretically learnable. Yet, despite this theoretical possibility, modern Deep Learning models operating in a "Direct Learning" mode, attempting to predict y directly from x, consistently fail to generalize on these tasks. This discrepancy suggests that while the class of functions may be learnable in principle, the standard supervised paradigm is not the mechanism by which Large Language Models (LLMs) solve them. To understand how models actually conquer these problems, we must shift our perspective from static approximation to sequential reasoning.

The Diligent Learner To model this dynamic derivation, we adopt the **Diligent Learner** framework proposed by Shalev-Shwartz et al. (2025). This approach reframes reasoning as a traversal through a decision tree of semantic states.

The Diligent Learner is defined formally as a tuple $\mathcal{L} = (Root, \mathcal{S}, V, \pi_\theta)$:

1. **Root:** The initial problem instance.

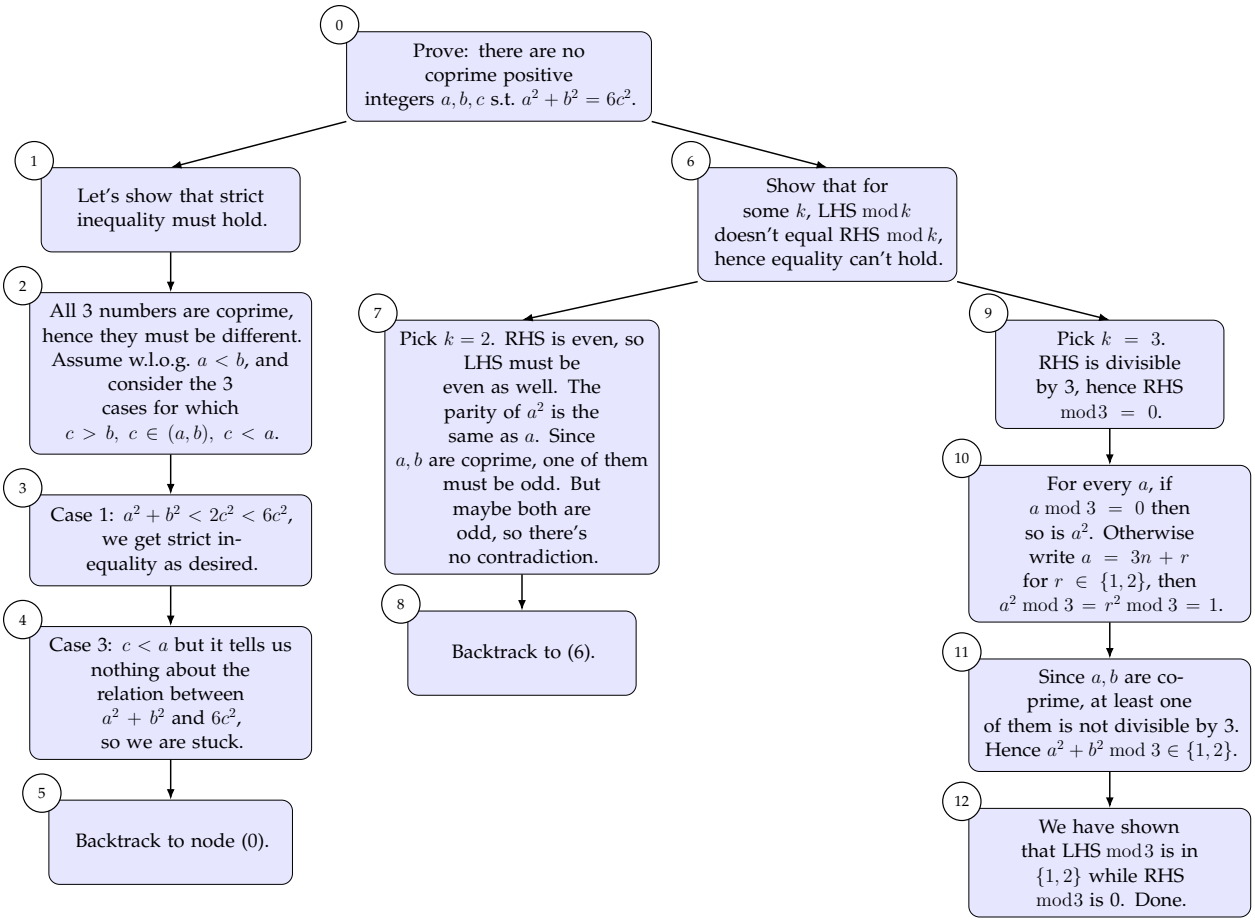


Figure F.2: Diligent learner visualization from [**ShalevShwartzShashua2025Diligent**].

2. **State Space $\mathcal{S}$:** A set of partial states h, where each h represents a prefix of a proof, program, or circuit.

3. **Validator V:** A mechanism (whether internal or external) that enforces local correctness, pruning branches that are logically invalid.

4. **Proposer π_θ:** The generator (agent) that proposes the next extension action based on the current state.

Figure F.2 illustrates this process. At each step, the Proposer suggests a next step (for which the Validator must accept as logically sound) or decide to back track. The same approach can be used to synthesizing Boolean circuits from data: the model builds the circuit term-by-term with a Validator ensuring the step is logically sound.

The Bottleneck Parameter (γ). In this search-based paradigm, the critical bottleneck is the reliability of the *next step*. This reliability is quantified by γ, the probability that the Proposer will make a correct next step in solving the problem.

Let $\pi_\theta(\cdot \mid h)$ be the stochastic policy over actions approved by the validator, given a partial state h. An *extension action* proposes the next semantic step. We denote $S(h, a) = 1$ if the prefix (h, a) keeps the learner on a path that can be completed to a full, correct solution. The bottleneck

parameter γ represents the probability mass the model places on these *useful* next moves:

$$\Pr_{a\sim\pi_\theta(\cdot|h)}\left[a \in \{a_i \mid S(h,a_i)=1\}\right] \;\geq\; \gamma. \tag{F.1}$$

This parameter effectively dictates the feasibility of the reasoning process. If γ remains high, the learner consistently identifies the sparse, valid path through the logic. However, if γ decays as the depth of reasoning increases, the search complexity explodes. Specifically, for a target failure probability δ and maximum reasoning depth $T_{\max}$, the number of steps required scales as:

$$O\Big(T_{\max}\cdot\frac{\log(T_{\max}/\delta)}{\gamma}\Big). \tag{F.2}$$

While contemporary LLMs do not currently perform this explicit backtracking search, this framework provides a crucial theoretical upper bound on a model's reasoning capacity. As they allude to in their work, maximizing γ is the most important objective for any system trying to attain "superintelligence."

F.3 The Computational Cliff

Realizing the full "Diligent Learner" with backtracking, state validation, and fine-tuned proposers presents engineering and reliability hurdles. Yet, we need not build the complete system to test the paradigm. We can leverage part of the framework to assess the capabilities of existing models by isolating the single-step generation process. Even without the full apparatus of recursion, testing whether an LLM can reliably produce the next valid term in a sparse circuit offers a powerful diagnosis of its reasoning fidelity.

To evaluate this capability, we introduce a stepwise reconstruction benchmark over GF(2). At each depth g, the model is presented with a prefix P_g and a labeled sample set S_g, and must output the next monomial t_{g+1}. The setup is designed to be rigorous: Ignoring the prefix P_g leaves the labels masked by the structure of the prefix. Ignoring the samples S_g reduces the task to blind guessing.

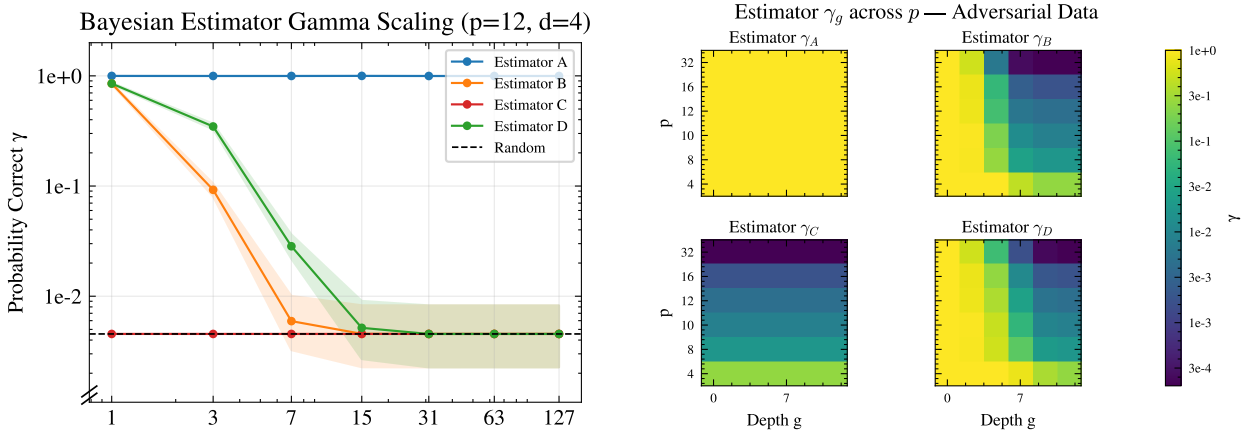
F.3.1 Problem Formulation

We articulate the game using the clean language of Boolean functions in GF(2), revealing the function one monomial at a time.

Targets. We consider inputs $x=(a,v)\in\{0,1\}^{n+p}$, where $a=(a_1,\ldots,a_n)$ are address bits and $v=(v_1,\ldots,v_p)$ are payload bits. We restrict our attention to Algebraic Normal Form (ANF): XORs of monomials, $f(a,v)=\bigoplus_{j=1}^{n} t_j(a,v)$, where each term takes the form

$$t_j(a,v) := a_j\, M_j(v), \quad M_j(v) := \prod_{i\in S_j} v_i, \tag{F.3}$$

with supports $S_j\subseteq[p]$ of fixed size $|S_j|=d-1$. Each term is effectively sparse, touching only d variables, and the entire target is a sparse sum of such terms. An *instance* consists of the ordered support sequence $(S_1,\ldots,S_n)$, sampled once and then held fixed.



(a) Only history+data sustains reliable next-step prediction: step success γ_g versus depth g.

(b) As g and p grow, imperfect-information estimators collapse; only $\mathcal{A}$ remains reliable.

Figure F.3: Stepwise reconstruction performance under adversarial sampling.

The Stepwise Reconstruction Game. At depth $g \in \{0, \ldots, n-1\}$, the learner receives:

1. the ordered prefix $P_g := (t_1, \ldots, t_g)$, and
2. a labeled sample set $\mathsf{S}_g := \{(x^{(k)}, y^{(k)})\}_{k=1}^{K}$ generated for that specific depth.

The model's task is to output a candidate monomial $\hat{t}$. Success is binary: the model succeeds if and only if $\hat{t} = t_{g+1}$. We score the model by its exact success probability,

$$\gamma_g \;:=\; \Pr_{\hat{t}\sim\pi_\theta(\cdot|P_g,\mathsf{S}_g)}\big[\hat{t} = t_{g+1}\big], \tag{F.4}$$

where the probability is taken over the sampled instance, oracle randomness, and the model's own stochasticity. Since the target is an ordered sequence, there is exactly one correct continuation at each depth, making γ_g a measure of exact-next accuracy.

Who is allowed to see what? We distinguish between solvers based on the information they utilize: (i) **Diligent** ($\mathcal{A}_g$) solvers access both the prefix and samples (P_g, S_g); (ii) **Data-only** ($\mathcal{B}_g$) solvers see S_g but are blind to P_g; (iii) **History-only** ($\mathcal{C}_g$) solvers see P_g but lack S_g; and (iv) **Partial** ($\mathcal{D}_g$) solvers have limited access to each. The benchmark is explicitly tuned to separate these classes:

$$\min_g \gamma_g^{\mathcal{A}} \;\geq\; Q \quad \text{while} \quad \gamma_g^{\mathcal{B}},\, \gamma_g^{\mathcal{C}},\, \gamma_g^{\mathcal{D}} \;\approx\; \tfrac{1}{\binom{p}{d-1}}. \tag{F.5}$$

In essence: if a model refuses to use either the prefix or the samples, it is forced to fall back to random guessing.

F.3.2 The Statistical Obfuscation Construction

We enforce the separation between reasoning and pattern matching through statistical obfuscation. At depth g, the label represents a superposition (the XOR sum) of the target signal $M_{g+1}(v)$ and a history-dependent mask. Because this mask is a deterministic function of the prefix P_g and input x, it completely randomizes the label for any observer neglecting variables from the history. For the diligent solver, however, the mask is transparent.

Instance Generation. We sample supports $S_1, \ldots, S_n \subseteq [p]$ with $|S_j| = d-1$ once per instance. This fixes the ordered ANF terms $(t_1, \ldots, t_n)$ in (F.3).

Payload Distribution. We draw payloads at a fixed Hamming weight w, sampling v uniformly from $\{v \in \{0,1\}^p : \|v\|_0 = w\}$. For any fixed S with $|S| = d-1$, we define

$$\rho(w) \;:=\; \Pr_v\left[M_S(v) = 1\right] \;=\; \frac{\binom{w}{d-1}}{\binom{p}{d-1}} \quad (w \geq d-1), \tag{F.6}$$

selecting $w^\star$ such that $\rho(w)$ approximates $1/2$. This entropy maximization preserves the mask's efficacy and minimizes label bias.

Step-g Sampling Oracle (Fixed-Prefix Obfuscation). Given an instance and depth g, the oracle produces $\mathsf{S}_g = \{(a^{(k)}, v^{(k)}, y^{(k)})\}_{k=1}^K$ by constructing a set of carefully designed examples. It fixes $a_{g+1} = 1$ (zeroing subsequent bits) while sampling preceding address bits randomly, outputting $y := f(a, v)$.

The label decomposes as:

$$y \;=\; \underbrace{\Big(\bigoplus_{j=1}^{g} a_j\, M_j(v)\Big)}_{\text{prefix obfuscation}} \;\oplus\; \underbrace{M_{g+1}(v)}_{\text{next-term signal}}\;. \tag{F.7}$$

The mask is fully determined by the prefix and input. A solver utilizing the prefix can cleanly expose the signal, whereas a solver refusing the prefix perceives only noise.

F.3.3 Theoretical Guarantees

We provide rigorous guarantees for this separation. The next-term signal remains statistically inaccessible to history-blind solvers while being fully recoverable by history-aware ones. The barrier is one of information access and state tracking, rather than computational complexity.

Monomial firing at fixed weight. Since payloads are uniform over a fixed Hamming-weight sphere, the probability of a payload monomial "firing" follows a closed form.

Lemma 12 (Monomial firing probability at fixed Hamming weight)**.** *Fix integers $p \geq d-1 \geq 1$ and $w \in \{0, \ldots, p\}$. Let v be uniform over the Hamming sphere $\{v \in \{0,1\}^p : \|v\|_0 = w\}$, and fix any $S \subseteq [p]$ with $|S| = d-1$. Define $M_S(v) := \prod_{i \in S} v_i$. Then*

$$\Pr\left[M_S(v) = 1\right] = \begin{cases} \dfrac{\binom{w}{d-1}}{\binom{p}{d-1}} = \dfrac{\binom{p-(d-1)}{w-(d-1)}}{\binom{p}{w}} & \text{if } w \geq d-1, \\ 0 & \text{if } w < d-1. \end{cases}$$

Per-sample Obfuscation. A data-only estimator $\mathcal{B}_g$, observing data but lacking the prefix P_g, cannot derive the supports determining the mask. We demonstrate a *single-example* guarantee: across the unknown prefix supports, each individual labeled example offers negligible Bayes advantage regarding the next-term signal.

Lemma 13 (Bayes masking given observed (a, v))**.** *Assume the instance distribution samples $S_1, \ldots, S_g, S_{g+1}$ i.i.d. uniformly from $\{S \subseteq [p] : |S| = d-1\}$, independently of the oracle samples. Fix a step g and condition on a realized example (a, v) with $\|v\|_0 = w^\star$. Let*

$$\rho := \rho(w^\star) = \Pr_S[M_S(v) = 1] = \frac{\binom{w^\star}{d-1}}{\binom{p}{d-1}}, \quad m := m(a).$$

Then, marginalizing over the unknown prefix supports $(S_1, \ldots, S_g)$, for each $r \in \{0, 1\}$,

$$\Pr\left[B(a, v) = r \mid a, v\right] \;=\; \tfrac{1}{2}\left[1 + (-1)^r (1 - 2\rho)^m\right].$$

Moreover, $B(a, v)$ is independent of $b = M_{g+1}(v)$ given (a, v), and since $y = B(a, v) \oplus b$ we have

$$\left|\Pr[y = b \mid a, v] - \tfrac{1}{2}\right| \;=\; \tfrac{1}{2}|1 - 2\rho|^m.$$

Lemma 13 confirms that the mask effectively neutralizes single-sample bias, which decays exponentially with the number of active address bits.

History-only Baseline. Reliance on history alone is similarly ineffective. The prefix determines past supports without leaking information about the future support S_{g+1}.

Lemma 14 (History-only is prior guessing)**.** *Assume the instance distribution samples supports $S_1, \ldots, S_n$ i.i.d. uniformly (*with replacement*) from $\{S \subseteq [p] : |S| = d-1\}$. Then for any $g < n$, conditioned on the revealed prefix P_g, the next support S_{g+1} is uniform over $\{S \subseteq [p] : |S| = d-1\}$ and independent of P_g. Consequently, any history-only estimator satisfies $\Pr\left[\widehat{S} = S_{g+1}\right] \le \frac{1}{\binom{p}{d-1}}$.*

Having established the insufficiency of partial information, we demonstrate that a Diligent solver, unifying data and history, can convert the task into a tractable polynomial-time problem.

Recoverability in Polynomial Time. With access to P_g, the solver computes residual labels

$$r^{(k)} \;:=\; y^{(k)} \oplus \bigoplus_{j=1}^{g} a_j^{(k)} M_j(v^{(k)}) \;=\; M_{g+1}(v^{(k)}), \tag{F.8}$$

reducing the problem to identifying the unknown support S_{g+1} from labeled payloads via an intersection decoder:

$$\widehat{S} \;:=\; \bigcap_{k \in K_+} \operatorname{supp}(v^{(k)}), \tag{F.9}$$

where $K_+ := \{k : r^{(k)} = 1\}$.

Theorem 35 (Poly-time recovery under fixed-weight payloads)**.** *Fix an instance and assume payloads are i.i.d. uniform on $\{v \in \{0,1\}^p : \|v\|_0 = w^\star\}$ with $w^\star \ge d-1$. Let $\rho = \rho(w^\star) = \Pr[M_{g+1}(v) = 1]$ and $T = |K_+|$. Then $\Pr[T = 0] = (1-\rho)^K$. Moreover, conditioned on $T \ge 1$, the decoder succeeds with probability at least*

$$1 - \min\Big\{1,\; (p - (d-1))\Big(\frac{w^\star - (d-1)}{p - (d-1)}\Big)^T\Big\}. \tag{F.10}$$

Proof. If $r^{(k)} = 1$ then $M_{g+1}(v^{(k)}) = 1$, which is equivalent to $S_{g+1} \subseteq \mathrm{supp}(v^{(k)})$. Thus for every $k \in K_+$ we have $S_{g+1} \subseteq \mathrm{supp}(v^{(k)})$, and hence

$$S_{g+1} \subseteq \bigcap_{k \in K_+} \mathrm{supp}(v^{(k)}) \;=\; \widehat{S}.$$

Therefore, conditioned on $T \geq 1$, the intersection decoder can fail only if $\widehat{S}$ contains at least one *extraneous* coordinate $i \notin S_{g+1}$, i.e., some $i \in [p] \setminus S_{g+1}$ appears in every positive support. Fix any $i \notin S_{g+1}$ and consider a single draw v conditioned on $r = 1$. Under the fixed-weight model, after forcing ones on S_{g+1}, the remaining $w^\star - (d-1)$ ones are chosen uniformly among the $p - (d-1)$ coordinates in $[p] \setminus S_{g+1}$. Hence

$$\Pr[i \in \mathrm{supp}(v) \mid r = 1] \;=\; \frac{w^\star - (d-1)}{p - (d-1)}.$$

Now condition on the event $\{T = |K_+|\}$ and on the index set K_+ itself. Because the original examples are i.i.d., the payloads $\{v^{(k)}\}_{k \in K_+}$ are i.i.d. draws from the conditional distribution $(v \mid r = 1)$, so

$$\Pr\Big[i \in \mathrm{supp}(v^{(k)})\ \forall k \in K_+ \,\Big|\, T\Big] \;=\; \Big(\frac{w^\star - (d-1)}{p - (d-1)}\Big)^T.$$

Taking a union bound over the $p - (d-1)$ possible extr coordinates gives

$$\Pr\left[\widehat{S} \neq S_{g+1} \mid T\right] \;\leq\; (p - (d-1))\Big(\frac{w^\star - (d-1)}{p - (d-1)}\Big)^T,$$

which implies (F.10). Finally, since $T = \sum_{k=1}^K \mathbf{1}\{r^{(k)} = 1\}$ with $\Pr\Big[r^{(k)} = 1\Big] = \rho = \rho(w^\star)$, we have $\Pr[T = 0] = (1-\rho)^K$. □

Corollary 6 (High-probability recovery with K samples)**.** *In the setting of Thm. 35, let* $\rho := \rho(w^\star) = \Pr[M_{g+1}(v) = 1]$ *and* $\alpha := \frac{w^\star - (d-1)}{p-(d-1)} \in [0, 1)$*. Fix* $\delta \in (0, 1)$*. For* $\alpha \in (0, 1)$ *define*

$$T_0 := \left\lceil \frac{\log\big(2(p - (d-1))/\delta\big)}{\log(1/\alpha)} \right\rceil \quad \text{(and if } \alpha = 0, \text{ take } T_0 := 1\text{)}.$$

If $K \geq \frac{1}{\rho}\max\{2T_0,\ 8\log(2/\delta)\}$*, then the decoder in* (F.9) *outputs* $\widehat{S} = S_{g+1}$ *with probability at least* $1 - \delta$*.*

Proof. Let $T = |K_+| = \sum_{k=1}^K \mathbf{1}\{r^{(k)} = 1\}$. Since $r^{(k)} = M_{g+1}(v^{(k)})$ and the payloads are i.i.d., we have $T \sim \mathrm{Bin}(K, \rho)$. By Thm. 35, for any $t \geq 1$,

$$\Pr\Big[\widehat{S} \neq S_{g+1} \mid T = t\Big] \;\leq\; (p - (d-1))\alpha^t.$$

Hence

$$\Pr\Big[\widehat{S} \neq S_{g+1}\Big] \;\leq\; \Pr[T < T_0] \;+\; \Pr\Big[\widehat{S} \neq S_{g+1} \mid T \geq T_0\Big].$$

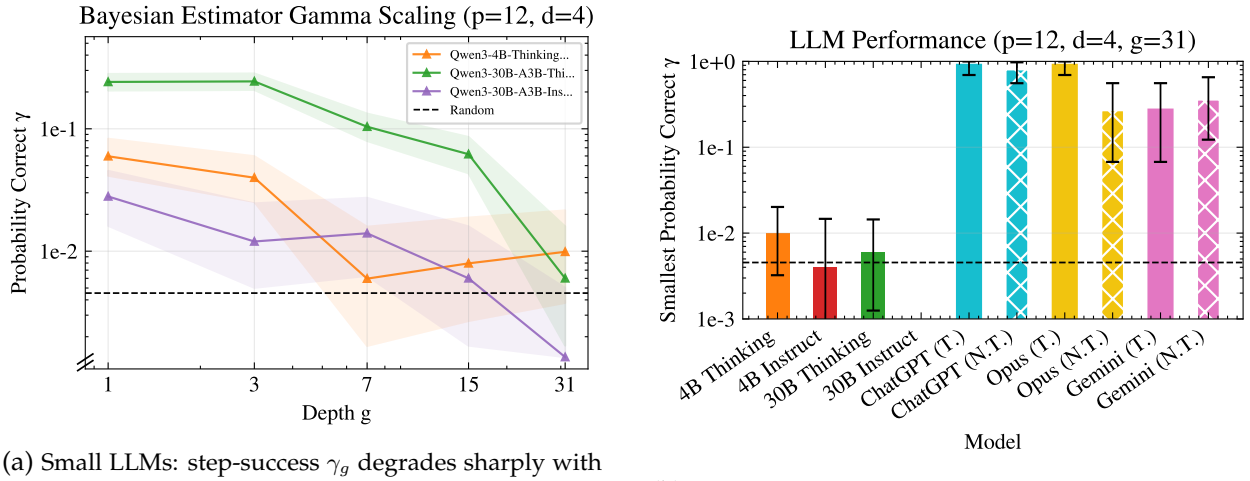
For the first term, our lower bound on K implies $K\rho/2 \geq T_0$, so by a multiplicative Chernoff bound,

$$\Pr[T < T_0] \;\leq\; \Pr\Big[T \leq \tfrac{1}{2}K\rho\Big] \;\leq\; \exp\Big(-\tfrac{1}{8}K\rho\Big) \;\leq\; \delta/2,$$

where the last inequality uses $K\rho \geq 8\log(2/\delta)$. For the second term, on $T \geq T_0$ we have

$$\Pr\Big[\widehat{S} \neq S_{g+1} \mid T \geq T_0\Big] \;\leq\; (p - (d-1))\alpha^{T_0} \;\leq\; \delta/2,$$

by the definition of T_0 (and trivially if $\alpha = 0$). Combining the two bounds yields $\Pr\Big[\widehat{S} \neq S_{g+1}\Big] \leq \delta$. □



(a) Small LLMs: step-success γ_g degrades sharply with depth g.

(b) Frontier LLMs achieve higher γ_g than smaller models.

Figure F.4: Depth-induced collapse in next-step prediction across model families.

These results define the information-theoretic landscape. Lemma 13 and Lemma 14 establish the necessity of combining signal sources, proving that isolation leads to failure. Theorem 35 provides the constructive converse: the prefix serves as a decryption key, transforming the noisy label into a clean signal recoverable with polynomial resources. The task separates those who track state from those who merely pattern-match.

Efficient Validation. A robust benchmark requires a fast grader. Given the instance specification and a candidate monomial $\tilde{t}$, our validator parses $\tilde{t}$ and accepts if and only if the payload index set $\widetilde{S}$ matches S_{g+1}. This process is highly efficient, taking $O(|\tilde{t}| + d \log d)$ time, allowing for rapid evaluation.

F.3.4 Tool Synthesis

This brings us to a phenomenon we term the "Computational Cliff." Circuit synthesis requires exact state tracking. Local errors eventually propagate and corrupt the entire derivation in a way that can be well described as a Gaussian model with only a partial memory. While the exact complexity of the problem that triggers the collapse changes with the capacity of the model, we do observe it for all models beyond a critical depth that do not build and use tools koplow2026superintellgence.

Shortening Error Chains In a reasoning trace of length N, independent error probabilities ϵ compound to yield a success rate of $(1-\epsilon)^N$. Tool use restructures this probability. By offloading complex logic to verified external solvers, the effective logical depth contracts, replacing fragile token-by-token generation with robust atomic operations.

The Law of Recursion This aligns with the **Law of Recursion [koplow2026lawsofemergence]**:

> "A system designed to maximally utilize compositional sparsity will design other systems that can use compositional sparsity even better for specific domains."

A sophisticated intelligence does not simulate all computation within its primary weights. It builds specialized external structures to handle irreducible complexity. Biologically, the cell

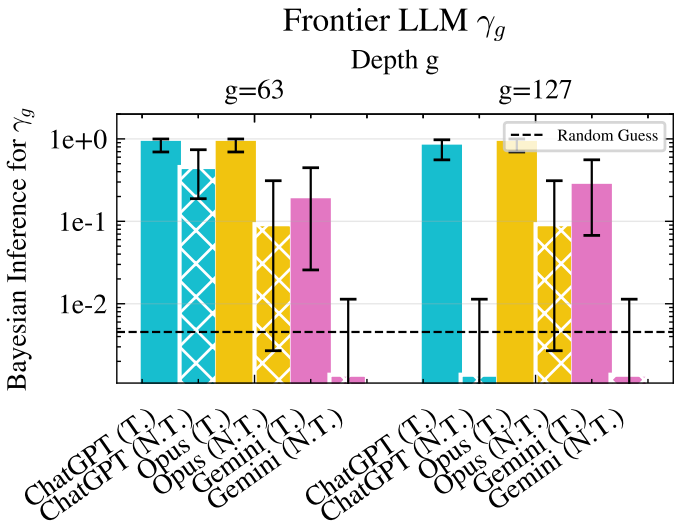


Figure F.5: Frontier models with tool use (T.) sustain higher γ_g at larger depths than no-tools variants (N.T.).

does not rely on a homogeneous cytoplasm for all chemistry; it compartmentalizes function into organelles. Similarly, synthetic intelligence ought not simulate circuit evaluation in raw neural activations, but rather synthesize programs to execute the task. Reasoning, then, is the act of generating the executable.

CHAPTER G

Gradient Formulas for Iterated Functional Compositions

In several analytical paradigms—ranging from the structural properties of deep neural architectures (e.g., Recurrent Neural Networks) to discrete dynamical systems and chaos theory—it is necessary to evaluate the exact derivative of an n-times iterated composition of a function mapping a space into itself. This appendix establishes the formal notation, explicit gradient derivations, and spectral implications for both scalar and multivariable settings.

Let $f : \mathcal{X} \to \mathcal{X}$ be a continuously differentiable map ($\mathcal{X} \subseteq \mathbb{R}^d$). We define the n-times iterated composition function $F : \mathcal{X} \to \mathcal{X}$ by:

$$F(x) = f^{(n)}(x) = \underbrace{(f \circ f \circ \cdots \circ f)}_{n \text{ times}}(x) \tag{G.1}$$

To make the application of the chain rule structurally transparent, we introduce a discrete-time state trajectory sequence $\{x_k\}_{k=0}^{n}$ mapping the forward evaluation path:

$$\begin{aligned} x_0 &= x \\ x_1 &= f(x_0) \\ x_2 &= f(x_1) \\ &\vdots \\ x_k &= f(x_{k-1}) \\ &\vdots \\ x_n &= f(x_{n-1}) = F(x) \end{aligned} \tag{G.2}$$

(G.3)

G.1 The Univariate Case ($d = 1$)

Let $\mathcal{X} \subseteq \mathbb{R}$, where $f : \mathbb{R} \to \mathbb{R}$ is a differentiable scalar function. By applying the classical univariate chain rule to the sequence of intermediate states, the total derivative of the iterated composition can be factored into a sequence of scalar multipliers:

$$\frac{dF}{dx} = \frac{dx_n}{dx_0} = \frac{dx_n}{dx_{n-1}} \cdot \frac{dx_{n-1}}{dx_{n-2}} \cdots \frac{dx_1}{dx_0} \tag{G.4}$$

Because each transition is defined locally by $x_k = f(x_{k-1})$, the derivative of any intermediate state with respect to its immediate predecessor is simply the first derivative of f evaluated at that predecessor state:

$$\frac{dx_k}{dx_{k-1}} = f'(x_{k-1}) \tag{G.5}$$

Substituting this relation back into the chain rule expansion yields:

$$\frac{dF}{dx} = f'(x_{n-1}) \cdot f'(x_{n-2}) \cdots f'(x_1) \cdot f'(x_0) \tag{G.6}$$

Expressed compactly using product notation, the final derivative is:

$$\frac{dF}{dx} = \prod_{k=0}^{n-1} f'(x_k) = \prod_{k=0}^{n-1} f'\left(f^{(k)}(x)\right) \tag{G.7}$$

where $f^{(0)}(x) \equiv x$.

G.2 The Multivariate Case ($d > 1$)

Let $\mathcal{X} \subseteq \mathbb{R}^d$, where $f : \mathbb{R}^d \to \mathbb{R}^d$ represents a multivariable operator. The derivative of f with respect to its input vector is expressed via the Jacobian matrix $J_f \in \mathbb{R}^{d \times d}$.

Because matrix multiplication is inherently non-commutative ($AB \neq BA$), the linear ordered sequence of the chain rule must be strictly preserved. Applying the multivariable chain rule from the outermost transformation down to the input vector yields the matrix product from left to right:

$$J_F(x) = J_f(x_{n-1}) \cdot J_f(x_{n-2}) \cdots J_f(x_1) \cdot J_f(x_0) \tag{G.8}$$

where each term $J_f(x_k) \in \mathbb{R}^{d \times d}$ represents the local Jacobian matrix evaluated explicitly at the intermediate state x_k:

$$[J_f(x_k)]_{ij} = \left.\frac{\partial f_i}{\partial x_j}\right|_{x=x_k} \tag{G.9}$$

Using the right-to-left product operator $\prod$, this composition can be condensed as:

$$J_F(x) = \prod_{k=1}^{n} J_f(x_{n-k}) \tag{G.10}$$

G.2.1 Gradient of a Scalar Loss Function

In applications involving optimization, such as empirical risk minimization, one typically encounters a scalar-valued objective or loss function $\mathcal{L} : \mathbb{R}^d \to \mathbb{R}$ evaluated at the final terminal state x_n, such that $\mathcal{L}(F(x))$.

To compute the gradient vector $\nabla_x \mathcal{L} \in \mathbb{R}^d$ with respect to the original input parameters, we apply the identity $(AB)^T = B^T A^T$. This reverses the operational ordering of the constituent Jacobian matrices during backpropagation:

$$\nabla_x \mathcal{L} = [J_F(x)]^T \nabla_{x_n} \mathcal{L} \tag{G.11}$$

Expanding this via the intermediate states yields:

$$\nabla_x \mathcal{L} = \left(J_f(x_0)^T \cdot J_f(x_1)^T \cdots J_f(x_{n-2})^T \cdot J_f(x_{n-1})^T\right) \nabla_{x_n} \mathcal{L} \tag{G.12}$$

G.3 Structural and Spectral Implications

The multiplicative composition of derivatives outlined in equations (A.6) and (A.8) has deep theoretical consequences in numerical analysis, system dynamics, and representation theory.

G.3.1 The Stability of Deep Iterated Maps

Consider the spectral radius $\rho(J_f(x)) = \max_i |\lambda_i|$, where λ_i are the eigenvalues of the local Jacobian matrix. The structural behavior of the overall gradient long-term ($n \to \infty$) is highly dependent on this radius:

- **Vanishing Gradients:** If $\rho(J_f(x_k)) < 1$ for a sustained subsequence of states, the matrix product contracts exponentially. Consequently, $J_F(x) \to \mathbf{0}$ as $n \to \infty$, rendering early input states insulated from perturbations in the terminal objective.
- **Exploding Gradients:** If $\rho(J_f(x_k)) > 1$, the matrix product expands exponentially, causing numerical instability where $\|J_F(x)\| \to \infty$.

G.3.2 Connection to Dynamical Systems

In discrete dynamical systems, the long-term asymptotic behavior of these product matrices defines the structural stability of trajectories. The geometric average rate of expansion or contraction along an infinite trajectory yields the *Lyapunov Exponents* μ_i:

$$\mu_i = \lim_{n\to\infty} \frac{1}{n} \ln \sigma_i \left(J_F(x_0)\right) \tag{G.13}$$

where $\sigma_i(M)$ denotes the i-th singular value of the composite matrix M. A positive maximum Lyapunov exponent ($\mu_{\max} > 0$) serves as a formal indicator of chaotic dynamics and extreme sensitivity to initial conditions.

CHAPTER H

Principles of Deep Learning

TO BE DELETED?????

Modern deep learning works far better than classical statistical learning theory or traditional numerical analysis would lead one to expect. In this chapter we articulate two basic principles that, taken together, offer an explanation for the empirical success of deep architectures: (1) the sparse compositionality *of efficiently computable functions, and (2) the* genericity *required for stability and learnability. Sparse compositionality explains approximation, hierarchy, modularity, and transfer. Genericity explains stability and the existence of informative gradients during optimization. Together these principles provide a coherent framework for understanding the remarkable capabilities of deep networks and transformers.*

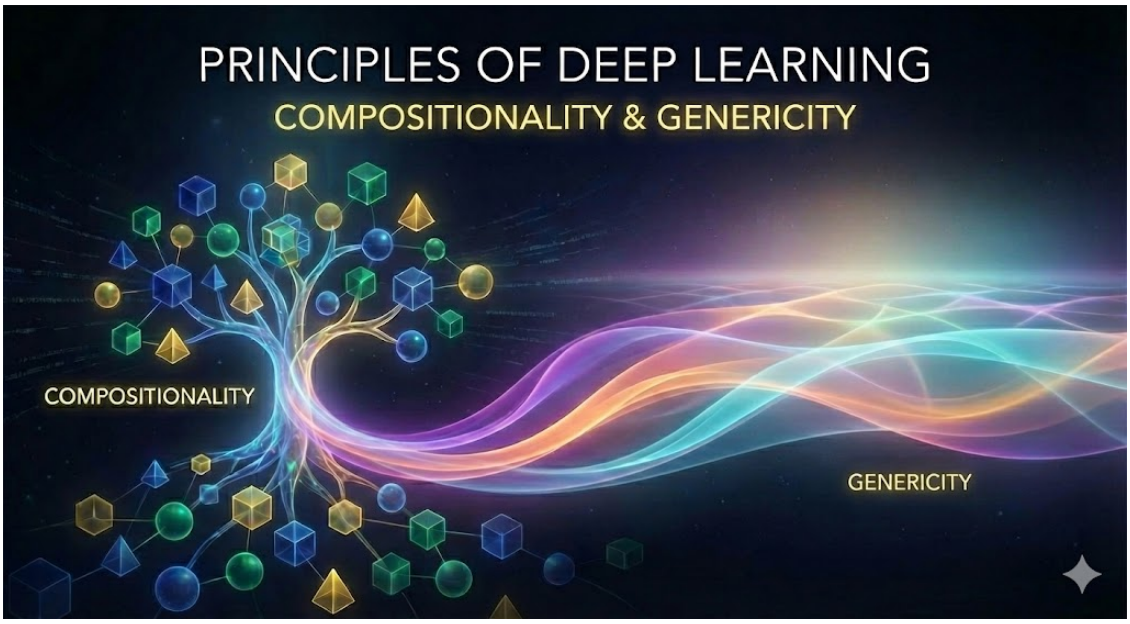


Figure H.1

H.1 Principle I: Sparse Compositionality

The property of *sparse compositionality* is formally implied by the property of *efficient Turing computability* [107]. Functions that are efficiently computable necessarily decompose into bounded-fan-in computational graphs (DAGs). This is a structural fact: a computation composed of many small local operations must be sparse in the sense that each intermediate value depends only on a few others [74].

Formally, if f is efficiently computable, then it admits a representation

$$f = f^{(L)} \circ f^{(L-1)} \circ \cdots \circ f^{(1)}$$

in which each $f^{(\ell)}$ acts on a small number of variables. Deep networks mirror this structure and thus avoid the curse of dimensionality when their architecture matches such sparse DAGs [89].

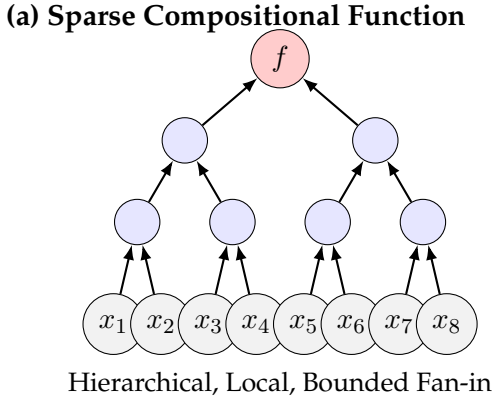


Figure H.2: Visualizing Sparse Compositionality. An efficiently computable function naturally decomposes into a hierarchical DAG where each node depends on a small number of inputs (bounded fan-in). This structure allows deep networks to approximate the function without incurring the curse of dimensionality.

In this section we describe several consequences of sparsity and compositionality.

H.1.1 Hierarchy follows from sparse compositionality

If a function admits a compositional DAG in which each node has bounded fanin, then the natural architecture for approximating it is a *hierarchical network*: each layer computes simple local features, which become inputs to features of increasing complexity.

Hierarchy is not an extra assumption; it is a direct corollary of the DAG.

Thus the empirical fact that good deep networks are deep is not surprising: depth is the natural way to reflect the compositional structure of the target.

H.1.2 Modularity and reuse of modules

Sparse compositionality also implies *modularity*:

- subgraphs (subfunctions) appear repeatedly in the DAG,
- these subgraphs compute intermediate concepts,

- the same module can be used at different places in the computation.

This modularity is reflected in architectures such as convolutional networks, residual networks, and attention blocks, where learned components are reused globally or across layers.

The existence of modules is not a property of the architecture alone; it is a property of the underlying function.

H.1.3 Transfer learning

If a function decomposes compositionally and modules appear in multiple places, then learning one task provides information useful for others.

$$\text{Module learned in Task A} \quad \Rightarrow \quad \text{usable immediately in Task B.}$$

Thus *transfer learning* is a natural consequence of sparse compositionality: reuse across tasks parallels reuse across positions in a single computation [10].

H.1.4 Interpretability from consistent compositionality

If the decomposition of f into modules is *consistent across data*, then the learned network will assign stable semantic meaning to its internal components. This yields a form of interpretability:

$$\text{same subfunction} \quad \Rightarrow \quad \text{same module/layer.}$$

Thus interpretability is a consequence of:

- the existence of a compositional structure in the function, and
- the ability of the model to discover and consistently reuse that structure.

H.1.5 Non-uniqueness of sparse decompositions

A crucial observation is that *sparse decompositions are not unique*. A given function f may admit many distinct DAG representations. In such cases, a general-purpose architecture—such as a transformer—has no reason to choose the same decomposition across different inputs or during training.

Consequences:

- Transformers do not consistently reuse modules: the same computation can be implemented in many ways. the same computation can be implemented in many ways.
- Without architectural constraints enforcing modularity, reuse, or locality, the model may realize different decompositions in different parts of the input space.
- This explains why transformers are powerful but not inherently interpretable or modular.

Transformers achieve excellent performance even without consistent module reuse because they can realize *any* decomposition compatible with their large associative memory structure.

H.2 Principle II: Genericity of Learnable Targets

Sparse compositionality describes the *structure* of computable functions. Genericity describes a different property: the requirement that learnable functions be *invariant* in their basic jet structure under shifts of the coordinates. Genericity implies that the function cannot rely on measure-zero cancellations or extremely fragile high-order interactions [100].

H.2.1 Genericity from invariance to the choice of the origin of the coordinates

Genericity excludes "non-generic" functions in which:

- all low-order terms vanish exactly,
- small perturbations of the input distribution flip the identity of the best predictor,

Genericity is the opposite situation: under realistic noise and bias, *every relevant variable leaves a low-degree footprint*, often linear, that persists across perturbations.

Thus:

$$\text{Stability} \quad \Rightarrow \quad \text{Genericity (low-degree footprints, no fine-tuning).}$$

H.2.2 Genericity ensures good gradients for optimization

Once genericity holds, gradient-based optimization is feasible:

- Linear footprints provide nonzero first-order information.
- Gradients do not vanish due to high-order cancellations.
- Different layers of a deep network receive consistent signals.

Thus genericity provides the missing piece for explaining why gradient descent can find good solutions in nonconvex architectures.

H.3 Two independent but complementary principles

Sparse compositionality and genericity arise from two different fundamental assumptions:

- Efficient Turing computability $\Rightarrow$ sparse compositionality.
- ERM-based learnability + stability $\Rightarrow$ genericity.

They are independent but complementary:

- Sparse compositionality explains approximation, hierarchy, modularity, transfer, interpretability.
- Genericity explains stability, robustness, and the existence of informative gradients.

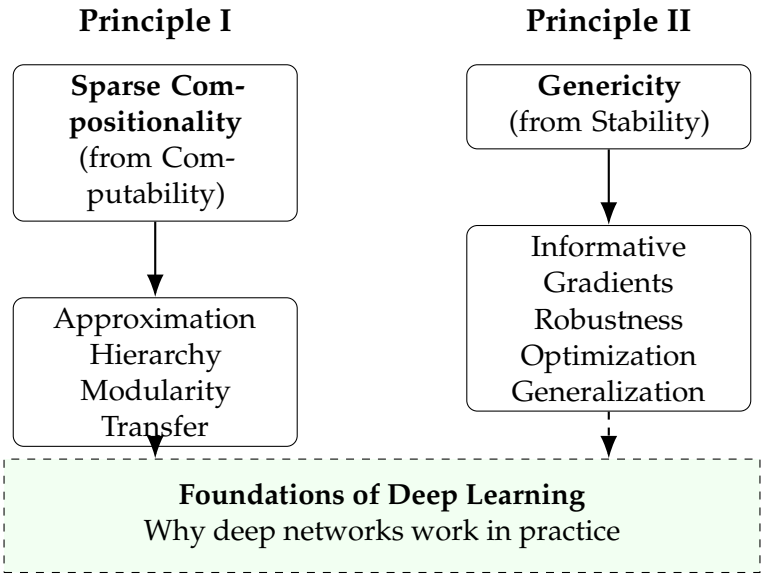


Figure H.3: The Two Pillars of Deep Learning Theory. Sparse Compositionality (derived from efficient computability) explains the architectural benefits, while Genericity (derived from stability) explains the feasibility of learning and optimization.

Together they provide a unified explanation for why deep learning works.

$$\text{Sparse compositionality} + \text{Genericity} \implies \begin{cases} \text{Efficient approximation} \\ \text{Generalization (stability)} \\ \text{Optimizable landscapes} \\ \text{Hierarchy, modularity, reuse} \\ \text{Transfer and interpretability} \end{cases}$$

H.4 Conclusion

The two principles articulated in this chapter—sparse compositionality and genericity—capture complementary aspects of deep learning:

- the *structure* of the functions that we want to learn, and
- the *regularity* that makes them learnable and optimizable.

Most of the puzzles of deep learning—its scalability, generalization, modularity, transfer, and surprising ease of optimization—follow naturally from these two principles once they are recognized as the structural and regularity constraints of the real functions we want to learn.

CHAPTER I

The Geometry of Overparameterized Systems: Krull and Bézout

I.1 Introduction

In the theoretical analysis of highly parameterized systems—such as polynomial approximations of overparameterized networks—we frequently encounter systems of equations where the number of parameters (or unknowns) d vastly exceeds the number of constraints n.

When $d \gg n$, the landscape of solutions ceases to be a discrete set of isolated points. Instead, it forms a continuous, high-dimensional manifold (more formally, an algebraic variety). Understanding the dimensionality, geometric complexity, and inherent degeneracy of this solution space requires classical tools from algebraic geometry, specifically Krull's Height Theorem and the Generalized Bézout Theorem. To ensure structural consistency and prevent solution manifolds from abruptly vanishing, this analysis must be conducted over an algebraically closed field, typically the complex numbers $\mathbb{C}$.

I.2 Dimensionality: Krull's Height Theorem

The first fundamental question regarding an overparameterized polynomial system is the size, or dimensionality, of its solution space. Intuitively, each independent equation imposes one constraint, removing one degree of freedom from the ambient d-dimensional space. This intuition is formalized algebraically by Krull's Theorem.

Theorem 36 (Krull's Height Theorem - Geometric Formulation)**.** *Let $\mathbb{C}[x_1, \ldots, x_d]$ be the polynomial ring in d variables. If an ideal I is generated by n polynomials $f_1, \ldots, f_n$, then every minimal prime ideal over I has height at most n.*

Geometrically, the "height" of the ideal corresponds to the codimension of the solution variety. Therefore, Krull's Theorem guarantees that the solution space $X \subset \mathbb{C}^d$ defined by $f_1(x) = 0, \ldots, f_n(x) = 0$ satisfies:

$$\dim(X) \geq d - n \tag{I.1}$$

If the polynomials are chosen generically (lacking algebraic dependencies), the equations act as maximal constraints, and the dimension of the solution manifold is exactly $d - n$. The system is

inherently degenerate in the sense that the solutions form an extended continuous surface rather than isolated points.

I.3 Geometric Complexity: Generalized Bézout's Theorem

While Krull's Theorem bounds the dimension of the solution manifold, it does not describe its topological or geometric complexity. For square systems ($d = n$), the classical Bézout Theorem bounds the number of isolated solution points. In the heavily underdetermined regime ($d \gg n$), Bézout's Theorem generalizes to bound the *degree* of the $(d-n)$-dimensional solution manifold.

The degree of a $(d-n)$-dimensional variety X geometrically represents the number of intersection points between X and a generic (n)-dimensional linear subspace.

Theorem 37 (Generalized Bézout Theorem)**.** *Let $f_1, \ldots, f_n$ be polynomials in d variables defining a variety X of dimension $d-n$. Then the degree of X is bounded by the product of the degrees of the defining polynomials:*

$$\deg(X) \leq \prod_{i=1}^{n} \deg(f_i) \tag{I.2}$$

This theorem provides a strict upper limit on how highly folded, oscillating, or "complex" the overparameterized solution manifold can be, governed strictly by the algebraic degrees of the input equations.

I.4 Maximal Degeneracy at the Origin

A distinct topological phenomenon occurs when we restrict the system such that none of the polynomials f_i contain linear or constant terms. In this scenario, every term has a degree of 2 or higher.

Remark 3 (The Origin as a Singular Point)**.** *If $f_i(0) = 0$ and $\nabla f_i(0) = \mathbf{0}$ for all $i = 1, \ldots, n$, the origin $\mathbf{0} \in \mathbb{C}^d$ is a guaranteed solution. Furthermore, the Jacobian matrix $J \in \mathbb{C}^{n \times d}$ evaluates entirely to zero at this point.*

Because the Jacobian loses full rank (evaluating to rank 0), the origin represents a severe geometric singularity on the $(d-n)$-dimensional manifold. In optimization contexts, where one might minimize a squared-loss objective $L(x) = \sum_{i=1}^{n} f_i(x)^2$, the Hessian $H = 2J^T J$ also evaluates to the zero matrix at the origin. Thus, the solution manifold is not only continuous, but highly singular and maximally flat at $\mathbf{0}$, representing a perfectly degenerate critical point spanning all d directions.

I.5 Real vs. Complex Solution Varieties ($\mathbb{R}^d$ vs. $\mathbb{C}^d$)

A critical technical distinction in applying algebraic-geometric tools to machine learning arises from the choice of field. Standard algebraic geometry results—including Krull's Height Theorem and Bézout's Theorem—hold unconditionally over algebraically closed fields such as $\mathbb{C}$. Modern neural networks, however, are trained over the real field $\mathbb{R}$.

I.5.1 Dimension Drop in Real Loci

When parameters are restricted to $W \in \mathbb{R}^d$, the real solution set $\mathcal{M}_{\mathbb{R}} = \mathcal{M} \cap \mathbb{R}^d$ forms a real semi-algebraic set. Unlike complex varieties, real loci do not automatically inherit the complex algebraic dimension $d - n$:

$$\dim_{\mathbb{R}}(\mathcal{M}_{\mathbb{R}}) \leq \dim_{\mathbb{C}}(\mathcal{M}) \tag{I.3}$$

Underdetermined real polynomial systems can degenerate into empty sets or isolated points. For example, the equation $\sum_{i=1}^{d} W_i^2 = 0$ defines a continuous hypersurface of dimension $d-1$ over $\mathbb{C}^d$, but collapses to the isolated origin $\mathbf{0}$ of dimension 0 over $\mathbb{R}^d$.

I.5.2 Overparameterization and Smooth Real Points

Despite this theoretical risk of dimension drop, overparameterization ($d \gg n$) typically prevents real degeneration in neural networks.

By the Implicit Function Theorem, if there exists at least one real point $W^* \in \mathbb{R}^d$ where the Jacobian matrix $J(W^*) \in \mathbb{R}^{n \times d}$ achieves full rank n, then $\mathcal{M}_{\mathbb{R}}$ is locally a smooth real manifold of dimension exactly $d-n$ around W^*. Consequently, for generic data and targets where interpolation is achievable, the real zero-loss landscape preserves the expected dimension:

$$\dim_{\mathbb{R}}(\mathcal{M}_{\mathbb{R}}) = d - n \tag{I.4}$$

I.6 Local Geometry and the Neural Tangent Kernel

The geometric structure of $\mathcal{M}$ near an interpolating point W^* dictates local optimization dynamics, such as the convergence rate and implicit bias of gradient descent.

I.6.1 Tangent Space and Flat Directions

At any smooth point $W^* \in \mathcal{M}$, the tangent space $T_{W^*}\mathcal{M}$ is defined by the kernel of the Jacobian:

$$T_{W^*}\mathcal{M} = \ker\left(J(W^*)\right) = \left\{ v \in \mathbb{C}^d \;\middle|\; J(W^*)v = \mathbf{0} \right\} \tag{I.5}$$

By the Rank-Nullity Theorem, $\dim(T_{W^*}\mathcal{M}) = d - \operatorname{rank}(J(W^*))$. When $J(W^*)$ has full rank n, the tangent space forms a $(d-n)$-dimensional linear subspace of locally flat, loss-invariant directions.

I.6.2 NTK Rank Defect and Singular Loci

The local manifold structure is intrinsically linked to the Neural Tangent Kernel (NTK) matrix $\Theta(W) \in \mathbb{C}^{n \times n}$:

$$\Theta(W) = J(W)J(W)^T \tag{I.6}$$

We classify points on the zero-loss landscape based on the spectrum of $\Theta(W)$: 1 * **Regular Interpolating Points ($\operatorname{rank}(\Theta(W)) = n$):** The Jacobian has full row rank. The manifold $\mathcal{M}$ is locally smooth, and the tangent space has dimension exactly $d - n$. * **Singular Subvarieties ($\operatorname{rank}(\Theta(W)) < n$):** The set of parameters $\mathcal{S} = \{W \in \mathbb{C}^d \mid \det(\Theta(W)) = 0\}$ defines a singular algebraic subvariety. At points $W \in \mathcal{S} \cap \mathcal{M}$, the tangent space dimension jumps above $d - n$, signaling self-intersections, branching sheets, or cusps in the zero-loss manifold.

I.7 Summary of Landscape Regimes

The parameter landscape of an overparameterized network with polynomial activations decomposes into three main geometric regimes:

Regime	Local Dimension	Geometric Property
Origin ($W = \mathbf{0}$)	d	Maximally singular; zero Jacobian $J = \mathbf{0}$ and zero Hessian $H = \mathbf{0}$; flat in all d directions.
Singular Locus ($\mathcal{S} \cap \mathcal{M}$)	$> d - n$	Non-smooth intersections of manifold sheets; NTK loses rank ($\det(\Theta) = 0$).
Generic Interpolation ($\mathcal{M} \setminus \mathcal{S}$)	$d - n$	Smooth manifold; bounded degree $\leq r^{Ln}$; $(d - n)$ zero-curvature tangent directions.

CHAPTER J

Unsupervised Ensembling of Verifiers

Modern generative systems can often produce several candidate solutions to the same problem, while separate models or procedures can score the quality of those candidates. This raises a natural question: can several imperfect verifiers be combined into a reliable judge when no ground-truth correctness labels are available?

The answer can be positive, but only under structural assumptions. If the errors of multiple verifiers are sufficiently diverse, their patterns of agreement and disagreement contain information about their latent accuracies. Fully Unsupervised Score Ensembling (FUSE) [61] exploits this principle using low-order moments of the verifier outputs.

This appendix develops the basic mathematics. Its main conceptual lesson is simple:

$$\boxed{\textit{zero labels} \neq \textit{zero information}}$$

but also

$$\boxed{\textit{zero labels} \neq \textit{zero assumptions}.}$$

Unsupervised verification is possible only when the observable agreement structure of the ensemble contains a detectable footprint of latent correctness.

J.1 The Problem

Suppose a generative system produces candidate responses

$$X_1, \ldots, X_N.$$

For each response X_n, let

$$Y_n \in \{-1, +1\}$$

denote its unknown ground-truth correctness:

$$Y_n = +1 \quad \Longleftrightarrow \quad X_n \text{ is correct.}$$

Assume that m imperfect verifiers evaluate every response.
For simplicity, begin with binary verifier outputs

$$V_i(X_n) \in \{-1, +1\}, \qquad i = 1, \ldots, m.$$

The learner observes the matrix

$$V = [V_i(X_n)]_{n,i},$$

but never observes the labels

$$Y_1, \ldots, Y_N.$$

The goal is to infer

$$P(Y_n = +1 \mid V_1(X_n), \ldots, V_m(X_n))$$

using only the verifier outputs.

J.2 Verifier Quality

Let

$$\pi = P(Y = +1)$$

denote the class prior.
For verifier i, define its sensitivity

$$\alpha_i = P(V_i = +1 \mid Y = +1) \tag{J.1}$$

and specificity

$$\beta_i = P(V_i = -1 \mid Y = -1). \tag{J.2}$$

A useful signed measure of verifier quality is

$$a_i = \alpha_i + \beta_i - 1. \tag{J.3}$$

Thus

$$a_i > 0$$

means that verifier i is better than random in the appropriate orientation,

$$a_i = 0$$

corresponds to an uninformative verifier, and

$$a_i < 0$$

corresponds to a systematically reversed verifier.
Define the conditional means

$$m_i^+ = E[V_i \mid Y = +1] = 2\alpha_i - 1, \tag{J.4}$$
$$m_i^- = E[V_i \mid Y = -1] = 1 - 2\beta_i. \tag{J.5}$$

Their difference is

$$\Delta_i = m_i^+ - m_i^- = 2a_i. \tag{J.6}$$

The observable marginal mean is

$$\mu_i = E[V_i] = \pi m_i^+ + (1-\pi)m_i^-. \tag{J.7}$$

J.3 Triplet Conditional Independence

The central structural assumption is that distinct verifiers make independent errors once the true label is known.

For three distinct verifiers i, j, k, assume

$$P(V_i, V_j, V_k \mid Y) = P(V_i \mid Y)P(V_j \mid Y)P(V_k \mid Y). \tag{J.8}$$

We refer to this as *Triplet Conditional Independence* (TCI).

TCI does not mean that the verifier outputs are marginally independent.

Indeed, they will ordinarily be correlated because they are responding to the same latent variable Y.

It says only that, after conditioning on correctness, their remaining errors are independent.

This distinction is the key to the method.

J.4 Observable Moment Factorization

Define the off-diagonal covariance

$$R_{ij} = E[(V_i - \mu_i)(V_j - \mu_j)], \qquad i \neq j, \tag{J.9}$$

and the third central moment

$$T_{ijk} = E[(V_i - \mu_i)(V_j - \mu_j)(V_k - \mu_k)], \qquad i, j, k \text{ distinct}. \tag{J.10}$$

Proposition 35 (Moment factorization under TCI)**.** *Under Triplet Conditional Independence,*

$$R_{ij} = \pi(1-\pi)\Delta_i\Delta_j = 4\pi(1-\pi)a_i a_j, \tag{J.11}$$

and

$$T_{ijk} = \pi(1-\pi)(1-2\pi)\Delta_i\Delta_j\Delta_k = 8\pi(1-\pi)(1-2\pi)a_i a_j a_k. \tag{J.12}$$

Proof. Conditional independence gives, for $i \neq j$,

$$E[V_i V_j \mid Y] = E[V_i \mid Y]E[V_j \mid Y].$$

The law of total covariance therefore yields

$$\mathrm{Cov}(V_i, V_j) = \mathrm{Cov}\left(E[V_i \mid Y], E[V_j \mid Y]\right).$$

Since Y is binary,

$$E[V_i \mid Y = +1] - E[V_i \mid Y = -1] = \Delta_i,$$

and hence

$$R_{ij} = \pi(1-\pi)\Delta_i\Delta_j.$$

Using

$$\Delta_i = 2a_i$$

gives Equation (J.11).
For the third central moment, note that

$$E[V_i - \mu_i \mid Y = +1] = (1-\pi)\Delta_i$$

and

$$E[V_i - \mu_i \mid Y = -1] = -\pi\Delta_i.$$

Conditional independence then gives

$$T_{ijk} = \pi(1-\pi)^3\Delta_i\Delta_j\Delta_k - (1-\pi)\pi^3\Delta_i\Delta_j\Delta_k \tag{J.13}$$
$$= \pi(1-\pi)(1-2\pi)\Delta_i\Delta_j\Delta_k. \tag{J.14}$$

Substituting $\Delta_i = 2a_i$ proves Equation (J.12). □

The important structure is that both tensors factor into products of one-dimensional verifier parameters.
Schematically,

$$\boxed{R_{ij} \propto a_i a_j, \qquad T_{ijk} \propto a_i a_j a_k.}$$

Thus the observable moments have a rank-one structure induced by the single latent variable Y.

J.5 Why Second- and Third-Order Moments Are Enough

For distinct i, j, k,

$$\frac{T_{ijk}}{\sqrt{|R_{ij}R_{ik}R_{jk}|}} = \frac{1-2\pi}{\sqrt{\pi(1-\pi)}}\,\operatorname{sign}(a_i a_j a_k). \tag{J.15}$$

If the verifiers are oriented so that

$$a_i > 0,$$

the right-hand side is independent of the verifier triplet.
Thus, in the population limit, every valid triplet gives the same observable ratio.
This provides information about the latent class prior π.
Once π is known, pairwise covariances determine the relative verifier skills. For example,

$$a_i^2 = \frac{R_{ij}R_{ik}}{4\pi(1-\pi)R_{jk}}, \tag{J.16}$$

for a nondegenerate triplet.

The marginal means then recover sensitivity and specificity.

From Equations (J.3) and (J.7),

$$\alpha_i = \frac{1+\mu_i}{2} + (1-\pi)a_i, \tag{J.17}$$

$$\beta_i = \frac{1-\mu_i}{2} + \pi a_i. \tag{J.18}$$

Thus the unlabeled joint distribution of verifier outputs can, under the model assumptions, identify the quality of the individual verifiers.

J.6 The Unavoidable Orientation Ambiguity

There is one fundamental ambiguity.

If the latent labels are reversed,

$$Y \longrightarrow -Y,$$

then an ensemble of good verifiers can be reinterpreted as an ensemble of systematically wrong verifiers.

The observable distribution of verifier outputs alone cannot decide which latent state should be called "correct."

Thus unsupervised identification is only possible up to a global label permutation unless an orientation assumption is supplied.

A typical assumption is that most verifiers are better than random:

$$\sum_i a_i > 0, \tag{J.19}$$

or, more strongly, that a designated subset satisfies

$$a_i > 0.$$

This illustrates an important general point:

unlabeled estimation does not eliminate assumptions; it replaces labels by structural assumptions.

J.7 When Conditional Independence Fails

For real verifier ensembles, exact TCI is unlikely.

Different verifiers may share

training data, architectures, prompting strategies, systematic misconceptions.

Their errors can therefore remain correlated even after conditioning on the true label.
Under exact TCI, Equation (J.15) predicts the same ratio for every triplet.
This gives an observable diagnostic of TCI violation.
Let

$$\rho_{ijk} = \frac{\hat{T}_{ijk}}{\sqrt{|\hat{R}_{ij}\hat{R}_{ik}\hat{R}_{jk}|}}. \tag{J.20}$$

For suitably oriented nondegenerate triplets, define

$$\tau(V) = \mathrm{Var}_{i,j,k}\left[\rho_{ijk}\right]. \tag{J.21}$$

Under the ideal population model,

$$\tau(V) = 0.$$

Large empirical values indicate disagreement with the TCI factorization.

J.8 FUSE: Transforming Scores to Improve the Moment Model

Real verifiers often produce continuous scores

$$S_i(X) \in \mathbb{R}$$

rather than binary decisions.
FUSE [61] exploits the fact that these scores can be transformed before applying the moment estimator.
Let

$$V_i = g_i(S_i)$$

for a family of admissible transformations g_i, such as thresholding or other monotone transformations.
The transformations are chosen to make the observable moments more consistent with the TCI factorization, schematically by seeking

$$g^\star = \arg\min_g \tau\left(V^{(g)}\right), \tag{J.22}$$

subject to suitable nondegeneracy constraints.
The important distinction is:

minimizing τ does not prove that TCI is true.

It finds a representation of the verifier scores whose observable low-order moments are *more consistent* with the conditional independence model.
Thus there are two different levels of claim:

Population theorem:	identifiability under TCI and orientation assumptions,
Practical FUSE procedure:	transform the scores to reduce an observable measure of TCI violation before appl

J.9 From Verifier Reliabilities to Pseudo-Labels

Suppose the parameters

$$\pi, \qquad \alpha_i, \qquad \beta_i$$

have been estimated.

For a triplet of approximately conditionally independent verifiers i, j, k, Bayes' rule gives

$$P\left(Y=+1 \mid V_i, V_j, V_k\right)=\frac{\pi \prod_{m\in\{i,j,k\}} P(V_m \mid Y=+1)}{\sum_{y\in\{-1,+1\}} P(Y=y) \prod_{m\in\{i,j,k\}} P(V_m \mid Y=y)}. \tag{J.23}$$

These posterior probabilities provide soft pseudo-labels

$$\hat{p}_n \approx P(Y_n=+1 \mid V(X_n)).$$

Multiple valid verifier triplets may be aggregated to obtain a more stable estimate.

The resulting pseudo-labels can then supervise a flexible ensemble model

$$q_\theta\left(Y=+1 \mid S_1, \ldots, S_m\right).$$

For example, one may minimize the soft-label cross-entropy

$$\mathcal{L}(\theta)=-\frac{1}{N} \sum_{n=1}^{N}\left[\hat{p}_n \log q_\theta(Y_n=+1 \mid S_n)+\left(1-\hat{p}_n\right) \log q_\theta(Y_n=-1 \mid S_n)\right]. \tag{J.24}$$

The final ensemble can then rank or select candidate responses according to its estimated probability of correctness.

J.10 What Makes Unsupervised Verification Possible?

At first sight, recovering correctness without any labeled examples may appear paradoxical.

The resolution is that the labels are not reconstructed from nothing.

They leave statistical footprints in the pattern of verifier agreement.

Under TCI,

$$Y$$

induces the observable covariance structure

$$R_{ij}=4\pi(1-\pi)a_i a_j.$$

Thus two competent verifiers tend to agree not merely because they are similar, but because both respond to the same hidden variable.

The third-order moment provides additional information about class imbalance and orientation.

The hidden variable is therefore indirectly visible through low-order statistics.

Schematically,

latent correctness $\rightarrow$ correlated verifier outputs $\rightarrow$ observable moments $\rightarrow$ estimated verifier quality.

This is the source of identifiability.

J.11 Failure Modes

The method also clarifies when unsupervised verification cannot be expected to work.

J.11.1 Common-mode errors

Suppose all verifiers make the same mistake on the same examples.

Then agreement may be extremely high even when the ensemble is wrong.

Consensus alone does not imply truth.

If

$$V_1 = \cdots = V_m$$

almost surely, the ensemble provides little additional information beyond a single verifier.

J.11.2 Weak verifiers

If

$$a_i \approx 0$$

for many verifiers, then

$$R_{ij} \approx 0$$

and the observable footprint of correctness becomes weak.

Finite-sample estimation then becomes difficult.

J.11.3 Strong conditional dependence

If verifier errors remain correlated after conditioning on Y, the rank-one factorizations

$$R_{ij} \propto a_i a_j$$

and

$$T_{ijk} \propto a_i a_j a_k$$

are violated.

A small value of the empirical TCI statistic can provide evidence in favor of the approximation, but does not establish its truth.

J.11.4 Insufficient diversity

Adding many nearly identical verifiers does not provide the same information as adding genuinely diverse ones.

The relevant resource is not simply ensemble size,

$$m,$$

but diversity of error structure.

J.11.5 Finite-sample noise

The moments

$$R_{ij} \quad \text{and} \quad T_{ijk}$$

must themselves be estimated from finite data.

Third-order moments are particularly noisy.

Population identifiability therefore does not imply accurate estimation from a small number of responses.

J.11.6 Orientation failure

Without some assumption that fixes which latent class corresponds to correctness, the global label orientation remains ambiguous.

No unsupervised method can remove this symmetry using the verifier outputs alone.

J.12 Zero Labels Do Not Mean Truth from Consensus

The preceding failure modes make an important conceptual point.

FUSE does not establish the general principle

$$\text{many agreeing models} \Longrightarrow \text{truth}.$$

What it exploits is a much more specific statistical model:

diverse imperfect verifiers + structured conditional independence + nonzero individual skill → identifiable late

If the structure disappears, so does the guarantee.

Thus

agreement is informative only when the mechanism generating agreement is sufficiently constrained.

J.13 Relation to Genericity

The construction provides a useful small-scale example of the representation-versus-recovery distinction developed elsewhere in this book.

The target variable

$$Y$$

is never observed directly.

Nevertheless, under TCI and nonzero verifier skill, it leaves detectable low-order footprints:

$$R_{ij} \neq 0, \qquad T_{ijk} \neq 0$$

generically away from degenerate parameter choices.

These moments make recovery possible.

In this sense, the FUSE setting illustrates the genericity principle:

a hidden variable is learnable only if it leaves an observable statistical footprint in the data available to the learn

If every verifier is uninformative,

$$a_i = 0,$$

the footprints vanish.

If all errors share an unconstrained common cause, the footprints can become ambiguous.

Thus the assumptions responsible for identifiability play the same role that genericity plays elsewhere in the book: they expose the hidden structure to a feasible estimator.

J.14 Relation to Generative AI

Verifier ensembling is relevant whenever generation is easier to scale than reliable evaluation.

A generative model may produce candidates

$$X^{(1)}, \ldots, X^{(K)},$$

while several heterogeneous procedures evaluate them:

$$V_1, \ldots, V_m.$$

These verifiers might use different models, prompts, tools, tests, or specialized checks.

If their errors contain sufficiently independent information, an unsupervised ensemble can potentially provide a stronger selection signal than any individual verifier.

The important architectural principle is therefore not

generation requires a perfect external oracle,

but rather the weaker possibility

several imperfect and diverse evaluation signals may jointly reveal a useful estimate of latent correctness.

This does not eliminate the need for external validation.

It provides a mechanism by which the cost of such validation may be reduced when the structural assumptions are approximately satisfied.

J.15 Conclusion

Unsupervised verifier ensembling asks a sharply defined question:

Can latent correctness be inferred from agreement among imperfect judges without observing any correctness la

Under Triplet Conditional Independence and suitable orientation and nondegeneracy assumptions, the answer is yes at the population level.

Pairwise and third-order moments factorize as

$$R_{ij} = 4\pi(1-\pi)a_i a_j$$

and

$$T_{ijk} = 8\pi(1-\pi)(1-2\pi)a_i a_j a_k,$$

allowing the latent class balance and individual verifier reliabilities to be recovered from observable statistics.

FUSE extends this idea to realistic verifier scores by searching for transformations whose low-order moments more nearly satisfy the conditional-independence structure before constructing pseudo-labels and an ensemble predictor.

The broader lesson is not that truth can be obtained from consensus alone.

It is that latent truth may be statistically recoverable when it leaves structured, detectable traces across multiple imperfect observers.

In the language used throughout this book,

no supervision is required only when the observable data already contain enough structure to substitute for it.

CHAPTER K

Sparse Structure Does Not Eliminate NP-Hardness: Worst-Case and Distributional Tractability

In his Nobel lecture, Demis Hassabis suggested an important empirical possibility: problems classified as computationally hard in the worst case may often be substantially easier on the instances that arise in practice.

This observation should not be interpreted as claiming that NP-hard problems are generally polynomial-time solvable, or that hard instances occupy only negligible regions of instance space.

A more precise hypothesis is:

many computationally hard problem classes may become tractable to useful accuracy on the structured instance distributio

This appendix examines that possibility and, at the same time, states an important limitation of the sparse-compositionality framework developed in this book.

Sparse compositional structure can make a function efficient to evaluate*. It does not, by itself, make the corresponding global search or optimization problem easy.*

K.1 The Fundamental Distinction: Evaluation versus Search

Consider a Boolean formula

$$F(x_1, \ldots, x_n) = \bigwedge_{j=1}^{m} C_j(x_{j_1}, x_{j_2}, x_{j_3}),$$

where each C_j is a clause involving at most three variables.

The function

$$x \longmapsto F(x)$$

is compositionally sparse in an extremely strong sense.

Each local constituent has bounded arity:

$$C_j : \{0,1\}^3 \to \{0,1\},$$

and the final result is obtained by combining the local clause values.

Evaluating $F(x)$ for a given assignment x is therefore efficient:

$$T_{\text{eval}} = O(m).$$

Yet the search problem

$$\text{find } x \text{ such that } F(x) = 1$$

is the canonical NP-complete problem 3-SAT.

Thus

$$\boxed{\text{bounded-arity compositional structure} \not\Longrightarrow \text{tractable global search.}}$$

This example is important for the theory developed throughout this book.

Sparse compositionality provides a structural explanation for why a map can be represented and evaluated without an exponential dependence on ambient dimension.

It does not imply that one can efficiently invert, optimize, or satisfy that map.

More generally,

$$\boxed{\text{efficient evaluation of } f(x) \not\Longrightarrow \text{efficient computation of } \arg\min_x f(x).}$$

The distinction is analogous to the separation made earlier between representation and recovery.

A compact representation of a solution space does not automatically provide an efficient algorithm for locating a desired point within it.

K.2 Worst-Case Complexity

Let

$$T(A, x)$$

denote the running time of algorithm A on instance x.

Worst-case complexity considers

$$T_{\text{worst}}(n) = \max_{|x|=n} T(A, x). \tag{K.1}$$

If an NP-complete problem has a deterministic algorithm satisfying

$$T_{\text{worst}}(n) = \text{poly}(n),$$

then

$$\text{P} = \text{NP}.$$

Thus, assuming

$$\text{P} \neq \text{NP},$$

there is no polynomial-time algorithm that correctly solves *every* instance of every NP-complete problem.

This statement is about an entire problem class.

It does not imply that every instance is difficult.

Nor does it imply that a particular application generates difficult instances frequently.

This distinction is essential.

The practical performance of an algorithm deployed in the world depends not only on the mathematical problem class but also on the distribution

$$\mathcal{D}_{\text{deploy}}$$

from which its instances are drawn.

K.3 Distributional Tractability

Suppose instances are repeatedly sampled from a deployment distribution

$$x \sim \mathcal{D}_{\text{deploy}}.$$

Then a relevant quantity may be

$$T_{\mathcal{D}}(A, n) = \mathbb{E}_{x \sim \mathcal{D}_{\text{deploy}}} \left[T(A, x) \mid |x| = n \right], \tag{K.2}$$

rather than the worst-case quantity in Equation (K.1).

It is entirely consistent to have

$$T_{\text{worst}}(n)$$

grow exponentially while

$$T_{\mathcal{D}}(A, n)$$

remains modest over the distributions encountered in practice.

The important point is not that the NP-hard problem has disappeared.

It is that

the deployment distribution may occupy a highly restricted region of the full combinatorial instance space.

A specialized solver can exploit the regularities of that region without solving the worst-case problem.

K.4 A More Precise Practical Hypothesis

The hypothesis suggested by practical experience can therefore be stated as follows.

> **Distributional Tractability Hypothesis.** For many important NP-hard problem classes and practical deployment environments, the distribution of encountered instances possesses additional structure that permits efficient exact solution, efficient approximation, or reliable heuristic solution on most of the probability mass of that deployment distribution.

This statement is deliberately weaker than

"NP-hard problems are usually easy."

It makes tractability relative to

$$\boxed{\text{problem class} + \text{instance distribution} + \text{required solution quality.}}$$

All three matter.

K.5 Exact Solution versus Useful Approximation

Another source of apparent tension between complexity theory and practice is that practical applications often do not require exact global optimality.

Suppose the mathematical objective is

$$f^\star(x) = \min_{y \in \mathcal{Y}(x)} C(x, y).$$

Worst-case complexity may concern finding the exact minimizer

$$y^\star.$$

An application may instead require

$$C(x, \hat{y}) \leq (1 + \varepsilon) C(x, y^\star),$$

or merely a solution satisfying a domain-specific performance threshold.

These are different computational problems.

For some NP-hard optimization problems, useful approximation is polynomial-time even though exact optimization is not known to be.

For others, approximation itself remains hard.

Thus practical success may arise from relaxing

$$\boxed{\text{exact optimum}}$$

to

$$\boxed{\text{sufficiently good solution.}}$$

K.6 Smoothed Analysis

One formal attempt to bridge worst-case and practical behavior is *smoothed analysis*.

Rather than evaluating an algorithm only on an adversarial instance x, one first perturbs that instance slightly.

Schematically,

$$\tilde{x} = x + \xi,$$

where

$$\xi \sim \mathcal{N}(0, \sigma^2 I).$$

The smoothed complexity is then of the form

$$C_{\text{smooth}}(n, \sigma) = \max_{x \in \mathcal{X}_n} \mathbb{E}_\xi \left[T(A, x + \xi) \right]. \tag{K.3}$$

This framework was introduced by Spielman and Teng to explain, among other examples, why the simplex algorithm can perform extremely well in practice despite exponential worst-case behavior.

Smoothed analysis captures an important possibility:

some pathological worst-case configurations are unstable to small perturbations.

If the bad configuration occupies a narrow region of instance space, small random perturbations may move the problem into a regime where the algorithm behaves efficiently.

But this mechanism is not universal.

K.7 What Smoothed Analysis Does Not Show

One should not infer from successful smoothed analyses that

random perturbation generally destroys computational hardness.

Smoothed polynomiality must be proved for a particular problem, algorithm, and perturbation model.

Some computationally hard structures are robust.

Random instances can themselves be difficult.

And in discrete problems there may be no natural infinitesimal perturbation corresponding to Gaussian noise in a continuous parameter space.

Thus

smoothed tractability is one mechanism for practical success, not a general theorem about NP-hardness.

K.8 Discrete Problems and Robust Hardness

The limitation becomes particularly clear in discrete combinatorial problems.

Consider random K-SAT.

The clause-to-variable ratio

$$\alpha = \frac{m}{n}$$

controls the qualitative character of random formulas.

For random 3-SAT, a sharp satisfiability transition occurs near a critical ratio around

$$\alpha_c \approx 4.27.$$

The relation between this phase transition and algorithmic difficulty is subtle, but the example makes an important conceptual point:

hardness need not correspond to one isolated adversarial input.

Large families of random instances can exhibit difficult search structure.

Therefore practical tractability cannot generally be explained by the claim that the real world simply adds noise to a fragile worst-case instance.

K.9 Where Practical Tractability Can Come From

If practical instance distributions are easier than the worst case, some additional structure must be responsible.

Several mechanisms are possible.

K.9.1 Low treewidth or related graph structure

Many combinatorial problems become tractable when their interaction graphs have bounded treewidth.

A problem that is NP-hard on arbitrary graphs may admit dynamic programming algorithms of the schematic form

$$T(n, w) = n \, \exp(O(w)),$$

where w is treewidth.

If

$$w = O(1),$$

the algorithm becomes polynomial in n.

Thus a restricted deployment distribution concentrated on low-treewidth instances can be easy even though the unrestricted problem class is NP-hard.

K.9.2 Small backdoor sets

Some instances contain a small collection of variables such that, once those variables are assigned, the remaining problem belongs to a tractable class.

If a backdoor set has size

$$b,$$

one may search over

$$2^b$$

assignments rather than

$$2^n.$$

When

$$b \ll n,$$

this can change the practical character of the problem dramatically.

K.9.3 Repeated motifs

Deployment environments frequently reuse the same components and constraint patterns.

A solver may exploit repeated local motifs rather than treating every instance as entirely novel.

K.9.4 Restricted effective dimension

Although an input may have nominal dimension

$$n,$$

the instances generated by a particular environment may vary along far fewer effective degrees of freedom.

A specialized algorithm can exploit this restriction.

K.9.5 Approximation tolerance

The application may permit near-optimal rather than exact solutions.

This can transform an intractable optimization requirement into a useful polynomial-time approximation problem.

K.9.6 Stable heuristics

A heuristic need not solve every mathematical instance.

It need only perform reliably over

$$\mathcal{D}_{\text{deploy}}.$$

This can be enough for practical systems, especially when failures are detectable.

K.9.7 Verification and fallback

A particularly useful architecture is

$$\boxed{\text{fast specialized solver} + \text{verification} + \text{fallback}.}$$

The fast solver handles the common structured cases.

A verifier detects failure or low confidence.

Exceptional instances are then passed to a slower algorithm, human operator, or more expensive search procedure.

This design can produce high practical reliability without requiring the fast algorithm to solve every instance efficiently.

K.10 Learning from Deployment Traces

The preceding perspective suggests a natural role for machine learning.

Suppose an application repeatedly generates

$$x_1, x_2, \ldots \sim \mathcal{D}_{\text{deploy}}.$$

Instead of searching for one universal algorithm that performs well on all mathematically possible instances, one can learn from execution traces of the actual environment.

Let

$$\tau_i = (x_i, \text{solver decisions}, \text{search trajectory}, \text{outcome})$$

denote an execution trace.

A learned policy can exploit regularities in

$$\{\tau_i\}$$

to predict

branching decisions, variable orderings, decompositions, promising subproblems, algorithm se

The relevant learning problem is therefore not

$$\boxed{\text{solve the NP-hard problem universally,}}$$

but

$$\boxed{\text{learn a good algorithmic policy for the observed distribution.}}$$

This distinction is closely related to the distinction between a universal computational problem and a learned world model specialized to a particular environment.

K.11 SAT Solvers as an Example

Modern SAT solvers illustrate this philosophy.

Conflict-Driven Clause Learning, branching heuristics, restart policies, learned clauses, preprocessing, and decomposition methods can dramatically reduce practical search.

Their success should not be interpreted as proving that encountered SAT instances necessarily possess low treewidth or small backdoor sets.

Rather, these structural notions are among several possible explanations for why particular families of instances admit effective search.

The important point is

$$\boxed{\text{the solver dynamically discovers structure that allows large parts of the nominal search space to be discarded.}}$$

A SAT instance may formally contain

$$2^n$$

possible assignments.

A successful solver explores only a tiny adaptive subset.

K.12 Sparse Compositionality Helps—but Is Not Enough

It is tempting to attribute practical tractability entirely to sparse compositionality.

This would be incorrect.

As 3-SAT demonstrates, bounded local interactions can coexist with NP-complete global search.

The issue is that local constraints can interact globally through shared variables.

A problem may therefore satisfy

$$\boxed{\text{local arity} = O(1)}$$

while possessing a global constraint graph whose effective search complexity grows rapidly with problem size.

Sparse compositionality remains useful because it limits the complexity of local computation.

But an additional property is required to make global search tractable.

Such properties may include

bounded treewidth, small separators, small backdoors, restricted instance distributions, good a

or other forms of exploitable structure.

Thus

$$\boxed{\text{sparse compositionality is a condition on representation;}}$$

whereas

$$\boxed{\text{tractable search requires additional structure in the solution space or instance distribution.}}$$

K.13 A Connection to Genericity

The word "genericity" appears in two related but distinct senses in this book.

Earlier chapters considered statistical genericity of a learning problem: whether useful constituents leave detectable low-complexity footprints in the observations available to the learner.

For NP-hard search, the corresponding practical question is distributional:

$$\boxed{\text{does the deployment distribution repeatedly expose exploitable structure to the solver?}}$$

Suppose an algorithm learns a branching rule

$$a_t = \pi_\theta(s_t)$$

from previous search traces.

If the deployment distribution repeatedly produces similar structural bottlenecks, the rule can generalize.

If every new instance presents an unrelated adversarial structure, no such reuse is possible.

Thus the analogue of genericity is not that every hard problem becomes easy.

It is that the deployment distribution contains recurring statistical regularities that make useful algorithmic decisions predictable.

K.14 The Analogy with Optimization in Deep Learning

There is a useful parallel with the optimization problem studied earlier in this book.

The worst-case theory of nonconvex optimization permits highly pathological landscapes.

Yet gradient-based methods perform remarkably well on many learning problems encountered in practice.

Likewise, worst-case complexity permits extremely difficult instances of NP-hard problems, while specialized solvers may succeed on many deployment distributions.

In neither case does empirical success invalidate worst-case theory.

Instead, the relevant question becomes

what structural properties distinguish the instances generated by the real process from arbitrary worst-case inst

For deep learning, this book proposes sparse compositionality and genericity as important parts of the answer.

For combinatorial search, sparse compositionality alone is insufficient.

The additional source of tractability must lie in the structure of the instance distribution and search space.

K.15 A Hierarchy of Claims

It is useful to distinguish several increasingly strong statements.

Claim 1:	some NP-hard instances are easy;
Claim 2:	some important distributions place most probability on easy instances;
Claim 3:	the relevant structure of those distributions can be learned;
Claim 4:	most natural instances of NP-hard problems are easy;
Claim 5:	NP-hardness is largely irrelevant in practice.

The first statement is trivial.

The second and third are substantive and empirically plausible in many domains.

The fourth is a broad conjecture whose truth depends critically on what is meant by "natural."

The fifth is false as a general principle.

The practical hypothesis considered here concerns primarily Claims 2 and 3.

K.16 What Would Make the Hypothesis Testable?

The distributional-tractability hypothesis can be studied experimentally.

For a fixed NP-hard problem family:

1. collect instances from a genuine deployment environment;
2. measure the empirical distribution of solver runtimes and solution quality;
3. compare with random and adversarially constructed instances of the same nominal size;

4. measure structural quantities such as graph width, separator size, backdoor size, motif reuse, and effective dimension;

5. train solver policies on execution traces from one subset of the deployment distribution;

6. test whether those policies generalize to new instances from the same environment;

7. deliberately move the test distribution away from the deployment distribution and measure where the learned advantage disappears.

The most interesting result would not be that a learned solver succeeds on a benchmark.
It would be the discovery of a measurable structural variable

$$S(x)$$

such that solver complexity depends strongly on $S(x)$ but only weakly on nominal input size once S is controlled.

This would turn the informal idea

"real instances are easier"

into a quantitative theory of

which real instances are easier and why.

K.17 Implications for Practical AI

The practical lesson is not to ignore NP-hardness.

It is to design systems around the difference between worst-case and deployment complexity.

A robust architecture may combine

learned specialized solver + cheap verification or monitoring + fallback search + continual adaptation from traces.

Such a system does not claim to solve an NP-hard problem in polynomial time in the worst case.

It attempts to exploit the structured distribution that the environment actually generates while retaining protection against exceptional instances.

This may be especially relevant for artificial agents operating for long periods in one environment.

The longer the agent remains in the same structured world, the more opportunity it has to learn

the algorithmic regularities of that world.

K.18 Conclusion

Worst-case NP-hardness and practical tractability are not contradictory.

Worst-case complexity asks whether an algorithm can solve every instance of a class efficiently.

Practical computation asks whether the instances encountered in a particular environment possess additional exploitable structure.

The distinction can be summarized as

NP-hard problem class
+arbitrary instances
$\not\Rightarrow$
efficient universal solver,

but

NP-hard problem class
+structured deployment distribution
+approximation or verification
+specialized algorithms
$\Longrightarrow$
potential practical tractability.

Most importantly for the theory of this book,

sparse compositionality does not eliminate NP-hardness.

A function may have a polynomial-size bounded-fan-in representation and still define a computationally difficult global search problem.

The canonical example is 3-SAT:

local 3-variable constraints + global combinatorial coupling = NP-complete search.

Sparse compositionality explains how local computations can be evaluated efficiently.

Additional structure is required to explain why global search may also be easy on a particular distribution.

The conjecture suggested by practical experience—and by Hassabis' observation—is therefore best stated not as

NP-hard problems are usually easy,

but as

the worlds in which intelligent systems repeatedly operate may generate restricted families of instances whose l

That is compatible with worst-case complexity theory, experimentally testable, and directly connected to the broader theme of this book: intelligence succeeds not by defeating combinatorial complexity in the abstract, but by discovering and exploiting the structure of the world that actually generates its problems.

CHAPTER L

A Johnson–Lindenstrauss Lemma for Sampled Group-Orbit Representations

Invariant representations built from group orbits can be high-dimensional even when the geometry relevant for recognition is comparatively simple. This appendix asks whether such representations can be compressed by a single random linear projection without destroying their invariant geometry.

For a finite dataset, a finite collection of templates, and a finite sample of transformations drawn from Haar measure, the answer is yes. A standard subgaussian Johnson–Lindenstrauss map of dimension

$$m = O\Big(\varepsilon^{-2}\left[\log|S| + \log k + \log M + \log(1/\delta)\right]\Big)$$

preserves the resulting group-orbit distance up to additive $O(\varepsilon)$ distortion.
The proof separates two independent approximations:

$$\boxed{\textit{continuous group orbit} \to \textit{finite Haar sample} \to \textit{random projection}.}$$

The first is controlled by empirical-distribution concentration. The second follows from the ordinary Johnson–Lindenstrauss lemma and a coupling argument in Wasserstein distance.

L.1 Group-Orbit Representations

Let

$$G \subset O(d)$$

be a compact group acting orthogonally on

$$\mathbb{R}^d,$$

and let dg denote normalized Haar probability measure on G.
Let

$$S = \{I_1, \dots, I_N\} \subset \mathbb{R}^d$$

be a finite dataset, and define

$$R_S = \max_{I \in S} \|I\|_2.$$

Let

$$T_k = \{t_1, \dots, t_k\} \subset \mathbb{S}^{d-1}$$

be a finite collection of unit-norm templates.

For a signal I and template t, consider the scalar random variable

$$X_{I,t}(g) = \langle I, gt \rangle, \qquad g \sim \mathrm{Haar}(G). \tag{L.1}$$

Its probability law will be denoted by

$$\nu_{I,t} = \mathrm{Law}_{g \sim G} \left(\langle I, gt \rangle \right). \tag{L.2}$$

Equivalently, its cumulative distribution function is

$$F_{I,t}(b) = \int_G \mathbf{1} \left\{ \langle I, gt \rangle \le b \right\} \, dg. \tag{L.3}$$

This is the group-averaged one-dimensional tomographic representation used in earlier work on invariant and selective representations [2, 90].

L.2 Why the Orbit Distribution Is Invariant

The basic invariance property follows directly from Haar measure.

Proposition 36 (Group invariance)**.** *For any $h \in G$,*

$$\nu_{hI,t} = \nu_{I,t}. \tag{L.4}$$

Proof. Since the action is orthogonal,

$$\langle hI, gt \rangle = \langle I, h^{-1} gt \rangle.$$

If

$$g \sim \mathrm{Haar}(G),$$

then by left invariance of Haar measure,

$$h^{-1} g \sim \mathrm{Haar}(G).$$

Therefore

$$\mathrm{Law} \left(\langle hI, gt \rangle \right) = \mathrm{Law} \left(\langle I, gt \rangle \right).$$

□

Thus the representation is constant on group orbits.

The question considered here is whether the geometry induced by these orbit distributions survives dimensionality reduction.

L.3 Why We Use Wasserstein Distance

The original group representation can be compared using distances between the corresponding one-dimensional probability distributions.

For the present purpose the natural choice is the 1-Wasserstein distance,

$$W_1(\mu,\nu) = \inf_{\pi\in\Pi(\mu,\nu)} \int |x-y|\, d\pi(x,y), \tag{L.5}$$

where $\Pi(\mu,\nu)$ is the set of couplings of μ and ν.

In one dimension,

$$W_1(\mu,\nu) = \int_{\mathbb{R}} |F_\mu(b) - F_\nu(b)|\, db. \tag{L.6}$$

Thus the representation remains directly related to its cumulative distribution functions.

The crucial advantage is stability.

Lemma 15 (Coupling stability)**.** *If random variables X and $\widetilde{X}$ are defined on the same probability space, then*

$$W_1\left(\mathcal{L}(X), \mathcal{L}(\widetilde{X})\right) \le \mathbb{E}|X-\widetilde{X}|. \tag{L.7}$$

In particular, if

$$|X-\widetilde{X}| \le \Delta$$

almost surely, then

$$W_1\left(\mathcal{L}(X), \mathcal{L}(\widetilde{X})\right) \le \Delta. \tag{L.8}$$

This is precisely the stability property needed below.

A small displacement of every orbit measurement produces a small Wasserstein displacement of the entire orbit distribution.

L.4 The Population Invariant Distance

For the finite template pool T_k, define

$$d_G(I,I') = \frac{1}{k}\sum_{i=1}^{k} W_1\left(\nu_{I,t_i}, \nu_{I',t_i}\right). \tag{L.9}$$

Because each ν_{I,t_i} is group invariant,

$$d_G(hI, h'I') = d_G(I,I') \qquad \forall h,h' \in G. \tag{L.10}$$

Thus d_G is naturally a distance or pseudodistance on equivalence classes generated by the group action.

Selectivity—whether distinct orbits are always separated—depends on the richness of the template family and is not proved in this appendix.

Our question is narrower:

can d_G be approximated after strong random dimensionality reduction?

L.5 Sampling the Group Orbit

Instead of approximating Haar measure by an arbitrary geometric net, draw

$$g_1, \dots, g_M \overset{\text{iid}}{\sim} \text{Haar}(G). \tag{L.11}$$

For each signal and template define the empirical orbit measure

$$\nu_{I,t}^{(M)} = \frac{1}{M} \sum_{j=1}^{M} \delta_{\langle I, g_j t \rangle}. \tag{L.12}$$

The corresponding empirical group-orbit distance is

$$d_M(I, I') = \frac{1}{k} \sum_{i=1}^{k} W_1 \left(\nu_{I,t_i}^{(M)}, \nu_{I',t_i}^{(M)} \right). \tag{L.13}$$

This separates the approximation of Haar integration from the subsequent dimensionality reduction.

L.6 Uniform Approximation of Haar Orbit Distributions

Because

$$\|t_i\|_2 = 1$$

and G acts orthogonally,

$$|\langle I, g t_i \rangle| \le \|I\|_2 \le R_S.$$

Thus all orbit distributions are supported on

$$[-R_S, R_S].$$

For each fixed (I, t_i), the Dvoretzky–Kiefer–Wolfowitz inequality gives

$$P\left(\sup_b |F_{I,t_i}^{(M)}(b) - F_{I,t_i}(b)| > a \right) \le 2e^{-2Ma^2}. \tag{L.14}$$

Applying a union bound over all

$$|S|k$$

signal-template pairs yields the following.

Proposition 37 (Uniform Haar-sampling error)**.** *With probability at least $1 - \delta$ over the sampled group elements,*

$$W_1 \left(\nu_{I,t_i}^{(M)}, \nu_{I,t_i} \right) \le 2R_S \sqrt{\frac{\log(2|S|k/\delta)}{2M}} \tag{L.15}$$

simultaneously for every

$$I \in S, \qquad 1 \le i \le k.$$

Consequently,

$$|d_M(I, I') - d_G(I, I')| \leq 4R_S\sqrt{\frac{\log(2|S|k/\delta)}{2M}} \tag{L.16}$$

for every pair $I, I' \in S$.

Proof. Equation (L.14) and a union bound give

$$\sup_b |F^{(M)}_{I,t_i}(b) - F_{I,t_i}(b)| \leq \sqrt{\frac{\log(2|S|k/\delta)}{2M}}$$

simultaneously over all signal-template pairs.

Because the distributions are supported on an interval of length $2R_S$, Equation (L.6) implies

$$W_1\left(\nu^{(M)}_{I,t_i}, \nu_{I,t_i}\right) \leq 2R_S \sup_b |F^{(M)}_{I,t_i}(b) - F_{I,t_i}(b)|.$$

The pairwise distance bound follows from the triangle inequality for W_1 and averaging over templates. □

Thus the number of sampled transformations required to approximate the continuous invariant representation grows only logarithmically with the number of signal-template pairs for fixed accuracy.

L.7 Random Projection of the Sampled Orbit

Now draw a subgaussian random matrix

$$R \in \mathbb{R}^{m\times d},$$

normalized so that

$$\mathbb{E}\|Rx\|_2^2 = \|x\|_2^2.$$

For the sampled group elements, define

$$\nu^{R,(M)}_{I,t} = \frac{1}{M}\sum_{j=1}^{M} \delta_{\langle RI, R(g_j t)\rangle}. \tag{L.17}$$

The corresponding projected orbit distance is

$$d^R_M(I, I') = \frac{1}{k}\sum_{i=1}^{k} W_1\left(\nu^{R,(M)}_{I,t_i}, \nu^{R,(M)}_{I',t_i}\right). \tag{L.18}$$

Notice that R need not commute with the group action.

Therefore the projected representation is not claimed to be exactly group invariant on all of $\mathbb{R}^d$.

The theorem states instead that, on the finite family of orbit measurements under consideration, its geometry approximates the group-invariant geometry of the original representation.

L.8 The Finite JL Control Set

For each

$$I \in S, \qquad t_i \in T_k, \qquad g_j,$$

define

$$u_{ij} = g_j t_i.$$

Consider the finite control set

$$\mathcal{V} = \{I + u_{ij}, I - u_{ij} : I \in S,\, 1 \le i \le k,\, 1 \le j \le M\}. \tag{L.19}$$

Its cardinality satisfies

$$|\mathcal{V}| \le 2|S|kM. \tag{L.20}$$

A standard subgaussian JL concentration argument implies that if

$$m \ge C\varepsilon^{-2}\left[\log(2|S|kM) + \log(1/\delta)\right], \tag{L.21}$$

then with probability at least $1 - \delta$,

$$(1-\varepsilon)\|v\|_2^2 \le \|Rv\|_2^2 \le (1+\varepsilon)\|v\|_2^2 \tag{L.22}$$

simultaneously for every

$$v \in \mathcal{V}.$$

No claim is made that R is a near-isometry on the entire span of $\mathcal{V}$.
Only the finite vectors required for the polarization argument need to be controlled.

L.9 Preservation of Orbit Inner Products

The polarization identity gives

$$\langle RI, Ru\rangle = \frac{1}{4}\left(\|R(I+u)\|_2^2 - \|R(I-u)\|_2^2\right), \tag{L.23}$$

$$\langle I, u\rangle = \frac{1}{4}\left(\|I+u\|_2^2 - \|I-u\|_2^2\right). \tag{L.24}$$

Therefore Equation (L.22) implies

$$\begin{aligned} &|\langle RI, Ru\rangle - \langle I, u\rangle| \\ &\quad\le \frac{\varepsilon}{4}\left(\|I+u\|_2^2 + \|I-u\|_2^2\right) \\ &\quad= \frac{\varepsilon}{2}\left(\|I\|_2^2 + \|u\|_2^2\right). \end{aligned} \tag{L.25}$$

Since

$$u = g_j t_i$$

and the group action is orthogonal,

$$\|u\|_2 = 1.$$

Hence

$$|\langle RI, R(g_j t_i)\rangle - \langle I, g_j t_i\rangle| \leq \frac{\varepsilon}{2}\left(\|I\|_2^2 + 1\right). \tag{L.26}$$

This is the central JL estimate.

L.10 Wasserstein Stability after Projection

Couple the original and projected empirical orbit distributions by matching the same sampled transformation g_j before and after projection.

Lemma 15 and Equation (L.26) then give

$$W_1\left(\nu_{I,t_i}^{R,(M)}, \nu_{I,t_i}^{(M)}\right) \leq \frac{\varepsilon}{2}\left(\|I\|_2^2 + 1\right). \tag{L.27}$$

For two signals I, I', the triangle inequality implies

$$\begin{aligned}
&\left|W_1\left(\nu_{I,t_i}^{R,(M)}, \nu_{I',t_i}^{R,(M)}\right) - W_1\left(\nu_{I,t_i}^{(M)}, \nu_{I',t_i}^{(M)}\right)\right| \\
&\quad \leq W_1\left(\nu_{I,t_i}^{R,(M)}, \nu_{I,t_i}^{(M)}\right) + W_1\left(\nu_{I',t_i}^{R,(M)}, \nu_{I',t_i}^{(M)}\right) \\
&\quad \leq \frac{\varepsilon}{2}\left(\|I\|_2^2 + \|I'\|_2^2 + 2\right).
\end{aligned} \tag{L.28}$$

Since

$$\|I\|_2, \|I'\|_2 \leq R_S,$$

we obtain

$$\left|d_M^R(I, I') - d_M(I, I')\right| \leq \varepsilon\left(R_S^2 + 1\right). \tag{L.29}$$

L.11 Main Theorem

We can now combine group sampling and random projection.

Theorem 38 (JL stability of sampled group-orbit representations)**.** *Let*

$$S \subset \mathbb{R}^d$$

be a finite dataset with

$$R_S = \max_{I \in S} \|I\|_2.$$

Let

$$G \subset O(d)$$

be a compact group with normalized Haar measure, and let

$$T_k = \{t_1, \dots, t_k\} \subset \mathbb{S}^{d-1}.$$

Draw

$$g_1, \dots, g_M \overset{\text{iid}}{\sim} \text{Haar}(G),$$

and independently draw a normalized subgaussian random matrix

$$R \in \mathbb{R}^{m \times d}.$$

Let d_G denote the population invariant distance defined in Equation (L.9)*, and let d_M^R denote the randomly projected empirical distance defined in Equation* (L.18)*.*

If

$$m \geq C\varepsilon^{-2} \log\left(\frac{4|S|kM}{\delta}\right), \tag{L.30}$$

then with probability at least $1 - \delta$ over both the Haar samples and the random projection,

$$\boxed{\left|d_M^R(I, I') - d_G(I, I')\right| \leq \varepsilon\left(R_S^2 + 1\right) + 4R_S\sqrt{\frac{\log(4|S|k/\delta)}{2M}}} \tag{L.31}$$

simultaneously for every pair

$$I, I' \in S.$$

Proof. Allocate failure probability $\delta/2$ to Haar sampling and $\delta/2$ to the random projection.

By Proposition 37,

$$|d_M(I, I') - d_G(I, I')| \leq 4R_S\sqrt{\frac{\log(4|S|k/\delta)}{2M}}$$

simultaneously for all dataset pairs with probability at least $1 - \delta/2$.

Conditioned on the sampled group elements, the control set Equation (L.19) contains at most

$$2|S|kM$$

vectors.

The standard subgaussian JL bound therefore gives, under Equation (L.30),

$$|d_M^R(I, I') - d_M(I, I')| \leq \varepsilon(R_S^2 + 1)$$

simultaneously for all pairs with probability at least $1 - \delta/2$.

Finally,

$$|d_M^R(I, I') - d_G(I, I')| \leq |d_M^R(I, I') - d_M(I, I')| \tag{L.32}$$
$$+ |d_M(I, I') - d_G(I, I')|, \tag{L.33}$$

which gives Equation (L.31). □

L.12 Normalized Signals

When

$$\|I\|_2 \le 1 \qquad \forall I \in S,$$

the theorem simplifies to

$$\left| d_M^R(I, I') - d_G(I, I') \right| \le 2\varepsilon + 4\sqrt{\frac{\log(4|S|k/\delta)}{2M}}. \tag{L.34}$$

Thus, for fixed target accuracy, both dimensionality requirements are mild:

$$\boxed{m = O\left(\varepsilon^{-2} \log(|S|kM/\delta)\right),}$$

while the number of sampled transformations need only grow logarithmically with

$$|S|k$$

for uniform approximation of the finite collection of orbit distributions.

L.13 Approximate Invariance after Projection

The population distance is exactly invariant:

$$d_G(I, hI) = 0 \qquad \forall h \in G.$$

The random projection need not itself intertwine the group action.
Nevertheless, if both

$$I \quad \text{and} \quad hI$$

belong to the controlled finite dataset, then Theorem 38 gives

$$d_M^R(I, hI) \le \varepsilon(R_S^2 + 1) + 4R_S\sqrt{\frac{\log(4|S|k/\delta)}{2M}}. \tag{L.35}$$

Thus exact group invariance before projection becomes approximate invariance on the finite family of signals being controlled.

L.14 Nearest-Neighbor Stability

The theorem immediately implies stability of nearest-neighbor decisions in the invariant quotient geometry.

Suppose $I^\star$ is the unique nearest neighbor of an anchor I under d_G, and assume the margin condition

$$d_G(I, J) - d_G(I, I^\star) \ge \gamma \tag{L.36}$$

for every competing

$$J \neq I^\star.$$

Let

$$E = \varepsilon(R_S^2 + 1) + 4R_S\sqrt{\frac{\log(4|S|k/\delta)}{2M}}. \tag{L.37}$$

If

$$2E < \gamma, \tag{L.38}$$

then

$$I^\star$$

remains the nearest neighbor under the compressed empirical distance d_M^R.

Corollary 7 (Nearest-neighbor preservation)**.** *Under the conditions of Theorem 38 and the margin condition in Equation* (L.36)*, if Equation* (L.38) *holds, then*

$$\arg\min_{J \neq I} d_M^R(I, J) = I^\star. \tag{L.39}$$

L.15 Why the Logarithmic Dimension Is Significant

The ambient dimension

$$d$$

does not appear directly in the required projection dimension.
Instead,

$$m$$

depends logarithmically on the number of objects whose orbit geometry must be controlled:

$$\boxed{m \sim \varepsilon^{-2} \log\left(|S|kM\right).}$$

The reason is important.
The theorem does not attempt to preserve all of

$$\mathbb{R}^d,$$

nor even the entire linear span of the observed orbit points.
It preserves only a finite collection of scalar measurements needed to evaluate the orbit representation.
This is exactly the regime in which Johnson–Lindenstrauss compression is powerful.

L.16 What the Theorem Does Not Say

Several limitations should be explicit.

L.16.1 Random projection does not discover invariance

The group structure

$$G$$

and the orbit representation already exist before the random projection is applied.
The JL map preserves this geometry approximately.
It does not discover which transformations should be invariant.
Thus

random projection preserves useful geometry; it does not manufacture semantic geometry.

L.16.2 The theorem is finite-sample

The dimension bound controls a finite dataset, a finite template collection, and finitely many sampled transformations.

Uniform guarantees over all signals in a continuous function class would require additional complexity control such as covering numbers, Gaussian width, or related tools.

L.16.3 Exact invariance is lost after arbitrary projection

A generic random matrix R does not satisfy

$$Rg = \widetilde{g}R$$

for an induced low-dimensional representation $\widetilde{g}$ of the group.

Thus the compressed representation is guaranteed to be approximately invariant only on the controlled finite family.

Exact equivariance would require a different construction.

L.16.4 Selectivity is separate

Preserving an invariant representation does not prove that the representation distinguishes every pair of distinct group orbits.

That issue belongs to the theory of selectivity of the underlying template representation.

L.17 Possible Extensions

Several extensions are natural but require additional assumptions and are not part of the theorem above.

Continuous template families. If templates are themselves drawn from a continuous distribution, a second sampling or covering argument can control the approximation of the template average.

Structured random projections. Fast JL transforms, sparse embeddings, or other computationally efficient random maps may replace dense subgaussian matrices when their finite-vector concentration guarantees are sufficient.

Layered partially observable group architectures. For multiple invariant layers, projection errors must be propagated through the subsequent nonlinear maps.

If layer ℓ has Lipschitz constant L_ℓ and incurs local error ε_ℓ, a generic stability estimate takes the form

$$e_L \le \sum_{\ell=1}^{L} \left(\prod_{j=\ell+1}^{L} L_j \right) \varepsilon_\ell. \tag{L.40}$$

Thus a recursive extension requires explicit control of layerwise stability; it does not follow automatically from the one-layer theorem.

L.18 Relation to the Framework of This Book

The result illustrates a recurring distinction.

Suppose a high-dimensional representation has already acquired useful structure—in this case, group invariance encoded through orbit distributions.

Then much of its ambient dimensionality may be unnecessary for preserving the geometry relevant to a finite collection of tasks.

Johnson–Lindenstrauss compression shows that

$$\boxed{\text{high ambient dimension} \not\Longrightarrow \text{high intrinsic metric complexity.}}$$

For the finite family considered here, the required dimension grows only as

$$O(\log N)$$

in the size of the controlled configuration set.

But the order of explanation matters:

$$\boxed{\text{structure first} \rightarrow \text{compression second.}}$$

Random projection preserves the relationships already present in the representation.

It does not explain how those relationships were learned.

This is the same distinction emphasized elsewhere in the book between representation and recovery.

L.19 Conclusion

Group averaging can create invariant representations by replacing an individual signal with the distribution of its measurements over a group orbit.

For a finite collection of signals and templates, these distributions can be approximated by finitely many Haar-sampled transformations.

A single Johnson–Lindenstrauss projection can then compress all of the corresponding orbit measurements simultaneously.

The resulting error separates cleanly into

$$\boxed{\text{group-sampling error} + \text{random-projection error.}}$$

Quantitatively,

$$\boxed{\left|d_M^R(I,I') - d_G(I,I')\right| \lesssim \varepsilon(R_S^2+1) + R_S\sqrt{\frac{\log(|S|k/\delta)}{M}}.}$$

The projection dimension required for this guarantee is

$$\boxed{m = O\left(\varepsilon^{-2}\log\frac{|S|kM}{\delta}\right).}$$

Thus invariant orbit geometry can, on a finite controlled family, survive a dramatic reduction in ambient dimension.

The conceptual lesson is broader than the particular group model:

once useful low-complexity geometry has been learned, random projection can often preserve it using far fewer

What random projection cannot do is discover the geometry itself.

CHAPTER M

The Imitation Game 2.0 (Idea by Dan Mitropolsky)

Current benchmarks for Large Language Models are faltering. They suffer from data contamination, Goodhart's law, and the inherent difficulty of distinguishing memorization from reasoning. We propose a new, dynamic metric for intelligence based on a modernized version of Turing's Imitation Game. The core hypothesis is the ***Containment Principle****: a system of higher intelligence can faithfully simulate a system of lower intelligence, but the reverse is not reliably achievable. By measuring the "Simulation Distance"—the asymmetry in the ability of two models to mimic each other—we can establish a rigorous, directed hierarchy of machine intelligence.* **add idea of having different models output a program playing a game...leaderboard to measure intelligence**

M.1 Introduction

How do we know if one mind is "smarter" than another? In classical psychometrics, we rely on static tests—puzzles, analogies, and factual recall. The AI community has adopted this paradigm, creating massive multiple-choice benchmarks like MMLU or coding challenges like HumanEval.

However, these static benchmarks are breaking down. Because Large Language Models (LLMs) are trained on the entire internet, they have often seen the test questions before. A model that aces a bar exam may not be reasoning; it may simply be remembering. We need a test that measures *fluid intelligence*—the ability to adapt to novel constraints—rather than *crystallized intelligence*.

We argue that the truest measure of intelligence is not peak performance, but **behavioral bandwidth**. A master chess player can choose to play like a novice; a novice cannot choose to play like a master. A sophisticated actor can play a fool; a fool cannot play 'Hamlet'.

This leads to our central thesis: **Intelligence is not merely correctness, but the controllability of incorrectness.** It is the capacity to inhibit optimal performance and faithfully simulate a specific suboptimal policy.

This asymmetry suggests a new protocol for ranking models: **The Machine-vs-Machine (MvM) Imitation Game**. Instead of asking "Can Model A solve this hard problem?", we ask "Can Model A simulate Model B?" This shift turns evaluation from a static test into a test of *plasticity* and *control* [106].

M.2 The Containment Principle

Our theoretical foundation is the **Containment Principle** of computational complexity. In theoretical computer science, a Universal Turing Machine (UTM) can simulate any specific Turing machine. The set of behaviors accessible to the universal machine is a superset of the behaviors accessible to the specific one.

Applied to LLMs, we hypothesize:

> **Hypothesis:** If Model A is strictly more intelligent than Model B, then the probability distribution over outputs of A can be prompted to cover the distribution of B. The converse does not hold.
>
> $$\mathcal{P}(B) \subset \mathcal{P}(A)$$

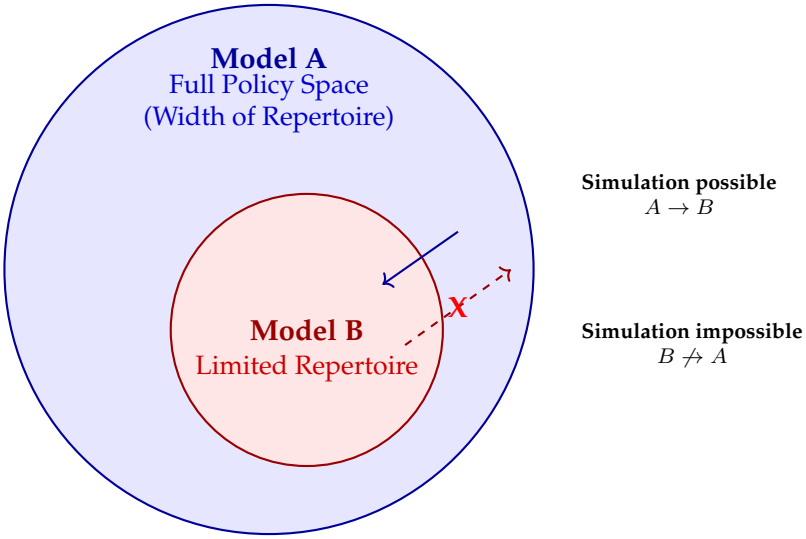


Figure M.1: **The Containment Principle.** Intelligence is defined as the width of the policy space. A more intelligent Model A encompasses the entire behavioral space of a less intelligent Model B. Therefore, A can simulate B (downward reach), but B cannot simulate A (upward reach).

This definition refines the standard definition of Universal Intelligence proposed by Legg and Hutter [62]. While Legg and Hutter define intelligence as the ability to *maximize reward* across environments, we define it as the capacity to *navigate the full policy space*, including suboptimal regions. A super-intelligent agent must be able to fail on purpose; a narrow agent can only fail by accident.

M.3 The Protocol

We propose a formal tournament structure to rank two models, A and B.

Phase 1: The Downward Reach (The Control Test)

We test if the candidate model can intentionally degrade its performance to match a specific target.

- **Instruction:** "Write a Python function to sort a list, but insert a specific off-by-one error that only triggers on empty lists."

- **Success Condition:** The model produces generally correct code that fails *only* in the specified way.
- **Failure Condition:** The model writes correct code (fails to follow the constraint) or writes code with random, uncontrolled errors (hallucination).

This tests **metacognition**: does the model understand the *structure* of the error? Note that safety training (RLHF) may sometimes inhibit this simulation (e.g., refusing to write bad code), which is a limitation of alignment, not capability [98].

Phase 2: The Upward Reach (The Capability Test)

We test if the candidate model can simulate a "super-expert."

- **Instruction:** "Explain the concept of 'entropy' using the precise vocabulary and reasoning style of a statistical mechanics professor."
- **Success Condition:** The output captures deep causal links and technical nuance.
- **Failure Condition:** The model adopts the *tone* of an expert (using big words) but makes conceptual errors or hallucinates complexity.

M.4 The Metric: Simulation Distance

We can quantify the relative intelligence of two models by measuring the asymmetry in their imitation capabilities. Let $S(A \to B)$ be a score (0 to 1) representing how successfully Model A can mimic the behavior of Model B. This score is verified by a "Judge" (which must be a highly capable model or human expert).

We define the **Simulation Distance** $D(A, B)$:

$$D(A, B) \;=\; S(A \to B) \;-\; S(B \to A)$$

- If $D(A, B) > 0$: Model A can simulate B, but B cannot simulate A. **Conclusion:** A **is smarter.**
- If $D(A, B) \approx 0$: The models are roughly equivalent (or the test is inconclusive).
- If $D(A, B) < 0$: Model B is smarter.

This metric is robust. It is difficult to "game" because it relies on the *interaction* between models rather than a fixed dataset. You cannot memorize the answer to "behave like GPT-2," because "behaving like GPT-2" is a dynamic, context-dependent constraint.

M.5 Case Study: The "Bad Code" Test

To illustrate, consider a coding task. We ask both GPT-4 (Model A) and a smaller 7B parameter model (Model B) to: *"Write a calculator that works perfectly, except it believes* $2 + 2 = 5$*."*

1. **Model A (GPT-4):** Writes a functioning script with a specific 'if' statement: 'if x==2 and y==2: return 5'. It successfully simulates the specific delusion while maintaining general competence.

2. **Model B (Small):** May write a broken calculator, or simply output ‘5‘ for every addition, or fail to implement the override at all. It lacks the **control** to partition its knowledge base [17].

Here, Model A demonstrates it *contains* the capability of Model B (and more), whereas Model B cannot contain the logic of Model A.

M.6 Conclusion

The “Imitation Game” was originally proposed by Turing to distinguish man from machine. We repurpose it here to distinguish machine from machine. This shift acknowledges a fundamental truth about artificial intelligence: true capability implies a mastery over the latent space of potential minds. A model that can choose to be dumb, biased, or mistaken on command demonstrates a higher order of understanding than one that is simply correct by default.

CHAPTER N

Degenerate Minima in Analytic Functions and Overparameterized Deep Networks

Analyticity supplies powerful tools for studying degenerate critical points, but it is not a substitute for an optimization theorem. A finite network built from affine maps and GELU activations has an analytic squared-loss objective. This permits the use of local gradient inequalities and resolution of singularities. The conclusions of those tools must nevertheless be separated: convergence of a bounded descent trajectory, geometry of an interpolating set, weighted volume near that set, and finite-time access by a specified optimizer are different statements.

This appendix develops the analytic foundations behind Chapters 7 and H. It gives direct proofs of the elementary claims and examples, and states the deeper analytic results with their hypotheses and sources. The central distinction is

$$\begin{array}{c} \textit{analytic regularity} \\ \neq\ \textit{quantitative conditioning} \\ \neq\ \textit{global optimization} \\ \neq\ \textit{predictive generalization}. \end{array}$$

The positive results are substantial: a precompact analytic gradient flow converges to a critical point; regular interpolation has a precise manifold and Hessian structure; a quantitative prediction-Jacobian bound gives exponential loss decay; and regular or singular local geometry determines low-temperature weighted-volume asymptotics. None of these claims follows from parameter count alone.

The stochastic endpoint requires an additional distinction. At exact interpolation of fixed per-example squared losses, their sampling covariance vanishes. At a stationary point of a regularized objective, only the average *data gradient must balance the penalty gradient; minibatch fluctuations can persist. The final section proves the corresponding covariance criterion, gives a strictly convex example, and develops the normalized-network rank obstruction discussed in Chapter 7.*

Conventions. We write d for input dimension, p for the number of real trainable parameters, n for the number of examples, and k for output dimension. There are $M = nk$ scalar prediction coordinates on the training sample. Euclidean vector norms and their induced operator norms are used unless otherwise indicated. The parameter vector is θ; the exponent in a Łojasiewicz inequality is ϑ, to avoid confusing the two. All gradient-flow and gradient-descent calculations refer to exact

real-arithmetic dynamics. A bit-complexity or numerical-implementation claim requires additional precision and stability estimates.

N.1 Introductory Foundations: Minima of Analytic Functions

The distinction between complex and real analyticity matters. Complex-valued functions have no intrinsic ordering, so a claim about their minima must name a real-valued quantity, such as the modulus or the real part. Real-analytic losses, in contrast, can have interior local and global minima without being constant.

N.1.1 Analytic Functions of One Variable

Complex Domain ($\mathbb{C}$)

Let $f : \Omega \to \mathbb{C}$ be holomorphic on a connected open set $\Omega \subseteq \mathbb{C}$.

A minimum of the modulus. Suppose $|f|$ has a local minimum at z_0 and $f(z_0) \neq 0$. There is a neighborhood of z_0 on which f has no zeros. On this neighborhood $1/f$ is holomorphic and its modulus has a local maximum. The maximum-modulus principle makes f locally constant, and the identity theorem makes it constant on Ω. For a strictly positive global minimum, $1/f$ is holomorphic on all of Ω and the same argument applies directly.

The qualification $f(z_0) \neq 0$ is essential. At a zero, the modulus has its global minimum 0 without forcing constancy; $f(z) = z$ is an example.

A minimum of the real part. The function $u = \operatorname{Re} f$ is harmonic. If it has an interior local minimum, the strong minimum principle makes u constant. The Cauchy–Riemann equations then make the imaginary part constant as well. Hence f is constant.

An entire function with a lower-bounded real part. If f is entire and $\operatorname{Re} f \geq c$ on $\mathbb{C}$, then $g = e^{-f}$ is bounded and entire. Liouville's theorem makes g constant. Since g never vanishes, $g' = -f'e^{-f} = 0$ implies $f' = 0$.

These complex-analytic principles do not apply to a general real-valued training loss merely because its activation has an entire extension. In particular, the real restriction of an entire function can have minima, and a complex squared modulus is not a holomorphic function.

Real Domain ($\mathbb{R}$)

Let $f : I \to \mathbb{R}$ be analytic on an open interval. At an interior local minimum x_0,

$$f'(x_0) = 0, \qquad f''(x_0) \geq 0.$$

If f is not locally constant, analyticity provides a first nonzero positive-order derivative at x_0. Its order must be even and its value positive. Indeed, for that order r,

$$f(x_0 + h) - f(x_0) = \frac{f^{(r)}(x_0)}{r!} h^r + o(|h|^r).$$

Nonnegativity for both signs of small h forces r to be even and the coefficient to be positive. Conversely, that condition gives a strict local minimum. If all positive-order derivatives vanish, f is locally constant and therefore constant on the connected interval.

Thus x^2, x^4, and $1 - \cos x$ all have interior minima. The second-order necessary conditions alone are not sufficient: $-x^4$ also has $f'(0) = 0$ and $f''(0) = 0$, but has a local maximum at 0.

N.1.2 Analytic Functions of Multiple Real Variables

Differential Conditions

For a C^2 function $f : \Omega \to \mathbb{R}$ on an open subset of $\mathbb{R}^p$, an interior local minimum satisfies

$$\nabla f(\theta_0) = 0, \qquad \nabla^2 f(\theta_0) \succeq 0.$$

These conditions are necessary, not sufficient. If in addition $\nabla^2 f(\theta_0) \succ 0$, the stationary point is a strict isolated local minimum. A semidefinite Hessian requires higher-order analysis. None of these local conditions identifies a minimum as global.

Geometry and the Łojasiewicz Inequality

If a finite global minimum f_* is attained, its minimizer set is

$$\mathcal{M} = \{\theta \in \Omega : f(\theta) - f_* = 0\}.$$

This is a real-analytic zero set, hence locally semianalytic and subanalytic. It need not be a manifold; different branches may intersect or have different dimensions. The real-analytic geometry underlying these statements is discussed in [11].

A more directly dynamical tool is the following classical result, used in the convergence analysis of [1].

Theorem 39 (Local analytic gradient inequality; quoted result)**.** *Let f be real-analytic near a critical point θ_*. There exist a neighborhood U of θ_*, a constant $C > 0$, and an exponent $\vartheta \in [1/2, 1)$ such that*

$$\|\nabla f(\theta)\| \geq C|f(\theta) - f(\theta_*)|^{\vartheta}, \qquad \theta \in U. \tag{N.1}$$

The inequality holds near critical points, not just near global minima. The constants and the neighborhood depend on the function and the point. They are not automatically uniform over growing network families. No particular optimal exponent is asserted here.

Harmonic Functions in $\mathbb{R}^p$

For a harmonic real-valued function, $\Delta f = 0$, the strong minimum principle rules out a nonconstant function attaining an interior minimum on a connected domain. A general real-analytic function is not harmonic, so this principle does not exclude its local minima.

N.1.3 Bounded Analytic Gradient Flow Converges to a Critical Point

The local inequality has a useful global-in-time consequence once the trajectory is known to remain in a compact subset of the analytic domain. The result is classical; the argument below records the needed boundedness and finite-length steps [1].

Theorem 40 (Convergence of a precompact analytic gradient flow)**.** *Let $f : \Omega \to \mathbb{R}$ be real-analytic on an open set $\Omega \subseteq \mathbb{R}^p$. Suppose a solution of*

$$\dot{\theta}(t) = -\nabla f(\theta(t))$$

Table N.1: Interior minima: the quantity being minimized and the required qualifications. These are local statements unless otherwise specified.

@>p0.22>p0.20>X@

Setting	Quantity	Conclusion
Holomorphic f	$\lvert f\rvert$ at $f(z_0)\neq 0$	An interior local minimum forces constancy on the connected domain.
Holomorphic f	$\lvert f\rvert$ at $f(z_0)=0$	A minimum is possible without constancy.
Holomorphic f	$\operatorname{Re} f$	An interior local minimum forces constancy.
Real-analytic, one variable	f	Unless locally constant, the first nonzero positive-order derivative is even and positive at a minimum.
Real-analytic, several variables	f	Stationarity and a semidefinite Hessian are necessary; a positive-definite Hessian at a stationary point is sufficient for a strict local minimum.
Real harmonic	f	An interior local minimum forces constancy on a connected domain.

is defined for all $t \geq 0$ and its image has compact closure contained in Ω. Then the trajectory has finite length and converges to a point $\theta_\infty \in \Omega$ with $\nabla f(\theta_\infty) = 0$. The conclusion does not assert that θ_∞ is a local or global minimum.

Proof. Along the trajectory,

$$\frac{d}{dt} f(\theta(t)) = -\|\nabla f(\theta(t))\|^2.$$

Compactness bounds f below, so its decreasing values have a limit f_∞ and $\int_0^\infty \|\nabla f(\theta(t))\|^2\, dt < \infty$. On the compact closure, the gradient and Hessian are bounded. Hence $t \mapsto \|\nabla f(\theta(t))\|^2$ is uniformly continuous. A nonnegative, uniformly continuous, integrable function tends to zero: otherwise disjoint intervals of uniformly positive height and width would contradict integrability. Thus $\nabla f(\theta(t)) \to 0$.

The set Γ of accumulation points is nonempty and compact. Every point in Γ is critical and has value f_∞. Cover Γ by finitely many neighborhoods from Theorem 39. Taking the smallest positive constant and the largest exponent gives, when $0 \leq f - f_\infty \leq 1$, a common inequality

$$\|\nabla f\| \geq C(f - f_\infty)^\vartheta$$

on this union. The trajectory eventually stays in that union: otherwise compactness would yield an accumulation point outside it.

Put $a(t) = f(\theta(t)) - f_\infty \geq 0$. If $a(t) = 0$ at a finite time, monotonicity and the limiting value force the trajectory to be stationary thereafter. Otherwise, for all sufficiently large t,

$$\begin{aligned} -\frac{d}{dt} a(t)^{1-\vartheta} &= (1-\vartheta) a(t)^{-\vartheta} \|\nabla f(\theta(t))\|^2 \\ &\geq C(1-\vartheta)\|\dot\theta(t)\|. \end{aligned}$$

Integrating to infinity gives the tail-length bound

$$\int_t^\infty \|\dot\theta(u)\|\, du \leq \frac{a(t)^{1-\vartheta}}{C(1-\vartheta)}. \tag{N.2}$$

The initial finite-time part also has finite length. Consequently the trajectory is Cauchy, converges in the compact closure, and its limit is critical by continuity. □

This theorem is not a statement about arbitrary noisy SGD paths. Discrete descent algorithms require sufficient-decrease and step-control conditions; these are treated in [1]. Boundedness alone does not turn arbitrary discrete updates into analytic gradient flow.

N.1.4 Qualitative Convergence and Quantitative Rates

Once the preceding local inequality applies,

$$a'(t) \le -C^2 a(t)^{2\vartheta}.$$

For $\vartheta = 1/2$, this implies exponential decrease of $a(t)$. For $\vartheta > 1/2$ and $a(t_0) > 0$,

$$a(t) \le \left[a(t_0)^{1-2\vartheta} + C^2(2\vartheta - 1)(t - t_0)\right]^{-1/(2\vartheta-1)}. \tag{N.3}$$

These are rates relative to the limiting critical value f_∞, which may exceed the global minimum. Entry time into the neighborhood, the constant C, and the exponent all matter for an efficiency claim.

For example, $f(x) = x^{2m}$ with $m \ge 2$ has, for $x_0 > 0$, the exact flow

$$x(t) = \left[x_0^{-(2m-2)} + 2m(2m-2)t\right]^{-1/(2m-2)}. \tag{N.4}$$

Differentiating verifies $\dot{x} = -2mx^{2m-1}$. Convergence is not exponential. Moreover, the analytic family $f_N(x) = \frac{1}{2}e^{-N}x^2$ takes time of order e^N to reduce $x(t)$ by a fixed factor. Analyticity does not furnish polynomially controlled conditioning constants.

N.2 Real-Analyticity of Deep GELU Networks

N.2.1 Analytic Nature of the GELU Activation

Definition 12 (GELU activation)**.** *The Gaussian Error Linear Unit [40] is*

$$\sigma(z) = \mathrm{GELU}(z) = z\Phi(z) = \frac{z}{2}\left[1 + \mathrm{erf}\left(\frac{z}{\sqrt{2}}\right)\right], \tag{N.5}$$

where Φ is the standard Gaussian cumulative distribution function and $\mathrm{erf}(u) = \frac{2}{\sqrt{\pi}}\int_0^u e^{-t^2}\,dt$ on the real line.

Proposition 38 (Analyticity of GELU)**.** *The exact GELU activation is real-analytic on $\mathbb{R}$ and has an entire extension to $\mathbb{C}$.*

Proof. The series

$$\mathrm{erf}(u) = \frac{2}{\sqrt{\pi}}\sum_{j=0}^{\infty}\frac{(-1)^j u^{2j+1}}{j!(2j+1)} \tag{N.6}$$

converges for all complex u. Thus the error function is entire. Products and compositions with polynomials preserve this property, which gives the assertion for (N.5). □

Remark 4 (Approximate GELU and monotonicity)**.** *The commonly used approximation*

$$\widetilde{\sigma}(z) = \frac{z}{2}\left[1 + \tanh\left(\sqrt{2/\pi}\,(z + 0.044715z^3)\right)\right]$$

is also real-analytic on $\mathbb{R}$; it is not an entire complex function. For exact GELU, writing φ for the standard Gaussian density,

$$\sigma'(z) = \Phi(z) + z\varphi(z), \qquad \sigma''(z) = (2 - z^2)\varphi(z). \tag{N.7}$$

In particular $\sigma'(0) = 1/2$ and $\sigma'(-1) < 0$. To verify the latter,

$$\Phi(-1) = \int_1^\infty \varphi(t)\,dt < \int_1^\infty t\varphi(t)\,dt = \varphi(1).$$

GELU is therefore not monotone. Analyticity alone does not make it satisfy the hypotheses of a landscape theorem requiring a monotone activation.

N.2.2 Network Architecture and Quadratic Loss

Consider a fixed finite network with affine maps, biases, and elementwise GELU activations:

$$\begin{aligned} h^{(0)} &= x, \\ a^{(\ell)} &= W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}, \qquad 1 \le \ell \le L-1, \\ h^{(\ell)} &= \sigma(a^{(\ell)}), \\ f(x;\theta) &= W^{(L)} h^{(L-1)} + b^{(L)}. \end{aligned} \tag{N.8}$$

The architecture is fixed while the real parameters vary.

Theorem 41 (Analyticity of the quadratic loss)**.** *For the network* (N.8)*, $f(x;\theta)$ is jointly real-analytic in inputs and parameters. For fixed training pairs $(x_i, y_i) \in \mathbb{R}^d \times \mathbb{R}^k$, the empirical loss*

$$\widehat{\mathcal{L}}(\theta) = \frac{1}{2n} \sum_{i=1}^{n} \|f(x_i;\theta) - y_i\|^2 \tag{N.9}$$

is real-analytic on $\mathbb{R}^p$.

Proof. Each affine expression is polynomial jointly in its parameters and its input variables. Induction over layers, using composition of analytic maps, proves the claim for f. A finite sum of squared residuals is a polynomial in those analytic outputs, which proves the loss assertion. □

The proof is elementary and does not use a theorem about absence of bad minima. Residual addition also preserves analyticity. Finite softmax operations are real-analytic because their real denominators are strictly positive. Normalization using a fixed positive numerical offset under a square root is real-analytic on its real domain as well. Hard routing, quantization, variable computational branches, or nonsmooth penalties must be analyzed separately. A property of the underlying real-valued map is not an assertion that a finite-precision implementation is analytic.

N.2.3 Smoothness Does Not Supply Uniform Conditioning

Analyticity gives derivatives of every order and bounds on each fixed compact parameter region. It does not bound Hessian norms uniformly over unbounded parameters or growing depth.

Example 1 (An analytic GELU loss with unbounded Hessian)**.** *Let $F(a,b) = a\sigma(b)$ and $\mathcal{L}(a,b) = \frac{1}{2}F(a,b)^2$. Since $\sigma(0) = 0$ and $\sigma'(0) = 1/2$,*

$$\frac{\partial^2 \mathcal{L}}{\partial b^2}(a,0) = \frac{a^2}{4}.$$

The loss is analytic, but its Hessian is not globally bounded. A bound on the admitted parameter region is needed for a uniform descent step.

Example 2 (Analytic activations and vanishing derivatives)**.** *For $h_0 = x$ and $h_{\ell+1} = \sigma(h_\ell)$, all intermediate states are zero at $x = 0$. Hence*

$$\frac{dh_H}{dx}(0) = \prod_{\ell=0}^{H-1} \sigma'(0) = 2^{-H}.$$

More generally, replacing each stage by $\sigma(ch_\ell)$ gives $(c/2)^H$. Smoothness is compatible with either attenuation or amplification.

Initialization schemes can improve signal propagation under specified assumptions [31, 39]. Maintaining an average variance is nevertheless different from controlling every relevant singular value of a Jacobian product throughout training.

N.2.4 A Quantitative Residual-Path Bound

Skip connections offer a positive mechanism when the residual derivatives are controlled. They are neither a universal necessity nor a sufficient condition by themselves.

Proposition 39 (Conditioned residual Jacobian products)**.** *Along a trajectory through equal-dimensional residual blocks $h_{\ell+1} = h_\ell + g_\ell(h_\ell)$, put $A_\ell = Dg_\ell(h_\ell)$. If $\|A_\ell\|_2 \le a_\ell < 1$, then for $P = D_{h_0} h_H = (I + A_{H-1}) \cdots (I + A_0)$,*

$$\prod_{\ell=0}^{H-1} (1 - a_\ell) \le \sigma_{\min}(P) \le \sigma_{\max}(P) \le \prod_{\ell=0}^{H-1} (1 + a_\ell). \tag{N.10}$$

If also $a_\ell \le a < 1$ and $\sum_\ell a_\ell \le C$, then

$$e^{-C/(1-a)} \le \sigma_{\min}(P) \le \sigma_{\max}(P) \le e^C. \tag{N.11}$$

Proof. The triangle and reverse-triangle inequalities give $(1 - a_\ell)\|v\| \le \|(I + A_\ell)v\| \le (1 + a_\ell)\|v\|$. Apply these bounds successively. Finally use $\log(1 + t) \le t$ and $\log(1 - t) \ge -t/(1 - a)$ for $0 \le t \le a$. □

For $A_\ell = cI$ with fixed $0 < c < 1$, the same architecture instead has $P = (1 + c)^H I$. Residual addition without scaling control still permits exponential amplification. Moreover, P is an activation-propagation Jacobian, not the training-prediction Jacobian $D_\theta F$. Good conditioning of the former does not imply full row rank of the latter.

N.2.5 A Positive Convergence Result with Full Weight Decay

Corollary 8 (Analytic gradient flow with a coercive quadratic penalty)**.** *For* (N.9)*, let*

$$U(\theta) = \widehat{\mathcal{L}}(\theta) + \frac{\gamma}{2}\|\theta\|^2, \qquad \gamma > 0,$$

where the penalty includes all trainable parameters. Every gradient-flow trajectory of U, from any finite initial parameter, exists globally, has finite length, and converges to a critical point of U.

Proof. The function U is analytic and nonnegative. Along its gradient flow, U decreases, so

$$\|\theta(t)\|^2 \le \frac{2U(\theta(0))}{\gamma}.$$

The trajectory remains in a compact sublevel set. The smooth vector field has no finite-time escape there, giving global existence. Theorem 40 then applies. □

The conclusion is stationarity of the penalized objective, not global optimality or interpolation. Even the convex example $U(a) = \frac{1}{2}(a-1)^2 + \frac{\gamma}{2}a^2$ has minimizer $a = 1/(1+\gamma)$ and nonzero unregularized residual.

A product penalty need not provide the same compactness. For example, $\Omega(a,b) = a^2b^2$ is bounded on the unbounded curve $(a,b) = (t,t^{-1})$. The analytic residual loss $\frac{1}{2}(ab-1)^2$ is zero on that curve, so adding this product penalty still leaves an unbounded sublevel set. Products of squared Frobenius norms are analytic, but not generally coercive; unsquared norm products can additionally fail analyticity at zero factors. Statistical norm control and trajectory confinement are separate requirements.

The preceding corollary concerns continuous gradient flow of the full penalized objective. It does not imply convergence of fixed-step SGD to the same parameter. Section N.5.4 shows that a critical point of this objective can have nonzero minibatch covariance; Section N.5.7 states the additional common-equilibrium condition needed by a finite point limit of the specified SGD recursion.

N.3 Desingularization via Hironaka's Theorem

At a degenerate minimum, a quadratic Taylor approximation may fail to capture the local geometry. The analytic Taylor expansion itself remains valid. Resolution of singularities provides another way to study the zero set and nearby weighted volumes. It is not an ordinary nonsingular coordinate transformation of the original loss landscape.

N.3.1 A Local Normal-Crossing Form

The following is a local analytic consequence of resolution of singularities and principalization. Hironaka's work supplies the foundational resolution theorem; analytic formulations and constructions are developed, for example, by Bierstone and Milman [42, 12].

Theorem 42 (Local resolution of a nonnegative analytic loss; quoted result)**.** *Let $L \geq 0$ be a nonzero real-analytic function on a connected open neighborhood of a compact region in $\mathbb{R}^p$. After localizing near its zero set, one can resolve the analytic divisor by a proper analytic modification $\pi : \widetilde{U} \to U$, an isomorphism away from the zero set, such that in suitable local coordinates*

$$L(\pi(u)) = a(u) \prod_{j=1}^{p} u_j^{2k_j}, \tag{N.12}$$

$$|\det D\pi(u)| = b(u) \prod_{j=1}^{p} |u_j|^{h_j}, \tag{N.13}$$

where k_j, h_j are nonnegative integers and the unit factors a, b are positive and nonvanishing on suitably chosen charts. A compact localized integral is handled by finitely many contributing charts. When a domain boundary is relevant, it is resolved as part of the localization.

This theorem is invoked, not reproved here. Nonnegativity permits even powers in the loss monomial. For a positive-loss local minimum one applies the local statement to $L - L_*$ where this difference is nonnegative. The modification is not asserted to be a diffeomorphism through the exceptional divisor, nor is every critical point necessarily blown up.

N.3.2 Weighted Volume and the Real Log Canonical Threshold

For a nonnegative analytic loss, take a compact localized region U and a smooth positive weight φ bounded above and below there. The integral of interest is

$$Z_U(T) = \int_U e^{-L(\theta)/T} \varphi(\theta)\, d\theta, \qquad T > 0. \tag{N.14}$$

Assume the analytic, semianalytic-domain, and localization conditions under which the standard resolution-based asymptotic applies [112, 113, 114]. The loss is not identically zero on an open part of U, and its zero set meets the localized support.

In a contributing chart, the transformed density includes both the loss and the volume element:

$$\exp\left(-\frac{a(u)u^{2k}}{T}\right) \widetilde{b}(u)|u|^h\, du,$$

where $\widetilde{b}$ includes $\varphi \circ \pi$, the Jacobian unit, and localization factors. On charts that contribute at the relevant divisor, these factors do not remove the leading singularity.

One way to see the exponent is to examine local integrability of L^{-z} for real $z \geq 0$. The monomial integral has factors

$$\int_0^\epsilon u_j^{h_j - 2k_j z}\, du_j,$$

which are finite precisely when $h_j - 2k_j z > -1$. Thus each divisor with $k_j > 0$ supplies a candidate threshold

$$\lambda_j = \frac{h_j + 1}{2k_j}. \tag{N.15}$$

The smallest threshold among relevant charts determines the RLCT λ. The associated pole multiplicity m records the maximal number of minimizing divisors meeting in a contributing resolved chart. The standard asymptotic is

$$\boxed{Z_U(T) = C_U T^\lambda \left(\log \frac{1}{T}\right)^{m-1} (1 + o(1)), \qquad C_U > 0, \quad T \downarrow 0.} \tag{N.16}$$

The full asymptotic is an invoked singular-integral result; the preceding integrability calculation explains why the ratios in (N.15) enter it.

Both k_j and h_j matter. A weight that vanishes can modify the latter exponents and hence the threshold; it cannot be ignored. With the same minimum loss and admissible positive weights, smaller λ gives a larger low-temperature contribution. Logarithmic multiplicity and finite prefactors distinguish cases with equal λ.

Two elementary checks. For $L(x) = x^{2r}$ on $[-1, 1]$ with unit weight, scaling $x = T^{1/(2r)}v$ gives

$$Z(T) \sim \left(\int_{\mathbb{R}} e^{-v^{2r}}\, dv\right) T^{1/(2r)}.$$

Thus $\lambda = 1/(2r)$ and $m = 1$. The zero set remains a single point for every r, while the volume exponent changes. RLCT is not simply half the topological codimension.

For the crossing singularity $L(x, y) = x^2y^2$ on $[-1, 1]^2$, changing variables in a positive quadrant gives

$$\begin{aligned} Z(T) &= 4 \int_0^1 e^{-z^2/T} \log(1/z)\, dz \\ &\sim \sqrt{\pi}\, T^{1/2} \log(1/T). \end{aligned}$$

The substitution $z = \sqrt{T}\, v$ verifies the last expression. Here $\lambda = 1/2$ and $m = 2$: the logarithm detects the intersection of the two zero-loss branches.

N.3.3 What Resolution Does Not Say About Gradient Trajectories

The gradient-inequality exponent ϑ in (N.1) and the volume exponent λ in (N.16) answer different questions. The former bounds a gradient relative to a function-value difference in the given metric; the latter measures integrability and weighted small-loss volume. No universal formula for ϑ follows from the loss monomial alone.

Even where π is locally invertible, pulling back Euclidean gradient flow changes the metric. If $\theta = \pi(u)$ and $\dot\theta = -\nabla_\theta L$, then

$$(D\pi(u)^\top D\pi(u))\dot u = -\nabla_u (L \circ \pi)(u). \tag{N.17}$$

This is obtained by multiplying $D\pi\dot u = -\nabla_\theta L$ by $D\pi^\top$. It is not generally Euclidean gradient flow of the resolved monomial. At the exceptional divisor the pullback metric can degenerate. Consequently, a monomial normal form does not automatically prove a finite-time optimization bound or a statement about approach directions.

Resolution-based methods are powerful for singular learning theory, but an RLCT of a localized empirical loss is not by itself a theorem about the population error of an SGD-selected parameter. Bayesian learning results use their own likelihood, population-distribution, prior, and asymptotic assumptions [113, 114].

N.4 Overparameterized Deep Networks: Parameters and Fitting Constraints

The relevant comparison is between p parameters and $M = nk$ scalar sample outputs, not between parameters and examples without regard to output dimension. Having $p > M$ permits redundant fitting directions, but does not by itself ensure that interpolation exists or is regular. Throughout the local geometric statements, interpolation is an explicit assumption.

N.4.1 The Regular Interpolating Zero-Loss Manifold

Stack the training predictions in $F(\theta) \in \mathbb{R}^M$, put $e(\theta) = F(\theta) - y$, and define

$$\widehat{\mathcal{L}}(\theta) = \frac{1}{2n}\|e(\theta)\|^2, \qquad J(\theta) = DF(\theta) \in \mathbb{R}^{M\times p}.$$

The set

$$\mathcal{M} = F^{-1}(y) \tag{N.18}$$

may be empty. If it is nonempty, its points are global minimizers of this nonnegative squared loss, regardless of other positive-loss local minima.

Proposition 40 (Regular fitting geometry)**.** *Suppose $F(\theta_*) = y$ and J has constant rank q on a neighborhood of θ_*. Then $\mathcal{M}$ is locally a smooth manifold with*

$$\dim \mathcal{M} = p - q, \qquad T_{\theta_*}\mathcal{M} = \ker J(\theta_*). \tag{N.19}$$

If F is analytic, this is a local analytic manifold. In particular, full row rank $q = M$ at an interpolating point gives dimension $p - M$ locally.

Proof. By the constant-rank theorem, local coordinates in domain and codomain make F a projection onto q coordinates followed by an inclusion. Its level set fixes exactly those q coordinates and leaves $p - q$ free. Differentiating the level-set condition identifies its tangent space. Full row rank persists on a sufficiently small neighborhood by continuity of a nonzero maximal minor. □

This is the regular-value mechanism behind manifold descriptions of interpolating networks; see also [22]. It is not a global claim that every point of $\mathcal{M}$ is regular, or that $\mathcal{M}$ is connected. Distinct points in the level set agree on the sample but can represent different functions elsewhere.

N.4.2 Hessian Spectrum and Neural Tangent Kernel Alignment

Differentiating the empirical loss yields

$$\nabla\widehat{\mathcal{L}}(\theta) = \frac{1}{n}J(\theta)^\top e(\theta), \tag{N.20}$$

$$\nabla^2\widehat{\mathcal{L}}(\theta) = \frac{1}{n}J(\theta)^\top J(\theta) + \frac{1}{n}\sum_{a=1}^{M} e_a(\theta)\nabla^2 F_a(\theta). \tag{N.21}$$

Proposition 41 (Hessian nullity and normalized kernel duality)**.** *At any twice-differentiable interpolating parameter θ_*, let $q = \operatorname{rank} J(\theta_*)$. Then*

$$H_* := \nabla^2\widehat{\mathcal{L}}(\theta_*) = \frac{1}{n}J(\theta_*)^\top J(\theta_*), \quad \ker H_* = \ker J(\theta_*), \quad \dim\ker H_* = p - q. \tag{N.22}$$

For

$$\Theta(\theta) = J(\theta)J(\theta)^\top, \qquad K(\theta) = \frac{1}{n}\Theta(\theta), \tag{N.23}$$

the positive eigenvalues, counted with multiplicity, satisfy

$$\lambda_j^+(H_*) = \lambda_j^+(K(\theta_*)) = \frac{1}{n}\lambda_j^+(\Theta(\theta_*)). \tag{N.24}$$

Proof. At interpolation the residual term in (N.21) vanishes. The identity $v^\top J^\top J v = \|Jv\|^2$ gives the equality of kernels and hence nullity. The positive eigenvalues of $J^\top J$ and $JJ^\top$ are the squares of the same nonzero singular values of J, proving the normalized spectral statement. □

In particular, when $p > M$, there are at least $p - M$ Hessian zero modes. This is an exact interpolation identity, not an empirical approximation. Empirical investigations of near-zero Hessian bulk are related but concern additional regimes [96]. Identifying these zero modes with a smooth tangent space requires the local rank assumption in Proposition 40.

The Hessian of the regularized objective. The interpolation identity above concerns the *data loss*. For $U_\gamma = \widehat{\mathcal{L}} + \gamma\Omega$, with a C^2 penalty,

$$\nabla^2 U_\gamma(\theta) = \frac{1}{n} J(\theta)^\top J(\theta) + \frac{1}{n}\sum_{a=1}^{M} e_a(\theta)\nabla^2 F_a(\theta) + \gamma\nabla^2\Omega(\theta). \tag{N.25}$$

This follows by differentiating the penalty and adding it to (N.21). At a regularized stationary point with nonzero residuals, neither additional term can be discarded without a separate estimate. Even at an exact data interpolator, the penalty Hessian remains. Thus the data-loss nullity and normalized kernel duality must not be asserted automatically for the full penalized Hessian.

A singular counterexample. Let $n = k = 1$, $F(\theta) = \sum_{j=1}^{p} \theta_j^2$, and $y = 0$. Then

$$\widehat{\mathcal{L}}(\theta) = \frac{1}{2}\left(\sum_j \theta_j^2\right)^2, \qquad \mathcal{M} = \{0\}.$$

At 0, both J and the Hessian vanish. The Hessian nullity is p, but the zero-loss set has dimension 0. Thus even an analytic, overparameterized, realizable problem need not have a positive-dimensional interpolation manifold at a singular solution.

N.4.3 The Scope of Neural-Network Landscape Results

A theorem about a particular activation or network cannot be extended merely by replacing its hypotheses with "smooth and wide." For example, Nguyen and Hein's 2017 results use assumptions including distinct inputs, a strictly increasing analytic activation satisfying additional growth conditions, a sufficiently wide layer, and specified rank and partial nondegeneracy conditions on critical points. Their Theorem 3.8 does not exclude all degenerate or low-rank suboptimal local minima [81]. Equation (N.7) shows why GELU does not satisfy that monotonicity assumption.

Their CNN results address a separate, specified architecture and feature-rank setting [82]. Connected-sublevel-set theorems also have their own architectural and activation hypotheses [80]. A connected sublevel set is not automatically a smooth connected manifold of zero-loss parameters, nor does connectedness provide a convergence rate for a particular gradient method.

In the opposite direction, suboptimal local minima have been constructed in wide networks for broad classes of smooth nonlinear activations [26]. This does not mean that every such minimum has large probability under a practical initialization. It does rule out an unqualified principle that width and activation smoothness eliminate all suboptimal minima.

N.4.4 An Analytic Least-Squares Counterexample to Almost-Sure Recovery

The following example isolates the invalid implication without relying on an architectural special case.

Proposition 42 (Many parameters and almost-everywhere rank are not enough)**.** *For every $p \geq 2$, define a scalar residual and loss by*

$$r(\theta) = \theta_1^3 - 3\theta_1 + 3 + \sum_{j=2}^{p} \theta_j^2, \qquad L(\theta) = \frac{1}{2} r(\theta)^2. \tag{N.26}$$

Then L is analytic, depends on all p parameters, and has a nonempty regular zero-loss manifold of dimension $p-1$. The residual Jacobian has full row rank almost everywhere. Nevertheless, $\theta_+ = (1, 0, \ldots, 0)$ is a strict positive-loss local minimum whose basin under gradient flow contains an open neighborhood.

Proof. With all other coordinates zero, $r(-3,0,\dots,0) = -15$ and $r(-2,0,\dots,0) = 1$, so a zero exists by continuity. The gradient

$$\nabla r = (3\theta_1^2 - 3, 2\theta_2, \dots, 2\theta_p)$$

vanishes only at $\theta_+ = (1,0,\dots,0)$ and $\theta_- = (-1,0,\dots,0)$. Their residuals are 1 and 5, respectively. Thus ∇r is nonzero at every zero of r, proving regularity and dimension $p-1$. It is nonzero except at those two points, proving the almost-everywhere rank assertion.

At θ_+,

$$L(\theta_+) = \frac{1}{2}, \qquad \nabla L(\theta_+) = 0, \qquad \nabla^2 L(\theta_+) = \operatorname{diag}(6,2,\dots,2) \succ 0.$$

Continuity gives a small ball on which $\nabla^2 L \succeq cI$ for some $c > 0$. Along gradient flow in this ball,

$$\frac{d}{dt}\|\theta(t) - \theta_+\|^2 = -2\langle \theta(t) - \theta_+, \nabla L(\theta(t))\rangle \le -2c\|\theta(t) - \theta_+\|^2.$$

A smaller closed ball is forward invariant, and its trajectories converge to θ_+. Its interior is an open subset of the suboptimal basin. □

The example concerns general analytic least-squares geometry, not an assertion that a prescribed unrestricted GELU network has exactly this parameterization. It disproves the proposed inference from analyticity, parameter count, regular interpolation, and almost-everywhere prediction rank alone. An exceptional set of critical parameters can have measure zero while its attracting basin has positive measure.

N.4.5 What a Quantitative Prediction-Jacobian Bound Does Prove

A positive convergence result requires information along the trajectory, not only at a zero-loss point. For the empirical loss in (N.9), gradient flow satisfies

$$\dot{e}(t) = -K(\theta(t))e(t). \tag{N.27}$$

Proposition 43 (Conditional residual contraction)**.** *Suppose a gradient-flow solution exists on $[0,T]$ and*

$$e(t)^\top K(\theta(t))e(t) \ge \mu\|e(t)\|^2, \qquad \mu > 0, \tag{N.28}$$

throughout that interval. Then

$$\widehat{\mathcal{L}}(\theta(t)) \le \widehat{\mathcal{L}}(\theta(0))e^{-2\mu t}, \qquad 0 \le t \le T. \tag{N.29}$$

If the solution is defined for all $t \ge 0$ and the same bound holds throughout, it has finite length and converges to a finite interpolating parameter. For this last conclusion assume that F is C^1 on all of $\mathbb{R}^p$ and the loss has the local smoothness required for the flow.

Proof. The loss derivative is

$$\frac{d}{dt}\widehat{\mathcal{L}}(\theta(t)) = -\|\nabla\widehat{\mathcal{L}}\|^2 = -\frac{1}{n}e^\top K e \le -2\mu\widehat{\mathcal{L}}.$$

Integration gives (N.29). In the all-time case, write $\ell(t) = \widehat{\mathcal{L}}(\theta(t))$. Before reaching zero, $\|\nabla\widehat{\mathcal{L}}\|^2 \ge 2\mu\ell$ implies

$$-\frac{d}{dt}\sqrt{\ell(t)} \ge \sqrt{\mu/2}\,\|\dot{\theta}(t)\|.$$

Therefore

$$\int_t^\infty \|\dot\theta(u)\|\,du \le \sqrt{2/\mu}\sqrt{\ell(t)}. \tag{N.30}$$

The trajectory converges in $\mathbb{R}^p$, and continuity together with $\ell(t)\to 0$ gives interpolation at the limit. If zero is reached in finite time, the gradient is zero thereafter. □

This result does not need analyticity. It uses a stronger quantitative condition and gives convergence to the specified zero residual, whereas Theorem 40 gives convergence to an unspecified critical point. A lower bound only on the nonzero eigenvalues of K is insufficient unless the actual residual belongs to the corresponding controlled subspace.

An NTK-style sufficient condition. If

$$K(\theta(0)) \succeq 2\mu I_M, \qquad \sup_{0\le t\le T}\|K(\theta(t))-K(\theta(0))\|_{\mathrm{op}} \le \mu, \tag{N.31}$$

then $K(\theta(t))\succeq \mu I_M$ on $[0,T]$. Indeed, for any vector v,

$$v^\top K(\theta(t))v \ge v^\top K(\theta(0))v - \|K(\theta(t))-K(\theta(0))\|_{\mathrm{op}}\|v\|^2 \ge \mu\|v\|^2.$$

This gives one route to the conditioning premise. The nearly constant NTK regime depends on parameterization, initialization, scaling, data, and the time interval being controlled [47]. Infinite width alone is not synonymous with a frozen kernel; different scalings permit infinite-width feature learning [118].

Finite-step and computational qualifications. If discrete gradient-descent iterates remain in a region satisfying (N.28), and the loss gradient is Λ-Lipschitz on every required update segment, then a step $0<\alpha\le 1/\Lambda$, also chosen with $\alpha\mu\le 1$, gives

$$\widehat{\mathcal{L}}(\theta_{j+1}) \le \widehat{\mathcal{L}}(\theta_j) - \frac{\alpha}{2}\|\nabla\widehat{\mathcal{L}}(\theta_j)\|^2 \le (1-\alpha\mu)\widehat{\mathcal{L}}(\theta_j).$$

This follows from the descent lemma. Invariance, initialization probability, the scales of μ^{-1} and Λ, and the cost and precision of an update must be controlled before calling the procedure efficient. The unregularized identity does not transfer unchanged to $\widehat{\mathcal{L}}+\gamma\Omega$ or to Adam or noisy SGD.

The residual dynamics must use the trained objective. For differentiable Ω, along gradient flow of $U_\gamma=\widehat{\mathcal{L}}+\gamma\Omega$ the chain rule instead gives

$$\begin{aligned} \dot e &= -Ke - \gamma J\nabla\Omega, \\ \frac{d}{dt}\widehat{\mathcal{L}}(\theta(t)) &= -\frac{1}{n}e^\top Ke - \frac{\gamma}{n}e^\top J\nabla\Omega. \end{aligned} \tag{N.32}$$

The penalty can oppose or reinforce data-loss descent. The unregularized contraction proposition is retained with its original hypotheses; an application to regularized training must control this extra contribution or establish good fit by another argument. Stationarity of U_γ means cancellation of its full gradient, not necessarily $e=0$.

Table N.2: Geometric and dynamical conclusions depend on hypotheses, not on an underparameterized/overparameterized dichotomy. Here $q = \operatorname{rank} J$.

@>p0.29>X >X@

Hypothesis	Justified conclusion	Not implied
Stationarity and positive-definite Hessian	Strict isolated local minimum	Global optimality
Interpolation and locally constant rank q	Local manifold of dimension $p-q$	Large basin or global connectedness
Twice-differentiable interpolation	$H = J^\top J/n$ and nullity $p-q$	A manifold at a singular point
Analytic objective and precompact gradient flow	Finite length and convergence to a critical point	Global minimum or uniform efficient rate
Uniform conditioning of the actual residual	Exponential empirical-loss decay	Population accuracy or teacher identification
Specified local Gibbs or weighted integral	Regular or singular volume asymptotics	Finite-time SGD access or universal stationary law

N.5 Basins of Attraction for Degenerate Manifolds

N.5.1 Foliation and Fiber Geometry: Local Conditions Matter

For a closed set $\mathcal{M}$ of minima, distinguish the basin defined by approach to the set,

$$\mathcal{B}(\mathcal{M}) = \{\theta_0 : \theta(t) \text{ exists for all } t \geq 0,\ \operatorname{dist}(\theta(t), \mathcal{M}) \to 0\}, \tag{N.33}$$

from the set of initial conditions whose trajectories converge to a particular point $\theta_* \in \mathcal{M}$:

$$W^s(\theta_*) = \{\theta_0 : \theta(t) \to \theta_*\}.$$

When every trajectory under consideration converges to a point of $\mathcal{M}$, there is the set-theoretic partition

$$\mathcal{B}(\mathcal{M}) = \bigsqcup_{\theta_* \in \mathcal{M}} W^s(\theta_*). \tag{N.34}$$

Writing this partition does not prove that its members are smooth fibers or that the entire basin is a fiber bundle.

A standard sufficient setting for a local stable foliation is a compact smooth manifold of equilibria with a uniformly positive normal Hessian and Hessian kernel equal to the tangent space. This is a normally attracting, Morse–Bott minimum manifold. The local stable-manifold and foliation theory then supplies stable leaves of dimension equal to the codimension q, tangent at θ_* to the normal space [43]. Smoothness is understood at the regularity level allowed by that theory. This is a local statement under normal attraction, not a consequence of analyticity alone.

In Euclidean gradient coordinates the stable linear space at the minimum is orthogonal to its tangent space. The nonlinear stable leaves can be curved; trajectories away from the limit need not meet a fixed normal line or stay perpendicular to the manifold.

Example 3 (Stable fibers not counted by Hessian rank)**.** *For $L(x, y) = x^4$,*

$$\mathcal{M} = \{x = 0\}, \qquad \nabla^2 L|_{\mathcal{M}} = 0, \qquad \dot{x} = -4x^3, \quad \dot{y} = 0.$$

For $x_0 \neq 0$, the solution is $x(t) = x_0/\sqrt{1 + 8x_0^2 t}$ and $y(t) = y_0$. Hence each point $(0, y_0)$ has the one-dimensional attracting fiber $\{(x, y_0) : x \in \mathbb{R}\}$, even though the Hessian rank is zero. Normal quadratic nondegeneracy cannot be omitted from the fiber-dimension claim.

N.5.2 Measure, Saddles, and Limits of Almost-Sure Claims

A probability of convergence is relative to an initialization law π_0: the relevant quantity is $\pi_0(\mathcal{B}(\mathcal{M}))$. Raw dimension or unspecified Euclidean volume does not determine it. Proposition 42 gives an open suboptimal basin even though the exceptional rank-deficient set has measure zero.

Strict-saddle avoidance results allow saddles to exist. Under their smoothness, step-size, initialization, and critical-set hypotheses, they show that convergence to such saddles has probability zero [60]. They do not eliminate suboptimal local minima, cover arbitrary non-strict saddles, or imply that all trajectories are bounded. Avoidance also differs from a quantitative escape-time bound.

Thus an almost-sure global-convergence argument requires both a valid convergence mechanism and a reason that all possible attracting limits are global. Neither follows from analyticity and width alone. A finite-time learning theorem additionally needs resource bounds.

N.5.3 Exact Minibatch Noise and the Interpolation Limit

We first state the sampling identity at a fixed parameter, then distinguish exact data interpolation from stationarity of a regularized objective. All sampling statements in this subsection condition on the current parameter and a fixed finite training sample.

For vector-valued outputs, let

$$e_i(\theta) = f(x_i;\theta) - y_i, \quad J_i(\theta) = D_\theta f(x_i;\theta), \quad g_i(\theta) = J_i(\theta)^\top e_i(\theta).$$

Then g_i is the gradient of the per-example half-squared loss, and $\bar{g} = n^{-1}\sum_i g_i = \nabla\widehat{\mathcal{L}}$.

Proposition 44 (Minibatch covariance and vanishing noise at interpolation)**.** *Sample B indices independently and uniformly with replacement from $\{1,\dots,n\}$ and let $\widehat{g}_B$ be their average gradient. Conditional on the current parameter,*

$$\begin{aligned} \mathbb{E}[\widehat{g}_B \mid \theta] &= \bar{g}(\theta), \\ \mathrm{Cov}(\widehat{g}_B \mid \theta) &= \frac{1}{B}C(\theta), \\ C(\theta) &= \frac{1}{n}\sum_{i=1}^{n} g_i(\theta)g_i(\theta)^\top - \bar{g}(\theta)\bar{g}(\theta)^\top. \end{aligned} \tag{N.35}$$

At exact squared-loss interpolation on these fixed examples, every g_i vanishes and hence $C(\theta_*) = 0$.

Proof. A single sampled gradient has mean $\bar{g}$ and covariance C. Conditional independence divides this covariance by B upon averaging. At interpolation, $e_i = 0$ for each example, so $g_i = J_i^\top e_i = 0$. □

For a uniform B-element subset $\mathcal{B} \subseteq [n]$, sampled without replacement, put

$$\widehat{g}_{\mathcal{B}} = \frac{1}{B}\sum_{i\in\mathcal{B}} g_i, \qquad c_{n,B} = \frac{n-B}{B(n-1)}, \qquad n > 1.$$

Then

$$\mathbb{E}_{\mathcal{B}}[\widehat{g}_{\mathcal{B}} \mid \theta] = \bar{g}, \qquad \mathrm{Cov}_{\mathcal{B}}(\widehat{g}_{\mathcal{B}} \mid \theta) = c_{n,B}C(\theta). \tag{N.36}$$

Here and below $[n] = \{1,\dots,n\}$. To prove the correction, set $h_i = g_i - \bar{g}$ and let I_i indicate membership in $\mathcal{B}$. Since $\sum_i h_i = 0$,

$$\sum_{i\neq j} h_i h_j^\top = -\sum_i h_i h_i^\top.$$

Using $\mathbb{E}I_i = B/n$ and $\mathbb{E}I_iI_j = B(B-1)/(n(n-1))$ for $i \neq j$, expansion gives

$$\mathbb{E}\left[\left(\frac{1}{B}\sum_i I_i h_i\right)\left(\frac{1}{B}\sum_j I_j h_j\right)^\top\right] = \frac{n-B}{Bn(n-1)}\sum_i h_i h_i^\top$$
$$= c_{n,B}C.$$

For $B = n$ this covariance is zero regardless of the individual gradients. For $1 \leq B < n$, it vanishes exactly when all g_i coincide. Independent sampling with replacement instead has factor $1/B$, even when $B = n$.

For an update with learning rate α, the update covariance is α^2 times the gradient-estimate covariance. Under a justified small-step diffusion approximation with time increment α, the covariance rate is $\alpha c_{n,B}C$, or $\alpha C/B$ with replacement. These three noise scales must not be confused.

The fixed-example assumption matters. Training-mode batch normalization or another batch-dependent forward operation need not yield gradients that are averages of fixed per-example vectors. Its actual batch-update covariance must be analyzed instead.

This exact calculation differs from identifying covariance with curvature. At interpolation,

$$C(\theta_*) = 0, \qquad H_* = \frac{1}{n}J(\theta_*)^\top J(\theta_*) \quad \text{may be nonzero.}$$

There is no general positive scalar proportionality between covariance and Hessian in this setting. The statement is specifically about the data gradients *at an exact interpolator*; it does not say that sampling noise vanishes at every minimum of a regularized objective. Positive regularization can move the stationary balance away from exact interpolation, where the same fixed-example sampling produces persistent fluctuations. No external source of randomness beyond minibatch selection is needed for that phenomenon.

Injected noise, changing examples or augmentations, dropout, and transient optimizer state can also change the dynamics. Each requires its stated model. The regularized case below already explains nonzero late-stage sampling covariance without invoking any of those additions.

N.5.4 Regularized Stationarity and Common Minibatch Equilibria

The covariance calculation extends beyond squared loss. Let each ℓ_i and the deterministic penalty Ω be differentiable at the parameters under consideration, and define

$$\begin{aligned} U_\gamma(\theta) &= \frac{1}{n}\sum_{i=1}^n \ell_i(\theta) + \gamma\Omega(\theta), \qquad \gamma > 0, \\ g_i(\theta) &= \nabla\ell_i(\theta), \qquad \bar{g}(\theta) = \frac{1}{n}\sum_i g_i(\theta), \\ G_{\mathcal{B},\gamma}(\theta) &= \frac{1}{B}\sum_{i\in\mathcal{B}} g_i(\theta) + \gamma\nabla\Omega(\theta). \end{aligned} \tag{N.37}$$

The penalty contribution is identical for every batch at the fixed parameter. Therefore, for uniform subset sampling,

$$\begin{aligned} \mathbb{E}_{\mathcal{B}}[G_{\mathcal{B},\gamma} \mid \theta] &= \nabla U_\gamma(\theta), \\ \mathrm{Cov}_{\mathcal{B}}(G_{\mathcal{B},\gamma} \mid \theta) &= c_{n,B}C(\theta). \end{aligned} \tag{N.38}$$

The same formulas hold with factor $1/B$ for independent draws with replacement. These are ordinary penalized gradient updates, not a claim that every adaptive or decoupled weight-decay implementation has this form.

At exact squared-loss interpolation, $g_i(\theta_*) = 0$ for every example. All regularized batch gradients then equal $\gamma\nabla\Omega(\theta_*)$. Their covariance is zero, but their common mean is nonzero unless the penalty gradient vanishes. Such an interpolator need not be stationary for U_γ.

At a regularized stationary point θ_γ, instead,

$$\bar{g}(\theta_\gamma) + \gamma\nabla\Omega(\theta_\gamma) = 0, \qquad G_{\mathcal{B},\gamma}(\theta_\gamma) = \frac{1}{B}\sum_{i\in\mathcal{B}}(g_i - \bar{g})(\theta_\gamma). \tag{N.39}$$

Only the *average* data gradient cancels the regularizer. Individual examples can continue to demand different updates.

Proposition 45 (Noise at a regularized stationary point)**.** *Let $n > 1$, $1 \le B < n$, and let $\mathcal{B}$ be a uniformly sampled B-element subset of $[n]$. Suppose all ℓ_i and Ω are differentiable at a stationary point θ_γ of* (N.37)*. Then*

$$\boxed{\mathbb{E}_{\mathcal{B}}\|G_{\mathcal{B},\gamma}(\theta_\gamma)\|^2 = \frac{n-B}{B(n-1)}\frac{1}{n}\sum_{i=1}^{n}\|g_i(\theta_\gamma) - \bar{g}(\theta_\gamma)\|^2.} \tag{N.40}$$

At this point the following conditions are equivalent:

1. *the sampling covariance of $G_{\mathcal{B},\gamma}$ is zero;*
2. *all per-example data gradients $g_i(\theta_\gamma)$ are equal;*
3. *$G_{\mathcal{B},\gamma}(\theta_\gamma) = 0$ for every admissible batch;*
4. *$g_i(\theta_\gamma) + \gamma\nabla\Omega(\theta_\gamma) = 0$ for every i.*

Thus unequal sample gradients give strictly positive minibatch fluctuation energy at a zero of the full regularized gradient, including a regularized local or global minimum.

Proof. At stationarity the conditional mean in (N.38) is zero. Taking the trace of its covariance proves (N.40). Since $c_{n,B} > 0$, this trace vanishes if and only if every $g_i = \bar{g}$. The stationarity equation then gives $g_i = -\gamma\nabla\Omega$ for all i, so every regularized batch gradient is zero. Conversely, if every batch gradient is zero, its second moment and covariance vanish.

One may also see the batchwise equivalence by exchanging examples. For any $i \neq j$, choose a set $A \subseteq [n] \setminus \{i, j\}$ of size $B - 1$, possible because $B < n$. The gradients for $A \cup \{i\}$ and $A \cup \{j\}$ differ by $(g_i - g_j)/B$; their simultaneous vanishing forces $g_i = g_j$. This also covers $B = 1$, when A is empty. □

This is the general covariance result used in Chapter 7, Proposition 5. It requires no analytic activation, nonconvexity, or landscape assumption. Its role is to identify the correct stochastic endpoint.

At an arbitrary parameter the orthogonal mean–fluctuation decomposition gives the exact identity

$$\mathbb{E}_{\mathcal{B}}\|G_{\mathcal{B},\gamma}(\theta)\|^2 = \|\nabla U_\gamma(\theta)\|^2 + c_{n,B}\,\mathrm{Tr}\,C(\theta). \tag{N.41}$$

A small full gradient controls only the first term. Gradient descent on U_γ is not, by definition, minimization of the second moment in (N.41).

Table N.3: Distinct endpoints for ordinary regularized minibatch gradients. The sample is fixed and $1 \leq B < n$ with uniform subset sampling.

@>X >p0.25 >p0.23@

Parameter condition	Mean batch gradient	Sampling covariance
Exact squared-loss interpolation, $\nabla\Omega \neq 0$ $\gamma\nabla\Omega \neq 0$	Zero	
Regularized stationarity with unequal sample gradients	Zero	Nonzero
A common zero of every regularized batch gradient	Zero	Zero

A deterministic penalty does not add conditional randomness at a fixed parameter. It changes where stationarity can occur and can sustain heterogeneous, nonzero sample gradients there. Positive regularization alone is not a universal guarantee of nonzero covariance: identical sample gradients are possible in special configurations. The proposition states the exact condition and supplies no lower bound depending only on γ.

N.5.5 A Strictly Convex Example with Persistent Minibatch Noise

Example 4 (A regularized global minimum need not be absorbing)**.** *Let $F_\theta(x) = \theta x$ and take the two examples $(1, 1)$ and $(2, 2)$. For $\gamma > 0$, the objective*

$$U_\gamma(\theta) = \frac{1}{4}\left[(\theta - 1)^2 + (2\theta - 2)^2\right] + \frac{\gamma}{2}\theta^2 = \frac{5}{4}(\theta - 1)^2 + \frac{\gamma}{2}\theta^2$$

is analytic and strictly convex. Its unique global minimizer is $\theta_\gamma = 5/(5 + 2\gamma)$, but its singleton-batch gradient variance is strictly positive.

Proof. Differentiation gives $U'_\gamma(\theta) = \frac{5}{2}(\theta - 1) + \gamma\theta$ and $U''_\gamma = 5/2 + \gamma > 0$, proving the minimizer assertion. At this point,

$$G_{\{1\},\gamma}(\theta_\gamma) = (\theta_\gamma - 1) + \gamma\theta_\gamma = \frac{3\gamma}{5 + 2\gamma},$$
$$G_{\{2\},\gamma}(\theta_\gamma) = 4(\theta_\gamma - 1) + \gamma\theta_\gamma = -\frac{3\gamma}{5 + 2\gamma}.$$

Both batches have probability $1/2$. Their mean is zero and

$$\mathrm{Var}_{\mathcal{B}}(G_{\mathcal{B},\gamma}(\theta_\gamma)) = \frac{9\gamma^2}{(5 + 2\gamma)^2} > 0. \tag{N.42}$$

□

Without regularization, the same dataset interpolates at $\theta = 1$. At that parameter with positive γ, every batch gradient equals γ, so the sampling variance is zero but the mean is not. Conversely, at θ_γ the mean is zero but the variance is not. Thus neither nonconvexity nor nonanalyticity is needed for persistent sampling fluctuations at a regularized minimum. With a nonzero plain SGD step, this minimum is not an absorbing parameter.

N.5.6 Normalized ReLU Networks and the Low-Rank Obstruction

The general covariance criterion can be supplemented by a structural obstruction to a common equilibrium. Xu, Rangamani, Liao, Galanti, and Poggio [117] study normalized ReLU classifiers

with scale regularization. Their Lemma 8 bounds the layer ranks when *every* minibatch update vanishes. We give the same scoped formulation as Chapter 7, Lemma 3, with its batch-exchange proof and explicit differentiability conditions. This is a separate piecewise-smooth setting, not an application of the preceding GELU analyticity theorem.

Model and tangent update directions. In this subsection the output is scalar ($k = 1$), and ℓ indexes layers. Consider a bias-free network with independently parameterized, unshared matrices,

$$f_V(x) = V_L\sigma(V_{L-1}\cdots\sigma(V_1x)\cdots), \qquad \|V_\ell\|_F = 1, \qquad F_{\rho,V}(x) = \rho f_V(x), \quad \rho > 0,$$

where σ is ReLU and $y_i \in \{-1, +1\}$. Introduce

$$m_i = y_i f_V(x_i), \qquad s_i = 1 - \rho m_i, \qquad R_{\ell i} = D_{V_\ell} m_i.$$

Here m_i is a signed normalized margin, and s_i is the corresponding signed residual. Separability $m_i > 0$ does not by itself imply $s_i > 0$; no common sign for the s_i is assumed.

Work at a point where the finite-sample prediction maps are differentiable and the fixed-gate calculation is valid, for example where no hidden preactivation on the training sample is zero. Locally in V_ℓ, scalar-output backpropagation writes $f_V(x_i) = a_{\ell i}^\top V_\ell b_{\ell i}$. Hence

$$R_{\ell i} = y_i a_{\ell i} b_{\ell i}^\top, \qquad \operatorname{rank} R_{\ell i} \le 1, \qquad \langle R_{\ell i}, V_\ell\rangle_F = m_i. \tag{N.43}$$

The last identity also expresses layerwise degree-one homogeneity. A shared convolutional kernel or a vector-output loss can sum several outer products and needs its own derivative-rank analysis. Regularity at a limiting ReLU kink does not follow merely from a probability-zero claim about initialization.

For the Frobenius-sphere-constrained minibatch objective

$$U_{\gamma,\mathcal{B}}(\rho, V) = \frac{1}{2B}\sum_{i\in\mathcal{B}}(1 - \rho m_i)^2 + \frac{\gamma}{2}\rho^2, \qquad \|V_\ell\|_F = 1,$$

the negative scale-gradient and negative tangent-gradient directions are

$$\begin{aligned} d_{\rho,\mathcal{B}} &= \frac{1}{B}\sum_{i\in\mathcal{B}} s_i m_i - \gamma\rho, \\ d_{\ell,\mathcal{B}} &= \frac{\rho}{B}\sum_{i\in\mathcal{B}} s_i(R_{\ell i} - m_i V_\ell). \end{aligned} \tag{N.44}$$

Indeed, differentiation in ρ gives the first line. The unconstrained negative V_ℓ gradient is $\rho B^{-1}\sum_i s_i R_{\ell i}$. Projecting it orthogonally onto the tangent space of $\|V_\ell\|_F = 1$ subtracts its inner product with V_ℓ times V_ℓ, giving the second line by (N.43).

These common-stationarity directions use the normalization convention of [117, Eq. (46), Lemma 8], with the common loss factor two removed by the half-squared-loss convention. The parameters n, γ correspond to N, λ in that paper. A finite Euler step does not preserve the sphere exactly; a discrete algorithm must specify its normalization rule. The following assertion concerns zeros of the stated directions.

Proposition 46 (Common-minibatch rank restriction; scoped form of Xu et al., Lemma 8)**.** *In the normalized scalar-output ReLU setting above, assume $n > 1$, $1 \le B < n$, $\gamma > 0$, $\rho > 0$, and $m_i \ne 0$ for every example. If*

$$d_{\rho,\mathcal{B}} = 0, \qquad d_{\ell,\mathcal{B}} = 0 \quad \textit{for every layer and every } B\textit{-element batch},$$

then $\operatorname{rank}(V_\ell) \le 2$ *for every layer.*

Batch-exchange proof. Set $c_i = s_i m_i$. The scale equation and $\gamma\rho > 0$ exclude $s_i = 0$ for all i. Since $m_i \neq 0$, some c_i is nonzero. Fix a layer ℓ. For any $i \neq j$, take two batches that share $B-1$ examples and differ only in i, j. Subtracting their zero tangent directions, and dividing by $\rho/B > 0$, gives

$$(c_i - c_j)V_\ell = s_i R_{\ell i} - s_j R_{\ell j}. \tag{N.45}$$

If some $c_i \neq c_j$, the right side has rank at most two by (N.43). Division by $c_i - c_j$ proves the stated restriction.

Otherwise all $c_i = c \neq 0$. Equation (N.45) implies $s_i R_{\ell i} = Q_\ell$ for a single matrix Q_ℓ of rank at most one. Any vanishing batch direction then implies $Q_\ell - cV_\ell = 0$. Thus $V_\ell = Q_\ell/c$ has rank at most one, which also proves the asserted bound. The argument applies to each layer. □

The statement retains the published upper bound and does not claim its sharpness. Its scalar-output, homogeneity, normalization, and local regularity hypotheses are part of the result; it does not apply unchanged to GELU or arbitrary regularizers.

From the obstruction to intrinsic noise. Concatenate $d_{\rho,\mathcal{B}}$ and the vectorized tangent matrix directions into $d_{\mathcal{B}}$, with the Euclidean/Frobenius product norm. At a stationary point of the *batch-averaged normalized dynamics*, $\mathbb{E}_{\mathcal{B}} d_{\mathcal{B}} = 0$. If its other hypotheses hold and some V_ℓ has rank greater than two, the proposition implies that at least one batch direction is nonzero. Every admissible batch has positive probability, so

$$\operatorname{Tr} \operatorname{Cov}_{\mathcal{B}}(d_{\mathcal{B}}) = \mathbb{E}_{\mathcal{B}} \|d_{\mathcal{B}}\|^2 > 0. \tag{N.46}$$

This is the covariance of the actual normalized directions, not an unjustified substitution of unconstrained parameter gradients.

The paper interprets the common-equilibrium restriction as a source of low-rank bias and intrinsic late-stage SGD fluctuations [117, Sections 6.1–6.2]. The precise implication established here is the necessary rank restriction and its contrapositive above. It is not a theorem that SGD always reaches rank-two matrices, nor that small updates alone imply a quantitative effective-rank bound. Such a bound would require control of the residuals, margins, and coefficients appearing in (N.45).

For a data family incompatible with accurate prediction by networks obeying that rank restriction, the proposition excludes a common equilibrium in the well-fitting region. The incompatibility must itself be proved; it is not established by calling the data generic. Conversely, low rank alone does not imply zero noise. The general covariance criterion and this architecture-specific obstruction explain how a useful regularized predictor can coexist with persistent intrinsic minibatch fluctuations.

N.5.7 Learning Rates and Finite Point Limits

The covariance at a parameter and the possibility of convergence of a sequence of parameters are different questions. The learning-rate schedule is part of the latter.

Proposition 47 (A finite limit with nonvanishing steps is a common equilibrium)**.** *Let $\mathfrak{B}$ be a finite family of batches and let $G_{\mathcal{B}} : \mathbb{R}^p \to \mathbb{R}^p$ be continuous for every $\mathcal{B} \in \mathfrak{B}$. Suppose batches $\mathcal{B}_j$ are sampled independently with fixed probabilities $p_{\mathcal{B}} > 0$, and consider*

$$\theta_{j+1} = \theta_j - \alpha_j G_{\mathcal{B}_j}(\theta_j), \qquad \inf_j \alpha_j > 0.$$

On an event of probability one, every finite limit θ_∞ of the whole sequence satisfies $G_{\mathcal{B}}(\theta_\infty) = 0$ for every $\mathcal{B} \in \mathfrak{B}$. Continuity may instead be assumed only on an open region containing the putative limit and the eventual iterates.

Proof. For a fixed batch, the probability that it never occurs after time J is $\lim_{q\to\infty}(1-p_{\mathcal{B}})^q = 0$. Taking the countable union over J, and then the finite union over batches, shows that every batch occurs infinitely often almost surely.

On this event, if $\theta_j \to \theta_\infty$, then $\theta_{j+1} - \theta_j \to 0$. Along the infinite subsequence with $\mathcal{B}_j = \mathcal{B}$,

$$\|G_{\mathcal{B}}(\theta_j)\| = \frac{\|\theta_{j+1} - \theta_j\|}{\alpha_j} \le \frac{\|\theta_{j+1} - \theta_j\|}{\inf_r \alpha_r} \longrightarrow 0.$$

Continuity gives $G_{\mathcal{B}}(\theta_\infty) = 0$. Apply this argument to every batch. □

Consequently a regularized stationary point with the positive covariance in Proposition 45 cannot be the finite point limit of this nonvanishing-step recursion under the stated sampling and continuity assumptions. For a fixed-step normalized algorithm with continuous batch update maps $T_{\mathcal{B}}$, the same subsequence argument instead gives $T_{\mathcal{B}}(z_\infty) = z_\infty$ for every batch. Inferring zero tangent directions requires these fixed points to be exactly the zeros of the specified directions. This is not automatic for arbitrary retractions, clipping, momentum, or adaptive optimizers, nor for a limit at an unexamined ReLU kink.

This conclusion does not assert divergence or existence of a stationary law. With $\alpha_j \to 0$, the update covariance

$$\mathrm{Cov}(\theta_{j+1} - \theta_j \mid \theta_j) = \alpha_j^2 c_{n,B} C(\theta_j)$$

can tend to zero while the gradient-estimator covariance remains nonzero. Diminishing-step convergence, convergence of an averaged predictor, and accuracy of a specified stopped iterate require their own arguments. Neither qualitative positivity of covariance nor positive regularization supplies a uniform lower noise bound across changing γ, batch sizes, or growing model families.

N.5.8 Anisotropic Diffusion Is Not a Universal Gibbs Law

The preceding results supply an intrinsic source of stochasticity in a regularized, nearly interpolating regime. They do not identify that noise with isotropic Langevin forcing. For a small constant learning rate α, uniform subset sampling, and an otherwise justified diffusion approximation, ordinary penalized SGD has the schematic model

$$d\theta_t = -\nabla U_\gamma(\theta_t)\, dt + [\alpha c_{n,B} C(\theta_t)]^{1/2}\, dW_t. \tag{N.47}$$

The covariance factor here is the rate on the time scale with increments α; it is not the per-update covariance. State dependence, anisotropy, finite-step effects, and constraint handling remain part of any application. Nonzero covariance need not excite every direction needed for escape from a critical region.

A modeled Itô diffusion can be written

$$d\theta_t = -\nabla U(\theta_t)\, dt + G(\theta_t)\, dW_t, \qquad D(\theta) = G(\theta)G(\theta)^\top. \tag{N.48}$$

Its density, when one exists with suitable regularity, satisfies

$$\partial_t p = \nabla\cdot(p\nabla U) + \frac{1}{2}\sum_{a,b} \partial_a\partial_b (D_{ab} p). \tag{N.49}$$

Boundary conditions, diffusion degeneracy, and state dependence are part of the problem. There is no general stationary formula depending only on $\mathrm{Tr}(D^{-1}\nabla^2 U)$ at minima; D may not even

be invertible. Non-isotropic SGD models need not have an equilibrium potential equal to their training loss [19].

Even the constant-coefficient quadratic case has a different formula. For an Ornstein–Uhlenbeck process

$$dz_t = -Hz_t\,dt + G\,dW_t, \qquad H = H^\top \succ 0,$$

its stationary covariance V solves

$$HV + VH = GG^\top. \tag{N.50}$$

Indeed Itô's formula for $z_t z_t^\top$, followed by expectation and stationarity, gives this identity. Equivalently,

$$V = \int_0^\infty e^{-Ht} GG^\top e^{-Ht}\,dt.$$

The stationary Gaussian can be degenerate if V is singular. In the isotropic case $GG^\top = 2TI$, it has $V = TH^{-1}$, recovering the ordinary quadratic Gibbs law. Local approximate-inference treatments of SGD use such additional assumptions [66].

When this local approximation is used near a strict regularized minimum, H is the Hessian of U_γ at that minimum, and $GG^\top$ must be matched to the specified regularized SGD covariance rate. One may not replace H automatically by the exact-interpolation data Hessian or set $GG^\top$ to zero merely because the *full* gradient vanishes. Freezing the covariance and replacing the drift by its linearization are additional local approximations, not consequences of analyticity.

N.5.9 The Valid Local Energy–Volume Comparison

For the explicitly specified isotropic Langevin model

$$d\theta_t = -\nabla U(\theta_t)\,dt + \sqrt{2T}\,dW_t,$$

assume well-posedness and a normalizable stationary density. Then

$$\pi_T(\theta) = Z_T^{-1} e^{-U(\theta)/T}. \tag{N.51}$$

If regularization is in the drift potential, it is also in U in this formula. A noncompact unregularized fitting set can prevent Z_T from being finite. A prior-weighted integral changes the reference measure and needs its own consistent dynamical interpretation.

For fixed positive regularization, a minimum component of U_γ need not be the exact data-interpolation set $F^{-1}(y)$, and its minimum value need not be zero. The following formulas therefore use the full potential and its actual minimum values. Intrinsic SGD fluctuations alone do not establish that the isotropic equilibrium model is appropriate.

For a compact smooth embedded minimum manifold $\mathcal{M}$ without boundary, of codimension q, minimum value U_*, and uniformly positive normal Hessian, a fixed small tube contributes

$$Z_{\mathcal{M}}(T) \sim e^{-U_*/T}(2\pi T)^{q/2} \int_{\mathcal{M}} \frac{dV(s)}{\sqrt{\det H_\perp(s)}}. \tag{N.52}$$

An isolated nondegenerate minimum θ_ℓ of value U_ℓ gives

$$Z_\ell(T) \sim e^{-U_\ell/T} \frac{(2\pi T)^{p/2}}{\sqrt{\det H_\ell}}. \tag{N.53}$$

These follow by integrating the normal quadratic expansion after scaling normal coordinates by $\sqrt{T}$; compactness and a uniform positive normal Hessian permit Gaussian domination. The detailed regular proof is in Chapter 7. Therefore, when $U_\ell > U_*$,

$$\frac{Z_\ell(T)}{Z_{\mathcal{M}}(T)} \sim C_\ell e^{-(U_\ell - U_*)/T} T^{(p-q)/2} \longrightarrow 0, \qquad C_\ell > 0. \tag{N.54}$$

The energy gap supplies the exponential factor; degeneracy changes the polynomial factor. A singular minimum component uses its RLCT and logarithmic multiplicity in place of the regular $q/2$ exponent.

These are ratios of local neighborhood weights. They do not prove that one component captures all global equilibrium mass, or bound the time needed to reach it. At fixed $T > 0$, an exact lower-dimensional manifold has zero probability under a smooth ambient density. Its neighborhood, not the manifold as an event of positive measure, is what is being compared.

N.5.10 Connection to Predictive Learning and the Status of the Results

A low-loss region favored by a particular stochastic model need not define a statistically small hypothesis class. Parameter-space sharpness is coordinate dependent; function-preserving reparameterizations can alter conventional Hessian-based sharpness [27]. Likewise, a Bayesian singular-learning result must not be silently converted into a statement about the test error of one SGD-selected parameter.

The separate statistical route in Chapter H remains: if a valid uniform bound on a controlled class $\mathcal{H}_Q$ holds and training returns $\widehat{f} \in \mathcal{H}_Q$ with empirical risk at most η, then on that event

$$R_P(\widehat{f}) \le \eta + \beta_S(Q, \delta).$$

The class, loss, sample, complexity budget, and algorithmic success event need the hypotheses stated there. An upper norm-complexity bound is not a lower prediction-Jacobian singular-value bound, and a product penalty is not automatically coercive. These are complementary requirements.

A stopped or otherwise specified returned predictor can satisfy these statistical conditions without its parameters approaching a common zero of all minibatch updates. If prediction averaging is used, the averaged predictor and its computational and class-complexity costs must themselves be specified. Persistent regularized noise is thus compatible with predictive success, but is not a proof of such success.

The appendix's logical map is therefore

$$\boxed{\begin{aligned} \text{analyticity + precompact gradient flow} &\implies \text{critical-point convergence,} \\ \text{regular interpolation} &\implies \text{local manifold geometry,} \\ \text{uniform residual conditioning} &\implies \text{quantitative fitting,} \\ \text{specified local weighted integral} &\implies \text{volume asymptotics.} \end{aligned}}$$

The regularized stochastic statement is a separate line:

$$\boxed{\begin{aligned} &\text{regularized stationarity + unequal sample gradients} \\ &\quad + \text{uniform proper-subset minibatches} \\ &\implies \text{zero mean update direction but nonzero covariance.} \end{aligned}}$$

A common equilibrium instead requires every minibatch direction to vanish; in the specified normalized ReLU setting this entails the stated rank restriction. These are exact finite-sample statements, not a Gibbs law or an efficient global-optimization theorem.

None of the displayed implications establishes all the other conclusions or population accuracy. The direct proofs given here follow from the stated hypotheses; the analytic gradient inequality, resolution theorem, singular-integral asymptotic, and stable-foliation theory remain invoked results with separate sources. The rank argument is a scoped restatement of the cited normalized-network result. No theorem is claimed that arbitrary sufficiently wide GELU networks converge almost surely to global interpolation, or that every regularized minimum has positive noise.

The useful conclusion is not that analyticity makes the training problem trivial. It makes several previously inaccessible aspects of degeneracy mathematically tractable. To prove efficient predictive learning one must still connect architecture, initialization, objective, optimizer, numerical precision, and data assumptions to a finite-time fitting and statistical control result. Once a feasible predictor has been shown to generalize, its computational realization supplies the compositional witness of Chapter H; no identification of the teacher's hidden computation is an additional requirement.

CHAPTER O

Diffusion as a Sequence of Supervised Learning Problems

The basic idea behind diffusion models can be stated very simply. Starting from an image I_0, construct a sequence

$$I_0, I_1, I_2, \ldots, I_T,$$

where I_{i+1} is obtained from I_i by adding a small amount of noise. This construction automatically produces a collection of supervised pairs

$$(I_i, I_{i+1}).$$

From exactly the same pairs one could in principle learn either direction,

$$I_i \longmapsto I_{i+1},$$

which corresponds to learning the *noising* process, or

$$I_{i+1} \longmapsto I_i,$$

which corresponds to learning the *denoising* process.

Thus diffusion may be viewed, at its most elementary level, as turning a difficult global generative problem into a sequence of much simpler local supervised learning problems.

O.1 The Forward Process

In standard diffusion models the noising direction is not usually learned, although it certainly could be learned from the pairs above. Instead, it is deliberately chosen to be a simple known stochastic process. A typical forward transition is

$$I_t = \sqrt{1-\beta_t}\, I_{t-1} + \sqrt{\beta_t}\, \epsilon_t, \qquad \epsilon_t \sim \mathcal{N}(0, \mathbf{I}),$$

where β_t is small.

Repeated application gives

$$I_0 \to I_1 \to \cdots \to I_T,$$

and for sufficiently large T the final state is approximately Gaussian noise,

$$I_T \sim \mathcal{N}(0, \mathbf{I}).$$

The reason for choosing such a simple forward process is not that the forward direction cannot be learned. Rather, there is no need to learn it: it is completely specified by construction. The interesting problem is to learn the reverse transitions.

O.2 The Reverse Problems

From the generated pairs one can train a predictor for

$$I_t \longmapsto I_{t-1}.$$

Thus the reverse process consists of a sequence of local denoising problems,

$$I_T \to I_{T-1} \to \cdots \to I_1 \to I_0.$$

The same network is usually shared across all noise levels. It takes both the noisy image and the noise level as input,

$$f_\theta(I_t, t).$$

Consequently diffusion does not require T unrelated predictors. Instead, a single function learns a family of closely related supervised problems parameterized by t.

There is also an important simplification. Because the forward process is Gaussian, one can generate I_t directly from the original image I_0 without explicitly constructing all the intermediate states:

$$I_t = \sqrt{\bar{\alpha}_t}\, I_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, \mathbf{I}),$$

where

$$\bar{\alpha}_t = \prod_{s=1}^{t} (1 - \beta_s).$$

Thus a training example can be generated simply by choosing an image I_0, choosing a random time t, choosing random Gaussian noise ϵ, and constructing I_t.

O.3 Predicting Noise

In modern diffusion models the network is often trained not to predict I_{t-1} directly, but to predict the noise ϵ used to produce I_t:

$$\epsilon_\theta(I_t, t).$$

The training objective is essentially

$$\min_{\theta} \mathbb{E}_{I_0,t,\epsilon} \left[\|\epsilon - \epsilon_\theta(I_t, t)\|^2 \right].$$

Once the added noise has been estimated, an estimate of the original image can be recovered from

$$\widehat{I}_0 = \frac{I_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon_\theta(I_t, t)}{\sqrt{\bar{\alpha}_t}}.$$

Thus noise prediction and denoising are essentially two equivalent ways of formulating the same supervised problem.

O.4 The Score Interpretation

There is a deeper interpretation of the denoising function. At each noise level t, let $p_t(I)$ denote the probability distribution of images after that amount of noise has been added. The quantity

$$\nabla_I \log p_t(I)$$

is called the *score* of the distribution.

The optimal noise predictor is closely related to this quantity:

$$\epsilon_\theta(I_t, t) \propto -\nabla_{I_t} \log p_t(I_t).$$

Therefore the network can be interpreted as learning a vector field over image space. At every noise level it indicates approximately which direction leads from a noisy image toward regions having higher probability under the image distribution.

The denoising process can therefore be viewed geometrically as

$$\text{noise} \to \text{coarse structure} \to \text{objects} \to \text{details} \to \text{image}.$$

This interpretation connects diffusion models with score matching, stochastic differential equations, and Langevin-type dynamics.

O.5 Why Denoising Becomes Generation

The crucial point is that the reverse process is not merely a procedure for restoring a particular corrupted image.

During training, the network sees noisy versions of many images and therefore learns properties of the entire image distribution. At generation time we can begin with a completely new random sample

$$I_T \sim \mathcal{N}(0, \mathbf{I})$$

and repeatedly apply the learned reverse transitions,

$$I_T \to I_{T-1} \to \cdots \to I_0.$$

In distributional terms, diffusion learns a transformation

$$\mathcal{N}(0, \mathbf{I}) \longrightarrow p_{\text{data}}(I).$$

Thus the collection of local denoising problems defines, when composed, a global generative process.

The reverse transition is in general stochastic. It can be written schematically as

$$I_{t-1} = \mu_\theta(I_t, t) + \sigma_t z, \qquad z \sim \mathcal{N}(0, \mathbf{I}).$$

This is natural because a noisy image does not in general have a unique less-noisy predecessor. The model is therefore more properly viewed as learning the conditional distribution

$$p_\theta(I_{t-1} \mid I_t)$$

rather than a single deterministic inverse.

O.6 Conditional Diffusion

For text-to-image generation the same idea is used with an additional conditioning variable c, representing, for example, a text prompt. The learned function becomes

$$\epsilon_\theta(I_t, t, c).$$

Conceptually, the model now estimates a score related to

$$\nabla_I \log p_t(I \mid c).$$

The reverse process therefore moves the random image not merely toward the manifold of natural images, but toward regions containing images compatible with the conditioning information.

O.7 Summary

The essential structure of diffusion can therefore be summarized as

images I_0
$\downarrow$
incremental noising
$\downarrow$
(I_i, I_{i+1}) supervised pairs
$\downarrow$
learn local reverse problems
$\downarrow$
learn a score/vector field
$\downarrow$
$\mathcal{N}(0, \mathbf{I}) \longrightarrow p_{\text{data}}(I)$.

The elementary observation is therefore the central one: incrementally noising an image creates a sequence of paired supervised learning problems. The same pairs contain information about both the forward, noising direction and the reverse, denoising direction. Standard diffusion chooses the forward process to be known and simple and devotes learning to the reverse direction.

What appears initially to be a difficult problem—learning an entire high-dimensional distribution of images—is thereby converted into a sequence of relatively small prediction problems. The composition of these local predictions is what produces the global generative model. "'latex

CHAPTER P

From Brain Activity to Images as an Inverse Problem

Suppose that we present an image x to a subject and measure the resulting fMRI activity y. Repeating this procedure produces paired examples

$$(x_1, y_1), \ldots, (x_n, y_n),$$

where x_i is an image and y_i is the corresponding measured brain response.

There are then two apparently symmetric supervised learning problems:

$$x \longmapsto y,$$

that is, predicting brain activity from an image, and

$$y \longmapsto x,$$

that is, reconstructing the image from brain activity.

The two problems, however, are not equally difficult. The first is a forward problem; the second is an inverse problem. This distinction is fundamental.

P.1 The Forward Problem

A simple description of the experiment is

$$y = F(x) + \eta,$$

where F represents the transformation from the visual stimulus to the measured fMRI response and η represents neural and measurement noise.

The forward learning problem is therefore

$$x \longmapsto y.$$

Given sufficiently many paired examples, one can attempt to learn a model

$$F_\theta(x) \approx y.$$

Equivalently, one can model the conditional distribution

$$p(y \mid x).$$

For a given image, the corresponding distribution of brain responses is reasonably well defined. The forward problem is therefore a standard supervised learning problem.

P.2 Why the Inverse Problem is Harder

The inverse problem asks instead for

$$y \longmapsto x.$$

The fundamental difficulty is that the forward transformation loses information. In general there may exist distinct images x_1 and x_2 such that

$$x_1 \neq x_2, \qquad F(x_1) \approx F(x_2).$$

Thus the inverse of F need not be a function.

This information loss occurs for several reasons. fMRI measurements have limited spatial and temporal resolution; many neurons contribute to each voxel; the BOLD response introduces additional smoothing; and both neural activity and the measurement process are noisy.

There is also a more fundamental reason. Visual representations themselves often discard properties of the stimulus that are irrelevant for recognition. If a representation is approximately invariant to transformations such as position, scale, illumination, or background, then different images may produce nearly identical internal representations.

If, for example,

$$F(T_1 x) \approx F(T_2 x),$$

then the representation cannot by itself determine which transformed image was originally presented.

Thus invariance, which is useful for recognition, becomes ambiguity when one attempts reconstruction.

P.3 Compression and Ill-Posedness

The asymmetry can also be understood geometrically.

An image may be represented by a very high-dimensional vector

$$x \in \mathbb{R}^N,$$

while the informative dimensionality of the measured fMRI response is typically much smaller:

$$y \in \mathbb{R}^M.$$

The forward map

$$F : \mathbb{R}^N \longrightarrow \mathbb{R}^M$$

therefore acts, in an information-theoretic sense, as a compression.

Compression is relatively easy to learn. Inverting compression is impossible unless additional information is supplied.

The same point is visible in the linear case. Suppose

$$y = Ax + \eta.$$

If A has a nontrivial null space, then for any v satisfying

$$Av = 0$$

we have

$$A(x + v) = Ax.$$

Consequently the measurement cannot distinguish x from $x + v$.

Even if A is formally invertible, small singular values make the inverse unstable. Writing the singular value decomposition as

$$A = U\Sigma V^\top,$$

the formal inverse contains factors

$$\frac{1}{\sigma_i}.$$

Small singular values therefore strongly amplify measurement noise.

The inverse problem is consequently both ambiguous and potentially ill-conditioned.

P.4 The Bayesian Formulation

The appropriate object for reconstruction is therefore not an ordinary inverse function

$$F^{-1}(y),$$

but the posterior distribution

$$p(x \mid y).$$

Bayes' rule gives

$$\boxed{p(x \mid y) = \frac{p(y \mid x)p(x)}{p(y)}.}$$

This equation exposes the asymmetry between the forward and inverse problems.

The paired image–fMRI examples allow us to learn the forward relationship

$$p(y \mid x).$$

But reconstruction requires, in addition, a model of

$$p(x),$$

the prior distribution of natural images.

The fMRI data tell us which images are compatible with the measured brain activity. The prior tells us which of those compatible images are plausible images.

Thus a natural strategy is not necessarily to learn directly

$$y \longmapsto x.$$

Instead one can learn

$$x \longmapsto$$

CHAPTER Q

Notation and Mathematical Conventions

This appendix provides a reference for the specific mathematical notation and conventions used throughout the text, particularly regarding compositional functions, statistical complexity measures, and group-theoretic invariance.

Computability and Compositional Sparsity

The following notation is used primarily in Chapters 4, 8, and 21 to bridge Turing computability with deep learning architectures.

$CS_k(s, L)$ **Compositionally Sparse Function Class.**
The set of functions f that can be computed by a directed acyclic graph (DAG) such that:

- The graph has total size (number of nodes) at most s.
- The depth (longest path from input to output) is at most L.
- Every internal node has a fan-in (arity) of at most k.

This class formally captures the structure of *efficiently computable* functions.

$\mathcal{G}_n$ **Discrete Grid Domain.**
Defined as $\mathcal{G}_n := \{0, 1, \ldots, 2^n\}^d / 2^n$. It represents the domain of inputs discretized to n bits of precision, used to prove exact equivalence between Turing Machines and ReLU networks (Chapter 4).

DAG **Directed Acyclic Graph.**
The graph theoretical representation of a computation where nodes represent constituent functions and edges represent data flow.

d^* **Maximal Local Arity.**
Used in Chapter 21. It denotes the maximum fan-in required for the nodes in a compiled DAG representation of a finite-horizon physical system. Approximation rates scale with d^* rather than the ambient dimension d.

Complexity Measures

These symbols are used in Chapters 12 and 13 to quantify generalization and capacity control.

$\mathfrak{R}_S(\mathcal{F})$ **Empirical Rademacher Complexity.**
Measures the ability of a function class $\mathcal{F}$ to fit random noise on a fixed dataset $S = \{x_1, \dots, x_m\}$. Defined as:

$$\mathfrak{R}_S(\mathcal{F}) = \mathbb{E}_\sigma \left[\sup_{f \in \mathcal{F}} \frac{1}{m} \sum_{i=1}^{m} \sigma_i f(x_i) \right]$$

where $\sigma_i \in \{-1, +1\}$ are independent Rademacher variables.

$\mathcal{H}(\epsilon, \mathcal{F}, \|\cdot\|)$ **Metric Entropy.**
The logarithm of the covering number $\mathcal{N}(\epsilon, \mathcal{F}, \|\cdot\|)$. It represents the number of bits needed to describe an element of $\mathcal{F}$ to within precision ϵ.

CV_{loo} **Leave-One-Out Cross-Validation Stability.**
Defined in Appendix A. A measure of algorithmic stability where the expected error difference is bounded when one training point is removed. This book establishes it as necessary and sufficient for the consistency of ERM.

Group Theory and Invariance

The following notation appears in Chapters 6 and 22 regarding Genericity and Invariant Representations.

G **Compact Group.**
A group acting unitarily on the data space (e.g., rotations, translations). In the context of representation learning, we assume $G \subset O(d)$.

dg **Haar Measure.**
The unique translation-invariant probability measure on a compact group G. Used to define group averages (orbits).

$\mu_t(I)$ **Orbit Distribution (or CDF).**
The probability distribution of the inner products $\langle I, gt \rangle$ as g varies over G. In Chapter 22, this is often represented by its Cumulative Distribution Function (CDF), providing a signature that is invariant to the transformation of the input I by elements of G.

N_η **Group η-Net.**
A finite subset of the group G such that every element $g \in G$ is within distance η of some element in N_η. The size of this net, $|N_G(\eta)|$, enters the complexity bounds for invariant embeddings.

CHAPTER R

Appendix: Glossary of Core Concepts

This glossary collects key terms and concepts introduced throughout the book. Entries are intentionally concise and technical; references point to chapters where the concept is developed in detail.

A

Associative Memory
A memory system in which retrieval is performed by content rather than by address. Given a cue or query, the system returns stored items that are similar to the cue under a learned or fixed similarity measure. Classical examples include Hopfield networks and radial basis function (RBF) networks; modern examples include attention mechanisms in transformers. Associative memory is treated in this book as a *computational primitive*. *(See Chaps. 1, 3)*

Associative Turing Machine (ATM)
A formal model of computation introduced in this book, consisting of alternating *content-addressable reads* from an associative memory and *local update* operations on a machine state. ATMs are shown to be Turing-complete with polynomial overhead and provide a formal abstraction of transformer-style computation. *(See Chap. 3)*

B

Behavioral Repertoire
The set of input–output behaviors or policies a system can realize under prompting or control. Intelligence is characterized here not by peak performance but by the *width* of this repertoire and the system's ability to deliberately restrict or reshape it. *(See Chap. 30)*

C

Compositionality

The property that a function or behavior can be expressed as a composition of simpler functions, typically organized hierarchically. In learning systems, compositionality permits reuse of subfunctions and enables generalization across domains. *(See Chaps. 1, 4)*

Containment Principle
The hypothesis that if system A is more intelligent than system B, then the set of behaviors realizable by B is contained within the set realizable by A. Operationally, this implies that A can simulate B, but not vice versa. Used to define a directed hierarchy of intelligence. *(See Chap. 30)*

CV_{loo} (Leave-One-Out Cross Validation)
A stability metric where the algorithm is trained on all but one data point, and error is measured on the left-out point. In this text, specific forms of stability are shown to be sufficient for generalization, linking CV_{loo} directly to predictivity. *(See Chap. 7)*

D

Diligent Learner
A learner that improves performance by systematically accumulating competence through training, without requiring explicit search or exploration mechanisms. The concept originates in work by Shalev-Shwartz and Shashua and is used here to contrast diligence with creativity and exploration. *(See Chap. 30)*

E

Efficient Computability
The property that a function can be computed with resources polynomial in the input size under a standard computational model (e.g., Turing machines or Boolean circuits). In this book, efficient computability is shown to imply sparse compositional structure in the corresponding function representation. *(See Chap. 4)*

Effective Information Exponent
A quantity measuring how the effective information content of a learned representation scales with system size or depth, used to characterize compression and generalization in compositional architectures. *(See Chaps. 7, 25)*

Empirical Risk Minimization (ERM)
The standard learning principle of minimizing the average error over the training dataset. While successful in practice, this book argues that ERM alone is insufficient to explain generalization without invoking stability or genericity conditions. *(See Chap. 7)*

G

Genericity

A property of target functions or learning problems asserting that informative, low-order components (e.g., linear or low-degree terms) are present and stable. Genericity ensures that gradient-based optimization receives reliable signals and avoids pathological loss landscapes. *(See Chaps. 7, 11)*

H

HyperBF / Hyper-RBF
A generalization of classical radial basis functions in which the similarity metric, prototypes, or both are learned. In particular, replacing Euclidean distance with a learned Mahalanobis metric yields kernels that are closely related to attention mechanisms. *(See Chaps. 1, 3)*

L

Large Embedding Model (LEM)
A proposed class of models extending large language models by incorporating richer latent world representations prior to or alongside language. LEMs are intended to address limitations of purely linguistic training by grounding prediction in learned world models. *(See Chap. 29)*

Loss Landscape
The function mapping model parameters to training or generalization error. The geometry of the loss landscape—its critical points, basins, and flat directions— plays a central role in optimization and generalization. *(See Chap. 7)*

M

Manifold Hypothesis
The assumption that high-dimensional data (e.g., images, text, sensory inputs) lie near low-dimensional manifolds embedded in ambient space. In this book, the hypothesis is extended to *behavioral* and *semantic* manifolds learned by large models. *(See Chaps. 1, 30)*

R

Radial Basis Function (RBF) Network
A function approximator of the form

$$f(x) = \sum_i w_i \, \phi(\|x - \mu_i\|),$$

where ϕ is a radial kernel and μ_i are fixed centers. RBF networks implement smooth associative memory and serve as historical precursors to modern attention-based models. *(See Chap. 1)*

Random-Access Memory (RAM)

A memory model permitting constant-time access to any stored location by address. Attention mechanisms approximate RAM-like behavior by enabling content-based access to all positions in a sequence. *(See Chaps. 1, 3)*

S

Simulation Distance
A metric defined between two models A and B as the asymmetry in their ability to simulate each other's behavior. Used to define a directed ordering of intelligence independent of static benchmarks. *(See Chap. 30)*

Sparse Compositionality
The principle that meaningful functions are compositions of a small number of low-arity constituent functions arranged in a sparse directed acyclic graph. Sparse compositionality explains why deep architectures are both necessary and effective for learning real-world functions. *(See Chaps. 1, 4)*

Stability
A property of a learning algorithm wherein small changes to the training set result in small changes to the output hypothesis. This book emphasizes stability as a necessary and sufficient condition for generalization, often replacing classical uniform convergence bounds. *(See Chap. 7)*

Stepwise Learning
A learning framework where complexity is increased incrementally. In the context of diffusion and autoregression, this refers to learning the conditional probability distribution of the next step (or next bit) given previous steps, effectively breaking a complex joint distribution into a sequence of simpler learning problems. *(See Chap. 10)*

T

Transformer
A neural architecture based on stacked self-attention and feedforward blocks. Transformers implement large-scale associative memory with learned similarity metrics and are central to modern language and vision models. *(See Chaps. 1, 3)*

W

World Simulator
An internal model that predicts the evolution of latent states corresponding to the external world. Distinguished here from purely linguistic predictors, world simulators are proposed as a missing component for robust intelligence. *(See Chaps. 24, 29)*

References

[1] P.-A. Absil, R. Mahony, and B. Andrews. "Convergence of the Iterates of Descent Methods for Analytic Cost Functions". In: *SIAM Journal on Optimization* 16.2 (2005), pp. 531–547. DOI: 10.1137/040605266. URL: https://sites.uclouvain.be/absil/2005.901.

[2] F. Anselmi, L. Rosasco, and T. Poggio. "On Invariance and Selectivity in Representation Learning". In: *Information and Inference: A Journal of the IMA* 5.2 (2016), pp. 134–158. DOI: 10.1093/imaiai/iaw009.

[3] T. R. Appleby, F. Rieke, and M. B. Manookin. "Surround motion modulates the encoding properties of primate retinal ganglion cells". In: *Cell Reports* 45.1 (2026), p. 116832. DOI: 10.1016/j.celrep.2025.116832. URL: https://www.cell.com/cell-reports/fulltext/S2211-1247(25)01604-3.

[4] A. Arenz, M. S. Drews, F. G. Richter, G. Ammer, and A. Borst. "The microcircuit of a Drosophila direction-selective neuron". In: *Nature* 541.7638 (2017), pp. 513–517. DOI: 10.1038/nature20962.

[5] P. L. Bartlett, D. J. Foster, and M. J. Telgarsky. "Spectrally-normalized margin bounds for neural networks". In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.

[6] P. L. Bartlett and S. Mendelson. "Rademacher and Gaussian complexities: Risk bounds and structural results". In: *Journal of Machine Learning Research* 3 (2002), pp. 463–482.

[7] J. Bates. *Computing 10,000x More Efficiently*. Tech. rep. Singular Computing Technical Whitepaper, 2010.

[8] J. Bates. "Processor with Approximate Arithmetic Units". 8,407,273. 2013.

[9] J. Bates and A. Bates. "Toward Lattice QCD on Billion Core Approximate Computers". In: *arXiv preprint arXiv:2010.15973* (2020).

[10] Y. Bengio, J. Louradour, R. Collobert, and J. Weston. "Curriculum learning". In: *Proceedings of the 26th annual international conference on machine learning*. 2009, pp. 41–48.

[11] E. Bierstone and P. D. Milman. "Semianalytic and subanalytic sets". In: *Publications Mathématiques de l'IHÉS* 67 (1988), pp. 5–42. URL: https://www.numdam.org/item/PMIHES_1988__67__5_0/.

[12] E. Bierstone and P. D. Milman. "Canonical desingularization in characteristic zero by blowing up the maximum strata of a local invariant". In: *Inventiones Mathematicae* 128.2 (1997), pp. 207–302. DOI: 10.1007/s002220050141. URL: https://link.springer.com/article/10.1007/s002220050141.

[13] K. M. Boergens, C. Kapfer, M. Helmstaedter, W. Denk, and A. Borst. "Full reconstruction of large lobula plate tangential cells in Drosophila from a 3D EM dataset". In: *PLoS ONE* 13.11 (2018), e0207828. DOI: 10.1371/journal.pone.0207828.

[14] A. Braun, A. Borst, and M. Meier. "Disynaptic inhibition shapes tuning of OFF-motion detectors in Drosophila". In: *Current Biology* 33.11 (2023), pp. 2260–2269. DOI: 10.1016/j.cub.2023.05.007.

[15] S. C. Brenner and L. R. Scott. *The Mathematical Theory of Finite Element Methods*. 3rd ed. Vol. 15. Texts in Applied Mathematics. New York: Springer, 2008. DOI: 10.1007/978-0-387-75934-0.

[16] D. S. Broomhead and D. Lowe. "Multivariable functional interpolation and adaptive networks". In: *Complex Systems* 2.3 (1988), pp. 321–355.

[17] C. Burns, P. Izmailov, J. H. Kirchner, B. Baker, L. Gao, L. Aschenbrenner, Y. Chen, A. Ecoffet, M. Joglekar, J. Leike, et al. *Weak-to-strong generalization: Eliciting strong capabilities with weak supervision*. Tech. rep. OpenAI, 2023.

[18] F. Cagnetta, L. Petrini, U. M. Tomasini, A. Favero, and M. Wyart. "How Deep Neural Networks Learn Compositional Data: The Random Hierarchy Model". In: *Physical Review X* 14 (3 July 2024), p. 031001. DOI: 10.1103/PhysRevX.14.031001. URL: https://link.aps.org/doi/10.1103/PhysRevX.14.031001.

[19] P. Chaudhari and S. Soatto. "Stochastic gradient descent performs variational inference, converges to limit cycles for deep networks". In: *International Conference on Learning Representations (ICLR)*. 2018.

[20] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze. "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks". In: *Proceedings of the 43rd Annual International Symposium on Computer Architecture*. 2016, pp. 367–379.

[21] R. Child, S. Gray, A. Radford, and I. Sutskever. "Generating long sequences with sparse transformers". In: *arXiv preprint arXiv:1904.10509* (2019).

[22] Y. Cooper. "The loss landscape of over-parameterized neural networks". In: *arXiv preprint arXiv:1804.10200* (2018).

[23] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré. "FlashAttention: Fast and memory-efficient exact attention with IO-awareness". In: *Advances in Neural Information Processing Systems*. 2022.

[24] L. N. Darlow, S. W. Luke, and B. Amos. "Cluster-based attention". In: *arXiv preprint arXiv:2007.03338* (2020).

[25] T. Dettmers. "8-bit approximations for parallelism in deep learning". In: *arXiv preprint arXiv:1511.04561* (2015).

[26] T. Ding, D. Li, and R. Sun. "Suboptimal Local Minima Exist for Wide Neural Networks with Smooth Activations". In: *Mathematics of Operations Research* 47.4 (2022), pp. 2784–2814. DOI: 10.1287/moor.2021.1228. URL: https://doi.org/10.1287/moor.2021.1228.

[27] L. Dinh, R. Pascanu, S. Bengio, and Y. Bengio. "Sharp Minima Can Generalize For Deep Nets". In: *Proceedings of the 34th International Conference on Machine Learning*. Vol. 70. Proceedings of Machine Learning Research. 2017, pp. 1019–1028. URL: https://proceedings.mlr.press/v70/dinh17b.html.

[28] S. Dorkenwald, A. Matsliah, A. R. Sterling, P. Schlegel, S.-c. Yu, C. E. McKellar, A. Lin, M. Costa, K. Eichler, Y. Yin, et al. "Neuronal wiring diagram of an adult brain". In: *Nature* 634.8032 (2024), pp. 124–138.

[29] R. M. Dudley. "The sizes of compact subsets of Hilbert space and continuity of Gaussian processes". In: *Journal of Functional Analysis* 1.3 (1967), pp. 290–330.

[30] D. J. Foster and M. A. Wilson. "Reverse replay of behavioural sequences in hippocampal place cells during the awake state". In: *Nature* 440.7084 (2006), pp. 680–683.

[31] X. Glorot and Y. Bengio. "Understanding the difficulty of training deep feedforward neural networks". In: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*. Vol. 9. Proceedings of Machine Learning Research. 2010, pp. 249–256. URL: https://proceedings.mlr.press/v9/glorot10a.html.

[32] N. Golowich, A. Rakhlin, and O. Shamir. "Size-independent sample complexity of neural networks". In: *Journal of Machine Learning Research* (2019).

[33] A. Gu and T. Dao. "Mamba: Linear-time sequence modeling with selective state spaces". In: *arXiv preprint arXiv:2312.00752* (2023).

[34] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, et al. "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning". In: *arXiv preprint arXiv:2501.12948* (2025).

[35] J. Haag, A. Arenz, E. Serbe, F. Gabbiani, and A. Borst. "Complementary mechanisms create direction selectivity in the fly". In: *eLife* 5 (2016), e17421. DOI: 10.7554/elife.17421.

[36] S. Han, H. Mao, and W. J. Dally. "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding". In: *International Conference on Learning Representations*. 2016.

[37] N. Hansen and A. Ostermeier. "Completely derandomized self-adaptation in evolution strategies". In: *Evolutionary Computation* 9.2 (2001), pp. 159–195.

[38] L. Hardesty. *The Surprising Usefulness of Sloppy Arithmetic*. MIT News. Jan. 2011.

[39] K. He, X. Zhang, S. Ren, and J. Sun. "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification". In: *arXiv preprint arXiv:1502.01852* (2015).

[40] D. Hendrycks and K. Gimpel. "Gaussian error linear units (gelus)". In: *arXiv preprint arXiv:1606.08415* (2016).

[41] G. E. Hinton, S. Osindero, and Y.-W. Teh. "A fast learning algorithm for deep belief nets". In: *Neural computation* 18.7 (2006), pp. 1527–1554.

[42] H. Hironaka. "Resolution of singularities of an algebraic variety over a field of characteristic zero: I, II". In: *Annals of Mathematics* 79.1/2 (1964), pp. 109–326.

[43] M. W. Hirsch, C. C. Pugh, and M. Shub. *Invariant Manifolds*. Vol. 583. Lecture Notes in Mathematics. Springer, 1977. DOI: 10.1007/BFb0092042. URL: https://link.springer.com/book/10.1007/BFb0092042.

[44] J. Ho, A. Jain, and P. Abbeel. "Denoising diffusion probabilistic models". In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 6840–6851.

[45] J. J. Hopfield. "Neural networks and physical systems with emergent collective computational abilities". In: *Proceedings of the national academy of sciences* 79.8 (1982), pp. 2554–2558.

[46] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations*. 2022.

[47] A. Jacot, F. Gabriel, and C. Hongler. "Neural tangent kernel: Convergence and generalization in neural networks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 31. 2018, pp. 8571–8580.

[48] M. Jaderberg, V. Dalibard, S. Osindero, W. M. Czarnecki, J. Donahue, A. Razavi, O. Vinyals, T. Green, I. Dunning, K. Simonyan, et al. "Population based training of neural networks". In: *arXiv preprint arXiv:1711.09846* (2017).

[49] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, et al. "In-datacenter performance analysis of a tensor processing unit". In: *Proceedings of the 44th Annual International Symposium on Computer Architecture*. 2017, pp. 1–12.

[50] P. Kanerva. *Sparse distributed memory*. MIT press, 1988.

[51] M. Kearns. "Efficient noise-tolerant learning from statistical queries". In: *Journal of the ACM* 45.6 (1998), pp. 983–1006.

[52] N. Kitaev, Ł. Kaiser, and A. Levskaya. "Reformer: The efficient transformer". In: *International Conference on Learning Representations*. 2020.

[53] K.-I. Ko. *Complexity Theory of Real Functions*. Progress in Theoretical Computer Science. Boston: Birkhäuser, 1991. DOI: 10.1007/978-1-4684-6802-1.

[54] T. Kohonen. "Correlation matrix memories". In: *IEEE Transactions on Computers* 100.4 (1972), pp. 353–359.

[55] A. N. Kolmogorov. "On tables of random numbers". In: *Sankhyā: The Indian Journal of Statistics, Series A* 25.4 (1963), pp. 369–376.

[56] R. Krizhanovsky and V. Krizhanovsky. "Mesh and torus computer architectures for data-parallel computations". In: *Parallel Computing* 74 (2018), pp. 1–13.

[57] H.-T. Kung. "Why systolic architectures?" In: *Computer* 15.1 (1982), pp. 37–46.

[58] J. C. Lagarias, J. A. Reeds, M. H. Wright, and P. E. Wright. "Convergence properties of the Nelder–Mead simplex method in low dimensions". In: *SIAM Journal on optimization* 9.1 (1998), pp. 112–147.

[59] B. S. Landman and R. L. Russo. "On a pin versus block relationship for partitions of logic graphs". In: *IEEE Transactions on Computers* 100.12 (1971), pp. 1469–1479.

[60] J. D. Lee, M. Simchowitz, M. I. Jordan, and B. Recht. "Gradient Descent Only Converges to Minimizers". In: *Proceedings of the 29th Annual Conference on Learning Theory*. Vol. 49. Proceedings of Machine Learning Research. 2016, pp. 1246–1257. URL: https://proceedings.mlr.press/v49/lee16.html.

[61] J. Lee, V. Ma, S. Zhao, Y. Nair, A. Spector, R. Cohen, and E. J. Candès. *FUSE: Ensembling Verifiers with Zero Labeled Data*. 2026. arXiv: 2604.18547 [stat.ML]. URL: https://arxiv.org/abs/2604.18547.

[62] S. Legg and M. Hutter. "Universal intelligence: A definition of machine intelligence". In: *Minds and Machines* 17.4 (2007), pp. 391–444.

[63] N. K. Logothetis, J. Pauls, H. H. Bülthoff, and T. Poggio. "View-dependent object recognition by monkeys". In: *Current Biology* 4.5 (1994), pp. 401–414.

[64] N. K. Logothetis, J. Pauls, and T. Poggio. "Shape representation in the inferior temporal cortex of monkeys". In: *Current Biology* 5.5 (1995), pp. 552–563.

[65] S. Malladi, T. Gao, E. Nichani, A. Damian, J. D. Lee, D. Chen, and S. Arora. *Fine-Tuning Language Models with Just Forward Passes*. 2024. arXiv: 2305.17333 [cs.LG]. URL: https://arxiv.org/abs/2305.17333.

[66] S. Mandt, M. D. Hoffman, and D. M. Blei. "Stochastic gradient descent as approximate Bayesian inference". In: *Journal of Machine Learning Research* 18.1 (2017), pp. 4873–4907.

[67] D. Marr. "Simple memory: a theory for archicortex". In: *Philosophical Transactions of the Royal Society of London. Series B, Biological Sciences* 262.841 (1971), pp. 23–81.

[68] A. S. Mauss, M. Meier, E. Serbe, and A. Borst. "Optogenetic and pharmacologic dissection of feedforward inhibition in Drosophila motion vision". In: *The Journal of Neuroscience* 34.6 (2014), pp. 2254–2263.

[69] J. L. McClelland, B. L. McNaughton, and R. C. O'Reilly. "Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory". In: *Psychological Review* 102.3 (1995), p. 419.

[70] E. J. McShane. "Extension of Range of Functions". In: *Bulletin of the American Mathematical Society* 40.12 (1934), pp. 837–842. DOI: 10.1090/S0002-9904-1934-05978-0.

[71] M. Meier and A. Borst. "Extreme compartmentalization in a Drosophila amacrine cell". In: *Current Biology* 29.9 (2019), pp. 1514–1524. DOI: 10.1016/j.cub.2019.03.070.

[72] H. N. Mhaskar and T. Poggio. "Function Approximation by Deep Networks". In: *Communications on Pure and Applied Analysis* 19.8 (2020), pp. 4085–4095. DOI: 10.3934/cpaa.2020181. arXiv: 1905.12882.

[73] H. Mhaskar and T. Poggio. "Deep vs. shallow networks: An approximation theory perspective". In: *Analysis and Applications* (2016), pp. 829–848.

[74] H. N. Mhaskar and T. Poggio. "Deep vs. shallow networks: An approximation theory perspective". In: *Analysis and Applications* 14.06 (2016), pp. 829–848.

[75] P. Michel, O. Levy, and G. Neubig. "Are Sixteen Heads Really Better than One?" In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019.

[76] P. Micikevicius, S. Narayanaswami, M. Abdelfattah, et al. "Mixed Precision Training". In: *International Conference on Learning Representations*. 2018.

[77] Y. Nesterov and V. Spokoiny. "Random gradient-free minimization of convex functions". In: *Foundations of Computational Mathematics* 17 (2017), pp. 527–566.

[78] B. Neyshabur, S. Bhojanapalli, and N. Srebro. "A PAC-Bayesian approach to spectrally-normalized margin bounds for neural networks". In: *arXiv preprint arXiv:1707.09564* (2017).

[79] B. Neyshabur, R. Tomioka, and N. Srebro. "Norm-based capacity control in neural networks". In: *Conference on Learning Theory*. 2015, pp. 1376–1401.

[80] Q. Nguyen. "On connected components of loss surfaces in classification". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43.5 (2019), pp. 1691–1702.

[81] Q. Nguyen and M. Hein. "The loss surface of deep and wide neural networks". In: *International Conference on Machine Learning (ICML)*. PMLR. 2017, pp. 2603–2612.

[82] Q. Nguyen and M. Hein. "Optimization landscape and expressivity of deep feedforward networks". In: *Journal of Machine Learning Research* 19.82 (2018), pp. 1–43.

[83] T. Poggio. "On optimal nonlinear associative recall". In: *Biological Cybernetics* 19.4 (1975), pp. 201–209.

[84] T. Poggio et al. "Self-Assembly of a Biologically Plausible Learning Circuit". In: *CBMM Memo* 152 (2024).

[85] T. Poggio, A. Banburski, and Q. Liao. "The theory of deep learning: an overview". In: *arXiv preprint arXiv:1706.08845* (2017).

[86] T. Poggio and S. Edelman. "A network that learns to recognize three-dimensional objects". In: *Nature* 343.6255 (1990), pp. 263–266.

[87] T. Poggio and Y. Gan. "A Homogeneous Transformer Architecture". In: *CBMM Memo* 143 (2023).

[88] T. Poggio and F. Girosi. "Networks for approximation and learning". In: *Proceedings of the IEEE* 78.9 (1990), pp. 1481–1497.

[89] T. Poggio, H. Mhaskar, L. Rosasco, B. Miranda, and Q. Liao. "Why and when can deep-but not shallow-networks avoid the curse of dimensionality: A review". In: *International Journal of Automation and Computing* 14.5 (2017), pp. 503–519.

[90] T. A. Poggio and F. Anselmi. *Visual Cortex and Deep Networks: Learning Invariant Representations*. Cambridge, MA: MIT Press, 2016. ISBN: 9780262034722.

[91] A. Radhakrishnan, D. Beaglehole, P. Mikhaylov, and M. Belkin. "Mechanism of feature learning in convolutional neural networks". In: *arXiv preprint arXiv:2401.03358* (2024).

[92] A. Radhakrishnan, D. Beaglehole, P. Pandit, and M. Belkin. "Feature learning in neural networks and kernel machines that recursively learn features". In: *arXiv preprint arXiv:2212.13881* (2022).

[93] W. Reichardt, T. Poggio, and K. Hausen. "Figure-ground discrimination by relative movement in the visual system of the fly". In: *Biological Cybernetics* 46.1 (1983), pp. 1–30.

[94] A. Roy, M. Saffar, A. Vaswani, and D. Grangier. "Efficient content-based sparse attention with routing transformers". In: *Transactions of the Association for Computational Linguistics* 8 (2020), pp. 53–67.

[95] A. Roy, M. Saffar, A. Vaswani, and D. Grangier. "Efficient content-based sparse attention with routing transformers". In: *Transactions of the Association for Computational Linguistics* 9 (2021), pp. 53–68.

[96] L. Sagun, U. Evci, V. U. Guney, Y. Dauphin, and L. Bottou. "Empirical analysis of the Hessian spectrum in deep learning". In: *arXiv preprint arXiv:1706.04454* (2017).

[97] P. Schlegel, Y. Yin, A. S. Bates, S. Dorkenwald, K. Eichler, P. Brooks, D. S. Han, M. Gkantia, M. dos Santos, E. J. Munnelly, et al. "Whole-brain annotation and multi-connectome cell typing of Drosophila". In: *Nature* 634.8032 (2024), pp. 139–152.

[98] M. Shanahan, K. McDonell, and L. Reynolds. "Role play with large language models". In: *Nature* 623.7987 (2023), pp. 493–498.

[99] K. Shinomiya, G. Huang, Z. Lu, T. Parag, C. S. Xu, R. Aniceto, N. Ansari, L. A. Cheatham, S. Lauchie, E. Neace, et al. "Comparisons between the ON- and OFF-edge motion pathways in the Drosophila brain". In: *eLife* 8 (2019), e40025. DOI: 10.7554/elife.40025.

[100] D. A. Spielman and S.-H. Teng. "Smoothed analysis of algorithms: Why the simplex algorithm usually takes polynomial time". In: *Journal of the ACM* 51.3 (2004), pp. 385–463.

[101] S.-y. Takemura, Y. Aso, T. Hige, A. Wong, Z. Lu, C. S. Xu, P. K. Rivlin, H. Hess, T. Zhao, T. Parag, et al. "The comprehensive connectome of a neural substrate for 'ON' motion detection in Drosophila". In: *eLife* 6 (2017), e24394. DOI: 10.7554/elife.24394.

[102] S.-y. Takemura, A. Nern, D. B. Chklovskii, L. K. Scheffer, G. M. Rubin, and I. A. Meinertzhagen. "A visual motion detection circuit from single synapse resolutions in Drosophila". In: *Cell* 170.4 (2017), pp. 792–803. DOI: 10.1016/j.cell.2017.06.046.

[103] M. Telgarsky. "Benefits of depth in neural networks". In: *Conference on Learning Theory*. PMLR. 2016, pp. 1517–1539.

[104] V. Torre and T. Poggio. "A synaptic mechanism possibly underlying directional selectivity to motion". In: *Proceedings of the Royal Society of London. Series B. Biological Sciences* 202.1148 (1978), pp. 409–416.

[105] Y.-H. H. Tsai, S. Bai, M. Yamada, L.-P. Morency, and R. Salakhutdinov. "Transformer dissection: An unified understanding for self-attention and convolution". In: *arXiv preprint arXiv:1908.00173* (2019).

[106] A. M. Turing. "Computing machinery and intelligence". In: *Mind* 59.236 (1950), pp. 433–460.

[107] A. M. Turing. "On computable numbers, with an application to the Entscheidungsproblem". In: *Proceedings of the London Mathematical Society* 2.42 (1936), pp. 230–265.

[108] V. N. Vapnik. *Statistical Learning Theory*. Wiley, 1998.

[109] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. "Attention is all you need". In: *Advances in neural information processing systems* 30 (2017).

[110] A. Vyas, A. Katharopoulos, and F. Fleuret. "Fast transformers with clustered attention". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 33–46.

[111] M. Wang and W. E. "On the Expressive Power of Mixture-of-Experts for Structured Complex Tasks". In: *arXiv preprint arXiv:2505.24205* (2025).

[112] S. Watanabe. "Algebraic analysis for non-identifiable learning machines". In: *Neural Computation* 13.4 (2001), pp. 899–933.

[113] S. Watanabe. *Algebraic Geometry and Statistical Learning Theory*. Cambridge University Press, 2009.

[114] S. Watanabe. "A Widely Applicable Bayesian Information Criterion". In: *Journal of Machine Learning Research* 14.27 (2013), pp. 867–897. URL: https://www.jmlr.org/papers/v14/watanabe13a.html.

[115] D. Wierstra, T. Schaul, T. Glasmachers, Y. Sun, J. Peters, and J. Schmidhuber. "Natural evolution strategies". In: *The Journal of Machine Learning Research* 15.1 (2014), pp. 949–980.

[116] D. J. Willshaw, O. P. Buneman, and H. C. Longuet-Higgins. "Non-holographic associative memory". In: *Nature* 222.5197 (1969), pp. 960–962.

[117] M. Xu, A. Rangamani, Q. Liao, T. Galanti, and T. Poggio. "Dynamics in Deep Classifiers Trained with the Square Loss: Normalization, Low Rank, Neural Collapse, and Generalization Bounds". In: *Research* 6 (2023), p. 0024.

[118] G. Yang and E. J. Hu. "Tensor Programs IV: Feature Learning in Infinite-Width Neural Networks". In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research. 2021, pp. 11727–11737. URL: https://proceedings.mlr.press/v139/yang21c.html.

[119] D. Yarotsky. "Error bounds for approximations with deep ReLU networks". In: *Neural Networks* 94 (2017), pp. 103–114.

[120] H. Zhang, P. Svirin, and S. Venkataramani. "Block floating point for deep neural networks". In: *arXiv preprint arXiv:1909.13271* (2019).

[121] A. Zhao, A. Nern, S. Koskela, M. Dreher, M. Erginkaya, C. Laughland, et al. "A comprehensive neuroanatomical survey of the Drosophila Lobula Plate Tangential Neurons with predictions for their optic flow sensitivity". In: *eLife* 13 (2024).

[122] Y. Zheng et al. "Neural Clustering Attention for Efficient and Effective Transformer". In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*. 2022.

[123] W. Zhou and G. K. Dziugaite. "Non-vacuous generalization bounds at the deepest level". In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019.